

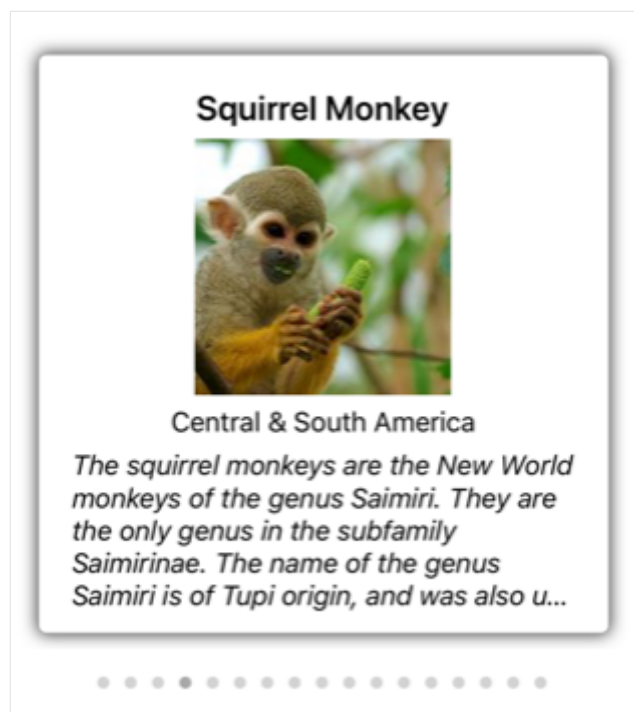
CarouselView

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [CarouselView](#) is a view for presenting data in a scrollable layout, where users can swipe to move through a collection of items.

By default, [CarouselView](#) will display its items in a horizontal orientation. A single item will be displayed on screen, with swipe gestures resulting in forwards and backwards navigation through the collection of items. In addition, indicators can be displayed that represent each item in the [CarouselView](#):



By default, [CarouselView](#) provides looped access to its collection of items. Therefore, swiping backwards from the first item in the collection will display the last item in the collection. Similarly, swiping forwards from the last item in the collection will return to the first item in the collection.

[CarouselView](#) shares much of its implementation with [CollectionView](#). However, the two controls have different use cases. [CollectionView](#) is typically used to present lists of data of any length, whereas [CarouselView](#) is typically used to highlight information in a list of limited length. For more information about [CollectionView](#), see [CollectionView](#).

Populate a CarouselView with data

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [CarouselView](#) includes the following properties that define the data to be displayed, and its appearance:

- `ItemsSource`, of type `IEnumerable`, specifies the collection of items to be displayed, and has a default value of `null`.
- `ItemTemplate`, of type [DataTemplate](#), specifies the template to apply to each item in the collection of items to be displayed.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings.

[CarouselView](#) defines a `ItemsUpdatingScrollMode` property that represents the scrolling behavior of the [CarouselView](#) when new items are added to it. For more information about this property, see [Control scroll position when new items are added](#).

[CarouselView](#) supports incremental data virtualization as the user scrolls. For more information, see [Load data incrementally](#).

Populate a CarouselView with data

A [CarouselView](#) is populated with data by setting its `ItemsSource` property to any collection that implements `IEnumerable`. By default, [CarouselView](#) displays items horizontally.

Important

If the [CarouselView](#) is required to refresh as items are added, removed, or changed in the underlying collection, the underlying collection should be an `IEnumerable` collection that sends property change notifications, such as `ObservableCollection`.

[CarouselView](#) can be populated with data by using data binding to bind its `ItemsSource` property to an `IEnumerable` collection. In XAML, this is achieved with the `Binding` markup extension:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}" />
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView();  
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
```

In this example, the `ItemsSource` property data binds to the `Monkeys` property of the connected viewmodel.

ⓘ Note

Compiled bindings can be enabled to improve data binding performance in .NET MAUI applications. For more information, see [Compiled bindings](#).

For information on how to change the [CarouselView](#) orientation, see [Specify CarouselView layout](#). For information on how to define the appearance of each item in the [CarouselView](#), see [Define item appearance](#). For more information about data binding, see [Data binding](#).

Define item appearance

The appearance of each item in the [CarouselView](#) can be defined by setting the `CarouselView.ItemTemplate` property to a [DataTemplate](#):

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}">  
  <CarouselView.ItemTemplate>  
    <DataTemplate>  
      <StackLayout>  
        <Frame HasShadow="True"  
          BorderColor="DarkGray"  
          CornerRadius="5"  
          Margin="20"  
          HeightRequest="300"  
          HorizontalOptions="Center"  
          VerticalOptions="CenterAndExpand">  
          <StackLayout>  
            <Label Text="{Binding Name}"  
              FontAttributes="Bold"  
              FontSize="18"/>
```

```

        HorizontalOptions="Center"
        VerticalOptions="Center" />
        <Image Source="{Binding ImageUrl}"
            Aspect="AspectFill"
            HeightRequest="150"
            WidthRequest="150"
            HorizontalOptions="Center" />
        <Label Text="{Binding Location}"
            HorizontalOptions="Center" />
        <Label Text="{Binding Details}"
            FontAttributes="Italic"
            HorizontalOptions="Center"
            MaxLines="5"
            LineBreakMode="TailTruncation" />
    </StackLayout>
</Frame>
</StackLayout>
</DataTemplate>
</CarouselView.ItemTemplate>
</CarouselView>

```

The equivalent C# code is:

C#

```

CarouselView carouselView = new CarouselView();
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");

carouselView.ItemTemplate = new DataTemplate(() =>
{
    Label nameLabel = new Label { ... };
    nameLabel.SetBinding(Label.TextProperty, "Name");

    Image image = new Image { ... };
    image.SetBinding(Image.SourceProperty, "ImageUrl");

    Label locationLabel = new Label { ... };
    locationLabel.SetBinding(Label.TextProperty, "Location");

    Label detailsLabel = new Label { ... };
    detailsLabel.SetBinding(Label.TextProperty, "Details");

    StackLayout stackLayout = new StackLayout();
    stackLayout.Add(nameLabel);
    stackLayout.Add(image);
    stackLayout.Add(locationLabel);
    stackLayout.Add(detailsLabel);

    Frame frame = new Frame { ... };
    StackLayout rootStackLayout = new StackLayout();
    rootStackLayout.Add(frame);
}

```



```
        return rootStackLayout;  
    });
```

The elements specified in the [DataTemplate](#) define the appearance of each item in the [CarouselView](#). In the example, layout within the [DataTemplate](#) is managed by a [StackLayout](#), and the data is displayed with an [Image](#) object, and three [Label](#) objects, that all bind to properties of the `Monkey` class:

C#

```
public class Monkey  
{  
    public string Name { get; set; }  
    public string Location { get; set; }  
    public string Details { get; set; }  
    public string ImageUrl { get; set; }  
}
```

The following screenshot shows an example of templating each item:



For more information about data templates, see [Data templates](#).

Choose item appearance at runtime

The appearance of each item in the [CarouselView](#) can be chosen at runtime, based on the item value, by setting the `CarouselView.ItemTemplate` property to a [DataTemplateSelector](#) object:

XAML

```

<ContentPage ...
    xmlns:controls="clr-namespace:CarouselViewDemos.Controls"

x:Class="CarouselViewDemos.Views.HorizontalLayoutDataTemplateSelectorPage">
    <ContentPage.Resources>
        <DataTemplate x:Key="AmericanMonkeyTemplate">
            ...
        </DataTemplate>

        <DataTemplate x:Key="OtherMonkeyTemplate">
            ...
        </DataTemplate>

        <controls:MonkeyDataTemplateSelector x:Key="MonkeySelector"
            AmericanMonkey="{StaticResource
AmericanMonkeyTemplate}"
            OtherMonkey="{StaticResource
OtherMonkeyTemplate}" />
    </ContentPage.Resources>

    <CarouselView ItemsSource="{Binding Monkeys}"
        ItemTemplate="{StaticResource MonkeySelector}" />
</ContentPage>

```

The equivalent C# code is:

C#

```

CarouselView carouselView = new CarouselView
{
    ItemTemplate = new MonkeyDataTemplateSelector { ... }
};
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");

```

The `ItemTemplate` property is set to a `MonkeyDataTemplateSelector` object. The following example shows the `MonkeyDataTemplateSelector` class:

C#

```

public class MonkeyDataTemplateSelector : DataTemplateSelector
{
    public DataTemplate AmericanMonkey { get; set; }
    public DataTemplate OtherMonkey { get; set; }

    protected override DataTemplate OnSelectTemplate(object item,
BindableObject container)
    {
        return ((Monkey)item).Location.Contains("America") ? AmericanMonkey
: OtherMonkey;
    }
}

```

```
}  
}
```

The `MonkeyDataTemplateSelector` class defines `AmericanMonkey` and `OtherMonkey` [DataTemplate](#) properties that are set to different data templates. The `OnSelectTemplate` override returns the `AmericanMonkey` template when the monkey name contains "America". When the monkey name doesn't contain "America", the `OnSelectTemplate` override returns the `OtherMonkey` template, which displays its data grayed out:



For more information about data template selectors, see [Create a DataTemplateSelector](#).

Important

When using [CarouselView](#), never set the root element of your [DataTemplate](#) objects to a [ViewCell](#). This will result in an exception being thrown because [CarouselView](#) has no concept of cells.

Display indicators

Indicators, that represent the number of items and current position in a [CarouselView](#), can be displayed next to the [CarouselView](#). This can be accomplished with the [IndicatorView](#) control:

XAML

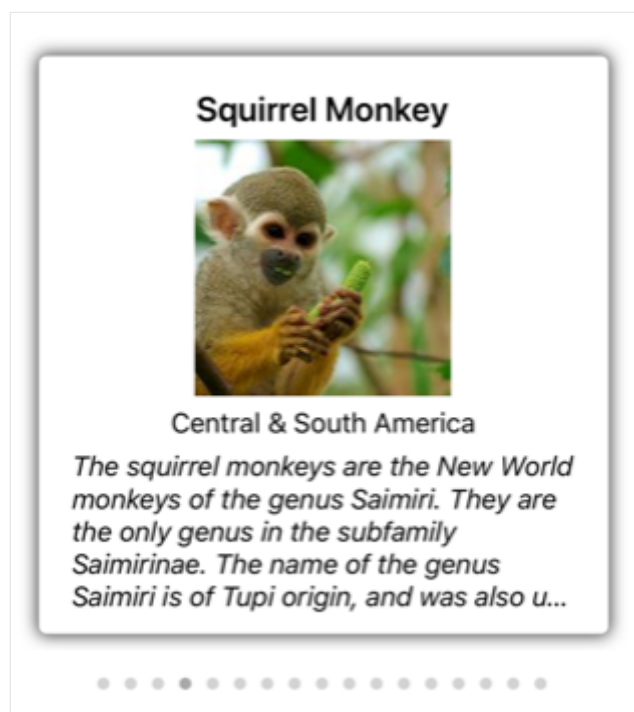
```
<StackLayout>  
    <CarouselView ItemsSource="{Binding Monkeys}"
```

```

        IndicatorView="indicatorView">
    <CarouselView.ItemTemplate>
        <!-- DataTemplate that defines item appearance -->
    </CarouselView.ItemTemplate>
</CarouselView>
<IndicatorView x:Name="indicatorView"
    IndicatorColor="LightGray"
    SelectedIndicatorColor="DarkGray"
    HorizontalOptions="Center" />
</StackLayout>

```

In this example, the [IndicatorView](#) is rendered beneath the [CarouselView](#), with an indicator for each item in the [CarouselView](#). The [IndicatorView](#) is populated with data by setting the `CarouselView.IndicatorView` property to the [IndicatorView](#) object. Each indicator is a light gray circle, while the indicator that represents the current item in the [CarouselView](#) is dark gray:



Important

Setting the `CarouselView.IndicatorView` property results in the `IndicatorView.Position` property binding to the `CarouselView.Position` property, and the `IndicatorView.ItemsSource` property binding to the `CarouselView.ItemsSource` property.

For more information about indicators, see [IndicatorView](#).

Context menus

[CarouselView](#) supports context menus for items of data through the [SwipeView](#), which reveals the context menu with a swipe gesture. The [SwipeView](#) is a container control that wraps around an item of content, and provides context menu items for that item of content. Therefore, context menus are implemented for a [CarouselView](#) by creating a [SwipeView](#) that defines the content that the [SwipeView](#) wraps around, and the context menu items that are revealed by the swipe gesture. This is achieved by adding a [SwipeView](#) to the [DataTemplate](#) that defines the appearance of each item of data in the [CarouselView](#):

XAML

```
<CarouselView x:Name="carouselView"
    ItemsSource="{Binding Monkeys}">
    <CarouselView.ItemTemplate>
        <DataTemplate>
            <StackLayout>
                <Frame HasShadow="True"
                    BorderColor="DarkGray"
                    CornerRadius="5"
                    Margin="20"
                    HeightRequest="300"
                    HorizontalOptions="Center"
                    VerticalOptions="CenterAndExpand">
                    <SwipeView>
                        <SwipeView.TopItems>
                            <SwipeItems>
                                <SwipeItem Text="Favorite"

IconImageSource="favorite.png"

                                BackgroundColor="LightGreen"
                                Command="{Binding Source=
{x:Reference carouselView}, Path=BindingContext.FavoriteCommand}"
                                CommandParameter="{Binding}"
                            />
                        </SwipeItems>
                    </SwipeView.TopItems>
                    <SwipeView.BottomItems>
                        <SwipeItems>
                            <SwipeItem Text="Delete"
                                IconImageSource="delete.png"
                                BackgroundColor="LightPink"
                                Command="{Binding Source=
{x:Reference carouselView}, Path=BindingContext.DeleteCommand}"
                                CommandParameter="{Binding}"
                            />
                        </SwipeItems>
                    </SwipeView.BottomItems>
                </StackLayout>
                <!-- Define item appearance -->
            </StackLayout>
        </SwipeView>
    </Frame>
```

```

        </StackLayout>
    </DataTemplate>
</CarouselView.ItemTemplate>
</CarouselView>

```

The equivalent C# code is:

C#

```

CarouselView carouselView = new CarouselView();
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");

carouselView.ItemTemplate = new DataTemplate(() =>
{
    StackLayout stackLayout = new StackLayout();
    Frame frame = new Frame { ... };

    SwipeView swipeView = new SwipeView();
    SwipeItem favoriteSwipeItem = new SwipeItem
    {
        Text = "Favorite",
        IconImageSource = "favorite.png",
        BackgroundColor = Colors.LightGreen
    };
    favoriteSwipeItem.SetBinding(MenuItem.CommandProperty, new
Binding("BindingContext.FavoriteCommand", source: carouselView));
    favoriteSwipeItem.SetBinding(MenuItem.CommandParameterProperty, ".");

    SwipeItem deleteSwipeItem = new SwipeItem
    {
        Text = "Delete",
        IconImageSource = "delete.png",
        BackgroundColor = Colors.LightPink
    };
    deleteSwipeItem.SetBinding(MenuItem.CommandProperty, new
Binding("BindingContext.DeleteCommand", source: carouselView));
    deleteSwipeItem.SetBinding(MenuItem.CommandParameterProperty, ".");

    swipeView.TopItems = new SwipeItems { favoriteSwipeItem };
    swipeView.BottomItems = new SwipeItems { deleteSwipeItem };

    StackLayout swipeViewStackLayout = new StackLayout { ... };
    swipeView.Content = swipeViewStackLayout;
    frame.Content = swipeView;
    stackLayout.Add(frame);

    return stackLayout;
});

```

In this example, the [SwipeView](#) content is a [StackLayout](#) that defines the appearance of each item that's surrounded by a [Frame](#) in the [CarouselView](#). The swipe items are used

to perform actions on the [SwipeView](#) content, and are revealed when the control is swiped from the bottom and from the top:



[SwipeView](#) supports four different swipe directions, with the swipe direction being defined by the directional `SwipeItems` collection the `SwipeItems` objects are added to. By default, a swipe item is executed when it's tapped by the user. In addition, once a swipe item has been executed the swipe items are hidden and the [SwipeView](#) content is re-displayed. However, these behaviors can be changed.

For more information about the [SwipeView](#) control, see [SwipeView](#).

Pull to refresh

[CarouselView](#) supports pull to refresh functionality through the [RefreshView](#), which enables the data being displayed to be refreshed by pulling down on the items. The [RefreshView](#) is a container control that provides pull to refresh functionality to its child, provided that the child supports scrollable content. Therefore, pull to refresh is implemented for a [CarouselView](#) by setting it as the child of a [RefreshView](#):

XAML

```
<RefreshView IsRefreshing="{Binding IsRefreshing}"
             Command="{Binding RefreshCommand}">
    <CarouselView ItemsSource="{Binding Animals}">
        ...
    </CarouselView>
</RefreshView>
```

The equivalent C# code is:

C#

```
RefreshView refreshView = new RefreshView();
ICommand refreshCommand = new Command(() =>
```

```
{
    // IsRefreshing is true
    // Refresh data here
    refreshView.IsRefreshing = false;
});
refreshView.Command = refreshCommand;

CarouselView carouselView = new CarouselView();
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Animals");
refreshView.Content = carouselView;
// ...
```

When the user initiates a refresh, the [ICommand](#) defined by the `Command` property is executed, which should refresh the items being displayed. A refresh visualization is shown while the refresh occurs, which consists of an animated progress circle:



The value of the `RefreshView.IsRefreshing` property indicates the current state of the [RefreshView](#). When a refresh is triggered by the user, this property will automatically transition to `true`. Once the refresh completes, you should reset the property to `false`.

For more information about [RefreshView](#), see [RefreshView](#).

Load data incrementally

[CarouselView](#) supports incremental data virtualization as the user scrolls. This enables scenarios such as asynchronously loading a page of data from a web service, as the user scrolls. In addition, the point at which more data is loaded is configurable so that users don't see blank space, or are stopped from scrolling.

[CarouselView](#) defines the following properties to control incremental loading of data:

- `RemainingItemsThreshold`, of type `int`, the threshold of items not yet visible in the list at which the `RemainingItemsThresholdReached` event will be fired.
- `RemainingItemsThresholdReachedCommand`, of type `ICommand`, which is executed when the `RemainingItemsThreshold` is reached.
- `RemainingItemsThresholdReachedCommandParameter`, of type `object`, which is the parameter that's passed to the `RemainingItemsThresholdReachedCommand`.

[CarouselView](#) also defines a `RemainingItemsThresholdReached` event that is fired when the [CarouselView](#) is scrolled far enough that `RemainingItemsThreshold` items have not been displayed. This event can be handled to load more items. In addition, when the `RemainingItemsThresholdReached` event is fired, the `RemainingItemsThresholdReachedCommand` is executed, enabling incremental data loading to take place in a viewmodel.

The default value of the `RemainingItemsThreshold` property is -1, which indicates that the `RemainingItemsThresholdReached` event will never be fired. When the property value is 0, the `RemainingItemsThresholdReached` event will be fired when the final item in the `ItemsSource` is displayed. For values greater than 0, the `RemainingItemsThresholdReached` event will be fired when the `ItemsSource` contains that number of items not yet scrolled to.

ⓘ Note

[CarouselView](#) validates the `RemainingItemsThreshold` property so that its value is always greater than or equal to -1.

The following XAML example shows a [CarouselView](#) that loads data incrementally:

XAML

```
<CarouselView ItemsSource="{Binding Animals}"
               RemainingItemsThreshold="2"

               RemainingItemsThresholdReached="OnCarouselViewRemainingItemsThresholdReached"
               RemainingItemsThresholdReachedCommand="{Binding LoadMoreDataCommand}">
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView
{
    RemainingItemsThreshold = 2
};
carouselView.RemainingItemsThresholdReached +=
OnCollectionViewRemainingItemsThresholdReached;
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Animals");
```

In this code example, the `RemainingItemsThresholdReached` event fires when there are 2 items not yet scrolled to, and in response executes the `OnCollectionViewRemainingItemsThresholdReached` event handler:

C#

```
void OnCollectionViewRemainingItemsThresholdReached(object sender, EventArgs e)
{
    // Retrieve more data here and add it to the CollectionView's
    ItemsSource collection.
}
```

⚠ Note

Data can also be loaded incrementally by binding the `RemainingItemsThresholdReachedCommand` to an [ICommand](#) implementation in the viewmodel.

Specify CarouselView layout

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [CarouselView](#) defines the following properties that control layout:

- `ItemsLayout`, of type `LinearItemsLayout`, specifies the layout to be used.
- `PeekAreaInsets`, of type `Thickness`, specifies how much to make adjacent items partially visible by.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings.

By default, a [CarouselView](#) will display its items in a horizontal orientation. A single item will be displayed on screen, with swipe gestures resulting in forwards and backwards navigation through the collection of items. However, a vertical orientation is also possible. This is because the `ItemsLayout` property is of type `LinearItemsLayout`, which inherits from the `ItemsLayout` class. The `ItemsLayout` class defines the following properties:

- `Orientation`, of type `ItemsLayoutOrientation`, specifies the direction in which the [CarouselView](#) expands as items are added.
- `SnapPointsAlignment`, of type `SnapPointsAlignment`, specifies how snap points are aligned with items.
- `SnapPointsType`, of type `SnapPointsType`, specifies the behavior of snap points when scrolling.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings. For more information about snap points, see [Snap points](#) in [Control scrolling in a CarouselView](#) guide.

The `ItemsLayoutOrientation` enumeration defines the following members:

- `Vertical` indicates that the [CarouselView](#) will expand vertically as items are added.
- `Horizontal` indicates that the [CarouselView](#) will expand horizontally as items are added.

The `LinearItemsLayout` class inherits from the `ItemsLayout` class, and defines an `ItemSpacing` property, of type `double`, that represents the empty space around each item. The default value of this property is 0, and its value must always be greater than or

equal to 0. The `LinearLayout` class also defines static `Vertical` and `Horizontal` members. These members can be used to create vertical or horizontal lists, respectively. Alternatively, a `LinearLayout` object can be created, specifying an `ItemsLayoutOrientation` enumeration member as an argument.

❗ Note

[CarouselView](#) uses the native layout engines to perform layout.

Horizontal layout

By default, `CarouselView` will display its items horizontally. Therefore, it's not necessary to set the `ItemsLayout` property to use this layout:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}">
  <CarouselView.ItemTemplate>
    <DataTemplate>
      <StackLayout>
        <Frame HasShadow="True"
          BorderColor="DarkGray"
          CornerRadius="5"
          Margin="20"
          HeightRequest="300"
          HorizontalOptions="Center"
          VerticalOptions="CenterAndExpand">
          <StackLayout>
            <Label Text="{Binding Name}"
              FontAttributes="Bold"
              FontSize="18"
              HorizontalOptions="Center"
              VerticalOptions="Center" />
            <Image Source="{Binding ImageUrl}"
              Aspect="AspectFill"
              HeightRequest="150"
              WidthRequest="150"
              HorizontalOptions="Center" />
            <Label Text="{Binding Location}"
              HorizontalOptions="Center" />
            <Label Text="{Binding Details}"
              FontAttributes="Italic"
              HorizontalOptions="Center"
              MaxLines="5"
              LineBreakMode="TailTruncation" />
          </StackLayout>
        </Frame>
      </StackLayout>
    </DataTemplate>
  </CarouselView.ItemTemplate>
</CarouselView>
```

```
        </DataTemplate>
    </CarouselView.ItemTemplate>
</CarouselView>
```

Alternatively, this layout can also be accomplished by setting the `ItemsLayout` property to a `LinearItemsLayout` object, specifying the `Horizontal` `ItemsLayoutOrientation` enumeration member as the `Orientation` property value:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}">
    <CarouselView.ItemsLayout>
        <LinearItemsLayout Orientation="Horizontal" />
    </CarouselView.ItemsLayout>
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView
{
    ...
    ItemsLayout = LinearItemsLayout.Horizontal
};
```

This results in a layout that grows horizontally as new items are added.

Vertical layout

`CarouselView` can display its items vertically by setting the `ItemsLayout` property to a `LinearItemsLayout` object, specifying the `Vertical` `ItemsLayoutOrientation` enumeration member as the `Orientation` property value:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}">
    <CarouselView.ItemsLayout>
        <LinearItemsLayout Orientation="Vertical" />
    </CarouselView.ItemsLayout>
    <CarouselView.ItemTemplate>
        <DataTemplate>
            <StackLayout>
                <Frame HasShadow="True"
                    BorderColor="DarkGray"
```

```

        CornerRadius="5"
        Margin="20"
        HeightRequest="300"
        HorizontalOptions="Center"
        VerticalOptions="CenterAndExpand">
<StackLayout>
    <Label Text="{Binding Name}"
        FontAttributes="Bold"
        FontSize="18"
        HorizontalOptions="Center"
        VerticalOptions="Center" />
    <Image Source="{Binding ImageUrl}"
        Aspect="AspectFill"
        HeightRequest="150"
        WidthRequest="150"
        HorizontalOptions="Center" />
    <Label Text="{Binding Location}"
        HorizontalOptions="Center" />
    <Label Text="{Binding Details}"
        FontAttributes="Italic"
        HorizontalOptions="Center"
        MaxLines="5"
        LineBreakMode="TailTruncation" />
</StackLayout>
</Frame>
</StackLayout>
</DataTemplate>
</CarouselView.ItemTemplate>
</CarouselView>

```

The equivalent C# code is:

```

C#

CarouselView carouselView = new CarouselView
{
    ...
    ItemsLayout = LinearItemsLayout.Vertical
};

```

This results in a layout that grows vertically as new items are added.

Partially visible adjacent items

By default, [CarouselView](#) displays full items at once. However, this behavior can be changed by setting the `PeekAreaInsets` property to a `Thickness` value that specifies how much to make adjacent items partially visible by. This can be useful to indicate to users that there are additional items to view. The following XAML shows an example of setting this property:

XAML

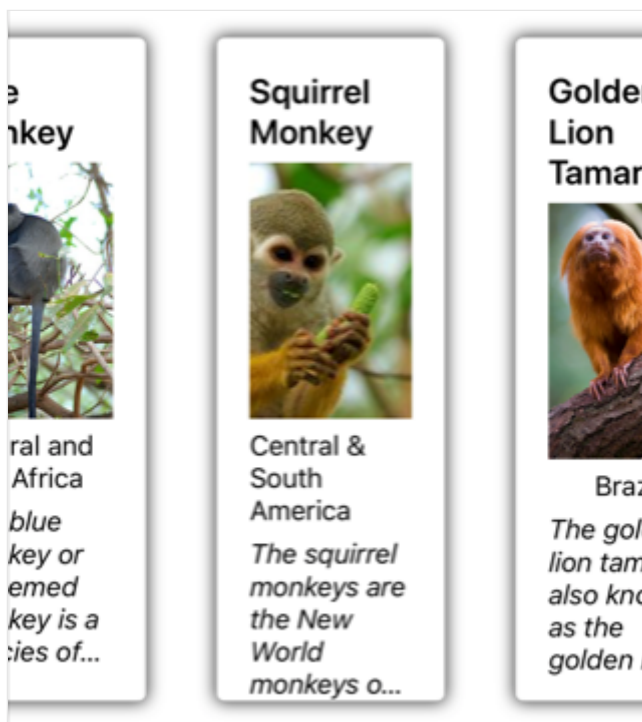
```
<CarouselView ItemsSource="{Binding Monkeys}"
    PeekAreaInsets="100">
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView
{
    ...
    PeekAreaInsets = new Thickness(100)
};
```

The result is that adjacent items are partially exposed on screen:



Item spacing

By default, there is no space between each item in a [CarouselView](#). This behavior can be changed by setting the `ItemSpacing` property on the items layout used by the [CarouselView](#).

When a [CarouselView](#) sets its `ItemsLayout` property to a `LinearItemsLayout` object, the `LinearItemsLayout.ItemSpacing` property can be set to a `double` value that represents the space between items:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}">
    <CarouselView.ItemsLayout>
        <LinearItemsLayout Orientation="Vertical"
            ItemSpacing="20" />
    </CarouselView.ItemsLayout>
    ...
</CarouselView>
```

ⓘ Note

The `LinearItemsLayout.ItemSpacing` property has a validation callback set, which ensures that the value of the property is always greater than or equal to 0.

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView
{
    ...
    ItemsLayout = new LinearItemsLayout(ItemsLayoutOrientation.Vertical)
    {
        ItemSpacing = 20
    }
};
```

This code results in a vertical layout that has a spacing of 20 between items.

Dynamic resizing of items

Items in a [CarouselView](#) can be dynamically resized at runtime by changing layout related properties of elements within the [DataTemplate](#). For example, the following code example changes the [HeightRequest](#) and [WidthRequest](#) properties of an [Image](#) object, and the [HeightRequest](#) property of its parent [Frame](#):

C#

```
void OnImageTapped(object sender, EventArgs e)
{
    Image image = sender as Image;
    image.HeightRequest = image.WidthRequest =
    image.HeightRequest.Equals(150) ? 200 : 150;
    Frame frame = ((Frame)image.Parent.Parent);
```



```
frame.HeightRequest = frame.HeightRequest.Equals(300) ? 350 : 300;
}
```

The `OnImageTapped` event handler is executed in response to an `Image` object being tapped, and changes the dimensions of the image (and its parent `Frame`, so that it's more easily viewed:



Right-to-left layout

`CarouselView` can layout its content in a right-to-left flow direction by setting its `FlowDirection` property to `RightToLeft`. However, the `FlowDirection` property should ideally be set on a page or root layout, which causes all the elements within the page, or root layout, to respond to the flow direction:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"

             x:Class="CarouselViewDemos.Views.HorizontalTemplateLayoutRTLPage"
             Title="Horizontal layout (RTL FlowDirection)"
             FlowDirection="RightToLeft">
    <CarouselView ItemsSource="{Binding Monkeys}">
        ...
    </CarouselView>
</ContentPage>
```

The default `FlowDirection` for an element with a parent is `MatchParent`. Therefore, the `CarouselView` inherits the `FlowDirection` property value from the `ContentPage`.

For more information about flow direction, see [Right to left localization](#).

Configure CarouselView interaction

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [CarouselView](#) defines the following properties that control user interaction:

- `CurrentItem`, of type `object`, the current item being displayed. This property has a default binding mode of `TwoWay`, and has a `null` value when there isn't any data to display.
- `CurrentItemChangedCommand`, of type [ICommand](#), which is executed when the current item changes.
- `CurrentItemChangedCommandParameter`, of type `object`, which is the parameter that's passed to the `CurrentItemChangedCommand`.
- `IsBounceEnabled`, of type `bool`, which specifies whether the [CarouselView](#) will bounce at a content boundary. The default value is `true`.
- `IsSwipeEnabled`, of type `bool`, which determines whether a swipe gesture will change the displayed item. The default value is `true`.
- `Loop`, of type `bool`, which determines whether the [CarouselView](#) provides looped access to its collection of items. The default value is `true`.
- `Position`, of type `int`, the index of the current item in the underlying collection. This property has a default binding mode of `TwoWay`, and has a 0 value when there isn't any data to display.
- `PositionChangedCommand`, of type [ICommand](#), which is executed when the position changes.
- `PositionChangedCommandParameter`, of type `object`, which is the parameter that's passed to the `PositionChangedCommand`.
- `VisibleViews`, of type `ObservableCollection<View>`, which is a read-only property that contains the objects for the items that are currently visible.

All of these properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings.

[CarouselView](#) defines a `CurrentItemChanged` event that's fired when the `CurrentItem` property changes, either due to user scrolling, or when an application sets the property. The `CurrentItemChangedEventArgs` object that accompanies the `CurrentItemChanged` event has two properties, both of type `object`:

- `PreviousItem` – the previous item, after the property change.

- `CurrentItem` – the current item, after the property change.

`CarouselView` also defines a `PositionChanged` event that's fired when the `Position` property changes, either due to user scrolling, or when an application sets the property. The `PositionChangedEventArgs` object that accompanies the `PositionChanged` event has two properties, both of type `int`:

- `PreviousPosition` – the previous position, after the property change.
- `CurrentPosition` – the current position, after the property change.

Respond to the current item changing

When the currently displayed item changes, the `CurrentItem` property will be set to the value of the item. When this property changes, the `CurrentItemChangedCommand` is executed with the value of the `CurrentItemChangedCommandParameter` being passed to the `ICommand`. The `Position` property is then updated, and the `CurrentItemChanged` event fires.

Important

The `Position` property changes when the `CurrentItem` property changes. This will result in the `PositionChangedCommand` being executed, and the `PositionChanged` event firing.

Event

The following XAML example shows a `CarouselView` that uses an event handler to respond to the current item changing:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}"
               CurrentItemChanged="OnCurrentItemChanged">
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView();
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
carouselView.CurrentItemChanged += OnCurrentItemChanged;
```

In this example, the `OnCurrentItemChanged` event handler is executed when the `CurrentItemChanged` event fires:

C#

```
void OnCurrentItemChanged(object sender, CurrentItemChangedEventArgs e)
{
    Monkey previousItem = e.PreviousItem as Monkey;
    Monkey currentItem = e.CurrentItem as Monkey;
}
```

In this example, the `OnCurrentItemChanged` event handler exposes the previous and current items:



Command

The following XAML example shows a `CarouselView` that uses a command to respond to the current item changing:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}"
    CurrentItemChangedCommand="{Binding ItemChangedCommand}"
    CurrentItemChangedCommandParameter="{Binding Source=
```

```
{RelativeSource Self}, Path=CurrentItem}">
...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView();
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
carouselView.SetBinding(CarouselView.CurrentItemChangedCommandProperty,
    "ItemChangedCommand");
carouselView.SetBinding(CarouselView.CurrentItemChangedCommandParameterPrope
    rty, new Binding("CurrentItem", source: RelativeBindingSource.Self));
```

In this example, the `CurrentItemChangedCommand` property binds to the `ItemChangedCommand` property, passing the `CurrentItem` property value to it as an argument. The `ItemChangedCommand` can then respond to the current item changing, as required:

C#

```
public ICommand ItemChangedCommand => new Command<Monkey>((item) =>
{
    PreviousMonkey = CurrentMonkey;
    CurrentMonkey = item;
}));
```

In this example, the `ItemChangedCommand` updates objects that store the previous and current items.

Respond to the position changing

When the currently displayed item changes, the `Position` property will be set to the index of the current item in the underlying collection. When this property changes, the `PositionChangedCommand` is executed with the value of the `PositionChangedCommandParameter` being passed to the `ICommand`. The `PositionChanged` event then fires. If the `Position` property has been programmatically changed, the `CarouselView` will be scrolled to the item that corresponds to the `Position` value.

📌 Note

Setting the `Position` property to 0 will result in the first item in the underlying collection being displayed.

Event

The following XAML example shows a `CarouselView` that uses an event handler to respond to the `Position` property changing:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}"
               PositionChanged="OnPositionChanged">
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView();
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
carouselView.PositionChanged += OnPositionChanged;
```

In this example, the `OnPositionChanged` event handler is executed when the `PositionChanged` event fires:

C#

```
void OnPositionChanged(object sender, PositionChangedEventArgs e)
{
    int previousItemPosition = e.PreviousPosition;
    int currentItemPosition = e.CurrentPosition;
}
```

In this example, the `OnCurrentItemChanged` event handler exposes the previous and current positions:

Previous item: Capuchin Monkey

Current item: Blue Monkey

Previous item position: 1

Current item position: 2

Blue Monkey



Central and East Africa

The blue monkey or diademed monkey is a species of Old World monkey native to Central and East Africa, ranging from the upper Congo River basin east to the East African...

Command

The following XAML example shows a [CarouselView](#) that uses a command to respond to the `Position` property changing:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}"
               PositionChangedCommand="{Binding PositionChangedCommand}"
               PositionChangedCommandParameter="{Binding Source=
{RelativeSource Self}, Path=Position}">
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView();
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
carouselView.SetBinding(CarouselView.PositionChangedCommandProperty,
"PositionChangedCommand");
carouselView.SetBinding(CarouselView.PositionChangedCommandParameterProperty
, new Binding("Position", source: RelativeBindingSource.Self));
```

In this example, the `PositionChangedCommand` property binds to the `PositionChangedCommand` property, passing the `Position` property value to it as an

argument. The `PositionChangedCommand` can then respond to the position changing, as required:

C#

```
public ICommand PositionChangedCommand => new Command<int>((position) =>
{
    PreviousPosition = CurrentPosition;
    CurrentPosition = position;
});
```

In this example, the `PositionChangedCommand` updates objects that store the previous and current positions.

Preset the current item

The current item in a `CarouselView` can be programmatically set by setting the `CurrentItem` property to the item. The following XAML example shows a `CarouselView` that pre-chooses the current item:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}"
               CurrentItem="{Binding CurrentItem}">
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView();
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
carouselView.SetBinding(CarouselView.CurrentItemProperty, "CurrentItem");
```

ⓘ Note

The `CurrentItem` property has a default binding mode of `TwoWay`.

The `CarouselView.CurrentItem` property data binds to the `CurrentItem` property of the connected view model, which is of type `Monkey`. By default, a `TwoWay` binding is used so that if the user changes the current item, the value of the `CurrentItem` property will be

set to the current `Monkey` object. The `CurrentItem` property is defined in the `MonkeysViewModel` class:

C#

```
public class MonkeysViewModel : INotifyPropertyChanged
{
    // ...
    public ObservableCollection<Monkey> Monkeys { get; private set; }

    public Monkey CurrentItem { get; set; }

    public MonkeysViewModel()
    {
        // ...
        CurrentItem = Monkeys.Skip(3).FirstOrDefault();
        OnPropertyChanged("CurrentItem");
    }
}
```

In this example, the `CurrentItem` property is set to the fourth item in the `Monkeys` collection:

Current item: Squirrel Monkey



Preset the position

The displayed item in a `CarouselView` can be programmatically set by setting the `Position` property to the index of the item in the underlying collection. The following XAML example shows a `CarouselView` that sets the displayed item:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}"
               Position="{Binding Position}">
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView();
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
carouselView.SetBinding(CarouselView.PositionProperty, "Position");
```

⚠ Note

The `Position` property has a default binding mode of `TwoWay`.

The `CarouselView.Position` property data binds to the `Position` property of the connected view model, which is of type `int`. By default, a `TwoWay` binding is used so that if the user scrolls through the `CarouselView`, the value of the `Position` property will be set to the index of the displayed item. The `Position` property is defined in the `MonkeysViewModel` class:

C#

```
public class MonkeysViewModel : INotifyPropertyChanged
{
    // ...
    public int Position { get; set; }

    public MonkeysViewModel()
    {
        // ...
        Position = 3;
        OnPropertyChanged("Position");
    }
}
```

In this example, the `Position` property is set to the fourth item in the `Monkeys` collection:

Current item: Squirrel Monkey

Position: 3

Squirrel Monkey



Central & South America

The squirrel monkeys are the New World monkeys of the genus Saimiri. They are the only genus in the subfamily Saimirinae. The name of the genus Saimiri is of Tupi origin, a...

Define visual states

[CarouselView](#) defines four visual states:

- `CurrentItem` represents the visual state for the currently displayed item.
- `PreviousItem` represents the visual state for the previously displayed item.
- `NextItem` represents the visual state for the next item.
- `DefaultItem` represents the visual state for the remainder of the items.

These visual states can be used to initiate visual changes to the items displayed by the [CarouselView](#).

The following XAML example shows how to define the `CurrentItem`, `PreviousItem`, `NextItem`, and `DefaultItem` visual states:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}"
    PeekAreaInsets="100">
    <CarouselView.ItemTemplate>
        <DataTemplate>
            <StackLayout>
                <VisualStateManager.VisualStateGroups>
                    <VisualStateGroup x:Name="CommonStates">
                        <VisualState x:Name="CurrentItem">
                            <VisualState.Setters>
                                <Setter Property="Scale"
                                    Value="1.1" />
                            </VisualState.Setters>
                        </VisualState>
                    </VisualStateGroup>
                </VisualStateManager.VisualStateGroups>
            </StackLayout>
        </DataTemplate>
    </CarouselView.ItemTemplate>
</CarouselView>
```

```

        </VisualState>
        <VisualState x:Name="PreviousItem">
            <VisualState.Setters>
                <Setter Property="Opacity"
                    Value="0.5" />
            </VisualState.Setters>
        </VisualState>
        <VisualState x:Name="NextItem">
            <VisualState.Setters>
                <Setter Property="Opacity"
                    Value="0.5" />
            </VisualState.Setters>
        </VisualState>
        <VisualState x:Name="DefaultItem">
            <VisualState.Setters>
                <Setter Property="Opacity"
                    Value="0.25" />
            </VisualState.Setters>
        </VisualState>
    </VisualStateGroup>
</VisualStateManager.VisualStateGroups>

    <!-- Item template content -->
    <Frame HasShadow="true">
        ...
    </Frame>
</StackLayout>
</DataTemplate>
</CarouselView.ItemTemplate>
</CarouselView>

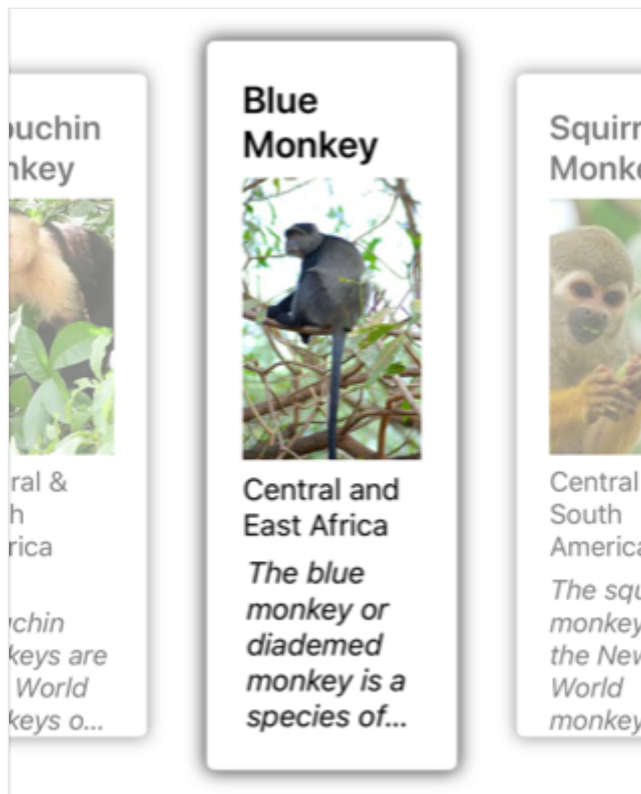
```

In this example, the `CurrentItem` visual state specifies that the current item displayed by the `CarouselView` will have its `Scale` property changed from its default value of 1 to 1.1. The `PreviousItem` and `NextItem` visual states specify that the items surrounding the current item will be displayed with an `opacity` value of 0.5. The `DefaultItem` visual state specifies that the remainder of the items displayed by the `CarouselView` will be displayed with an `opacity` value of 0.25.

⚠ Note

Alternatively, the visual states can be defined in a `Style` that has a `TargetType` property value that's the type of the root element of the `DataTemplate`, which is set as the `ItemTemplate` property value.

The following screenshot shows the `CurrentItem`, `PreviousItem`, and `NextItem` visual states:



For more information about visual states, see [Visual states](#).

Clear the current item

The `CurrentItem` property can be cleared by setting it, or the object it binds to, to `null`.

Disable bounce

By default, [CarouselView](#) bounces items at content boundaries. This can be disabled by setting the `IsBounceEnabled` property to `false`.

Disable loop

By default, [CarouselView](#) provides looped access to its collection of items. Therefore, swiping backwards from the first item in the collection will display the last item in the collection. Similarly, swiping forwards from the last item in the collection will return to the first item in the collection. This behavior can be disabled by setting the `Loop` property to `false`.

Disable swipe interaction

By default, [CarouselView](#) allows users to move through items using a swipe gesture. This swipe interaction can be disabled by setting the `IsSwipeEnabled` property to `false`.

Define an EmptyView for a CarouselView

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [CarouselView](#) defines the following properties that can be used to provide user feedback when there's no data to display:

- `EmptyView`, of type `object`, the string, binding, or view that will be displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. The default value is `null`.
- `EmptyViewTemplate`, of type [DataTemplate](#), the template to use to format the specified `EmptyView`. The default value is `null`.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings.

The main usage scenarios for setting the `EmptyView` property are displaying user feedback when a filtering operation on a [CarouselView](#) yields no data, and displaying user feedback while data is being retrieved from a web service.

Note

The `EmptyView` property can be set to a view that includes interactive content if required.

For more information about data templates, see [Data templates](#).

Display a string when data is unavailable

The `EmptyView` property can be set to a string, which will be displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. The following XAML shows an example of this scenario:

XAML

```
<CarouselView ItemsSource="{Binding EmptyMonkeys}"
    EmptyView="No items to display." />
```


The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView
{
    EmptyView = "No items to display."
};
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "EmptyMonkeys");
```

The result is that, because the data bound collection is `null`, the string set as the `EmptyView` property value is displayed.

Display views when data is unavailable

The `EmptyView` property can be set to a view, which will be displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. This can be a single view, or a view that contains multiple child views. The following XAML example shows the `EmptyView` property set to a view that contains multiple child views:

XAML

```
<StackLayout Margin="20">
    <SearchBar SearchCommand="{Binding FilterCommand}"
        SearchCommandParameter="{Binding Source={RelativeSource
Self}, Path=Text}"
        Placeholder="Filter" />
    <CarouselView ItemsSource="{Binding Monkeys}">
        <CarouselView.EmptyView>
            <ContentView>
                <StackLayout HorizontalOptions="CenterAndExpand"
                    VerticalOptions="CenterAndExpand">
                    <Label Text="No results matched your filter."
                        Margin="10,25,10,10"
                        FontAttributes="Bold"
                        FontSize="18"
                        HorizontalOptions="Fill"
                        HorizontalTextAlignment="Center" />
                    <Label Text="Try a broader filter?"
                        FontAttributes="Italic"
                        FontSize="12"
                        HorizontalOptions="Fill"
                        HorizontalTextAlignment="Center" />
                </StackLayout>
            </ContentView>
        </CarouselView.EmptyView>
    </CarouselView.ItemTemplate>
```

```

        ...
    </CarouselView.ItemTemplate>
</CarouselView>
</StackLayout>

```

In this example, what looks like a redundant has been added as the root element of the `EmptyView`. This is because internally, the `EmptyView` is added to a native container that doesn't provide any context for .NET MAUI layout. Therefore, to position the views that comprise your `EmptyView`, you must add a root layout, whose child is a layout that can position itself within the root layout.

The equivalent C# code is:

```

C#

StackLayout stackLayout = new StackLayout();
stackLayout.Add(new Label { Text = "No results matched your filter.", ... }
);
stackLayout.Add(new Label { Text = "Try a broader filter?", ... } );

SearchBar searchBar = new SearchBar { ... };
CarouselView carouselView = new CarouselView
{
    EmptyView = new ContentView
    {
        Content = stackLayout
    }
};
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");

```

When the `SearchBar` executes the `FilterCommand`, the collection displayed by the `CarouselView` is filtered for the search term stored in the `SearchBar.Text` property. If the filtering operation yields no data, the `StackLayout` set as the `EmptyView` property value is displayed.

Display a templated custom type when data is unavailable

The `EmptyView` property can be set to a custom type, whose template is displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. The `EmptyViewTemplate` property can be set to a `DataTemplate` that defines the appearance of the `EmptyView`. The following XAML shows an example of this scenario:

XAML

```
<StackLayout Margin="20">
    <SearchBar x:Name="searchBar"
        SearchCommand="{Binding FilterCommand}"
        SearchCommandParameter="{Binding Source={RelativeSource
Self}, Path=Text}"
        Placeholder="Filter" />
    <CarouselView ItemsSource="{Binding Monkeys}">
        <CarouselView.EmptyView>
            <controls:FilterData Filter="{Binding Source={x:Reference
searchBar}, Path=Text}" />
        </CarouselView.EmptyView>
        <CarouselView.EmptyViewTemplate>
            <DataTemplate>
                <Label Text="{Binding Filter, StringFormat='Your filter term
of {0} did not match any records.'}"
                    Margin="10,25,10,10"
                    FontAttributes="Bold"
                    FontSize="18"
                    HorizontalOptions="Fill"
                    HorizontalTextAlignment="Center" />
            </DataTemplate>
        </CarouselView.EmptyViewTemplate>
        <CarouselView.ItemTemplate>
            ...
        </CarouselView.ItemTemplate>
    </CarouselView>
</StackLayout>
```

The equivalent C# code is:

C#

```
SearchBar searchBar = new SearchBar { ... };
CarouselView carouselView = new CarouselView
{
    EmptyView = new FilterData { Filter = searchBar.Text },
    EmptyViewTemplate = new DataTemplate(() =>
    {
        return new Label { ... };
    })
};
```

The `FilterData` type defines a `Filter` property, and a corresponding [BindableProperty](#):

C#

```
public class FilterData : BindableObject
{
    public static readonly BindableProperty FilterProperty =
```

```
BindableProperty.Create(nameof(Filter), typeof(string), typeof(FilterData),
    null);

    public string Filter
    {
        get { return (string)GetValue(FilterProperty); }
        set { SetValue(FilterProperty, value); }
    }
}
```

The `EmptyView` property is set to a `FilterData` object, and the `Filter` property data binds to the `SearchBar.Text` property. When the `SearchBar` executes the `FilterCommand`, the collection displayed by the `CarouselView` is filtered for the search term stored in the `Filter` property. If the filtering operation yields no data, the `Label` defined in the `DataTemplate`, that's set as the `EmptyViewTemplate` property value, is displayed.

⚠ Note

When displaying a templated custom type when data is unavailable, the `EmptyViewTemplate` property can be set to a view that contains multiple child views.

Choose an EmptyView at runtime

Views that will be displayed as an `EmptyView` when data is unavailable, can be defined as `ContentView` objects in a `ResourceDictionary`. The `EmptyView` property can then be set to a specific `ContentView`, based on some business logic, at runtime. The following XAML example shows an example of this scenario:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:viewmodels="clr-namespace:CarouselViewDemos.ViewModels"
    x:Class="CarouselViewDemos.Views.EmptyViewSwapPage"
    Title="EmptyView (swap)">
    <ContentPage.BindingContext>
        <viewmodels:MonkeysViewModel />
    </ContentPage.BindingContext>
    <ContentPage.Resources>
        <ContentView x:Key="BasicEmptyView">
            <StackLayout>
                <Label Text="No items to display."
                    Margin="10,25,10,10"
                    FontAttributes="Bold"
                    FontSize="18"
                    HorizontalOptions="Fill">
```

```

        HorizontalTextAlignment="Center" />
    </StackLayout>
</ContentView>
<ContentView x:Key="AdvancedEmptyView">
    <StackLayout>
        <Label Text="No results matched your filter."
            Margin="10,25,10,10"
            FontAttributes="Bold"
            FontSize="18"
            HorizontalOptions="Fill"
            HorizontalTextAlignment="Center" />
        <Label Text="Try a broader filter?"
            FontAttributes="Italic"
            FontSize="12"
            HorizontalOptions="Fill"
            HorizontalTextAlignment="Center" />
    </StackLayout>
</ContentView>
</ContentPage.Resources>
<StackLayout Margin="20">
    <SearchBar SearchCommand="{Binding FilterCommand}"
        SearchCommandParameter="{Binding Source={RelativeSource
Self}, Path=Text}"
        Placeholder="Filter" />
    <StackLayout Orientation="Horizontal">
        <Label Text="Toggle EmptyViews" />
        <Switch Toggled="OnEmptyViewSwitchToggled" />
    </StackLayout>
    <CarouselView x:Name="carouselView"
        ItemsSource="{Binding Monkeys}">
        <CarouselView.ItemTemplate>
            ...
        </CarouselView.ItemTemplate>
    </CarouselView>
</StackLayout>
</ContentPage>

```

This XAML defines two `ContentView` objects in the page-level `ResourceDictionary`, with the `Switch` object controlling which `ContentView` object will be set as the `EmptyView` property value. When the `Switch` is toggled, the `OnEmptyViewSwitchToggled` event handler executes the `ToggleEmptyView` method:

C#

```

void ToggleEmptyView(bool isToggled)
{
    carouselView.EmptyView = isToggled ? Resources["BasicEmptyView"] :
Resources["AdvancedEmptyView"];
}

```

The `ToggleEmptyView` method sets the `EmptyView` property of the `CarouselView` object to one of the two `ContentView` objects stored in the `ResourceDictionary`, based on the value of the `Switch.IsToggled` property. When the `SearchBar` executes the `FilterCommand`, the collection displayed by the `CarouselView` is filtered for the search term stored in the `SearchBar.Text` property. If the filtering operation yields no data, the `ContentView` object set as the `EmptyView` property is displayed.

For more information about resource dictionaries, see [Resource dictionaries](#).

Choose an EmptyViewTemplate at runtime

The appearance of the `EmptyView` can be chosen at runtime, based on its value, by setting the `CarouselView.EmptyViewTemplate` property to a `DataTemplateSelector` object:

XAML

```
<ContentPage ...
    xmlns:controls="clr-namespace:CarouselViewDemos.Controls">
    <ContentPage.Resources>
        <DataTemplate x:Key="AdvancedTemplate">
            ...
        </DataTemplate>

        <DataTemplate x:Key="BasicTemplate">
            ...
        </DataTemplate>

        <controls:SearchTermDataTemplateSelector x:Key="SearchSelector"
            DefaultTemplate="
{StaticResource AdvancedTemplate}"
            OtherTemplate="
{StaticResource BasicTemplate}" />
    </ContentPage.Resources>

    <StackLayout Margin="20">
        <SearchBar x:Name="searchBar"
            SearchCommand="{Binding FilterCommand}"
            SearchCommandParameter="{Binding Source={RelativeSource
Self}, Path=Text}"
            Placeholder="Filter" />
        <CarouselView ItemsSource="{Binding Monkeys}"
            EmptyView="{Binding Source={x:Reference searchBar},
Path=Text}"
            EmptyViewTemplate="{StaticResource SearchSelector}">
            <CarouselView.ItemTemplate>
                ...
            </CarouselView.ItemTemplate>
        </CarouselView>
    </StackLayout>
</ContentPage>
```

```
</StackLayout>
</ContentPage>
```

The equivalent C# code is:

C#

```
SearchBar searchBar = new SearchBar { ... };
CarouselView carouselView = new CarouselView()
{
    EmptyView = searchBar.Text,
    EmptyViewTemplate = new SearchTermDataTemplateSelector { ... }
};
carouselView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
```

The `EmptyView` property is set to the `SearchBar.Text` property, and the `EmptyViewTemplate` property is set to a `SearchTermDataTemplateSelector` object.

When the `SearchBar` executes the `FilterCommand`, the collection displayed by the `CarouselView` is filtered for the search term stored in the `SearchBar.Text` property. If the filtering operation yields no data, the `DataTemplate` chosen by the `SearchTermDataTemplateSelector` object is set as the `EmptyViewTemplate` property and displayed.

The following example shows the `SearchTermDataTemplateSelector` class:

C#

```
public class SearchTermDataTemplateSelector : DataTemplateSelector
{
    public DataTemplate DefaultTemplate { get; set; }
    public DataTemplate OtherTemplate { get; set; }

    protected override DataTemplate OnSelectTemplate(object item,
        BindableObject container)
    {
        string query = (string)item;
        return query.ToLower().Equals("xamarin") ? OtherTemplate :
        DefaultTemplate;
    }
}
```

The `SearchTermTemplateSelector` class defines `DefaultTemplate` and `OtherTemplate` `DataTemplate` properties that are set to different data templates. The `OnSelectTemplate` override returns `DefaultTemplate`, which displays a message to the user, when the search query isn't equal to "xamarin". When the search query is equal to "xamarin", the

`OnSelectTemplate` override returns `OtherTemplate`, which displays a basic message to the user.

For more information about data template selectors, see [Create a DataTemplateSelector](#).

Control scrolling in a CarouselView

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [CarouselView](#) defines the following scroll related properties:

- `HorizontalScrollBarVisibility`, of type `ScrollBarVisibility`, which specifies when the horizontal scroll bar is visible.
- `IsDragging`, of type `bool`, which indicates whether the [CarouselView](#) is scrolling. This is a read only property, whose default value is `false`.
- `IsScrollAnimated`, of type `bool`, which specifies whether an animation will occur when scrolling the [CarouselView](#). The default value is `true`.
- `ItemsUpdatingScrollMode`, of type `ItemsUpdatingScrollMode`, which represents the scrolling behavior of the [CarouselView](#) when new items are added to it.
- `VerticalScrollBarVisibility`, of type `ScrollBarVisibility`, which specifies when the vertical scroll bar is visible.

All of these properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings.

[CarouselView](#) also defines two `ScrollTo` methods, that scroll items into view. One of the overloads scrolls the item at the specified index into view, while the other scrolls the specified item into view. Both overloads have additional arguments that can be specified to indicate the exact position of the item after the scroll has completed, and whether to animate the scroll.

[CarouselView](#) defines a `ScrollToRequested` event that is fired when one of the `ScrollTo` methods is invoked. The `ScrollToRequestedEventArgs` object that accompanies the `ScrollToRequested` event has many properties, including `IsAnimated`, `Index`, `Item`, and `ScrollToPosition`. These properties are set from the arguments specified in the `ScrollTo` method calls.

In addition, [CarouselView](#) defines a `Scrolled` event that is fired to indicate that scrolling occurred. The `ItemsViewScrolledEventArgs` object that accompanies the `Scrolled` event has many properties. For more information, see [Detect scrolling](#).

When a user swipes to initiate a scroll, the end position of the scroll can be controlled so that items are fully displayed. This feature is known as snapping, because items snap to position when scrolling stops. For more information, see [Snap points](#).

[CarouselView](#) can also load data incrementally as the user scrolls. For more information, see [Load data incrementally](#).

Detect scrolling

The `IsDragging` property can be examined to determine whether the [CarouselView](#) is currently scrolling through items.

In addition, [CarouselView](#) defines a `Scrolled` event which is fired to indicate that scrolling occurred. This event should be consumed when data about the scroll is required.

The following XAML example shows a [CarouselView](#) that sets an event handler for the `Scrolled` event:

XAML

```
<CarouselView Scrolled="OnCollectionViewScrolled">
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView();
carouselView.Scrolled += OnCarouselViewScrolled;
```

In this code example, the `OnCarouselViewScrolled` event handler is executed when the `Scrolled` event fires:

C#

```
void OnCarouselViewScrolled(object sender, ItemsViewScrolledEventArgs e)
{
    Debug.WriteLine("HorizontalDelta: " + e.HorizontalDelta);
    Debug.WriteLine("VerticalDelta: " + e.VerticalDelta);
    Debug.WriteLine("HorizontalOffset: " + e.HorizontalOffset);
    Debug.WriteLine("VerticalOffset: " + e.VerticalOffset);
    Debug.WriteLine("FirstVisibleItemIndex: " + e.FirstVisibleItemIndex);
    Debug.WriteLine("CenterItemIndex: " + e.CenterItemIndex);
    Debug.WriteLine("LastVisibleItemIndex: " + e.LastVisibleItemIndex);
}
```

In this example, the `OnCarouselViewScrolled` event handler outputs the values of the `ItemsViewScrolledEventArgs` object that accompanies the event.

❗ Important

The `Scrolled` event is fired for user initiated scrolls, and for programmatic scrolls.

Scroll an item at an index into view

One `ScrollTo` method overload scrolls the item at the specified index into view. Given a `CarouselView` object named `CarouselView`, the following example shows how to scroll the item at index 6 into view:

C#

```
carouselView.ScrollTo(6);
```

❗ Note

The `ScrollToRequested` event is fired when the `ScrollTo` method is invoked.

Scroll an item into view

Another `ScrollTo` method overload scrolls the specified item into view. Given a `CarouselView` object named `CarouselView`, the following example shows how to scroll the Proboscis Monkey item into view:

C#

```
MonkeysViewModel viewModel = BindingContext as MonkeysViewModel;  
Monkey monkey = viewModel.Monkeys.FirstOrDefault(m => m.Name == "Proboscis  
Monkey");  
carouselView.ScrollTo(monkey);
```

❗ Note

The `ScrollToRequested` event is fired when the `ScrollTo` method is invoked.

Disable scroll animation

A scrolling animation is displayed when moving between items in a [CarouselView](#). This animation occurs both for user initiated scrolls, and for programmatic scrolls. Setting the `IsScrollAnimated` property to `false` will disable the animation for both scrolling categories.

Alternatively, the `animate` argument of the `ScrollTo` method can be set to `false` to disable the scrolling animation on programmatic scrolls:

C#

```
carouselView.ScrollTo(monkey, animate: false);
```

Control scroll position

When scrolling an item into view, the exact position of the item after the scroll has completed can be specified with the `position` argument of the `ScrollTo` methods. This argument accepts a `ScrollToPosition` enumeration member.

MakeVisible

The `ScrollToPosition.MakeVisible` member indicates that the item should be scrolled until it's visible in the view:

C#

```
carouselView.ScrollTo(monkey, position: ScrollToPosition.MakeVisible);
```

This example code results in the minimal scrolling required to scroll the item into view.

ⓘ Note

The `ScrollToPosition.MakeVisible` member is used by default, if the `position` argument is not specified when calling the `ScrollTo` method.

Start

The `ScrollToPosition.Start` member indicates that the item should be scrolled to the start of the view:

C#

```
carouselView.ScrollTo(monkey, position: ScrollToPosition.Start);
```

This example code results in the item being scrolled to the start of the view.

Center

The `ScrollToPosition.Center` member indicates that the item should be scrolled to the center of the view:

C#

```
carouselViewView.ScrollTo(monkey, position: ScrollToPosition.Center);
```

This example code results in the item being scrolled to the center of the view.

End

The `ScrollToPosition.End` member indicates that the item should be scrolled to the end of the view:

C#

```
carouselViewView.ScrollTo(monkey, position: ScrollToPosition.End);
```

This example code results in the item being scrolled to the end of the view.

Control scroll position when new items are added

`CarouselView` defines a `ItemsUpdatingScrollMode` property, which is backed by a bindable property. This property gets or sets a `ItemsUpdatingScrollMode` enumeration value that represents the scrolling behavior of the `CarouselView` when new items are added to it. The `ItemsUpdatingScrollMode` enumeration defines the following members:

- `KeepItemsInView` keeps the first item in the list displayed when new items are added.
- `KeepScrollOffset` ensures that the current scroll position is maintained when new items are added.

- `KeepLastItemInView` adjusts the scroll offset to keep the last item in the list displayed when new items are added.

The default value of the `ItemsUpdatingScrollMode` property is `KeepItemsInView`.

Therefore, when new items are added to a `CarouselView` the first item in the list will remain displayed. To ensure that the last item in the list is displayed when new items are added, set the `ItemsUpdatingScrollMode` property to `KeepLastItemInView`:

XAML

```
<CarouselView ItemsUpdatingScrollMode="KeepLastItemInView">
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView
{
    ItemsUpdatingScrollMode = ItemsUpdatingScrollMode.KeepLastItemInView
};
```

Scroll bar visibility

`CarouselView` defines `HorizontalScrollBarVisibility` and `VerticalScrollBarVisibility` properties, which are backed by bindable properties. These properties get or set a `ScrollBarVisibility` enumeration value that represents when the horizontal, or vertical, scroll bar is visible. The `ScrollBarVisibility` enumeration defines the following members:

- `Default` indicates the default scroll bar behavior for the platform, and is the default value for the `HorizontalScrollBarVisibility` and `VerticalScrollBarVisibility` properties.
- `Always` indicates that scroll bars will be visible, even when the content fits in the view.
- `Never` indicates that scroll bars will not be visible, even if the content doesn't fit in the view.

Snap points

When a user swipes to initiate a scroll, the end position of the scroll can be controlled so that items are fully displayed. This feature is known as snapping, because items snap to position when scrolling stops, and is controlled by the following properties from the `ItemsLayout` class:

- `SnapPointsType`, of type `SnapPointsType`, specifies the behavior of snap points when scrolling.
- `SnapPointsAlignment`, of type `SnapPointsAlignment`, specifies how snap points are aligned with items.

These properties are backed by `BindableProperty` objects, which means that the properties can be targets of data bindings.

ⓘ Note

When snapping occurs, it will occur in the direction that produces the least amount of motion.

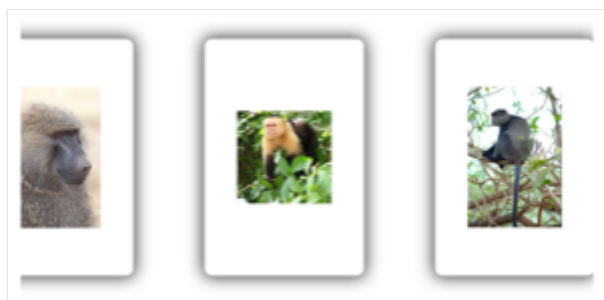
Snap points type

The `SnapPointsType` enumeration defines the following members:

- `None` indicates that scrolling does not snap to items.
- `Mandatory` indicates that content always snaps to the closest snap point to where scrolling would naturally stop, along the direction of inertia.
- `MandatorySingle` indicates the same behavior as `Mandatory`, but only scrolls one item at a time.

By default on a `CarouselView`, the `SnapPointsType` property is set to `SnapPointsType.MandatorySingle`, which ensures that scrolling only scrolls one item at a time.

The following screenshot shows a `CarouselView` with snapping turned off:



Snap points alignment

The `SnapPointsAlignment` enumeration defines `Start`, `Center`, and `End` members.

Important

The value of the `SnapPointsAlignment` property is only respected when the `SnapPointsType` property is set to `Mandatory`, or `MandatorySingle`.

Start

The `SnapPointsAlignment.Start` member indicates that snap points are aligned with the leading edge of items. The following XAML example shows how to set this enumeration member:

XAML

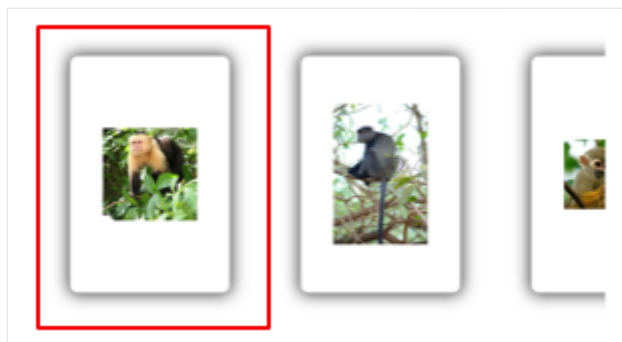
```
<CarouselView ItemsSource="{Binding Monkeys}"
    PeekAreaInsets="100">
    <CarouselView.ItemsLayout>
        <LinearItemsLayout Orientation="Horizontal"
            SnapPointsType="MandatorySingle"
            SnapPointsAlignment="Start" />
    </CarouselView.ItemsLayout>
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView
{
    ItemsLayout = new LinearItemsLayout(ItemsLayoutOrientation.Horizontal)
    {
        SnapPointsType = SnapPointsType.MandatorySingle,
        SnapPointsAlignment = SnapPointsAlignment.Start
    },
    // ...
};
```

When a user swipes to initiate a scroll in a horizontally scrolling `CarouselView`, the left item will be aligned with the left of the view:



Center

The `SnapPointsAlignment.Center` member indicates that snap points are aligned with the center of items.

By default on a [CarouselView](#), the `SnapPointsAlignment` property is set to `Center`. However, for completeness, the following XAML example shows how to set this enumeration member:

XAML

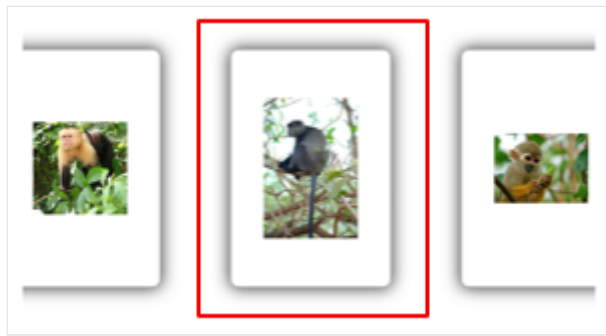
```
<CarouselView ItemsSource="{Binding Monkeys}"
    PeekAreaInsets="100">
    <CarouselView.ItemsLayout>
        <LinearItemsLayout Orientation="Horizontal"
            SnapPointsType="MandatorySingle"
            SnapPointsAlignment="Center" />
    </CarouselView.ItemsLayout>
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView
{
    ItemsLayout = new LinearItemsLayout(ItemsLayoutOrientation.Horizontal)
    {
        SnapPointsType = SnapPointsType.MandatorySingle,
        SnapPointsAlignment = SnapPointsAlignment.Center
    },
    // ...
};
```

When a user swipes to initiate a scroll in a horizontally scrolling [CarouselView](#), the center item will be aligned with the center of the view:



End

The `SnapPointsAlignment.End` member indicates that snap points are aligned with the trailing edge of items. The following XAML example shows how to set this enumeration member:

XAML

```
<CarouselView ItemsSource="{Binding Monkeys}"
    PeekAreaInsets="100">
    <CarouselView.ItemsLayout>
        <LinearItemsLayout Orientation="Horizontal"
            SnapPointsType="MandatorySingle"
            SnapPointsAlignment="End" />
    </CarouselView.ItemsLayout>
    ...
</CarouselView>
```

The equivalent C# code is:

C#

```
CarouselView carouselView = new CarouselView
{
    ItemsLayout = new LinearItemsLayout(ItemsLayoutOrientation.Horizontal)
    {
        SnapPointsType = SnapPointsType.MandatorySingle,
        SnapPointsAlignment = SnapPointsAlignment.End
    },
    // ...
};
```

When a user swipes to initiate a scroll in a horizontally scrolling [CarouselView](#), the right item will be aligned with the right of the view.



CheckBox

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [CheckBox](#) is a type of button that can either be checked or empty. When a checkbox is checked, it's considered to be on. When a checkbox is empty, it's considered to be off.

[CheckBox](#) defines the following properties:

- `IsChecked`, of type `bool`, which indicates whether the [CheckBox](#) is checked. This property has a default binding mode of `TwoWay`.
- `Color`, of type [Color](#), which indicates the color of the [CheckBox](#).

These properties are backed by [BindableProperty](#) objects, which means that they can be styled, and be the target of data bindings.

[CheckBox](#) defines a `CheckedChanged` event that's raised when the `IsChecked` property changes, either through user manipulation or when an application sets the `IsChecked` property. The `CheckedChangedEventArgs` object that accompanies the `CheckedChanged` event has a single property named `value`, of type `bool`. When the event is raised, the value of the `value` property is set to the new value of the `IsChecked` property.

Create a CheckBox

The following example shows how to instantiate a [CheckBox](#) in XAML:

XAML

```
<CheckBox />
```

This XAML results in the appearance shown in the following screenshot:



By default, the [CheckBox](#) is empty. The [CheckBox](#) can be checked by user manipulation, or by setting the `IsChecked` property to `true`:

XAML

```
<CheckBox IsChecked="true" />
```

This XAML results in the appearance shown in the following screenshot:



Alternatively, a `CheckBox` can be created in code:

C#

```
CheckBox checkBox = new CheckBox { IsChecked = true };
```

Respond to a `CheckBox` changing state

When the `IsChecked` property changes, either through user manipulation or when an application sets the `IsChecked` property, the `CheckedChanged` event fires. An event handler for this event can be registered to respond to the change:

XAML

```
<CheckBox CheckedChanged="OnCheckBoxCheckedChanged" />
```

The code-behind file contains the handler for the `CheckedChanged` event:

C#

```
void OnCheckBoxCheckedChanged(object sender, CheckedChangedEventArgs e)
{
    // Perform required operation after examining e.Value
}
```

The `sender` argument is the `CheckBox` responsible for this event. You can use this to access the `CheckBox` object, or to distinguish between multiple `CheckBox` objects sharing the same `CheckedChanged` event handler.

Alternatively, an event handler for the `CheckedChanged` event can be registered in code:

C#

```
CheckBox checkBox = new CheckBox { ... };
checkBox.CheckedChanged += (sender, e) =>
{
```

```
// Perform required operation after examining e.Value  
};
```

Data bind a CheckBox

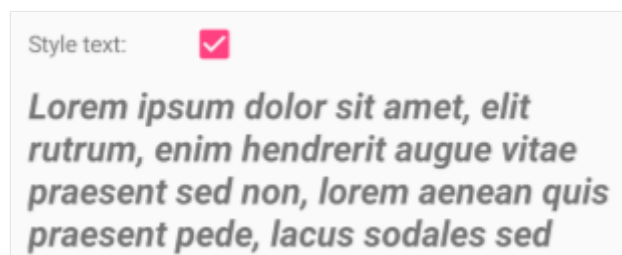
The `CheckedChanged` event handler can be eliminated by using data binding and triggers to respond to a `CheckBox` being checked or empty:

XAML

```
<CheckBox x:Name="checkBox" />  
<Label Text="Lorem ipsum dolor sit amet, elit rutrum, enim hendrerit augue  
vitae praesent sed non, lorem aenean quis praesent pede.">  
    <Label.Triggers>  
        <DataTrigger TargetType="Label"  
            Binding="{Binding Source={x:Reference checkBox},  
Path=IsChecked}"  
            Value="true">  
            <Setter Property="FontAttributes"  
                Value="Italic, Bold" />  
            <Setter Property="FontSize"  
                Value="18" />  
        </DataTrigger>  
    </Label.Triggers>  
</Label>
```

In this example, the `Label` uses a binding expression in a data trigger to monitor the `IsChecked` property of the `CheckBox`. When this property becomes `true`, the `FontAttributes` and `FontSize` properties of the `Label` change. When the `IsChecked` property returns to `false`, the `FontAttributes` and `FontSize` properties of the `Label` are reset to their initial state.

The following screenshot shows the `Label` formatting when the `CheckBox` is checked:



For more information about triggers, see [Triggers](#).

Disable a Checkbox

Sometimes an application enters a state where a [CheckBox](#) being checked is not a valid operation. In such cases, the [CheckBox](#) can be disabled by setting its `IsEnabled` property to `false`.

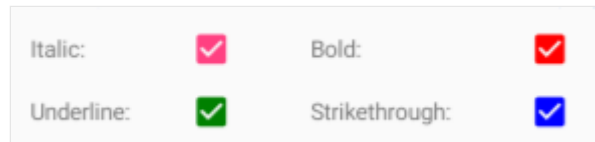
CheckBox appearance

In addition to the properties that [CheckBox](#) inherits from the [View](#) class, [CheckBox](#) also defines a `Color` property that sets its color to a [Color](#):

XAML

```
<CheckBox Color="Red" />
```

The following screenshot shows a series of checked [CheckBox](#) objects, where each object has its `Color` property set to a different [Color](#):



CheckBox visual states

[CheckBox](#) has an `IsChecked` [VisualState](#) that can be used to initiate a visual change to the [CheckBox](#) when it becomes checked.

The following XAML example shows how to define a visual state for the `IsChecked` state:

XAML

```
<CheckBox ...>
    <VisualStateManager.VisualStateGroups>
        <VisualStateGroupList>
            <VisualStateGroup x:Name="CommonStates">
                <VisualState x:Name="Normal">
                    <VisualState.Setters>
                        <Setter Property="Color"
                            Value="Red" />
                    </VisualState.Setters>
                </VisualState>

                <VisualState x:Name="IsChecked">
                    <VisualState.Setters>
                        <Setter Property="Color"
                            Value="Green" />
                    </VisualState.Setters>
                </VisualState>
            </VisualStateGroup>
        </VisualStateGroupList>
    </VisualStateManager.VisualStateGroups>
</CheckBox>
```

```
        </VisualState>
      </VisualStateGroup>
    </VisualStateGroupList>
  </VisualStateManager.VisualStateGroups>
</CheckBox>
```

In this example, the `IsChecked` [VisualState](#) specifies that when the [CheckBox](#) is checked, its `Color` property will be set to green. The `Normal` [VisualState](#) specifies that when the [CheckBox](#) is in a normal state, its `Color` property will be set to red. Therefore, the overall effect is that the [CheckBox](#) is red when it's empty, and green when it's checked.

For more information about visual states, see [Visual states](#).

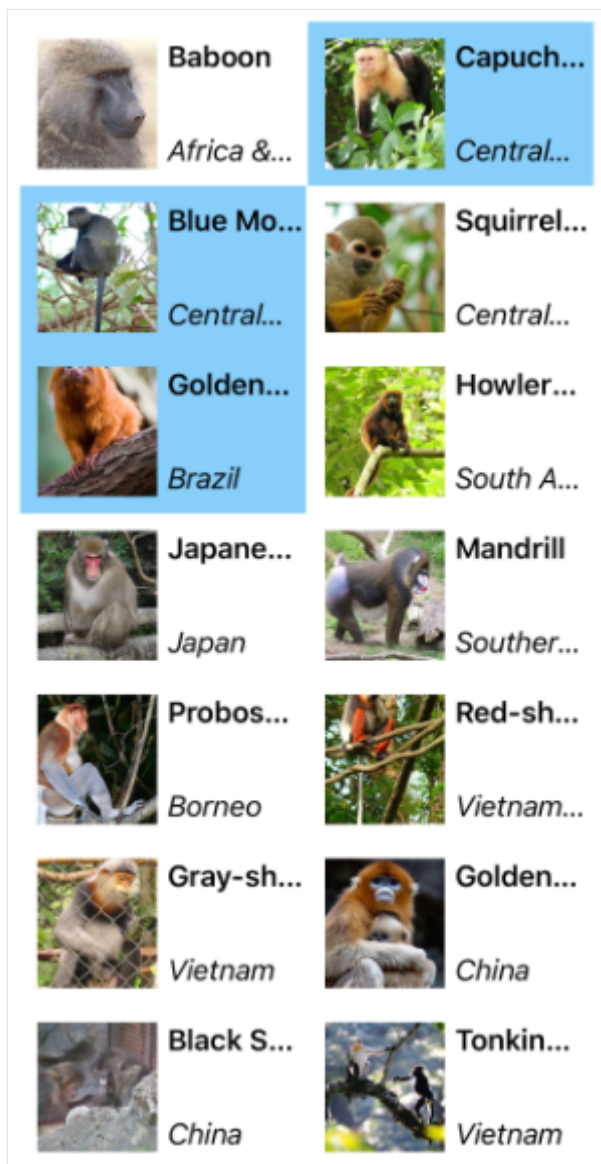
CollectionView

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [CollectionView](#) is a view for presenting lists of data using different layout specifications. It aims to provide a more flexible, and performant alternative to [ListView](#).

The following screenshot shows a [CollectionView](#) that uses a two-column vertical grid and allows multiple selections:



[CollectionView](#) should be used for presenting lists of data that require scrolling or selection. A bindable layout can be used when the data to be displayed doesn't require scrolling or selection. For more information, see [BindableLayout](#).

CollectionView and ListView differences

While the [CollectionView](#) and [ListView](#) APIs are similar, there are some notable differences:

- [CollectionView](#) has a flexible layout model, which allows data to be presented vertically or horizontally, in a list or a grid.
- [CollectionView](#) supports single and multiple selection.
- [CollectionView](#) has no concept of cells. Instead, a data template is used to define the appearance of each item of data in the list.
- [CollectionView](#) automatically utilizes the virtualization provided by the underlying native controls.
- [CollectionView](#) reduces the API surface of [ListView](#). Many properties and events from [ListView](#) are not present in [CollectionView](#).
- [CollectionView](#) does not include built-in separators.
- [CollectionView](#) will throw an exception if its `ItemsSource` is updated off the UI thread.

Move from ListView to CollectionView

[ListView](#) implementations can be migrated to [CollectionView](#) implementations with the help of the following table:

 Expand table

Concept	ListView API	CollectionView
Data	<code>ItemsSource</code>	A CollectionView is populated with data by setting its <code>ItemsSource</code> property. For more information, see Populate a CollectionView with data .
Item appearance	<code>ItemTemplate</code>	The appearance of each item in a CollectionView can be defined by setting the <code>ItemTemplate</code> property to a DataTemplate . For more information, see Define item appearance .
Cells	TextCell , ImageCell , ViewCell	CollectionView has no concept of cells, and therefore no concept of disclosure indicators. Instead, a data template is used to define the appearance of each item of data in the list.

Concept	ListView API	CollectionView
Row separators	<code>SeparatorColor</code> , <code>SeparatorVisibility</code>	CollectionView does not include built-in separators. These can be provided, if desired, in the item template.
Selection	<code>SelectionMode</code> , <code>SelectedItem</code>	CollectionView supports single and multiple selection. For more information, see Configure CollectionView item selection .
Row height	<code>HasUnevenRows</code> , <code>RowHeight</code>	In a CollectionView , the row height of each item is determined by the <code>ItemSizingStrategy</code> property. For more information, see Item sizing .
Caching	<code>CachingStrategy</code>	CollectionView automatically uses the virtualization provided by the underlying native controls.
Headers and footers	<code>Header</code> , <code>HeaderElement</code> , <code>HeaderTemplate</code> , <code>Footer</code> , <code>FooterElement</code> , <code>FooterTemplate</code>	CollectionView can present a header and footer that scroll with the items in the list, via the <code>Header</code> , <code>Footer</code> , <code>HeaderTemplate</code> , and <code>FooterTemplate</code> properties. For more information, see Headers and footers .
Grouping	<code>GroupDisplayBinding</code> , <code>GroupHeaderTemplate</code> , <code>GroupShortNameBinding</code> , <code>IsGroupingEnabled</code>	CollectionView displays correctly grouped data by setting its <code>IsGrouped</code> property to <code>true</code> . Group headers and group footers can be customized by setting the <code>GroupHeaderTemplate</code> and <code>GroupFooterTemplate</code> properties to DataTemplate objects. For more information, see Display grouped data in a CollectionView .
Pull to refresh	<code>IsPullToRefreshEnabled</code> , <code>IsRefreshing</code> , <code>RefreshAllowed</code> , <code>RefreshCommand</code> , <code>RefreshControlColor</code> , <code>BeginRefresh()</code> , <code>EndRefresh()</code>	Pull to refresh functionality is supported by setting a CollectionView as the child of a RefreshView . For more information, see Pull to refresh .
Context menu items	<code>ContextActions</code>	Context menu items are supported by setting a SwipeView as the root view in the DataTemplate that defines the appearance of each item of data in the CollectionView . For more information, see Context menus .
Scrolling	<code>ScrollTo()</code>	CollectionView defines <code>ScrollTo</code> methods, which scroll items into view. For more information, see Control scrolling in a CollectionView .

Populate a CollectionView with data

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [CollectionView](#) includes the following properties that define the data to be displayed, and its appearance:

- `ItemsSource`, of type `IEnumerable`, specifies the collection of items to be displayed, and has a default value of `null`.
- `ItemTemplate`, of type [DataTemplate](#), specifies the template to apply to each item in the collection of items to be displayed.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings.

[CollectionView](#) defines a `ItemsUpdatingScrollMode` property that represents the scrolling behavior of the [CollectionView](#) when new items are added to it. For more information about this property, see [Control scroll position when new items are added](#).

[CollectionView](#) supports incremental data virtualization as the user scrolls. For more information, see [Load data incrementally](#).

Populate a CollectionView with data

A [CollectionView](#) is populated with data by setting its `ItemsSource` property to any collection that implements `IEnumerable`. By default, [CollectionView](#) displays items in a vertical list.

Important

If the [CollectionView](#) is required to refresh as items are added, removed, or changed in the underlying collection, the underlying collection should be an `IEnumerable` collection that sends property change notifications, such as `ObservableCollection`.

[CollectionView](#) can be populated with data by using data binding to bind its `ItemsSource` property to an `IEnumerable` collection. In XAML, this is achieved with the `Binding` markup extension:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}" />
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView();  
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
```

In this example, the `ItemsSource` property data binds to the `Monkeys` property of the connected viewmodel.

ⓘ Note

Compiled bindings can be enabled to improve data binding performance in .NET MAUI applications. For more information, see [Compiled bindings](#).

For information on how to change the [CollectionView](#) layout, see [Specify CollectionView layout](#). For information on how to define the appearance of each item in the [CollectionView](#), see [Define item appearance](#). For more information about data binding, see [Data binding](#).

⚠ Warning

[CollectionView](#) will throw an exception if its `ItemsSource` is updated off the UI thread.

Define item appearance

The appearance of each item in the [CollectionView](#) can be defined by setting the `CollectionView.ItemTemplate` property to a [DataTemplate](#):

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}">  
  <CollectionView.ItemTemplate>  
    <DataTemplate>  
      <Grid Padding="10">  
        <Grid.RowDefinitions>  
          <RowDefinition Height="Auto" />  
        </Grid.RowDefinitions>  
      </Grid>  
    </DataTemplate>  
  </CollectionView.ItemTemplate>  
</CollectionView>
```

```

        <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="Auto" />
        <ColumnDefinition Width="Auto" />
    </Grid.ColumnDefinitions>
    <Image Grid.RowSpan="2"
        Source="{Binding ImageUrl}"
        Aspect="AspectFill"
        HeightRequest="60"
        WidthRequest="60" />
    <Label Grid.Column="1"
        Text="{Binding Name}"
        FontAttributes="Bold" />
    <Label Grid.Row="1"
        Grid.Column="1"
        Text="{Binding Location}"
        FontAttributes="Italic"
        VerticalOptions="End" />
    </Grid>
</DataTemplate>
</CollectionView.ItemTemplate>
...
</CollectionView>

```

The equivalent C# code is:

C#

```

CollectionView collectionView = new CollectionView();
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");

collectionView.ItemTemplate = new DataTemplate(() =>
{
    Grid grid = new Grid { Padding = 10 };
    grid.RowDefinitions.Add(new RowDefinition { Height = GridLength.Auto });
    grid.RowDefinitions.Add(new RowDefinition { Height = GridLength.Auto });
    grid.ColumnDefinitions.Add(new ColumnDefinition { Width =
GridLength.Auto });
    grid.ColumnDefinitions.Add(new ColumnDefinition { Width =
GridLength.Auto });

    Image image = new Image { Aspect = Aspect.AspectFill, HeightRequest =
60, WidthRequest = 60 };
    image.SetBinding(Image.SourceProperty, "ImageUrl");

    Label nameLabel = new Label { FontAttributes = FontAttributes.Bold };
    nameLabel.SetBinding(Label.TextProperty, "Name");

    Label locationLabel = new Label { FontAttributes =
FontAttributes.Italic, VerticalOptions = LayoutOptions.End };
    locationLabel.SetBinding(Label.TextProperty, "Location");

```

```
Grid.SetRowSpan(image, 2);

grid.Add(image);
grid.Add(nameLabel, 1, 0);
grid.Add(locationLabel, 1, 1);

return grid;
});
```

The elements specified in the [DataTemplate](#) define the appearance of each item in the list. In the example, layout within the [DataTemplate](#) is managed by a [Grid](#). The [Grid](#) contains an [Image](#) object, and two [Label](#) objects, that all bind to properties of the `Monkey` class:

C#

```
public class Monkey
{
    public string Name { get; set; }
    public string Location { get; set; }
    public string Details { get; set; }
    public string ImageUrl { get; set; }
}
```

The following screenshot shows the result of templating each item in the list:

	Baboon <i>Africa & Asia</i>
	Capuchin Monkey <i>Central & South America</i>
	Blue Monkey <i>Central and East Africa</i>
	Squirrel Monkey <i>Central & South America</i>
	Golden Lion Tamarin <i>Brazil</i>
	Howler Monkey <i>South America</i>
	Japanese Macaque <i>Japan</i>
	Mandrill <i>Southern Cameroon, Gabon, Equatorial Guinea, and Congo</i>

💡 Tip

Placing a [CollectionView](#) inside a [VerticalStackLayout](#) can stop the [CollectionView](#) scrolling, and can limit the number of items that are displayed. In this situation, replace the [VerticalStackLayout](#) with a [Grid](#).

For more information about data templates, see [Data templates](#).

Choose item appearance at runtime

The appearance of each item in the [CollectionView](#) can be chosen at runtime, based on the item value, by setting the `CollectionView.ItemTemplate` property to a [DataTemplateSelector](#) object:

XAML

```
<ContentPage ...
    xmlns:controls="clr-namespace:CollectionViewDemos.Controls">
```

```

<ContentPage.Resources>
    <DataTemplate x:Key="AmericanMonkeyTemplate">
        ...
    </DataTemplate>

    <DataTemplate x:Key="OtherMonkeyTemplate">
        ...
    </DataTemplate>

    <controls:MonkeyDataTemplateSelector x:Key="MonkeySelector"
        AmericanMonkey="{StaticResource
AmericanMonkeyTemplate}"
        OtherMonkey="{StaticResource
OtherMonkeyTemplate}" />
</ContentPage.Resources>

<CollectionView ItemsSource="{Binding Monkeys}"
    ItemTemplate="{StaticResource MonkeySelector}" />
</ContentPage>

```

The equivalent C# code is:

```

C#

CollectionView collectionView = new CollectionView
{
    ItemTemplate = new MonkeyDataTemplateSelector { ... }
};
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");

```

The `ItemTemplate` property is set to a `MonkeyDataTemplateSelector` object. The following example shows the `MonkeyDataTemplateSelector` class:

```

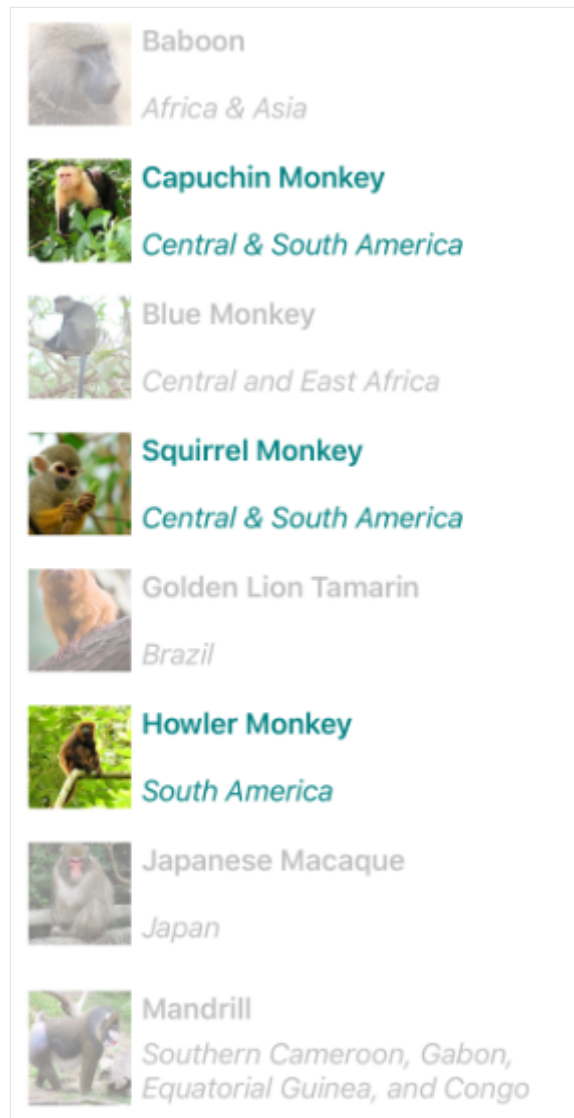
C#

public class MonkeyDataTemplateSelector : DataTemplateSelector
{
    public DataTemplate AmericanMonkey { get; set; }
    public DataTemplate OtherMonkey { get; set; }

    protected override DataTemplate OnSelectTemplate(object item,
BindableObject container)
    {
        return ((Monkey)item).Location.Contains("America") ? AmericanMonkey
: OtherMonkey;
    }
}

```

The `MonkeyDataTemplateSelector` class defines `AmericanMonkey` and `OtherMonkey` `DataTemplate` properties that are set to different data templates. The `OnSelectTemplate` override returns the `AmericanMonkey` template, which displays the monkey name and location in teal, when the monkey name contains "America". When the monkey name doesn't contain "America", the `OnSelectTemplate` override returns the `OtherMonkey` template, which displays the monkey name and location in silver:



For more information about data template selectors, see [Create a DataTemplateSelector](#).

Important

When using [CollectionView](#), never set the root element of your [DataTemplate](#) objects to a [ViewCell](#). This will result in an exception being thrown because [CollectionView](#) has no concept of cells.

Context menus

[CollectionView](#) supports context menus for items of data through the [SwipeView](#), which reveals the context menu with a swipe gesture. The [SwipeView](#) is a container control that wraps around an item of content, and provides context menu items for that item of content. Therefore, context menus are implemented for a [CollectionView](#) by creating a [SwipeView](#) that defines the content that the [SwipeView](#) wraps around, and the context menu items that are revealed by the swipe gesture. This is achieved by setting the [SwipeView](#) as the root view in the [DataTemplate](#) that defines the appearance of each item of data in the [CollectionView](#):

XAML

```
<CollectionView x:Name="collectionView"
    ItemsSource="{Binding Monkeys}">
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <SwipeView>
                <SwipeView.LeftItems>
                    <SwipeItems>
                        <SwipeItem Text="Favorite"
                            IconImageSource="favorite.png"
                            BackgroundColor="LightGreen"
                            Command="{Binding Source={x:Reference
collectionView}, Path=BindingContext.FavoriteCommand}"
                            CommandParameter="{Binding}" />
                        <SwipeItem Text="Delete"
                            IconImageSource="delete.png"
                            BackgroundColor="LightPink"
                            Command="{Binding Source={x:Reference
collectionView}, Path=BindingContext.DeleteCommand}"
                            CommandParameter="{Binding}" />
                    </SwipeItems>
                </SwipeView.LeftItems>
                <Grid BackgroundColor="White"
                    Padding="10">
                    <!-- Define item appearance -->
                </Grid>
            </SwipeView>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView();
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");

collectionView.ItemTemplate = new DataTemplate(() =>
{
```

```

// Define item appearance
Grid grid = new Grid { Padding = 10, BackgroundColor = Colors.White };
// ...

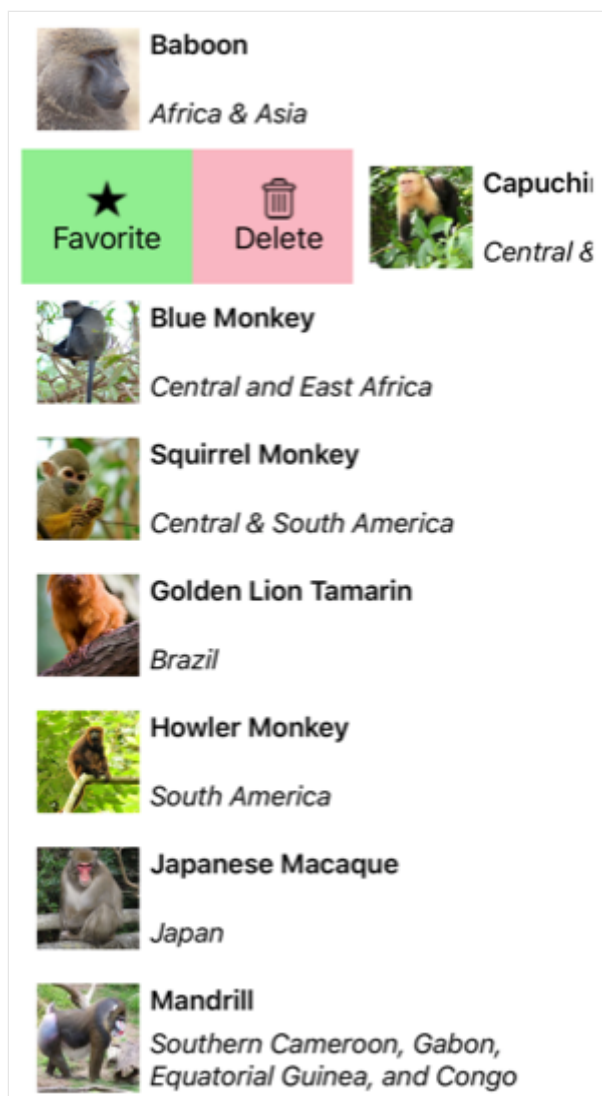
SwipeView swipeView = new SwipeView();
SwipeItem favoriteSwipeItem = new SwipeItem
{
    Text = "Favorite",
    IconImageSource = "favorite.png",
    BackgroundColor = Colors.LightGreen
};
favoriteSwipeItem.SetBinding(MenuItem.CommandProperty, new
Binding("BindingContext.FavoriteCommand", source: collectionView));
favoriteSwipeItem.SetBinding(MenuItem.CommandParameterProperty, ".");

SwipeItem deleteSwipeItem = new SwipeItem
{
    Text = "Delete",
    IconImageSource = "delete.png",
    BackgroundColor = Colors.LightPink
};
deleteSwipeItem.SetBinding(MenuItem.CommandProperty, new
Binding("BindingContext.DeleteCommand", source: collectionView));
deleteSwipeItem.SetBinding(MenuItem.CommandParameterProperty, ".");

swipeView.LeftItems = new SwipeItems { favoriteSwipeItem,
deleteSwipeItem };
swipeView.Content = grid;
return swipeView;
});

```

In this example, the [SwipeView](#) content is a [Grid](#) that defines the appearance of each item in the [CollectionView](#). The swipe items are used to perform actions on the [SwipeView](#) content, and are revealed when the control is swiped from the left side:



[SwipeView](#) supports four different swipe directions, with the swipe direction being defined by the directional `SwipeItems` collection the `SwipeItems` objects are added to. By default, a swipe item is executed when it's tapped by the user. In addition, once a swipe item has been executed the swipe items are hidden and the [SwipeView](#) content is re-displayed. However, these behaviors can be changed.

For more information about the [SwipeView](#) control, see [SwipeView](#).

Pull to refresh

[CollectionView](#) supports pull to refresh functionality through the [RefreshView](#), which enables the data being displayed to be refreshed by pulling down on the list of items. The [RefreshView](#) is a container control that provides pull to refresh functionality to its child, provided that the child supports scrollable content. Therefore, pull to refresh is implemented for a [CollectionView](#) by setting it as the child of a [RefreshView](#):

XAML

```

<RefreshView IsRefreshing="{Binding IsRefreshing}"
              Command="{Binding RefreshCommand}">
    <CollectionView ItemsSource="{Binding Animals}">
        ...
    </CollectionView>
</RefreshView>

```

The equivalent C# code is:

C#

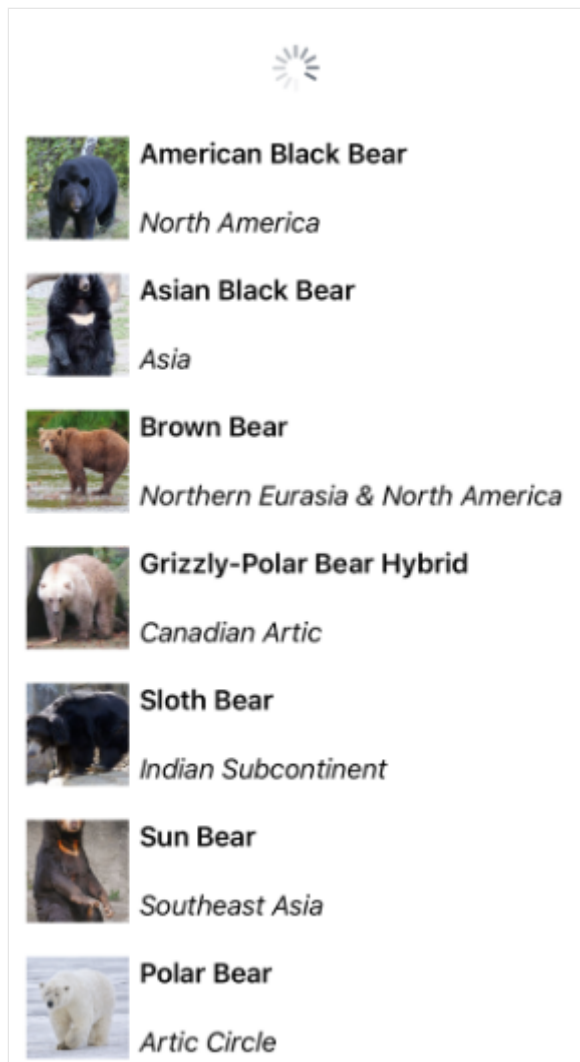
```

RefreshView refreshView = new RefreshView();
ICommand refreshCommand = new Command(() =>
{
    // IsRefreshing is true
    // Refresh data here
    refreshView.IsRefreshing = false;
});
refreshView.Command = refreshCommand;

CollectionView collectionView = new CollectionView();
collectionView.SetBinding(CollectionView.ItemsSourceProperty, "Animals");
refreshView.Content = collectionView;
// ...

```

When the user initiates a refresh, the `ICommand` defined by the `Command` property is executed, which should refresh the items being displayed. A refresh visualization is shown while the refresh occurs, which consists of an animated progress circle:



The value of the `RefreshView.IsRefreshing` property indicates the current state of the [RefreshView](#). When a refresh is triggered by the user, this property will automatically transition to `true`. Once the refresh completes, you should reset the property to `false`.

For more information about [RefreshView](#), see [RefreshView](#).

Load data incrementally

[CollectionView](#) supports incremental data virtualization as the user scrolls. This enables scenarios such as asynchronously loading a page of data from a web service, as the user scrolls. In addition, the point at which more data is loaded is configurable so that users don't see blank space, or are stopped from scrolling.

Warning

Don't attempt to load data incrementally in a [CollectionView](#) that's in a [StackLayout](#). This scenario will cause an infinite loop to occur where the [CollectionView](#) will keep expanding.

[CollectionView](#) defines the following properties to control incremental loading of data:

- `RemainingItemsThreshold`, of type `int`, the threshold of items not yet visible in the list at which the `RemainingItemsThresholdReached` event will be fired.
- `RemainingItemsThresholdReachedCommand`, of type `ICommand`, which is executed when the `RemainingItemsThreshold` is reached.
- `RemainingItemsThresholdReachedCommandParameter`, of type `object`, which is the parameter that's passed to the `RemainingItemsThresholdReachedCommand`.

[CollectionView](#) also defines a `RemainingItemsThresholdReached` event that is fired when the [CollectionView](#) is scrolled far enough that `RemainingItemsThreshold` items have not been displayed. This event can be handled to load more items. In addition, when the `RemainingItemsThresholdReached` event is fired, the `RemainingItemsThresholdReachedCommand` is executed, enabling incremental data loading to take place in a viewmodel.

The default value of the `RemainingItemsThreshold` property is -1, which indicates that the `RemainingItemsThresholdReached` event will never be fired. When the property value is 0, the `RemainingItemsThresholdReached` event will be fired when the final item in the `ItemsSource` is displayed. For values greater than 0, the `RemainingItemsThresholdReached` event will be fired when the `ItemsSource` contains that number of items not yet scrolled to.

❗ Note

[CollectionView](#) validates the `RemainingItemsThreshold` property so that its value is always greater than or equal to -1.

The following XAML example shows a [CollectionView](#) that loads data incrementally:

XAML

```
<CollectionView ItemsSource="{Binding Animals}"
                RemainingItemsThreshold="5"

                RemainingItemsThresholdReached="OnCollectionViewRemainingItemsThresholdReached">
    ...
</CollectionView>
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView
{
    RemainingItemsThreshold = 5
};
collectionView.RemainingItemsThresholdReached +=
OnCollectionViewRemainingItemsThresholdReached;
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Animals");
```

In this code example, the `RemainingItemsThresholdReached` event fires when there are 5 items not yet scrolled to, and in response executes the `OnCollectionViewRemainingItemsThresholdReached` event handler:

C#

```
void OnCollectionViewRemainingItemsThresholdReached(object sender, EventArgs e)
{
    // Retrieve more data here and add it to the CollectionView's
    ItemsSource collection.
}
```

ⓘ Note

Data can also be loaded incrementally by binding the `RemainingItemsThresholdReachedCommand` to an [ICommand](#) implementation in the viewmodel.

Specify `CollectionView` layout

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) `CollectionView` defines the following properties that control layout:

- `ItemsLayout`, of type `IItemsLayout`, specifies the layout to be used.
- `ItemSizingStrategy`, of type `ItemSizingStrategy`, specifies the item measure strategy to be used.

These properties are backed by `BindableProperty` objects, which means that the properties can be targets of data bindings.

By default, a `CollectionView` will display its items in a vertical list. However, any of the following layouts can be used:

- Vertical list – a single column list that grows vertically as new items are added.
- Horizontal list – a single row list that grows horizontally as new items are added.
- Vertical grid – a multi-column grid that grows vertically as new items are added.
- Horizontal grid – a multi-row grid that grows horizontally as new items are added.

These layouts can be specified by setting the `ItemsLayout` property to class that derives from the `ItemsLayout` class. This class defines the following properties:

- `Orientation`, of type `ItemsLayoutOrientation`, specifies the direction in which the `CollectionView` expands as items are added.
- `SnapPointsAlignment`, of type `SnapPointsAlignment`, specifies how snap points are aligned with items.
- `SnapPointsType`, of type `SnapPointsType`, specifies the behavior of snap points when scrolling.

These properties are backed by `BindableProperty` objects, which means that the properties can be targets of data bindings. For more information about snap points, see [Snap points in Control scrolling in a CollectionView](#).

The `ItemsLayoutOrientation` enumeration defines the following members:

- `Vertical` indicates that the `CollectionView` will expand vertically as items are added.

- `Horizontal` indicates that the `CollectionView` will expand horizontally as items are added.

The `LinearItemsLayout` class inherits from the `ItemsLayout` class, and defines an `ItemSpacing` property, of type `double`, that represents the empty space around each item. The default value of this property is 0, and its value must always be greater than or equal to 0. The `LinearItemsLayout` class also defines static `Vertical` and `Horizontal` members. These members can be used to create vertical or horizontal lists, respectively. Alternatively, a `LinearItemsLayout` object can be created, specifying an `ItemsLayoutOrientation` enumeration member as an argument.

The `GridItemsLayout` class inherits from the `ItemsLayout` class, and defines the following properties:

- `VerticalItemSpacing`, of type `double`, that represents the vertical empty space around each item. The default value of this property is 0, and its value must always be greater than or equal to 0.
- `HorizontalItemSpacing`, of type `double`, that represents the horizontal empty space around each item. The default value of this property is 0, and its value must always be greater than or equal to 0.
- `Span`, of type `int`, that represents the number of columns or rows to display in the grid. The default value of this property is 1, and its value must always be greater than or equal to 1.

These properties are backed by `BindableProperty` objects, which means that the properties can be targets of data bindings.

ⓘ Note

`CollectionView` uses the native layout engines to perform layout.

Vertical list

By default, `CollectionView` will display its items in a vertical list layout. Therefore, it's not necessary to set the `ItemsLayout` property to use this layout:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}">
  <CollectionView.ItemTemplate>
    <DataTemplate>
      <Grid Padding="10">
```

```

        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="Auto" />
            <ColumnDefinition Width="Auto" />
        </Grid.ColumnDefinitions>
        <Image Grid.RowSpan="2"
            Source="{Binding ImageUrl}"
            Aspect="AspectFill"
            HeightRequest="60"
            WidthRequest="60" />
        <Label Grid.Column="1"
            Text="{Binding Name}"
            FontAttributes="Bold" />
        <Label Grid.Row="1"
            Grid.Column="1"
            Text="{Binding Location}"
            FontAttributes="Italic"
            VerticalOptions="End" />
    </Grid>
</DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>

```

However, for completeness, in XAML a [CollectionView](#) can be set to display its items in a vertical list by setting its `ItemsLayout` property to `VerticalList`:

XAML

```

<CollectionView ItemsSource="{Binding Monkeys}"
    ItemsLayout="VerticalList">
    ...
</CollectionView>

```

Alternatively, this can also be accomplished by setting the `ItemsLayout` property to a `LinearItemsLayout` object, specifying the `Vertical` `ItemsLayoutOrientation` enumeration member as the `Orientation` property value:

XAML

```

<CollectionView ItemsSource="{Binding Monkeys}">
    <CollectionView.ItemsLayout>
        <LinearItemsLayout Orientation="Vertical" />
    </CollectionView.ItemsLayout>
    ...
</CollectionView>

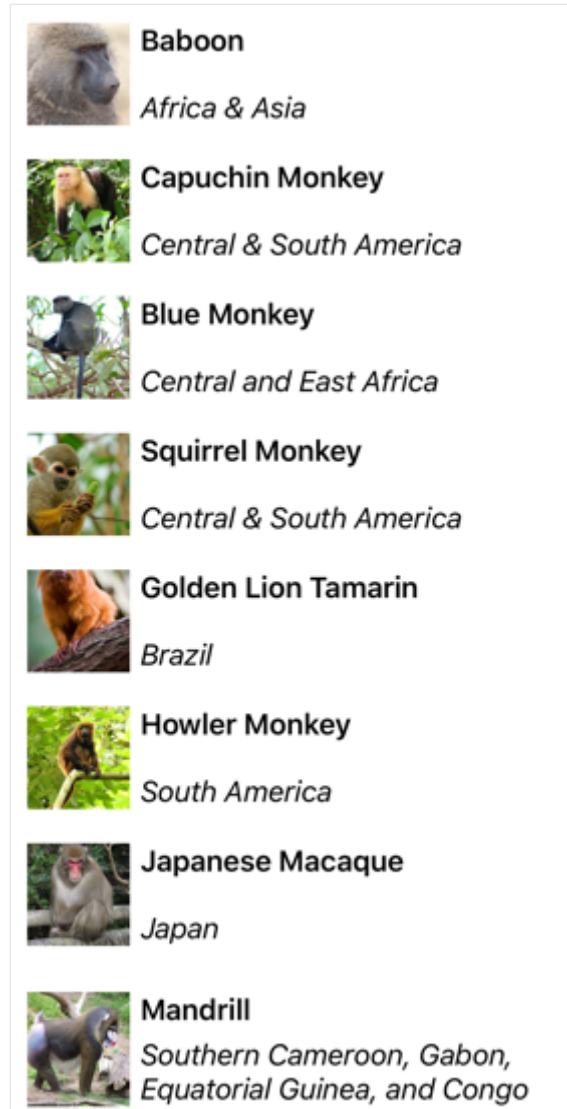
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView
{
    ...
    ItemsLayout = LinearItemsLayout.Vertical
};
```

This results in a single column list, which grows vertically as new items are added:



💡 Tip

Placing a [CollectionView](#) inside a [VerticalStackLayout](#) can stop the [CollectionView](#) scrolling, and can limit the number of items that are displayed. In this situation, replace the [VerticalStackLayout](#) with a [Grid](#).

Horizontal list

In XAML, a `CollectionView` can display its items in a horizontal list by setting its `ItemsLayout` property to `HorizontalList`:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}"
    ItemsLayout="HorizontalList">
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <Grid Padding="10">
                <Grid.RowDefinitions>
                    <RowDefinition Height="35" />
                    <RowDefinition Height="35" />
                </Grid.RowDefinitions>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="70" />
                    <ColumnDefinition Width="140" />
                </Grid.ColumnDefinitions>
                <Image Grid.RowSpan="2"
                    Source="{Binding ImageUrl}"
                    Aspect="AspectFill"
                    HeightRequest="60"
                    WidthRequest="60" />
                <Label Grid.Column="1"
                    Text="{Binding Name}"
                    FontAttributes="Bold"
                    LineBreakMode="TailTruncation" />
                <Label Grid.Row="1"
                    Grid.Column="1"
                    Text="{Binding Location}"
                    LineBreakMode="TailTruncation"
                    FontAttributes="Italic"
                    VerticalOptions="End" />
            </Grid>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
```

Alternatively, this layout can also be accomplished by setting the `ItemsLayout` property to a `LinearItemsLayout` object, specifying the `Horizontal` `ItemsLayoutOrientation` enumeration member as the `Orientation` property value:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}">
    <CollectionView.ItemsLayout>
        <LinearItemsLayout Orientation="Horizontal" />
    </CollectionView.ItemsLayout>
```

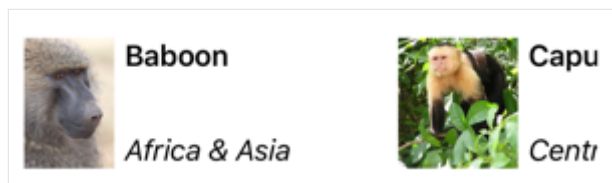
```
...
</CollectionView>
```

The equivalent C# code is:

```
C#

CollectionView collectionView = new CollectionView
{
    ...
    ItemsLayout = LinearItemsLayout.Horizontal
};
```

This results in a single row list, which grows horizontally as new items are added:



Vertical grid

In XAML, a `CollectionView` can display its items in a vertical grid by setting its `ItemsLayout` property to `VerticalGrid`:

```
XAML

<CollectionView ItemsSource="{Binding Monkeys}"
    ItemsLayout="VerticalGrid, 2">
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <Grid Padding="10">
                <Grid.RowDefinitions>
                    <RowDefinition Height="35" />
                    <RowDefinition Height="35" />
                </Grid.RowDefinitions>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="70" />
                    <ColumnDefinition Width="80" />
                </Grid.ColumnDefinitions>
                <Image Grid.RowSpan="2"
                    Source="{Binding ImageUrl}"
                    Aspect="AspectFill"
                    HeightRequest="60"
                    WidthRequest="60" />
                <Label Grid.Column="1"
                    Text="{Binding Name}"
                    FontAttributes="Bold">
```



```

        LineBreakMode="TailTruncation" />
        <Label Grid.Row="1"
            Grid.Column="1"
            Text="{Binding Location}"
            LineBreakMode="TailTruncation"
            FontAttributes="Italic"
            VerticalOptions="End" />
    </Grid>
</DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>

```

Alternatively, this layout can also be accomplished by setting the `ItemsLayout` property to a `GridItemsLayout` object whose `Orientation` property is set to `Vertical`:

XAML

```

<CollectionView ItemsSource="{Binding Monkeys}">
    <CollectionView.ItemsLayout>
        <GridItemsLayout Orientation="Vertical"
            Span="2" />
    </CollectionView.ItemsLayout>
    ...
</CollectionView>

```

The equivalent C# code is:

C#

```

CollectionView collectionView = new CollectionView
{
    ...
    ItemsLayout = new GridItemsLayout(2, ItemsLayoutOrientation.Vertical)
};

```

By default, a vertical `GridItemsLayout` will display items in a single column. However, this example sets the `GridItemsLayout.Span` property to 2. This results in a two-column grid, which grows vertically as new items are added:

	Baboon <i>Africa &...</i>		Capuch... <i>Central...</i>
	Blue Mo... <i>Central...</i>		Squirrel... <i>Central...</i>
	Golden... <i>Brazil</i>		Howler... <i>South A...</i>
	Japane... <i>Japan</i>		Mandrill <i>Souther...</i>
	Probos... <i>Borneo</i>		Red-sh... <i>Vietnam...</i>
	Gray-sh... <i>Vietnam</i>		Golden... <i>China</i>
	Black S... <i>China</i>		Tonkin... <i>Vietnam</i>

Horizontal grid

In XAML, a [CollectionView](#) can display its items in a horizontal grid by setting its `ItemsLayout` property to `HorizontalGrid`:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}"
    ItemsLayout="HorizontalGrid, 4">
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <Grid Padding="10">
                <Grid.RowDefinitions>
                    <RowDefinition Height="35" />
                    <RowDefinition Height="35" />
                </Grid.RowDefinitions>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="70" />
                    <ColumnDefinition Width="140" />
                </Grid.ColumnDefinitions>
                <Image Grid.RowSpan="2"
```

```

        Source="{Binding ImageUrl}"
        Aspect="AspectFill"
        HeightRequest="60"
        WidthRequest="60" />
<Label Grid.Column="1"
        Text="{Binding Name}"
        FontAttributes="Bold"
        LineBreakMode="TailTruncation" />
<Label Grid.Row="1"
        Grid.Column="1"
        Text="{Binding Location}"
        LineBreakMode="TailTruncation"
        FontAttributes="Italic"
        VerticalOptions="End" />
    </Grid>
</DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>

```

Alternatively, this layout can also be accomplished by setting the `ItemsLayout` property to a `GridItemsLayout` object whose `Orientation` property is set to `Horizontal`:

XAML

```

<CollectionView ItemsSource="{Binding Monkeys}">
    <CollectionView.ItemsLayout>
        <GridItemsLayout Orientation="Horizontal"
            Span="4" />
    </CollectionView.ItemsLayout>
    ...
</CollectionView>

```

The equivalent C# code is:

C#

```

CollectionView collectionView = new CollectionView
{
    ...
    ItemsLayout = new GridItemsLayout(4, ItemsLayoutOrientation.Horizontal)
};

```

By default, a horizontal `GridItemsLayout` will display items in a single row. However, this example sets the `GridItemsLayout.Span` property to 4. This results in a four-row grid, which grows horizontally as new items are added:

	Baboon <i>Africa & Asia</i>		Go <i>Brē</i>
	Capuchin Monk... <i>Central & South...</i>		Ho <i>Soi</i>
	Blue Monkey <i>Central and East...</i>		Jaḡ <i>Jaḡ</i>
	Squirrel Monkey <i>Central & South...</i>		Ma <i>Soi</i>

Headers and footers

[CollectionView](#) can present a header and footer that scroll with the items in the list. The header and footer can be strings, views, or [DataTemplate](#) objects.

[CollectionView](#) defines the following properties for specifying the header and footer:

- `Header`, of type `object`, specifies the string, binding, or view that will be displayed at the start of the list.
- `HeaderTemplate`, of type [DataTemplate](#), specifies the [DataTemplate](#) to use to format the `Header`.
- `Footer`, of type `object`, specifies the string, binding, or view that will be displayed at the end of the list.
- `FooterTemplate`, of type [DataTemplate](#), specifies the [DataTemplate](#) to use to format the `Footer`.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings.

When a header is added to a layout that grows horizontally, from left to right, the header is displayed to the left of the list. Similarly, when a footer is added to a layout that grows horizontally, from left to right, the footer is displayed to the right of the list.

Display strings in the header and footer

The `Header` and `Footer` properties can be set to `string` values, as shown in the following example:

XAML

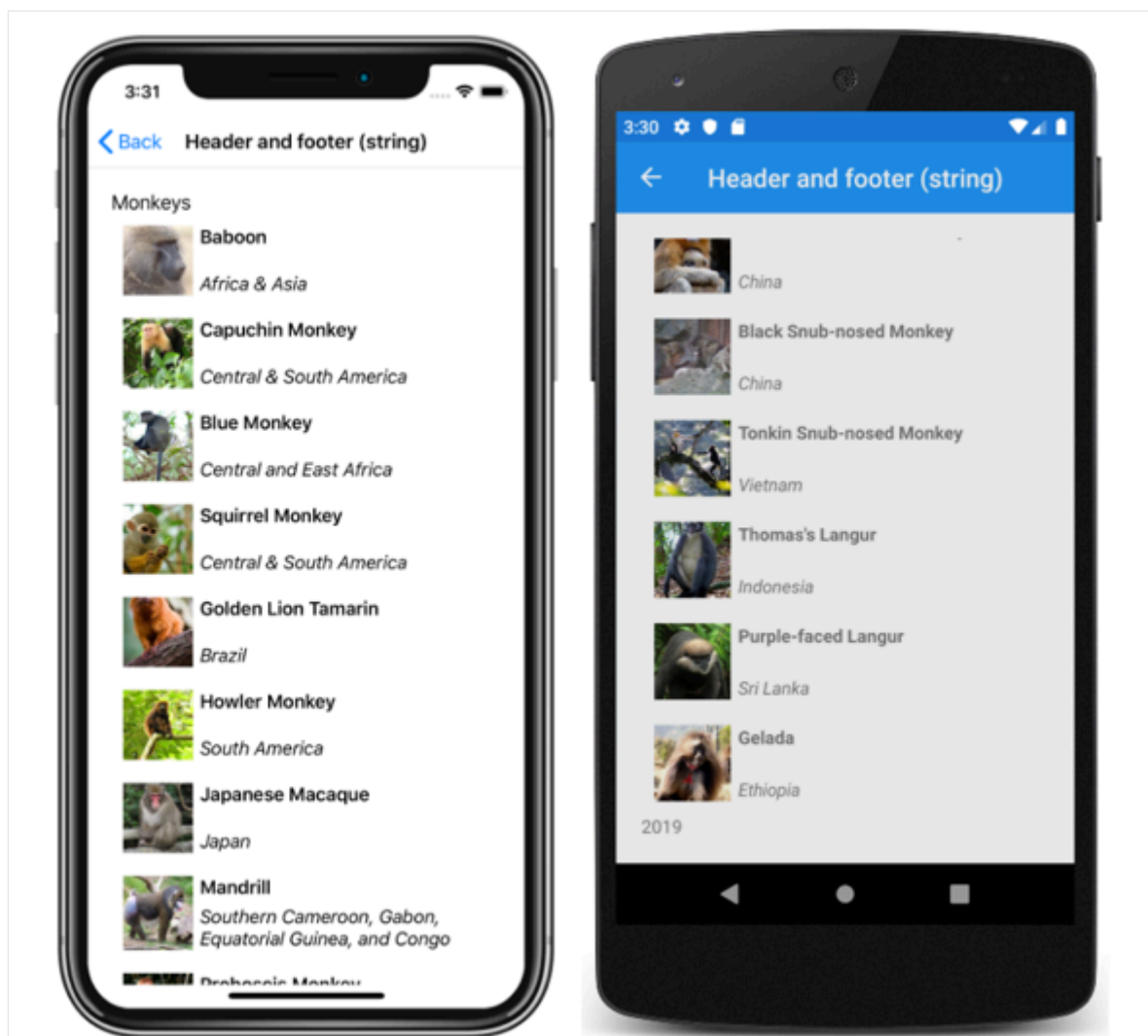
```
<CollectionView ItemsSource="{Binding Monkeys}"
                Header="Monkeys"
                Footer="2019">
    ...
</CollectionView>
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView
{
    Header = "Monkeys",
    Footer = "2019"
};
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
```

This code results in the following screenshots, with the header shown in the iOS screenshot, and the footer shown in the Android screenshot:



Display views in the header and footer

The `Header` and `Footer` properties can each be set to a view. This can be a single view, or a view that contains multiple child views. The following example shows the `Header` and `Footer` properties each set to a `StackLayout` object that contains a `Label` object:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}">
  <CollectionView.Header>
    <StackLayout BackgroundColor="LightGray">
      <Label Margin="10,0,0,0"
        Text="Monkeys"
        FontSize="12"
        FontAttributes="Bold" />
    </StackLayout>
  </CollectionView.Header>
  <CollectionView.Footer>
    <StackLayout BackgroundColor="LightGray">
      <Label Margin="10,0,0,0"
        Text="Friends of Xamarin Monkey"
        FontSize="12"
        FontAttributes="Bold" />
    </StackLayout>
  </CollectionView.Footer>
</CollectionView>
```

```
        </StackLayout>
    </CollectionView.Footer>
    ...
</CollectionView>
```

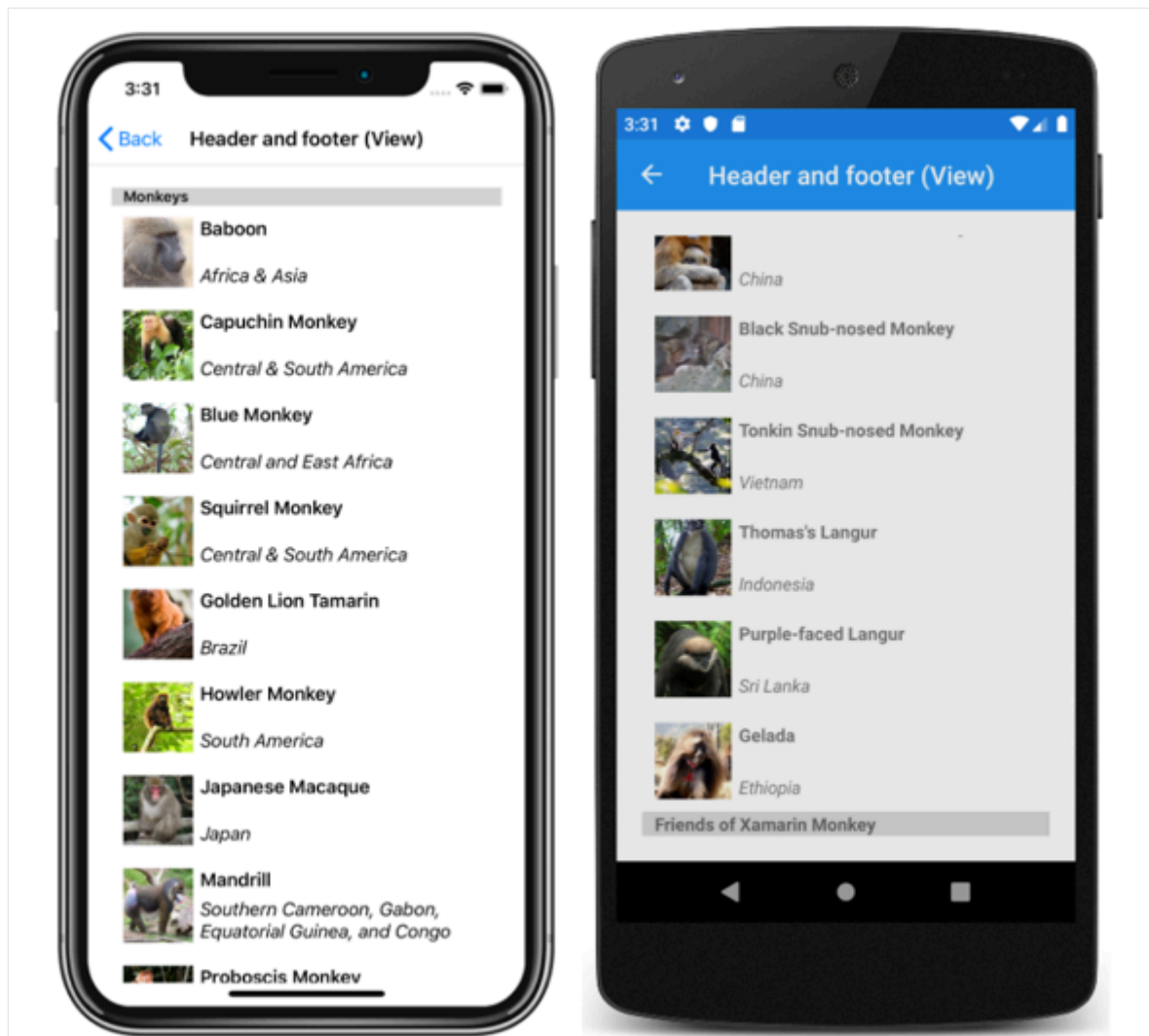
The equivalent C# code is:

C#

```
StackLayout headerStackLayout = new StackLayout();
header.StackLayout.Add(new Label { Text = "Monkeys", ... } );
StackLayout footerStackLayout = new StackLayout();
footerStackLayout.Add(new Label { Text = "Friends of Xamarin Monkey", ... }
);

CollectionView collectionView = new CollectionView
{
    Header = headerStackLayout,
    Footer = footerStackLayout
};
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
```

This code results in the following screenshots, with the header shown in the iOS screenshot, and the footer shown in the Android screenshot:



Display a templated header and footer

The `HeaderTemplate` and `FooterTemplate` properties can be set to `DataTemplate` objects that are used to format the header and footer. In this scenario, the `Header` and `Footer` properties must bind to the current source for the templates to be applied, as shown in the following example:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}"
    Header="{Binding .}"
    Footer="{Binding .}">
    <CollectionView.HeaderTemplate>
        <DataTemplate>
            <StackLayout BackgroundColor="LightGray">
                <Label Margin="10,0,0,0"
                    Text="Monkeys"
                    FontSize="12"
                    FontAttributes="Bold" />
            </StackLayout>
        </DataTemplate>
    </CollectionView.HeaderTemplate>
```



```

<CollectionView.FooterTemplate>
    <DataTemplate>
        <StackLayout BackgroundColor="LightGray">
            <Label Margin="10,0,0,0"
                Text="Friends of Xamarin Monkey"
                FontSize="12"
                FontAttributes="Bold" />
        </StackLayout>
    </DataTemplate>
</CollectionView.FooterTemplate>
...
</CollectionView>

```

The equivalent C# code is:

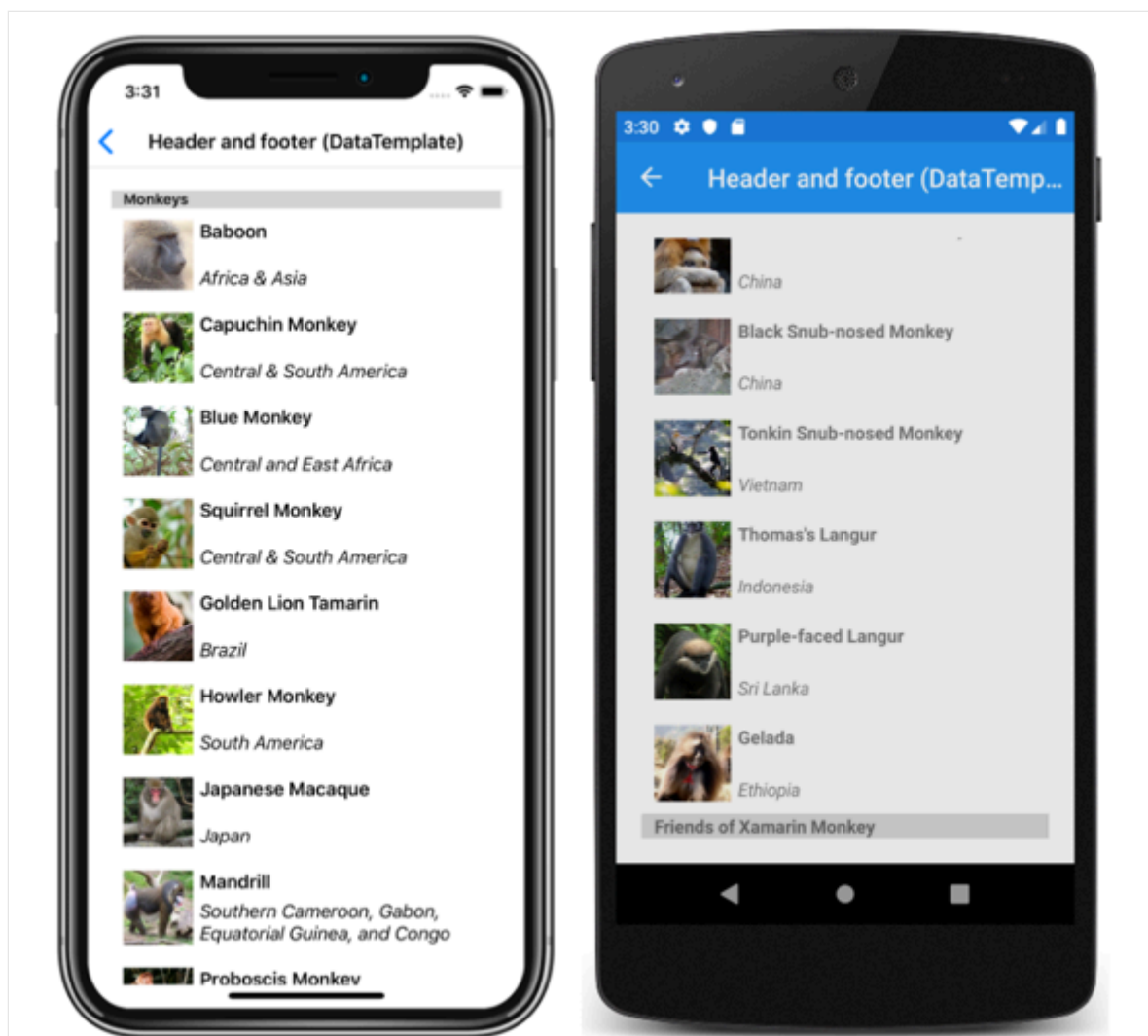
C#

```

CollectionView collectionView = new CollectionView
{
    HeaderTemplate = new DataTemplate(() =>
    {
        return new StackLayout { };
    }),
    FooterTemplate = new DataTemplate(() =>
    {
        return new StackLayout { };
    })
};
collectionView.SetBinding(ItemsView.HeaderProperty, ".");
collectionView.SetBinding(ItemsView.FooterProperty, ".");
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");

```

This code results in the following screenshots, with the header shown in the iOS screenshot, and the footer shown in the Android screenshot:



Item spacing

By default, there is no space between each item in a [CollectionView](#). This behavior can be changed by setting properties on the items layout used by the [CollectionView](#).

When a [CollectionView](#) sets its `ItemsLayout` property to a `LinearItemsLayout` object, the `LinearItemsLayout.ItemSpacing` property can be set to a `double` value that represents the space between items:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}">
    <CollectionView.ItemsLayout>
        <LinearItemsLayout Orientation="Vertical"
            ItemSpacing="20" />
    </CollectionView.ItemsLayout>
    ...
</CollectionView>
```

ⓘ Note

The `LinearLayout.ItemSpacing` property has a validation callback set, which ensures that the value of the property is always greater than or equal to 0.

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView
{
    ...
    ItemLayout = new LinearLayout(LinearLayoutOrientation.Vertical)
    {
        ItemSpacing = 20
    }
};
```

This code results in a vertical single column list that has a spacing of 20 between items:



Baboon

Africa & Asia



Capuchin Monkey

Central & South America



Blue Monkey

Central and East Africa



Squirrel Monkey

Central & South America



Golden Lion Tamarin

Brazil



Howler Monkey

South America



Japanese Macaque

Japan

When a `CollectionView` sets its `ItemsLayout` property to a `GridItemsLayout` object, the `GridItemsLayout.VerticalItemSpacing` and `GridItemsLayout.HorizontalItemSpacing` properties can be set to `double` values that represent the empty space vertically and horizontally between items:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}">
    <CollectionView.ItemsLayout>
        <GridItemsLayout Orientation="Vertical"
            Span="2"
            VerticalItemSpacing="20"
            HorizontalItemSpacing="30" />
    </CollectionView.ItemsLayout>
    ...
</CollectionView>
```

ⓘ Note

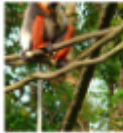
The `GridItemsLayout.VerticalItemSpacing` and `GridItemsLayout.HorizontalItemSpacing` properties have validation callbacks set, which ensures that the values of the properties are always greater than or equal to 0.

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView
{
    ...
    ItemsLayout = new GridItemsLayout(2, ItemsLayoutOrientation.Vertical)
    {
        VerticalItemSpacing = 20,
        HorizontalItemSpacing = 30
    }
};
```

This code results in a vertical two-column grid that has a vertical spacing of 20 between items and a horizontal spacing of 30 between items:

	Baboon <i>Africa &...</i>		Capuchi. <i>Central...</i>
	Blue Mo... <i>Central a...</i>		Squirrel... <i>Central...</i>
	Golden... <i>Brazil</i>		Howler... <i>South A...</i>
	Japanes... <i>Japan</i>		Mandrill <i>Southern</i>
	Probosc... <i>Borneo</i>		Red-sha... <i>Vietnam,.</i>
	Gray-sh... <i>Vietnam</i>		Golden... <i>China</i>

Item sizing

By default, each item in a [CollectionView](#) is individually measured and sized, provided that the UI elements in the [DataTemplate](#) don't specify fixed sizes. This behavior, which can be changed, is specified by the `CollectionView.ItemSizingStrategy` property value. This property value can be set to one of the `ItemSizingStrategy` enumeration members:

- `MeasureAllItems` – each item is individually measured. This is the default value.
- `MeasureFirstItem` – only the first item is measured, with all subsequent items being given the same size as the first item.

Important

The `MeasureFirstItem` sizing strategy will result in increased performance when used in situations where the item size is intended to be uniform across all items.

The following code example shows setting the `ItemSizingStrategy` property:

XAML

```
<CollectionView ...  
    ItemSizingStrategy="MeasureFirstItem">  
    ...  
</CollectionView>
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView  
{  
    ...  
    ItemSizingStrategy = ItemSizingStrategy.MeasureFirstItem  
};
```

Dynamic resizing of items

Items in a `CollectionView` can be dynamically resized at runtime by changing layout related properties of elements within the `DataTemplate`. For example, the following code example changes the `HeightRequest` and `WidthRequest` properties of an `Image` object:

C#

```
void OnImageTapped(object sender, EventArgs e)  
{  
    Image image = sender as Image;  
    image.HeightRequest = image.WidthRequest =  
    image.HeightRequest.Equals(60) ? 100 : 60;  
}
```

The `OnImageTapped` event handler is executed in response to an `Image` object being tapped, and changes the dimensions of the image so that it's more easily viewed:



Baboon

Africa & Asia



Capuchin Monkey

Central & South America



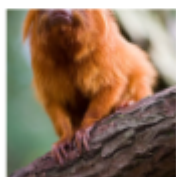
Blue Monkey

Central and East Africa



Squirrel Monkey

Central & South America



Golden Lion Tamarin

Brazil



Howler Monkey

South America



Japanese Macaque

Japan

Right-to-left layout

[CollectionView](#) can layout its content in a right-to-left flow direction by setting its `FlowDirection` property to `RightToLeft`. However, the `FlowDirection` property should ideally be set on a page or root layout, which causes all the elements within the page, or root layout, to respond to the flow direction:

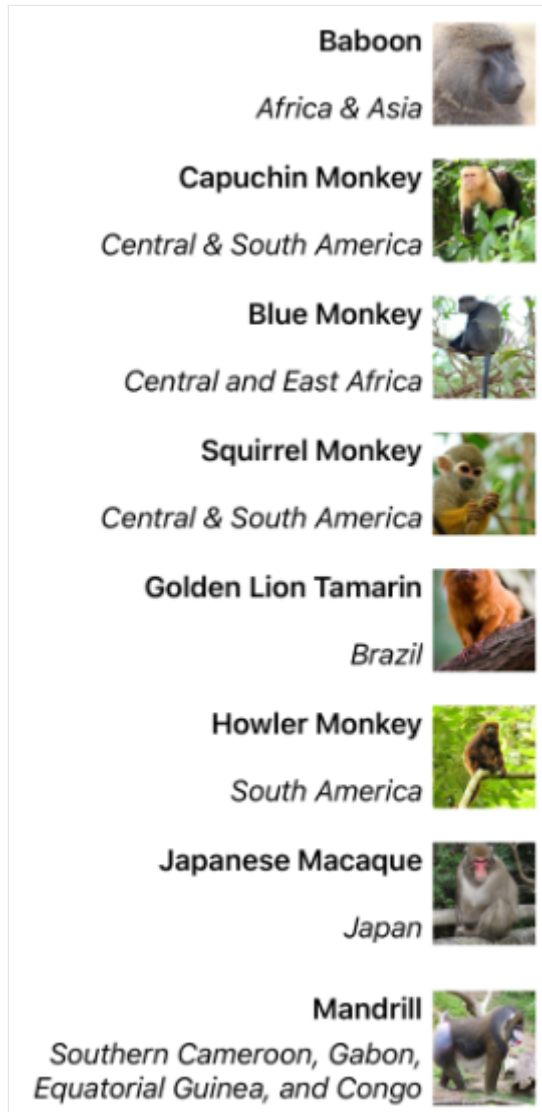
XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"

              x:Class="CollectionViewDemos.Views.VerticallyListFlowDirectionPage"
              Title="Vertical list (RTL FlowDirection)"
              FlowDirection="RightToLeft">
    <Grid Margin="20">
        <CollectionView ItemsSource="{Binding Monkeys}">
            ...
        </CollectionView>
    </Grid>
</ContentPage>
```

```
</Grid>  
</ContentPage>
```

The default `FlowDirection` for an element with a parent is `MatchParent`. Therefore, the `CollectionView` inherits the `FlowDirection` property value from the `Grid`, which in turn inherits the `FlowDirection` property value from the `ContentPage`. This results in the right-to-left layout shown in the following screenshot:



For more information about flow direction, see [Right to left localization](#).

Configure CollectionView item selection

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [CollectionView](#) defines the following properties that control item selection:

- `SelectionMode`, of type `SelectionMode`, the selection mode.
- `SelectedItem`, of type `object`, the selected item in the list. This property has a default binding mode of `TwoWay`, and has a `null` value when no item is selected.
- `SelectedItems`, of type `IList<object>`, the selected items in the list. This property has a default binding mode of `OneWay`, and has a `null` value when no items are selected.
- `SelectionChangedCommand`, of type [ICommand](#), which is executed when the selected item changes.
- `SelectionChangedCommandParameter`, of type `object`, which is the parameter that's passed to the `SelectionChangedCommand`.

All of these properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings.

By default, [CollectionView](#) selection is disabled. However, this behavior can be changed by setting the `SelectionMode` property value to one of the `SelectionMode` enumeration members:

- `None` – indicates that items can't be selected. This is the default value.
- `Single` – indicates that a single item can be selected, with the selected item being highlighted.
- `Multiple` – indicates that multiple items can be selected, with the selected items being highlighted.

[CollectionView](#) defines a `SelectionChanged` event that is fired when the `SelectedItem` property changes, either due to the user selecting an item from the list, or when an application sets the property. In addition, this event is also fired when the `SelectedItems` property changes. The `SelectionChangedEventArgs` object that accompanies the `SelectionChanged` event has two properties, both of type `IReadOnlyList<object>`:

- `PreviousSelection` – the list of items that were selected, before the selection changed.
- `CurrentSelection` – the list of items that are selected, after the selection change.

In addition, `CollectionView` has a `UpdateSelectedItems` method that updates the `SelectedItems` property with a list of selected items, while only firing a single change notification.

Single selection

When the `SelectionMode` property is set to `Single`, a single item in the `CollectionView` can be selected. When an item is selected, the `SelectedItem` property is set to the value of the selected item. When this property changes, the `SelectionChangedCommand` is executed (with the value of the `SelectionChangedCommandParameter` being passed to the `ICommand`), and the `SelectionChanged` event fires.

The following XAML example shows a `CollectionView` that can respond to single item selection:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}"
                SelectionMode="Single"
                SelectionChanged="OnCollectionViewSelectionChanged">
    ...
</CollectionView>
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView
{
    SelectionMode = SelectionMode.Single
};
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
collectionView.SelectionChanged += OnCollectionViewSelectionChanged;
```

In this code example, the `OnCollectionViewSelectionChanged` event handler is executed when the `SelectionChanged` event fires, with the event handler retrieving the previously selected item, and the current selected item:

C#

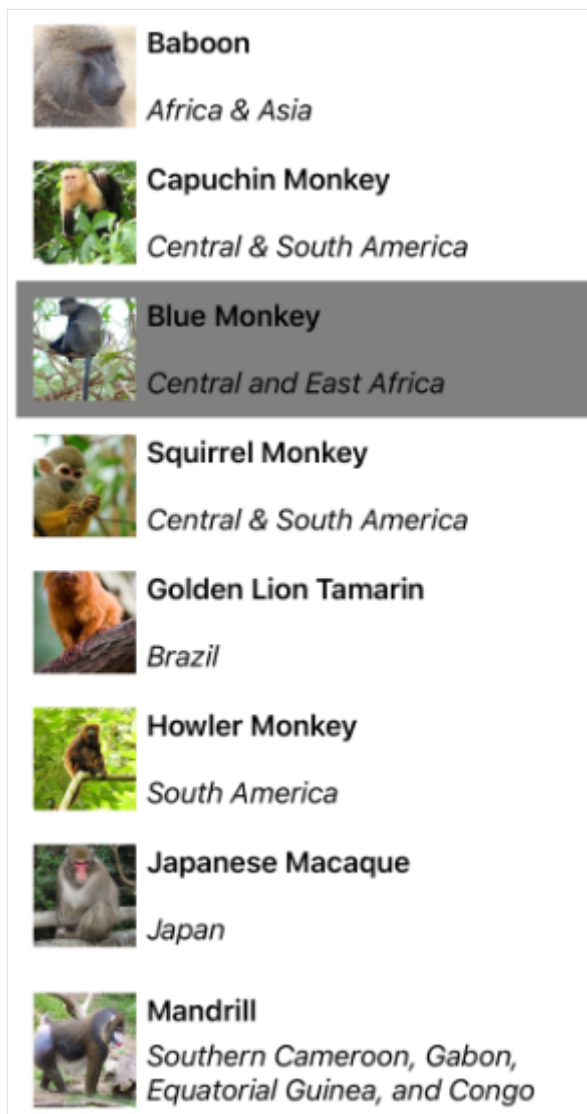
```
void OnCollectionViewSelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    string previous = (e.PreviousSelection.FirstOrDefault() as
Monkey)?.Name;
```

```
string current = (e.CurrentSelection.FirstOrDefault() as Monkey)?.Name;  
...  
}
```

❗ Important

The `SelectionChanged` event can be fired by changes that occur as a result of changing the `SelectionMode` property.

The following screenshot shows single item selection in a [CollectionView](#):



Multiple selection

When the `SelectionMode` property is set to `Multiple`, multiple items in the [CollectionView](#) can be selected. When items are selected, the `SelectedItems` property is set to the selected items. When this property changes, the `SelectionChangedCommand` is

executed (with the value of the `SelectionChangedCommandParameter` being passed to the `ICommand`, and the `SelectionChanged` event fires.

The following XAML example shows a `CollectionView` that can respond to multiple item selection:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}"
                SelectionMode="Multiple"
                SelectionChanged="OnCollectionViewSelectionChanged">
    ...
</CollectionView>
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView
{
    SelectionMode = SelectionMode.Multiple
};
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
collectionView.SelectionChanged += OnCollectionViewSelectionChanged;
```

In this code example, the `OnCollectionViewSelectionChanged` event handler is executed when the `SelectionChanged` event fires, with the event handler retrieving the previously selected items, and the current selected items:

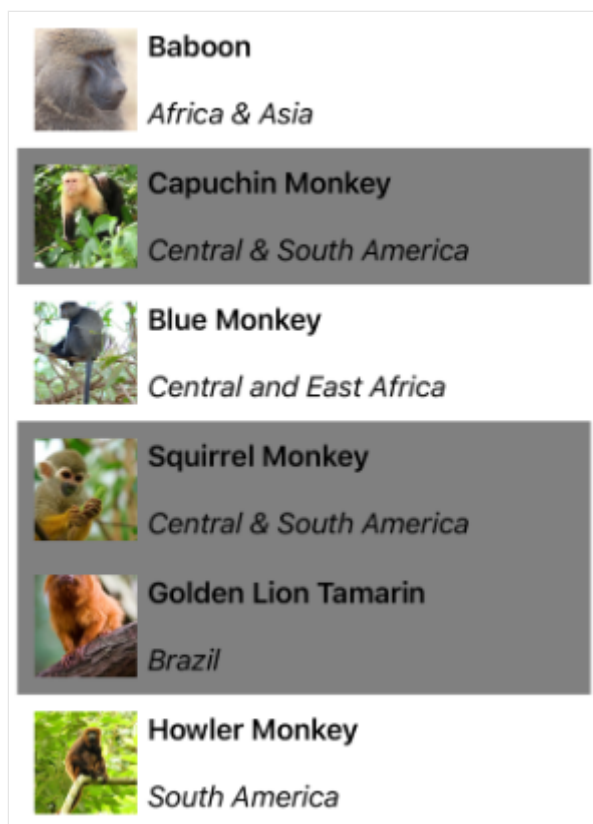
C#

```
void OnCollectionViewSelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    var previous = e.PreviousSelection;
    var current = e.CurrentSelection;
    ...
}
```

Important

The `SelectionChanged` event can be fired by changes that occur as a result of changing the `SelectionMode` property.

The following screenshot shows multiple item selection in a `CollectionView`:



Single preselection

When the `SelectionMode` property is set to `Single`, a single item in the [CollectionView](#) can be preselected by setting the `SelectedItem` property to the item. The following XAML example shows a [CollectionView](#) that preselects a single item:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}"
                SelectionMode="Single"
                SelectedItem="{Binding SelectedMonkey}">
    ...
</CollectionView>
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView
{
    SelectionMode = SelectionMode.Single
};
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
collectionView.SetBinding(SelectableItemsView.SelectedItemProperty,
    "SelectedMonkey");
```

❗ Note

The `SelectedItem` property has a default binding mode of `TwoWay`.

The `SelectedItem` property data binds to the `SelectedMonkey` property of the connected view model, which is of type `Monkey`. By default, a `TwoWay` binding is used so that if the user changes the selected item, the value of the `SelectedMonkey` property is set to the selected `Monkey` object. The `SelectedMonkey` property is defined in the `MonkeysViewModel` class, and is set to the fourth item of the `Monkeys` collection:

C#

```
public class MonkeysViewModel : INotifyPropertyChanged
{
    ...
    public ObservableCollection<Monkey> Monkeys { get; private set; }

    Monkey selectedMonkey;
    public Monkey SelectedMonkey
    {
        get
        {
            return selectedMonkey;
        }
        set
        {
            if (selectedMonkey != value)
            {
                selectedMonkey = value;
            }
        }
    }

    public MonkeysViewModel()
    {
        ...
        selectedMonkey = Monkeys.Skip(3).FirstOrDefault();
    }
    ...
}
```

Therefore, when the `CollectionView` appears, the fourth item in the list is preselected:

	Baboon <i>Africa & Asia</i>
	Capuchin Monkey <i>Central & South America</i>
	Blue Monkey <i>Central and East Africa</i>
	Squirrel Monkey <i>Central & South America</i>
	Golden Lion Tamarin <i>Brazil</i>
	Howler Monkey <i>South America</i>
	Japanese Macaque <i>Japan</i>
	Mandrill <i>Southern Cameroon, Gabon, Equatorial Guinea, and Congo</i>

Multiple preselection

When the `SelectionMode` property is set to `Multiple`, multiple items in the [CollectionView](#) can be preselected. The following XAML example shows a [CollectionView](#) that enables the preselection of multiple items:

XAML

```
<CollectionView x:Name="collectionView"
    ItemsSource="{Binding Monkeys}"
    SelectionMode="Multiple"
    SelectedItems="{Binding SelectedMonkeys}">
    ...
</CollectionView>
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView
{
    SelectionMode = SelectionMode.Multiple
};
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
collectionView.SetBinding(SelectableItemsView.SelectedItemsProperty,
    "SelectedMonkeys");
```

ⓘ Note

The `SelectedItems` property has a default binding mode of `OneWay`.

The `SelectedItems` property data binds to the `SelectedMonkeys` property of the connected view model, which is of type `ObservableCollection<object>`. The `SelectedMonkeys` property is defined in the `MonkeysViewModel` class, and is set to the second, fourth, and fifth items in the `Monkeys` collection:

C#

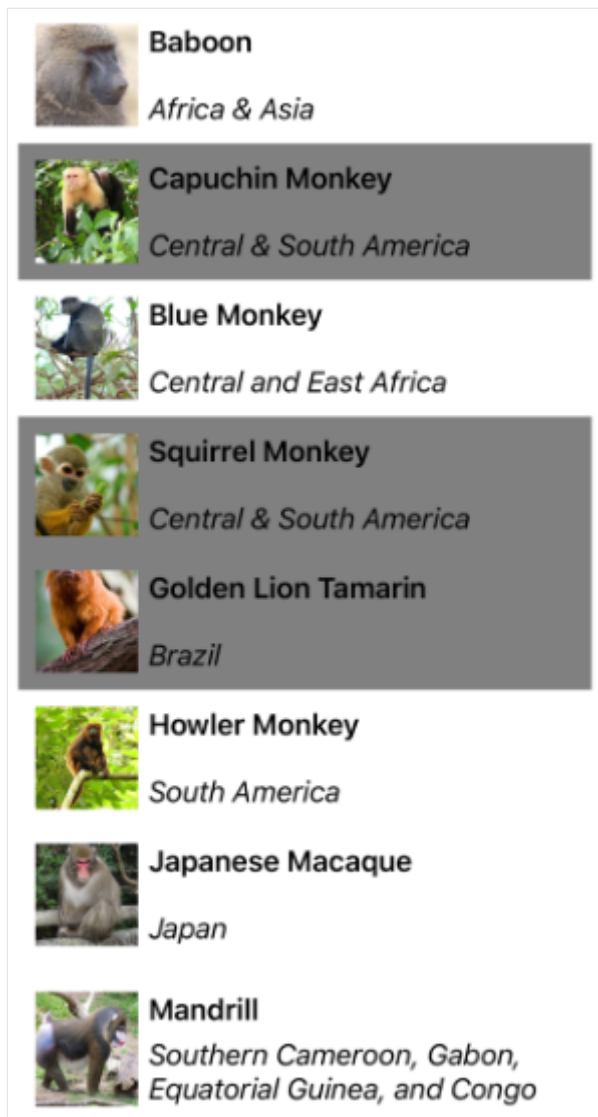
```
namespace CollectionViewDemos.ViewModels
{
    public class MonkeysViewModel : INotifyPropertyChanged
    {
        ...
        ObservableCollection<object> selectedMonkeys;
        public ObservableCollection<object> SelectedMonkeys
        {
            get
            {
                return selectedMonkeys;
            }
            set
            {
                if (selectedMonkeys != value)
                {
                    selectedMonkeys = value;
                }
            }
        }

        public MonkeysViewModel()
        {
            ...
            SelectedMonkeys = new ObservableCollection<object>()
            {
                Monkeys[1], Monkeys[3], Monkeys[4]
            };
        }
        ...
    }
}
```



```
}  
}
```

Therefore, when the [CollectionView](#) appears, the second, fourth, and fifth items in the list are preselected:



Clear selections

The `SelectedItem` and `SelectedItems` properties can be cleared by setting them, or the objects they bind to, to `null`. When either of these properties are cleared, the `SelectionChanged` event is raised with an empty `CurrentSelection` property, and the `SelectionChangedCommand` is executed.

Handle reselection

A common scenario is that users will select an item in the `CollectionView` and then navigate to another page. When they navigate back the item is still selected, which will

result in them being unable to reselect the given item. To enable reselection you should clear the item selection on the `CollectionView`:

XAML

```
<CollectionView ...
    SelectionChanged="OnCollectionViewSelectionChanged" />
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView();
collectionView.SelectionChanged += OnCollectionViewSelectionChanged;
```

The following example shows the event handler code for the `SelectionChanged` event:

C#

```
void OnCollectionViewSelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    var cv = (CollectionView)sender;
    if (cv.SelectedItem == null)
        return;

    cv.SelectedItem = null;
}
```

Change selected item color

`CollectionView` has a `Selected VisualState` that can be used to initiate a visual change to the selected item in the `CollectionView`. A common use case for this `VisualState` is to change the background color of the selected item, which is shown in the following XAML example:

XAML

```
<ContentPage ...>
  <ContentPage.Resources>
    <Style TargetType="Grid">
      <Setter Property="VisualStateManager.VisualStateGroups">
        <VisualStateGroupList>
          <VisualStateGroup x:Name="CommonStates">
            <VisualState x:Name="Normal" />
            <VisualState x:Name="Selected">
```

```

        <VisualState.Setters>
            <Setter Property="BackgroundColor"
                Value="LightSkyBlue" />
        </VisualState.Setters>
    </VisualState>
</VisualStateGroup>
</VisualStateGroupList>
</Setter>
</Style>
</ContentPage.Resources>
<Grid Margin="20">
    <CollectionView ItemsSource="{Binding Monkeys}"
        SelectionMode="Single">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Grid Padding="10">
                    ...
                </Grid>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</Grid>
</ContentPage>

```

❗ Important

The [Style](#) that contains the `Selected` [VisualState](#) must have a `TargetType` property value that's the type of the root element of the [DataTemplate](#), which is set as the `ItemTemplate` property value.

The equivalent C# code for the style containing the visual state is:

```

C#

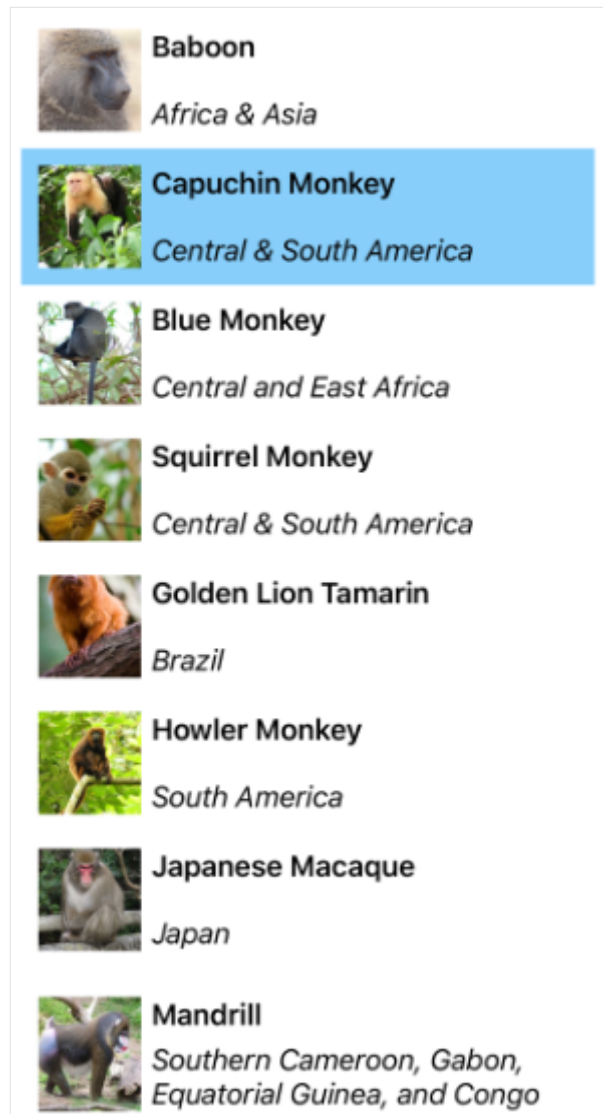
using static Microsoft.Maui.Controls.VisualStateManager;
...

Setter backgroundColorSetter = new() { Property = BackgroundColorProperty,
    Value = Colors.LightSkyBlue };
VisualState stateSelected = new() { Name = CommonStates.Selected, Setters =
    { backgroundColorSetter } };
VisualState stateNormal = new() { Name = CommonStates.Normal };
VisualStateGroup visualStateGroup = new() { Name = nameof(CommonStates),
    States = { stateSelected, stateNormal } };
VisualStateGroupList visualStateGroupList = new() { visualStateGroup };
Setter vsgSetter = new() { Property = VisualStateGroupsProperty, Value =
    visualStateGroupList };
Style style = new(typeof(Grid)) { Setters = { vsgSetter } };

```

```
// Add the style to the resource dictionary
Resources.Add(style);
```

In this example, the `Style.TargetType` property value is set to `Grid` because the root element of the `ItemTemplate` is a `Grid`. The `Selected VisualState` specifies that when an item in the `CollectionView` is selected, the `BackgroundColor` of the item is set to `LightSkyBlue`:



For more information about visual states, see [Visual states](#).

Disable selection

`CollectionView` selection is disabled by default. However, if a `CollectionView` has selection enabled, it can be disabled by setting the `SelectionMode` property to `None`:

XAML

```
<CollectionView ...
```

```
SelectionMode="None" />
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView  
{  
    ...  
    SelectionMode = SelectionMode.None  
};
```

When the `SelectionMode` property is set to `None`, items in the `CollectionView` can't be selected, the `SelectedItem` property remains `null`, and the `SelectionChanged` event isn't fired.

ⓘ Note

When an item has been selected and the `SelectionMode` property is changed from `Single` to `None`, the `SelectedItem` property will be set to `null` and the `SelectionChanged` event will be fired with an empty `CurrentSelection` property.

Define an EmptyView for a CollectionView

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [CollectionView](#) defines the following properties that can be used to provide user feedback when there's no data to display:

- `EmptyView`, of type `object`, the string, binding, or view that will be displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. The default value is `null`.
- `EmptyViewTemplate`, of type [DataTemplate](#), the template to use to format the specified `EmptyView`. The default value is `null`.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings.

The main usage scenarios for setting the `EmptyView` property are displaying user feedback when a filtering operation on a [CollectionView](#) yields no data, and displaying user feedback while data is being retrieved from a web service.

Note

The `EmptyView` property can be set to a view that includes interactive content if required.

For more information about data templates, see [Data templates](#).

Display a string when data is unavailable

The `EmptyView` property can be set to a string, which will be displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. The following XAML shows an example of this scenario:

XAML

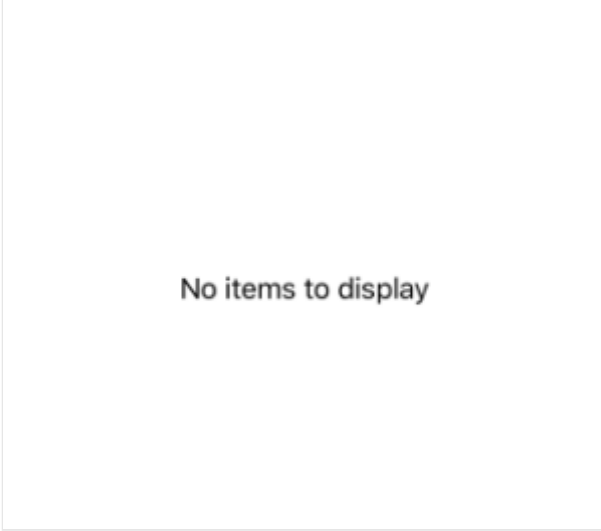
```
<CollectionView ItemsSource="{Binding EmptyMonkeys}"
    EmptyView="No items to display" />
```

The equivalent C# code is:

```
C#

CollectionView collectionView = new CollectionView
{
    EmptyView = "No items to display"
};
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "EmptyMonkeys");
```

The result is that, because the data bound collection is `null`, the string set as the `EmptyView` property value is displayed:



No items to display

Display views when data is unavailable

The `EmptyView` property can be set to a view, which will be displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. This can be a single view, or a view that contains multiple child views. The following XAML example shows the `EmptyView` property set to a view that contains multiple child views:

XAML

```
<Grid Margin="20" RowDefinitions="Auto,*">
    <SearchBar x:Name="searchBar"
        SearchCommand="{Binding FilterCommand}"
        SearchCommandParameter="{Binding Source={x:Reference
searchBar}, Path=Text}"
        Placeholder="Filter" />
    <CollectionView ItemsSource="{Binding Monkeys}"
        Grid.Row="1">
        <CollectionView.ItemTemplate>
            <DataTemplate>
```

```

        ...
    </DataTemplate>
</CollectionView.ItemTemplate>
<CollectionView.EmptyView>
    <ContentView>
        <StackLayout HorizontalOptions="Center"
            VerticalOptions="Center">
            <Label Text="No results matched your filter."
                Margin="10,25,10,10"
                FontAttributes="Bold"
                FontSize="18"
                HorizontalOptions="Fill"
                HorizontalTextAlignment="Center" />
            <Label Text="Try a broader filter?"
                FontAttributes="Italic"
                FontSize="12"
                HorizontalOptions="Fill"
                HorizontalTextAlignment="Center" />
        </StackLayout>
    </ContentView>
</CollectionView.EmptyView>
</CollectionView>
</Grid>

```

In this example, what looks like a redundant has been added as the root element of the `EmptyView`. This is because internally, the `EmptyView` is added to a native container that doesn't provide any context for .NET MAUI layout. Therefore, to position the views that comprise your `EmptyView`, you must add a root layout, whose child is a layout that can position itself within the root layout.

The equivalent C# code is:

C#

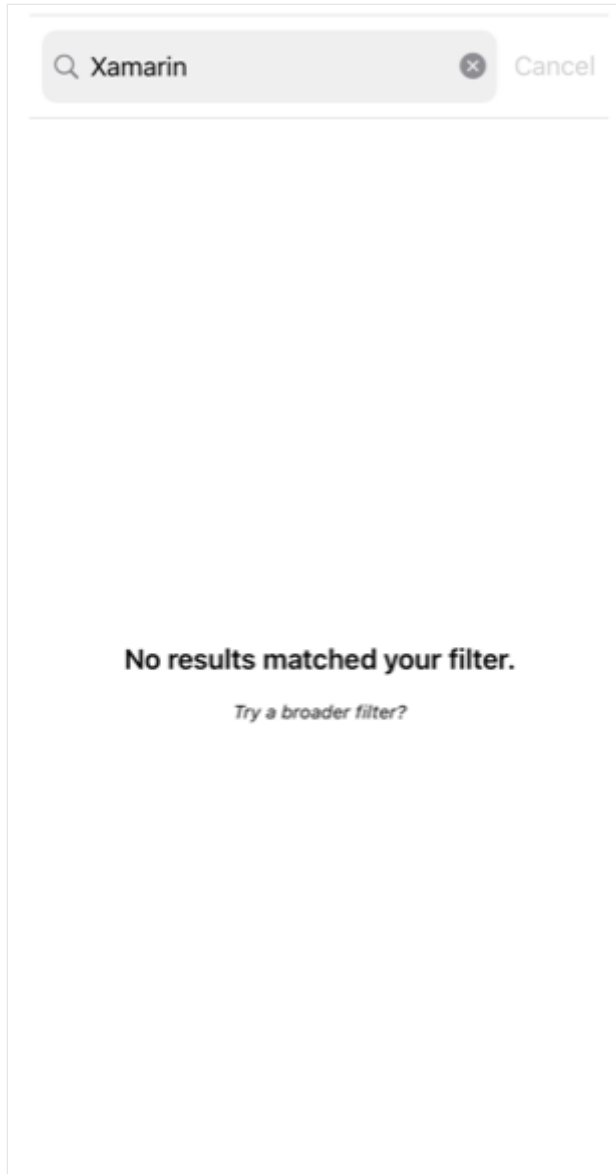
```

StackLayout stackLayout = new StackLayout();
stackLayout.Add(new Label { Text = "No results matched your filter.", ... }
);
stackLayout.Add(new Label { Text = "Try a broader filter?", ... } );

SearchBar searchBar = new SearchBar { ... };
CollectionView collectionView = new CollectionView
{
    EmptyView = new ContentView
    {
        Content = stackLayout
    }
};
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");

```


When the [SearchBar](#) executes the `FilterCommand`, the collection displayed by the [CollectionView](#) is filtered for the search term stored in the `SearchBar.Text` property. If the filtering operation yields no data, the [StackLayout](#) set as the `EmptyView` property value is displayed:



Display a templated custom type when data is unavailable

The `EmptyView` property can be set to a custom type, whose template is displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. The `EmptyViewTemplate` property can be set to a [DataTemplate](#) that defines the appearance of the `EmptyView`. The following XAML shows an example of this scenario:

XAML

```

<Grid Margin="20" RowDefinitions="Auto,*">
    <SearchBar x:Name="searchBar"
        SearchCommand="{Binding FilterCommand}"
        SearchCommandParameter="{Binding Source={x:Reference
searchBar}, Path=Text}"
        Placeholder="Filter" />
    <CollectionView ItemsSource="{Binding Monkeys}"
        Grid.Row="1">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                ...
            </DataTemplate>
        </CollectionView.ItemTemplate>
        <CollectionView.EmptyView>
            <views:FilterData Filter="{Binding Source={x:Reference
searchBar}, Path=Text}" />
        </CollectionView.EmptyView>
        <CollectionView.EmptyViewTemplate>
            <DataTemplate>
                <Label Text="{Binding Filter, StringFormat='Your filter term
of {0} did not match any records.'}"
                    Margin="10,25,10,10"
                    FontAttributes="Bold"
                    FontSize="18"
                    HorizontalOptions="Fill"
                    HorizontalTextAlignment="Center" />
            </DataTemplate>
        </CollectionView.EmptyViewTemplate>
    </CollectionView>
</Grid>

```

The equivalent C# code is:

C#

```

SearchBar searchBar = new SearchBar { ... };
CollectionView collectionView = new CollectionView
{
    EmptyView = new FilterData { Filter = searchBar.Text },
    EmptyViewTemplate = new DataTemplate(() =>
    {
        return new Label { ... };
    })
};

```

The `FilterData` type defines a `Filter` property, and a corresponding [BindableProperty](#):

C#

```

public class FilterData : BindableObject
{

```

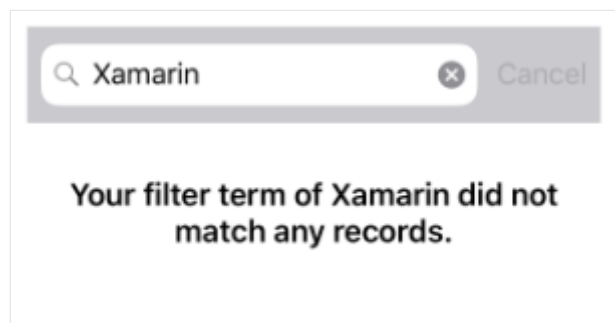
```

public static readonly BindableProperty FilterProperty =
BindableProperty.Create(nameof(Filter), typeof(string), typeof(FilterData),
null);

public string Filter
{
    get { return (string)GetValue(FilterProperty); }
    set { SetValue(FilterProperty, value); }
}
}

```

The `EmptyView` property is set to a `FilterData` object, and the `Filter` property data binds to the `SearchBar.Text` property. When the `SearchBar` executes the `FilterCommand`, the collection displayed by the `CollectionView` is filtered for the search term stored in the `Filter` property. If the filtering operation yields no data, the `Label` defined in the `DataTemplate`, that's set as the `EmptyViewTemplate` property value, is displayed:



⚠ Note

When displaying a templated custom type when data is unavailable, the `EmptyViewTemplate` property can be set to a view that contains multiple child views.

Choose an EmptyView at runtime

Views that will be displayed as an `EmptyView` when data is unavailable, can be defined as `ContentView` objects in a `ResourceDictionary`. The `EmptyView` property can then be set to a specific `ContentView`, based on some business logic, at runtime. The following XAML shows an example of this scenario:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="CollectionViewDemos.Views.EmptyViewSwapPage"
              Title="EmptyView (swap)">
    <ContentPage.Resources>

```

```

<ContentView x:Key="BasicEmptyView">
    <StackLayout>
        <Label Text="No items to display."
            Margin="10,25,10,10"
            FontAttributes="Bold"
            FontSize="18"
            HorizontalOptions="Fill"
            HorizontalTextAlignment="Center" />
    </StackLayout>
</ContentView>
<ContentView x:Key="AdvancedEmptyView">
    <StackLayout>
        <Label Text="No results matched your filter."
            Margin="10,25,10,10"
            FontAttributes="Bold"
            FontSize="18"
            HorizontalOptions="Fill"
            HorizontalTextAlignment="Center" />
        <Label Text="Try a broader filter?"
            FontAttributes="Italic"
            FontSize="12"
            HorizontalOptions="Fill"
            HorizontalTextAlignment="Center" />
    </StackLayout>
</ContentView>
</ContentPage.Resources>

<Grid Margin="20" RowDefinitions="Auto, Auto, *">
    <SearchBar x:Name="searchBar"
        SearchCommand="{Binding FilterCommand}"
        SearchCommandParameter="{Binding Source={x:Reference
searchBar}, Path=Text}"
        Placeholder="Filter" />
    <HorizontalStackLayout Grid.Row="1">
        <Label Text="Toggle EmptyViews" />
        <Switch Toggled="OnEmptyViewSwitchToggled" />
    </HorizontalStackLayout>
    <CollectionView x:Name="collectionView"
        ItemsSource="{Binding Monkeys}"
        Grid.Row="2">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                ...
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</Grid>
</ContentPage>

```

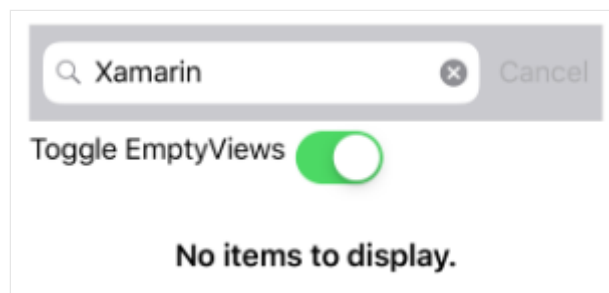
This XAML defines two [ContentView](#) objects in the page-level [ResourceDictionary](#), with the [Switch](#) object controlling which [ContentView](#) object will be set as the `EmptyView`

property value. When the [Switch](#) is toggled, the `OnEmptyViewSwitchToggled` event handler executes the `ToggleEmptyView` method:

C#

```
void ToggleEmptyView(bool isToggled)
{
    collectionView.EmptyView = isToggled ? Resources["BasicEmptyView"] :
Resources["AdvancedEmptyView"];
}
```

The `ToggleEmptyView` method sets the `EmptyView` property of the [CollectionView](#) object to one of the two [ContentView](#) objects stored in the [ResourceDictionary](#), based on the value of the `Switch.IsToggled` property. When the [SearchBar](#) executes the `FilterCommand`, the collection displayed by the [CollectionView](#) is filtered for the search term stored in the `SearchBar.Text` property. If the filtering operation yields no data, the [ContentView](#) object set as the `EmptyView` property is displayed:



For more information about resource dictionaries, see [Resource dictionaries](#).

Choose an EmptyViewTemplate at runtime

The appearance of the `EmptyView` can be chosen at runtime, based on its value, by setting the `CollectionView.EmptyViewTemplate` property to a [DataTemplateSelector](#) object:

XAML

```
<ContentPage ...
    xmlns:controls="clr-namespace:CollectionViewDemos.Controls">
    <ContentPage.Resources>
        <DataTemplate x:Key="AdvancedTemplate">
            ...
        </DataTemplate>

        <DataTemplate x:Key="BasicTemplate">
            ...
        </DataTemplate>
```

```

        <controls:SearchTermDataTemplateSelector x:Key="SearchSelector"
                                                DefaultTemplate="
{StaticResource AdvancedTemplate}"
                                                OtherTemplate="
{StaticResource BasicTemplate}" />
    </ContentPage.Resources>

    <Grid Margin="20" RowDefinitions="Auto,*">
        <SearchBar x:Name="searchBar"
                    SearchCommand="{Binding FilterCommand}"
                    SearchCommandParameter="{Binding Source={x:Reference
searchBar}, Path=Text}"
                    Placeholder="Filter" />
        <CollectionView ItemsSource="{Binding Monkeys}"
                        EmptyView="{Binding Source={x:Reference searchBar},
Path=Text}"
                        EmptyViewTemplate="{StaticResource SearchSelector}"
                        Grid.Row="1" />
    </Grid>
</ContentPage>

```

The equivalent C# code is:

```

C#

SearchBar searchBar = new SearchBar { ... };
CollectionView collectionView = new CollectionView
{
    EmptyView = searchBar.Text,
    EmptyViewTemplate = new SearchTermDataTemplateSelector { ... }
};
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");

```

The `EmptyView` property is set to the `SearchBar.Text` property, and the `EmptyViewTemplate` property is set to a `SearchTermDataTemplateSelector` object.

When the `SearchBar` executes the `FilterCommand`, the collection displayed by the `CollectionView` is filtered for the search term stored in the `SearchBar.Text` property. If the filtering operation yields no data, the `DataTemplate` chosen by the `SearchTermDataTemplateSelector` object is set as the `EmptyViewTemplate` property and displayed.

The following example shows the `SearchTermDataTemplateSelector` class:

```

C#

public class SearchTermDataTemplateSelector : DataTemplateSelector
{

```

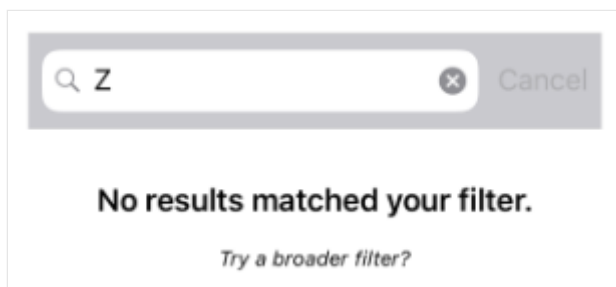
```

public DataTemplate DefaultTemplate { get; set; }
public DataTemplate OtherTemplate { get; set; }

protected override DataTemplate OnSelectTemplate(object item,
BindableObject container)
{
    string query = (string)item;
    return query.ToLower().Equals("xamarin") ? OtherTemplate :
DefaultTemplate;
}
}

```

The `SearchTermTemplateSelector` class defines `DefaultTemplate` and `OtherTemplate` `DataTemplate` properties that are set to different data templates. The `OnSelectTemplate` override returns `DefaultTemplate`, which displays a message to the user, when the search query isn't equal to "xamarin". When the search query is equal to "xamarin", the `OnSelectTemplate` override returns `OtherTemplate`, which displays a basic message to the user:



For more information about data template selectors, see [Create a DataTemplateSelector](#).

Control scrolling in a `CollectionView`

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) `CollectionView` defines two `ScrollTo` methods, that scroll items into view. One of the overloads scrolls the item at the specified index into view, while the other scrolls the specified item into view. Both overloads have additional arguments that can be specified to indicate the group the item belongs to, the exact position of the item after the scroll has completed, and whether to animate the scroll.

`CollectionView` defines a `ScrollToRequested` event that is fired when one of the `ScrollTo` methods is invoked. The `ScrollToRequestedEventArgs` object that accompanies the `ScrollToRequested` event has many properties, including `IsAnimated`, `Index`, `Item`, and `ScrollToPosition`. These properties are set from the arguments specified in the `ScrollTo` method calls.

In addition, `CollectionView` defines a `Scrolled` event that is fired to indicate that scrolling occurred. The `ItemsViewScrolledEventArgs` object that accompanies the `Scrolled` event has many properties. For more information, see [Detect scrolling](#).

`CollectionView` also defines a `ItemsUpdatingScrollMode` property that represents the scrolling behavior of the `CollectionView` when new items are added to it. For more information about this property, see [Control scroll position when new items are added](#).

When a user swipes to initiate a scroll, the end position of the scroll can be controlled so that items are fully displayed. This feature is known as snapping, because items snap to position when scrolling stops. For more information, see [Snap points](#).

`CollectionView` can also load data incrementally as the user scrolls. For more information, see [Load data incrementally](#).

Tip

Placing a `CollectionView` inside a `VerticalStackLayout` can stop the `CollectionView` scrolling, and can limit the number of items that are displayed. In this situation, replace the `VerticalStackLayout` with a `Grid`.

Detect scrolling

[CollectionView](#) defines a `Scrolled` event which is fired to indicate that scrolling occurred. The `ItemsViewScrolledEventArgs` class, which represents the object that accompanies the `Scrolled` event, defines the following properties:

- `HorizontalDelta`, of type `double`, represents the change in the amount of horizontal scrolling. This is a negative value when scrolling left, and a positive value when scrolling right.
- `VerticalDelta`, of type `double`, represents the change in the amount of vertical scrolling. This is a negative value when scrolling upwards, and a positive value when scrolling downwards.
- `HorizontalOffset`, of type `double`, defines the amount by which the list is horizontally offset from its origin.
- `VerticalOffset`, of type `double`, defines the amount by which the list is vertically offset from its origin.
- `FirstVisibleItemIndex`, of type `int`, is the index of the first item that's visible in the list.
- `CenterItemIndex`, of type `int`, is the index of the center item that's visible in the list.
- `LastVisibleItemIndex`, of type `int`, is the index of the last item that's visible in the list.

The following XAML example shows a [CollectionView](#) that sets an event handler for the `Scrolled` event:

XAML

```
<CollectionView Scrolled="OnCollectionViewScrolled">
    ...
</CollectionView>
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView();
collectionView.Scrolled += OnCollectionViewScrolled;
```

In this code example, the `OnCollectionViewScrolled` event handler is executed when the `Scrolled` event fires:

C#

```
void OnCollectionViewScrolled(object sender, ItemsViewScrolledEventArgs e)
{
    // Custom logic
}
```

❗ Important

The `Scrolled` event is fired for user initiated scrolls, and for programmatic scrolls.

Scroll an item at an index into view

One `ScrollTo` method overload scrolls the item at the specified index into view. Given a `CollectionView` object named `CollectionView`, the following example shows how to scroll the item at index 12 into view:

C#

```
collectionView.ScrollTo(12);
```

Alternatively, an item in grouped data can be scrolled into view by specifying the item and group indexes. The following example shows how to scroll the third item in the second group into view:

C#

```
// Items and groups are indexed from zero.
collectionView.ScrollTo(2, 1);
```

❗ Note

The `ScrollToRequested` event is fired when the `ScrollTo` method is invoked.

Scroll an item into view

Another `ScrollTo` method overload scrolls the specified item into view. Given a `CollectionView` object named `CollectionView`, the following example shows how to scroll the Proboscis Monkey item into view:

C#

```
MonkeysViewModel viewModel = BindingContext as MonkeysViewModel;
Monkey monkey = viewModel.Monkeys.FirstOrDefault(m => m.Name == "Proboscis
Monkey");
collectionView.ScrollTo(monkey);
```

Alternatively, an item in grouped data can be scrolled into view by specifying the item and the group. The following example shows how to scroll the Proboscis Monkey item in the Monkeys group into view:

```
C#

GroupedAnimalsViewModel viewModel = BindingContext as
GroupedAnimalsViewModel;
AnimalGroup group = viewModel.Animals.FirstOrDefault(a => a.Name ==
"Monkeys");
Animal monkey = group.FirstOrDefault(m => m.Name == "Proboscis Monkey");
collectionView.ScrollTo(monkey, group);
```

ⓘ Note

The `ScrollToRequested` event is fired when the `ScrollTo` method is invoked.

Disable scroll animation

A scrolling animation is displayed when scrolling an item into view. However, this animation can be disabled by setting the `animate` argument of the `ScrollTo` method to `false`:

```
C#

collectionView.ScrollTo(monkey, animate: false);
```

Control scroll position

When scrolling an item into view, the exact position of the item after the scroll has completed can be specified with the `position` argument of the `ScrollTo` methods. This argument accepts a `ScrollToPosition` enumeration member.

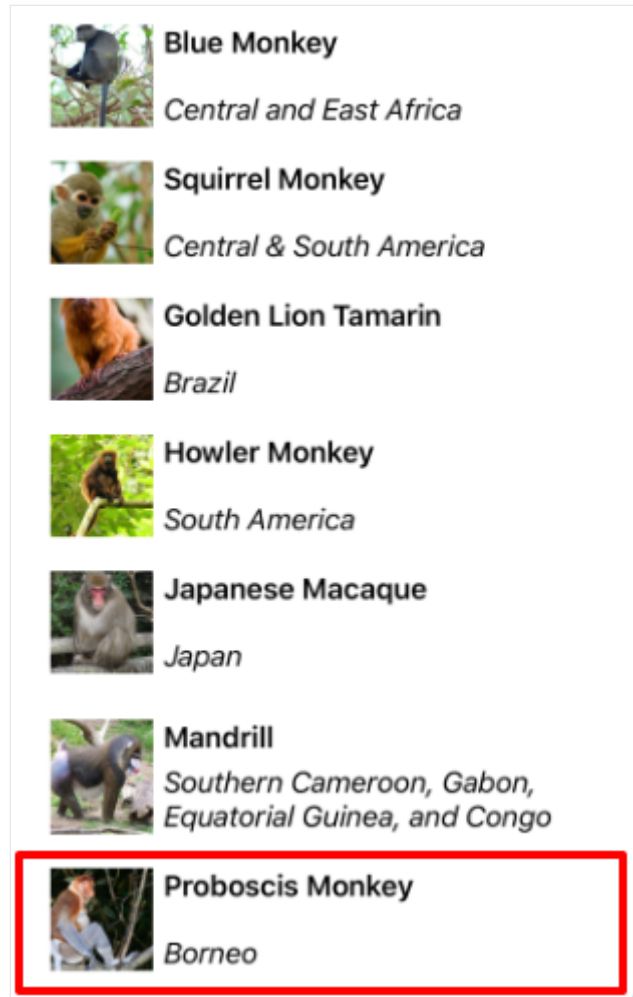
MakeVisible

The `ScrollToPosition.MakeVisible` member indicates that the item should be scrolled until it's visible in the view:

C#

```
collectionView.ScrollTo(monkey, position: ScrollToPosition.MakeVisible);
```

This example code results in the minimal scrolling required to scroll the item into view:



ⓘ Note

The `ScrollToPosition.MakeVisible` member is used by default, if the `position` argument is not specified when calling the `ScrollTo` method.

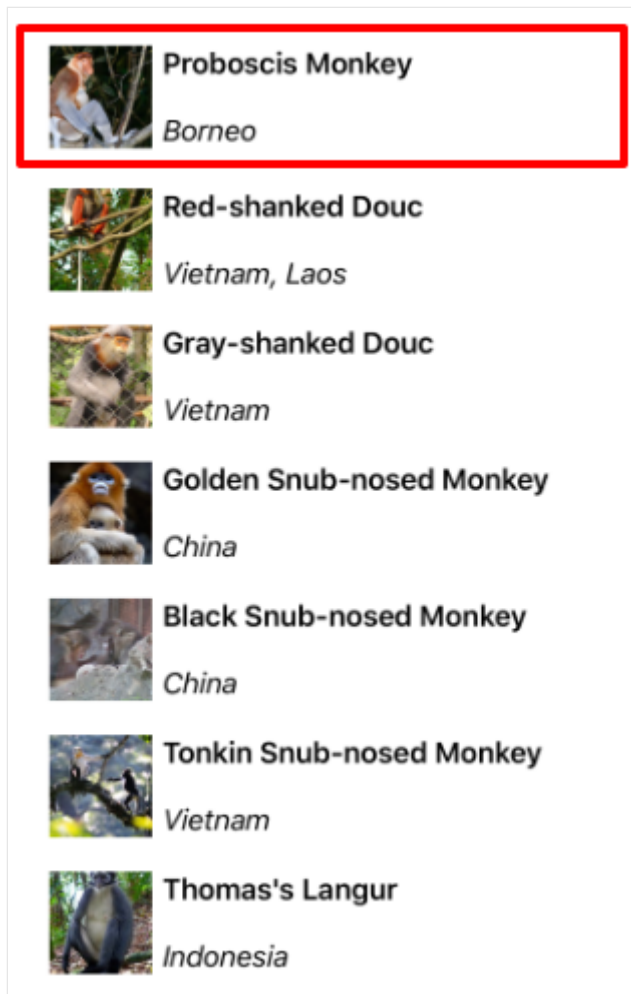
Start

The `ScrollToPosition.Start` member indicates that the item should be scrolled to the start of the view:

C#

```
collectionView.ScrollTo(monkey, position: ScrollToPosition.Start);
```

This example code results in the item being scrolled to the start of the view:



Center

The `ScrollToPosition.Center` member indicates that the item should be scrolled to the center of the view:

C#

```
collectionView.ScrollTo(monkey, position: ScrollToPosition.Center);
```

This example code results in the item being scrolled to the center of the view:

**Howler Monkey***South America***Japanese Macaque***Japan***Mandrill***Southern Cameroon, Gabon,
Equatorial Guinea, and Congo***Proboscis Monkey***Borneo***Red-shanked Douc***Vietnam, Laos***Gray-shanked Douc***Vietnam***Golden Snub-nosed Monkey***China*

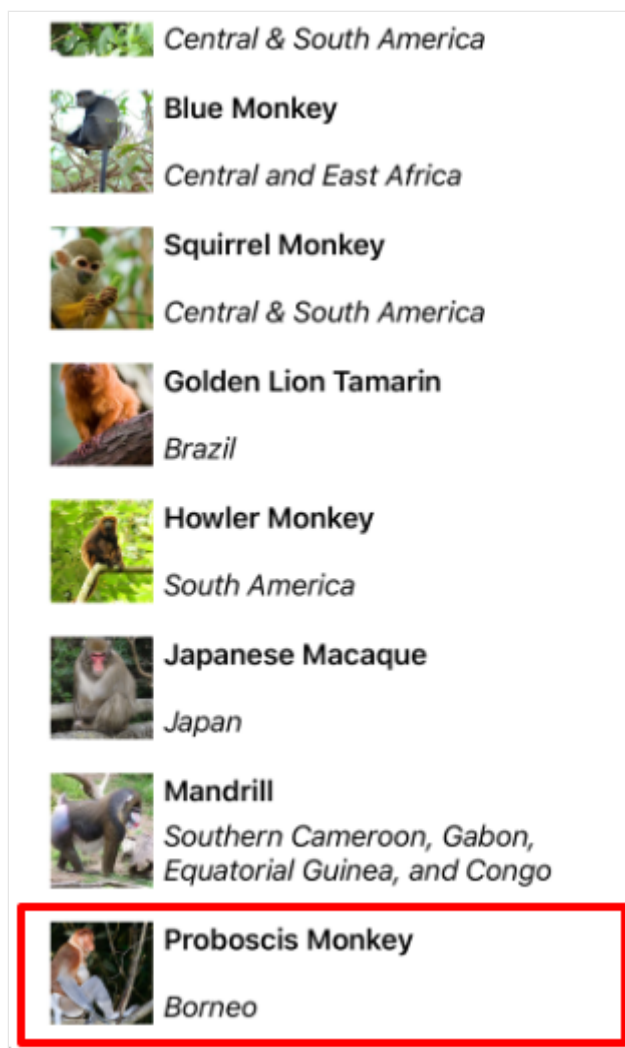
End

The `ScrollToPosition.End` member indicates that the item should be scrolled to the end of the view:

C#

```
collectionView.ScrollTo(monkey, position: ScrollToPosition.End);
```

This example code results in the item being scrolled to the end of the view:



Control scroll position when new items are added

[CollectionView](#) defines a `ItemsUpdatingScrollMode` property, which is backed by a bindable property. This property gets or sets a `ItemsUpdatingScrollMode` enumeration value that represents the scrolling behavior of the [CollectionView](#) when new items are added to it. The `ItemsUpdatingScrollMode` enumeration defines the following members:

- `KeepItemsInView` keeps the first item in the list displayed when new items are added.
- `KeepScrollOffset` ensures that the current scroll position is maintained when new items are added.
- `KeepLastItemInView` adjusts the scroll offset to keep the last item in the list displayed when new items are added.

The default value of the `ItemsUpdatingScrollMode` property is `KeepItemsInView`.

Therefore, when new items are added to a [CollectionView](#) the first item in the list will

remain displayed. To ensure that the last item in the list is displayed when new items are added, set the `ItemsUpdatingScrollMode` property to `KeepLastItemInView`:

XAML

```
<CollectionView ItemsUpdatingScrollMode="KeepLastItemInView">
    ...
</CollectionView>
```

The equivalent C# code is:

C#

```
CollectionView collectionView = new CollectionView
{
    ItemsUpdatingScrollMode = ItemsUpdatingScrollMode.KeepLastItemInView
};
```

Scroll bar visibility

`CollectionView` defines `HorizontalScrollBarVisibility` and `VerticalScrollBarVisibility` properties, which are backed by bindable properties. These properties get or set a `ScrollBarVisibility` enumeration value that represents when the horizontal, or vertical, scroll bar is visible. The `ScrollBarVisibility` enumeration defines the following members:

- `Default` indicates the default scroll bar behavior for the platform, and is the default value for the `HorizontalScrollBarVisibility` and `VerticalScrollBarVisibility` properties.
- `Always` indicates that scroll bars will be visible, even when the content fits in the view.
- `Never` indicates that scroll bars will not be visible, even if the content doesn't fit in the view.

Snap points

When a user swipes to initiate a scroll, the end position of the scroll can be controlled so that items are fully displayed. This feature is known as snapping, because items snap to position when scrolling stops, and is controlled by the following properties from the `ItemsLayout` class:

- `SnapPointsType`, of type `SnapPointsType`, specifies the behavior of snap points when scrolling.
- `SnapPointsAlignment`, of type `SnapPointsAlignment`, specifies how snap points are aligned with items.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings.

ⓘ Note

When snapping occurs, it will occur in the direction that produces the least amount of motion.

Snap points type

The `SnapPointsType` enumeration defines the following members:

- `None` indicates that scrolling does not snap to items.
- `Mandatory` indicates that content always snaps to the closest snap point to where scrolling would naturally stop, along the direction of inertia.
- `MandatorySingle` indicates the same behavior as `Mandatory`, but only scrolls one item at a time.

By default, the `SnapPointsType` property is set to `SnapPointsType.None`, which ensures that scrolling does not snap items, as shown in the following screenshot:

	<i>Central & South America</i>
	Blue Monkey <i>Central and East Africa</i>
	Squirrel Monkey <i>Central & South America</i>
	Golden Lion Tamarin <i>Brazil</i>
	Howler Monkey <i>South America</i>
	Japanese Macaque <i>Japan</i>
	Mandrill <i>Southern Cameroon, Gabon, Equatorial Guinea, and Congo</i>
	Proboscis Monkey <i>Borneo</i>

Snap points alignment

The `SnapPointsAlignment` enumeration defines `Start`, `Center`, and `End` members.

❗ Important

The value of the `SnapPointsAlignment` property is only respected when the `SnapPointsType` property is set to `Mandatory`, Or `MandatorySingle`.

Start

The `SnapPointsAlignment.Start` member indicates that snap points are aligned with the leading edge of items.

By default, the `SnapPointsAlignment` property is set to `SnapPointsAlignment.Start`. However, for completeness, the following XAML example shows how to set this enumeration member:

```
XAML
```

```

<CollectionView ItemsSource="{Binding Monkeys}">
  <CollectionView.ItemsLayout>
    <LinearItemsLayout Orientation="Vertical"
                      SnapPointsType="MandatorySingle"
                      SnapPointsAlignment="Start" />
  </CollectionView.ItemsLayout>
  ...
</CollectionView>

```

The equivalent C# code is:

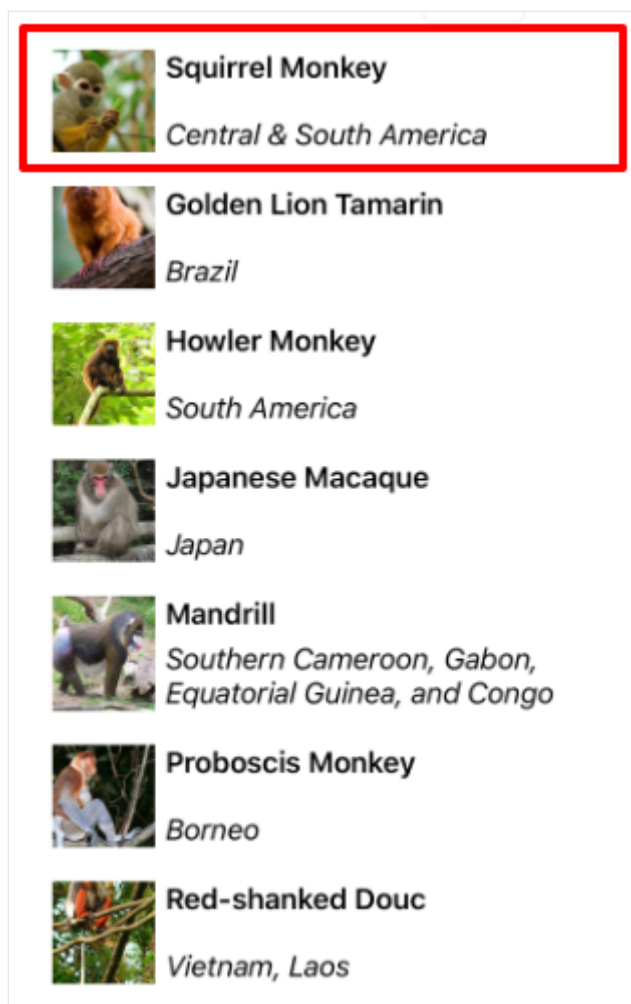
```

C#

CollectionView collectionView = new CollectionView
{
    ItemsLayout = new LinearItemsLayout(ItemsLayoutOrientation.Vertical)
    {
        SnapPointsType = SnapPointsType.MandatorySingle,
        SnapPointsAlignment = SnapPointsAlignment.Start
    },
    // ...
};

```

When a user swipes to initiate a scroll, the top item will be aligned with the top of the view:



Center

The `SnapPointsAlignment.Center` member indicates that snap points are aligned with the center of items. The following XAML example shows how to set this enumeration member:

XAML

```
<CollectionView ItemsSource="{Binding Monkeys}">
    <CollectionView.ItemsLayout>
        <LinearItemsLayout Orientation="Vertical"
                           SnapPointsType="MandatorySingle"
                           SnapPointsAlignment="Center" />
    </CollectionView.ItemsLayout>
    ...
</CollectionView>
```

The equivalent C# code is:

C#

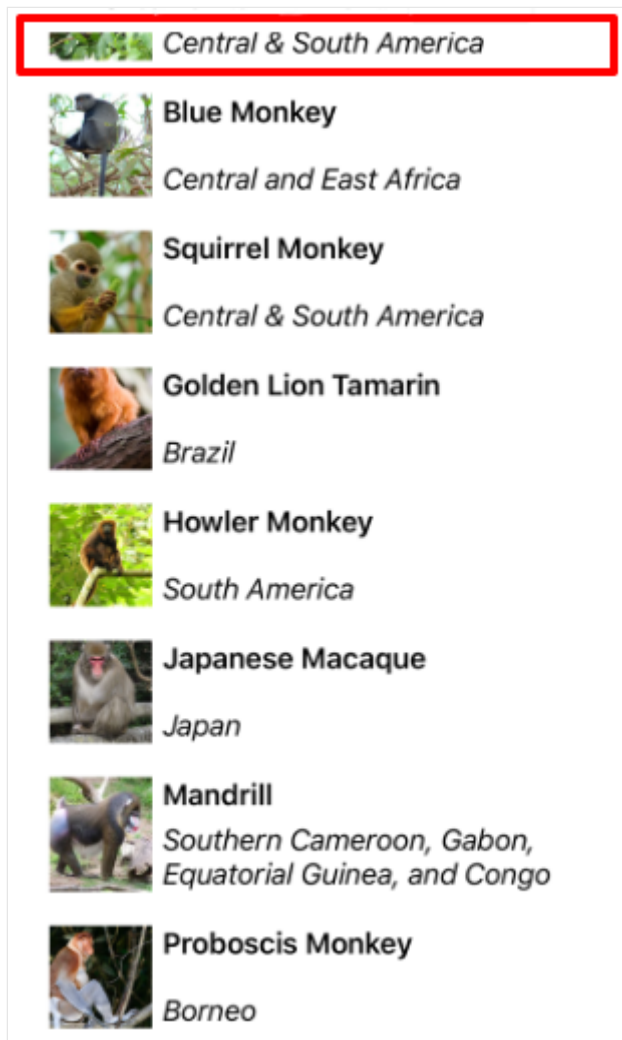
```
CollectionView collectionView = new CollectionView
{
```

```

ItemsLayout = new LinearItemsLayout(ItemsLayoutOrientation.Vertical)
{
    SnapPointsType = SnapPointsType.MandatorySingle,
    SnapPointsAlignment = SnapPointsAlignment.Center
},
// ...
};

```

When a user swipes to initiate a scroll, the top item will be center aligned at the top of the view:



End

The `SnapPointsAlignment.End` member indicates that snap points are aligned with the trailing edge of items. The following XAML example shows how to set this enumeration member:

XAML

```

<CollectionView ItemsSource="{Binding Monkeys}">
    <CollectionView.ItemsLayout>
        <LinearItemsLayout Orientation="Vertical"

```

```

                SnapPointsType="MandatorySingle"
                SnapPointsAlignment="End" />
            </CollectionView.ItemsLayout>
            ...
        </CollectionView>

```

The equivalent C# code is:

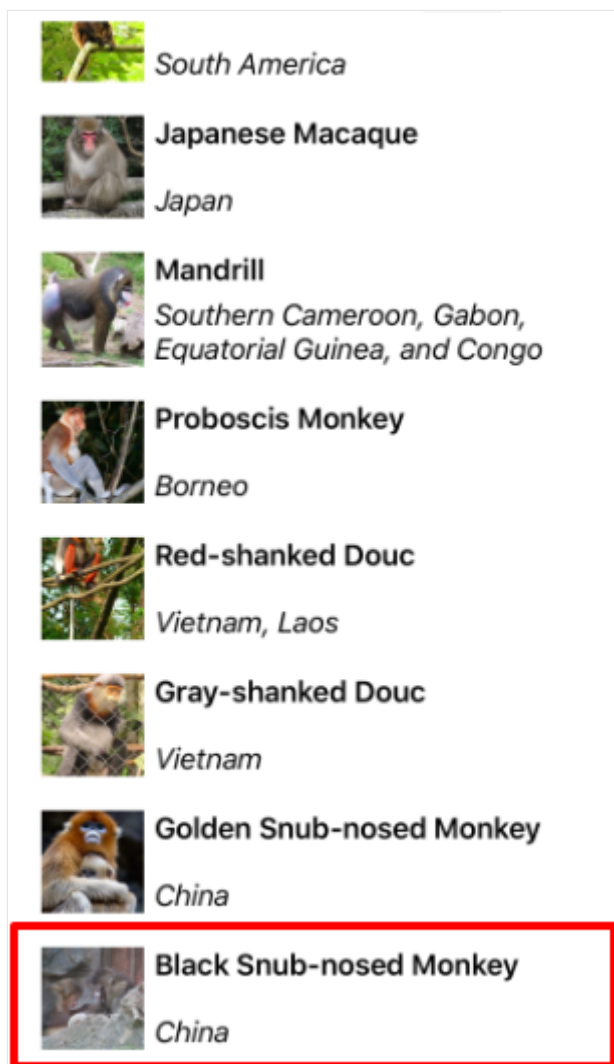
```

C#

CollectionView collectionView = new CollectionView
{
    ItemsLayout = new LinearItemsLayout(ItemsLayoutOrientation.Vertical)
    {
        SnapPointsType = SnapPointsType.MandatorySingle,
        SnapPointsAlignment = SnapPointsAlignment.End
    },
    // ...
};

```

When a user swipes to initiate a scroll, the bottom item will be aligned with the bottom of the view:



Display grouped data in a `CollectionView`

Article • 09/30/2024



[Browse the sample](#)

Large data sets can often become unwieldy when presented in a continually scrolling list. In this scenario, organizing the data into groups can improve the user experience by making it easier to navigate the data.

The .NET Multi-platform App UI (.NET MAUI) [CollectionView](#) supports displaying grouped data, and defines the following properties that control how it will be presented:

- `IsGrouped`, of type `bool`, indicates whether the underlying data should be displayed in groups. The default value of this property is `false`.
- `GroupHeaderTemplate`, of type [DataTemplate](#), the template to use for the header of each group.
- `GroupFooterTemplate`, of type [DataTemplate](#), the template to use for the footer of each group.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings.

The following screenshot shows a [CollectionView](#) displaying grouped data:

Bears	
	American Black Bear <i>North America</i>
	Asian Black Bear <i>Asia</i>
	Brown Bear <i>Northern Eurasia & North America</i>
	Grizzly-Polar Bear Hybrid <i>Canadian Arctic</i>
	Sloth Bear <i>Indian Subcontinent</i>
	Sun Bear <i>Southeast Asia</i>
	Polar Bear <i>Arctic Circle</i>
	Spectacled Bear <i>South America</i>

For more information about data templates, see [Data templates](#).

Group data

Data must be grouped before it can be displayed. This can be accomplished by creating a list of groups, where each group is a list of items. The list of groups should be an

`IEnumerable<T>` collection, where `T` defines two pieces of data:

- A group name.
- An `IEnumerable` collection that defines the items belonging to the group.

The process for grouping data, therefore, is to:

- Create a type that models a single item.
- Create a type that models a single group of items.
- Create an `IEnumerable<T>` collection, where `T` is the type that models a single group of items. This collection is a collection of groups, which stores the grouped data.

- Add data to the `IEnumerable<T>` collection.

Example

When grouping data, the first step is to create a type that models a single item. The following example shows the `Animal` class from the sample application:

C#

```
public class Animal
{
    public string Name { get; set; }
    public string Location { get; set; }
    public string Details { get; set; }
    public string ImageUrl { get; set; }
}
```

The `Animal` class models a single item. A type that models a group of items can then be created. The following example shows the `AnimalGroup` class from the sample application:

C#

```
public class AnimalGroup : List<Animal>
{
    public string Name { get; private set; }

    public AnimalGroup(string name, List<Animal> animals) : base(animals)
    {
        Name = name;
    }
}
```

The `AnimalGroup` class inherits from the `List<T>` class and adds a `Name` property that represents the group name.

An `IEnumerable<T>` collection of groups can then be created:

C#

```
public List<AnimalGroup> Animals { get; private set; } = new
List<AnimalGroup>();
```

This code defines a collection named `Animals`, where each item in the collection is an `AnimalGroup` object. Each `AnimalGroup` object comprises a name, and a `List<Animal>`

collection that defines the `Animal` objects in the group.

Grouped data can then be added to the `Animals` collection:

C#

```
Animals.Add(new AnimalGroup("Bears", new List<Animal>
{
    new Animal
    {
        Name = "American Black Bear",
        Location = "North America",
        Details = "Details about the bear go here.",
        ImageUrl =
"https://upload.wikimedia.org/wikipedia/commons/0/08/01_Schwarzbär.jpg"
    },
    new Animal
    {
        Name = "Asian Black Bear",
        Location = "Asia",
        Details = "Details about the bear go here.",
        ImageUrl =
"https://upload.wikimedia.org/wikipedia/commons/thumb/b/b7/Ursus_thibetanus_3_%28Wroclaw_zoo%29.JPG/180px-Ursus_thibetanus_3_%28Wroclaw_zoo%29.JPG"
    },
    // ...
}));

Animals.Add(new AnimalGroup("Monkeys", new List<Animal>
{
    new Animal
    {
        Name = "Baboon",
        Location = "Africa & Asia",
        Details = "Details about the monkey go here.",
        ImageUrl =
"https://upload.wikimedia.org/wikipedia/commons/thumb/f/fc/Papio_anubis_%28Serengeti%2C_2009%29.jpg/200px-Papio_anubis_%28Serengeti%2C_2009%29.jpg"
    },
    new Animal
    {
        Name = "Capuchin Monkey",
        Location = "Central & South America",
        Details = "Details about the monkey go here.",
        ImageUrl =
"https://upload.wikimedia.org/wikipedia/commons/thumb/4/40/Capuchin_Costa_Rica.jpg/200px-Capuchin_Costa_Rica.jpg"
    },
    new Animal
    {
        Name = "Blue Monkey",
        Location = "Central and East Africa",
        Details = "Details about the monkey go here.",
        ImageUrl =
```

```

"https://upload.wikimedia.org/wikipedia/commons/thumb/8/83/BlueMonkey.jpg/220px-BlueMonkey.jpg"
    },
    // ...
});

```

This code creates two groups in the `Animals` collection. The first `AnimalGroup` is named `Bears`, and contains a `List<Animal>` collection of bear details. The second `AnimalGroup` is named `Monkeys`, and contains a `List<Animal>` collection of monkey details.

Display grouped data

`CollectionView` will display grouped data, provided that the data has been grouped correctly, by setting the `IsGrouped` property to `true`:

XAML

```

<CollectionView ItemsSource="{Binding Animals}"
    IsGrouped="true">
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <Grid Padding="10">
                ...
                <Image Grid.RowSpan="2"
                    Source="{Binding ImageUrl}"
                    Aspect="AspectFill"
                    HeightRequest="60"
                    WidthRequest="60" />
                <Label Grid.Column="1"
                    Text="{Binding Name}"
                    FontAttributes="Bold" />
                <Label Grid.Row="1"
                    Grid.Column="1"
                    Text="{Binding Location}"
                    FontAttributes="Italic"
                    VerticalOptions="End" />
            </Grid>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>

```

The equivalent C# code is:

C#

```

CollectionView collectionView = new CollectionView
{
    IsGrouped = true
}

```

```
};  
collectionView.SetBinding(ItemsView.ItemsSourceProperty, "Animals");  
// ...
```

The appearance of each item in the [CollectionView](#) is defined by setting the `CollectionView.ItemTemplate` property to a [DataTemplate](#). For more information, see [Define item appearance](#).

ⓘ Note

By default, [CollectionView](#) will display the group name in the group header and footer. This behavior can be changed by customizing the group header and group footer.

Customize the group header

The appearance of each group header can be customized by setting the `CollectionView.GroupHeaderTemplate` property to a [DataTemplate](#):

XAML

```
<CollectionView ItemsSource="{Binding Animals}"  
                IsGrouped="true">  
    ...  
    <CollectionView.GroupHeaderTemplate>  
        <DataTemplate>  
            <Label Text="{Binding Name}"  
                  BackgroundColor="LightGray"  
                  FontSize="18"  
                  FontAttributes="Bold" />  
        </DataTemplate>  
    </CollectionView.GroupHeaderTemplate>  
</CollectionView>
```

In this example, each group header is set to a [Label](#) that displays the group name, and that has other appearance properties set. The following screenshot shows the customized group header:

Bears	
	American Black Bear <i>North America</i>
	Asian Black Bear <i>Asia</i>
	Brown Bear <i>Northern Eurasia & North America</i>
	Grizzly-Polar Bear Hybrid <i>Canadian Arctic</i>
	Sloth Bear <i>Indian Subcontinent</i>
	Sun Bear <i>Southeast Asia</i>
	Polar Bear <i>Arctic Circle</i>
	Spectacled Bear <i>South America</i>

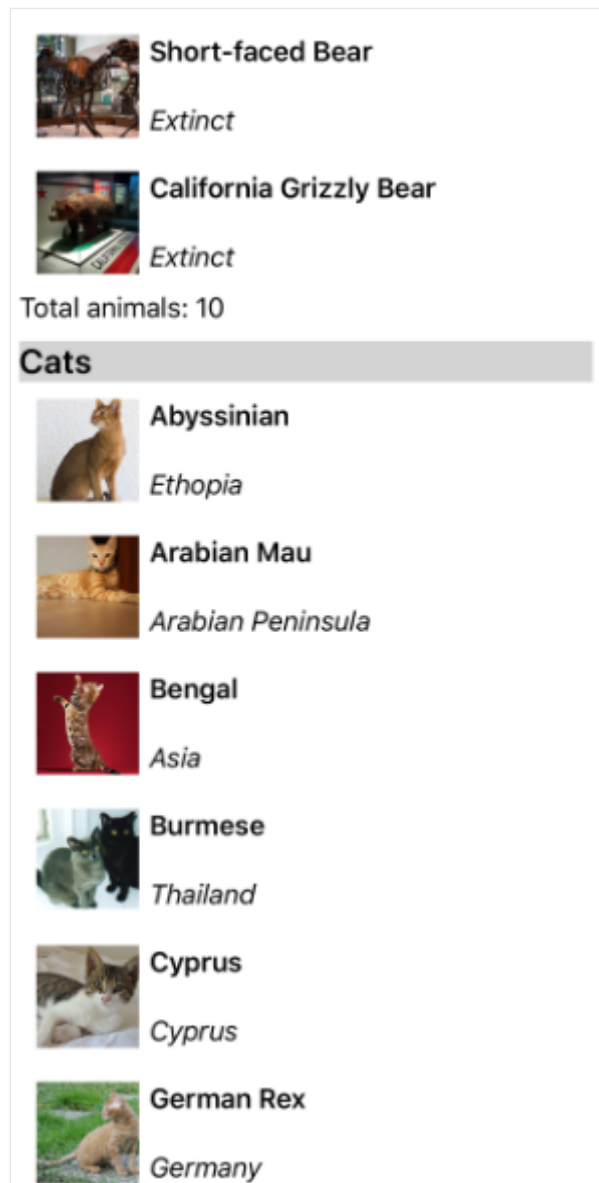
Customize the group footer

The appearance of each group footer can be customized by setting the `CollectionView.GroupFooterTemplate` property to a [DataTemplate](#):

XAML

```
<CollectionView ItemsSource="{Binding Animals}"
    IsGrouped="true">
    ...
    <CollectionView.GroupFooterTemplate>
        <DataTemplate>
            <Label Text="{Binding Count, StringFormat='Total animals:
{0:D}}'"
                Margin="0,0,0,10" />
        </DataTemplate>
    </CollectionView.GroupFooterTemplate>
</CollectionView>
```

In this example, each group footer is set to a [Label](#) that displays the number of items in the group. The following screenshot shows the customized group footer:



Empty groups

When a [CollectionView](#) displays grouped data, it will display any groups that are empty. Such groups will be displayed with a group header and footer, indicating that the group is empty. The following screenshot shows an empty group:

Aardvarks	
Total animals: 0	
Bears	
	American Black Bear <i>North America</i>
	Asian Black Bear <i>Asia</i>
	Brown Bear <i>Northern Eurasia & North America</i>
	Grizzly-Polar Bear Hybrid <i>Canadian Artic</i>
	Sloth Bear <i>Indian Subcontinent</i>
	Sun Bear <i>Southeast Asia</i>
	Polar Bear <i>Arctic Circle</i>

ⓘ Note

On iOS 10, group headers and footers for empty groups may all be displayed at the top of the [CollectionView](#).

Group without templates

[CollectionView](#) can display correctly grouped data without setting the `CollectionView.ItemTemplate` property to a [DataTemplate](#):

XAML

```
<CollectionView ItemsSource="{Binding Animals}"
                IsGrouped="true" />
```

In this scenario, meaningful data can be displayed by overriding the `ToString` method in the type that models a single item, and the type that models a single group of items.

ContentView

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [ContentView](#) is a control that enables the creation of custom, reusable controls.

The [ContentView](#) class defines a `Content` property, of type [View](#), which represents the content of the [ContentView](#). This property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled.

The [ContentView](#) class derives from the `TemplatedView` class, which defines the [ControlTemplate](#) bindable property, of type [ControlTemplate](#), which defines the appearance of the control. For more information about the [ControlTemplate](#) property, see [Customize appearance with a ControlTemplate](#).

ⓘ Note

A [ContentView](#) can only contain a single child.

Create a custom control

The [ContentView](#) class offers little functionality by itself but can be used to create a custom control. The process for creating a custom control is to:

1. Create a class that derives from the [ContentView](#) class.
2. Define any control properties or events in the code-behind file for the custom control.
3. Define the UI for the custom control.

This article demonstrates how to create a `CardView` control, which is a UI element that displays an image, title, and description in a card-like layout.

Create a ContentView-derived class

A [ContentView](#)-derived class can be created using the ContentView item template in Visual Studio. This template creates a XAML file in which the UI for the custom control can be defined, and a code-behind file in which any control properties, events, and other logic can be defined.

Define control properties

Any control properties, events, and other logic should be defined in the code-behind file for the [ContentView](#)-derived class.

The `CardView` custom control defines the following properties:

- `CardTitle`, of type `string`, which represents the title shown on the card.
- `CardDescription`, of type `string`, which represents the description shown on the card.
- `IconImageSource`, of type [ImageSource](#), which represents the image shown on the card.
- `IconBackgroundColor`, of type [Color](#), which represents the background color for the image shown on the card.
- `BorderColor`, of type [Color](#), which represents the color of the card border, image border, and divider line.
- `CardColor`, of type [Color](#), which represents the background color of the card.

Each property is backed by a [BindableProperty](#) instance.

The following example shows the `CardTitle` bindable property in the code-behind file for the `CardView` class:

```
C#

public partial class CardView : ContentView
{
    public static readonly BindableProperty CardTitleProperty =
        BindableProperty.Create(nameof(CardTitle), typeof(string), typeof(CardView),
            string.Empty);

    public string CardTitle
    {
        get => (string)GetValue(CardView.CardTitleProperty);
        set => SetValue(CardView.CardTitleProperty, value);
    }
    // ...

    public CardView()
    {
        InitializeComponent();
    }
}
```

For more information about [BindableProperty](#) objects, see [Bindable properties](#).

Define the UI

The custom control UI can be defined in the XAML file for the [ContentView](#)-derived class, which uses a [ContentView](#) as the root element of the control:

XAML

```
<ContentView ...
    x:Name="this"
    x:Class="CardViewDemo.Controls.CardView">
    <Frame BindingContext="{x:Reference this}"
        BackgroundColor="{Binding CardColor}"
        BorderColor="{Binding BorderColor}"
        ...>
        <Grid>
            ...
            <Frame BorderColor="{Binding BorderColor,
FallbackValue='Black'}"
                BackgroundColor="{Binding IconBackgroundColor,
FallbackValue='Grey'}"
                ...>
                <Image Source="{Binding IconImageSource}"
                    .. />
            </Frame>
            <Label Text="{Binding CardTitle, FallbackValue='Card Title'}"
                ... />
            <BoxView BackgroundColor="{Binding BorderColor,
FallbackValue='Black'}"
                ... />
            <Label Text="{Binding CardDescription, FallbackValue='Card
description text.'}"
                ... />
        </Grid>
    </Frame>
</ContentView>
```

The [ContentView](#) element sets the `x:Name` property to `this`, which can be used to access the object bound to the `CardView` instance. Elements in the layout set bindings on their properties to values defined on the bound object. For more information about data binding, see [Data binding](#).

📌 Note

The `FallbackValue` property in the `Binding` expression provides a default value in case the binding is `null`.

Instantiate a custom control

A reference to the custom control namespace must be added to the page that instantiates the custom control. Once the reference has been added the `CardView` can be instantiated, and its properties defined:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:controls="clr-namespace:CardViewDemo.Controls"
             x:Class="CardViewDemo.CardViewXamlPage">
    <ScrollView>
        <StackLayout>
            <controls:CardView BorderColor="DarkGray"
                              CardTitle="Slavko Vlasic"
                              CardDescription="Lorem ipsum dolor sit amet,
consectetur adipiscing elit. Nulla elit dolor, convallis non interdum."
                              IconBackgroundColor="SlateGray"
                              IconImageSource="user.png" />
            <!-- More CardView objects -->
        </StackLayout>
    </ScrollView>
</ContentPage>
```

The following screenshot shows multiple `CardView` objects:



Customize appearance with a ControlTemplate

A custom control that derives from the [ContentView](#) class can define its UI using XAML or code, or may not define its UI at all. A [ControlTemplate](#) can be used to override the control's appearance, regardless of how that appearance is defined.

For example, a `CardView` layout might occupy too much space for some use cases. A [ControlTemplate](#) can be used to override the `CardView` layout to provide a more compact view, suitable for a condensed list:

XAML

```
<ContentPage.Resources>
  <ResourceDictionary>
    <ControlTemplate x:Key="CardViewCompressed">
      <Grid>
        <Grid.RowDefinitions>
          <RowDefinition Height="100" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
          <ColumnDefinition Width="100" />
          <ColumnDefinition Width="100*" />
        </Grid.ColumnDefinitions>
        <Image Source="{TemplateBinding IconImageSource}"
              BackgroundColor="{TemplateBinding
IconBackgroundColor}"
              WidthRequest="100"
              HeightRequest="100"
              Aspect="AspectFill"
              HorizontalOptions="Center"
              VerticalOptions="Center" />
        <StackLayout Grid.Column="1">
          <Label Text="{TemplateBinding CardTitle}"
                FontAttributes="Bold" />
          <Label Text="{TemplateBinding CardDescription}" />
        </StackLayout>
      </Grid>
    </ControlTemplate>
  </ResourceDictionary>
</ContentPage.Resources>
```

Data binding in a [ControlTemplate](#) uses the `TemplateBinding` markup extension to specify bindings. The [ControlTemplate](#) property can then be set to the defined [ControlTemplate](#) object, by using its `x:Key` value. The following example shows the [ControlTemplate](#) property set on a `CardView` instance:

XAML

```
<controls:CardView ControlTemplate="{StaticResource CardViewCompressed}" />
```

The following screenshot shows a standard `CardView` instance, and multiple `CardView` instances whose control templates have been overridden:



For more information about control templates, see [Control templates](#).

DatePicker

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [DatePicker](#) invokes the platform's date-picker control and allows you to select a date.

[DatePicker](#) defines eight properties:

- `MinimumDate` of type [DateTime](#), which defaults to the first day of the year 1900.
- `MaximumDate` of type `DateTime`, which defaults to the last day of the year 2100.
- `Date` of type `DateTime`, the selected date, which defaults to the value [DateTime.Today](#).
- `Format` of type `string`, a [standard](#) or [custom](#) .NET formatting string, which defaults to "D", the long date pattern.
- `TextColor` of type [Color](#), the color used to display the selected date.
- `FontAttributes` of type `FontAttributes`, which defaults to `FontAttributes.None`.
- `FontFamily` of type `string`, which defaults to `null`.
- `FontSize` of type `double`, which defaults to -1.0.
- `CharacterSpacing`, of type `double`, is the spacing between characters of the [DatePicker](#) text.

All eight properties are backed by [BindableProperty](#) objects, which means that they can be styled, and the properties can be targets of data bindings. The `Date` property has a default binding mode of `BindingMode.TwoWay`, which means that it can be a target of a data binding in an application that uses the Model-View-ViewModel (MVVM) pattern.

⚠ Warning

When setting `MinimumDate` and `MaximumDate`, make sure that `MinimumDate` is always less than or equal to `MaximumDate`. Otherwise, [DatePicker](#) will raise an exception.

The [DatePicker](#) ensures that `Date` is between `MinimumDate` and `MaximumDate`, inclusive. If `MinimumDate` or `MaximumDate` is set so that `Date` is not between them, [DatePicker](#) will adjust the value of `Date`.

The [DatePicker](#) raises a `DateSelected` event when the user selects a date.

Create a DatePicker

When a `DateTime` value is specified in XAML, the XAML parser uses the `DateTime.Parse` method with a `CultureInfo.InvariantCulture` argument to convert the string to a `DateTime` value. The dates must be specified in a precise format: two-digit months, two-digit days, and four-digit years separated by slashes:

XAML

```
<DatePicker MinimumDate="01/01/2022"
            MaximumDate="12/31/2022"
            Date="06/21/2022" />
```

If the `BindingContext` property of `DatePicker` is set to an instance of a viewmodel containing properties of type `DateTime` named `MinDate`, `MaxDate`, and `SelectedDate` (for example), you can instantiate the `DatePicker` like this:

XAML

```
<DatePicker MinimumDate="{Binding MinDate}"
            MaximumDate="{Binding MaxDate}"
            Date="{Binding SelectedDate}" />
```

In this example, all three properties are initialized to the corresponding properties in the viewmodel. Because the `Date` property has a binding mode of `TwoWay`, any new date that the user selects is automatically reflected in the viewmodel.

If the `DatePicker` does not contain a binding on its `Date` property, your app should attach a handler to the `DateSelected` event to be informed when the user selects a new date.

In code, you can initialize the `MinimumDate`, `MaximumDate`, and `Date` properties to values of type `DateTime`:

C#

```
DatePicker datePicker = new DatePicker
{
    MinimumDate = new DateTime(2018, 1, 1),
    MaximumDate = new DateTime(2018, 12, 31),
    Date = new DateTime(2018, 6, 21)
};
```

For information about setting font properties, see [Fonts](#).

DatePicker and layout

It's possible to use an unconstrained horizontal layout option such as `Center`, `Start`, or `End` with `DatePicker`:

XAML

```
<DatePicker ...  
    HorizontalOptions="Center" />
```

However, this is not recommended. Depending on the setting of the `Format` property, selected dates might require different display widths. For example, the "D" format string causes `DateTime` to display dates in a long format, and "Wednesday, September 12, 2018" requires a greater display width than "Friday, May 4, 2018". Depending on the platform, this difference might cause the `DateTime` view to change width in layout, or for the display to be truncated.

Tip

It's best to use the default `HorizontalOptions` setting of `Fill` with `DatePicker`, and not to use a width of `Auto` when putting `DatePicker` in a `Grid` cell.

Localize a DatePicker on Windows

For apps targeting Windows, ensuring that the `DatePicker` displays dates in a format that's localized to the user's settings, including the names of months and days in the picker's dialog, requires specific configuration in your project's *Package.appxmanifest* file. Localizing the elements in the package manifest improves the user experience by adhering to the cultural norms of the user's locale.

Localizing the date formats and strings in the

`<xref:Microsoft.Maui.Controls.DatePicker>` requires declaring the supported languages within your *Package.appxmanifest* file.

Follow these steps to configure your `DatePicker` for localization on Windows:

1. Locate the Resources section.

Navigate to the `Platforms\Windows` folder of your project and open the *Package.appxmanifest* file in a code editor or Visual Studio. If using Visual Studio,

ensure you're viewing the file's raw XML. Look for the `<Resources>` section, which may initially include:

XML

```
<Resources>
  <Resource Language="x-generate" />
</Resources>
```

2. Specify the supported languages.

Replace the `<Resource Language="x-generate">` with `<Resource />` elements for each of your supported languages. The language code should be in the form of a BCP-47 language tag, such as `en-US` for English (United States), `es-ES` for Spanish (Spain), `fr-FR` for French (France) or `de-DE` for German (Germany). For example, to add support for both English (United States) and Spanish (Spain), your `<Resources>` section should be modified to look like this:

XML

```
<Resources>
  <Resource Language="en-US" />
  <Resource Language="es-ES" />
</Resources>
```

This configuration ensures that the [DatePicker](#) will display date formats, months, and days according to the user's locale, significantly enhancing the app's usability and accessibility across different regions.

For more information about localization in .NET MAUI apps, see [Localization](#).

Editor

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [Editor](#) allows you to enter and edit multiple lines of text.

[Editor](#) defines the following properties:

- `AutoSize`, of type `EditorAutoSizeOption`, defines whether the editor will change size to accommodate user input. By default, the editor doesn't auto size.
- `HorizontalTextAlignment`, of type [TextAlignment](#), defines the horizontal alignment of the text.
- `VerticalTextAlignment`, of type [TextAlignment](#), defines the vertical alignment of the text.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

In addition, [Editor](#) defines a `Completed` event, which is raised when the user finalizes text in the [Editor](#) with the return key.

[Editor](#) derives from the [InputView](#) class, from which it inherits the following properties:

- `CharacterSpacing`, of type `double`, sets the spacing between characters in the entered text.
- `CursorPosition`, of type `int`, defines the position of the cursor within the editor.
- `FontAttributes`, of type `FontAttributes`, determines text style.
- `FontAutoScalingEnabled`, of type `bool`, defines whether the text will reflect scaling preferences set in the operating system. The default value of this property is `true`.
- `FontFamily`, of type `string`, defines the font family.
- `FontSize`, of type `double`, defines the font size.
- `IsReadOnly`, of type `bool`, defines whether the user should be prevented from modifying text. The default value of this property is `false`.
- `IsSpellCheckEnabled`, of type `bool`, controls whether spell checking is enabled.
- `IsTextPredictionEnabled`, of type `bool`, controls whether text prediction and automatic text correction is enabled.
- `Keyboard`, of type `Keyboard`, specifies the soft input keyboard that's displayed when entering text.
- `MaxLength`, of type `int`, defines the maximum input length.

- `Placeholder`, of type `string`, defines the text that's displayed when the control is empty.
- `PlaceholderColor`, of type `Color`, defines the color of the placeholder text.
- `SelectionLength`, of type `int`, represents the length of selected text within the control.
- `Text`, of type `string`, defines the text entered into the control.
- `TextColor`, of type `Color`, defines the color of the entered text.
- `TextTransform`, of type `TextTransform`, specifies the casing of the entered text.

These properties are backed by `BindableProperty` objects, which means that they can be targets of data bindings, and styled.

In addition, `InputView` defines a `TextChanged` event, which is raised when the text in the `Editor` changes. The `TextChangedEventArgs` object that accompanies the `TextChanged` event has `NewTextValue` and `OldTextValue` properties, which specify the new and old text, respectively.

For information about specifying fonts on an `Editor`, see [Fonts](#).

Create an Editor

The following example shows how to create an `Editor`:

XAML

```
<Editor x:Name="editor"
    Placeholder="Enter your response here"
    HeightRequest="250"
    TextChanged="OnEditorTextChanged"
    Completed="OnEditorCompleted" />
```

The equivalent C# code is:

C#

```
Editor editor = new Editor { Placeholder = "Enter text", HeightRequest = 250
};
editor.TextChanged += OnEditorTextChanged;
editor.Completed += OnEditorCompleted;
```

The following screenshot shows the resulting `Editor` on Android:

Enter your response here.

❗ Note

On iOS, the soft input keyboard can cover a text input field when the field is near the bottom of the screen, making it difficult to enter text. However, in a .NET MAUI iOS app, pages automatically scroll when the soft input keyboard would cover a text entry field, so that the field is above the soft input keyboard. The `KeyboardAutoManagerScroll.Disconnect` method, in the `Microsoft.Maui.Platform` namespace, can be called to disable this default behavior. The `KeyboardAutoManagerScroll.Connect` method can be called to re-enable the behavior after it's been disabled.

Entered text can be accessed by reading the `Text` property, and the `TextChanged` and `Completed` events signal that the text has changed or been completed.

The `TextChanged` event is raised when the text in the `Editor` changes, and the `TextChangedEventArgs` provide the text before and after the change via the `OldTextValue` and `NewTextValue` properties:

C#

```
void OnEditorTextChanged(object sender, TextChangedEventArgs e)
{
    string oldText = e.OldTextValue;
    string newText = e.NewTextValue;
    string myText = editor.Text;
}
```

The `Completed` event is only raised on Windows when the user has ended input by pressing the `Tab` key on the keyboard, or by focusing another control. The handler for the event is a generic event handler:

C#

```
void OnEditorCompleted(object sender, EventArgs e)
{
```

```
string text = ((Editor)sender).Text;  
}
```

Set character spacing

Character spacing can be applied to an `Editor` by setting the `CharacterSpacing` property to a `double` value:

XAML

```
<Editor ...  
    CharacterSpacing="10" />
```

The result is that characters in the text displayed by the `Editor` are spaced `CharacterSpacing` device-independent units apart.

ⓘ Note

The `CharacterSpacing` property value is applied to the text displayed by the `Text` and `Placeholder` properties.

Limit input length

The `MaxLength` property can be used to limit the input length that's permitted for the `Editor`. This property should be set to a positive integer:

XAML

```
<Editor ... MaxLength="10" />
```

A `MaxLength` property value of 0 indicates that no input will be allowed, and a value of `int.MaxValue`, which is the default value for an `Editor`, indicates that there is no effective limit on the number of characters that may be entered.

Auto-size an Editor

An `Editor` can be made to auto-size to its content by setting the `Editor.AutoSize` property to `TextChanges`, which is a value of the `EditorAutoSizeOption` enumeration. This enumeration has two values:

- `Disabled` indicates that automatic resizing is disabled, and is the default value.
- `TextChanges` indicates that automatic resizing is enabled.

This can be accomplished as follows:

XAML

```
<Editor Text="Enter text here"
        AutoSize="TextChanges" />
```

When auto-resizing is enabled, the height of the `Editor` will increase when the user fills it with text, and the height will decrease as the user deletes text. This can be used to ensure that `Editor` objects in a `DataTemplate` in a `CollectionView` size correctly.

Important

An `Editor` will not auto-size if the `HeightRequest` property has been set.

Transform text

An `Editor` can transform the casing of its text, stored in the `Text` property, by setting the `TextTransform` property to a value of the `TextTransform` enumeration. This enumeration has four values:

- `None` indicates that the text won't be transformed.
- `Default` indicates that the default behavior for the platform will be used. This is the default value of the `TextTransform` property.
- `Lowercase` indicates that the text will be transformed to lowercase.
- `Uppercase` indicates that the text will be transformed to uppercase.

The following example shows transforming text to uppercase:

XAML

```
<Editor Text="This text will be displayed in uppercase."
        TextTransform="Uppercase" />
```

Customize the keyboard

The keyboard that's presented when users interact with an `Editor` can be set programmatically via the `Keyboard` property, to one of the following properties from the

`Keyboard` class:

- `Chat` – used for texting and places where emoji are useful.
- `Default` – the default keyboard.
- `Email` – used when entering email addresses.
- `Numeric` – used when entering numbers.
- `Plain` – used when entering text, without any `KeyboardFlags` specified.
- `Telephone` – used when entering telephone numbers.
- `Text` – used when entering text.
- `Url` – used for entering file paths & web addresses.

The following example shows setting the `Keyboard` property:

XAML

```
<Editor Keyboard="Chat" />
```

The `Keyboard` class also has a `Create` factory method that can be used to customize a keyboard by specifying capitalization, spellcheck, and suggestion behavior.

`KeyboardFlags` enumeration values are specified as arguments to the method, with a customized `Keyboard` being returned. The `KeyboardFlags` enumeration contains the following values:

- `None` – no features are added to the keyboard.
- `CapitalizeSentence` – indicates that the first letter of the first word of each entered sentence will be automatically capitalized.
- `Spellcheck` – indicates that spellcheck will be performed on entered text.
- `Suggestions` – indicates that word completions will be offered on entered text.
- `CapitalizeWord` – indicates that the first letter of each word will be automatically capitalized.
- `CapitalizeCharacter` – indicates that every character will be automatically capitalized.
- `CapitalizeNone` – indicates that no automatic capitalization will occur.
- `All` – indicates that spellcheck, word completions, and sentence capitalization will occur on entered text.

The following XAML code example shows how to customize the default `Keyboard` to offer word completions and capitalize every entered character:

XAML


```

<Editor>
  <Editor.Keyboard>
    <Keyboard x:FactoryMethod="Create">
      <x:Arguments>

<KeyboardFlags>Suggestions,CapitalizeCharacter</KeyboardFlags>
      </x:Arguments>
    </Keyboard>
  </Editor.Keyboard>
</Editor>

```

The equivalent C# code is:

C#

```

Editor editor = new Editor();
editor.Keyboard = Keyboard.Create(KeyboardFlags.Suggestions |
KeyboardFlags.CapitalizeCharacter);

```

Hide and show the soft input keyboard

The `SoftInputExtensions` class, in the `Microsoft.Maui` namespace, provides a series of extension methods that support interacting with the soft input keyboard on controls that support text input. The class defines the following methods:

- `IsSoftInputShowing`, which checks to see if the device is currently showing the soft input keyboard.
- `HideSoftInputAsync`, which will attempt to hide the soft input keyboard if it's currently showing.
- `ShowSoftInputAsync`, which will attempt to show the soft input keyboard if it's currently hidden.

The following example shows how to hide the soft input keyboard on an `Editor` named `editor`, if it's currently showing:

C#

```

if (editor.IsSoftInputShowing())
    await
editor.HideSoftInputAsync(System.Threading.CancellationToken.None);

```

Enable and disable spell checking

The `IsSpellCheckEnabled` property controls whether spell checking is enabled. By default, the property is set to `true`. As the user enters text, misspellings are indicated.

However, for some text entry scenarios, such as entering a username, spell checking provides a negative experience and so should be disabled by setting the `IsSpellCheckEnabled` property to `false`:

XAML

```
<Editor ... IsSpellCheckEnabled="false" />
```

ⓘ Note

When the `IsSpellCheckEnabled` property is set to `false`, and a custom keyboard isn't being used, the native spell checker will be disabled. However, if a `Keyboard` has been set that disables spell checking, such as `Keyboard.Chat`, the `IsSpellCheckEnabled` property is ignored. Therefore, the property cannot be used to enable spell checking for a `Keyboard` that explicitly disables it.

Enable and disable text prediction

The `IsTextPredictionEnabled` property controls whether text prediction and automatic text correction is enabled. By default, the property is set to `true`. As the user enters text, word predictions are presented.

However, for some text entry scenarios, such as entering a username, text prediction and automatic text correction provides a negative experience and should be disabled by setting the `IsTextPredictionEnabled` property to `false`:

XAML

```
<Editor ... IsTextPredictionEnabled="false" />
```

ⓘ Note

When the `IsTextPredictionEnabled` property is set to `false`, and a custom keyboard isn't being used, text prediction and automatic text correction is disabled. However, if a `Keyboard` has been set that disables text prediction, the

`IsTextPredictionEnabled` property is ignored. Therefore, the property cannot be used to enable text prediction for a `Keyboard` that explicitly disables it.

Prevent text entry

Users can be prevented from modifying the text in an [Editor](#) by setting the `IsReadOnly` property, which has a default value of `false`, to `true`:

XAML

```
<Editor Text="This is a read-only Editor"
        IsReadOnly="true" />
```

ⓘ Note

The `IsReadOnly` property does not alter the visual appearance of an [Editor](#), unlike the `IsEnabled` property that also changes the visual appearance of the [Editor](#) to gray.

Entry

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [Entry](#) allows you to enter and edit a single line of text. In addition, the [Entry](#) can be used as a password field.

[Entry](#) defines the following properties:

- `ClearButtonVisibility`, of type `ClearButtonVisibility`, controls whether a clear button is displayed, which enables the user to clear the text. The default value of this property ensures that a clear button isn't displayed.
- `HorizontalTextAlignment`, of type `TextAlignment`, defines the horizontal alignment of the text.
- `IsPassword`, of type `bool`, specifies whether the entry should visually obscure typed text.
- `ReturnCommand`, of type `ICommand`, defines the command to be executed when the return key is pressed.
- `ReturnCommandParameter`, of type `object`, specifies the parameter for the `ReturnCommand`.
- `ReturnType`, of type `ReturnType`, specifies the appearance of the return button.
- `VerticalTextAlignment`, of type `TextAlignment`, defines the vertical alignment of the text.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

In addition, [Entry](#) defines a `Completed` event, which is raised when the user finalizes text in the [Entry](#) with the return key.

[Entry](#) derives from the [InputView](#) class, from which it inherits the following properties:

- `CharacterSpacing`, of type `double`, sets the spacing between characters in the entered text.
- `CursorPosition`, of type `int`, defines the position of the cursor within the editor.
- `FontAttributes`, of type `FontAttributes`, determines text style.
- `FontAutoScalingEnabled`, of type `bool`, defines whether the text will reflect scaling preferences set in the operating system. The default value of this property is `true`.
- `FontFamily`, of type `string`, defines the font family.
- `FontSize`, of type `double`, defines the font size.

- `IsReadOnly`, of type `bool`, defines whether the user should be prevented from modifying text. The default value of this property is `false`.
- `IsSpellCheckEnabled`, of type `bool`, controls whether spell checking is enabled.
- `IsTextPredictionEnabled`, of type `bool`, controls whether text prediction and automatic text correction is enabled.
- `Keyboard`, of type `Keyboard`, specifies the soft input keyboard that's displayed when entering text.
- `MaxLength`, of type `int`, defines the maximum input length.
- `Placeholder`, of type `string`, defines the text that's displayed when the control is empty.
- `PlaceholderColor`, of type `Color`, defines the color of the placeholder text.
- `SelectionLength`, of type `int`, represents the length of selected text within the control.
- `Text`, of type `string`, defines the text entered into the control.
- `TextColor`, of type `Color`, defines the color of the entered text.
- `TextTransform`, of type `TextTransform`, specifies the casing of the entered text.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

In addition, [InputView](#) defines a `TextChanged` event, which is raised when the text in the [Entry](#) changes. The `TextChangedEventArgs` object that accompanies the `TextChanged` event has `NewTextValue` and `OldTextValue` properties, which specify the new and old text, respectively.

For information about specifying fonts on an [Entry](#), see [Fonts](#).

Create an Entry

The following example shows how to create an [Entry](#):

XAML

```
<Entry x:Name="entry"
    Placeholder="Enter text"
    TextChanged="OnEntryTextChanged"
    Completed="OnEntryCompleted" />
```

The equivalent C# code is:

C#

```
Entry entry = new Entry { Placeholder = "Enter text" };
entry.TextChanged += OnEntryTextChanged;
entry.Completed += OnEntryCompleted;
```

The following screenshot shows the resulting [Entry](#) on Android:



ⓘ Note

On iOS, the soft input keyboard can cover a text input field when the field is near the bottom of the screen, making it difficult to enter text. However, in a .NET MAUI iOS app, pages automatically scroll when the soft input keyboard would cover a text entry field, so that the field is above the soft input keyboard. The `KeyboardAutoManagerScroll.Disconnect` method, in the `Microsoft.Maui.Platform` namespace, can be called to disable this default behavior. The `KeyboardAutoManagerScroll.Connect` method can be called to re-enable the behavior after it's been disabled.

Entered text can be accessed by reading the `Text` property, and the `TextChanged` and `Completed` events signal that the text has changed or been completed.

The `TextChanged` event is raised when the text in the [Entry](#) changes, and the `TextChangedEventArgs` provide the text before and after the change via the `OldTextValue` and `NewTextValue` properties:

```
C#

void OnEntryTextChanged(object sender, TextChangedEventArgs e)
{
    string oldText = e.OldTextValue;
    string newText = e.NewTextValue;
    string myText = entry.Text;
}
```

The `Completed` event is raised when the user has ended input by pressing the `Return` key on the keyboard, or by pressing the Tab key on Windows. The handler for the event is a generic event handler:

```
C#
```

```
void OnEntryCompleted(object sender, EventArgs e)
{
    string text = ((Entry)sender).Text;
}
```

After the `Completed` event fires, any `ICommand` specified by the `ReturnCommand` property is executed, with the `object` specified by the `ReturnCommandParameter` property being passed to the `ReturnCommand`.

ⓘ Note

The `VisualElement` class, which is in the `Entry` inheritance hierarchy, also has `Focused` and `Unfocused` events.

Set character spacing

Character spacing can be applied to an `Entry` by setting the `CharacterSpacing` property to a `double` value:

XAML

```
<Entry ...
    CharacterSpacing="10" />
```

The result is that characters in the text displayed by the `Entry` are spaced `CharacterSpacing` device-independent units apart.

ⓘ Note

The `CharacterSpacing` property value is applied to the text displayed by the `Text` and `Placeholder` properties.

Limit input length

The `MaxLength` property can be used to limit the input length that's permitted for the `Entry`. This property should be set to a positive integer:

XAML

```
<Entry ...  
    MaxLength="10" />
```

A `MaxLength` property value of 0 indicates that no input will be allowed, and a value of `int.MaxValue`, which is the default value for an `Entry`, indicates that there is no effective limit on the number of characters that may be entered.

Set the cursor position and text selection length

The `CursorPosition` property can be used to return or set the position at which the next character will be inserted into the string stored in the `Text` property:

XAML

```
<Entry Text="Cursor position set"  
    CursorPosition="5" />
```

The default value of the `CursorPosition` property is 0, which indicates that text will be inserted at the start of the `Entry`.

In addition, the `SelectionLength` property can be used to return or set the length of text selection within the `Entry`:

XAML

```
<Entry Text="Cursor position and selection length set"  
    CursorPosition="2"  
    SelectionLength="10" />
```

The default value of the `SelectionLength` property is 0, which indicates that no text is selected.

Display a clear button

The `ClearButtonVisibility` property can be used to control whether an `Entry` displays a clear button, which enables the user to clear the text. This property should be set to a `ClearButtonVisibility` enumeration member:

- `Never` indicates that a clear button will never be displayed. This is the default value for the `ClearButtonVisibility` property.
- `WhileEditing` indicates that a clear button will be displayed in the [Entry](#), while it has focus and text.

The following example shows setting the property:

XAML

```
<Entry Text=".NET MAUI"
      ClearButtonVisibility="WhileEditing" />
```

The following screenshot shows an [Entry](#) on Android with the clear button enabled:



Transform text

An [Entry](#) can transform the casing of its text, stored in the `Text` property, by setting the `TextTransform` property to a value of the `TextTransform` enumeration. This enumeration has four values:

- `None` indicates that the text won't be transformed.
- `Default` indicates that the default behavior for the platform will be used. This is the default value of the `TextTransform` property.
- `Lowercase` indicates that the text will be transformed to lowercase.
- `Uppercase` indicates that the text will be transformed to uppercase.

The following example shows transforming text to uppercase:

XAML

```
<Entry Text="This text will be displayed in uppercase."
      TextTransform="Uppercase" />
```

Obscure text entry

[Entry](#) provides the `IsPassword` property which visually obscures entered text when it's set to `true`:

XAML

```
<Entry IsPassword="true" />
```

The following screenshot shows an `Entry` whose input has been obscured:



Customize the keyboard

The soft input keyboard that's presented when users interact with an `Entry` can be set programmatically via the `Keyboard` property, to one of the following properties from the `Keyboard` class:

- `Chat` – used for texting and places where emoji are useful.
- `Default` – the default keyboard.
- `Email` – used when entering email addresses.
- `Numeric` – used when entering numbers.
- `Plain` – used when entering text, without any `KeyboardFlags` specified.
- `Telephone` – used when entering telephone numbers.
- `Text` – used when entering text.
- `Url` – used for entering file paths & web addresses.

The following example shows setting the `Keyboard` property:

XAML

```
<Entry Keyboard="Chat" />
```

The `Keyboard` class also has a `Create` factory method that can be used to customize a keyboard by specifying capitalization, spellcheck, and suggestion behavior.

`KeyboardFlags` enumeration values are specified as arguments to the method, with a customized `Keyboard` being returned. The `KeyboardFlags` enumeration contains the following values:

- `None` – no features are added to the keyboard.
- `CapitalizeSentence` – indicates that the first letter of the first word of each entered sentence will be automatically capitalized.
- `Spellcheck` – indicates that spellcheck will be performed on entered text.

- `Suggestions` – indicates that word completions will be offered on entered text.
- `CapitalizeWord` – indicates that the first letter of each word will be automatically capitalized.
- `CapitalizeCharacter` – indicates that every character will be automatically capitalized.
- `CapitalizeNone` – indicates that no automatic capitalization will occur.
- `All` – indicates that spellcheck, word completions, and sentence capitalization will occur on entered text.

The following XAML code example shows how to customize the default `Keyboard` to offer word completions and capitalize every entered character:

XAML

```
<Entry Placeholder="Enter text here">
    <Entry.Keyboard>
        <Keyboard x:FactoryMethod="Create">
            <x:Arguments>

<KeyboardFlags>Suggestions,CapitalizeCharacter</KeyboardFlags>
            </x:Arguments>
        </Keyboard>
    </Entry.Keyboard>
</Entry>
```

The equivalent C# code is:

C#

```
Entry entry = new Entry { Placeholder = "Enter text here" };
entry.Keyboard = Keyboard.Create(KeyboardFlags.Suggestions |
KeyboardFlags.CapitalizeCharacter);
```

Customize the return key

The appearance of the return key on the soft input keyboard, which is displayed when an `Entry` has focus, can be customized by setting the `ReturnType` property to a value of the `ReturnType` enumeration:

- `Default` – indicates that no specific return key is required and that the platform default will be used.
- `Done` – indicates a "Done" return key.
- `Go` – indicates a "Go" return key.
- `Next` – indicates a "Next" return key.

- `Search` – indicates a "Search" return key.
- `Send` – indicates a "Send" return key.

The following XAML example shows how to set the return key:

XAML

```
<Entry ReturnType="Send" />
```

ⓘ Note

The exact appearance of the return key is dependent upon the platform. On iOS, the return key is a text-based button. However, on Android and Windows, the return key is a icon-based button.

When the `Return` key is pressed, the `Completed` event fires and any [ICommand](#) specified by the `ReturnCommand` property is executed. In addition, any `object` specified by the `ReturnCommandParameter` property will be passed to the [ICommand](#) as a parameter. For more information about commands, see [Commanding](#).

Hide and show the soft input keyboard

The `SoftInputExtensions` class, in the `Microsoft.Maui` namespace, provides a series of extension methods that support interacting with the soft input keyboard on controls that support text input. The class defines the following methods:

- `IsSoftInputShowing`, which checks to see if the device is currently showing the soft input keyboard.
- `HideSoftInputAsync`, which will attempt to hide the soft input keyboard if it's currently showing.
- `ShowSoftInputAsync`, which will attempt to show the soft input keyboard if it's currently hidden.

The following example shows how to hide the soft input keyboard on an [Entry](#) named `entry`, if it's currently showing:

C#

```
if (entry.IsSoftInputShowing())  
    await entry.HideSoftInputAsync(System.Threading.CancellationToken.None);
```

Enable and disable spell checking

The `IsSpellCheckEnabled` property controls whether spell checking is enabled. By default, the property is set to `true`. As the user enters text, misspellings are indicated.

However, for some text entry scenarios, such as entering a username, spell checking provides a negative experience and should be disabled by setting the

`IsSpellCheckEnabled` property to `false`:

XAML

```
<Entry ... IsSpellCheckEnabled="false" />
```

ⓘ Note

When the `IsSpellCheckEnabled` property is set to `false`, and a custom keyboard isn't being used, the native spell checker will be disabled. However, if a `Keyboard` has been set that disables spell checking, such as `Keyboard.Chat`, the `IsSpellCheckEnabled` property is ignored. Therefore, the property cannot be used to enable spell checking for a `Keyboard` that explicitly disables it.

Enable and disable text prediction

The `IsTextPredictionEnabled` property controls whether text prediction and automatic text correction is enabled. By default, the property is set to `true`. As the user enters text, word predictions are presented.

However, for some text entry scenarios, such as entering a username, text prediction and automatic text correction provides a negative experience and should be disabled by setting the `IsTextPredictionEnabled` property to `false`:

XAML

```
<Entry ... IsTextPredictionEnabled="false" />
```

ⓘ Note

When the `IsTextPredictionEnabled` property is set to `false`, and a custom keyboard isn't being used, text prediction and automatic text correction is disabled.

However, if a `Keyboard` has been set that disables text prediction, the `IsTextPredictionEnabled` property is ignored. Therefore, the property cannot be used to enable text prediction for a `Keyboard` that explicitly disables it.

Prevent text entry

Users can be prevented from modifying the text in an `Entry` by setting the `IsReadOnly` property to `true`:

XAML

```
<Entry Text="User input won't be accepted."
      IsReadOnly="true" />
```

ⓘ Note

The `IsReadOnly` property does not alter the visual appearance of an `Entry`, unlike the `IsEnabled` property that also changes the visual appearance of the `Entry` to gray.

Frame

Article • 11/12/2024

The .NET Multi-platform App UI (.NET MAUI) [Frame](#) is used to wrap a view or layout with a border that can be configured with color, shadow, and other options. Frames can be used to create borders around controls but can also be used to create more complex UI.

❗ Important

The [Frame](#) control is marked as obsolete in .NET MAUI 9, and will be completely removed in a future release. The [Border](#) control should be used in its place. For more information see [Border](#).

The [Frame](#) class defines the following properties:

- `BorderColor`, of type [Color](#), determines the color of the [Frame](#) border.
- `CornerRadius`, of type `float`, determines the rounded radius of the corner.
- `HasShadow`, of type `bool`, determines whether the frame has a drop shadow.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [Frame](#) class inherits from [ContentView](#), which provides a `Content` bindable property. The `Content` property is the [ContentProperty](#) of the [Frame](#) class, and therefore does not need to be explicitly set from XAML.

❗ Note

The [Frame](#) class existed in Xamarin.Forms and is present in .NET MAUI for users who are migrating their apps from Xamarin.Forms to .NET MAUI. If you're building a new .NET MAUI app it's recommended to use [Border](#) instead, and to set shadows using the `Shadow` bindable property on [VisualElement](#). For more information, see [Border](#) and [Shadow](#).

Create a Frame

A [Frame](#) object typically wraps another control, such as a [Label](#):

XAML

```
<Frame>
  <Label Text="Frame wrapped around a Label" />
</Frame>
```

The appearance of [Frame](#) objects can be customized by setting properties:

XAML

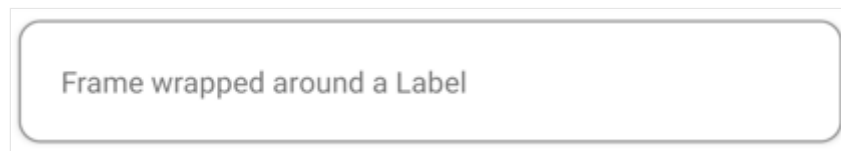
```
<Frame BorderColor="Gray"
  CornerRadius="10">
  <Label Text="Frame wrapped around a Label" />
</Frame>
```

The equivalent C# code is:

C#

```
Frame frame = new Frame
{
    BorderColor = Colors.Gray,
    CornerRadius = 10,
    Content = new Label { Text = "Frame wrapped around a Label" }
};
```

The following screenshot shows the example [Frame](#):



Create a card with a Frame

Combining a [Frame](#) object with a layout such as a [StackLayout](#) enables the creation of more complex UI.

The following XAML shows how to create a card with a [Frame](#):

XAML

```
<Frame BorderColor="Gray"
  CornerRadius="5"
  Padding="8">
  <StackLayout>
    <Label Text="Card Example"
      FontSize="14"
      FontAttributes="Bold" />
  </StackLayout>
</Frame>
```

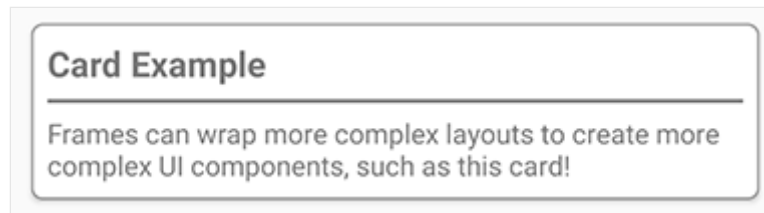


```

<BoxView Color="Gray"
    HeightRequest="2"
    HorizontalOptions="Fill" />
<Label Text="Frames can wrap more complex layouts to create more complex
UI components, such as this card!"/>
</StackLayout>
</Frame>

```

The following screenshot shows the example card:



Round elements

The `CornerRadius` property of the `Frame` control is one approach to creating a circle image. The following XAML shows how to create a circle image with a `Frame`:

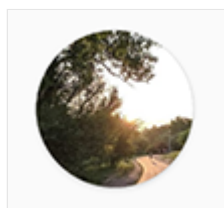
XAML

```

<Frame Margin="10"
    BorderColor="Black"
    CornerRadius="50"
    HeightRequest="60"
    WidthRequest="60"
    IsClippedToBounds="True"
    HorizontalOptions="Center"
    VerticalOptions="Center">
    <Image Source="outdoors.jpg"
        Aspect="AspectFill"
        Margin="-20"
        HeightRequest="100"
        WidthRequest="100" />
</Frame>

```

The following screenshot shows the example circle image:



GraphicsView

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [GraphicsView](#) is a graphics canvas on which 2D graphics can be drawn using types from the [Microsoft.Maui.Graphics](#) namespace. For more information about [Microsoft.Maui.Graphics](#), see [Graphics](#).

[GraphicsView](#) defines the `Drawable` property, of type `IDrawable`, which specifies the content that will be drawn. This property is backed by a [BindableProperty](#), which means it can be the target of data binding, and styled.

[GraphicsView](#) defines the following events:

- `StartHoverInteraction`, with `TouchEventArgs`, which is raised when a pointer enters the hit test area of the [GraphicsView](#).
- `MoveHoverInteraction`, with `TouchEventArgs`, which is raised when a pointer moves while the pointer remains within the hit test area of the [GraphicsView](#).
- `EndHoverInteraction`, which is raised when a pointer leaves the hit test area of the [GraphicsView](#).
- `StartInteraction`, with `TouchEventArgs`, which is raised when the [GraphicsView](#) is pressed.
- `DragInteraction`, with `TouchEventArgs`, which is raised when the [GraphicsView](#) is dragged.
- `EndInteraction`, with `TouchEventArgs`, which is raised when the press that raised the `StartInteraction` event is released.
- `CancelInteraction`, which is raised when the press that made contact with the [GraphicsView](#) loses contact.

Create a GraphicsView

A [GraphicsView](#) must define an `IDrawable` object that specifies the content that will be drawn on the control. This can be achieved by creating an object that derives from `IDrawable`, and by implementing its `Draw` method:

C#

```
namespace MyMauiApp
{
    public class GraphicsDrawable : IDrawable
```

```

{
    public void Draw(ICanvas canvas, RectF dirtyRect)
    {
        // Drawing code goes here
    }
}

```

The `Draw` method has `ICanvas` and `RectF` arguments. The `ICanvas` argument is the drawing canvas on which you draw graphical objects. The `RectF` argument is a `struct` that contains data about the size and location of the drawing canvas. For more information about drawing on an `ICanvas`, see [Draw graphical objects](#).

In XAML, the `IDrawable` object can be declared as a resource and then consumed by a `GraphicsView` by specifying its key as the value of the `Drawable` property:

XAML

```

<ContentPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui
    xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml
    xmlns:drawable="clr-namespace:MyMauiApp"
    x:Class="MyMauiApp.MainPage">
    <ContentPage.Resources>
        <drawable:GraphicsDrawable x:Key="drawable" />
    </ContentPage.Resources>
    <VerticalStackLayout>
        <GraphicsView Drawable="{StaticResource drawable}"
            HeightRequest="300"
            WidthRequest="400" />
    </VerticalStackLayout>
</ContentPage>

```

Position and size graphical objects

The location and size of the `ICanvas` on a page can be determined by examining properties of the `RectF` argument in the `Draw` method.

The `RectF` struct defines the following properties:

- `Bottom`, of type `float`, which represents the y-coordinate of the bottom edge of the canvas.
- `Center`, of type `PointF`, which specifies the coordinates of the center of the canvas.
- `Height`, of type `float`, which defines the height of the canvas.
- `IsEmpty`, of type `bool`, which indicates whether the canvas has a zero size and location.

- `Left`, of type `float`, which represents the x-coordinate of the left edge of the canvas.
- `Location`, of type `PointF`, which defines the coordinates of the upper-left corner of the canvas.
- `Right`, of type `float`, which represents the x-coordinate of the right edge of the canvas.
- `Size`, of type `SizeF`, which defines the width and height of the canvas.
- `Top`, of type `float`, which represents the y-coordinate of the top edge of the canvas.
- `Width`, of type `float`, which defines the width of the canvas.
- `X`, of type `float`, which defines the x-coordinate of the upper-left corner of the canvas.
- `Y`, of type `float`, which defines the y-coordinate of the upper-left corner of the canvas.

These properties can be used to position and size graphical objects on the [ICanvas](#). For example, graphical objects can be placed at the center of the `Canvas` by using the `Center.X` and `Center.Y` values as arguments to a drawing method. For information about drawing on an [ICanvas](#), see [Draw graphical objects](#).

Invalidate the canvas

[GraphicsView](#) has an `Invalidate` method that informs the canvas that it needs to redraw itself. This method must be invoked on a [GraphicsView](#) instance:

C#

```
graphicsView.Invalidate();
```

.NET MAUI automatically invalidates the [GraphicsView](#) as needed by the UI. For example, when the element is first shown, comes into view, or is revealed by moving an element from on top of it, it's redrawn. The only time you need to call `Invalidate` is when you want to force the [GraphicsView](#) to redraw itself, such as if you have changed its content while it's still visible.

HybridWebView

Article • 11/14/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [HybridWebView](#) enables hosting arbitrary HTML/JS/CSS content in a web view, and enables communication between the code in the web view (JavaScript) and the code that hosts the web view (C#/.NET). For example, if you have an existing React JS app, you could host it in a cross-platform .NET MAUI native app, and build the back-end of the app using C# and .NET.

[HybridWebView](#) defines the following properties:

- [DefaultFile](#), of type `string?`, which specifies the file within the [HybridRoot](#) that should be served as the default file. The default value is *index.html*.
- [HybridRoot](#), of type `string?`, which is the path within the app's raw asset resources that contain the web app's contents. The default value is *wwwroot*, which maps to *Resources/Raw/wwwroot*.

In addition, [HybridWebView](#) defines a [RawMessageReceived](#) event that's raised when a raw message is received. The [HybridWebViewRawMessageReceivedEventArgs](#) object that accompanies the event defines a [Message](#) property that contains the message.

Your app's C# code can invoke synchronous and asynchronous JavaScript methods within the [HybridWebView](#) with the [InvokeJavaScriptAsync](#) and [EvaluateJavaScriptAsync](#) methods. Your app's JavaScript code can also synchronously invoke C# methods. For more information, see [Invoke JavaScript from C#](#) and [Invoke C# from JavaScript](#).

To create a .NET MAUI app with [HybridWebView](#) you need:

- The web content of the app, which consists of static HTML, JavaScript, CSS, images, and other files.
- A [HybridWebView](#) control as part of the app's UI. This can be achieved by referencing it in the app's XAML.
- Code in the web content, and in C#/.NET, that uses the [HybridWebView](#) APIs to send messages between the two components.

The entire app, including the web content, is packaged and runs locally on a device, and can be published to applicable app stores. The web content is hosted within a native web view control and runs within the context of the app. Any part of the app can access external web services, but isn't required to.

Important

By default, the [HybridWebView](#) control won't be available when full trimming or Native AOT is enabled. To change this behavior, see [Trimming feature switches](#).

Create a .NET MAUI HybridWebView app

To create a .NET MAUI app with a [HybridWebView](#):

1. Open an existing .NET MAUI app project or create a new .NET MAUI app project.
2. Add your web content to the .NET MAUI app project.

Your app's web content should be included as part of a .NET MAUI project as raw assets. A raw asset is any file in the app's *Resources\Raw* folder, and includes sub-folders. For a default [HybridWebView](#), web content should be placed in the *Resources\Raw\wwwroot* folder, with the main file named *index.html*.

A simple app might have the following files and contents:

- *Resources\Raw\wwwroot\index.html* with content for the main UI:

HTML

```
<!DOCTYPE html>

<html lang="en" xmlns="http://www.w3.org/1999/xhtml">
<head>
  <meta charset="utf-8" />
  <title></title>
  <link rel="icon" href="data:,">
  <link rel="stylesheet" href="styles/app.css">
  <script src="scripts/HybridWebView.js"></script>
</script>
  function LogMessage(msg) {
    var messageLog =
document.getElementById("messageLog");
    messageLog.value += '\r\n' + msg;
  }

  window.addEventListener(
    "HybridWebViewMessageReceived",
    function (e) {
      LogMessage("Raw message: " + e.detail.message);
    });

  function AddNumbers(a, b) {
    var result = {
```

```

        "result": a + b,
        "operationName": "Addition"
    };
    return result;
}

var count = 0;

async function EvaluateMeWithParamsAndAsyncReturn(s1, s2)
{
    const response = await fetch("/asyncdata.txt");
    if (!response.ok) {
        throw new Error(`HTTP error: ${response.status}`);
    }
    var jsonData = await response.json();

    jsonData[s1] = s2;

    const msg = 'JSON data is available: ' +
JSON.stringify(jsonData);
    window.HybridWebView.SendRawMessage(msg)

    return jsonData;
}

async function InvokeDoSyncWork() {
    LogMessage("Invoking DoSyncWork");
    await window.HybridWebView.InvokeDotNet('DoSyncWork');
    LogMessage("Invoked DoSyncWork");
}

async function InvokeDoSyncWorkParams() {
    LogMessage("Invoking DoSyncWorkParams");
    await
window.HybridWebView.InvokeDotNet('DoSyncWorkParams', [123,
'hello']);
    LogMessage("Invoked DoSyncWorkParams");
}

async function InvokeDoSyncWorkReturn() {
    LogMessage("Invoking DoSyncWorkReturn");
    const retValue = await
window.HybridWebView.InvokeDotNet('DoSyncWorkReturn');
    LogMessage("Invoked DoSyncWorkReturn, return value: "
+ retValue);
}

async function InvokeDoSyncWorkParamsReturn() {
    LogMessage("Invoking DoSyncWorkParamsReturn");
    const retValue = await
window.HybridWebView.InvokeDotNet('DoSyncWorkParamsReturn', [123,
'hello']);
    LogMessage("Invoked DoSyncWorkParamsReturn, return
value: message=" + retValue.Message + ", value=" +
retValue.Value);
}

```

```

    }

</script>
</head>
<body>
    <div>
        Hybrid sample!
    </div>
    <div>
        <button
onclick="window.HybridWebView.SendRawMessage('Message from JS! ' +
(count++))">Send message to C#</button>
    </div>
    <div>
        <button onclick="InvokeDoSyncWork()">Call C# sync method
(no params)</button>
        <button onclick="InvokeDoSyncWorkParams()">Call C# sync
method (params)</button>
        <button onclick="InvokeDoSyncWorkReturn()">Call C# method
(no params) and get simple return value</button>
        <button onclick="InvokeDoSyncWorkParamsReturn()">Call C#
method (params) and get complex return value</button>
    </div>
    <div>
        Log: <textarea readonly id="messageLog" style="width: 80%;
height: 10em;"></textarea>
    </div>
    <div>
        Consider checking out this PDF: <a
href="docs/sample.pdf">sample.pdf</a>
    </div>
</body>
</html>

```

- *Resources\Raw\wwwroot\scripts\HybridWebView.js* with the standard [HybridWebView](#) JavaScript library:

JavaScript

```

window.HybridWebView = {
    "Init": function Init() {
        function DispatchHybridWebViewMessage(message) {
            const event = new
CustomEvent("HybridWebViewMessageReceived", { detail: { message:
message } });
            window.dispatchEvent(event);
        }

        if (window.chrome && window.chrome.webview) {
            // Windows WebView2
            window.chrome.webview.addEventListener('message', arg
=> {
                DispatchHybridWebViewMessage(arg.data);
            });
        }
    }
}

```



```

    });
  }
  else if (window.webkit && window.webkit.messageHandlers &&
window.webkit.messageHandlers.webwindowinterop) {
    // iOS and MacCatalyst WKWebView
    window.external = {
      "receiveMessage": message => {
        DispatchHybridWebViewMessage(message);
      }
    };
  }
  else {
    // Android WebView
    window.addEventListener('message', arg => {
      DispatchHybridWebViewMessage(arg.data);
    });
  }
},

"SendRawMessage": function SendRawMessage(message) {
  window.HybridWebView.__SendMessageInternal('__RawMessage',
message);
},

"InvokeDotNet": async function InvokeDotNetAsync(methodName,
paramValues) {
  const body = {
    MethodName: methodName
  };

  if (typeof paramValues !== 'undefined') {
    if (!Array.isArray(paramValues)) {
      paramValues = [paramValues];
    }

    for (var i = 0; i < paramValues.length; i++) {
      paramValues[i] = JSON.stringify(paramValues[i]);
    }

    if (paramValues.length > 0) {
      body.ParamValues = paramValues;
    }
  }

  const message = JSON.stringify(body);

  var requestUrl =
`${window.location.origin}/__hvvInvokeDotNet?
data=${encodeURIComponent(message)}`;

  const rawResponse = await fetch(requestUrl, {
    method: 'GET',
    headers: {
      'Accept': 'application/json'
    }
  })

```

```

    });
    const response = await rawResponse.json();

    if (response) {
        if (response.IsJson) {
            return JSON.parse(response.Result);
        }

        return response.Result;
    }

    return null;
},

    "__SendMessageInternal": function __SendMessageInternal(type,
message) {

        const messageToSend = type + '|' + message;

        if (window.chrome && window.chrome.webview) {
            // Windows WebView2
            window.chrome.webview.postMessage(messageToSend);
        }
        else if (window.webkit && window.webkit.messageHandlers &&
window.webkit.messageHandlers.webwindowinterop) {
            // iOS and MacCatalyst WKWebView

window.webkit.messageHandlers.webwindowinterop.postMessage(message
ToSend);
        }
        else {
            // Android WebView
            hybridWebViewHost.sendMessage(messageToSend);
        }
    },

    "__InvokeJavaScript": function __InvokeJavaScript(taskId,
methodName, args) {
        if (methodName[Symbol.toStringTag] === 'AsyncFunction') {
            // For async methods, we need to call the method and
            then trigger the callback when it's done
            const asyncPromise = methodName(...args);
            asyncPromise
                .then(asyncResult => {

window.HybridWebView.__TriggerAsyncCallback(taskId, asyncResult);
                })
                .catch(error => console.error(error));
        } else {
            // For sync methods, we can call the method and
            trigger the callback immediately
            const syncResult = methodName(...args);
            window.HybridWebView.__TriggerAsyncCallback(taskId,
syncResult);
        }
    }

```

```

    },

    "__TriggerAsyncCallback": function
__TriggerAsyncCallback(taskId, result) {
    // Make sure the result is a string
    if (result && typeof (result) !== 'string') {
        result = JSON.stringify(result);
    }

    window.HybridWebView.__SendMessageInternal('__InvokeJavaScriptCompleted', taskId + '|' + result);
}

}

window.HybridWebView.Init();

```

Then, add any additional web content to your project.

Warning

In some cases Visual Studio might add entries to the project's `.csproj` file that are incorrect. When using the default location for raw assets there shouldn't be any entries for these files or folders in the `.csproj` file.

3. Add the [HybridWebView](#) control to your app:

XAML

```

<Grid RowDefinitions="Auto,*"
      ColumnDefinitions="*">
    <Button Text="Send message to JavaScript"
           Clicked="OnSendMessageButtonClicked" />
    <HybridWebView x:Name="hybridWebView"

RawMessageReceived="OnHybridWebViewRawMessageReceived"
                  Grid.Row="1" />
</Grid>

```

4. Modify the `CreateMauiApp` method of your `MauiProgram` class to enable developer tools on the underlying WebView controls when your app is running in debug configuration. To do this, call the [AddHybridWebViewDeveloperTools](#) method on the `IServiceCollection` object:

C#

```

using Microsoft.Extensions.Logging;

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf",
"OpenSansRegular");
                fonts.AddFont("OpenSans-Semibold.ttf",
"OpenSansSemibold");
            });

#if DEBUG
        builder.Services.AddHybridWebViewDeveloperTools();
        builder.Logging.AddDebug();
#endif

        // Register any app services on the IServiceCollection object

        return builder.Build();
    }
}

```

5. Use the [HybridWebView](#) APIs to send messages between the JavaScript and C# code:

```

C#

private void OnSendMessageButtonClicked(object sender, EventArgs e)
{
    hybridWebView.SendRawMessage($"Hello from C#!");
}

private async void OnHybridWebViewRawMessageReceived(object sender,
HybridWebViewRawMessageReceivedEventArgs e)
{
    await DisplayAlert("Raw Message Received", e.Message, "OK");
}

```

The messages above are classed as raw because no additional processing is performed. You can also encode data within the message to perform more advanced messaging.

Invoke JavaScript from C#

Your app's C# code can synchronously and asynchronously invoke JavaScript methods within the [HybridWebView](#), with optional parameters and an optional return value. This can be achieved with the [InvokeJavaScriptAsync](#) and [EvaluateJavaScriptAsync](#) methods:

- The [EvaluateJavaScriptAsync](#) method runs the JavaScript code provided via a parameter and returns the result as a string.
- The [InvokeJavaScriptAsync](#) method invokes a specified JavaScript method, optionally passing in parameter values, and specifies a generic argument that indicates the type of the return value. It returns an object of the generic argument type that contains the return value of the called JavaScript method. Internally, parameters and return values are JSON encoded.

Invoke synchronous JavaScript

Synchronous JavaScript methods can be invoked with the [EvaluateJavaScriptAsync](#) and [InvokeJavaScriptAsync](#) methods. In the following example the [InvokeJavaScriptAsync](#) method is used to demonstrate invoking JavaScript that's embedded in an app's web content. For example, a simple Javascript method to add two numbers could be defined in your web content:

JavaScript

```
function AddNumbers(a, b) {  
    return a + b;  
}
```

The `AddNumbers` JavaScript method can be invoked from C# with the [InvokeJavaScriptAsync](#) method:

C#

```
double x = 123d;  
double y = 321d;  
  
double result = await hybridWebView.InvokeJavaScriptAsync<double>(   
    "AddNumbers", // JavaScript method name  
    HybridSampleJSContext.Default.Double, // JSON serialization info for  
    return type  
    [x, y], // Parameter values  
    [HybridSampleJSContext.Default.Double,  
    HybridSampleJSContext.Default.Double]); // JSON serialization info for each  
    parameter
```

The method invocation requires specifying `JsonTypeInfo` objects that include serialization information for the types used in the operation. These objects are

automatically created by including the following `partial` class in your project:

C#

```
[JsonSourceGenerationOptions(WriteIndented = true)]
[JsonSerializable(typeof(double))]
internal partial class HybridSampleJsContext : JsonSerializerContext
{
    // This type's attributes specify JSON serialization info to preserve
    // type structure
    // for trimmed builds.
}
```

Important

The `HybridSampleJsContext` class must be `partial` so that code generation can provide the implementation when the project is compiled. If the type is nested into another type, then that type must also be `partial`.

Invoke asynchronous JavaScript

Asynchronous JavaScript methods can be invoked with the `EvaluateJavaScriptAsync` and `InvokeJavaScriptAsync` methods. In the following example the `InvokeJavaScriptAsync` method is used to demonstrate invoking JavaScript that's embedded in an app's web content. For example, a Javascript method that asynchronously retrieves data could be defined in your web content:

JavaScript

```
async function EvaluateMeWithParamsAndAsyncReturn(s1, s2) {
    const response = await fetch("/asyncdata.txt");
    if (!response.ok) {
        throw new Error(`HTTP error: ${response.status}`);
    }
    var jsonData = await response.json();
    jsonData[s1] = s2;

    return jsonData;
}
```

The `EvaluateMeWithParamsAndAsyncReturn` JavaScript method can be invoked from C# with the `InvokeJavaScriptAsync` method:

C#

```
Dictionary<string, string> asyncResult = await
hybridWebView.InvokeJavaScriptAsync<Dictionary<string, string>>(
    "EvaluateMeWithParamsAndAsyncReturn", // JavaScript method name
    HybridSampleJSContext.Default.DictionaryStringString, // JSON
    serialization info for return type
    ["new_key", "new_value"], // Parameter values
    [HybridSampleJSContext.Default.String,
HybridSampleJSContext.Default.String]); // JSON serialization info for each
parameter
```

In this example, `asyncResult` is a `Dictionary<string, string>` that contains the JSON data from the web request.

The method invocation requires specifying `JsonTypeInfo` objects that include serialization information for the types used in the operation. These objects are automatically created by including the following `partial` class in your project:

C#

```
[JsonSourceGenerationOptions(WriteIndented = true)]
[JsonSerializable(typeof(Dictionary<string, string>))]
[JsonSerializable(typeof(string))]
internal partial class HybridSampleJSContext : JsonSerializerContext
{
    // This type's attributes specify JSON serialization info to preserve
    // type structure
    // for trimmed builds.
}
```

Important

The `HybridSampleJsContext` class must be `partial` so that code generation can provide the implementation when the project is compiled. If the type is nested into another type, then that type must also be `partial`.

Invoke C# from JavaScript

Your app's JavaScript code within the `HybridWebView` can synchronously invoke C# methods, with optional parameters and an optional return value. This can be achieved by:

- Defining public C# methods that will be invoked from JavaScript.

- Calling the [SetInvokeJavaScriptTarget](#) method to set the object that will be the target of JavaScript calls from the [HybridWebView](#).
- Calling the C# methods from JavaScript.

Important

Asynchronously invoking C# methods from JavaScript isn't currently supported.

The following example defines four public methods for invoking from JavaScript:

```
C#

public partial class MainPage : ContentPage
{
    ...

    public void DoSyncWork()
    {
        Debug.WriteLine("DoSyncWork");
    }

    public void DoSyncWorkParams(int i, string s)
    {
        Debug.WriteLine($"DoSyncWorkParams: {i}, {s}");
    }

    public string DoSyncWorkReturn()
    {
        Debug.WriteLine("DoSyncWorkReturn");
        return "Hello from C#!";
    }

    public SyncReturn DoSyncWorkParamsReturn(int i, string s)
    {
        Debug.WriteLine($"DoSyncWorkParamReturn: {i}, {s}");
        return new SyncReturn
        {
            Message = "Hello from C#!" + s,
            Value = i
        };
    }

    public class SyncReturn
    {
        public string? Message { get; set; }
        public int Value { get; set; }
    }
}
```


You must then call the [SetInvokeJavaScriptTarget](#) method to set the object that will be the target of JavaScript calls from the [HybridWebView](#):

C#

```
public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();
        hybridWebView.SetInvokeJavaScriptTarget(this);
    }

    ...
}
```

The public methods on the object set via the [SetInvokeJavaScriptTarget](#) method can then be invoked from JavaScript with the `window.HybridWebView.InvokeDotNet` function:

JavaScript

```
await window.HybridWebView.InvokeDotNet('DoSyncWork');
await window.HybridWebView.InvokeDotNet('DoSyncWorkParams', [123, 'hello']);
const retValue = await
window.HybridWebView.InvokeDotNet('DoSyncWorkReturn');
const retValue = await
window.HybridWebView.InvokeDotNet('DoSyncWorkParamsReturn', [123, 'hello']);
```

The `window.HybridWebView.InvokeDotNet` JavaScript function invokes a specified C# method, with optional parameters and an optional return value.

ⓘ Note

Invoking the `window.HybridWebView.InvokeDotNet` JavaScript function requires your app to include the *HybridWebView.js* JavaScript library listed earlier in this article.

Image

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [Image](#) displays an image that can be loaded from a local file, a URI, or a stream. The standard platform image formats are supported, including animated GIFs, and local Scalable Vector Graphics (SVG) files are also supported. For more information about adding images to a .NET MAUI app project, see [Add images to a .NET MAUI app project](#).

[Image](#) defines the following properties:

- `Aspect`, of type `Aspect`, defines the scaling mode of the image.
- `IsAnimationPlaying`, of type `bool`, determines whether an animated GIF is playing or stopped. The default value of this property is `false`.
- `IsLoading`, of type `bool`, indicates the loading status of the image. The default value of this property is `false`.
- `IsOpaque`, of type `bool`, indicates whether the rendering engine may treat the image as opaque while rendering it. The default value of this property is `false`.
- `Source`, of type [ImageSource](#), specifies the source of the image.

These properties are backed by [BindableProperty](#) objects, which means that they can be styled, and be the target of data bindings.

ⓘ Note

Font icons can be displayed by an [Image](#) by specifying the font icon data as a [FontImageSource](#) object. For more information, see [Display font icons](#).

The [ImageSource](#) class defines the following methods that can be used to load an image from different sources:

- `FromFile` returns a `FileImageSource` that reads an image from a local file.
- `FromUri` returns an `UriImageSource` that downloads and reads an image from a specified URI.
- `FromStream` returns a `StreamImageSource` that reads an image from a stream that supplies image data.

In XAML, images can be loaded from files and URIs by specifying the filename or URI as a string value for the `Source` property. Images can also be loaded from streams in XAML through custom markup extensions.

Important

Images will be displayed at their full resolution unless the size of the [Image](#) is constrained by its layout, or the [HeightRequest](#) or [WidthRequest](#) property of the [Image](#) is specified.

For information about adding app icons and a splash screen to your app, see [App icons](#) and [Splash screen](#).

Load a local image

Images can be added to your app project by dragging them to the *Resources\Images* folder of your project, where its build action will automatically be set to **MauiImage**. At build time, vector images are resized to the correct resolutions for the target platform and device, and added to your app package. This is necessary because different platforms support different image resolutions, and the operating system chooses the appropriate image resolution at runtime based on the device's capabilities.

To comply with Android resource naming rules, all local image filenames must be lowercase, start and end with a letter character, and contain only alphanumeric characters or underscores. For more information, see [App resources overview](#)  on developer.android.com.

Important

.NET MAUI converts SVG files to PNG files. Therefore, when adding an SVG file to your .NET MAUI app project, it should be referenced from XAML or C# with a .png extension.

Adhering to these rules for file naming and placement enables the following XAML to load and display an image:

XAML

```
<Image Source="dotnet_bot.png" />
```

The equivalent C# code is:

C#

```
Image image = new Image
{
    Source = ImageSource.FromFile("dotnet_bot.png")
};
```

The `ImageSource.FromFile` method requires a `string` argument, and returns a new `FileImageSource` object that reads the image from the file. There's also an implicit conversion operator that enables the filename to be specified as a `string` argument to the `Image.Source` property:

C#

```
Image image = new Image { Source = "dotnet_bot.png" };
```

Load a remote image

Remote images can be downloaded and displayed by specifying a URI as the value of the `Source` property:

XAML

```
<Image Source="https://aka.ms/campus.jpg" />
```

The equivalent C# code is:

C#

```
Image image = new Image
{
    Source = ImageSource.FromUri(new Uri("https://aka.ms/campus.jpg"))
};
```

The `ImageSource.FromUri` method requires a `Uri` argument, and returns a new `UriImageSource` object that reads the image from the `Uri`. There's also an implicit conversion for string-based URIs:

C#

```
Image image = new Image { Source = "https://aka.ms/campus.jpg" };
```

Image caching

Caching of downloaded images is enabled by default, with cached images being stored for 1 day. This behavior can be changed by setting properties of the `UriImageSource` class.

The `UriImageSource` class defines the following properties:

- `Uri`, of type `Uri`, represents the URL of the image to be downloaded for display.
- `CacheValidity`, of type `TimeSpan`, specifies how long the image will be stored locally for. The default value of this property is 1 day.
- `CachingEnabled`, of type `bool`, defines whether image caching is enabled. The default value of this property is `true`.

These properties are backed by `BindableProperty` objects, which means that they can be styled, and be the target of data bindings.

To set a specific cache period, set the `Source` property to an `UriImageSource` object that sets its `CacheValidity` property:

XAML

```
<Image>
  <Image.Source>
    <UriImageSource Uri="https://aka.ms/campus.jpg"
                    CacheValidity="10:00:00:00" />
  </Image.Source>
</Image>
```

The equivalent C# code is:

C#

```
Image image = new Image();
image.Source = new UriImageSource
{
    Uri = new Uri("https://aka.ms/campus.jpg"),
    CacheValidity = new TimeSpan(10,0,0,0)
};
```

In this example, the caching period is set to 10 days.

Load an image from a stream

Images can be loaded from streams with the `ImageSource.FromStream` method:

C#

```
Image image = new Image
{
    Source = ImageSource.FromStream(() => stream)
};
```

❗ Important

Image caching is disabled on Android when loading an image from a stream with the [ImageSource.FromStream](#) method. This is due to the lack of data from which to create a reasonable cache key.

Load a font icon

The [FontImage](#) markup extension enables you to display a font icon in any view that can display an [ImageSource](#). It provides the same functionality as the [FontImageSource](#) class, but with a more concise representation.

The [FontImage](#) markup extension is supported by the [FontImageExtension](#) class, which defines the following properties:

- `FontFamily` of type `string`, the font family to which the font icon belongs.
- `Glyph` of type `string`, the unicode character value of the font icon.
- `Color` of type [Color](#), the color to be used when displaying the font icon.
- `Size` of type `double`, the size, in device-independent units, of the rendered font icon. The default value is 30. In addition, this property can be set to a named font size.

❗ Note

The XAML parser allows the [FontImageExtension](#) class to be abbreviated as `FontImage`.

The `Glyph` property is the content property of [FontImageExtension](#). Therefore, for XAML markup expressions expressed with curly braces, you can eliminate the `Glyph=` part of the expression provided that it's the first argument.

The following XAML example shows how to use the [FontImage](#) markup extension:

XAML

```
<Image BackgroundColor="#D1D1D1"
        Source="{FontImage &#xf30c;, FontFamily=Ionicons, Size=44}" />
```

In this example, the abbreviated version of the [FontImageExtension](#) class name is used to display an Xbox icon, from the Ionicons font family, in an [Image](#):



While the unicode character for the icon is `\uf30c`, it has to be escaped in XAML and so becomes ``.

For information about displaying font icons by specifying the font icon data in a [FontImageSource](#) object, see [Display font icons](#).

Load animated GIFs

.NET MAUI includes support for displaying small, animated GIFs. This is accomplished by setting the `Source` property to an animated GIF file:

XAML

```
<Image Source="demo.gif" />
```

Important

While the animated GIF support in .NET MAUI includes the ability to download files, it does not support caching or streaming animated GIFs.

By default, when an animated GIF is loaded it will not be played. This is because the `IsAnimationPlaying` property, that controls whether an animated GIF is playing or stopped, has a default value of `false`. Therefore, when an animated GIF is loaded it will not be played until the `IsAnimationPlaying` property is set to `true`. Playback can be stopped by resetting the `IsAnimationPlaying` property to `false`. Note that this property has no effect when displaying a non-GIF image source.

Control image scaling

The `Aspect` property determines how the image will be scaled to fit the display area, and should be set to one of the members of the `Aspect` enumeration:

- `AspectFit` - letterboxes the image (if required) so that the entire image fits into the display area, with blank space added to the top/bottom or sides depending on whether the image is wide or tall.
- `AspectFill` - clips the image so that it fills the display area while preserving the aspect ratio.
- `Fill` - stretches the image to completely and exactly fill the display area. This may result in the image being distorted.
- `Center` - centers the image in the display area while preserving the aspect ratio.

ImageButton

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [ImageButton](#) view combines the [Button](#) view and [Image](#) view to create a button whose content is an image. When you press the [ImageButton](#) with a finger or click it with a mouse, it directs the app to carry out a task. However, unlike the [Button](#) the [ImageButton](#) view has no concept of text and text appearance.

[ImageButton](#) defines the following properties:

- `Aspect`, of type `Aspect`, determines how the image is scaled to fit the display area.
- `BorderColor`, of type `Color`, describes the border color of the button.
- `BorderWidth`, of type `double`, defines the width of the button's border.
- `Command`, of type `ICommand`, defines the command that's executed when the button is tapped.
- `CommandParameter`, of type `object`, is the parameter that's passed to `Command`.
- `CornerRadius`, of type `int`, describes the corner radius of the button's border.
- `IsLoading`, of type `bool`, represents the loading status of the image. The default value of this property is `false`.
- `IsOpaque`, of type `bool`, determines whether .NET MAUI should treat the image as opaque when rendering it. The default value of this property is `false`.
- `IsPressed`, of type `bool`, represents whether the button is being pressed. The default value of this property is `false`.
- `Padding`, of type `Thickness`, determines the button's padding.
- `Source`, of type `ImageSource`, specifies an image to display as the content of the button.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The `Aspect` property can be set to one of the members of the `Aspect` enumeration:

- `Fill` - stretches the image to completely and exactly fill the [ImageButton](#). This may result in the image being distorted.
- `AspectFill` - clips the image so that it fills the [ImageButton](#) while preserving the aspect ratio.
- `AspectFit` - letterboxes the image (if necessary) so that the entire image fits into the [ImageButton](#), with blank space added to the top/bottom or sides depending

on whether the image is wide or tall. This is the default value of the `Aspect` enumeration.

- `Center` - centers the image in the `ImageButton` while preserving the aspect ratio.

In addition, `ImageButton` defines `Clicked`, `Pressed`, and `Released` events. The `Clicked` event is raised when an `ImageButton` tap with a finger or mouse pointer is released from the button's surface. The `Pressed` event is raised when a finger presses on an `ImageButton`, or a mouse button is pressed with the pointer positioned over the `ImageButton`. The `Released` event is raised when the finger or mouse button is released. Generally, a `Clicked` event is also raised at the same time as the `Released` event, but if the finger or mouse pointer slides away from the surface of the `ImageButton` before being released, the `Clicked` event might not occur.

❗ Important

An `ImageButton` must have its `IsEnabled` property set to `true` for it to respond to taps.

Create an ImageButton

To create an image button, create an `ImageButton` object, set its `Source` property and handle its `Clicked` event.

The following XAML example shows how to create an `ImageButton`:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="ControlGallery.Views.XAML.ImageButtonDemoPage"
    Title="ImageButton Demo">
    <StackLayout>
        <ImageButton Source="image.png"
            Clicked="OnImageButtonClicked"
            HorizontalOptions="Center"
            VerticalOptions="Center" />
    </StackLayout>
</ContentPage>
```

The `Source` property specifies the image that appears in the `ImageButton`. The `Clicked` event is set to an event handler named `OnImageButtonClicked`. This handler is located in the code-behind file:

C#

```
public partial class ImageButtonDemoPage : ContentPage
{
    int clickTotal;

    public ImageButtonDemoPage()
    {
        InitializeComponent();
    }

    void OnImageButtonClicked(object sender, EventArgs e)
    {
        clickTotal += 1;
        label.Text = $"{clickTotal} ImageButton click{(clickTotal == 1 ? ""
: "s")}";
    }
}
```

In this example, when the [ImageButton](#) is tapped, the `OnImageButtonClicked` method executes. The `sender` argument is the [ImageButton](#) responsible for this event. You can use this to access the [ImageButton](#) object, or to distinguish between multiple [ImageButton](#) objects sharing the same `Clicked` event. The `Clicked` handler increments a counter and displays the counter value in a [Label](#):



The equivalent C# code to create an [ImageButton](#) is:

C#

```
Label label;
int clickTotal = 0;
...

ImageButton imageButton = new ImageButton
{
```

```

        Source = "XamarinLogo.png",
        HorizontalOptions = LayoutOptions.Center,
        VerticalOptions = LayoutOptions.CenterAndExpand
    };
    imageButton.Clicked += (s, e) =>
    {
        clickTotal += 1;
        label.Text = $"{clickTotal} ImageButton click{(clickTotal == 1 ? "" :
"s")}";
    };
};

```

Use the command interface

An app can respond to [ImageButton](#) taps without handling the `Clicked` event. The [ImageButton](#) implements an alternative notification mechanism called the *command* or *commanding* interface. This consists of two properties:

- `Command` of type [ICommand](#), an interface defined in the [System.Windows.Input](#) namespace.
- `CommandParameter` property of type [Object](#).

This approach is suitable in connection with data-binding, and particularly when implementing the Model-View-ViewModel (MVVM) pattern. For more information about commanding, see [Use the command interface](#) in the [Button](#) article.

Press and release an ImageButton

The `Pressed` event is raised when a finger presses on a [ImageButton](#), or a mouse button is pressed with the pointer positioned over the [ImageButton](#). The `Released` event is raised when the finger or mouse button is released. Generally, the `Clicked` event is also raised at the same time as the `Released` event, but if the finger or mouse pointer slides away from the surface of the [ImageButton](#) before being released, the `Clicked` event might not occur.

For more information about these events, see [Press and release the button](#) in the [Button](#) article.

ImageButton visual states

[ImageButton](#) has a `Pressed` [VisualState](#) that can be used to initiate a visual change to the [ImageButton](#) when pressed, if it's enabled.

The following XAML example shows how to define a visual state for the `Pressed` state:

XAML

```
<ImageButton Source="image.png"
    ...>
    <VisualStateManager.VisualStateGroups>
        <VisualStateGroup x:Name="CommonStates">
            <VisualState x:Name="Normal">
                <VisualState.Setters>
                    <Setter Property="Scale"
                        Value="1" />
                </VisualState.Setters>
            </VisualState>
            <VisualState x:Name="Pressed">
                <VisualState.Setters>
                    <Setter Property="Scale"
                        Value="0.8" />
                </VisualState.Setters>
            </VisualState>
            <VisualState x:Name="PointerOver" />
        </VisualStateGroup>
    </VisualStateManager.VisualStateGroups>
</ImageButton>
```

In this example, the `Pressed` `VisualState` specifies that when the `ImageButton` is pressed, its `Scale` property will be changed from its default value of 1 to 0.8. The `Normal` `VisualState` specifies that when the `ImageButton` is in a normal state, its `Scale` property will be set to 1. Therefore, the overall effect is that when the `ImageButton` is pressed, it's rescaled to be slightly smaller, and when the `ImageButton` is released, it's rescaled to its default size.

Important

For an `ImageButton` to return to its `Normal` state the `VisualStateGroup` must also define a `PointerOver` state. If you use the styles `ResourceDictionary` created by the .NET MAUI app project template, you'll already have an implicit `ImageButton` style that defines the `PointerOver` state.

For more information about visual states, see [Visual states](#).

Disable an ImageButton

Sometimes an app enters a state where an `ImageButton` click is not a valid operation. In those cases, the `ImageButton` should be disabled by setting its `IsEnabled` property to

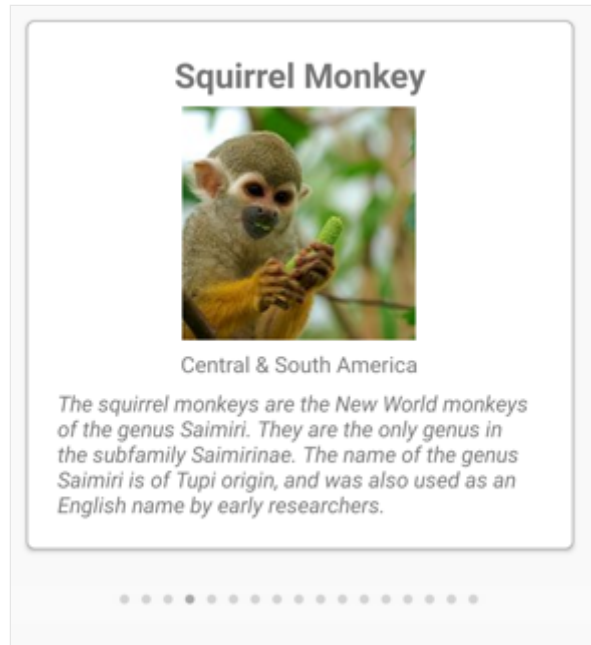
false.

IndicatorView

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [IndicatorView](#) is a control that displays indicators that represent the number of items, and current position, in a [CarouselView](#):



[IndicatorView](#) defines the following properties:

- `Count`, of type `int`, the number of indicators.
- `HideSingle`, of type `bool`, indicates whether the indicator should be hidden when only one exists. The default value is `true`.
- `IndicatorColor`, of type `Color`, the color of the indicators.
- `IndicatorSize`, of type `double`, the size of the indicators. The default value is 6.0.
- `IndicatorLayout`, of type `Layout<View>`, defines the layout class used to render the [IndicatorView](#). This property is set by .NET MAUI, and does not typically need to be set by developers.
- `IndicatorTemplate`, of type `DataTemplate`, the template that defines the appearance of each indicator.
- `IndicatorsShape`, of type `IndicatorShape`, the shape of each indicator.
- `ItemsSource`, of type `IEnumerable`, the collection that indicators will be displayed for. This property will automatically be set when the `CarouselView.IndicatorView` property is set.
- `MaximumVisible`, of type `int`, the maximum number of visible indicators. The default value is `int.MaxValue`.

- `Position`, of type `int`, the currently selected indicator index. This property uses a `TwoWay` binding. This property will automatically be set when the `CarouselView.IndicatorView` property is set.
- `SelectedIndicatorColor`, of type `Color`, the color of the indicator that represents the current item in the `CarouselView`.

These properties are backed by `BindableProperty` objects, which means that they can be targets of data bindings, and styled.

Create an IndicatorView

To add indicators to a page, create an `IndicatorView` object and set its `IndicatorColor` and `SelectedIndicatorColor` properties. In addition, set the `CarouselView.IndicatorView` property to the name of the `IndicatorView` object.

The following example shows how to create an `IndicatorView` in XAML:

XAML

```
<Grid RowDefinitions="*,Auto">
    <CarouselView ItemsSource="{Binding Monkeys}"
        IndicatorView="indicatorView">
        <CarouselView.ItemTemplate>
            <!-- DataTemplate that defines item appearance -->
        </CarouselView.ItemTemplate>
    </CarouselView>
    <IndicatorView x:Name="indicatorView"
        Grid.Row="1"
        IndicatorColor="LightGray"
        SelectedIndicatorColor="DarkGray"
        HorizontalOptions="Center" />
</Grid>
```

In this example, the `IndicatorView` is rendered beneath the `CarouselView`, with an indicator for each item in the `CarouselView`. The `IndicatorView` is populated with data by setting the `CarouselView.IndicatorView` property to the `IndicatorView` object. Each indicator is a light gray circle, while the indicator that represents the current item in the `CarouselView` is dark gray.

Important

Setting the `CarouselView.IndicatorView` property results in the `IndicatorView.Position` property binding to the `CarouselView.Position` property,

and the `IndicatorView.ItemsSource` property binding to the `CarouselView.ItemsSource` property.

Change indicator shape

The `IndicatorView` class has an `IndicatorsShape` property, which determines the shape of the indicators. This property can be set to one of the `IndicatorShape` enumeration members:

- `Circle` specifies that the indicator shapes will be circular. This is the default value of the `IndicatorView.IndicatorsShape` property.
- `Square` indicates that the indicator shapes will be square.

The following example shows an `IndicatorView` configured to use square indicators:

XAML

```
<IndicatorView x:Name="indicatorView"
    IndicatorsShape="Square"
    IndicatorColor="LightGray"
    SelectedIndicatorColor="DarkGray" />
```

Change indicator size

The `IndicatorView` class has an `IndicatorSize` property, of type `double`, which determines the size of the indicators in device-independent units. The default value of this property is 6.0.

The following example shows an `IndicatorView` configured to display larger indicators:

XAML

```
<IndicatorView x:Name="indicatorView"
    IndicatorSize="18" />
```

Limit the number of indicators displayed

The `IndicatorView` class has a `MaximumVisible` property, of type `int`, which determines the maximum number of visible indicators.

The following example shows an [IndicatorView](#) configured to display a maximum of six indicators:

XAML

```
<IndicatorView x:Name="indicatorView"
              MaximumVisible="6" />
```

Define indicator appearance

The appearance of each indicator can be defined by setting the `IndicatorView.IndicatorTemplate` property to a [DataTemplate](#):

XAML

```
<Grid RowDefinitions="*,Auto">
    <CarouselView ItemsSource="{Binding Monkeys}"
                  IndicatorView="indicatorView">
        <CarouselView.ItemTemplate>
            <!-- DataTemplate that defines item appearance -->
        </CarouselView.ItemTemplate>
    </CarouselView>
    <IndicatorView x:Name="indicatorView"
                  Grid.Row="1"
                  Margin="0,0,0,40"
                  IndicatorColor="Transparent"
                  SelectedIndicatorColor="Transparent"
                  HorizontalOptions="Center">
        <IndicatorView.IndicatorTemplate>
            <DataTemplate>
                <Label Text="&#xf30c;"
                      FontFamily="ionicons"
                      FontSize="12" />
            </DataTemplate>
        </IndicatorView.IndicatorTemplate>
    </IndicatorView>
</Grid>
```

The elements specified in the [DataTemplate](#) define the appearance of each indicator. In this example, each indicator is a [Label](#) that displays a font icon.

The following screenshot shows indicators rendered using a font icon:



Set visual states

[IndicatorView](#) has a `Selected` visual state that can be used to initiate a visual change to the indicator for the current position in the [IndicatorView](#). A common use case for this [VisualState](#) is to change the color of the indicator that represents the current position:

XAML

```
<ContentPage ...>
  <ContentPage.Resources>
    <Style x:Key="IndicatorLabelStyle"
      TargetType="Label">
      <Setter Property="VisualStateManager.VisualStateGroups">
        <VisualStateGroupList>
          <VisualStateGroup x:Name="CommonStates">
            <VisualState x:Name="Normal">
              <VisualState.Setters>
                <Setter Property="TextColor"
                  Value="LightGray" />
              </VisualState.Setters>
            </VisualState>
            <VisualState x:Name="Selected">
              <VisualState.Setters>
                <Setter Property="TextColor"
                  Value="Black" />
              </VisualState.Setters>
            </VisualState>
          </VisualStateGroup>
        </VisualStateGroupList>
      </Setter>
    </Style>
  </ContentPage.Resources>
```

```

<Grid RowDefinitions="*,Auto">
    ...
    <IndicatorView x:Name="indicatorView"
        Grid.Row="1"
        Margin="0,0,0,40"
        IndicatorColor="Transparent"
        SelectedIndicatorColor="Transparent"
        HorizontalOptions="Center">
        <IndicatorView.IndicatorTemplate>
            <DataTemplate>
                <Label Text="&#xf30c;"
                    FontFamily="ionicons"
                    FontSize="12"
                    Style="{StaticResource IndicatorLabelStyle}" />
            </DataTemplate>
        </IndicatorView.IndicatorTemplate>
    </IndicatorView>
</Grid>
</ContentPage>

```

In this example, the `Selected` visual state specifies that the indicator that represents the current position will have its `TextColor` set to black. Otherwise the `TextColor` of the indicator will be light gray:



For more information about visual states, see [Visual states](#).

Label

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [Label](#) displays single-line and multi-line text. Text displayed by a [Label](#) can be colored, spaced, and can have text decorations.

[Label](#) defines the following properties:

- `CharacterSpacing`, of type `double`, sets the spacing between characters in the displayed text.
- `FontAttributes`, of type `FontAttributes`, determines text style.
- `FontAutoScalingEnabled`, of type `bool`, defines whether the text will reflect scaling preferences set in the operating system. The default value of this property is `true`.
- `FontFamily`, of type `string`, defines the font family.
- `FontSize`, of type `double`, defines the font size.
- `FormattedText`, of type `FormattedString`, specifies the presentation of text with multiple presentation options such as fonts and colors.
- `HorizontalTextAlignment`, of type [TextAlignment](#), defines the horizontal alignment of the displayed text.
- `LineBreakMode`, of type `LineBreakMode`, determines how text should be handled when it can't fit on one line.
- `LineHeight`, of type `double`, specifies the multiplier to apply to the default line height when displaying text.
- `MaxLines`, of type `int`, indicates the maximum number of lines allowed in the [Label](#).
- `Padding`, of type `Thickness`, determines the label's padding.
- `Text`, of type `string`, defines the text displayed as the content of the label.
- `TextColor`, of type [Color](#), defines the color of the displayed text.
- `TextDecorations`, of type `TextDecorations`, specifies the text decorations (underline and strikethrough) that can be applied.
- `TextTransform`, of type `TextTransform`, specifies the casing of the displayed text.
- `TextType`, of type `TextType`, determines whether the [Label](#) should display plain text or HTML text.
- `VerticalTextAlignment`, of type [TextAlignment](#), defines the vertical alignment of the displayed text.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

For information about specifying fonts on a [Label](#), see [Fonts](#).

Create a Label

The following example shows how to create a [Label](#):

XAML

```
<Label Text="Hello world" />
```

The equivalent C# code is:

C#

```
Label label = new Label { Text = "Hello world" };
```

Set colors

Labels can be set to use a specific text color via the `TextColor` property.

The following example sets the text color of a [Label](#):

XAML

```
<Label TextColor="#77d065"  
      Text="This is a green label." />
```

For more information about colors, see [Colors](#).

Set character spacing

Character spacing can be applied to [Label](#) objects by setting the `CharacterSpacing` property to a `double` value:

XAML

```
<Label Text="Character spaced text"  
      CharacterSpacing="10" />
```

The result is that characters in the text displayed by the `Label` are spaced `CharacterSpacing` device-independent units apart.

Add new lines

There are two main techniques for forcing text in a `Label` onto a new line, from XAML:

1. Use the unicode line feed character, which is "
".
2. Specify your text using *property element* syntax.

The following code shows an example of both techniques:

XAML

```
<!-- Unicode line feed character -->
<Label Text="First line &#10; Second line" />

<!-- Property element syntax -->
<Label>
    <Label.Text>
        First line
        Second line
    </Label.Text>
</Label>
```

In C#, text can be forced onto a new line with the "\n" character:

C#

```
Label label = new Label { Text = "First line\nSecond line" };
```

Control text truncation and wrapping

Text wrapping and truncation can be controlled by setting the `LineBreakMode` property to a value of the `LineBreakMode` enumeration:

- `NoWrap` — does not wrap text, displaying only as much text as can fit on one line. This is the default value of the `LineBreakMode` property.
- `WordWrap` — wraps text at the word boundary.
- `CharacterWrap` — wraps text onto a new line at a character boundary.
- `HeadTruncation` — truncates the head of the text, showing the end.
- `MiddleTruncation` — displays the beginning and end of the text, with the middle replace by an ellipsis.

- `TailTruncation` — shows the beginning of the text, truncating the end.

Display a specific number of lines

The number of lines displayed by a `Label` can be specified by setting the `MaxLines` property to an `int` value:

- When `MaxLines` is -1, which is its default value, the `Label` respects the value of the `LineBreakMode` property to either show just one line, possibly truncated, or all lines with all text.
- When `MaxLines` is 0, the `Label` isn't displayed.
- When `MaxLines` is 1, the result is identical to setting the `LineBreakMode` property to `NoWrap`, `HeadTruncation`, `MiddleTruncation`, or `TailTruncation`. However, the `Label` will respect the value of the `LineBreakMode` property with regard to placement of an ellipsis, if applicable.
- When `MaxLines` is greater than 1, the `Label` will display up to the specified number of lines, while respecting the value of the `LineBreakMode` property with regard to placement of an ellipsis, if applicable. However, setting the `MaxLines` property to a value greater than 1 has no effect if the `LineBreakMode` property is set to `NoWrap`.

The following XAML example demonstrates setting the `MaxLines` property on a `Label`:

XAML

```
<Label Text="Lorem ipsum dolor sit amet, consectetur adipiscing elit. In  
facilisis nulla eu felis fringilla vulputate. Nullam porta eleifend lacinia.  
Donec at iaculis tellus."  
    LineBreakMode="WordWrap"  
    MaxLines="2" />
```

Set line height

The vertical height of a `Label` can be customized by setting the `Label.LineHeight` property to a `double` value.

❗ Note

- On iOS, the `Label.LineHeight` property changes the line height of text that fits on a single line, and text that wraps onto multiple lines.

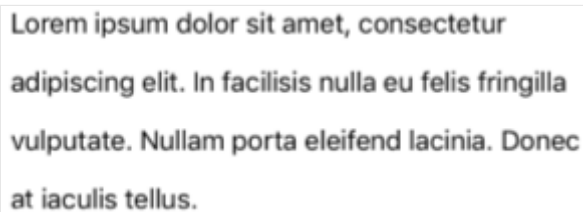
- On Android, the `Label.LineHeight` property only changes the line height of text that wraps onto multiple lines.
- On Windows, the `Label.LineHeight` property changes the line height of text that wraps onto multiple lines.

The following example demonstrates setting the `LineHeight` property on a [Label](#):

XAML

```
<Label Text="Lorem ipsum dolor sit amet, consectetur adipiscing elit. In  
facilisis nulla eu felis fringilla vulputate. Nullam porta eleifend lacinia.  
Donec at iaculis tellus."  
    LineBreakMode="WordWrap"  
    LineHeight="1.8" />
```

The following screenshot shows the result of setting the `Label.LineHeight` property to 1.8:

A screenshot of a UI element, likely a Label, showing text wrapped onto four lines. The text is: "Lorem ipsum dolor sit amet, consectetur adipiscing elit. In facilisis nulla eu felis fringilla vulputate. Nullam porta eleifend lacinia. Donec at iaculis tellus." The text is left-aligned and the line height is visibly increased, consistent with the example provided.

Display HTML

❗ Important

Displaying HTML in a [Label](#) is limited to the HTML tags that are supported by the underlying platform. For example, Android supports only a subset of HTML tags, focusing on basic styling and formatting for block level elements such as `<span>` and `<p>`. For more complex HTML rendering, consider using a [WebView](#) or `FormattedText`.

The [Label](#) class has a `TextType` property, which determines whether the [Label](#) object should display plain text, or HTML text. This property should be set to one of the members of the `TextType` enumeration:

- `Text` indicates that the [Label](#) will display plain text, and is the default value of the `TextType` property.
- `Html` indicates that the [Label](#) will display HTML text.

Therefore, `Label` objects can display HTML by setting the `TextType` property to `Html`, and the `Text` property to a HTML string:

C#

```
Label label = new Label
{
    Text = "This is <span style=\"color:red;\"><strong>HTML</strong></span>
    text.",
    TextType = TextType.Html
};
```

In the example above, the double quote characters in the HTML have to be escaped using the `\` symbol.

In XAML, HTML strings can become unreadable due to additionally escaping the `<` and `>` symbols:

XAML

```
<Label Text="This is &lt;span
style=&quot;color:red&quot;&gt;&lt;strong&gt;HTML&lt;/strong&gt;&lt;/span&gt;
; text."
    TextType="Html" />
```

Alternatively, for greater readability the HTML can be inlined in a `CDATA` section:

XAML

```
<Label TextType="Html">
    <![CDATA[
        <Label Text="This is &lt;span
style=&quot;color:red&quot;&gt;&lt;strong&gt;HTML&lt;/strong&gt;&lt;/span&gt;
; text."
        ]]>
</Label>
```

In this example, the `Text` property is set to the HTML string that's inlined in the `CDATA` section. This works because the `Text` property is the `ContentProperty` for the `Label` class.

Decorate text

Underline and strikethrough text decorations can be applied to `Label` objects by setting the `TextDecorations` property to one or more `TextDecorations` enumeration members:

- None
- Underline
- Strikethrough

The following example demonstrates setting the `TextDecorations` property:

XAML

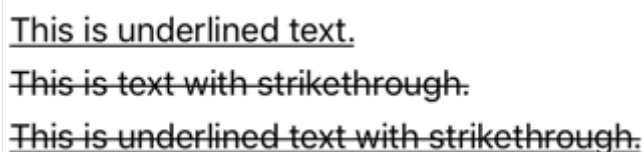
```
<Label Text="This is underlined text." TextDecorations="Underline" />
<Label Text="This is text with strikethrough."
TextDecorations="Strikethrough" />
<Label Text="This is underlined text with strikethrough."
TextDecorations="Underline, Strikethrough" />
```

The equivalent C# code is:

C#

```
Label underlineLabel = new Label { Text = "This is underlined text.",
TextDecorations = TextDecorations.Underline };
Label strikethroughLabel = new Label { Text = "This is text with
strikethrough.", TextDecorations = TextDecorations.Strikethrough };
Label bothLabel = new Label { Text = "This is underlined text with
strikethrough.", TextDecorations = TextDecorations.Underline |
TextDecorations.Strikethrough };
```

The following screenshot shows the `TextDecorations` enumeration members applied to `Label` instances:



This is underlined text.
~~This is text with strikethrough.~~
~~This is underlined text with strikethrough.~~

ⓘ Note

Text decorations can also be applied to `Span` instances. For more information about the `Span` class, see [Use formatted text](#).

Transform text

A `Label` can transform the casing of its text, stored in the `Text` property, by setting the `TextTransform` property to a value of the `TextTransform` enumeration. This enumeration

has four values:

- `None` indicates that the text won't be transformed.
- `Default` indicates that the default behavior for the platform will be used. This is the default value of the `TextTransform` property.
- `Lowercase` indicates that the text will be transformed to lowercase.
- `Uppercase` indicates that the text will be transformed to uppercase.

The following example shows transforming text to uppercase:

XAML

```
<Label Text="This text will be displayed in uppercase."
      TextTransform="Uppercase" />
```

Use formatted text

`Label` exposes a `FormattedText` property that allows the presentation of text with multiple fonts and colors in the same view. The `FormattedText` property is of type `FormattedString`, which comprises one or more `Span` instances, set via the `Spans` property.

ⓘ Note

It's not possible to display HTML in a `Span`.

`Span` defines the following properties:

- `BackgroundColor`, of type `Color`, which represents the color of the span background.
- `CharacterSpacing`, of type `double`, sets the spacing between characters in the displayed text.
- `FontAttributes`, of type `FontAttributes`, determines text style.
- `FontAutoScalingEnabled`, of type `bool`, defines whether the text will reflect scaling preferences set in the operating system. The default value of this property is `true`.
- `FontFamily`, of type `string`, defines the font family.
- `FontSize`, of type `double`, defines the font size.
- `LineHeight`, of type `double`, specifies the multiplier to apply to the default line height when displaying text.
- `Style`, of type `Style`, which is the style to apply to the span.

- `Text`, of type `string`, defines the text displayed as the content of the `Span`.
- `TextColor`, of type `Color`, defines the color of the displayed text.
- `TextDecorations`, of type `TextDecorations`, specifies the text decorations (underline and strikethrough) that can be applied.
- `TextTransform`, of type `TextTransform`, specifies the casing of the displayed text.

These properties are backed by `BindableProperty` objects, which means that they can be targets of data bindings, and styled.

❗ Note

The `Span.LineHeight` property has no effect on Windows.

In addition, the `GestureRecognizers` property can be used to define a collection of gesture recognizers that will respond to gestures on the `Span`.

The following XAML example demonstrates a `FormattedText` property that consists of three `Span` instances:

XAML

```
<Label LineBreakMode="WordWrap">
    <Label.FormattedText>
        <FormattedString>
            <Span Text="Red Bold, " TextColor="Red" FontAttributes="Bold" />
            <Span Text="default, " FontSize="14">
                <Span.GestureRecognizers>
                    <TapGestureRecognizer Command="{Binding TapCommand}" />
                </Span.GestureRecognizers>
            </Span>
            <Span Text="italic small." FontAttributes="Italic" FontSize="12"
        />
    </FormattedString>
</Label.FormattedText>
</Label>
```

The equivalent C# code is:

C#

```
FormattedString formattedString = new FormattedString ();
formattedString.Spans.Add (new Span { Text = "Red bold, ", TextColor =
Colors.Red, FontAttributes = FontAttributes.Bold });

Span span = new Span { Text = "default, " };
span.GestureRecognizers.Add(new TapGestureRecognizer { Command = new
Command(async () => await DisplayAlert("Tapped", "This is a tapped Span."),
```

```
"OK")) });
formattedString.Spans.Add(span);
formattedString.Spans.Add (new Span { Text = "italic small.", FontAttributes
= FontAttributes.Italic, FontSize = 14 });

Label label = new Label { FormattedText = formattedString };
```

The following screenshot shows the resulting [Label](#) that contains three [Span](#) objects:



A [Span](#) can also respond to any gestures that are added to the span's [GestureRecognizers](#) collection. For example, a [TapGestureRecognizer](#) has been added to the second [Span](#) in the above examples. Therefore, when this [Span](#) is tapped the [TapGestureRecognizer](#) will respond by executing the [ICommand](#) defined by the [Command](#) property. For more information about tap gesture recognition, see [Recognize a tap gesture](#).

Create a hyperlink

The text displayed by [Label](#) and [Span](#) instances can be turned into hyperlinks with the following approach:

1. Set the [TextColor](#) and [TextDecoration](#) properties of the [Label](#) or [Span](#).
2. Add a [TapGestureRecognizer](#) to the [GestureRecognizers](#) collection of the [Label](#) or [Span](#), whose [Command](#) property binds to a [ICommand](#), and whose [CommandParameter](#) property contains the URL to open.
3. Define the [ICommand](#) that will be executed by the [TapGestureRecognizer](#).
4. Write the code that will be executed by the [ICommand](#).

The following example, shows a [Label](#) whose content is set from multiple [Span](#) objects:

XAML

```
<Label>
  <Label.FormattedText>
    <FormattedString>
      <Span Text="Alternatively, click " />
      <Span Text="here"
        TextColor="Blue"
        TextDecorations="Underline">
        <Span.GestureRecognizers>
          <TapGestureRecognizer Command="{Binding TapCommand}"
            CommandParameter="https://learn.microsoft.com/dotnet/maui/" />
        </Span.GestureRecognizers>
```

```
        </Span>
        <Span Text=" to view .NET MAUI documentation." />
    </FormattedString>
</Label.FormattedText>
</Label>
```

In this example, the first and third `Span` instances contain text, while the second `Span` represents a tappable hyperlink. It has its text color set to blue, and has an underline text decoration. This creates the appearance of a hyperlink, as shown in the following screenshot:

Alternatively, click [here](#) to view .NET MAUI documentation.

When the hyperlink is tapped, the `TapGestureRecognizer` will respond by executing the `ICommand` defined by its `Command` property. In addition, the URL specified by the `CommandParameter` property will be passed to the `ICommand` as a parameter.

The code-behind for the XAML page contains the `TapCommand` implementation:

```
C#

using System.Windows.Input;

public partial class MainPage : ContentPage
{
    // Launcher.OpenAsync is provided by Essentials.
    public ICommand TapCommand => new Command<string>(async (url) => await
    Launcher.OpenAsync(url));

    public MainPage()
    {
        InitializeComponent();
        BindingContext = this;
    }
}
```

The `TapCommand` executes the `Launcher.OpenAsync` method, passing the `TapGestureRecognizer.CommandParameter` property value as a parameter. The `Launcher.OpenAsync` method opens the URL in a web browser. Therefore, the overall effect is that when the hyperlink is tapped on the page, a web browser appears and the URL associated with the hyperlink is navigated to.

Create a reusable hyperlink class

The previous approach to creating a hyperlink requires writing repetitive code every time you require a hyperlink in your app. However, both the [Label](#) and [Span](#) classes can be subclassed to create `HyperlinkLabel` and `HyperlinkSpan` classes, with the gesture recognizer and text formatting code added there.

The following example shows a `HyperlinkSpan` class:

C#

```
public class HyperlinkSpan : Span
{
    public static readonly BindableProperty UrlProperty =
        BindableProperty.Create(nameof(Url), typeof(string),
        typeof(HyperlinkSpan), null);

    public string Url
    {
        get { return (string)GetValue(UrlProperty); }
        set { SetValue(UrlProperty, value); }
    }

    public HyperlinkSpan()
    {
        TextDecorations = TextDecorations.Underline;
        TextColor = Colors.Blue;
        GestureRecognizers.Add(new TapGestureRecognizer
        {
            // Launcher.OpenAsync is provided by Essentials.
            Command = new Command(async () => await Launcher.OpenAsync(Url))
        });
    }
}
```

The `HyperlinkSpan` class defines a `Url` property, and associated `BindableProperty`, and the constructor sets the hyperlink appearance and the `TapGestureRecognizer` that will respond when the hyperlink is tapped. When a `HyperlinkSpan` is tapped, the `TapGestureRecognizer` will respond by executing the `Launcher.OpenAsync` method to open the URL, specified by the `Url` property, in a web browser.

The `HyperlinkSpan` class can be consumed by adding an instance of the class to the XAML:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:HyperlinkDemo"
    x:Class="HyperlinkDemo.MainPage">
    <StackLayout>
```



```
...
<Label>
    <Label.FormattedText>
        <FormattedString>
            <Span Text="Alternatively, click " />
            <local:HyperlinkSpan Text="here"
Url="https://learn.microsoft.com/dotnet/" />
            <Span Text=" to view .NET documentation." />
        </FormattedString>
    </Label.FormattedText>
</Label>
</StackLayout>
</ContentPage>
```

ListView

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [ListView](#) displays a scrollable vertical list of selectable data items. While [ListView](#) manages the appearance of the list, the appearance of each item in the list is defined by a [DataTemplate](#) that uses a [Cell](#) to display items. .NET MAUI includes cell types to display combinations of text and images, and you can also define custom cells that display any content you want. [ListView](#) also includes support for displaying headers and footers, grouped data, pull-to-refresh, and context menu items.

The [ListView](#) class derives from the `ItemsView<Cell>` class, from which it inherits the following properties:

- `ItemsSource`, of type `IEnumerable`, specifies the collection of items to be displayed, and has a default value of `null`.
- `ItemTemplate`, of type [DataTemplate](#), specifies the template to apply to each item in the collection of items to be displayed.

[ListView](#) defines the following properties:

- `Footer`, of type `object`, specifies the string or view that will be displayed at the end of the list.
- `FooterTemplate`, of type [DataTemplate](#), specifies the [DataTemplate](#) to use to format the `Footer`.
- `GroupHeaderTemplate`, of type [DataTemplate](#), defines the [DataTemplate](#) used to define the appearance of the header of each group. This property is mutually exclusive with the `GroupDisplayBinding` property. Therefore, setting this property will set `GroupDisplayBinding` to `null`.
- `HasUnevenRows`, of type `bool`, indicates whether items in the list can have rows of different heights. The default value of this property is `false`.
- `Header`, of type `object`, specifies the string or view that will be displayed at the start of the list.
- `HeaderTemplate`, of type [DataTemplate](#), specifies the [DataTemplate](#) to use to format the `Header`.
- `HorizontalScrollBarVisibility`, of type `ScrollBarVisibility`, indicates when the horizontal scroll bar will be visible.

- `IsGroupingEnabled`, of type `bool`, indicates whether the underlying data should be displayed in groups. The default value of this property is `false`.
- `IsPullToRefreshEnabled`, of type `bool`, indicates whether the user can swipe down to cause the `ListView` to refresh its data. The default value of this property is `false`.
- `IsRefreshing`, of type `bool`, indicates whether the `ListView` is currently refreshing. The default value of this property is `false`.
- `RefreshCommand`, of type `ICommand`, represents the command that will be executed when a refresh is triggered.
- `RefreshControlColor`, of type `Color`, determines the color of the refresh visualization that's shown while a refresh occurs.
- `RowHeight`, of type `int`, determines the height of each row when `HasUnevenRows` is `false`.
- `SelectedItem`, of type `object`, represents the currently selected item in the `ListView`.
- `SelectionMode`, of type `ListViewSelectionMode`, indicates whether items can be selected in the `ListView` or not. The default value of this property is `Single`.
- `SeparatorColor`, of type `Color`, defines the color of the bar that separates items in the list.
- `SeparatorVisibility`, of type `SeparatorVisibility`, defines whether separators are visible between items.
- `VerticalScrollBarVisibility`, of type `ScrollBarVisibility`, indicates when the vertical scroll bar will be visible.

All of these properties are backed by `BindableProperty` objects, which means that they can be targets of data bindings, and styled.

In addition, `ListView` defines the following properties that aren't backed by `BindableProperty` objects:

- `GroupDisplayBinding`, of type `BindingBase`, the binding to use for displaying the group header. This property is mutually exclusive with the `GroupHeaderTemplate` property. Therefore, setting this property will set `GroupHeaderTemplate` to `null`.
- `GroupShortNameBinding`, of type `BindingBase`, the binding for the name to display in grouped jump lists.
- `CachingStrategy`, of type `ListViewCachingStrategy`, defines the cell reuse strategy of the `ListView`. This is a read-only property.

`ListView` defines the following events:

- `ItemAppearing`, which is raised when the visual representation of an item is being added to the visual layout of the `ListView`. The `ItemVisibilityEventArgs` object

that accompanies this event defines `Item` and `Index` properties.

- `ItemDisappearing`, which is raised when the visual representation of an item is being removed from the visual layout of the `ListView`. The `ItemVisibilityEventArgs` object that accompanies this event defines `Item` and `Index` properties.
- `ItemSelected`, which is raised when a new item in the list is selected. The `SelectedItemChangedEventArgs` object that accompanies this event defines `SelectedItem` and `SelectedItemIndex` properties.
- `ItemTapped`, which is raised when an item in the `ListView` is tapped. The `ItemTappedEventArgs` object that accompanies this event defines `Group`, `Item`, and `ItemIndex` properties.
- `Refreshing`, which is raised when a pull to refresh operation is triggered on the `ListView`.
- `Scrolled`, . The `ScrolledEventArgs` object that accompanies this event defines `ScrollX` and `ScrollY` properties.
- `ScrollToRequested` . The `ScrollToRequestedEventArgs` object that accompanies this event defines `Element`, `Mode`, `Position`, `ScrollX`, `ScrollY`, and `ShouldAnimate` properties.

Populate a ListView with data

A `ListView` is populated with data by setting its `ItemsSource` property to any collection that implements `IEnumerable`.

❗ Important

If the `ListView` is required to refresh as items are added, removed, or changed in the underlying collection, the underlying collection should be an `IEnumerable` collection that sends property change notifications, such as `ObservableCollection`.

`ListView` can be populated with data by using data binding to bind its `ItemsSource` property to an `IEnumerable` collection. In XAML, this is achieved with the `Binding` markup extension:

XAML

```
<ListView ItemsSource="{Binding Monkeys}" />
```

The equivalent C# code is:

C#

```
ListView listView = new ListView();  
listView.SetBinding(ItemsView.ItemsSourceProperty, "Monkeys");
```

In this example, the `ItemsSource` property data binds to the `Monkeys` property of the connected viewmodel.

ⓘ Note

Compiled bindings can be enabled to improve data binding performance in .NET MAUI applications. For more information, see [Compiled bindings](#).

For more information about data binding, see [Data binding](#).

Define item appearance

The appearance of each item in the `ListView` can be defined by setting the `ItemTemplate` property to a `DataTemplate`:

XAML

```
<ListView ItemsSource="{Binding Monkeys}">  
  <ListView.ItemTemplate>  
    <DataTemplate>  
      <ViewCell>  
        <Grid Padding="10">  
          <Grid.RowDefinitions>  
            <RowDefinition Height="Auto" />  
            <RowDefinition Height="Auto" />  
          </Grid.RowDefinitions>  
          <Grid.ColumnDefinitions>  
            <ColumnDefinition Width="Auto" />  
            <ColumnDefinition Width="Auto" />  
          </Grid.ColumnDefinitions>  
          <Image Grid.RowSpan="2"  
            Source="{Binding ImageUrl}"  
            Aspect="AspectFill"  
            HeightRequest="60"  
            WidthRequest="60" />  
          <Label Grid.Column="1"  
            Text="{Binding Name}"  
            FontAttributes="Bold" />  
          <Label Grid.Row="1"  
            Grid.Column="1"  
            Text="{Binding Location}"  
            FontAttributes="Italic"
```

```

VerticalOptions="End" />
        </Grid>
    </ViewCell>
</DataTemplate>
</ListView.ItemTemplate>
</ListView>

```

The elements specified in the [DataTemplate](#) define the appearance of each item in the list, and the child of the [DataTemplate](#) must be a [Cell](#) object. In the example, layout within the [DataTemplate](#) is managed by a [Grid](#). The [Grid](#) contains an [Image](#) object, and two [Label](#) objects, that all bind to properties of the `Monkey` class:

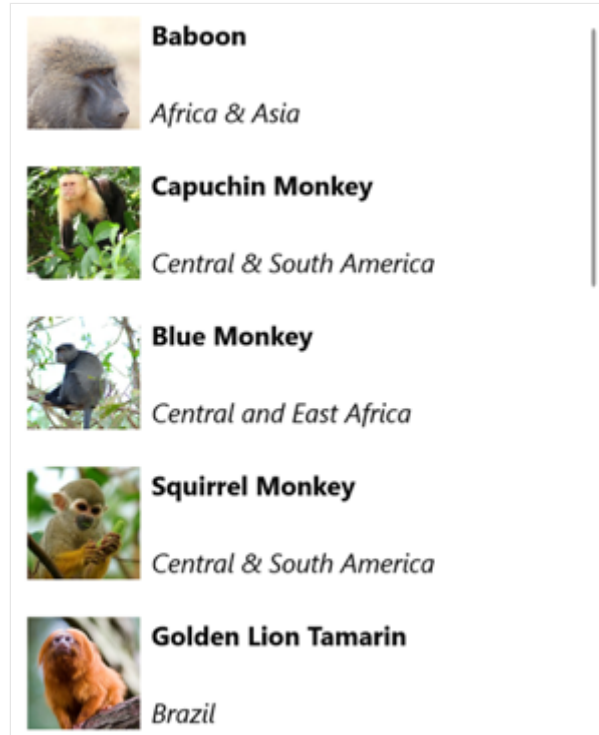
```

C#

public class Monkey
{
    public string Name { get; set; }
    public string Location { get; set; }
    public string Details { get; set; }
    public string ImageUrl { get; set; }
}

```

The following screenshot shows the result of templating each item in the list:



For more information about data templates, see [Data templates](#).

Cells

The appearance of each item in a [ListView](#) is defined by a [DataTemplate](#), and the [DataTemplate](#) must reference a [Cell](#) class to display items. Each cell represents an item of data in the [ListView](#). .NET MAUI includes the following built-in cells:

- [TextCell](#), which displays primary and secondary text on separate lines.
- [ImageCell](#), which displays an image with primary and secondary text on separate lines.
- [SwitchCell](#), which displays text and a switch that can be switched on or off.
- [EntryCell](#), which displays a label and text that's editable.
- [ViewCell](#), which is a custom cell whose appearance is defined by a [View](#). This cell type should be used when you want to fully define the appearance of each item in a [ListView](#).

Typically, [SwitchCell](#) and [EntryCell](#) will only be used in a [TableView](#) and won't be used in a [ListView](#). For more information about [SwitchCell](#) and [EntryCell](#), see [TableView](#).

Text cell

A [TextCell](#) displays primary and secondary text on separate lines. [TextCell](#) defines the following properties:

- `Text`, of type `string`, defines the primary text to be displayed.
- `TextColor`, of type [Color](#), represents the color of the primary text.
- `Detail`, of type `string`, defines the secondary text to be displayed.
- `DetailColor`, of type [Color](#), indicates the color of the secondary text.
- `Command`, of type [ICommand](#), defines the command that's executed when the cell is tapped.
- `CommandParameter`, of type `object`, represents the parameter that's passed to the command.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The following example shows using a [TextCell](#) to define the appearance of items in a [ListView](#):

XAML

```
<ListView ItemsSource="{Binding Food}">
  <ListView.ItemTemplate>
    <DataTemplate>
      <TextCell Text="{Binding Name}"
                Detail="{Binding Description}" />
    </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

```
</ListView.ItemTemplate>
</ListView>
```

The following screenshot shows the resulting cell appearance:



Image cell

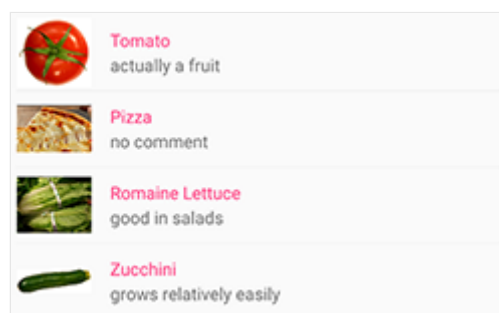
An [ImageCell](#) displays an image with primary and secondary text on separate lines. [ImageCell](#) inherits the properties from [TextCell](#), and defines the [ImageSource](#) property, of type [ImageSource](#), which specifies the image to be displayed in the cell. This property is backed by a [BindableProperty](#) object, which means it can be the target of data bindings, and be styled.

The following example shows using an [ImageCell](#) to define the appearance of items in a [ListView](#):

XAML

```
<ListView ItemsSource="{Binding Food}">
  <ListView.ItemTemplate>
    <DataTemplate>
      <ImageCell ImageSource="{Binding Image}"
                  Text="{Binding Name}"
                  Detail="{Binding Description}" />
    </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

The following screenshot shows the resulting cell appearance:



View cell

A [ViewCell](#) is a custom cell whose appearance is defined by a [View](#). [ViewCell](#) defines a [View](#) property, of type [View](#), which defines the view that represents the content of the cell. This property is backed by a [BindableProperty](#) object, which means it can be the target of data bindings, and be styled.

ⓘ Note

The [View](#) property is the content property of the [ViewCell](#) class, and therefore does not need to be explicitly set from XAML.

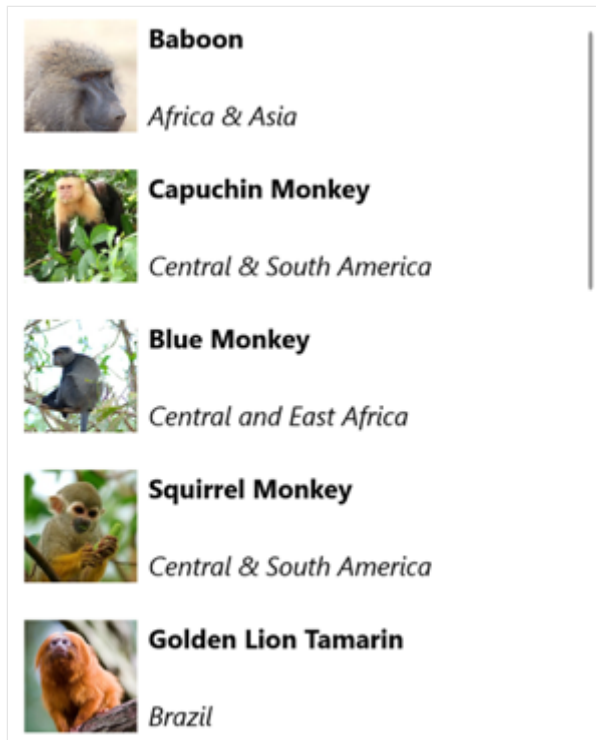
The following example shows using a [ViewCell](#) to define the appearance of items in a [ListView](#):

XAML

```
<ListView ItemsSource="{Binding Monkeys}">
  <ListView.ItemTemplate>
    <DataTemplate>
      <ViewCell>
        <Grid Padding="10">
          <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
          </Grid.RowDefinitions>
          <Grid.ColumnDefinitions>
            <ColumnDefinition Width="Auto" />
            <ColumnDefinition Width="Auto" />
          </Grid.ColumnDefinitions>
          <Image Grid.RowSpan="2"
            Source="{Binding ImageUrl}"
            Aspect="AspectFill"
            HeightRequest="60"
            WidthRequest="60" />
          <Label Grid.Column="1"
            Text="{Binding Name}"
            FontAttributes="Bold" />
          <Label Grid.Row="1"
            Grid.Column="1"
            Text="{Binding Location}"
            FontAttributes="Italic"
            VerticalOptions="End" />
        </Grid>
      </ViewCell>
    </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

Inside the [ViewCell](#), layout can be managed by any .NET MAUI layout. In this example, layout is managed by a [Grid](#). The [Grid](#) contains an [Image](#) object, and two [Label](#) objects, that all bind to properties of the `Monkey` class.

The following screenshot shows the result of templating each item in the list:



Choose item appearance at runtime

The appearance of each item in the [ListView](#) can be chosen at runtime, based on the item value, by setting the `ItemTemplate` property to a [DataTemplateSelector](#) object:

XAML

```
<ContentPage ...
    xmlns:templates="clr-namespace:ListViewDemos.Templates">
    <ContentPage.Resources>
        <DataTemplate x:Key="AmericanMonkeyTemplate">
            <ViewCell>
                ...
            </ViewCell>
        </DataTemplate>

        <DataTemplate x:Key="OtherMonkeyTemplate">
            <ViewCell>
                ...
            </ViewCell>
        </DataTemplate>

        <templates:MonkeyDataTemplateSelector x:Key="MonkeySelector"
            AmericanMonkey="{StaticResource
```

```

AmericanMonkeyTemplate}"
                                OtherMonkey="{StaticResource
OtherMonkeyTemplate}" />
</ContentPage.Resources>

<Grid Margin="20">
    <ListView ItemsSource="{Binding Monkeys}"
              ItemTemplate="{StaticResource MonkeySelector}" />
</Grid>
</ContentPage>

```

The `ItemTemplate` property is set to a `MonkeyDataTemplateSelector` object. The following example shows the `MonkeyDataTemplateSelector` class:

```

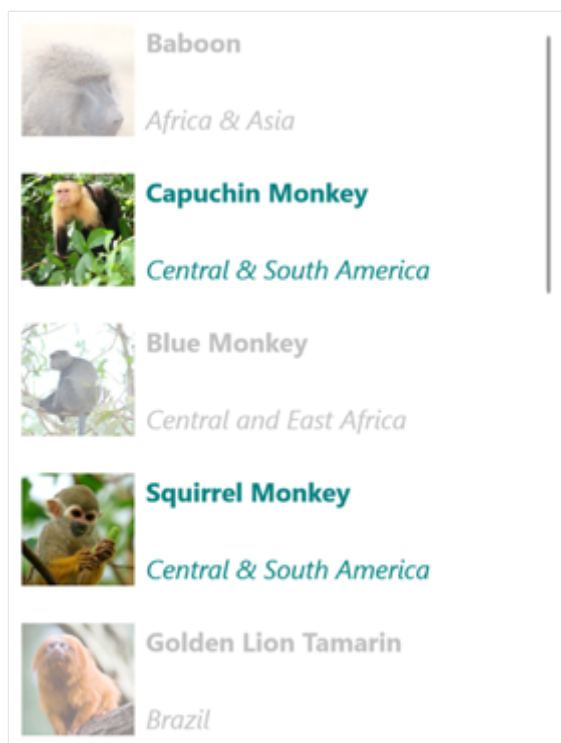
C#

public class MonkeyDataTemplateSelector : DataTemplateSelector
{
    public DataTemplate AmericanMonkey { get; set; }
    public DataTemplate OtherMonkey { get; set; }

    protected override DataTemplate OnSelectTemplate(object item,
BindableObject container)
    {
        return ((Monkey)item).Location.Contains("America") ? AmericanMonkey
: OtherMonkey;
    }
}

```

The `MonkeyDataTemplateSelector` class defines `AmericanMonkey` and `OtherMonkey` `DataTemplate` properties that are set to different data templates. The `OnSelectTemplate` override returns the `AmericanMonkey` template, which displays the monkey name and location in teal, when the monkey name contains "America". When the monkey name doesn't contain "America", the `OnSelectTemplate` override returns the `OtherMonkey` template, which displays the monkey name and location in silver:



For more information about data template selectors, see [Create a DataTemplateSelector](#).

Respond to item selection

By default, [ListView](#) selection is enabled. However, this behavior can be changed by setting the `SelectionMode` property. The `ListViewSelectionMode` enumeration defines the following members:

- `None` – indicates that items cannot be selected.
- `Single` – indicates that a single item can be selected, with the selected item being highlighted. This is the default value.

[ListView](#) defines an `ItemSelected` event that's raised when the `SelectedItem` property changes, either due to the user selecting an item from the list, or when an app sets the property. The `SelectedItemChangedEventArgs` object that accompanies this event has `SelectedItem` and `SelectedItemIndex` properties.

When the `SelectionMode` property is set to `Single`, a single item in the [ListView](#) can be selected. When an item is selected, the `SelectedItem` property will be set to the value of the selected item. When this property changes, the `ItemSelected` event is raised.

The following example shows a [ListView](#) that can respond to single item selection:

XAML

```
<ListView ItemsSource="{Binding Monkeys}"
  ItemSelected="OnItemSelected">
```

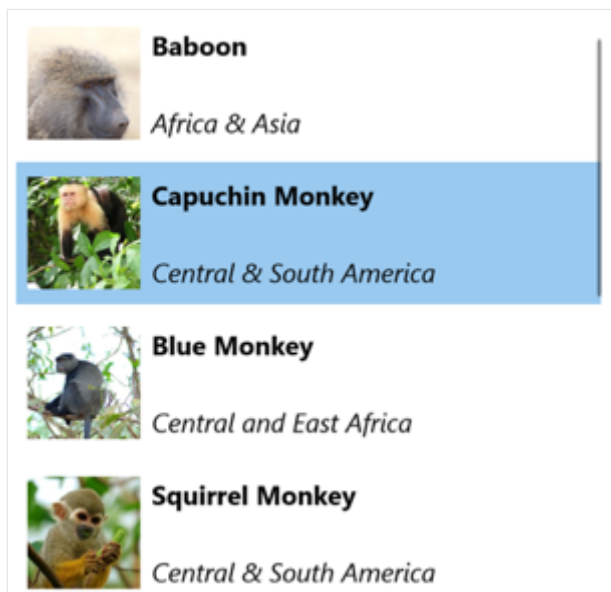
```
...  
</ListView>
```

In this example, the `OnItemSelected` event handler is executed when the `ItemSelected` event fires, with the event handler retrieving the selected item:

C#

```
void OnItemSelected(object sender, SelectedItemChangedEventArgs args)  
{  
    Monkey item = args.SelectedItem as Monkey;  
}
```

The following screenshot shows single item selection in a [ListView](#):



Clear the selection

The `SelectedItem` property can be cleared by setting it, or the object it binds to, to `null`.

Disable selection

[ListView](#) selection is enabled by default. However, it can be disabled by setting the `SelectionMode` property to `None`:

XAML

```
<ListView ...  
    SelectionMode="None" />
```

When the `SelectionMode` property is set to `None`, items in the `ListView` cannot be selected, the `SelectedItem` property will remain `null`, and the `ItemSelected` event will not be fired.

Cache data

`ListView` is a powerful view for displaying data, but it has some limitations. Scrolling performance can suffer when using custom cells, especially when they contain deeply nested view hierarchies or use certain layouts that require complex measurement. Fortunately, there are techniques you can use to avoid poor performance.

A `ListView` is often used to display much more data than fits onscreen. For example, a music app might have a library of songs with thousands of entries. Creating an item for every entry would waste valuable memory and perform poorly. Creating and destroying rows constantly would require the app to instantiate and cleanup objects constantly, which would also perform poorly.

To conserve memory, the native `ListView` equivalents for each platform have built-in features for reusing rows. Only the cells visible on screen are loaded in memory and the content is loaded into existing cells. This pattern prevents the app from instantiating thousands of objects, saving time and memory.

.NET MAUI permits `ListView` cell reuse through the `ListViewCachingStrategy` enumeration, which defines the following members:

- `RetainElement`, specifies that the `ListView` will generate a cell for each item in the list.
- `RecycleElement`, specifies that the `ListView` will attempt to minimize its memory footprint and execution speed by recycling list cells.
- `RecycleElementAndDataTemplate`, as `RecycleElement` while also ensuring that when a `ListView` uses a `DataTemplateSelector`, `DataTemplate` objects are cached by the type of item in the list.

Retain elements

The `RetainElement` caching strategy specifies that the `ListView` will generate a cell for each item in the list, and is the default `ListView` behavior. It should be used in the following circumstances:

- Each cell has a large number of bindings (20-30+).
- The cell template changes frequently.

- Testing reveals that the `RecycleElement` caching strategy results in a reduced execution speed.

It's important to recognize the consequences of the `RetainElement` caching strategy when working with custom cells. Any cell initialization code will need to run for each cell creation, which may be multiple times per second. In this circumstance, layout techniques that were fine on a page, like using multiple nested `Grid` objects, become performance bottlenecks when they're set up and destroyed in real time as the user scrolls.

Recycle elements

The `RecycleElement` caching strategy specifies that the `ListView` will attempt to minimize its memory footprint and execution speed by recycling list cells. This mode doesn't always offer a performance improvement, and testing should be performed to determine any improvements. However, it's the preferred choice, and should be used in the following circumstances:

- Each cell has a small to moderate number of bindings.
- Each cell's `BindingContext` defines all of the cell data.
- Each cell is largely similar, with the cell template unchanging.

During virtualization the cell will have its binding context updated, and so if an app uses this mode it must ensure that binding context updates are handled appropriately. All data about the cell must come from the binding context or consistency errors may occur. This problem can be avoided by using data binding to display cell data. Alternatively, cell data should be set in the `OnBindingContextChanged` override, rather than in the custom cell's constructor, as shown in the following example:

C#

```
public class CustomCell : ViewCell
{
    Image image = null;

    public CustomCell()
    {
        image = new Image();
        View = image;
    }

    protected override void OnBindingContextChanged()
    {
        base.OnBindingContextChanged();

        var item = BindingContext as ImageItem;
```

```
        if (item != null)
        {
            image.Source = item.ImageUrl;
        }
    }
}
```

Recycle elements with a DataTemplateSelector

When a [ListView](#) uses a [DataTemplateSelector](#) to select a [DataTemplate](#), the `RecycleElement` caching strategy does not cache [DataTemplate](#) objects. Instead, a [DataTemplate](#) is selected for each item of data in the list.

ⓘ Note

The `RecycleElement` caching strategy requires that when a [DataTemplateSelector](#) is asked to select a [DataTemplate](#) that each [DataTemplate](#) must return the same [ViewCell](#) type. For example, given a [ListView](#) with a [DataTemplateSelector](#) that can return either `MyDataTemplateA` (where `MyDataTemplateA` returns a [ViewCell](#) of type `MyViewCellA`), or `MyDataTemplateB` (where `MyDataTemplateB` returns a [ViewCell](#) of type `MyViewCellB`), when `MyDataTemplateA` is returned it must return `MyViewCellA` or an exception will be thrown.

Recycle elements with DataTemplates

The `RecycleElementAndDataTemplate` caching strategy builds on the `RecycleElement` caching strategy by additionally ensuring that when a [ListView](#) uses a [DataTemplateSelector](#) to select a [DataTemplate](#), [DataTemplate](#) objects are cached by the type of item in the list. Therefore, [DataTemplate](#) objects are selected once per item type, instead of once per item instance.

ⓘ Note

The `RecycleElementAndDataTemplate` caching strategy requires that [DataTemplate](#) objects returned by the [DataTemplateSelector](#) must use the [DataTemplate](#) constructor that takes a `Type`.

Set the caching strategy

The [ListView](#) caching strategy can be defined by in XAML by setting the `CachingStrategy` attribute:

XAML

```
<ListView CachingStrategy="RecycleElement">
    <ListView.ItemTemplate>
        <DataTemplate>
            <ViewCell>
                ...
            </ViewCell>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
```

In C#, the caching strategy is set via a constructor overload:

C#

```
ListView listView = new ListView(ListViewCachingStrategy.RecycleElement);
```

Set the caching strategy in a subclassed ListView

Setting the `CachingStrategy` attribute from XAML on a subclassed [ListView](#) will not produce the desired behavior, because there's no `CachingStrategy` property on [ListView](#). The solution to this issue is to specify a constructor on the subclassed [ListView](#) that accepts a `ListViewCachingStrategy` parameter and passes it to the base class:

C#

```
public class CustomListView : ListView
{
    public CustomListView (ListViewCachingStrategy strategy) : base
    (strategy)
    {
    }
    ...
}
```

Then the `ListViewCachingStrategy` enumeration value can be specified from XAML by using the `x:Arguments` attribute:

XAML

```
<local:CustomListView>
    <x:Arguments>
```

```
<ListViewCachingStrategy>RecycleElement</ListViewCachingStrategy>
</x:Arguments>
</local:CustomListView>
```

Headers and footers

[ListView](#) can present a header and footer that scroll with the items in the list. The header and footer can be strings, views, or [DataTemplate](#) objects.

[ListView](#) defines the following properties for specifying the header and footer:

- `Header`, of type `object`, specifies the string, binding, or view that will be displayed at the start of the list.
- `HeaderTemplate`, of type [DataTemplate](#), specifies the [DataTemplate](#) to use to format the `Header`.
- `Footer`, of type `object`, specifies the string, binding, or view that will be displayed at the end of the list.
- `FooterTemplate`, of type [DataTemplate](#), specifies the [DataTemplate](#) to use to format the `Footer`.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings.

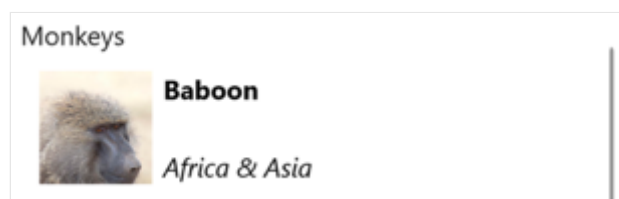
Display strings in the header and footer

The `Header` and `Footer` properties can be set to `string` values, as shown in the following example:

XAML

```
<ListView ItemsSource="{Binding Monkeys}"
    Header="Monkeys"
    Footer="2022">
    ...
</ListView>
```

The following screenshot shows the resulting header:



Display views in the header and footer

The `Header` and `Footer` properties can each be set to a view. This can be a single view, or a view that contains multiple child views. The following example shows the `Header` and `Footer` properties each set to a `Grid` object that contains a `Label` object:

XAML

```
<ListView ItemsSource="{Binding Monkeys}">
  <ListView.Header>
    <Grid BackgroundColor="LightGray">
      <Label Margin="10,0,0,0"
        Text="Monkeys"
        FontSize="12"
        FontAttributes="Bold" />
    </Grid>
  </ListView.Header>
  <ListView.Footer>
    <Grid BackgroundColor="LightGray">
      <Label Margin="10,0,0,0"
        Text="Friends of Monkey"
        FontSize="12"
        FontAttributes="Bold" />
    </Grid>
  </ListView.Footer>
  ...
</ListView>
```

The following screenshot shows the resulting header:



Display a templated header and footer

The `HeaderTemplate` and `FooterTemplate` properties can be set to `DataTemplate` objects that are used to format the header and footer. In this scenario, the `Header` and `Footer` properties must bind to the current source for the templates to be applied, as shown in the following example:

XAML

```
<ListView ItemsSource="{Binding Monkeys}"
  Header="{Binding .}"
```

```

        Footer="{Binding .}">
    <ListView.HeaderTemplate>
        <DataTemplate>
            <Grid BackgroundColor="LightGray">
                <Label Margin="10,0,0,0"
                    Text="Monkeys"
                    FontSize="12"
                    FontAttributes="Bold" />
            </Grid>
        </DataTemplate>
    </ListView.HeaderTemplate>
    <ListView.FooterTemplate>
        <DataTemplate>
            <Grid BackgroundColor="LightGray">
                <Label Margin="10,0,0,0"
                    Text="Friends of Monkey"
                    FontSize="12"
                    FontAttributes="Bold" />
            </Grid>
        </DataTemplate>
    </ListView.FooterTemplate>
    ...
</ListView>

```

Control item separators

By default, separators are displayed between [ListView](#) items on iOS and Android. This behavior can be changed by setting the `SeparatorVisibility` property, of type `SeparatorVisibility`, to `None`:

XAML

```

<ListView ...
    SeparatorVisibility="None" />

```

In addition, when the separator is enabled, its color can be set with the `SeparatorColor` property:

XAML

```

<ListView ...
    SeparatorColor="Blue" />

```

Size items

By default, all items in a [ListView](#) have the same height, which is derived from the contents of the [DataTemplate](#) that defines the appearance of each item. However, this behavior can be changed with the `HasUnevenRows` and `RowHeight` properties. By default, the `HasUnevenRows` property is `false`.

The `RowHeight` property can be set to an `int` that represents the height of each item in the [ListView](#), provided that `HasUnevenRows` is `false`. When `HasUnevenRows` is set to `true`, each item in the [ListView](#) can have a different height. The height of each item will be derived from the contents of the item's [DataTemplate](#), and so each item will be sized to its content.

Individual [ListView](#) items can be programmatically resized at runtime by changing layout related properties of elements within the [DataTemplate](#), provided that the `HasUnevenRows` property is `true`. The following example changes the height of an [Image](#) object when it's tapped:

C#

```
void OnImageTapped(object sender, EventArgs args)
{
    Image image = sender as Image;
    ViewCell viewCell = image.Parent.Parent as ViewCell;

    if (image.HeightRequest < 250)
    {
        image.HeightRequest = image.Height + 100;
        viewCell.ForceUpdateSize();
    }
}
```

In this example, the `OnImageTapped` event handler is executed in response to an [Image](#) object being tapped. The event handler updates the height of the [Image](#) and the `Cell.ForceUpdateSize` method updates the cell's size, even when it isn't currently visible.

Warning

Overuse of dynamic item sizing can cause [ListView](#) performance to degrade.

Right-to-left layout

[ListView](#) can layout its content in a right-to-left flow direction by setting its `FlowDirection` property to `RightToLeft`. However, the `FlowDirection` property should

ideally be set on a page or root layout, which causes all the elements within the page, or root layout, to respond to the flow direction:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="ListViewDemos.RightToLeftListPage"
             Title="Right to left list"
             FlowDirection="RightToLeft">
    <Grid Margin="20">
        <ListView ItemsSource="{Binding Monkeys}">
            ...
        </ListView>
    </Grid>
</ContentPage>
```

The default `FlowDirection` for an element with a parent is `MatchParent`. Therefore, the `ListView` inherits the `FlowDirection` property value from the `Grid`, which in turn inherits the `FlowDirection` property value from the `ContentPage`.

Display grouped data

Large data sets can often become unwieldy when presented in a continually scrolling list. In this scenario, organizing the data into groups can improve the user experience by making it easier to navigate the data.

Data must be grouped before it can be displayed. This can be accomplished by creating a list of groups, where each group is a list of items. The list of groups should be an `IEnumerable<T>` collection, where `T` defines two pieces of data:

- A group name.
- An `IEnumerable` collection that defines the items belonging to the group.

The process for grouping data, therefore, is to:

- Create a type that models a single item.
- Create a type that models a single group of items.
- Create an `IEnumerable<T>` collection, where `T` is the type that models a single group of items. This collection is a collection of groups, which stores the grouped data.
- Add data to the `IEnumerable<T>` collection.

Example

When grouping data, the first step is to create a type that models a single item. The following example shows the `Animal` class:

C#

```
public class Animal
{
    public string Name { get; set; }
    public string Location { get; set; }
    public string Details { get; set; }
    public string ImageUrl { get; set; }
}
```

The `Animal` class models a single item. A type that models a group of items can then be created. The following example shows the `AnimalGroup` class:

C#

```
public class AnimalGroup : List<Animal>
{
    public string Name { get; private set; }

    public AnimalGroup(string name, List<Animal> animals) : base(animals)
    {
        Name = name;
    }
}
```

The `AnimalGroup` class inherits from the `List<T>` class and adds a `Name` property that represents the group name.

An `IEnumerable<T>` collection of groups can then be created:

C#

```
public List<AnimalGroup> Animals { get; private set; } = new
List<AnimalGroup>();
```

This code defines a collection named `Animals`, where each item in the collection is an `AnimalGroup` object. Each `AnimalGroup` object comprises a name, and a `List<Animal>` collection that defines the `Animal` objects in the group.

Grouped data can then be added to the `Animals` collection:

C#

```

Animals.Add(new AnimalGroup("Bears", new List<Animal>
{
    new Animal
    {
        Name = "American Black Bear",
        Location = "North America",
        Details = "Details about the bear go here.",
        ImageUrl =
"https://upload.wikimedia.org/wikipedia/commons/0/08/01_Schwarzbär.jpg"
    },
    new Animal
    {
        Name = "Asian Black Bear",
        Location = "Asia",
        Details = "Details about the bear go here.",
        ImageUrl =
"https://upload.wikimedia.org/wikipedia/commons/thumb/b/b7/Ursus_thibetanus_3_%28Wroclaw_zoo%29.JPG/180px-Ursus_thibetanus_3_%28Wroclaw_zoo%29.JPG"
    },
    // ...
}));

Animals.Add(new AnimalGroup("Monkeys", new List<Animal>
{
    new Animal
    {
        Name = "Baboon",
        Location = "Africa & Asia",
        Details = "Details about the monkey go here.",
        ImageUrl =
"https://upload.wikimedia.org/wikipedia/commons/thumb/f/fc/Papio_anubis_%28Serengeti%2C_2009%29.jpg/200px-Papio_anubis_%28Serengeti%2C_2009%29.jpg"
    },
    new Animal
    {
        Name = "Capuchin Monkey",
        Location = "Central & South America",
        Details = "Details about the monkey go here.",
        ImageUrl =
"https://upload.wikimedia.org/wikipedia/commons/thumb/4/40/Capuchin_Costa_Rica.jpg/200px-Capuchin_Costa_Rica.jpg"
    },
    new Animal
    {
        Name = "Blue Monkey",
        Location = "Central and East Africa",
        Details = "Details about the monkey go here.",
        ImageUrl =
"https://upload.wikimedia.org/wikipedia/commons/thumb/8/83/BlueMonkey.jpg/220px-BlueMonkey.jpg"
    },
    // ...
}));

```


This code creates two groups in the `Animals` collection. The first `AnimalGroup` is named `Bears`, and contains a `List<Animal>` collection of bear details. The second `AnimalGroup` is named `Monkeys`, and contains a `List<Animal>` collection of monkey details.

`ListView` will display grouped data, provided that the data has been grouped correctly, by setting the `IsGroupingEnabled` property to `true`:

XAML

```
<ListView ItemsSource="{Binding Animals}"
    IsGroupingEnabled="True">
    <ListView.ItemTemplate>
        <DataTemplate>
            <ViewCell>
                <Grid Padding="10">
                    <Grid.RowDefinitions>
                        <RowDefinition Height="Auto" />
                        <RowDefinition Height="Auto" />
                    </Grid.RowDefinitions>
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="Auto" />
                        <ColumnDefinition Width="Auto" />
                    </Grid.ColumnDefinitions>
                    <Image Grid.RowSpan="2"
                        Source="{Binding ImageUrl}"
                        Aspect="AspectFill"
                        HeightRequest="60"
                        WidthRequest="60" />
                    <Label Grid.Column="1"
                        Text="{Binding Name}"
                        FontAttributes="Bold" />
                    <Label Grid.Row="1"
                        Grid.Column="1"
                        Text="{Binding Location}"
                        FontAttributes="Italic"
                        VerticalOptions="End" />
                </Grid>
            </ViewCell>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
```

The equivalent C# code is:

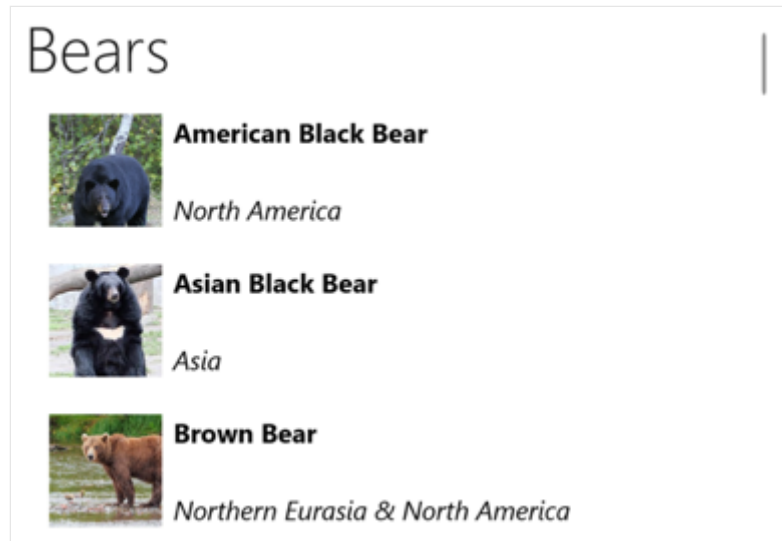
C#

```
ListView listView = new ListView
{
    IsGroupingEnabled = true
};
```

```
listView.SetBinding(ItemsView.ItemsSourceProperty, "Animals");  
// ...
```

The appearance of each item in the [ListView](#) is defined by setting its `ItemTemplate` property to a [DataTemplate](#). For more information, see [Define item appearance](#).

The following screenshot shows the [ListView](#) displaying grouped data:



ⓘ Note

By default, [ListView](#) will display the group name in the group header. This behavior can be changed by customizing the group header.

Customize the group header

The appearance of each group header can be customized by setting the `ListView.GroupHeaderTemplate` property to a [DataTemplate](#):

XAML

```
<ListView ItemsSource="{Binding Animals}"  
    IsGroupingEnabled="True">  
    <ListView.GroupHeaderTemplate>  
        <DataTemplate>  
            <ViewCell>  
                <Label Text="{Binding Name}"  
                    BackgroundColor="LightGray"  
                    FontSize="18"  
                    FontAttributes="Bold" />  
            </ViewCell>  
        </DataTemplate>  
    </ListView.GroupHeaderTemplate>
```

```
...  
</ListView>
```

In this example, each group header is set to a [Label](#) that displays the group name, and that has other appearance properties set. The following screenshot shows the customized group header:



Important

The `GroupHeaderTemplate` property is mutually exclusive with the `GroupDisplayBinding` property. Therefore, both properties should not be set.

Group without templates

[ListView](#) can display correctly grouped data without setting the `ItemTemplate` property to a [DataTemplate](#):

XAML

```
<ListView ItemsSource="{Binding Animals}"  
          IsGroupingEnabled="true" />
```

In this scenario, meaningful data can be displayed by overriding the `ToString` method in the type that models a single item, and the type that models a single group of items.

Control scrolling

[ListView](#) defines two `ScrollTo` methods, that scroll items into view. One of the overloads scrolls the specified item into view, while the other scrolls the specified item in the specified group into view. Both overloads have additional arguments that allow the exact position of the item after the scroll has completed to be specified, and whether to animate the scroll.

[ListView](#) defines a `ScrollToRequested` event that is fired when one of the `ScrollTo` methods is invoked. The `ScrollToRequestedEventArgs` object that accompanies the `ScrollToRequested` event has many properties, including `ShouldAnimate`, `Element`, `Mode`, and `Position`. Some of these properties are set from the arguments specified in the `ScrollTo` method calls.

In addition, [ListView](#) defines a `Scrolled` event that is fired to indicate that scrolling occurred. The `ScrolledEventArgs` object that accompanies the `Scrolled` event has `ScrollX` and `ScrollY` properties.

Detect scrolling

[ListView](#) defines a `Scrolled` event which is fired to indicate that scrolling occurred. The `ItemsViewScrolledEventArgs` class, which represents the object that accompanies the `Scrolled` event, defines the following properties:

- `ScrollX`, of type `double`, represents the X position of the scroll
- `ScrollY`, of type `double`, represents the Y position of the scroll.

The following XAML example shows a [ListView](#) that sets an event handler for the `Scrolled` event:

XAML

```
<ListView Scrolled="OnListViewScrolled">
    ...
</ListView>
```

The equivalent C# code is:

C#

```
ListView listView = new ListView();
listView.Scrolled += OnListViewScrolled;
```

In this code example, the `OnListViewScrolled` event handler is executed when the `Scrolled` event fires:

C#

```
void OnListViewScrolled(object sender, ScrolledEventArgs e)
{
    // Custom logic
}
```

❗ Important

The `Scrolled` event is fired for user initiated scrolls, and for programmatic scrolls.

Scroll an item into view

The `ScrollTo` method scrolls the specified item into view. Given a `ListView` object named `listView`, the following example shows how to scroll the Proboscis Monkey item into view:

C#

```
MonkeysViewModel viewModel = BindingContext as MonkeysViewModel;
Monkey monkey = viewModel.Monkeys.FirstOrDefault(m => m.Name == "Proboscis
Monkey");
listView.ScrollTo(monkey, ScrollToPosition.MakeVisible, true);
```

Alternatively, an item in grouped data can be scrolled into view by specifying the item and the group. The following example shows how to scroll the Proboscis Monkey item in the Monkeys group into view:

C#

```
GroupedAnimalsViewModel viewModel = BindingContext as
GroupedAnimalsViewModel;
AnimalGroup group = viewModel.Animals.FirstOrDefault(a => a.Name ==
"Monkeys");
Animal monkey = group.FirstOrDefault(m => m.Name == "Proboscis Monkey");
listView.ScrollTo(monkey, group, ScrollToPosition.MakeVisible, true);
```

❗ Note

The `ScrollToRequested` event is fired when the `ScrollTo` method is invoked.

Disable scroll animation

A scrolling animation is displayed when scrolling an item into view. However, this animation can be disabled by setting the `animated` argument of the `ScrollTo` method to `false`:

C#

```
listView.ScrollTo(monkey, position: ScrollToPosition.MakeVisible, animate: false);
```

Control scroll position

When scrolling an item into view, the exact position of the item after the scroll has completed can be specified with the `position` argument of the `ScrollTo` methods. This argument accepts a `ScrollToPosition` enumeration member.

MakeVisible

The `ScrollToPosition.MakeVisible` member indicates that the item should be scrolled until it's visible in the view:

C#

```
listView.ScrollTo(monkey, position: ScrollToPosition.MakeVisible, animate: true);
```

Start

The `ScrollToPosition.Start` member indicates that the item should be scrolled to the start of the view:

C#

```
listView.ScrollTo(monkey, position: ScrollToPosition.Start, animate: true);
```

Center

The `ScrollToPosition.Center` member indicates that the item should be scrolled to the center of the view:

C#

```
listView.ScrollTo(monkey, position: ScrollToPosition.Center, animate: true);
```

End

The `ScrollToPosition.End` member indicates that the item should be scrolled to the end of the view:

C#

```
listView.ScrollTo(monkey, position: ScrollToPosition.End, animate: true);
```

Scroll bar visibility

[ListView](#) defines `HorizontalScrollBarVisibility` and `VerticalScrollBarVisibility` properties, which are backed by bindable properties. These properties get or set a `ScrollBarVisibility` enumeration value that represents when the horizontal, or vertical, scroll bar is visible. The `ScrollBarVisibility` enumeration defines the following members:

- `Default` indicates the default scroll bar behavior for the platform, and is the default value for the `HorizontalScrollBarVisibility` and `VerticalScrollBarVisibility` properties.
- `Always` indicates that scroll bars will be visible, even when the content fits in the view.
- `Never` indicates that scroll bars will not be visible, even if the content doesn't fit in the view.

Add context menus

[ListView](#) supports context menu items, which are defined as [MenuItem](#) objects that are added to the `ViewCell.ContextActions` collection in the [DataTemplate](#) for each item:

XAML

```
<ListView x:Name="listView"
          ItemsSource="{Binding Monkeys}">
```

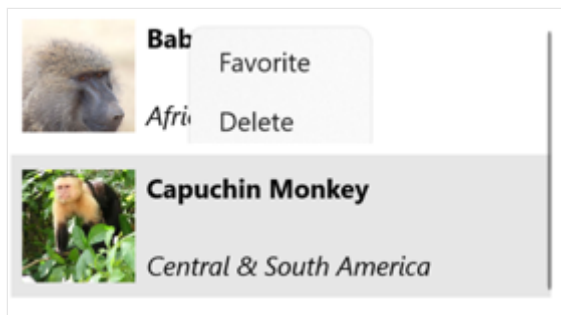
```

<ListView.ItemTemplate>
    <DataTemplate>
        <ViewCell>
            <ViewCell.ContextActions>
                <MenuItem Text="Favorite"
                    Command="{Binding Source={x:Reference
listView}, Path=BindingContext.FavoriteCommand}"
                    CommandParameter="{Binding}" />
                <MenuItem Text="Delete"
                    Command="{Binding Source={x:Reference
listView}, Path=BindingContext.DeleteCommand}"
                    CommandParameter="{Binding}" />
            </ViewCell.ContextActions>

            ...
        </ViewCell>
    </DataTemplate>
</ListView.ItemTemplate>
</ListView>

```

The [MenuItem](#) objects are revealed when an item in the [ListView](#) is right-clicked:



For more information about menu items, see [Display menu items](#).

Pull to refresh

[ListView](#) supports pull to refresh functionality, which enables the data being displayed to be refreshed by pulling down on the list of items.

To enable pull to refresh, set the `IsPullToRefreshEnabled` property to `true`. When a refresh is triggered, [ListView](#) raises the `Refreshing` event, and the `IsRefreshing` property will be set to `true`. The code required to refresh the contents of the [ListView](#) should then be executed by the handler for the `Refreshing` event, or by the [ICommand](#) implementation the `RefreshCommand` executes. Once the [ListView](#) is refreshed, the `IsRefreshing` property should be set to `false`, or the `EndRefresh` method should be called on the [ListView](#), to indicate that the refresh is complete.

The following example shows a [ListView](#) that uses pull to refresh:

XAML

```
<ListView ItemsSource="{Binding Animals}"
    IsPullToRefreshEnabled="true"
    RefreshCommand="{Binding RefreshCommand}"
    IsRefreshing="{Binding IsRefreshing}">
    <ListView.ItemTemplate>
        <DataTemplate>
            <ViewCell>
                ...
            </ViewCell>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
```

In this example, when the user initiates a refresh, the `ICommand` defined by the `RefreshCommand` property is executed, which should refresh the items being displayed. A refresh visualization is shown while the refresh occurs, which consists of an animated progress circle. The value of the `IsRefreshing` property indicates the current state of the refresh operation. When a refresh is triggered, this property will automatically transition to `true`. Once the refresh completes, you should reset the property to `false`.

Map

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [Map](#) control is a cross-platform view for displaying and annotating maps. The [Map](#) control uses the native map control on each platform, and is provided by the [Microsoft.Maui.Controls.Maps NuGet package](#) [↗](#).

Important

The [Map](#) control isn't supported on Windows due to lack of a map control in WinUI. However, the [CommunityToolkit.Maui.Maps](#) [↗](#) NuGet package provides access to Bing Maps through a [WebView](#) on Windows. For more information, see [Get started](#).

Setup

The [Map](#) control uses the native map control on each platform. This provides a fast, familiar maps experience for users, but means that some configuration steps are needed to adhere to each platform's API requirements.

Map initialization

The [Map](#) control is provided by the [Microsoft.Maui.Controls.Maps NuGet package](#) [↗](#), which should be added to your .NET MAUI app project.

After installing the NuGet package, it must be initialized in your app by calling the `UseMauiMap` method on the `MauiAppBuilder` object in the `CreateMauiApp` method of your `MauiProgram` class:

C#

```
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
```

```

        fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
        fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
    })
    .UseMauiMaps();

    return builder.Build();
}
}

```

Once the NuGet package has been added and initialized, [Map](#) APIs can be used in your project.

Platform configuration

Additional configuration is required on Android before the map will display. In addition, on iOS, Android, and Mac Catalyst, accessing the user's location requires location permissions to have been granted to your app.

iOS and Mac Catalyst

Displaying and interacting with a map on iOS and Mac Catalyst doesn't require any additional configuration. However, to access location services, you should set the required location services requests in **Info.plist**. These will typically be one or more of the following:

- [NSLocationAlwaysAndWhenInUseUsageDescription](#) [↗](#) – for using location services at all times.
- [NSLocationWhenInUseUsageDescription](#) [↗](#) – for using location services when the app is in use.

For more information, see [Choosing the location services authorization to request](#) [↗](#) on developer.apple.com.

The XML representation for these keys in **Info.plist** is shown below. You should update the `string` values to reflect how your app is using the location information:

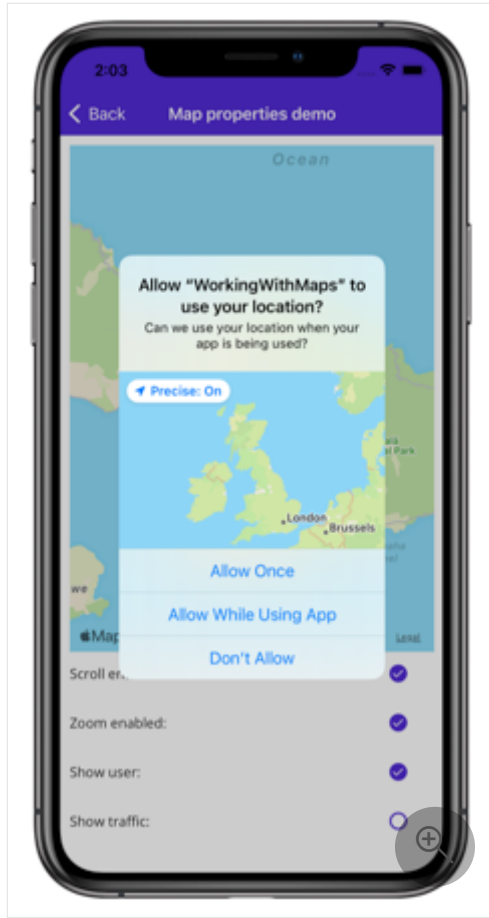
XML

```

<key>NSLocationAlwaysAndWhenInUseUsageDescription</key>
<string>Can we use your location at all times?</string>
<key>NSLocationWhenInUseUsageDescription</key>
<string>Can we use your location when your app is being used?</string>

```

A prompt is then displayed when your app attempts to access the user's location, requesting access:



Android

The configuration process for displaying and interacting with a map on Android is to:

1. Get a Google Maps API key and add it to your app manifest.
2. Specify the Google Play services version number in the manifest.
3. [optional] Specify location permissions in the manifest.
4. [optional] Specify the WRITE_EXTERNAL_STORAGE permission in the manifest.

Get a Google Maps API key

To use the [Map](#) control on Android you must generate an API key, which will be consumed by the [Google Maps SDK](#) on which the [Map](#) control relies on Android. To do this, follow the instructions in [Set up in the Google Cloud Console](#) and [Use API Keys](#) on [developers.google.com](#).

Once you've obtained an API key it must be added within the `<application>` element of your `Platforms/Android/AndroidManifest.xml` file, by specifying it as the value of the `com.google.android.geo.API_KEY` metadata:

XML

```
<application android:allowBackup="true" android:icon="@mipmap/appicon"
  android:roundIcon="@mipmap/appicon_round" android:supportRtl="true">
  <meta-data android:name="com.google.android.geo.API_KEY"
    android:value="PASTE-YOUR-API-KEY-HERE" />
</application>
```

This embeds the API key into the manifest. Without a valid API key the [Map](#) control will display a blank grid.

ⓘ Note

`com.google.android.geo.API_KEY` is the recommended metadata name for the API key. A key with this name can be used to authenticate to multiple Google Maps-based APIs on Android. For backwards compatibility, the `com.google.android.maps.v2.API_KEY` metadata name can be used, but only allows authentication to the Android Maps API v2. An app can only specify one of the API key metadata names.

Specify the Google Play services version number

Add the following declaration within the `<application>` element of **AndroidManifest.xml**:

XML

```
<meta-data android:name="com.google.android.gms.version"
  android:value="@integer/google_play_services_version" />
```

This embeds the version of Google Play services that the app was compiled with, into the manifest.

Specify location permissions

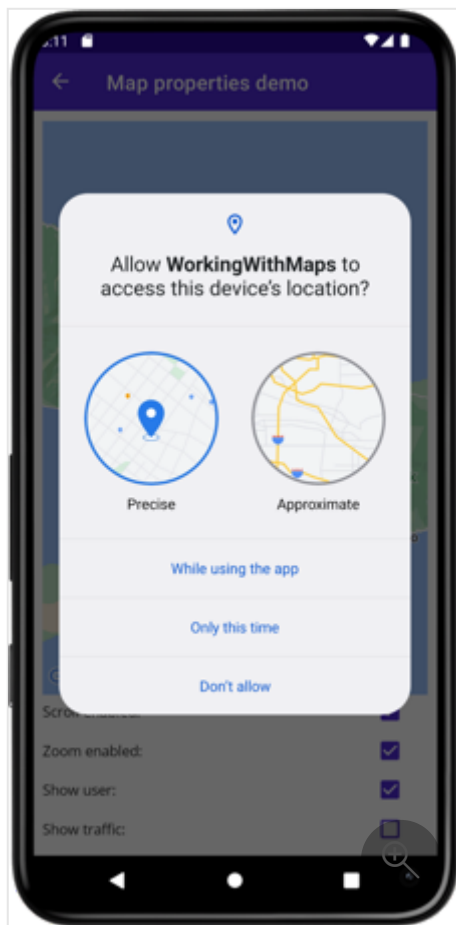
If your app needs to access the user's location, you must request permission by adding the `ACCESS_COARSE_LOCATION` or `ACCESS_FINE_LOCATION` permissions (or both) to the manifest, as a child of the `<manifest>` element:

XML

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
...
<!-- Required to access the user's location -->
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"
/>
</manifest>
```

The `ACCESS_COARSE_LOCATION` permission allows the API to use WiFi or mobile data, or both, to determine the device's location. The `ACCESS_FINE_LOCATION` permissions allows the API to use the Global Positioning System (GPS), WiFi, or mobile data to determine a precise a location as possible.

A prompt is then displayed when your app attempts to access the user's location, requesting access:



Alternatively, these permissions can be enabled in Visual Studio's Android manifest editor.

Specify the `WRITE_EXTERNAL_STORAGE` permission

If your app targets API 22 or lower, it will be necessary to add the `WRITE_EXTERNAL_STORAGE` permission to the manifest, as a child of the `<manifest>`

element:

XML

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
```

This is not required if your app targets API 23 or greater.

Map control

The [Map](#) class defines the following properties that control map appearance and behavior:

- `IsShowingUser`, of type `bool`, indicates whether the map is showing the user's current location.
- `ItemsSource`, of type `IEnumerable`, which specifies the collection of `IEnumerable` pin items to be displayed.
- `ItemTemplate`, of type [DataTemplate](#), which specifies the [DataTemplate](#) to apply to each item in the collection of displayed pins.
- `ItemTemplateSelector`, of type [DataTemplateSelector](#), which specifies the [DataTemplateSelector](#) that will be used to choose a [DataTemplate](#) for a pin at runtime.
- `IsScrollEnabled`, of type `bool`, determines whether the map is allowed to scroll.
- `IsTrafficEnabled`, of type `bool`, indicates whether traffic data is overlaid on the map.
- `IsZoomEnabled`, of type `bool`, determines whether the map is allowed to zoom.
- `MapElements`, of type `IList<MapElement>`, represents the list of elements on the map, such as polygons and polylines.
- `MapType`, of type `MapType`, indicates the display style of the map.
- `Pins`, of type `IList<Pin>`, represents the list of pins on the map.
- `VisibleRegion`, of type `MapSpan`, returns the currently displayed region of the map.

These properties, with the exception of the `MapElements`, `Pins`, and `VisibleRegion` properties, are backed by [BindableProperty](#) objects, which mean they can be targets of data bindings.

The [Map](#) class also defines a `MapClicked` event that's fired when the map is tapped. The `MapClickedEventArgs` object that accompanies the event has a single property named `Location`, of type `Location`. When the event is fired, the `Location` property is set to the

map location that was tapped. For information about the `Location` class, see [Location and distance](#).

For information about the `ItemsSource`, `ItemTemplate`, and `ItemTemplateSelector` properties, see [Display a pin collection](#).

Display a map

A `Map` can be displayed by adding it to a layout or page:

XAML

```
<ContentPage ...  
  
xmlns:maps="http://schemas.microsoft.com/dotnet/2021/maui/maps">  
    <maps:Map x:Name="map" />  
</ContentPage>
```

The equivalent C# code is:

C#

```
using Map = Microsoft.Maui.Controls.Maps.Map;  
  
namespace WorkingWithMaps  
{  
    public class MapTypesPageCode : ContentPage  
    {  
        public MapTypesPageCode()  
        {  
            Map map = new Map();  
            Content = map;  
        }  
    }  
}
```

This example calls the default `Map` constructor, which centers the map on Maui, Hawaii::



Alternatively, a `MapSpan` argument can be passed to a `Map` constructor to set the center point and zoom level of the map when it's loaded. For more information, see [Display a specific location on a map](#).

Important

.NET MAUI has two `Map` types - [Microsoft.Maui.Controls.Maps.Map](#) and [Microsoft.Maui.ApplicationModel.Map](#). Because the [Microsoft.Maui.ApplicationModel](#) namespace is one of .NET MAUI's `global using` directives, when using the [Microsoft.Maui.Controls.Maps.Map](#) control from code you'll have to fully qualify your `Map` usage or use a [using alias](#).

Map types

The `Map.MapType` property can be set to a `MapType` enumeration member to define the display style of the map. The `MapType` enumeration defines the following members:

- `Street` specifies that a street map will be displayed.
- `Satellite` specifies that a map containing satellite imagery will be displayed.
- `Hybrid` specifies that a map combining street and satellite data will be displayed.

By default, a `Map` will display a street map if the `MapType` property is undefined. Alternatively, the `MapType` property can be set to one of the `MapType` enumeration members:

XAML

```
<maps:Map MapType="Satellite" />
```

The equivalent C# code is:

C#

```
Map map = new Map
{
    MapType = MapType.Satellite
};
```

Display a specific location on a map

The region of a map to display when a map is loaded can be set by passing a `MapSpan` argument to the `Map` constructor:

XAML

```
<ContentPage ...
    xmlns:maps="http://schemas.microsoft.com/dotnet/2021/maui/maps"
    xmlns:sensors="clr-
namespace:Microsoft.Maui.Devices.Sensors;assembly=Microsoft.Maui.Essentials"
>
    <maps:Map>
        <x:Arguments>
            <maps:MapSpan>
                <x:Arguments>
                    <sensors:Location>
                        <x:Arguments>
                            <x:Double>36.9628066</x:Double>
                            <x:Double>-122.0194722</x:Double>
```

```

        </x:Arguments>
    </sensors:Location>
    <x:Double>0.01</x:Double>
    <x:Double>0.01</x:Double>
</x:Arguments>
</maps:MapSpan>
</x:Arguments>
</maps:Map>
</ContentPage>

```

The equivalent C# code is:

C#

```

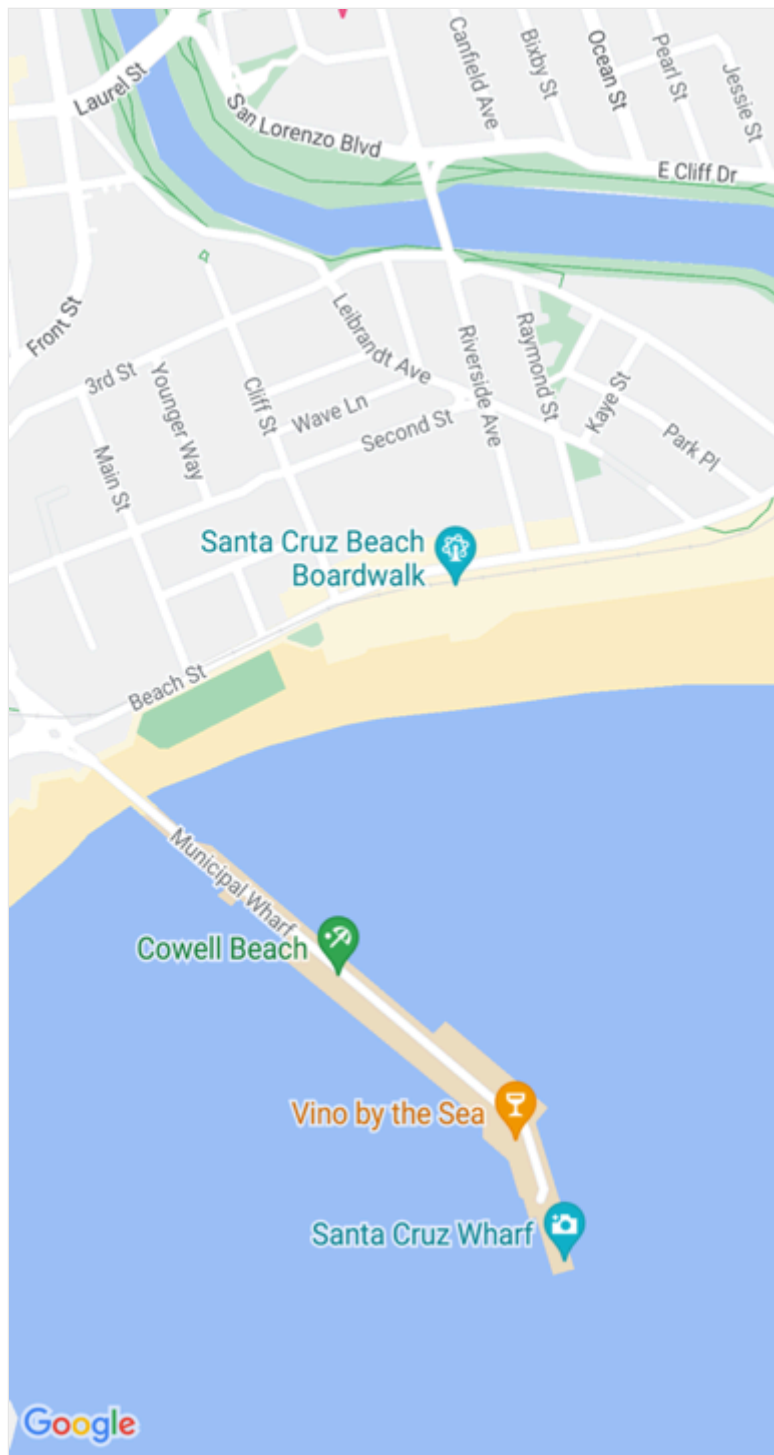
using Microsoft.Maui.Maps;
using Map = Microsoft.Maui.Controls.Maps.Map;
...

Location location = new Location(36.9628066, -122.0194722);
MapSpan mapSpan = new MapSpan(location, 0.01, 0.01);
Map map = new Map(mapSpan);

```

This example creates a [Map](#) object that shows the region that is specified by the `MapSpan` object. The `MapSpan` object is centered on the latitude and longitude represented by a `Location` object, and spans 0.01 latitude and 0.01 longitude degrees. For information about the `Location` class, see [Location and distance](#). For information about passing arguments in XAML, see [Pass arguments in XAML](#).

The result is that when the map is displayed, it's centered on a specific location, and spans a specific number of latitude and longitude degrees:



Create a MapSpan object

There are a number of approaches for creating `MapSpan` objects. A common approach is supply the required arguments to the `MapSpan` constructor. These are a latitude and longitude represented by a `Location` object, and `double` values that represent the degrees of latitude and longitude that are spanned by the `MapSpan`. For information about the `Location` class, see [Location and distance](#).

Alternatively, there are three methods in the `MapSpan` class that return new `MapSpan` objects:

1. `ClampLatitude` returns a `MapSpan` with the same `LongitudeDegrees` as the method's class instance, and a radius defined by its `north` and `south` arguments.
2. `FromCenterAndRadius` returns a `MapSpan` that is defined by its `Location` and `Distance` arguments.
3. `WithZoom` returns a `MapSpan` with the same center as the method's class instance, but with a radius multiplied by its `double` argument.

For information about the `Distance` struct, see [Location and distance](#).

Once a `MapSpan` has been created, the following properties can be accessed to retrieve data about it:

- `Center`, of type `Location`, which represents the location in the geographical center of the `MapSpan`.
- `LatitudeDegrees`, of type `double`, which represents the degrees of latitude that are spanned by the `MapSpan`.
- `LongitudeDegrees`, of type `double`, which represents the degrees of longitude that are spanned by the `MapSpan`.
- `Radius`, of type `Distance`, which represents the `MapSpan` radius.

Move the map

The `Map.MoveToRegion` method can be called to change the position and zoom level of a map. This method accepts a `MapSpan` argument that defines the region of the map to display, and its zoom level.

The following code shows an example of moving the displayed region on a map:

C#

```
using Microsoft.Maui.Maps;
using Microsoft.Maui.Controls.Maps.Map;
...

MapSpan mapSpan = MapSpan.FromCenterAndRadius(location,
Distance.FromKilometers(0.444));
map.MoveToRegion(mapSpan);
```

Zoom the map

The zoom level of a [Map](#) can be changed without altering its location. This can be accomplished using the map UI, or programatically by calling the `MoveToRegion` method

with a `MapSpan` argument that uses the current location as the `Location` argument:

C#

```
double zoomLevel = 0.5;
double latlongDegrees = 360 / (Math.Pow(2, zoomLevel));
if (map.VisibleRegion != null)
{
    map.MoveToRegion(new MapSpan(map.VisibleRegion.Center, latlongDegrees,
    latlongDegrees));
}
```

In this example, the `MoveToRegion` method is called with a `MapSpan` argument that specifies the current location of the map, via the `Map.VisibleRegion` property, and the zoom level as degrees of latitude and longitude. The overall result is that the zoom level of the map is changed, but its location isn't. An alternative approach for implementing zoom on a map is to use the `MapSpan.WithZoom` method to control the zoom factor.

❗ Important

Zooming a map, whether via the map UI or programatically, requires that the `Map.IsZoomEnabled` property is `true`. For more information about this property, see [Disable zoom](#).

Customize map behavior

The behavior of a `Map` can be customized by setting some of its properties, and by handling the `MapClicked` event.

❗ Note

Additional map behavior customization can be achieved by customizing its handler. For more information, see [Customize controls with handlers](#).

Show traffic data

The `Map` class defines a `IsTrafficEnabled` property of type `bool`. By default this property is `false`, which indicates that traffic data won't be overlaid on the map. When this property is set to `true`, traffic data is overlaid on the map:

XAML

```
<maps:Map IsTrafficEnabled="true" />
```

The equivalent C# code is:

C#

```
Map map = new Map
{
    IsTrafficEnabled = true
};
```

Disable scroll

The `Map` class defines a `IsScrollEnabled` property of type `bool`. By default this property is `true`, which indicates that the map is allowed to scroll. When this property is set to `false`, the map will not scroll:

XAML

```
<maps:Map IsScrollEnabled="false" />
```

The equivalent C# code is:

C#

```
Map map = new Map
{
    IsScrollEnabled = false
};
```

Disable zoom

The `Map` class defines a `IsZoomEnabled` property of type `bool`. By default this property is `true`, which indicates that zoom can be performed on the map. When this property is set to `false`, the map can't be zoomed:

XAML

```
<maps:Map IsZoomEnabled="false" />
```

The equivalent C# code is:

C#

```
Map map = new Map
{
    IsZoomEnabled = false
};
```

Show the user's location

The `Map` class defines a `IsShowingUser` property of type `bool`. By default this property is `false`, which indicates that the map is not showing the user's current location. When this property is set to `true`, the map shows the user's current location:

XAML

```
<maps:Map IsShowingUser="true" />
```

The equivalent C# code is:

C#

```
Map map = new Map
{
    IsShowingUser = true
};
```

Important

Accessing the user's location requires location permissions to have been granted to the application. For more information, see [Platform configuration](#).

Map clicks

The `Map` class defines a `MapClicked` event that's fired when the map is tapped. The `MapClickedEventArgs` object that accompanies the event has a single property named `Location`, of type `Location`. When the event is fired, the `Location` property is set to the map location that was tapped. For information about the `Location` class, see [Location and distance](#).

The following code example shows an event handler for the `MapClicked` event:

C#


```
void OnMapClicked(object sender, MapClickedEventArgs e)
{
    System.Diagnostics.Debug.WriteLine($"MapClick: {e.Location.Latitude},
    {e.Location.Longitude}");
}
```

In this example, the `OnMapClicked` event handler outputs the latitude and longitude that represents the tapped map location. The event handler must be registered with the `MapClicked` event:

XAML

```
<maps:Map MapClicked="OnMapClicked" />
```

The equivalent C# code is:

C#

```
Map map = new Map();
map.MapClicked += OnMapClicked;
```

Location and distance

The `Microsoft.Maui.Devices.Sensors` namespace contains a `Location` class that's typically used when positioning a map and its pins. The `Microsoft.Maui.Maps` namespace contains a `Distance` struct that can optionally be used when positioning a map.

Location

The `Location` class encapsulates a location stored as latitude and longitude values. This class defines the following properties:

- `Accuracy`, of type `double?`, which represents the horizontal accuracy of the `Location`, in meters.
- `Altitude`, of type `double?`, which represents the altitude in meters in a reference system that's specified by the `AltitudeReferenceSystem` property.
- `AltitudeReferenceSystem`, of type `AltitudeReferenceSystem`, which specifies the reference system in which the altitude value is provided.
- `Course`, of type `double?`, which indicates the degrees value relative to true north.

- `IsFromMockProvider`, of type `bool`, which indicates if the location is from the GPS or from a mock location provider.
- `Latitude`, of type `double`, which represents the latitude of the location in decimal degrees.
- `Longitude`, of type `double`, which represents the longitude of the location in decimal degrees.
- `Speed`, of type `double?`, which represents the speed in meters per second.
- `Timestamp`, of type `DateTimeOffset`, which represents the timestamp when the `Location` was created.
- `VerticalAccuracy`, of type `double?`, which specifies the vertical accuracy of the `Location`, in meters.

`Location` objects are created with one of the `Location` constructor overloads, which typically require at a minimum latitude and longitude arguments specified as `double` values:

C#

```
Location location = new Location(36.9628066, -122.0194722);
```

When creating a `Location` object, the latitude value will be clamped between -90.0 and 90.0, and the longitude value will be clamped between -180.0 and 180.0.

ⓘ Note

The `GeographyUtils` class has a `ToRadians` extension method that converts a `double` value from degrees to radians, and a `ToDegrees` extension method that converts a `double` value from radians to degrees.

The `Location` class also has `CalculateDistance` methods that calculate the distance between two locations.

Distance

The `Distance` struct encapsulates a distance stored as a `double` value, which represents the distance in meters. This struct defines three read-only properties:

- `Kilometers`, of type `double`, which represents the distance in kilometers that's spanned by the `Distance`.

- `Meters`, of type `double`, which represents the distance in meters that's spanned by the `Distance`.
- `Miles`, of type `double`, which represents the distance in miles that's spanned by the `Distance`.

`Distance` objects can be created with the `Distance` constructor, which requires a meters argument specified as a `double`:

C#

```
Distance distance = new Distance(1450.5);
```

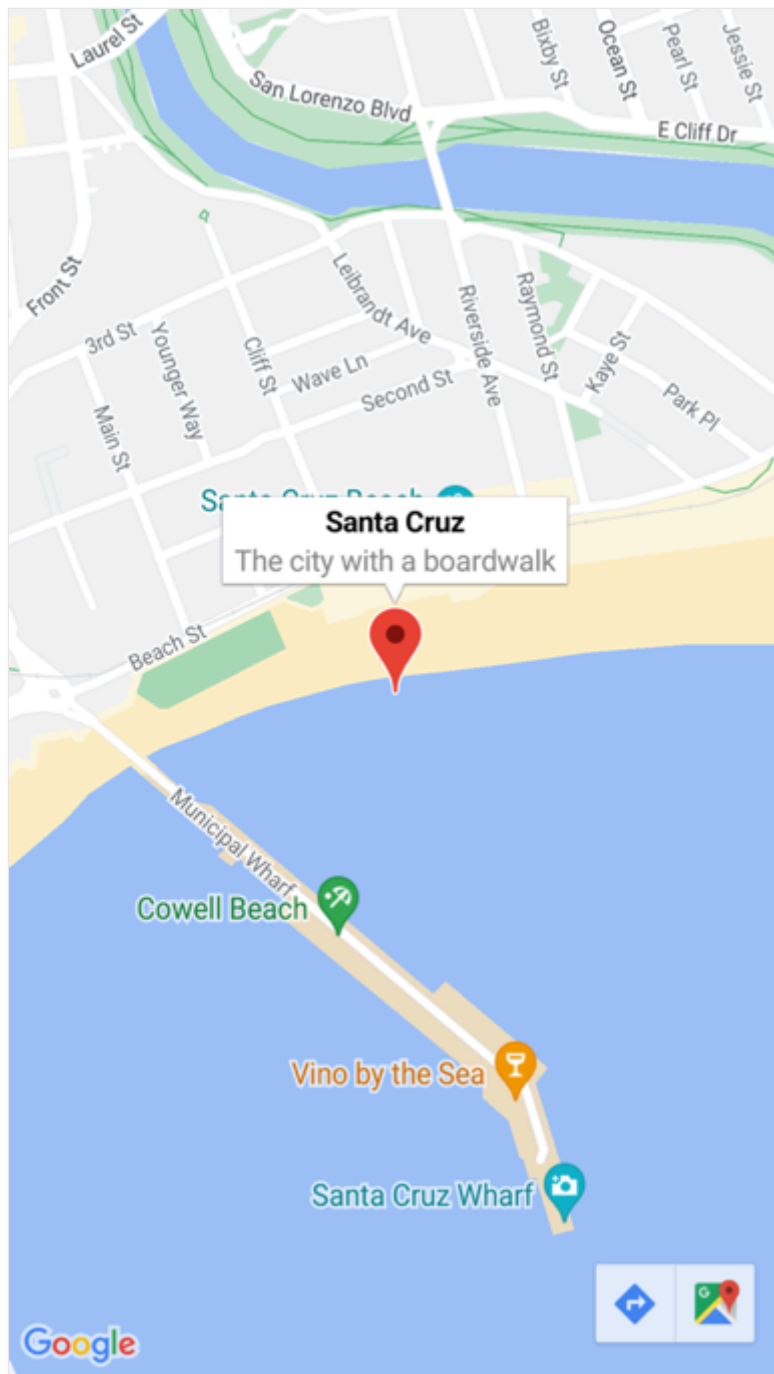
Alternatively, `Distance` objects can be created with the `FromKilometers`, `FromMeters`, `FromMiles`, and `BetweenPositions` factory methods:

C#

```
Distance distance1 = Distance.FromKilometers(1.45); // argument represents  
the number of kilometers  
Distance distance2 = Distance.FromMeters(1450.5);  // argument represents  
the number of meters  
Distance distance3 = Distance.FromMiles(0.969);    // argument represents  
the number of miles  
Distance distance4 = Distance.BetweenPositions(location1, location2);
```

Pins

The `Map` control allows locations to be marked with `Pin` objects. A `Pin` is a map marker that opens an information window when tapped:



When a `Pin` object is added to the `Map.Pins` collection, the pin is rendered on the map.

The `Pin` class has the following properties:

- `Address`, of type `string`, which typically represents the address for the pin location. However, it can be any `string` content, not just an address.
- `Label`, of type `string`, which typically represents the pin title.
- `Location`, of type `Location`, which represents the latitude and longitude of the pin.
- `Type`, of type `PinType`, which represents the type of pin.

These properties are backed by `BindableProperty` objects, which means a `Pin` can be the target of data bindings. For more information about data binding `Pin` objects, see [Display a pin collection](#).

In addition, the `Pin` class defines `MarkerClicked` and `InfoWindowClicked` events. The `MarkerClicked` event is fired when a pin is tapped, and the `InfoWindowClicked` event is fired when the information window is tapped. The `PinClickedEventArgs` object that accompanies both events has a single `HideInfoWindow` property, of type `bool`.

Display a pin

A `Pin` can be added to a `Map` in XAML:

XAML

```
<ContentPage ...
    xmlns:maps="http://schemas.microsoft.com/dotnet/2021/maui/maps"
    xmlns:sensors="clr-
namespace:Microsoft.Maui.Devices.Sensors;assembly=Microsoft.Maui.Essentials"
>
    <maps:Map x:Name="map">
        <x:Arguments>
            <maps:MapSpan>
                <x:Arguments>
                    <sensors:Location>
                        <x:Arguments>
                            <x:Double>36.9628066</x:Double>
                            <x:Double>-122.0194722</x:Double>
                        </x:Arguments>
                    </sensors:Location>
                    <x:Double>0.01</x:Double>
                    <x:Double>0.01</x:Double>
                </x:Arguments>
            </maps:MapSpan>
        </x:Arguments>
        <maps:Map.Pins>
            <maps:Pin Label="Santa Cruz"
                Address="The city with a boardwalk"
                Type="Place">
                <maps:Pin.Location>
                    <sensors:Location>
                        <x:Arguments>
                            <x:Double>36.9628066</x:Double>
                            <x:Double>-122.0194722</x:Double>
                        </x:Arguments>
                    </sensors:Location>
                </maps:Pin.Location>
            </maps:Pin>
        </maps:Map.Pins>
    </maps:Map>
</ContentPage>
```

This XAML creates a [Map](#) object that shows the region that is specified by the [MapSpan](#) object. The [MapSpan](#) object is centered on the latitude and longitude represented by a [Location](#) object, which extends 0.01 latitude and longitude degrees. A [Pin](#) object is added to the [Map.Pins](#) collection, and drawn on the [Map](#) at the location specified by its [Location](#) property. For information about the [Location](#) class, see [Location and distance](#). For information about passing arguments in XAML to objects that lack default constructors, see [Pass arguments in XAML](#).

The equivalent C# code is:

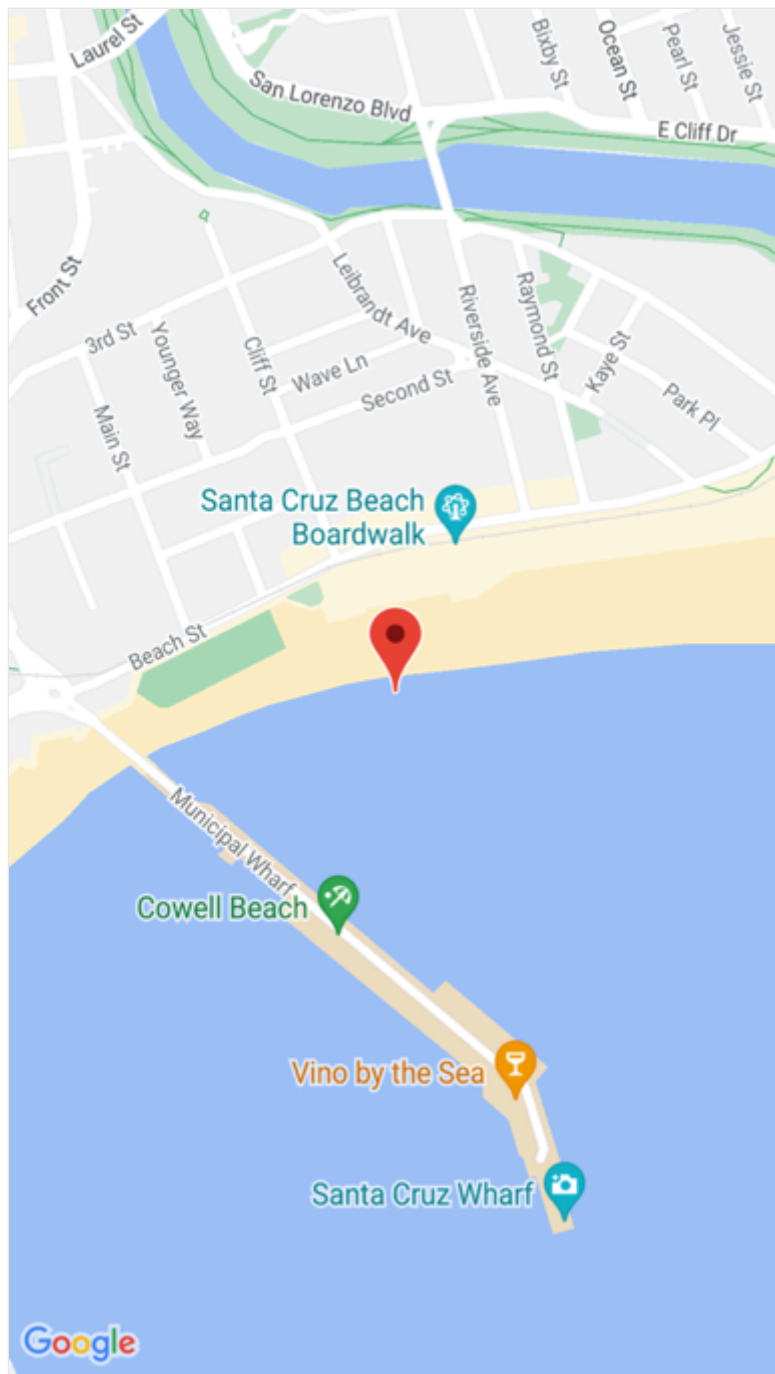
C#

```
using Microsoft.Maui.Controls.Maps;
using Microsoft.Maui.Maps;
using Map = Microsoft.Maui.Controls.Maps.Map;
...

Map map = new Map
{
    ...
};

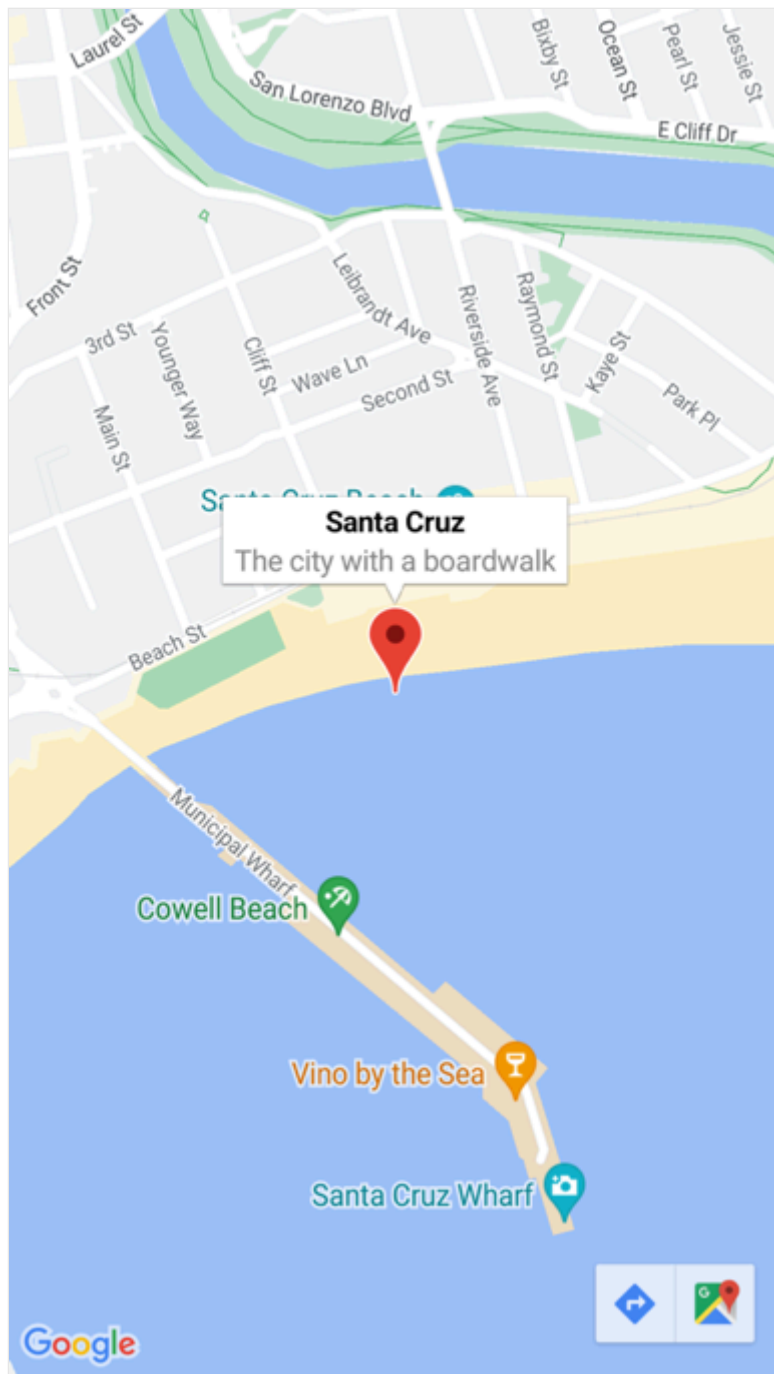
Pin pin = new Pin
{
    Label = "Santa Cruz",
    Address = "The city with a boardwalk",
    Type = PinType.Place,
    Location = new Location(36.9628066, -122.0194722)
};
map.Pins.Add(pin);
```

This example code results in a single pin being rendered on a map:



Interact with a pin

By default, when a `Pin` is tapped its information window is displayed:



Tapping elsewhere on the map closes the information window.

The `Pin` class defines a `MarkerClicked` event, which is fired when a `Pin` is tapped. It's not necessary to handle this event to display the information window. Instead, this event should be handled when there's a requirement to be notified that a specific pin has been tapped.

The `Pin` class also defines a `InfoWindowClicked` event that's fired when an information window is tapped. This event should be handled when there's a requirement to be notified that a specific information window has been tapped.

The following code shows an example of handling these events:

```
C#
```



```

using Microsoft.Maui.Controls.Maps;
using Microsoft.Maui.Maps;
using Map = Microsoft.Maui.Controls.Maps.Map;
...

Pin boardwalkPin = new Pin
{
    Location = new Location(36.9641949, -122.0177232),
    Label = "Boardwalk",
    Address = "Santa Cruz",
    Type = PinType.Place
};
boardwalkPin.MarkerClicked += async (s, args) =>
{
    args.HideInfoWindow = true;
    string pinName = ((Pin)s).Label;
    await DisplayAlert("Pin Clicked", $"{pinName} was clicked.", "Ok");
};

Pin wharfPin = new Pin
{
    Location = new Location(36.9571571, -122.0173544),
    Label = "Wharf",
    Address = "Santa Cruz",
    Type = PinType.Place
};
wharfPin.InfoWindowClicked += async (s, args) =>
{
    string pinName = ((Pin)s).Label;
    await DisplayAlert("Info Window Clicked", $"The info window was clicked for {pinName}.", "Ok");
};

```

The `PinClickedEventArgs` object that accompanies both events has a single `HideInfoWindow` property, of type `bool`. When this property is set to `true` inside an event handler, the information window will be hidden.

Pin types

`Pin` objects include a `Type` property, of type `PinType`, which represents the type of pin. The `PinType` enumeration defines the following members:

- `Generic`, represents a generic pin.
- `Place`, represents a pin for a place.
- `SavedPin`, represents a pin for a saved location.
- `SearchResult`, represents a pin for a search result.

However, setting the `Pin.Type` property to any `PinType` member does not change the appearance of the rendered pin. Instead, you must customize the `Pin` handler to customize pin appearance. For more information about handler customization, see [Customize controls with handlers](#).

Display a pin collection

The `Map` class defines the following bindable properties:

- `ItemsSource`, of type `IEnumerable`, which specifies the collection of `IEnumerable` pin items to be displayed.
- `ItemTemplate`, of type `DataTemplate`, which specifies the `DataTemplate` to apply to each item in the collection of displayed pins.
- `ItemTemplateSelector`, of type `DataTemplateSelector`, which specifies the `DataTemplateSelector` that will be used to choose a `DataTemplate` for a pin at runtime.

Important

The `ItemTemplate` property takes precedence when both the `ItemTemplate` and `ItemTemplateSelector` properties are set.

A `Map` can be populated with pins by using data binding to bind its `ItemsSource` property to an `IEnumerable` collection:

XAML

```
<ContentPage ...

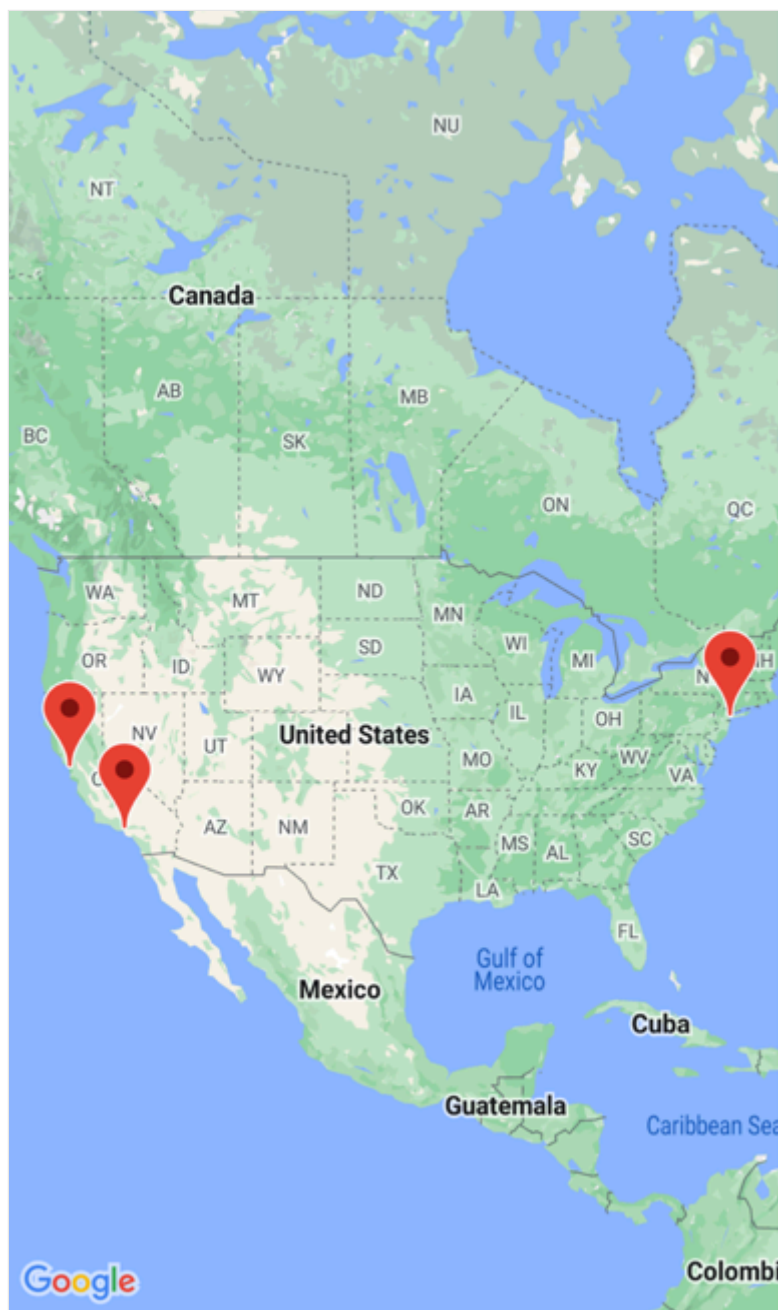
xmlns:maps="http://schemas.microsoft.com/dotnet/2021/maui/maps">
    <Grid>
        ...
        <maps:Map x:Name="map"
            ItemsSource="{Binding Positions}">
            <maps:Map.ItemTemplate>
                <DataTemplate>
                    <maps:Pin Location="{Binding Location}"
                        Address="{Binding Address}"
                        Label="{Binding Description}" />
                </DataTemplate>
            </maps:Map.ItemTemplate>
        </maps:Map>
        ...
    </Grid>
</ContentPage>
```

```
</Grid>
</ContentPage>
```

The `ItemsSource` property data binds to the `Positions` property of the connected viewmodel, which returns an `ObservableCollection` of `Position` objects, which is a custom type. Each `Position` object defines `Address` and `Description` properties, of type `string`, and a `Location` property, of type `Location`.

The appearance of each item in the `IEnumerable` collection is defined by setting the `ItemTemplate` property to a `DataTemplate` that contains a `Pin` object that data binds to appropriate properties.

The following screenshot shows a `Map` displaying a `Pin` collection using data binding:



Choose item appearance at runtime

The appearance of each item in the `IEnumerable` collection can be chosen at runtime, based on the item value, by setting the `ItemTemplateSelector` property to a [DataTemplateSelector](#):

XAML

```
<ContentPage ...
    xmlns:templates="clr-namespace:WorkingWithMaps.Templates"

xmlns:maps="http://schemas.microsoft.com/dotnet/2021/maui/maps">
    <ContentPage.Resources>
        <templates:MapItemTemplateSelector x:Key="MapItemTemplateSelector">
            <templates:MapItemTemplateSelector.DefaultTemplate>
                <DataTemplate>
                    <maps:Pin Location="{Binding Location}"
                        Address="{Binding Address}"
                        Label="{Binding Description}" />
                </DataTemplate>
            </templates:MapItemTemplateSelector.DefaultTemplate>
            <templates:MapItemTemplateSelector.SanFranTemplate>
                <DataTemplate>
                    <maps:Pin Location="{Binding Location}"
                        Address="{Binding Address}"
                        Label="Xamarin!" />
                </DataTemplate>
            </templates:MapItemTemplateSelector.SanFranTemplate>
        </templates:MapItemTemplateSelector>
    </ContentPage.Resources>

    <Grid>
        ...
        <maps:Map x:Name="map"
            ItemsSource="{Binding Positions}"
            ItemTemplateSelector="{StaticResource
MapItemTemplateSelector}">
        ...
    </Grid>
</ContentPage>
```

The following example shows the `MapItemTemplateSelector` class:

C#

```
using WorkingWithMaps.Models;

namespace WorkingWithMaps.Templates;

public class MapItemTemplateSelector : DataTemplateSelector
{
```

```

public DataTemplate DefaultTemplate { get; set; }
public DataTemplate SanFranTemplate { get; set; }

protected override DataTemplate OnSelectTemplate(object item,
BindableObject container)
{
    return ((Position)item).Address.Contains("San Francisco") ?
SanFranTemplate : DefaultTemplate;
}
}

```

The `MapItemTemplateSelector` class defines `DefaultTemplate` and `SanFranTemplate` `DataTemplate` properties that are set to different data templates. The `OnSelectTemplate` method returns the `SanFranTemplate`, which displays "Xamarin" as a label when a `Pin` is tapped, when the item has an address that contains "San Francisco". When the item doesn't have an address that contains "San Francisco", the `OnSelectTemplate` method returns the `DefaultTemplate`.

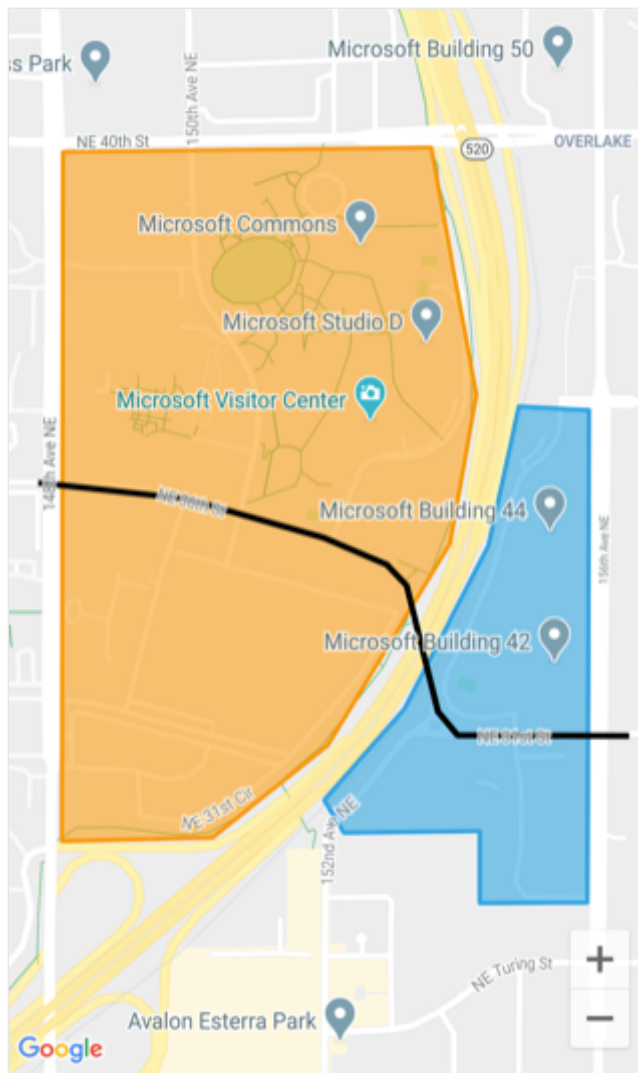
ⓘ Note

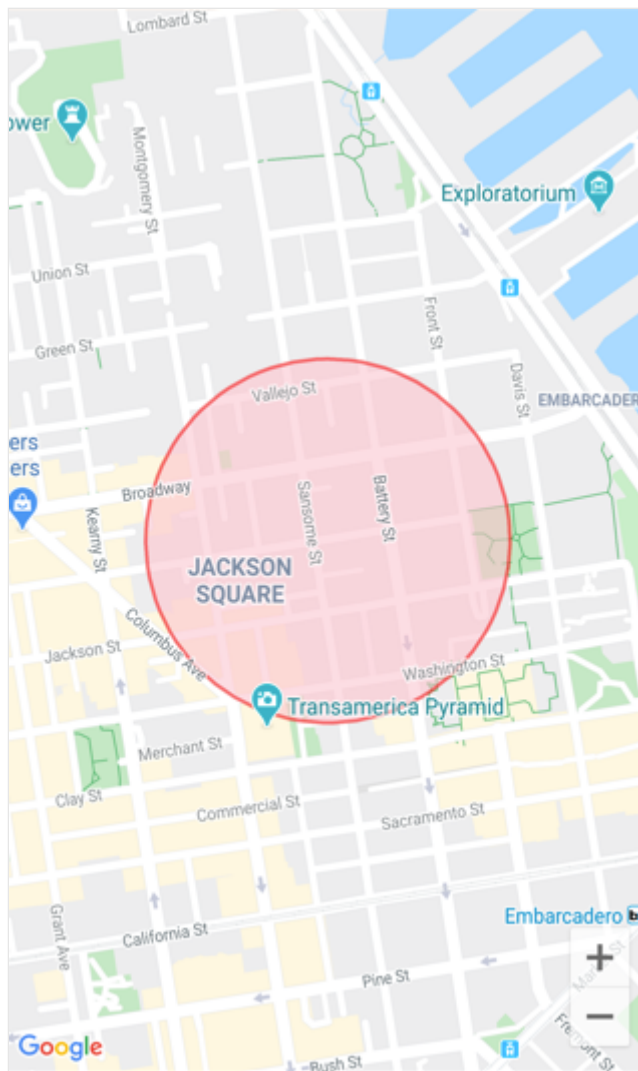
A use case for this functionality is binding properties of subclassed `Pin` objects to different properties, based on the `Pin` sub-type.

For more information about data template selectors, see [Create a DataTemplateSelector](#).

Polygons, polylines, and circles

`Polygon`, `Polyline`, and `Circle` elements allow you to highlight specific areas on a map. A `Polygon` is a fully enclosed shape that can have a stroke and fill color. A `Polyline` is a line that does not fully enclose an area. A `Circle` highlights a circular area of the map:





The `Polygon`, `Polyline`, and `Circle` classes derive from the `MapElement` class, which exposes the following bindable properties:

- `StrokeColor` is a `Color` object that determines the line color.
- `StrokeWidth` is a `float` object that determines the line width.

The `Polygon` class defines an additional bindable property:

- `FillColor` is a `Color` object that determines the polygon's background color.

In addition, the `Polygon` and `Polyline` classes both define a `GeoPath` property, which is a list of `Location` objects that specify the points of the shape.

The `Circle` class defines the following bindable properties:

- `Center` is a `Location` object that defines the center of the circle, in latitude and longitude.
- `Radius` is a `Distance` object that defines the radius of the circle in meters, kilometers, or miles.
- `FillColor` is a `Color` property that determines the color within the circle perimeter.

Create a polygon

A `Polygon` object can be added to a map by instantiating it and adding it to the map's `MapElements` collection:

XAML

```
<ContentPage ...
    xmlns:maps="http://schemas.microsoft.com/dotnet/2021/maui/maps"
    xmlns:sensors="clr-
namespace:Microsoft.Maui.Devices.Sensors;assembly=Microsoft.Maui.Essentials"
>
    <maps:Map>
        <maps:Map.MapElements>
            <maps:Polygon StrokeColor="#FF9900"
                StrokeWidth="8"
                FillColor="#88FF9900">
                <maps:Polygon.Geopath>
                    <sensors:Location>
                        <x:Arguments>
                            <x:Double>47.6458676</x:Double>
                            <x:Double>-122.1356007</x:Double>
                        </x:Arguments>
                    </sensors:Location>
                    <sensors:Location>
                        <x:Arguments>
                            <x:Double>47.6458097</x:Double>
                            <x:Double>-122.142789</x:Double>
                        </x:Arguments>
                    </sensors:Location>
                    ...
                </maps:Polygon.Geopath>
            </maps:Polygon>
        </maps:Map.MapElements>
    </maps:Map>
</ContentPage>
```

The equivalent C# code is:

C#

```
using Microsoft.Maui.Controls.Maps;
using Microsoft.Maui.Maps;
using Map = Microsoft.Maui.Controls.Maps.Map;
...

Map map = new Map();

// Instantiate a polygon
Polygon polygon = new Polygon
{
```



```

        StrokeWidth = 8,
        StrokeColor = Color.FromArgb("#1BA1E2"),
        FillColor = Color.FromArgb("#881BA1E2"),
        Geopath =
        {
            new Location(47.6368678, -122.137305),
            new Location(47.6368894, -122.134655),
            ...
        }
    };

    // Add the polygon to the map's MapElements collection
    map.MapElements.Add(polygon);

```

The `StrokeColor` and `StrokeWidth` properties are specified to set the polygon's outline. In this example, the `FillColor` property value matches the `StrokeColor` property value but has an alpha value specified to make it transparent, allowing the underlying map to be visible through the shape. The `GeoPath` property contains a list of `Location` objects defining the geographic coordinates of the polygon points. A `Polygon` object is rendered on the map once it has been added to the `MapElements` collection of the `Map`.

ⓘ Note

A `Polygon` is a fully enclosed shape. The first and last points will automatically be connected if they do not match.

Create a polyline

A `Polyline` object can be added to a map by instantiating it and adding it to the map's `MapElements` collection:

XAML

```

<ContentPage ...
    xmlns:maps="http://schemas.microsoft.com/dotnet/2021/maui/maps"
    xmlns:sensors="clr-
namespace:Microsoft.Maui.Devices.Sensors;assembly=Microsoft.Maui.Essentials"
>
    <maps:Map>
        <maps:Map.MapElements>
            <maps:Polyline StrokeColor="Black"
                StrokeWidth="12">
                <maps:Polyline.Geopath>
                    <sensors:Location>
                        <x:Arguments>
                            <x:Double>47.6381401</x:Double>

```

```

        <x:Double>-122.1317367</x:Double>
      </x:Arguments>
    </sensors:Location>
  </sensors:Location>
  <x:Arguments>
    <x:Double>47.6381473</x:Double>
    <x:Double>-122.1350841</x:Double>
  </x:Arguments>
</sensors:Location>
...
</maps:Polyline.Geopath>
</maps:Polyline>
</maps:Map.MapElements>
</maps:Map>
</ContentPage>

```

The equivalent C# code is:

C#

```

using Microsoft.Maui.Controls.Maps;
using Microsoft.Maui.Maps;
using Map = Microsoft.Maui.Controls.Maps.Map;
...

Map map = new Map();

// instantiate a polyline
Polyline polyline = new Polyline
{
    StrokeColor = Colors.Blue,
    StrokeWidth = 12,
    Geopath =
    {
        new Location(47.6381401, -122.1317367),
        new Location(47.6381473, -122.1350841),
        ...
    }
};

// Add the Polyline to the map's MapElements collection
map.MapElements.Add(polyline);

```

The `StrokeColor` and `StrokeWidth` properties are specified to set the line appearance. The `GeoPath` property contains a list of `Location` objects defining the geographic coordinates of the polyline points. A `Polyline` object is rendered on the map once it has been added to the `MapElements` collection of the [Map](#).

Create a circle

A `Circle` object can be added to a map by instantiating it and adding it to the map's `MapElements` collection:

XAML

```
<ContentPage ...
    xmlns:maps="http://schemas.microsoft.com/dotnet/2021/maui/maps"
    xmlns:sensors="clr-
namespace:Microsoft.Maui.Devices.Sensors;assembly=Microsoft.Maui.Essentials"
>
    <maps:Map>
        <maps:Map.MapElements>
            <maps:Circle StrokeColor="#88FF0000"
                StrokeWidth="8"
                FillColor="#88FFC0CB">
                <maps:Circle.Center>
                    <sensors:Location>
                        <x:Arguments>
                            <x:Double>37.79752</x:Double>
                            <x:Double>-122.40183</x:Double>
                        </x:Arguments>
                    </sensors:Location>
                </maps:Circle.Center>
                <maps:Circle.Radius>
                    <maps:Distance>
                        <x:Arguments>
                            <x:Double>250</x:Double>
                        </x:Arguments>
                    </maps:Distance>
                </maps:Circle.Radius>
            </maps:Circle>
        </maps:Map.MapElements>
    </maps:Map>
</ContentPage>
```

The equivalent C# code is:

C#

```
using Microsoft.Maui.Controls.Maps;
using Microsoft.Maui.Maps;
using Map = Microsoft.Maui.Controls.Maps.Map;

Map map = new Map();

// Instantiate a Circle
Circle circle = new Circle
{
    Center = new Location(37.79752, -122.40183),
    Radius = new Distance(250),
    StrokeColor = Color.FromArgb("#88FF0000"),
    StrokeWidth = 8,
```

```
FillColor = Color.FromArgb("#88FFC0CB")
};

// Add the Circle to the map's MapElements collection
map.MapElements.Add(circle);
```

The location of the `Circle` on the Map is determined by the value of the `Center` and `Radius` properties. The `Center` property defines the center of the circle, in latitude and longitude, while the `Radius` property defines the radius of the circle in meters. The `StrokeColor` and `StrokeWidth` properties are specified to set the circle's outline. The `FillColor` property value specifies the color within the circle perimeter. In this example, both of the color values specify an alpha channel, allowing the underlying map to be visible through the circle. The `Circle` object is rendered on the map once it has been added to the `MapElements` collection of the [Map](#).

ⓘ Note

The `GeographyUtils` class has a `ToCircumferencePositions` extension method that converts a `Circle` object (that defines `Center` and `Radius` property values) to a list of `Location` objects that make up the latitude and longitude coordinates of the circle perimeter.

Geocoding and geolocation

The `Geocoding` class, in the `Microsoft.Maui.Devices.Sensors` namespace, can be used to geocode a placemark to positional coordinates and reverse geocode coordinates to a placemark. For more information, see [Geocoding](#).

The `Geolocation` class, in the `Microsoft.Maui.Devices.Sensors` namespace, can be used to retrieve the device's current geolocation coordinates. For more information, see [Geolocation](#).

Launch the native map app

The native map app on each platform can be launched from a .NET MAUI app by the `Launcher` class. This class enables an app to open another app through its custom URI scheme. The launcher functionality can be invoked with the `OpenAsync` method, passing in a `string` or `Uri` argument that represents the custom URL scheme to open. For more information about the `Launcher` class, see [Launcher](#).

ⓘ Note

An alternative to using the `Launcher` class is to use `Map` class from the `Microsoft.Maui.ApplicationModel` namespace. For more information, see [Map](#).

The maps app on each platform uses a unique custom URI scheme. For information about the maps URI scheme on iOS, see [Map Links](#) on developer.apple.com. For information about the maps URI scheme on Android, see [Maps Developer Guide](#) and [Google Maps Intents for Android](#) on developers.android.com. For information about the maps URI scheme on Windows, see [Launch the Windows Maps app](#).

Launch the map app at a specific location

A location in the native maps app can be opened by adding appropriate query parameters to the custom URI scheme for each map app:

C#

```
if (DeviceInfo.Current.Platform == DevicePlatform.iOS ||
    DeviceInfo.Current.Platform == DevicePlatform.MacCatalyst)
{
    //
    https://developer.apple.com/library/ios/featuredarticles/iPhoneURLScheme_Reference/MapLinks/MapLinks.html
    await Launcher.OpenAsync("http://maps.apple.com/?
q=394+Pacific+Ave+San+Francisco+CA");
}
else if (DeviceInfo.Current.Platform == DevicePlatform.Android)
{
    // opens the Maps app directly
    await Launcher.OpenAsync("geo:0,0?q=394+Pacific+Ave+San+Francisco+CA");
}
else if (DeviceInfo.Current.Platform == DevicePlatform.WinUI)
{
    await Launcher.OpenAsync("bingmaps:?where=394 Pacific Ave San Francisco
CA");
}
```

This example code results in the native map app being launched on each platform, with the map centered on a pin representing the specified location.

Launch the map app with directions

The native maps app can be launched displaying directions, by adding appropriate query parameters to the custom URI scheme for each map app:

C#

```
if (DeviceInfo.Current.Platform == DevicePlatform.iOS ||
DeviceInfo.Current.Platform == DevicePlatform.MacCatalyst)
{
    //
    https://developer.apple.com/library/ios/featuredarticles/iPhoneURLScheme_Reference/MapLinks/MapLinks.html
    await Launcher.OpenAsync("http://maps.apple.com/?
daddr=San+Francisco,+CA&saddr=cupertino");
}
else if (DeviceInfo.Current.Platform == DevicePlatform.Android)
{
    // opens the 'task chooser' so the user can pick Maps, Chrome or other
    mapping app
    await Launcher.OpenAsync("http://maps.google.com/?
daddr=San+Francisco,+CA&saddr=Mountain+View");
}
else if (DeviceInfo.Current.Platform == DevicePlatform.WinUI)
{
    await Launcher.OpenAsync("bingmaps:?rtp=adr.394 Pacific Ave San
Francisco CA~adr.One Microsoft Way Redmond WA 98052");
}
```

This example code results in the native map app being launched on each platform, with the map centered on a route between the specified locations.

Picker

Article • 08/30/2024

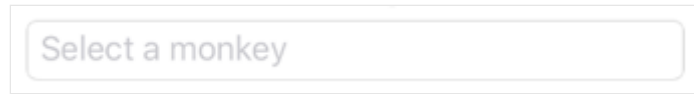
The .NET Multi-platform App UI (.NET MAUI) [Picker](#) displays a short list of items, from which the user can select an item.

[Picker](#) defines the following properties:

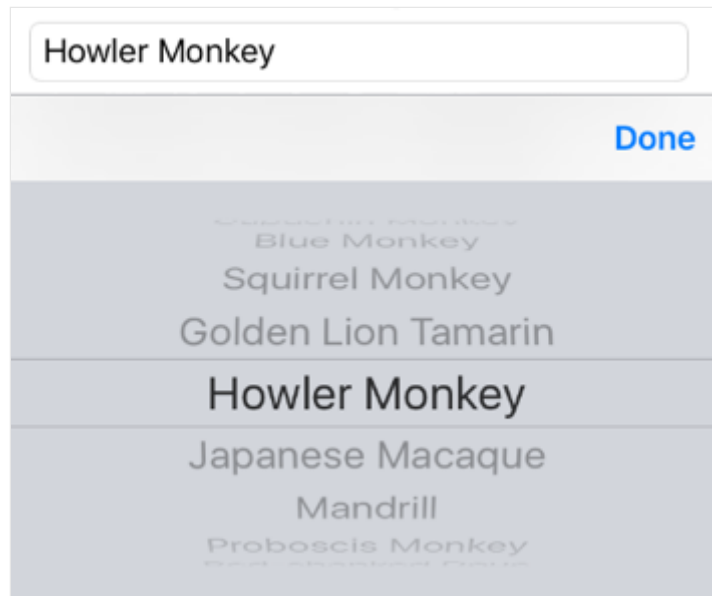
- `CharacterSpacing`, of type `double`, is the spacing between characters of the item displayed by the [Picker](#).
- `FontAttributes` of type `FontAttributes`, which defaults to `FontAttributes.None`.
- `FontAutoScalingEnabled`, of type `bool`, which determines whether the text respects scaling preferences set in the operating system. The default value of this property is `true`.
- `FontFamily` of type `string`, which defaults to `null`.
- `FontSize` of type `double`, which defaults to `-1.0`.
- `HorizontalTextAlignment`, of type [TextAlignment](#), is the horizontal alignment of the text displayed by the [Picker](#).
- `ItemsSource` of type `IList`, the source list of items to display, which defaults to `null`.
- `SelectedIndex` of type `int`, the index of the selected item, which defaults to `-1`.
- `SelectedItem` of type `object`, the selected item, which defaults to `null`.
- `ItemDisplayBinding`, of type [BindingBase](#), selects the property that will be displayed for each object in the list of items, if the `ItemsSource` is a complex object. For more information, see [Populate a Picker with data using data binding](#).
- `TextColor` of type [Color](#), the color used to display the text.
- `TextTransform`, of type `TextTransform`, which defines whether to transform the casing of text.
- `Title` of type `string`, which defaults to `null`.
- `TitleColor` of type [Color](#), the color used to display the `Title` text.
- `VerticalTextAlignment`, of type [TextAlignment](#), is the vertical alignment of the text displayed by the [Picker](#).

All of the properties, with the exception of `ItemDisplayBinding`, are backed by [BindableProperty](#) objects, which means that they can be styled, and the properties can be targets of data bindings. The `SelectedIndex` and `SelectedItem` properties have a default binding mode of `BindingMode.TwoWay`, which means that they can be targets of data bindings in an application that uses the Model-View-ViewModel (MVVM) pattern. For information about setting font properties, see [Fonts](#).

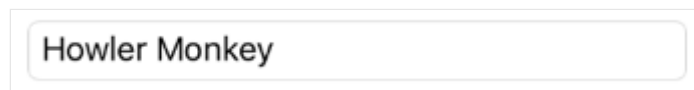
A [Picker](#) doesn't show any data when it's first displayed. Instead, the value of its `Title` property is shown as a placeholder, as shown in the following iOS screenshot:



When the [Picker](#) gains focus, its data is displayed and the user can select an item:



The [Picker](#) fires a `SelectedIndexChanged` event when the user selects an item. Following selection, the selected item is displayed by the [Picker](#):



There are two techniques for populating a [Picker](#) with data:

- Setting the `ItemsSource` property to the data to be displayed. This is the recommended technique for adding data to a [Picker](#). For more information, see [Set the ItemsSource property](#).
- Adding the data to be displayed to the `Items` collection. For more information, see [Add data to the Items collection](#).

Set the ItemsSource property

A [Picker](#) can be populated with data by setting its `ItemsSource` property to an `IList` collection. Each item in the collection must be of, or derived from, type `object`. Items can be added in XAML by initializing the `ItemsSource` property from an array of items:

XAML


```

<Picker x:Name="picker"
        Title="Select a monkey">
    <Picker.ItemsSource>
        <x:Array Type="{x:Type x:String}">
            <x:String>Baboon</x:String>
            <x:String>Capuchin Monkey</x:String>
            <x:String>Blue Monkey</x:String>
            <x:String>Squirrel Monkey</x:String>
            <x:String>Golden Lion Tamarin</x:String>
            <x:String>Howler Monkey</x:String>
            <x:String>Japanese Macaque</x:String>
        </x:Array>
    </Picker.ItemsSource>
</Picker>

```

ⓘ Note

The `x:Array` element requires a `Type` attribute indicating the type of the items in the array.

The equivalent C# code is:

C#

```

var monkeyList = new List<string>();
monkeyList.Add("Baboon");
monkeyList.Add("Capuchin Monkey");
monkeyList.Add("Blue Monkey");
monkeyList.Add("Squirrel Monkey");
monkeyList.Add("Golden Lion Tamarin");
monkeyList.Add("Howler Monkey");
monkeyList.Add("Japanese Macaque");

Picker picker = new Picker { Title = "Select a monkey" };
picker.ItemsSource = monkeyList;

```

Respond to item selection

A `Picker` supports selection of one item at a time. When a user selects an item, the `SelectedIndexChanged` event fires, the `SelectedIndex` property is updated to an integer representing the index of the selected item in the list, and the `SelectedItem` property is updated to the `object` representing the selected item. The `SelectedIndex` property is a zero-based number indicating the item the user selected. If no item is selected, which is the case when the `Picker` is first created and initialized, `SelectedIndex` will be -1.

❗ Note

Item selection behavior in a [Picker](#) can be customized on iOS with a platform-specific. For more information, see [Picker item selection on iOS](#).

The following XAML example shows how to retrieve the `SelectedItem` property value from the [Picker](#):

XAML

```
<Label Text="{Binding Source={x:Reference picker}, Path=SelectedItem}" />
```

The equivalent C# code is:

C#

```
Label monkeyNameLabel = new Label();  
monkeyNameLabel.SetBinding(Label.TextProperty, new Binding("SelectedItem",  
source: picker));
```

In addition, an event handler can be executed when the `SelectedIndexChanged` event fires:

C#

```
void OnPickerSelectedIndexChanged(object sender, EventArgs e)  
{  
    var picker = (Picker)sender;  
    int selectedIndex = picker.SelectedIndex;  
  
    if (selectedIndex != -1)  
    {  
        monkeyNameLabel.Text = (string)picker.ItemsSource[selectedIndex];  
    }  
}
```

In this example, the event handler obtains the `SelectedIndex` property value, and uses the value to retrieve the selected item from the `ItemsSource` collection. This is functionally equivalent to retrieving the selected item from the `SelectedItem` property. Each item in the `ItemsSource` collection is of type `object`, and so must be cast to a `string` for display.

❗ Note

A [Picker](#) can be initialized to display a specific item by setting the `SelectedIndex` or `SelectedItem` properties. However, these properties must be set after initializing the `ItemsSource` collection.

Populate a Picker with data using data binding

A [Picker](#) can be also populated with data by using data binding to bind its `ItemsSource` property to an `IList` collection. In XAML this is achieved with the `Binding` markup extension:

XAML

```
<Picker Title="Select a monkey"
        ItemsSource="{Binding Monkeys}"
        ItemDisplayBinding="{Binding Name}" />
```

The equivalent C# code is shown below:

C#

```
Picker picker = new Picker { Title = "Select a monkey" };
picker.SetBinding(Picker.ItemsSourceProperty, "Monkeys");
picker.ItemDisplayBinding = new Binding("Name");
```

In this example, the `ItemsSource` property data binds to the `Monkeys` property of the binding context, which returns an `IList<Monkey>` collection. The following code example shows the `Monkey` class, which contains four properties:

C#

```
public class Monkey
{
    public string Name { get; set; }
    public string Location { get; set; }
    public string Details { get; set; }
    public string ImageUrl { get; set; }
}
```

When binding to a list of objects, the [Picker](#) must be told which property to display from each object. This is achieved by setting the `ItemDisplayBinding` property to the required property from each object. In the code examples above, the [Picker](#) is set to display each `Monkey.Name` property value.

Respond to item selection

Data binding can be used to set an object to the `SelectedItem` property value when it changes:

XAML

```
<Picker Title="Select a monkey"
        ItemsSource="{Binding Monkeys}"
        ItemDisplayBinding="{Binding Name}"
        SelectedItem="{Binding SelectedMonkey}" />
<Label Text="{Binding SelectedMonkey.Name}" ... />
<Label Text="{Binding SelectedMonkey.Location}" ... />
<Image Source="{Binding SelectedMonkey.ImageUrl}" ... />
<Label Text="{Binding SelectedMonkey.Details}" ... />
```

The equivalent C# code is:

C#

```
Picker picker = new Picker { Title = "Select a monkey" };
picker.SetBinding(Picker.ItemsSourceProperty, "Monkeys");
picker.SetBinding(Picker.SelectedItemProperty, "SelectedMonkey");
picker.ItemDisplayBinding = new Binding("Name");

Label nameLabel = new Label { ... };
nameLabel.SetBinding(Label.TextProperty, "SelectedMonkey.Name");

Label locationLabel = new Label { ... };
locationLabel.SetBinding(Label.TextProperty, "SelectedMonkey.Location");

Image image = new Image { ... };
image.SetBinding(Image.SourceProperty, "SelectedMonkey.ImageUrl");

Label detailsLabel = new Label();
detailsLabel.SetBinding(Label.TextProperty, "SelectedMonkey.Details");
```

The `SelectedItem` property data binds to the `SelectedMonkey` property of the binding context, which is of type `Monkey`. Therefore, when the user selects an item in the `Picker`, the `SelectedMonkey` property will be set to the selected `Monkey` object. The `SelectedMonkey` object data is displayed in the user interface by `Label` and `Image` views.

ⓘ Note

The `SelectedItem` and `SelectedIndex` properties both support two-way bindings by default.

Add data to the Items collection

An alternative process for populating a [Picker](#) with data is to add the data to be displayed to the read-only `Items` collection, which is of type `IList<string>`. Each item in the collection must be of type `string`. Items can be added in XAML by initializing the `Items` property with a list of `x:String` items:

XAML

```
<Picker Title="Select a monkey">
  <Picker.Items>
    <x:String>Baboon</x:String>
    <x:String>Capuchin Monkey</x:String>
    <x:String>Blue Monkey</x:String>
    <x:String>Squirrel Monkey</x:String>
    <x:String>Golden Lion Tamarin</x:String>
    <x:String>Howler Monkey</x:String>
    <x:String>Japanese Macaque</x:String>
  </Picker.Items>
</Picker>
```

The equivalent C# code is:

C#

```
Picker picker = new Picker { Title = "Select a monkey" };
picker.Items.Add("Baboon");
picker.Items.Add("Capuchin Monkey");
picker.Items.Add("Blue Monkey");
picker.Items.Add("Squirrel Monkey");
picker.Items.Add("Golden Lion Tamarin");
picker.Items.Add("Howler Monkey");
picker.Items.Add("Japanese Macaque");
```

In addition to adding data using the `Items.Add` method, data can also be inserted into the collection by using the `Items.Insert` method.

Respond to item selection

A [Picker](#) supports selection of one item at a time. When a user selects an item, the `SelectedIndexChanged` event fires, and the `SelectedIndex` property is updated to an integer representing the index of the selected item in the list. The `SelectedIndex` property is a zero-based number indicating the item that the user selected. If no item is selected, which is the case when the [Picker](#) is first created and initialized, `SelectedIndex` will be -1.

ⓘ Note

Item selection behavior in a [Picker](#) can be customized on iOS with a platform-specific. For more information, see [Picker item selection on iOS](#).

The following code example shows the `OnPickerSelectedIndexChanged` event handler method, which is executed when the `SelectedIndexChanged` event fires:

C#

```
void OnPickerSelectedIndexChanged(object sender, EventArgs e)
{
    var picker = (Picker)sender;
    int selectedIndex = picker.SelectedIndex;

    if (selectedIndex != -1)
    {
        monkeyNameLabel.Text = picker.Items[selectedIndex];
    }
}
```

This method obtains the `SelectedIndex` property value, and uses the value to retrieve the selected item from the `Items` collection. Because each item in the `Items` collection is a `string`, they can be displayed by a [Label](#) without requiring a cast.

ⓘ Note

A [Picker](#) can be initialized to display a specific item by setting the `SelectedIndex` property. However, the `SelectedIndex` property must be set after initializing the `Items` collection.

ProgressBar

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [ProgressBar](#) indicates to users that the app is progressing through a lengthy activity. The progress bar is a horizontal bar that is filled to a percentage represented by a `double` value.

The appearance of a [ProgressBar](#) is platform-dependent, and the following screenshot shows a [ProgressBar](#) on Android:



[ProgressBar](#) defines two properties:

- `Progress` is a `double` value that represents the current progress as a value from 0 to 1. `Progress` values less than 0 will be clamped to 0, values greater than 1 will be clamped to 1. The default value of this property is 0.
- `ProgressColor` is a `Color` value that defines the color of the [ProgressBar](#).

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

[ProgressBar](#) also defines a `ProgressTo` method that animates the bar from its current value to a specified value. For more information, see [Animate a ProgressBar](#).

Create a ProgressBar

To indicate progress through a lengthy activity, create a [ProgressBar](#) object and set its properties to define its appearance.

The following XAML example shows how to display a [ProgressBar](#):

XAML

```
<ProgressBar Progress="0.5" />
```

The equivalent C# code is:

C#

```
ProgressBar progressBar = new ProgressBar { Progress = 0.5 };
```

Warning

Do not use unconstrained horizontal layout options such as `Center`, `Start`, or `End` with `ProgressBar`. Keep the default `HorizontalOptions` value of `Fill`.

The following XAML example shows how to change the color of a `ProgressBar`:

XAML

```
<ProgressBar Progress="0.5"
              ProgressColor="Orange" />
```

The equivalent C# code is:

C#

```
ProgressBar progressBar = new ProgressBar
{
    Progress = 0.5,
    ProgressColor = Colors.Orange
};
```

Animate a ProgressBar

The `ProgressTo` method animates the `ProgressBar` from its current `Progress` value to a provided value over time. The method accepts a `double` progress value, a `uint` duration in milliseconds, an `Easing` enum value and returns a `Task<bool>`. The following example demonstrates how to animate a `ProgressBar`:

C#

```
// animate to 75% progress over 500 milliseconds with linear easing
await progressBar.ProgressTo(0.75, 500, Easing.Linear);
```

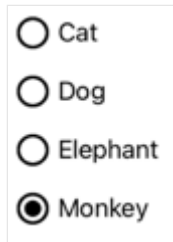
For more information about the `Easing` enumeration, see [Easing functions](#).

RadioButton

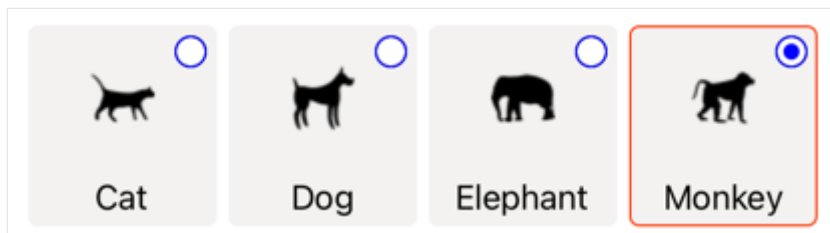
Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [RadioButton](#) is a type of button that allows users to select one option from a set. Each option is represented by one radio button, and you can only select one radio button in a group. By default, each [RadioButton](#) displays text:



However, on some platforms a [RadioButton](#) can display a [View](#), and on all platforms the appearance of each [RadioButton](#) can be redefined with a [ControlTemplate](#):



[RadioButton](#) defines the following properties:

- `Content`, of type `object`, which defines the `string` or [View](#) to be displayed by the [RadioButton](#).
- `IsChecked`, of type `bool`, which defines whether the [RadioButton](#) is checked. This property uses a `TwoWay` binding, and has a default value of `false`.
- `GroupName`, of type `string`, which defines the name that specifies which [RadioButton](#) controls are mutually exclusive. This property has a default value of `null`.
- `Value`, of type `object`, which defines an optional unique value associated with the [RadioButton](#).
- `BorderColor`, of type [Color](#), which defines the border stroke color.
- `BorderWidth`, of type `double`, which defines the width of the [RadioButton](#) border.
- `CharacterSpacing`, of type `double`, which defines the spacing between characters of any displayed text.
- `CornerRadius`, of type `int`, which defines the corner radius of the [RadioButton](#).

- `FontAttributes`, of type `FontAttributes`, which determines text style.
- `FontAutoScalingEnabled`, of type `bool`, which defines whether an app's UI reflects text scaling preferences set in the operating system. The default value of this property is `true`.
- `FontFamily`, of type `string`, which defines the font family.
- `FontSize`, of type `double`, which defines the font size.
- `TextColor`, of type `Color`, which defines the color of any displayed text.
- `TextTransform`, of type `TextTransform`, which defines the casing of any displayed text.

These properties are backed by `BindableProperty` objects, which means that they can be targets of data bindings, and styled.

`RadioButton` also defines a `CheckedChanged` event that's raised when the `IsChecked` property changes, either through user or programmatic manipulation. The `CheckedChangedEventArgs` object that accompanies the `CheckedChanged` event has a single property named `Value`, of type `bool`. When the event is raised, the value of the `CheckedChangedEventArgs.Value` property is set to the new value of the `IsChecked` property.

`RadioButton` grouping can be managed by the `RadioButtonGroup` class, which defines the following attached properties:

- `GroupName`, of type `string`, which defines the group name for `RadioButton` objects in an `ILayout`.
- `SelectedValue`, of type `object`, which represents the value of the checked `RadioButton` object within an `ILayout` group. This attached property uses a `TwoWay` binding by default.

Tip

Although not mandatory, it's strongly recommended that you set the `GroupName` property to ensure that the `SelectedValue` property works correctly across all platforms.

For more information about the `GroupName` attached property, see [Group RadioButtons](#). For more information about the `SelectedValue` attached property, see [Respond to RadioButton state changes](#).

Create RadioButtons

The appearance of a [RadioButton](#) is defined by the type of data assigned to the `RadioButton.Content` property:

- When the `RadioButton.Content` property is assigned a `string`, it will be displayed on each platform, horizontally aligned next to the radio button circle.
- When the `RadioButton.Content` is assigned a [View](#), it will be displayed on supported platforms (iOS, Windows), while unsupported platforms will fallback to a string representation of the [View](#) object (Android). In both cases, the content is displayed horizontally aligned next to the radio button circle.
- When a [ControlTemplate](#) is applied to a [RadioButton](#), a [View](#) can be assigned to the `RadioButton.Content` property on all platforms. For more information, see [Redefine RadioButton appearance](#).

Display string-based content

A [RadioButton](#) displays text when the `Content` property is assigned a `string`:

XAML

```
<StackLayout>
  <Label Text="What's your favorite animal?" />
  <RadioButton Content="Cat" />
  <RadioButton Content="Dog" />
  <RadioButton Content="Elephant" />
  <RadioButton Content="Monkey"
    IsChecked="true" />
</StackLayout>
```

In this example, [RadioButton](#) objects are implicitly grouped inside the same parent container. This XAML results in the appearance shown in the following screenshot:



Display arbitrary content

On iOS and Windows, a [RadioButton](#) can display arbitrary content when the `Content` property is assigned a [View](#):

```
<StackLayout>
  <Label Text="What's your favorite animal?" />
  <RadioButton>
    <RadioButton.Content>
      <Image Source="cat.png" />
    </RadioButton.Content>
  </RadioButton>
  <RadioButton>
    <RadioButton.Content>
      <Image Source="dog.png" />
    </RadioButton.Content>
  </RadioButton>
  <RadioButton>
    <RadioButton.Content>
      <Image Source="elephant.png" />
    </RadioButton.Content>
  </RadioButton>
  <RadioButton>
    <RadioButton.Content>
      <Image Source="monkey.png" />
    </RadioButton.Content>
  </RadioButton>
</StackLayout>
```

In this example, [RadioButton](#) objects are implicitly grouped inside the same parent container. This XAML results in the appearance shown in the following screenshot:



On Android, [RadioButton](#) objects will display a string-based representation of the [View](#) object that's been set as content.

ⓘ Note

When a [ControlTemplate](#) is applied to a [RadioButton](#), a [View](#) can be assigned to the `RadioButton.Content` property on all platforms. For more information, see [Redefine RadioButton appearance](#).

Associate values with RadioButtons

Each `RadioButton` object has a `Value` property, of type `object`, which defines an optional unique value to associate with the radio button. This enables the value of a `RadioButton` to be different to its content, and is particularly useful when `RadioButton` objects are displaying `View` objects.

The following XAML shows setting the `Content` and `Value` properties on each `RadioButton` object:

XAML

```
<StackLayout>
  <Label Text="What's your favorite animal?" />
  <RadioButton Value="Cat">
    <RadioButton.Content>
      <Image Source="cat.png" />
    </RadioButton.Content>
  </RadioButton>
  <RadioButton Value="Dog">
    <RadioButton.Content>
      <Image Source="dog.png" />
    </RadioButton.Content>
  </RadioButton>
  <RadioButton Value="Elephant">
    <RadioButton.Content>
      <Image Source="elephant.png" />
    </RadioButton.Content>
  </RadioButton>
  <RadioButton Value="Monkey">
    <RadioButton.Content>
      <Image Source="monkey.png" />
    </RadioButton.Content>
  </RadioButton>
</StackLayout>
```

In this example, each `RadioButton` has an `Image` as its content, while also defining a string-based value. This enables the value of the checked radio button to be easily identified.

Group RadioButtons

Radio buttons work in groups, and there are three approaches to grouping radio buttons:

- Place them inside the same parent container. This is known as *implicit* grouping.

- Set the `GroupName` property on each radio button in the group to the same value. This is known as *explicit* grouping.
- Set the `RadioButtonGroup.GroupName` attached property on a parent container, which in turn sets the `GroupName` property of any `RadioButton` objects in the container. This is also known as *explicit* grouping.

Important

[RadioButton](#) objects don't have to belong to the same parent to be grouped. They are mutually exclusive provided that they share a group name.

Explicit grouping with the `GroupName` property

The following XAML example shows explicitly grouping `RadioButton` objects by setting their `GroupName` properties:

XAML

```
<Label Text="What's your favorite color?" />
<RadioButton Content="Red"
              GroupName="colors" />
<RadioButton Content="Green"
              GroupName="colors" />
<RadioButton Content="Blue"
              GroupName="colors" />
<RadioButton Content="Other"
              GroupName="colors" />
```

In this example, each `RadioButton` is mutually exclusive because it shares the same `GroupName` value.

Explicit grouping with the `RadioButtonGroup.GroupName` attached property

The `RadioButtonGroup` class defines a `GroupName` attached property, of type `string`, which can be set on a `Layout<View>` object. This enables any layout to be turned into a radio button group:

XAML

```
<StackLayout RadioButtonGroup.GroupName="colors">
  <Label Text="What's your favorite color?" />
  <RadioButton Content="Red" />
```

```
<RadioButton Content="Green" />
<RadioButton Content="Blue" />
<RadioButton Content="Other" />
</StackLayout>
```

In this example, each `RadioButton` in the `StackLayout` will have its `GroupName` property set to `colors`, and will be mutually exclusive.

⚠ Note

When an `ILayout` object that sets the `RadioButtonGroup.GroupName` attached property contains a `RadioButton` that sets its `GroupName` property, the value of the `RadioButton.GroupName` property will take precedence.

Respond to RadioButton state changes

A radio button has two states: checked or unchecked. When a radio button is checked, its `IsChecked` property is `true`. When a radio button is unchecked, its `IsChecked` property is `false`. A radio button can be cleared by tapping another radio button in the same group, but it cannot be cleared by tapping it again. However, you can clear a radio button programmatically by setting its `IsChecked` property to `false`.

Respond to an event firing

When the `IsChecked` property changes, either through user or programmatic manipulation, the `CheckedChanged` event fires. An event handler for this event can be registered to respond to the change:

XAML

```
<RadioButton Content="Red"
              GroupName="colors"
              CheckedChanged="OnColorsRadioButtonCheckedChanged" />
```

The code-behind contains the handler for the `CheckedChanged` event:

C#

```
void OnColorsRadioButtonCheckedChanged(object sender,
CheckedChangedEventArgs e)
{
```

```
// Perform required operation  
}
```

The `sender` argument is the `RadioButton` responsible for this event. You can use this to access the `RadioButton` object, or to distinguish between multiple `RadioButton` objects sharing the same `CheckedChanged` event handler.

Respond to a property change

The `RadioButtonGroup` class defines a `SelectedValue` attached property, of type `object`, which can be set on an `ILayout` object. This attached property represents the value of the checked `RadioButton` within a group defined on a layout.

When the `IsChecked` property changes, either through user or programmatic manipulation, the `RadioButtonGroup.SelectedValue` attached property also changes. Therefore, the `RadioButtonGroup.SelectedValue` attached property can be data bound to a property that stores the user's selection:

XAML

```
<StackLayout RadioButtonGroup.GroupName="{Binding GroupName}"  
              RadioButtonGroup.SelectedValue="{Binding Selection}">  
  <Label Text="What's your favorite animal?" />  
  <RadioButton Content="Cat"  
               Value="Cat" />  
  <RadioButton Content="Dog"  
               Value="Dog" />  
  <RadioButton Content="Elephant"  
               Value="Elephant" />  
  <RadioButton Content="Monkey"  
               Value="Monkey"/>  
  <Label x:Name="animalLabel">  
    <Label.FormattedText>  
      <FormattedString>  
        <Span Text="You have chosen:" />  
        <Span Text="{Binding Selection}" />  
      </FormattedString>  
    </Label.FormattedText>  
  </Label>  
</StackLayout>
```

In this example, the value of the `RadioButtonGroup.GroupName` attached property is set by the `GroupName` property on the binding context. Similarly, the value of the `RadioButtonGroup.SelectedValue` attached property is set by the `Selection` property on

the binding context. In addition, the `Selection` property is updated to the `Value` property of the checked `RadioButton`.

RadioButton visual states

`RadioButton` objects have `Checked` and `Unchecked` visual states that can be used to initiate a visual change when a `RadioButton` is checked or unchecked.

The following XAML example shows how to define a visual state for the `Checked` and `Unchecked` states:

XAML

```
<ContentPage ...>
  <ContentPage.Resources>
    <Style TargetType="RadioButton">
      <Setter Property="VisualStateManager.VisualStateGroups">
        <VisualStateGroupList>
          <VisualStateGroup x:Name="CheckedStates">
            <VisualState x:Name="Checked">
              <VisualState.Setters>
                <Setter Property="TextColor"
                      Value="Green" />
                <Setter Property="Opacity"
                      Value="1" />
              </VisualState.Setters>
            </VisualState>
          <VisualState x:Name="Unchecked">
            <VisualState.Setters>
              <Setter Property="TextColor"
                    Value="Red" />
              <Setter Property="Opacity"
                    Value="0.5" />
            </VisualState.Setters>
          </VisualState>
        </VisualStateGroup>
      </VisualStateGroupList>
    </Setter>
  </Style>
</ContentPage.Resources>
<StackLayout>
  <Label Text="What's your favorite mode of transport?" />
  <RadioButton Content="Car" />
  <RadioButton Content="Bike" />
  <RadioButton Content="Train" />
  <RadioButton Content="Walking" />
</StackLayout>
</ContentPage>
```

In this example, the implicit `Style` targets `RadioButton` objects. The `Checked VisualState` specifies that when a `RadioButton` is checked, its `TextColor` property will be set to green with an `Opacity` value of 1. The `Unchecked VisualState` specifies that when a `RadioButton` is in a unchecked state, its `TextColor` property will be set to red with an `Opacity` value of 0.5. Therefore, the overall effect is that when a `RadioButton` is unchecked it's red and partially transparent, and is green without transparency when it's checked:

What's your favorite mode of transport?

☐ Car

☐ Bike

☐ Train

☒ Walking

For more information about visual states, see [Visual states](#).

Redefine RadioButton appearance

By default, `RadioButton` objects use handlers to utilize native controls on supported platforms. However, `RadioButton` visual structure can be redefined with a `ControlTemplate`, so that `RadioButton` objects have an identical appearance on all platforms. This is possible because the `RadioButton` class inherits from the `TemplatedView` class.

The following XAML shows a [ControlTemplate](#) that can be used to redefine the visual structure of [RadioButton](#) objects:

XAML

```
<ContentPage ...>
  <ContentPage.Resources>
    <ControlTemplate x:Key="RadioButtonTemplate">
      <Border Stroke="#F3F2F1"
        StrokeThickness="2"
        StrokeShape="RoundRectangle 10"
        BackgroundColor="#F3F2F1"
        HeightRequest="90"
        WidthRequest="90"
        HorizontalOptions="Start"
        VerticalOptions="Start">
        <VisualStateManager.VisualStateGroups>
          <VisualStateGroupList>
            <VisualStateGroup x:Name="CheckedStates">
              <VisualState x:Name="Checked">
                <VisualState.Setters>
                  <Setter Property="Stroke"
```

```

        Value="#FF3300" />
        <Setter TargetName="check"
            Property="Opacity"
            Value="1" />
    </VisualState.Setters>
</VisualState>
<VisualState x:Name="Unchecked">
    <VisualState.Setters>
        <Setter Property="BackgroundColor"
            Value="#F3F2F1" />
        <Setter Property="Stroke"
            Value="#F3F2F1" />
        <Setter TargetName="check"
            Property="Opacity"
            Value="0" />
    </VisualState.Setters>
</VisualState>
</VisualStateGroup>
</VisualStateManager.VisualStateGroups>
<Grid Margin="4"
    WidthRequest="90">
    <Grid Margin="0,0,4,0"
        WidthRequest="18"
        HeightRequest="18"
        HorizontalOptions="End"
        VerticalOptions="Start">
        <Ellipse Stroke="Blue"
            Fill="White"
            WidthRequest="16"
            HeightRequest="16"
            HorizontalOptions="Center"
            VerticalOptions="Center" />
        <Ellipse x:Name="check"
            Fill="Blue"
            WidthRequest="8"
            HeightRequest="8"
            HorizontalOptions="Center"
            VerticalOptions="Center" />
    </Grid>
</ContentPresenter />
</Grid>
</Border>
</ControlTemplate>

<Style TargetType="RadioButton">
    <Setter Property="ControlTemplate"
        Value="{StaticResource RadioButtonTemplate}" />
</Style>
</ContentPage.Resources>
<!-- Page content -->
</ContentPage>

```

In this example, the root element of the [ControlTemplate](#) is a [Border](#) object that defines `Checked` and `Unchecked` visual states. The [Border](#) object uses a combination of [Grid](#), [Ellipse](#), and [ContentPresenter](#) objects to define the visual structure of a [RadioButton](#). The example also includes an *implicit* style that will assign the `RadioButtonTemplate` to the [ControlTemplate](#) property of any [RadioButton](#) objects on the page.

ⓘ Note

The [ContentPresenter](#) object marks the location in the visual structure where [RadioButton](#) content will be displayed.

The following XAML shows [RadioButton](#) objects that consume the [ControlTemplate](#) via the *implicit* style:

XAML

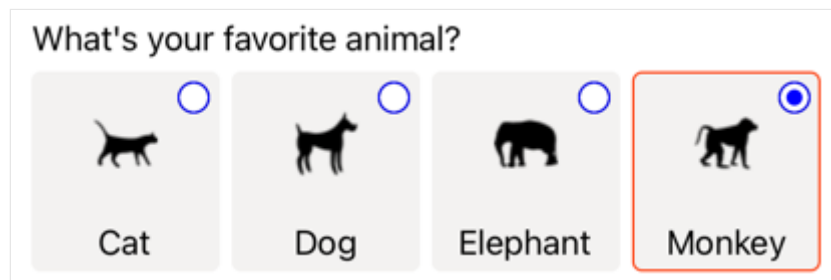
```
<StackLayout>
  <Label Text="What's your favorite animal?" />
  <StackLayout RadioButtonGroup.GroupName="animals"
    Orientation="Horizontal">
    <RadioButton Value="Cat">
      <RadioButton.Content>
        <StackLayout>
          <Image Source="cat.png"
            HorizontalOptions="Center"
            VerticalOptions="Center" />
          <Label Text="Cat"
            HorizontalOptions="Center"
            VerticalOptions="End" />
        </StackLayout>
      </RadioButton.Content>
    </RadioButton>
    <RadioButton Value="Dog">
      <RadioButton.Content>
        <StackLayout>
          <Image Source="dog.png"
            HorizontalOptions="Center"
            VerticalOptions="Center" />
          <Label Text="Dog"
            HorizontalOptions="Center"
            VerticalOptions="End" />
        </StackLayout>
      </RadioButton.Content>
    </RadioButton>
    <RadioButton Value="Elephant">
      <RadioButton.Content>
        <StackLayout>
          <Image Source="elephant.png"
            HorizontalOptions="Center"
            VerticalOptions="Center" />
          <Label Text="Elephant"
            HorizontalOptions="Center"
            VerticalOptions="End" />
        </StackLayout>
      </RadioButton.Content>
    </RadioButton>
  </StackLayout>
</StackLayout>
```

```

        VerticalOptions="Center" />
        <Label Text="Elephant"
            HorizontalOptions="Center"
            VerticalOptions="End" />
    </StackLayout>
</RadioButton.Content>
</RadioButton>
<RadioButton Value="Monkey">
    <RadioButton.Content>
        <StackLayout>
            <Image Source="monkey.png"
                HorizontalOptions="Center"
                VerticalOptions="Center" />
            <Label Text="Monkey"
                HorizontalOptions="Center"
                VerticalOptions="End" />
        </StackLayout>
    </RadioButton.Content>
</RadioButton>
</StackLayout>
</StackLayout>

```

In this example, the visual structure defined for each [RadioButton](#) is replaced with the visual structure defined in the [ControlTemplate](#), and so at runtime the objects in the [ControlTemplate](#) become part of the visual tree for each [RadioButton](#). In addition, the content for each [RadioButton](#) is substituted into the [ContentPresenter](#) defined in the control template. This results in the following [RadioButton](#) appearance:



For more information about control templates, see [Control templates](#).

Disable a RadioButton

Sometimes an app enters a state where a [RadioButton](#) being checked is not a valid operation. In such cases, the [RadioButton](#) can be disabled by setting its `IsEnabled` property to `false`.

RefreshView

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [RefreshView](#) is a container control that provides pull to refresh functionality for scrollable content. Therefore, the child of a [RefreshView](#) must be a scrollable control, such as [ScrollView](#), [CollectionView](#), or [ListView](#).

[RefreshView](#) defines the following properties:

- `Command`, of type [ICommand](#), which is executed when a refresh is triggered.
- `CommandParameter`, of type `object`, which is the parameter that's passed to the `Command`.
- `IsRefreshing`, of type `bool`, which indicates the current state of the [RefreshView](#).
- `RefreshColor`, of type [Color](#), the color of the progress circle that appears during the refresh.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

ⓘ Note

On Windows, the pull direction of a [RefreshView](#) can be set with a platform-specific. For more information, see [RefreshView pull direction on Windows](#).

Create a RefreshView

To add a [RefreshView](#) to a page, create a [RefreshView](#) object and set its `IsRefreshing` and `Command` properties. Then set its child to a scrollable control.

The following example shows how to instantiate a [RefreshView](#) in XAML:

XAML

```
<RefreshView IsRefreshing="{Binding IsRefreshing}"
             Command="{Binding RefreshCommand}">
    <ScrollView>
        <FlexLayout Direction="Row"
                    Wrap="Wrap"
                    AlignItems="Center"
                    AlignContent="Center">
```

```

        BindableLayout.ItemsSource="{Binding Items}"
        BindableLayout.ItemTemplate="{StaticResource
ColorItemTemplate}" />
    </ScrollView>
</RefreshView>

```

A [RefreshView](#) can also be created in code:

```

C#

RefreshView refreshView = new RefreshView();
ICommand refreshCommand = new Command(() =>
{
    // IsRefreshing is true
    // Refresh data here
    refreshView.IsRefreshing = false;
});
refreshView.Command = refreshCommand;

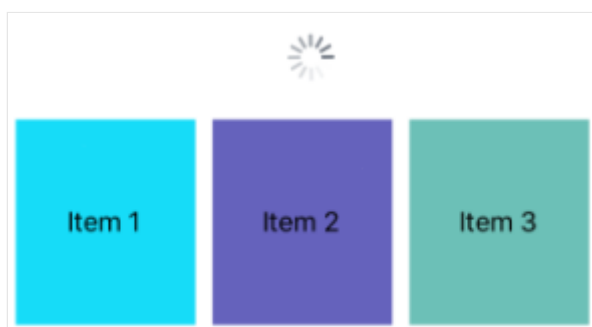
ScrollView scrollView = new ScrollView();
FlexLayout flexLayout = new FlexLayout { ... };
scrollView.Content = flexLayout;
refreshView.Content = scrollView;

```

In this example, the [RefreshView](#) provides pull to refresh functionality to a [ScrollView](#) whose child is a [FlexLayout](#). The [FlexLayout](#) uses a bindable layout to generate its content by binding to a collection of items, and sets the appearance of each item with a [DataTemplate](#). For more information about bindable layouts, see [Bindable layout](#).

The value of the `RefreshView.IsRefreshing` property indicates the current state of the [RefreshView](#). When a refresh is triggered by the user, this property will automatically transition to `true`. Once the refresh completes, you should reset the property to `false`.

When the user initiates a refresh, the [ICommand](#) defined by the `Command` property is executed, which should refresh the items being displayed. A refresh visualization is shown while the refresh occurs, which consists of an animated progress circle. The following screenshot shows the progress circle on iOS:



ⓘ Note

Manually setting the `IsRefreshing` property to `true` will trigger the refresh visualization, and will execute the `ICommand` defined by the `Command` property.

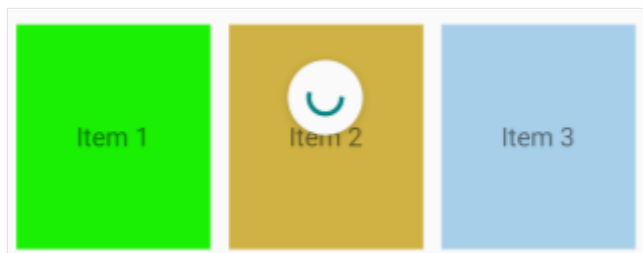
RefreshView appearance

In addition to the properties that `RefreshView` inherits from the `VisualElement` class, `RefreshView` also defines the `RefreshColor` property. This property can be set to define the color of the progress circle that appears during the refresh:

XAML

```
<RefreshView RefreshColor="Teal"
... />
```

The following Android screenshot shows a `RefreshView` with the `RefreshColor` property:



In addition, the `BackgroundColor` property can be set to a `Color` that represents the background color of the progress circle.

ⓘ Note

On iOS, the `BackgroundColor` property sets the background color of the `UIView` that contains the progress circle.

Disable a RefreshView

An app may enter a state where pull to refresh is not a valid operation. In such cases, the `RefreshView` can be disabled by setting its `IsEnabled` property to `false`. This will prevent users from being able to trigger pull to refresh.

Alternatively, when defining the `Command` property, the `CanExecute` delegate of the `ICommand` can be specified to enable or disable the command.

ScrollView

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [ScrollView](#) is a view that's capable of scrolling its content. By default, [ScrollView](#) scrolls its content vertically. A [ScrollView](#) can only have a single child, although this can be other layouts.

[ScrollView](#) defines the following properties:

- `Content`, of type [View](#), represents the content to display in the [ScrollView](#).
- `ContentSize`, of type `Size`, represents the size of the content. This is a read-only property.
- `HorizontalScrollBarVisibility`, of type `ScrollBarVisibility`, represents when the horizontal scroll bar is visible.
- `Orientation`, of type `ScrollOrientation`, represents the scrolling direction of the [ScrollView](#). The default value of this property is `Vertical`.
- `ScrollX`, of type `double`, indicates the current X scroll position. The default value of this read-only property is 0.
- `ScrollY`, of type `double`, indicates the current Y scroll position. The default value of this read-only property is 0.
- `VerticalScrollBarVisibility`, of type `ScrollBarVisibility`, represents when the vertical scroll bar is visible.

These properties are backed by [BindableProperty](#) objects, with the exception of the `Content` property, which means that they can be targets of data bindings and styled.

The `Content` property is the [ContentProperty](#) of the [ScrollView](#) class, and therefore does not need to be explicitly set from XAML.

Warning

[ScrollView](#) objects should not be nested. In addition, [ScrollView](#) objects should not be nested with other controls that provide scrolling, such as [CollectionView](#), [ListView](#), and [WebView](#).

ScrollView as a root layout

A [ScrollView](#) can only have a single child, which can be other layouts. It's therefore common for a [ScrollView](#) to be the root layout on a page. To scroll its child content, [ScrollView](#) computes the difference between the height of its content and its own height. That difference is the amount that the [ScrollView](#) can scroll its content.

A [StackLayout](#) will often be the child of a [ScrollView](#). In this scenario, the [ScrollView](#) causes the [StackLayout](#) to be as tall as the sum of the heights of its children. Then the [ScrollView](#) can determine the amount that its content can be scrolled. For more information about the [StackLayout](#), see [StackLayout](#).

⊗ Caution

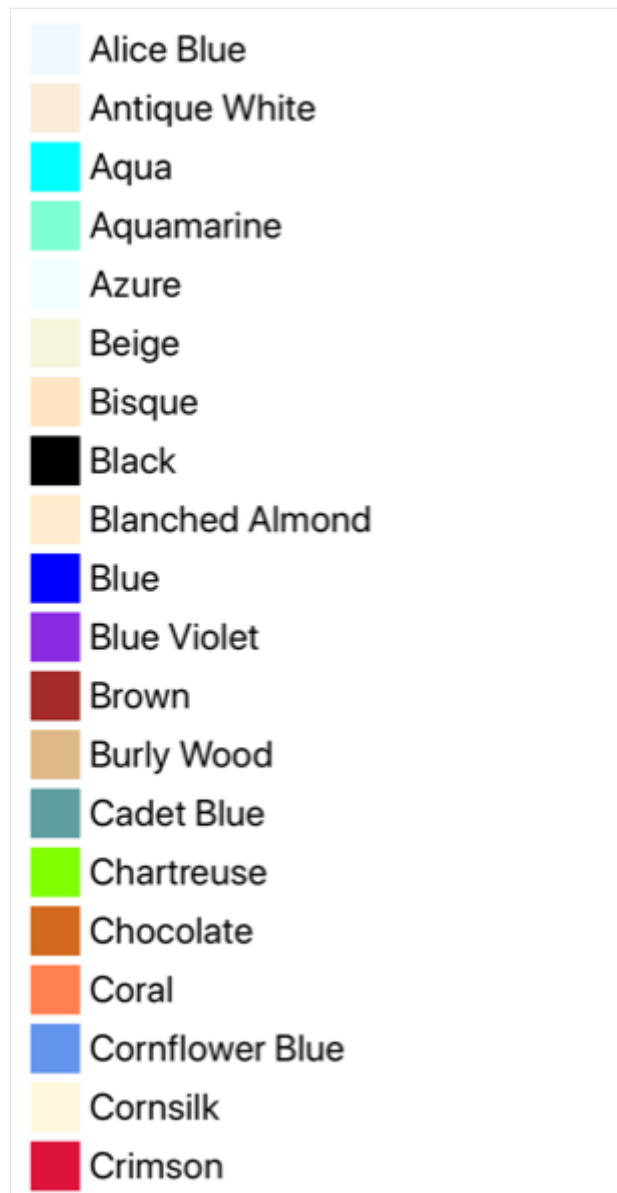
In a vertical [ScrollView](#), avoid setting the `VerticalOptions` property to `Start`, `Center`, or `End`. Doing so tells the [ScrollView](#) to be only as tall as it needs to be, which could be zero. While .NET MAUI protects against this eventuality, it's best to avoid code that suggests something you don't want to happen.

The following XAML example has a [ScrollView](#) as a root layout on a page:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:local="clr-namespace:ScrollViewDemos"
  x:Class="ScrollViewDemos.Views.XAML.ColorListPage"
  Title="ScrollView demo">
  <ScrollView Margin="20">
    <StackLayout BindableLayout.ItemsSource="{x:Static
local:NamedColor.All}">
      <BindableLayout.ItemTemplate>
        <DataTemplate>
          <StackLayout Orientation="Horizontal">
            <BoxView Color="{Binding Color}"
              HeightRequest="32"
              WidthRequest="32"
              VerticalOptions="Center" />
            <Label Text="{Binding FriendlyName}"
              FontSize="24"
              VerticalOptions="Center" />
          </StackLayout>
        </DataTemplate>
      </BindableLayout.ItemTemplate>
    </StackLayout>
  </ScrollView>
</ContentPage>
```

In this example, the [ScrollView](#) has its content set to a [StackLayout](#) that uses a bindable layout to display the `Colors` fields defined by .NET MAUI. By default, a [ScrollView](#) scrolls vertically, which reveals more content:



The equivalent C# code is:

C#

```
public class ColorListPage : ContentPage
{
    public ColorListPage()
    {
        DataTemplate dataTemplate = new DataTemplate(() =>
        {
            BoxView boxView = new BoxView
            {
                HeightRequest = 32,
                WidthRequest = 32,
                VerticalOptions = LayoutOptions.Center
            };
        });
    }
}
```

```

        boxView.SetBinding(BoxView.ColorProperty, "Color");

        Label label = new Label
        {
            FontSize = 24,
            VerticalOptions = LayoutOptions.Center
        };
        label.SetBinding(Label.TextProperty, "FriendlyName");

        StackLayout horizontalStackLayout = new StackLayout
        {
            Orientation = StackOrientation.Horizontal
        };
        horizontalStackLayout.Add(boxView);
        horizontalStackLayout.Add(label);

        return horizontalStackLayout;
    });

    StackLayout stackLayout = new StackLayout();
    BindableLayout.SetItemsSource(stackLayout, NamedColor.All);
    BindableLayout.SetItemTemplate(stackLayout, dataTemplate);

    ScrollView scrollView = new ScrollView
    {
        Margin = new Thickness(20),
        Content = stackLayout
    };

    Title = "ScrollView demo";
    Content = scrollView;
}
}

```

For more information about bindable layouts, see [BindableLayout](#).

ScrollView as a child layout

A [ScrollView](#) can be a child layout to a different parent layout.

A [ScrollView](#) will often be the child of a [Grid](#). A [ScrollView](#) requires a specific height to compute the difference between the height of its content and its own height, with the difference being the amount that the [ScrollView](#) can scroll its content. When a [ScrollView](#) is the child of a [Grid](#), it doesn't receive a specific height. The [Grid](#) wants the [ScrollView](#) to be as short as possible, which is either the height of the [ScrollView](#) contents or zero. To handle this scenario, the `RowDefinition` of the [Grid](#) row that contains the [ScrollView](#) should be set to `*`. This will cause the [Grid](#) to give the [ScrollView](#) all the extra space not required by the other children, and the [ScrollView](#) will then have a specific height.

The following XAML example has a [ScrollView](#) as a child layout to a [Grid](#):

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="ScrollViewDemos.Views.XAML.BlackCatPage"
             Title="ScrollView as a child layout demo">
    <Grid Margin="20"
          RowDefinitions="Auto,*,Auto">
        <Label Text="THE BLACK CAT by Edgar Allan Poe"
              FontSize="14"
              FontAttributes="Bold"
              HorizontalOptions="Center" />
        <ScrollView x:Name="scrollView"
                  Grid.Row="1"
                  VerticalOptions="FillAndExpand"
                  Scrolled="OnScrollViewScrolled">
            <StackLayout>
                <Label Text="FOR the most wild, yet most homely narrative
which I am about to pen, I neither expect nor solicit belief. Mad indeed
would I be to expect it, in a case where my very senses reject their own
evidence. Yet, mad am I not -- and very surely do I not dream. But to-morrow
I die, and to-day I would unburthen my soul. My immediate purpose is to
place before the world, plainly, succinctly, and without comment, a series
of mere household events. In their consequences, these events have terrified
-- have tortured -- have destroyed me. Yet I will not attempt to expound
them. To me, they have presented little but Horror -- to many they will seem
less terrible than barroques. Hereafter, perhaps, some intellect may be
found which will reduce my phantasm to the common-place -- some intellect
more calm, more logical, and far less excitable than my own, which will
perceive, in the circumstances I detail with awe, nothing more than an
ordinary succession of very natural causes and effects." />
                <!-- More Label objects go here -->
            </StackLayout>
        </ScrollView>
        <Button Grid.Row="2"
              Text="Scroll to end"
              Clicked="OnButtonClicked" />
    </Grid>
</ContentPage>
```

In this example, the root layout is a [Grid](#) that has a [Label](#), [ScrollView](#), and [Button](#) as its children. The [ScrollView](#) has a [StackLayout](#) as its content, with the [StackLayout](#) containing multiple [Label](#) objects. This arrangement ensures that the first [Label](#) is always on-screen, while text displayed by the other [Label](#) objects can be scrolled:

THE BLACK CAT by Edgar Allan Poe

FOR the most wild, yet most homely narrative which I am about to pen, I neither expect nor solicit belief. Mad indeed would I be to expect it, in a case where my very senses reject their own evidence. Yet, mad am I not -- and very surely do I not dream. But to-morrow I die, and to-day I would unburthen my soul. My immediate purpose is to place before the world, plainly, succinctly, and without comment, a series of mere household events. In their consequences, these events have terrified -- have tortured -- have destroyed me. Yet I will not attempt to expound them. To me, they have presented little but Horror -- to many they will seem less terrible than barroques. Hereafter, perhaps, some intellect may be found which will reduce my phantasm to the common-place -- some intellect more calm, more logical, and far less excitable than my own, which will perceive, in the circumstances I detail with awe, nothing more than an ordinary succession of very natural causes and effects.

From my infancy I was noted for the docility and humanity of my disposition. My tenderness of heart was even so conspicuous as to make me the jest of my companions. I was especially fond of animals, and was indulged by my parents with a great variety of pets. With these I spent most of my time, and never was so happy as when feeding and caressing them. This peculiarity of character grew with my growth, and, in my manhood, I derived from it one of my principal sources of pleasure. To those who have cherished an affection for a faithful and sagacious dog, I need hardly be at the trouble of explaining the nature or the intensity of the

The equivalent C# code is:

C#

```
public class BlackCatPage : ContentPage
{
    public BlackCatPage()
    {
        Label titleLabel = new Label
        {
            Text = "THE BLACK CAT by Edgar Allan Poe",
            // More properties set here to define the Label appearance
        };

        StackLayout stackLayout = new StackLayout();
        stackLayout.Add(new Label { Text = "FOR the most wild, yet most
homely narrative which I am about to pen, I neither expect nor solicit
belief. Mad indeed would I be to expect it, in a case where my very senses
reject their own evidence. Yet, mad am I not -- and very surely do I not
dream. But to-morrow I die, and to-day I would unburthen my soul. My
immediate purpose is to place before the world, plainly, succinctly, and
```

without comment, a series of mere household events. In their consequences, these events have terrified -- have tortured -- have destroyed me. Yet I will not attempt to expound them. To me, they have presented little but Horror -- to many they will seem less terrible than barroques. Hereafter, perhaps, some intellect may be found which will reduce my phantasm to the common-place -- some intellect more calm, more logical, and far less excitable than my own, which will perceive, in the circumstances I detail with awe, nothing more than an ordinary succession of very natural causes and effects." });

```
// More Label objects go here

ScrollView scrollView = new ScrollView();
scrollView.Content = stackLayout;
// ...

Title = "ScrollView as a child layout demo";
Grid grid = new Grid
{
    Margin = new Thickness(20),
    RowDefinitions =
    {
        new RowDefinition { Height = new GridLength(0,
GridUnitType.Auto) },
        new RowDefinition { Height = new GridLength(1,
GridUnitType.Star) },
        new RowDefinition { Height = new GridLength(0,
GridUnitType.Auto) }
    }
};
grid.Add(titleLabel);
grid.Add(scrollView, 0, 1);
grid.Add(button, 0, 2);

Content = grid;
}
}
```

Orientation

[ScrollView](#) has an `Orientation` property, which represents the scrolling direction of the [ScrollView](#). This property is of type `ScrollOrientation`, which defines the following members:

- `Vertical` indicates that the [ScrollView](#) will scroll vertically. This member is the default value of the `Orientation` property.
- `Horizontal` indicates that the [ScrollView](#) will scroll horizontally.
- `Both` indicates that the [ScrollView](#) will scroll horizontally and vertically.
- `Neither` indicates that the [ScrollView](#) won't scroll.

💡 Tip

Scrolling can be disabled by setting the `Orientation` property to `Neither`.

Detect scrolling

`ScrollView` defines a `Scrolled` event that's raised to indicate that scrolling occurred. The `ScrolledEventArgs` object that accompanies the `Scrolled` event has `ScrollX` and `ScrollY` properties, both of type `double`.

📌 Important

The `ScrolledEventArgs.ScrollX` and `ScrolledEventArgs.ScrollY` properties can have negative values, due to the bounce effect that occurs when scrolling back to the start of a [ScrollView](#).

The following XAML example shows a `ScrollView` that sets an event handler for the `Scrolled` event:

XAML

```
<ScrollView Scrolled="OnScrollViewScrolled">
    ...
</ScrollView>
```

The equivalent C# code is:

C#

```
ScrollView scrollView = new ScrollView();
scrollView.Scrolled += OnScrollViewScrolled;
```

In this example, the `OnScrollViewScrolled` event handler is executed when the `Scrolled` event fires:

C#

```
void OnScrollViewScrolled(object sender, ScrolledEventArgs e)
{
    Console.WriteLine($"ScrollX: {e.ScrollX}, ScrollY: {e.ScrollY}");
}
```

In this example, the `OnScrollViewScrolled` event handler outputs the values of the `ScrolledEventArgs` object that accompanies the event.

ⓘ Note

The `Scrolled` event is raised for user initiated scrolls, and for programmatic scrolls.

Scroll programmatically

`ScrollView` defines two `ScrollToAsync` methods, that asynchronously scroll the `ScrollView`. One of the overloads scrolls to a specified position in the `ScrollView`, while the other scrolls a specified element into view. Both overloads have an additional argument that can be used to indicate whether to animate the scroll.

ⓘ Important

The `ScrollToAsync` methods will not result in scrolling when the `ScrollView.Orientation` property is set to `Neither`.

Scroll a position into view

A position within a `ScrollView` can be scrolled to with the `ScrollToAsync` method that accepts `double x` and `y` arguments. Given a vertical `ScrollView` object named `scrollView`, the following example shows how to scroll to 150 device-independent units from the top of the `ScrollView`:

C#

```
await scrollView.ScrollToAsync(0, 150, true);
```

The third argument to the `ScrollToAsync` is the `animated` argument, which determines whether a scrolling animation is displayed when programmatically scrolling a `ScrollView`.

Scroll an element into view

An element within a `ScrollView` can be scrolled into view with the `ScrollToAsync` method that accepts `Element` and `ScrollToPosition` arguments. Given a vertical `ScrollView`

named `scrollView`, and a `Label` named `label`, the following example shows how to scroll an element into view:

C#

```
await scrollView.ScrollToAsync(label, ScrollToPosition.End, true);
```

The third argument to the `ScrollToAsync` is the `animated` argument, which determines whether a scrolling animation is displayed when programmatically scrolling a `ScrollView`.

When scrolling an element into view, the exact position of the element after the scroll has completed can be set with the second argument, `position`, of the `ScrollToAsync` method. This argument accepts a `ScrollToPosition` enumeration member:

- `MakeVisible` indicates that the element should be scrolled until it's visible in the `ScrollView`.
- `Start` indicates that the element should be scrolled to the start of the `ScrollView`.
- `Center` indicates that the element should be scrolled to the center of the `ScrollView`.
- `End` indicates that the element should be scrolled to the end of the `ScrollView`.

Scroll bar visibility

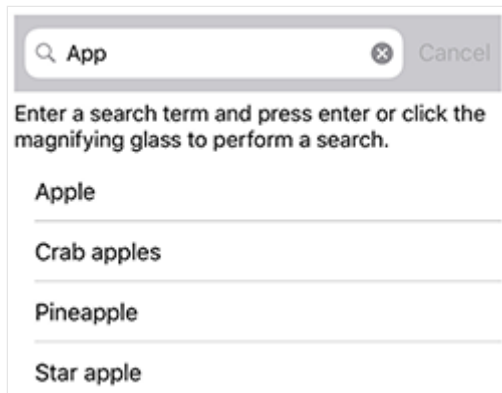
`ScrollView` defines `HorizontalScrollBarVisibility` and `VerticalScrollBarVisibility` properties, which are backed by bindable properties. These properties get or set a `ScrollBarVisibility` enumeration value that represents whether the horizontal, or vertical, scroll bar is visible. The `ScrollBarVisibility` enumeration defines the following members:

- `Default` indicates the default scroll bar behavior for the platform, and is the default value of the `HorizontalScrollBarVisibility` and `VerticalScrollBarVisibility` properties.
- `Always` indicates that scroll bars will be visible, even when the content fits in the view.
- `Never` indicates that scroll bars will not be visible, even if the content doesn't fit in the view.

SearchBar

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [SearchBar](#) is a user input control used to initiating a search. The [SearchBar](#) control supports placeholder text, query input, search execution, and cancellation. The following iOS screenshot shows a [SearchBar](#) query with results displayed in a [ListView](#):



[SearchBar](#) defines the following properties:

- `CancelButtonColor` is a [Color](#) that defines the color of the cancel button.
- `HorizontalTextAlignment` is a [TextAlignment](#) enum value that defines the horizontal alignment of the query text.
- `SearchCommand` is an [ICommand](#) that allows binding user actions, such as finger taps or clicks, to commands defined on a viewmodel.
- `SearchCommandParameter` is an `object` that specifies the parameter that should be passed to the `SearchCommand`.
- `VerticalTextAlignment` is a [TextAlignment](#) enum value that defines the vertical alignment of the query text.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

In addition, [SearchBar](#) defines a `SearchButtonPressed` event, which is raised when the search button is clicked, or the enter key is pressed.

[SearchBar](#) derives from the [InputView](#) class, from which it inherits the following properties:

- `CharacterSpacing`, of type `double`, sets the spacing between characters in the entered text.
- `CursorPosition`, of type `int`, defines the position of the cursor within the editor.

- `FontAttributes`, of type `FontAttributes`, determines text style.
- `FontAutoScalingEnabled`, of type `bool`, defines whether the text will reflect scaling preferences set in the operating system. The default value of this property is `true`.
- `FontFamily`, of type `string`, defines the font family.
- `FontSize`, of type `double`, defines the font size.
- `IsReadOnly`, of type `bool`, defines whether the user should be prevented from modifying text. The default value of this property is `false`.
- `IsSpellCheckEnabled`, of type `bool`, controls whether spell checking is enabled.
- `IsTextPredictionEnabled`, of type `bool`, controls whether text prediction and automatic text correction is enabled.
- `Keyboard`, of type `Keyboard`, specifies the soft input keyboard that's displayed when entering text.
- `MaxLength`, of type `int`, defines the maximum input length.
- `Placeholder`, of type `string`, defines the text that's displayed when the control is empty.
- `PlaceholderColor`, of type `Color`, defines the color of the placeholder text.
- `SelectionLength`, of type `int`, represents the length of selected text within the control.
- `Text`, of type `string`, defines the text entered into the control.
- `TextColor`, of type `Color`, defines the color of the entered text.
- `TextTransform`, of type `TextTransform`, specifies the casing of the entered text.

These properties are backed by `BindableProperty` objects, which means that they can be targets of data bindings, and styled.

In addition, `InputView` defines a `TextChanged` event, which is raised when the text in the `Entry` changes. The `TextChangedEventArgs` object that accompanies the `TextChanged` event has `NewTextValue` and `OldTextValue` properties, which specify the new and old text, respectively.

Create a SearchBar

To create a search bar, create a `SearchBar` object and set its `Placeholder` property to text that instructs the user to enter a search term.

The following XAML example shows how to create a `SearchBar`:

XAML

```
<SearchBar Placeholder="Search items..." />
```

The equivalent C# code is:

C#

```
SearchBar searchBar = new SearchBar { Placeholder = "Search items..." };
```

ⓘ Note

On iOS, the soft input keyboard can cover a text input field when the field is near the bottom of the screen, making it difficult to enter text. However, in a .NET MAUI iOS app, pages automatically scroll when the soft input keyboard would cover a text entry field, so that the field is above the soft input keyboard. The `KeyboardAutoManagerScroll.Disconnect` method, in the `Microsoft.Maui.Platform` namespace, can be called to disable this default behavior. The `KeyboardAutoManagerScroll.Connect` method can be called to re-enable the behavior after it's been disabled.

Perform a search with event handlers

A search can be executed using the `SearchBar` control by attaching an event handler to one of the following events:

- `SearchButtonPressed`, which is called when the user either clicks the search button or presses the enter key.
- `TextChanged`, which is called anytime the text in the query box is changed. This event is inherited from the `InputView` class.

The following XAML example shows an event handler attached to the `TextChanged` event and uses a `ListView` to display search results:

XAML

```
<SearchBar TextChanged="OnTextChanged" />
<ListView x:Name="searchResults" >
```

In this example, the `TextChanged` event is set to an event handler named `OnTextChanged`. This handler is located in the code-behind file:

C#

```
void OnTextChanged(object sender, EventArgs e)
{
```

```
        SearchBar searchBar = (SearchBar)sender;
        searchResults.ItemsSource =
            DataService.GetSearchResults(searchBar.Text);
    }
```

In this example, a `DataService` class with a `GetSearchResults` method is used to return items that match a query. The `SearchBar` control's `Text` property value is passed to the `GetSearchResults` method and the result is used to update the `ListView` control's `ItemsSource` property. The overall effect is that search results are displayed in the `ListView`.

Perform a search using a viewmodel

A search can be executed without event handlers by binding the `SearchCommand` property to an `ICommand` implementation. For more information about commanding, see [Commanding](#).

The following example shows a viewmodel class that contains an `ICommand` property named `PerformSearch`:

C#

```
public class SearchViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    protected virtual void NotifyPropertyChanged([CallerMemberName] string
propertyName = "")
    {
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
    }

    public ICommand PerformSearch => new Command<string>((string query) =>
    {
        SearchResults = DataService.GetSearchResults(query);
    });

    private List<string> searchResults = DataService.Fruits;
    public List<string> SearchResults
    {
        get
        {
            return searchResults;
        }
        set
        {
            searchResults = value;
        }
    }
}
```

```
        NotifyPropertyChanged();  
    }  
}  
}
```

❗ Note

The viewmodel assumes the existence of a `DataService` class capable of performing searches.

The following XAML example consumes the `SearchViewModel` class:

XAML

```
<ContentPage ...>  
    <ContentPage.BindingContext>  
        <viewmodels:SearchViewModel />  
    </ContentPage.BindingContext>  
    <StackLayout>  
        <SearchBar x:Name="searchBar"  
            SearchCommand="{Binding PerformSearch}"  
            SearchCommandParameter="{Binding Text, Source=  
{x:Reference searchBar}}"/>  
        <ListView x:Name="searchResults"  
            ItemsSource="{Binding SearchResults}" />  
    </StackLayout>  
</ContentPage>
```

In this example, the `BindingContext` is set to an instance of the `SearchViewModel` class. The `SearchBar.SearchCommand` property binds to `PerformSearch` viewmodel property, and the `SearchCommandParameter` property binds to the `SearchBar.Text` property. Similarly, the `ListView.ItemsSource` property is bound to the `SearchResults` property of the viewmodel.

Hide and show the soft input keyboard

The `SoftInputExtensions` class, in the `Microsoft.Maui` namespace, provides a series of extension methods that support interacting with the soft input keyboard on controls that support text input. The class defines the following methods:

- `IsSoftInputShowing`, which checks to see if the device is currently showing the soft input keyboard.

- `HideSoftInputAsync`, which will attempt to hide the soft input keyboard if it's currently showing.
- `ShowSoftInputAsync`, which will attempt to show the soft input keyboard if it's currently hidden.

The following example shows how to hide the soft input keyboard on a [SearchBar](#) named `searchBar`, if it's currently showing:

C#

```
if (searchBar.IsSoftInputShowing())  
    await  
searchBar.HideSoftInputAsync(System.Threading.CancellationToken.None);
```

Shapes

Article • 08/30/2024

 [Browse the sample](#)

A .NET Multi-platform App UI (.NET MAUI) [Shape](#) is a type of [View](#) that enables you to draw a shape to the screen. [Shape](#) objects can be used inside layout classes and most controls, because the [Shape](#) class derives from the [View](#) class. .NET MAUI Shapes is available in the [Microsoft.Maui.Controls.Shapes](#) namespace.

[Shape](#) defines the following properties:

- [Aspect](#), of type [Stretch](#), describes how the shape fills its allocated space. The default value of this property is `Stretch.None`.
- [Fill](#), of type [Brush](#), indicates the brush used to paint the shape's interior.
- [Stroke](#), of type [Brush](#), indicates the brush used to paint the shape's outline.
- [StrokeDashArray](#), of type `DoubleCollection`, which represents a collection of `double` values that indicate the pattern of dashes and gaps that are used to outline a shape.
- [StrokeDashOffset](#), of type `double`, specifies the distance within the dash pattern where a dash begins. The default value of this property is 0.0.
- [StrokeDashPattern](#), of type `float[]`, indicates the pattern of dashes and gaps that are used when drawing the stroke for a shape.
- [StrokeLineCap](#), of type [PenLineCap](#), describes the shape at the start and end of a line or segment. The default value of this property is `PenLineCap.Flat`.
- [StrokeLineJoin](#), of type [PenLineJoin](#), specifies the type of join that is used at the vertices of a shape. The default value of this property is `PenLineJoin.Miter`.
- [StrokeMiterLimit](#), of type `double`, specifies the limit on the ratio of the miter length to half the [StrokeThickness](#) of a shape. The default value of this property is 10.0.
- [StrokeThickness](#), of type `double`, indicates the width of the shape outline. The default value of this property is 1.0.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

.NET MAUI defines a number of objects that derive from the [Shape](#) class. These are [Ellipse](#), [Line](#), [Path](#), [Polygon](#), [Polyline](#), [Rectangle](#), and [RoundRectangle](#).

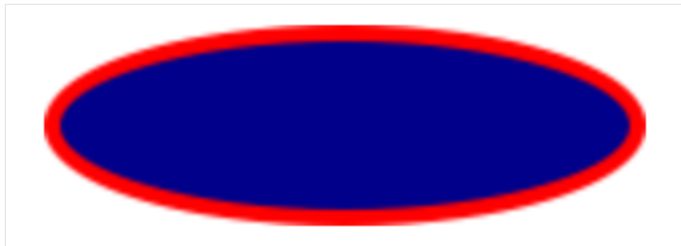
Paint shapes

[Brush](#) objects are used to paint a shape's [Stroke](#) and [Fill](#):

XAML

```
<Ellipse Fill="DarkBlue"
        Stroke="Red"
        StrokeThickness="4"
        WidthRequest="150"
        HeightRequest="50"
        HorizontalOptions="Start" />
```

In this example, the stroke and fill of an [Ellipse](#) are specified:



❗ Important

[Brush](#) objects use a type converter that enables [Color](#) values to be specified for the [Stroke](#) property.

If you don't specify a [Brush](#) object for [Stroke](#), or if you set [StrokeThickness](#) to 0, then the border around the shape is not drawn.

For more information about [Brush](#) objects, see [Brushes](#). For more information about valid [Color](#) values, see [Colors](#).

Stretch shapes

[Shape](#) objects have an [Aspect](#) property, of type [Stretch](#). This property determines how a [Shape](#) object's contents is stretched to fill the [Shape](#) object's layout space. A [Shape](#) object's layout space is the amount of space the [Shape](#) is allocated by the .NET MAUI layout system, because of either an explicit [WidthRequest](#) and [HeightRequest](#) setting or because of its `HorizontalOptions` and `VerticalOptions` settings.

The [Stretch](#) enumeration defines the following members:

- `None`, which indicates that the content preserves its original size. This is the default value of the `Shape.Aspect` property.

- **Fill**, which indicates that the content is resized to fill the destination dimensions. The aspect ratio is not preserved.
- **Uniform**, which indicates that the content is resized to fit the destination dimensions, while preserving the aspect ratio.
- **UniformToFill**, indicates that the content is resized to fill the destination dimensions, while preserving the aspect ratio. If the aspect ratio of the destination rectangle differs from the source, the source content is clipped to fit in the destination dimensions.

The following XAML shows how to set the **Aspect** property:

XAML

```
<Path Aspect="Uniform"
      Stroke="Yellow"
      Fill="Red"
      BackgroundColor="LightGray"
      HorizontalOptions="Start"
      HeightRequest="100"
      WidthRequest="100">
  <Path.Data>
    <!-- Path data goes here -->
  </Path.Data>
</Path>
```

In this example, a **Path** object draws a heart. The **Path** object's **WidthRequest** and **HeightRequest** properties are set to 100 device-independent units, and its **Aspect** property is set to **Uniform**. As a result, the object's contents are resized to fit the destination dimensions, while preserving the aspect ratio:



Draw dashed shapes

Shape objects have a **StrokeDashArray** property, of type **DoubleCollection**. This property represents a collection of **double** values that indicate the pattern of dashes and gaps that are used to outline a shape. A **DoubleCollection** is an **ObservableCollection** of **double** values. Each **double** in the collection specifies the length of a dash or gap. The

first item in the collection, which is located at index 0, specifies the length of a dash. The second item in the collection, which is located at index 1, specifies the length of a gap. Therefore, objects with an even index value specify dashes, while objects with an odd index value specify gaps.

[Shape](#) objects also have a [StrokeDashOffset](#) property, of type `double`, which specifies the distance within the dash pattern where a dash begins. Failure to set this property will result in the [Shape](#) having a solid outline.

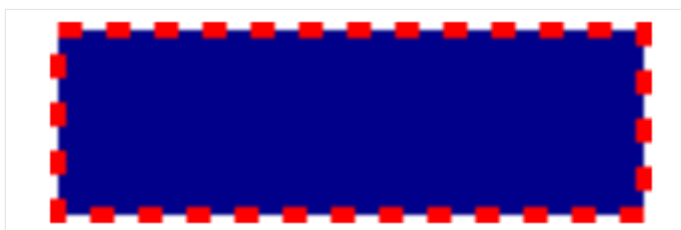
Dashed shapes can be drawn by setting both the [StrokeDashArray](#) and [StrokeDashOffset](#) properties. The [StrokeDashArray](#) property should be set to one or more `double` values, with each pair delimited by a single comma and/or one or more spaces. For example, "0.5 1.0" and "0.5,1.0" are both valid.

The following XAML example shows how to draw a dashed rectangle:

XAML

```
<Rectangle Fill="DarkBlue"
  Stroke="Red"
  StrokeThickness="4"
  StrokeDashArray="1,1"
  StrokeDashOffset="6"
  WidthRequest="150"
  HeightRequest="50"
  HorizontalOptions="Start" />
```

In this example, a filled rectangle with a dashed stroke is drawn:



Control line ends

A line has three parts: start cap, line body, and end cap. The start and end caps describe the shape at the start and end of a line, or segment.

[Shape](#) objects have a [StrokeLineCap](#) property, of type [PenLineCap](#), that describes the shape at the start and end of a line, or segment. The [PenLineCap](#) enumeration defines the following members:

- **Flat**, which represents a cap that doesn't extend past the last point of the line. This is comparable to no line cap, and is the default value of the [StrokeLineCap](#) property.
- **Square**, which represents a rectangle that has a height equal to the line thickness and a length equal to half the line thickness.
- **Round**, which represents a semicircle that has a diameter equal to the line thickness.

i Important

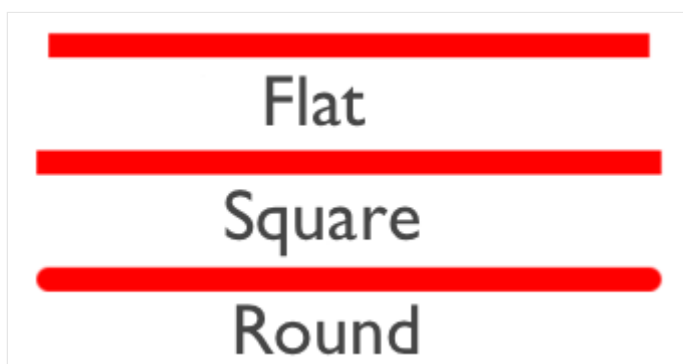
The [StrokeLineCap](#) property has no effect if you set it on a shape that has no start or end points. For example, this property has no effect if you set it on an [Ellipse](#), or [Rectangle](#).

The following XAML shows how to set the [StrokeLineCap](#) property:

XAML

```
<Line X1="0"  
      Y1="20"  
      X2="300"  
      Y2="20"  
      StrokeLineCap="Round"  
      Stroke="Red"  
      StrokeThickness="12" />
```

In this example, the red line is rounded at the start and end of the line:



Control line joins

[Shape](#) objects have a [StrokeLineJoin](#) property, of type [PenLineJoin](#), that specifies the type of join that is used at the vertices of the shape. The [PenLineJoin](#) enumeration defines the following members:

- `Miter`, which represents regular angular vertices. This is the default value of the `StrokeLineJoin` property.
- `Bevel`, which represents beveled vertices.
- `Round`, which represents rounded vertices.

⚠ Note

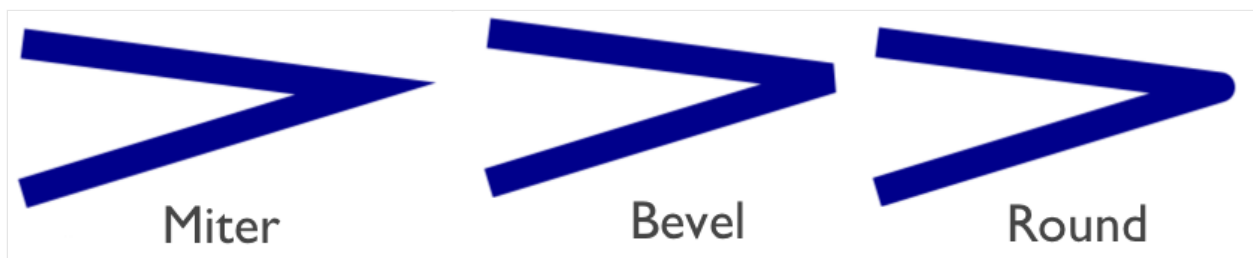
When the `StrokeLineJoin` property is set to `Miter`, the `StrokeMiterLimit` property can be set to a `double` to limit the miter length of line joins in the shape.

The following XAML shows how to set the `StrokeLineJoin` property:

XAML

```
<Polyline Points="20 20,250 50,20 120"  
          Stroke="DarkBlue"  
          StrokeThickness="20"  
          StrokeLineJoin="Round" />
```

In this example, the dark blue polyline has rounded joins at its vertices:



Ellipse

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [Ellipse](#) class derives from the [Shape](#) class, and can be used to draw ellipses and circles. For information on the properties that the [Ellipse](#) class inherits from the [Shape](#) class, see [Shapes](#).

The [Ellipse](#) class sets the [Aspect](#) property, inherited from the [Shape](#) class, to `Stretch.Fill`. For more information about the [Aspect](#) property, see [Stretch shapes](#).

Create an Ellipse

To draw an ellipse, create an [Ellipse](#) object and set its [WidthRequest](#) and [HeightRequest](#) properties. To paint the inside of the ellipse, set its [Fill](#) property to a [Brush](#)-derived object. To give the ellipse an outline, set its [Stroke](#) property to a [Brush](#)-derived object. The [StrokeThickness](#) property specifies the thickness of the ellipse outline. For more information about [Brush](#) objects, see [Brushes](#).

To draw a circle, make the [WidthRequest](#) and [HeightRequest](#) properties of the [Ellipse](#) object equal.

The following XAML example shows how to draw a filled ellipse:

XAML

```
<Ellipse Fill="Red"
        WidthRequest="150"
        HeightRequest="50"
        HorizontalOptions="Start" />
```

In this example, a red filled ellipse with dimensions 150x50 (device-independent units) is drawn:

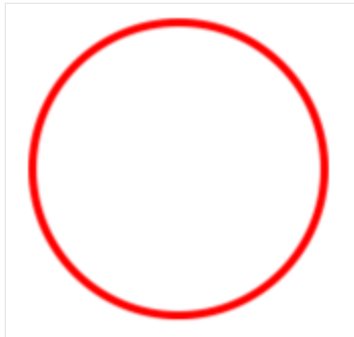


The following XAML example shows how to draw a circle:

XAML

```
<Ellipse Stroke="Red"  
    StrokeThickness="4"  
    WidthRequest="150"  
    HeightRequest="150"  
    HorizontalOptions="Start" />
```

In this example, a red circle with dimensions 150x150 (device-independent units) is drawn:



For information about drawing a dashed ellipse, see [Draw dashed shapes](#).

Fill rules

Article • 08/30/2024

 [Browse the sample](#)

Several .NET Multi-platform App UI (.NET MAUI) Shapes classes have `FillRule` properties, of type `FillRule`. These include `Polygon`, `Polyline`, and `GeometryGroup`.

The `FillRule` enumeration defines `EvenOdd` and `Nonzero` members. Each member represents a different rule for determining whether a point is in the fill region of a shape.

Important

All shapes are considered closed for the purposes of fill rules.

EvenOdd

The `EvenOdd` fill rule draws a ray from the point to infinity in any direction and counts the number of segments within the shape that the ray crosses. If this number is odd, the point is inside. If this number is even, the point is outside.

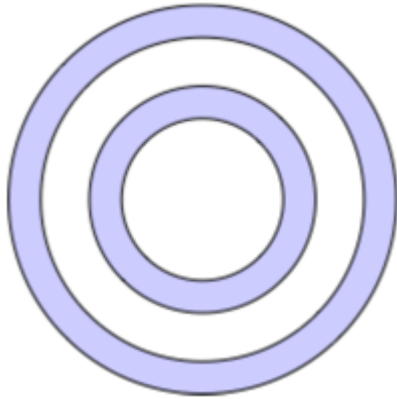
The following XAML example creates and renders a composite shape, with the `FillRule` defaulting to `EvenOdd`:

XAML

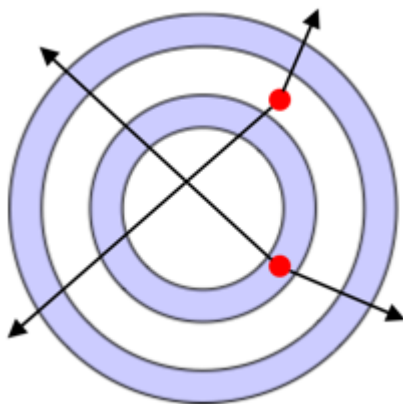
```
<Path Stroke="Black"
      Fill="#CCCCFF"
      Aspect="Uniform"
      HorizontalOptions="Start">
  <Path.Data>
    <!-- FillRule doesn't need to be set, because EvenOdd is the
    default. -->
    <GeometryGroup>
      <EllipseGeometry RadiusX="50"
                      RadiusY="50"
                      Center="75,75" />
      <EllipseGeometry RadiusX="70"
                      RadiusY="70"
                      Center="75,75" />
      <EllipseGeometry RadiusX="100"
                      RadiusY="100"
                      Center="75,75" />
      <EllipseGeometry RadiusX="120"
```

```
RadiusY="120"  
Center="75,75" />  
</GeometryGroup>  
</Path.Data>  
</Path>
```

In this example, a composite shape made up of a series of concentric rings is displayed:



In the composite shape, notice that the center and third rings are not filled. This is because a ray drawn from any point within either of those two rings passes through an even number of segments:



In the image above, the red circles represent points, and the lines represent arbitrary rays. For the upper point, the two arbitrary rays each pass through an even number of line segments. Therefore, the ring the point is in isn't filled. For the lower point, the two arbitrary rays each pass through an odd number of line segments. Therefore, the ring the point is in is filled.

Nonzero

The [Nonzero](#) fill rule draws a ray from the point to infinity in any direction and then examines the places where a segment of the shape crosses the ray. Starting with a count of zero, the count is incremented each time a segment crosses the ray from left to right and decremented each time a segment crosses the ray from right to left. After counting

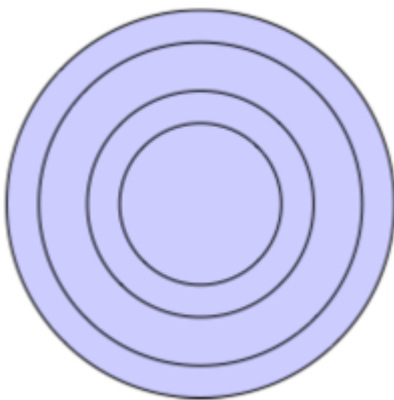
the crossings, if the result is zero then the point is outside the polygon. Otherwise, it's inside.

The following XAML example creates and renders a composite shape, with the `FillRule` set to `Nonzero`:

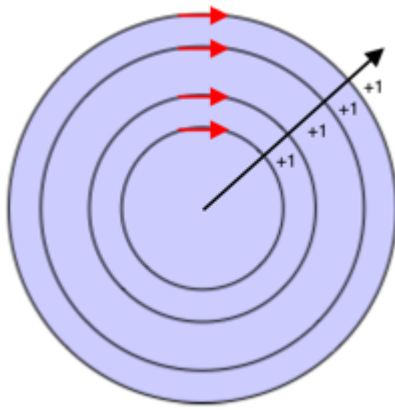
XAML

```
<Path Stroke="Black"
      Fill="#CCCCFF"
      Aspect="Uniform"
      HorizontalOptions="Start">
  <Path.Data>
    <GeometryGroup FillRule="Nonzero">
      <EllipseGeometry RadiusX="50"
                      RadiusY="50"
                      Center="75,75" />
      <EllipseGeometry RadiusX="70"
                      RadiusY="70"
                      Center="75,75" />
      <EllipseGeometry RadiusX="100"
                      RadiusY="100"
                      Center="75,75" />
      <EllipseGeometry RadiusX="120"
                      RadiusY="120"
                      Center="75,75" />
    </GeometryGroup>
  </Path.Data>
</Path>
```

In this example, a composite shape made up of a series of concentric rings is displayed:



In the composite shape, notice that all rings are filled. This is because all the segments are running in the same direction, and so a ray drawn from any point will cross one or more segments and the sum of the crossings will not equal zero:



In the image above the red arrows represent the direction the segments are drawn, and black arrow represents an arbitrary ray running from a point in the innermost ring. Starting with a value of zero, for each segment that the ray crosses, a value of one is added because the segment crosses the ray from left to right.

A more complex shape with segments running in different directions is required to better demonstrate the behavior of the [Nonzero](#) fill rule. The following XAML example creates a similar shape to the previous example, except that it's created with a [PathGeometry](#) rather than an [EllipseGeometry](#):

XAML

```
<Path Stroke="Black"
      Fill="#CCCCFF">
  <Path.Data>
    <GeometryGroup FillRule="Nonzero">
      <PathGeometry>
        <PathGeometry.Figures>
          <!-- Inner ring -->
          <PathFigure StartPoint="120,120">
            <PathFigure.Segments>
              <PathSegmentCollection>
                <ArcSegment Size="50,50"
                           IsLargeArc="True"

SweepDirection="CounterClockwise"
                           Point="140,120" />
              </PathSegmentCollection>
            </PathFigure.Segments>
          </PathFigure>

          <!-- Second ring -->
          <PathFigure StartPoint="120,100">
            <PathFigure.Segments>
              <PathSegmentCollection>
                <ArcSegment Size="70,70"
                           IsLargeArc="True"

SweepDirection="CounterClockwise"
                           Point="140,100" />
              </PathSegmentCollection>
            </PathFigure.Segments>
          </PathFigure>
        </PathGeometry.Figures>
      </PathGeometry>
    </GeometryGroup>
  </Path.Data>
</Path>
```

```

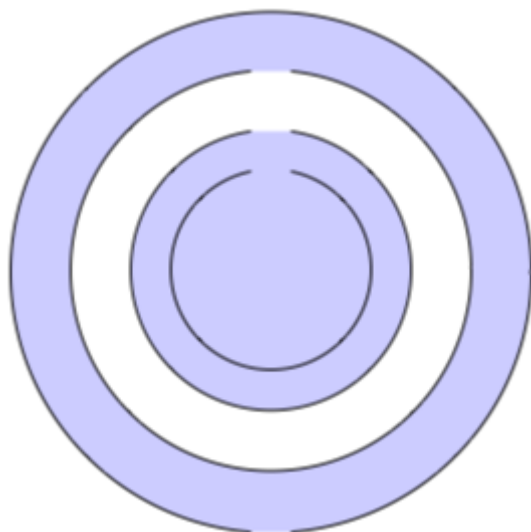
        </PathSegmentCollection>
    </PathFigure.Segments>
</PathFigure>

<!-- Third ring -->
    <PathFigure StartPoint="120,70">
        <PathFigure.Segments>
            <PathSegmentCollection>
                <ArcSegment Size="100,100"
                    IsLargeArc="True"
SweepDirection="CounterClockwise"
Point="140,70" />
            </PathSegmentCollection>
        </PathFigure.Segments>
    </PathFigure>

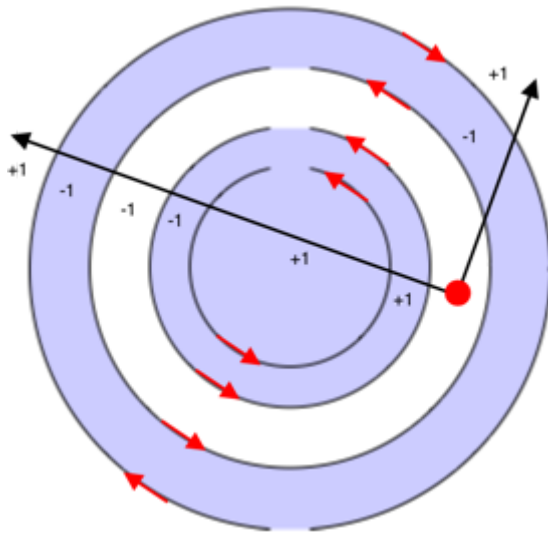
    <!-- Outer ring -->
    <PathFigure StartPoint="120,300">
        <PathFigure.Segments>
            <ArcSegment Size="130,130"
                IsLargeArc="True"
                SweepDirection="Clockwise"
                Point="140,300" />
        </PathFigure.Segments>
    </PathFigure>
</PathGeometry.Figures>
</PathGeometry>
</GeometryGroup>
</Path.Data>
</Path>

```

In this example, a series of arc segments are drawn, that aren't closed:



In the image above, the third arc from the center is not filled. This is because the sum of the values from a given ray crossing the segments in its path is zero:



In the image above, the red circle represents a point, the black lines represent arbitrary rays that move out from the point in the non-filled region, and the red arrows represent the direction the segments are drawn. As can be seen, the sum of the values from the rays crossing the segments is zero:

- The arbitrary ray that travels diagonally right crosses two segments that run in different directions. Therefore, the segments cancel each other out giving a value of zero.
- The arbitrary ray that travels diagonally left crosses a total of six segments. However, the crossings cancel each other out so that zero is the final sum.

A sum of zero results in the ring not being filled.

Geometries

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [Geometry](#) class, and the classes that derive from it, enable you to describe the geometry of a 2D shape. [Geometry](#) objects can be simple, such as rectangles and circles, or composite, created from two or more geometry objects. In addition, more complex geometries can be created that include arcs and curves.

The [Geometry](#) class is the parent class for several classes that define different categories of geometries:

- [EllipseGeometry](#), which represents the geometry of an ellipse or circle.
- [GeometryGroup](#), which represents a container that can combine multiple geometry objects into a single object.
- [LineGeometry](#), which represents the geometry of a line.
- [PathGeometry](#), which represents the geometry of a complex shape that can be composed of arcs, curves, ellipses, lines, and rectangles.
- [RectangleGeometry](#), which represents the geometry of a rectangle or square.

Note

There's also a [RoundRectangleGeometry](#) class that derives from the [GeometryGroup](#) class. For more information, see [RoundRectangleGeometry](#).

The [Geometry](#) and [Shape](#) classes seem similar, in that they both describe 2D shapes, but have an important difference. The [Geometry](#) class derives from the [BindableObject](#) class, while the [Shape](#) class derives from the [View](#) class. Therefore, [Shape](#) objects can render themselves and participate in the layout system, while [Geometry](#) objects cannot. While [Shape](#) objects are more readily usable than [Geometry](#) objects, [Geometry](#) objects are more versatile. While a [Shape](#) object is used to render 2D graphics, a [Geometry](#) object can be used to define the geometric region for 2D graphics, and define a region for clipping.

The following classes have properties that can be set to [Geometry](#) objects:

- The [Path](#) class uses a [Geometry](#) to describe its contents. You can render a [Geometry](#) by setting the `Path.Data` property to a [Geometry](#) object, and setting the [Path](#) object's [Fill](#) and [Stroke](#) properties.

- The [VisualElement](#) class has a [Clip](#) property, of type [Geometry](#), that defines the outline of the contents of an element. When the [Clip](#) property is set to a [Geometry](#) object, only the area that is within the region of the [Geometry](#) will be visible. For more information, see [Clip with a Geometry](#).

The classes that derive from the [Geometry](#) class can be grouped into three categories: simple geometries, path geometries, and composite geometries.

Simple geometries

The simple geometry classes are [EllipseGeometry](#), [LineGeometry](#), and [RectangleGeometry](#). They are used to create basic geometric shapes, such as circles, lines, and rectangles. These same shapes, as well as more complex shapes, can be created using a [PathGeometry](#) or by combining geometry objects together, but these classes provide a simpler approach for producing these basic geometric shapes.

EllipseGeometry

An ellipse geometry represents the geometry of an ellipse or circle, and is defined by a center point, an x-radius, and a y-radius.

The [EllipseGeometry](#) class defines the following properties:

- [Center](#), of type `Point`, which represents the center point of the geometry.
- [RadiusX](#), of type `double`, which represents the x-radius value of the geometry. The default value of this property is 0.0.
- [RadiusY](#), of type `double`, which represents the y-radius value of the geometry. The default value of this property is 0.0.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

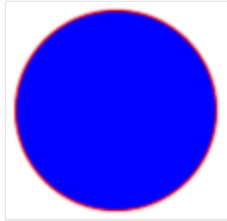
The following example shows how to create and render an [EllipseGeometry](#) in a [Path](#) object:

XAML

```
<Path Fill="Blue"
      Stroke="Red">
  <Path.Data>
    <EllipseGeometry Center="50,50"
                     RadiusX="50"
                     RadiusY="50" />
  </Path.Data>
</Path>
```

```
</Path.Data>  
</Path>
```

In this example, the center of the [EllipseGeometry](#) is set to (50,50) and the x-radius and y-radius are both set to 50. This creates a red circle with a diameter of 100 device-independent units, whose interior is painted blue:



LineGeometry

A line geometry represents the geometry of a line, and is defined by specifying the start point of the line and the end point.

The [LineGeometry](#) class defines the following properties:

- [StartPoint](#), of type `Point`, which represents the start point of the line.
- [EndPoint](#), of type `Point`, which represents the end point of the line.

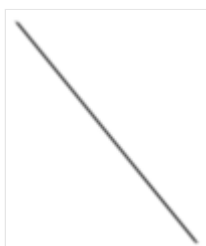
These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The following example shows how to create and render a [LineGeometry](#) in a [Path](#) object:

XAML

```
<Path Stroke="Black">  
  <Path.Data>  
    <LineGeometry StartPoint="10,20"  
                  EndPoint="100,130" />  
  </Path.Data>  
</Path>
```

In this example, a [LineGeometry](#) is drawn from (10,20) to (100,130):



ⓘ Note

Setting the [Fill](#) property of a [Path](#) that renders a [LineGeometry](#) will have no effect, because a line has no interior.

RectangleGeometry

A rectangle geometry represents the geometry of a rectangle or square, and is defined with a `Rect` structure that specifies its relative position and its height and width.

The [RectangleGeometry](#) class defines the `Rect` property, of type `Rect`, which represents the dimensions of the rectangle. This property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled.

The following example shows how to create and render a [RectangleGeometry](#) in a [Path](#) object:

XAML

```
<Path Fill="Blue"
      Stroke="Red">
  <Path.Data>
    <RectangleGeometry Rect="10,10,150,100" />
  </Path.Data>
</Path>
```

The position and dimensions of the rectangle are defined by a `Rect` structure. In this example, the position is (10,10), the width is 150, and the height is 100 device-independent units:



Path geometries

A path geometry describes a complex shape that can be composed of arcs, curves, ellipses, lines, and rectangles.

The [PathGeometry](#) class defines the following properties:

- [Figures](#), of type [PathFigureCollection](#), which represents the collection of [PathFigure](#) objects that describe the path's contents.
- [FillRule](#), of type [FillRule](#), which determines how the intersecting areas contained in the geometry are combined. The default value of this property is `FillRule.EvenOdd`.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

For more information about the [FillRule](#) enumeration, see [.NET MAUI Shapes: Fill rules](#).

ⓘ Note

The [Figures](#) property is the [ContentProperty](#) of the [PathGeometry](#) class, and so does not need to be explicitly set from XAML.

A [PathGeometry](#) is made up of a collection of [PathFigure](#) objects, with each [PathFigure](#) describing a shape in the geometry. Each [PathFigure](#) is itself comprised of one or more [PathSegment](#) objects, each of which describes a segment of the shape. There are many types of segments:

- [ArcSegment](#), which creates an elliptical arc between two points.
- [BezierSegment](#), which creates a cubic Bezier curve between two points.
- [LineSegment](#), which creates a line between two points.
- [PolyBezierSegment](#), which creates a series of cubic Bezier curves.
- [PolyLineSegment](#), which creates a series of lines.
- [PolyQuadraticBezierSegment](#), which creates a series of quadratic Bezier curves.
- [QuadraticBezierSegment](#), which creates a quadratic Bezier curve.

All the above classes derive from the abstract [PathSegment](#) class.

The segments within a [PathFigure](#) are combined into a single geometric shape with the end point of each segment being the start point of the next segment. The [StartPoint](#) property of a [PathFigure](#) specifies the point from which the first segment is drawn. Each subsequent segment starts at the end point of the previous segment. For example, a vertical line from `10,50` to `10,150` can be defined by setting the [StartPoint](#) property to `10,50` and creating a [LineSegment](#) with a [Point](#) property setting of `10,150`:

XAML

```
<Path Stroke="Black">
  <Path.Data>
    <PathGeometry>
```

```

        <PathGeometry.Figures>
            <PathFigureCollection>
                <PathFigure StartPoint="10,50">
                    <PathFigure.Segments>
                        <PathSegmentCollection>
                            <LineSegment Point="10,150" />
                        </PathSegmentCollection>
                    </PathFigure.Segments>
                </PathFigure>
            </PathFigureCollection>
        </PathGeometry.Figures>
    </PathGeometry>
</Path.Data>
</Path>

```

More complex geometries can be created by using a combination of [PathSegment](#) objects, and by using multiple [PathFigure](#) objects within a [PathGeometry](#).

Create an ArcSegment

An [ArcSegment](#) creates an elliptical arc between two points. An elliptical arc is defined by its start and end points, x- and y-radius, x-axis rotation factor, a value indicating whether the arc should be greater than 180 degrees, and a value describing the direction in which the arc is drawn.

The [ArcSegment](#) class defines the following properties:

- [Point](#), of type `Point`, which represents the endpoint of the elliptical arc. The default value of this property is (0,0).
- [Size](#), of type `Size`, which represents the x- and y-radius of the arc. The default value of this property is (0,0).
- [RotationAngle](#), of type `double`, which represents the amount in degrees by which the ellipse is rotated around the x-axis. The default value of this property is 0.
- [SweepDirection](#), of type [SweepDirection](#), which specifies the direction in which the arc is drawn. The default value of this property is `SweepDirection.CounterClockwise`.
- [IsLargeArc](#), of type `bool`, which indicates whether the arc should be greater than 180 degrees. The default value of this property is `false`.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

ⓘ Note

The [ArcSegment](#) class does not contain a property for the starting point of the arc. It only defines the end point of the arc it represents. The start point of the arc is the current point of the [PathFigure](#) to which the [ArcSegment](#) is added.

The [SweepDirection](#) enumeration defines the following members:

- [CounterClockwise](#), which specifies that arcs are drawn in a counter clockwise direction.
- [Clockwise](#), which specifies that arcs are drawn in a clockwise direction.

The following example shows how to create and render an [ArcSegment](#) in a [Path](#) object:

XAML

```
<Path Stroke="Black">
  <Path.Data>
    <PathGeometry>
      <PathGeometry.Figures>
        <PathFigureCollection>
          <PathFigure StartPoint="10,10">
            <PathFigure.Segments>
              <PathSegmentCollection>
                <ArcSegment Size="100,50"
                           RotationAngle="45"
                           IsLargeArc="True"

SweepDirection="CounterClockwise"
                           Point="200,100" />
              </PathSegmentCollection>
            </PathFigure.Segments>
          </PathFigure>
        </PathFigureCollection>
      </PathGeometry.Figures>
    </PathGeometry>
  </Path.Data>
</Path>
```

In this example, an elliptical arc is drawn from (10,10) to (200,100).

Create a BezierSegment

A [BezierSegment](#) creates a cubic Bezier curve between two points. A cubic Bezier curve is defined by four points: a start point, an end point, and two control points.

The [BezierSegment](#) class defines the following properties:

- [Point1](#), of type `Point`, which represents the first control point of the curve. The default value of this property is (0,0).
- [Point2](#), of type `Point`, which represents the second control point of the curve. The default value of this property is (0,0).
- [Point3](#), of type `Point`, which represents the end point of the curve. The default value of this property is (0,0).

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

ⓘ Note

The [BezierSegment](#) class does not contain a property for the starting point of the curve. The start point of the curve is the current point of the [PathFigure](#) to which the [BezierSegment](#) is added.

The two control points of a cubic Bezier curve behave like magnets, attracting portions of what would otherwise be a straight line toward themselves and producing a curve. The first control point affects the start portion of the curve. The second control point affects the end portion of the curve. The curve doesn't necessarily pass through either of the control points. Instead, each control point moves its portion of the line toward itself, but not through itself.

The following example shows how to create and render a [BezierSegment](#) in a [Path](#) object:

XAML

```
<Path Stroke="Black">
  <Path.Data>
    <PathGeometry>
      <PathGeometry.Figures>
        <PathFigureCollection>
          <PathFigure StartPoint="10,10">
            <PathFigure.Segments>
              <PathSegmentCollection>
                <BezierSegment Point1="100,0"
                              Point2="200,200"
                              Point3="300,10" />
              </PathSegmentCollection>
            </PathFigure.Segments>
          </PathFigure>
        </PathFigureCollection>
      </PathGeometry.Figures>
    </PathGeometry>
  </Path.Data>
</Path>
```

```
</Path.Data>
</Path>
```

In this example, a cubic Bezier curve is drawn from (10,10) to (300,10). The curve has two control points at (100,0) and (200,200):



Create a LineSegment

A [LineSegment](#) creates a line between two points.

The [LineSegment](#) class defines the `Point` property, of type `Point`, which represents the end point of the line segment. The default value of this property is (0,0), and it's backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled.

❗ Note

The [LineSegment](#) class does not contain a property for the starting point of the line. It only defines the end point. The start point of the line is the current point of the [PathFigure](#) to which the [LineSegment](#) is added.

The following example shows how to create and render [LineSegment](#) objects in a [Path](#) object:

XAML

```
<Path Stroke="Black"
      Aspect="Uniform"
      HorizontalOptions="Start">
  <Path.Data>
    <PathGeometry>
      <PathGeometry.Figures>
        <PathFigureCollection>
          <PathFigure IsClosed="True"
                      StartPoint="10,100">
            <PathFigure.Segments>
              <PathSegmentCollection>
                <LineSegment Point="100,100" />
                <LineSegment Point="100,50" />
              </PathSegmentCollection>
            </PathFigure.Segments>
          </PathFigure>
        </PathFigureCollection>
      </PathGeometry.Figures>
    </PathGeometry>
  </Path.Data>
</Path>
```



```

        </PathFigure>
    </PathFigureCollection>
</PathGeometry.Figures>
</PathGeometry>
</Path.Data>
</Path>

```

In this example, a line segment is drawn from (10,100) to (100,100), and from (100,100) to (100,50). In addition, the [PathFigure](#) is closed because its `IsClosed` property is set to `true`. This results in a triangle being drawn:



Create a PolyBezierSegment

A [PolyBezierSegment](#) creates one or more cubic Bezier curves.

The [PolyBezierSegment](#) class defines the `Points` property, of type [PointCollection](#), which represents the points that define the [PolyBezierSegment](#). A [PointCollection](#) is an [ObservableCollection](#) of [Point](#) objects. This property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled.

ⓘ Note

The [PolyBezierSegment](#) class does not contain a property for the starting point of the curve. The start point of the curve is the current point of the [PathFigure](#) to which the [PolyBezierSegment](#) is added.

The following example shows how to create and render a [PolyBezierSegment](#) in a [Path](#) object:

XAML

```

<Path Stroke="Black">
    <Path.Data>
        <PathGeometry>
            <PathGeometry.Figures>
                <PathFigureCollection>
                    <PathFigure StartPoint="10,10">
                        <PathFigure.Segments>
                            <PathSegmentCollection>
                                <PolyBezierSegment Points="0,0 100,0 150,100
150,0 200,0 300,10" />

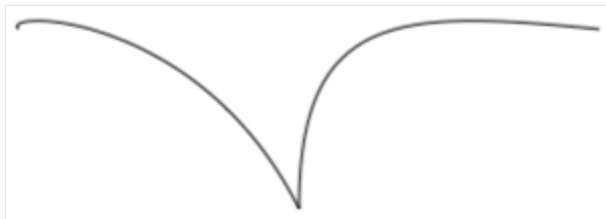
```

```

        </PathSegmentCollection>
    </PathFigure.Segments>
</PathFigure>
</PathFigureCollection>
</PathGeometry.Figures>
</PathGeometry>
</Path.Data>
</Path>

```

In this example, the [PolyBezierSegment](#) specifies two cubic Bezier curves. The first curve is from (10,10) to (150,100) with a control point of (0,0), and another control point of (100,0). The second curve is from (150,100) to (300,10) with a control point of (150,0) and another control point of (200,0):



Create a PolyLineSegment

A [PolyLineSegment](#) creates one or more line segments.

The [PolyLineSegment](#) class defines the `Points` property, of type [PointCollection](#), which represents the points that define the [PolyLineSegment](#). A [PointCollection](#) is an [ObservableCollection](#) of [Point](#) objects. This property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled.

⚠ Note

The [PolyLineSegment](#) class does not contain a property for the starting point of the line. The start point of the line is the current point of the [PathFigure](#) to which the [PolyLineSegment](#) is added.

The following example shows how to create and render a [PolyLineSegment](#) in a [Path](#) object:

XAML

```

<Path Stroke="Black">
    <Path.Data>
        <PathGeometry>
            <PathGeometry.Figures>
                <PathFigure StartPoint="10,10">

```

```

        <PathFigure.Segments>
            <PolyLineSegment Points="50,10 50,50" />
        </PathFigure.Segments>
    </PathFigure>
</PathGeometry.Figures>
</PathGeometry>
</Path.Data>
</Path>

```

In this example, the [PolyLineSegment](#) specifies two lines. The first line is from (10,10) to (50,10), and the second line is from (50,10) to (50,50):



Create a PolyQuadraticBezierSegment

A [PolyQuadraticBezierSegment](#) creates one or more quadratic Bezier curves.

The [PolyQuadraticBezierSegment](#) class defines the `Points` property, of type [PointCollection](#), which represents the points that define the [PolyQuadraticBezierSegment](#). A [PointCollection](#) is an `ObservableCollection` of [Point](#) objects. This property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled.

ⓘ Note

The [PolyQuadraticBezierSegment](#) class does not contain a property for the starting point of the curve. The start point of the curve is the current point of the [PathFigure](#) to which the [PolyQuadraticBezierSegment](#) is added.

The following example shows to create and render a [PolyQuadraticBezierSegment](#) in a [Path](#) object:

XAML

```

<Path Stroke="Black">
    <Path.Data>
        <PathGeometry>
            <PathGeometry.Figures>
                <PathFigureCollection>
                    <PathFigure StartPoint="10,10">
                        <PathFigure.Segments>
                            <PathSegmentCollection>
                                <PolyQuadraticBezierSegment Points="100,100

```

```

150,50 0,100 15,200" />
        </PathSegmentCollection>
    </PathFigure.Segments>
</PathFigure>
</PathFigureCollection>
</PathGeometry.Figures>
</PathGeometry>
</Path.Data>
</Path>

```

In this example, the [PolyQuadraticBezierSegment](#) specifies two Bezier curves. The first curve is from (10,10) to (150,50) with a control point at (100,100). The second curve is from (100,100) to (15,200) with a control point at (0,100):



Create a QuadraticBezierSegment

A [QuadraticBezierSegment](#) creates a quadratic Bezier curve between two points.

The [QuadraticBezierSegment](#) class defines the following properties:

- [Point1](#), of type `Point`, which represents the control point of the curve. The default value of this property is (0,0).
- [Point2](#), of type `Point`, which represents the end point of the curve. The default value of this property is (0,0).

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

ⓘ Note

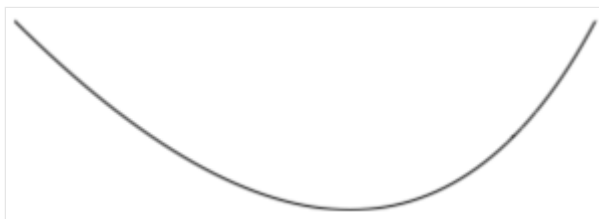
The [QuadraticBezierSegment](#) class does not contain a property for the starting point of the curve. The start point of the curve is the current point of the [PathFigure](#) to which the [QuadraticBezierSegment](#) is added.

The following example shows how to create and render a [QuadraticBezierSegment](#) in a [Path](#) object:

XAML

```
<Path Stroke="Black">
  <Path.Data>
    <PathGeometry>
      <PathGeometry.Figures>
        <PathFigureCollection>
          <PathFigure StartPoint="10,10">
            <PathFigure.Segments>
              <PathSegmentCollection>
                <QuadraticBezierSegment Point1="200,200"
                                         Point2="300,10" />
              </PathSegmentCollection>
            </PathFigure.Segments>
          </PathFigure>
        </PathFigureCollection>
      </PathGeometry.Figures>
    </PathGeometry>
  </Path.Data>
</Path>
```

In this example, a quadratic Bezier curve is drawn from (10,10) to (300,10). The curve has a control point at (200,200):



Create complex geometries

More complex geometries can be created by using a combination of [PathSegment](#) objects. The following example creates a shape using a [BezierSegment](#), a [LineSegment](#), and an [ArcSegment](#):

XAML

```
<Path Stroke="Black">
  <Path.Data>
    <PathGeometry>
      <PathGeometry.Figures>
        <PathFigure StartPoint="10,50">
          <PathFigure.Segments>
            <BezierSegment Point1="100,0"
                           Point2="200,200">
```

```

        Point3="300,100"/>
        <LineSegment Point="400,100" />
        <ArcSegment Size="50,50"
            RotationAngle="45"
            IsLargeArc="True"
            SweepDirection="Clockwise"
            Point="200,100"/>
    </PathFigure.Segments>
</PathFigure>
</PathGeometry.Figures>
</PathGeometry>
</Path.Data>
</Path>

```

In this example, a [BezierSegment](#) is first defined using four points. The example then adds a [LineSegment](#), which is drawn between the end point of the [BezierSegment](#) to the point specified by the [LineSegment](#). Finally, an [ArcSegment](#) is drawn from the end point of the [LineSegment](#) to the point specified by the [ArcSegment](#).

Even more complex geometries can be created by using multiple [PathFigure](#) objects within a [PathGeometry](#). The following example creates a [PathGeometry](#) from seven [PathFigure](#) objects, some of which contain multiple [PathSegment](#) objects:

XAML

```

<Path Stroke="Red"
    StrokeThickness="12"
    StrokeLineJoin="Round">
    <Path.Data>
        <PathGeometry>
            <!-- H -->
            <PathFigure StartPoint="0,0">
                <LineSegment Point="0,100" />
            </PathFigure>
            <PathFigure StartPoint="0,50">
                <LineSegment Point="50,50" />
            </PathFigure>
            <PathFigure StartPoint="50,0">
                <LineSegment Point="50,100" />
            </PathFigure>

            <!-- E -->
            <PathFigure StartPoint="125, 0">
                <BezierSegment Point1="60, -10"
                    Point2="60, 60"
                    Point3="125, 50" />
                <BezierSegment Point1="60, 40"
                    Point2="60, 110"
                    Point3="125, 100" />
            </PathFigure>
        </PathGeometry>
    </Path.Data>
</Path>

```

```

<!-- L -->
<PathFigure StartPoint="150, 0">
    <LineSegment Point="150, 100" />
    <LineSegment Point="200, 100" />
</PathFigure>

<!-- L -->
<PathFigure StartPoint="225, 0">
    <LineSegment Point="225, 100" />
    <LineSegment Point="275, 100" />
</PathFigure>

<!-- O -->
<PathFigure StartPoint="300, 50">
    <ArcSegment Size="25, 50"
                Point="300, 49.9"
                IsLargeArc="True" />
</PathFigure>
</PathGeometry>
</Path.Data>
</Path>

```

In this example, the word "Hello" is drawn using a combination of [LineSegment](#) and [BezierSegment](#) objects, along with a single [ArcSegment](#) object:



Composite geometries

Composite geometry objects can be created using a [GeometryGroup](#). The [GeometryGroup](#) class creates a composite geometry from one or more [Geometry](#) objects. Any number of [Geometry](#) objects can be added to a [GeometryGroup](#).

The [GeometryGroup](#) class defines the following properties:

- [Children](#), of type [GeometryCollection](#), which specifies the objects that define the [GeometryGroup](#). A [GeometryCollection](#) is an `ObservableCollection` of [Geometry](#) objects.
- [FillRule](#), of type [FillRule](#), which specifies how the intersecting areas in the [GeometryGroup](#) are combined. The default value of this property is `FillRule.EvenOdd`.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

❗ Note

The `Children` property is the [ContentProperty](#) of the [GeometryGroup](#) class, and so does not need to be explicitly set from XAML.

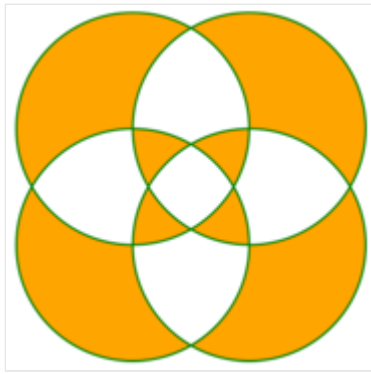
For more information about the [FillRule](#) enumeration, see [Fill rules](#).

To draw a composite geometry, set the required [Geometry](#) objects as the children of a [GeometryGroup](#), and display them with a [Path](#) object. The following XAML shows an example of this:

XAML

```
<Path Stroke="Green"
      StrokeThickness="2"
      Fill="Orange">
  <Path.Data>
    <GeometryGroup>
      <EllipseGeometry RadiusX="100"
                       RadiusY="100"
                       Center="150,150" />
      <EllipseGeometry RadiusX="100"
                       RadiusY="100"
                       Center="250,150" />
      <EllipseGeometry RadiusX="100"
                       RadiusY="100"
                       Center="150,250" />
      <EllipseGeometry RadiusX="100"
                       RadiusY="100"
                       Center="250,250" />
    </GeometryGroup>
  </Path.Data>
</Path>
```

In this example, four [EllipseGeometry](#) objects with identical x-radius and y-radius coordinates, but with different center coordinates, are combined. This creates four overlapping circles, whose interiors are filled orange due to the default [EvenOdd](#) fill rule:



RoundRectangleGeometry

A round rectangle geometry represents the geometry of a rectangle, or square, with rounded corners, and is defined by a corner radius and a `Rect` structure that specifies its relative position and its height and width.

The [RoundRectangleGeometry](#) class, which derives from the [GeometryGroup](#) class, defines the following properties:

- [CornerRadius](#), of type `CornerRadius`, which is the corner radius of the geometry.
- [Rect](#), of type `Rect`, which represents the dimensions of the rectangle.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

ⓘ Note

The fill rule used by the [RoundRectangleGeometry](#) is `FillRule.Nonzero`. For more information about fill rules, see [Fill rules](#).

The following example shows how to create and render a [RoundRectangleGeometry](#) in a [Path](#) object:

XAML

```
<Path Fill="Blue"
      Stroke="Red">
  <Path.Data>
    <RoundRectangleGeometry CornerRadius="5"
                           Rect="10,10,150,100" />
  </Path.Data>
</Path>
```

The position and dimensions of the rectangle are defined by a `Rect` structure. In this example, the position is (10,10), the width is 150, and the height is 100 device-independent units. In addition, the rectangle corners are rounded with a radius of 5 device-independent units.

Clip with a Geometry

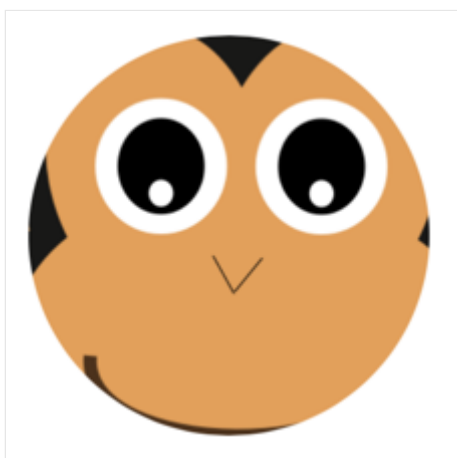
The `VisualElement` class has a `Clip` property, of type `Geometry`, that defines the outline of the contents of an element. When the `Clip` property is set to a `Geometry` object, only the area that is within the region of the `Geometry` will be visible.

The following example shows how to use a `Geometry` object as the clip region for an `Image`:

XAML

```
<Image Source="monkeyface.png">
  <Image.Clip>
    <EllipseGeometry RadiusX="100"
                      RadiusY="100"
                      Center="180,180" />
  </Image.Clip>
</Image>
```

In this example, an `EllipseGeometry` with `RadiusX` and `RadiusY` values of 100, and a `Center` value of (180,180) is set to the `Clip` property of an `Image`. Only the part of the image that is within the area of the ellipse will be displayed:



ⓘ Note

Simple geometries, path geometries, and composite geometries can all be used to clip `VisualElement` objects.

Other features

The [GeometryHelper](#) class provides the following helper methods:

- [FlattenGeometry](#), which flattens a [Geometry](#) into a [PathGeometry](#).
- [FlattenCubicBezier](#), which flattens a cubic Bezier curve into a `List<Point>` collection.
- [FlattenQuadraticBezier](#), which flattens a quadratic Bezier curve into a `List<Point>` collection.
- [FlattenArc](#), which flattens an elliptical arc into a `List<Point>` collection.

Line

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [Line](#) class derives from the [Shape](#) class, and can be used to draw lines. For information on the properties that the [Line](#) class inherits from the [Shape](#) class, see [Shapes](#).

[Line](#) defines the following properties:

- [X1](#), of type double, indicates the x-coordinate of the start point of the line. The default value of this property is 0.0.
- [Y1](#), of type double, indicates the y-coordinate of the start point of the line. The default value of this property is 0.0.
- [X2](#), of type double, indicates the x-coordinate of the end point of the line. The default value of this property is 0.0.
- [Y2](#), of type double, indicates the y-coordinate of the end point of the line. The default value of this property is 0.0.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

For information about controlling how line ends are drawn, see [Control line ends](#).

Create a Line

To draw a line, create a [Line](#) object and set its `X1` and `Y1` properties to its start point, and its `X2` and `Y2` properties to its end point. In addition, set its [Stroke](#) property to a [Brush](#)-derived object because a line without a stroke is invisible. For more information about [Brush](#) objects, see [Brushes](#).

Note

Setting the [Fill](#) property of a [Line](#) has no effect, because a line has no interior.

The following XAML example shows how to draw a line:

XAML

```
<Line X1="40"  
      Y1="0"
```

```
X2="0"  
Y2="120"  
Stroke="Red" />
```

In this example, a red diagonal line is drawn from (40,0) to (0,120):



Because the `X1`, `Y1`, `X2`, and `Y2` properties have default values of 0, it's possible to draw some lines with minimal syntax:

XAML

```
<Line Stroke="Red"  
      X2="200" />
```

In this example, a horizontal line that's 200 device-independent units long is defined. Because the other properties are 0 by default, a line is drawn from (0,0) to (200,0).

The following XAML example shows how to draw a dashed line:

XAML

```
<Line X1="40"  
      Y1="0"  
      X2="0"  
      Y2="120"  
      Stroke="DarkBlue"  
      StrokeDashArray="1,1"  
      StrokeDashOffset="6" />
```

In this example, a dark blue dashed diagonal line is drawn from (40,0) to (0,120):



For more information about drawing a dashed line, see [Draw dashed shapes](#).

Path

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [Path](#) class derives from the [Shape](#) class, and can be used to draw curves and complex shapes. These curves and shapes are often described using [Geometry](#) objects. For information on the properties that the [Path](#) class inherits from the [Shape](#) class, see [Shapes](#).

[Path](#) defines the following properties:

- [Data](#), of type [Geometry](#), which specifies the shape to be drawn.
- [RenderTransform](#), of type [Transform](#), which represents the transform that is applied to the geometry of a path prior to it being drawn.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

For more information about transforms, see [Path Transforms](#).

Create a Path

To draw a path, create a [Path](#) object and set its [Data](#) property. There are two techniques for setting the [Data](#) property:

- You can set a string value for [Data](#) in XAML, using path markup syntax. With this approach, the `Path.Data` value is consuming a serialization format for graphics. Typically, you don't edit this string value by hand after it's created. Instead, you use design tools to manipulate the data, and export it as a string fragment that's consumable by the [Data](#) property.
- You can set the [Data](#) property to a [Geometry](#) object. This can be a specific [Geometry](#) object, or a [GeometryGroup](#) which acts as a container that can combine multiple geometry objects into a single object.

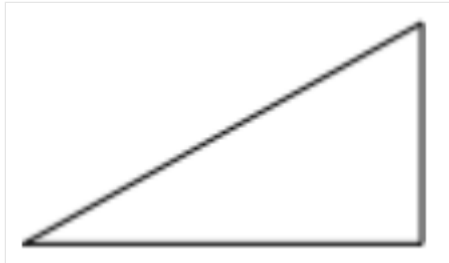
Create a Path with path markup syntax

The following XAML example shows how to draw a triangle using path markup syntax:

```
XAML
```

```
<Path Data="M 10,100 L 100,100 100,50Z"
      Stroke="Black"
      Aspect="Uniform"
      HorizontalOptions="Start" />
```

The [Data](#) string begins with the move command, indicated by **M**, which establishes an absolute start point for the path. **L** is the line command, which creates a straight line from the start point to the specified end point. **Z** is the close command, which creates a line that connects the current point to the starting point. The result is a triangle:



For more information about path markup syntax, see [Path markup syntax](#).

Create a Path with Geometry objects

Curves and shapes can be described using [Geometry](#) objects, which are used to set the [Path](#) object's [Data](#) property. There are a variety of [Geometry](#) objects to choose from. The [EllipseGeometry](#), [LineGeometry](#), and [RectangleGeometry](#) classes describe relatively simple shapes. To create more complex shapes or create curves, use a [PathGeometry](#).

[PathGeometry](#) objects are comprised of one or more [PathFigure](#) objects. Each [PathFigure](#) object represents a different shape. Each [PathFigure](#) object is itself comprised of one or more [PathSegment](#) objects, each representing a connection portion of the shape. Segment types include the following the [LineSegment](#), [BezierSegment](#), and [ArcSegment](#) classes.

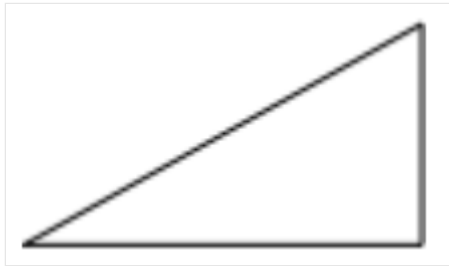
The following XAML example shows how to draw a triangle using a [PathGeometry](#) object:

XAML

```
<Path Stroke="Black"
      Aspect="Uniform"
      HorizontalOptions="Start">
  <Path.Data>
    <PathGeometry>
      <PathGeometry.Figures>
        <PathFigureCollection>
          <PathFigure IsClosed="True"
```

```
        StartPoint="10,100">
        <PathFigure.Segments>
            <PathSegmentCollection>
                <LineSegment Point="100,100" />
                <LineSegment Point="100,50" />
            </PathSegmentCollection>
        </PathFigure.Segments>
    </PathFigure>
</PathFigureCollection>
</PathGeometry.Figures>
</PathGeometry>
</Path.Data>
</Path>
```

In this example, the start point of the triangle is (10,100). A line segment is drawn from (10,100) to (100,100), and from (100,100) to (100,50). Then the figures first and last segments are connected, because the `PathFigure.IsClosed` property is set to `true`. The result is a triangle:



For more information about geometries, see [Geometries](#).

Path markup syntax

Article • 08/30/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) path markup syntax enables you to compactly specify path geometries in XAML.

Path markup syntax is specified as a string value to the `Path.Data` property:

XAML

```
<Path Stroke="Black"
      Data="M13.908992,16.207977 L32.000049,16.207977 32.000049,31.999985
13.908992,30.109983Z" />
```

Path markup syntax is composed of an optional `FillRule` value, and one or more figure descriptions. This syntax can be expressed as: `<Path Data=" [fillRule] figureDescription [figureDescription] * " ... />`

In this syntax:

- *fillRule* is an optional `FillRule` that specifies whether the geometry should use the `EvenOdd` or `Nonzero` fill rule. `F0` is used to specify the `EvenOdd` fill rule, while `F1` is used to specify the `Nonzero` fill rule. For more information about fill rules, see [Fill rules](#).
- *figureDescription* represents a figure composed of a move command, draw commands, and an optional close command. A move command specifies the start point of the figure. Draw commands describe the figure's contents, and the optional close command closes the figure.

In the example above, the path markup syntax specifies a start point using the move command (`M`), a series of straight lines using the line command (`L`), and closes the path with the close command (`Z`).

In path markup syntax, spaces are not required before or after commands. In addition, two numbers don't have to be separated by a comma or white space, but this can only be achieved when the string is unambiguous.

 **Tip**

Path markup syntax is compatible with Scalable Vector Graphics (SVG) image path definitions, and so it can be useful for porting graphics from SVG format.

While path markup syntax is intended for consumption in XAML, it can be converted to a [Geometry](#) object in code by invoking the `ConvertFromInvariantString` method in the [PathGeometryConverter](#) class:

C#

```
Geometry pathData = (Geometry)new  
PathGeometryConverter().ConvertFromInvariantString("M13.908992,16.207977  
L32.000049,16.207977 32.000049,31.999985 13.908992,30.109983Z");
```

Move command

The move command specifies the start point of a new figure. The syntax for this command is: `M` *startPoint* or `m` *startPoint*.

In this syntax, *startPoint* is a `Point` structure that specifies the start point of a new figure. If you list multiple points after the move command, a line is drawn to those points.

`M 10,10` is an example of a valid move command.

Draw commands

A draw command can consist of several shape commands. The following draw commands are available:

- Line (`L` or `l`).
- Horizontal line (`H` or `h`).
- Vertical line (`V` or `v`).
- Elliptical arc (`A` or `a`).
- Cubic Bezier curve (`C` or `c`).
- Quadratic Bezier curve (`Q` or `q`).
- Smooth cubic Bezier curve (`S` or `s`).
- Smooth quadratic Bezier curve (`T` or `t`).

Each draw command is specified with a case-insensitive letter. When sequentially entering more than one command of the same type, you can omit the duplicate command entry. For example `L 100,200 300,400` is equivalent to `L 100,200 L 300,400`.

Line command

The line command creates a straight line between the current point and the specified end point. The syntax for this command is: `L endPoint` or `l endPoint`.

In this syntax, *endPoint* is a `Point` that represents the end point of the line.

`L 20,30` and `L 20 30` are examples of valid line commands.

For information about creating a straight line as a [PathGeometry](#) object, see [Create a LineSegment](#).

Horizontal line command

The horizontal line command creates a horizontal line between the current point and the specified x-coordinate. The syntax for this command is: `H x` or `h x`.

In this syntax, *x* is a `double` that represents the x-coordinate of the end point of the line.

`H 90` is an example of a valid horizontal line command.

Vertical line command

The vertical line command creates a vertical line between the current point and the specified y-coordinate. The syntax for this command is: `v y` or `v y`.

In this syntax, *y* is a `double` that represents the y-coordinate of the end point of the line.

`v 90` is an example of a valid vertical line command.

Elliptical arc command

The elliptical arc command creates an elliptical arc between the current point and the specified end point. The syntax for this command is: `A size rotationAngle isLargeArcFlag sweepDirectionFlag endPoint` or `a size rotationAngle isLargeArcFlag sweepDirectionFlag endPoint`.

In this syntax:

- `size` is a `Size` that represents the x- and y-radius of the arc.
- `rotationAngle` is a `double` that represents the rotation of the ellipse, in degrees.
- `isLargeArcFlag` should be set to 1 if the angle of the arc should be 180 degrees or greater, otherwise set it to 0.

- `sweepDirectionFlag` should be set to 1 if the arc is drawn in a positive-angle direction, otherwise set it to 0.
- `endPoint` is a `Point` to which the arc is drawn.

A `150,150 0 1,0 150,-150` is an example of a valid elliptical arc command.

For information about creating an elliptical arc as a [PathGeometry](#) object, see [Create an ArcSegment](#).

Cubic Bezier curve command

The cubic Bezier curve command creates a cubic Bezier curve between the current point and the specified end point by using the two specified control points. The syntax for this command is: `C controlPoint1 controlPoint2 endPoint` or `c controlPoint1 controlPoint2 endPoint`.

In this syntax:

- `controlPoint1` is a `Point` that represents the first control point of the curve, which determines the starting tangent of the curve.
- `controlPoint2` is a `Point` that represents the second control point of the curve, which determines the ending tangent of the curve.
- `endPoint` is a `Point` that represents the point to which the curve is drawn.

`C 100,200 200,400 300,200` is an example of a valid cubic Bezier curve command.

For information about creating a cubic Bezier curve as a [PathGeometry](#) object, see [Create a BezierSegment](#).

Quadratic Bezier curve command

The quadratic Bezier curve command creates a quadratic Bezier curve between the current point and the specified end point by using the specified control point. The syntax for this command is: `Q controlPoint endPoint` or `q controlPoint endPoint`.

In this syntax:

- `controlPoint` is a `Point` that represents the control point of the curve, which determines the starting and ending tangents of the curve.
- `endPoint` is a `Point` that represents the point to which the curve is drawn.

`Q 100,200 300,200` is an example of a valid quadratic Bezier curve command.

For information about creating a quadratic Bezier curve as a [PathGeometry](#) object, see [Create a QuadraticBezierSegment](#).

Smooth cubic Bezier curve command

The smooth cubic Bezier curve command creates a cubic Bezier curve between the current point and the specified end point by using the specified control point. The syntax for this command is: `S controlPoint2 endPoint` or `s controlPoint2 endPoint`.

In this syntax:

- `controlPoint2` is a `Point` that represents the second control point of the curve, which determines the ending tangent of the curve.
- `endPoint` is a `Point` that represents the point to which the curve is drawn.

The first control point is assumed to be the reflection of the second control point of the previous command, relative to the current point. If there is no previous command, or the previous command was not a cubic Bezier curve command or a smooth cubic Bezier curve command, the first control point is assumed to be coincident with the current point.

`S 100,200 200,300` is an example of a valid smooth cubic Bezier curve command.

Smooth quadratic Bezier curve command

The smooth quadratic Bezier curve command creates a quadratic Bezier curve between the current point and the specified end point by using a control point. The syntax for this command is: `T endPoint` or `t endPoint`.

In this syntax, `endPoint` is a `Point` that represents the point to which the curve is drawn.

The control point is assumed to be the reflection of the control point of the previous command relative to the current point. If there is no previous command or if the previous command was not a quadratic Bezier curve or a smooth quadratic Bezier curve command, the control point is assumed to be coincident with the current point.

`T 100,30` is an example of a valid smooth quadratic cubic Bezier curve command.

Close command

The close command ends the current figure and creates a line that connects the current point to the starting point of the figure. Therefore, this command creates a line-join

between the last segment and the first segment of the figure.

The syntax for the close command is: `Z` or `z`.

Additional values

Instead of a standard numerical value, you can also use the following case-sensitive special values:

- `Infinity` represents `double.PositiveInfinity`.
- `-Infinity` represents `double.NegativeInfinity`.
- `NaN` represents `double.NaN`.

In addition, you may also use case-insensitive scientific notation. Therefore, `+1.e17` is a valid value.

Path transforms

Article • 08/30/2024

 [Browse the sample](#)

A .NET Multi-platform App UI (.NET MAUI) [Transform](#) defines how to transform a [Path](#) object from one coordinate space to another coordinate space. When a transform is applied to a [Path](#) object, it changes how the object is rendered in the UI.

Transforms can be categorized into four general classifications: rotation, scaling, skew, and translation. .NET MAUI defines a class for each of these transform classifications:

- [RotateTransform](#), which rotates a [Path](#) by a specified `Angle`.
- [ScaleTransform](#), which scales a [Path](#) object by specified `ScaleX` and `ScaleY` amounts.
- [SkewTransform](#), which skews a [Path](#) object by specified `AngleX` and `AngleY` amounts.
- [TranslateTransform](#), which moves a [Path](#) object by specified `x` and `y` amounts.

.NET MAUI also provides the following classes for creating more complex transformations:

- [TransformGroup](#), which represents a composite transform composed of multiple transform objects.
- [CompositeTransform](#), which applies multiple transform operations to a [Path](#) object.
- [MatrixTransform](#), which creates custom transforms that are not provided by the other transform classes.

All of these classes derive from the [Transform](#) class, which defines a `Value` property of type [Matrix](#), which represents the current transformation as a [Matrix](#) object. This property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled. For more information about the [Matrix](#) struct, see [Transform matrix](#).

To apply a transform to a [Path](#), you create a transform class and set it as the value of the `Path.RenderTransform` property.

Rotation transform

A rotate transform rotates a [Path](#) object clockwise about a specified point in a 2D x-y coordinate system.

The [RotateTransform](#) class, which derives from the [Transform](#) class, defines the following properties:

- [Angle](#), of type `double`, represents the angle, in degrees, of clockwise rotation. The default value of this property is 0.0.
- [CenterX](#), of type `double`, represents the x-coordinate of the rotation center point. The default value of this property is 0.0.
- [CenterY](#), of type `double`, represents the y-coordinate of the rotation center point. The default value of this property is 0.0.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [CenterX](#) and [CenterY](#) properties specify the point about which the [Path](#) object is rotated. This center point is expressed in the coordinate space of the object that's transformed. By default, the rotation is applied to (0,0), which is the upper-left corner of the [Path](#) object.

The following example shows how to rotate a [Path](#) object:

XAML

```
<Path Stroke="Black"
      Aspect="Uniform"
      HorizontalOptions="Center"
      HeightRequest="100"
      WidthRequest="100"
      Data="M13.908992,16.207977L32.000049,16.207977 32.000049,31.999985
13.908992,30.109983z">
  <Path.RenderTransform>
    <RotateTransform CenterX="0"
                    CenterY="0"
                    Angle="45" />
  </Path.RenderTransform>
</Path>
```

In this example, the [Path](#) object is rotated 45 degrees about its upper-left corner.

Scale transform

A scale transform scales a [Path](#) object in the 2D x-y coordinate system.

The [ScaleTransform](#) class, which derives from the [Transform](#) class, defines the following properties:

- [ScaleX](#), of type `double`, which represents the x-axis scale factor. The default value of this property is 1.0.
- [ScaleY](#), of type `double`, which represents the y-axis scale factor. The default value of this property is 1.0.
- [CenterX](#), of type `double`, which represents the x-coordinate of the center point of this transform. The default value of this property is 0.0.
- [CenterY](#), of type `double`, which represents the y-coordinate of the center point of this transform. The default value of this property is 0.0.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The value of [ScaleX](#) and [ScaleY](#) have a huge impact on the resulting scaling:

- Values between 0 and 1 decrease the width and height of the scaled object.
- Values greater than 1 increase the width and height of the scaled object.
- Values of 1 indicate that the object is not scaled.
- Negative values flip the scale object horizontally and vertically.
- Values between 0 and -1 flip the scale object and decrease its width and height.
- Values less than -1 flip the object and increase its width and height.
- Values of -1 flip the scaled object but do not change its horizontal or vertical size.

The [CenterX](#) and [CenterY](#) properties specify the point about which the [Path](#) object is scaled. This center point is expressed in the coordinate space of the object that's transformed. By default, scaling is applied to (0,0), which is the upper-left corner of the [Path](#) object. This has the effect of moving the [Path](#) object and making it appear larger, because when you apply a transform you change the coordinate space in which the [Path](#) object resides.

The following example shows how to scale a [Path](#) object:

XAML

```
<Path Stroke="Black"
      Aspect="Uniform"
      HorizontalOptions="Center"
      HeightRequest="100"
      WidthRequest="100"
      Data="M13.908992,16.207977L32.000049,16.207977 32.000049,31.999985
13.908992,30.109983z">
  <Path.RenderTransform>
    <ScaleTransform CenterX="0"
                    CenterY="0"
                    ScaleX="1.5"
                    ScaleY="1.5" />
  </Path.RenderTransform>
</Path>
```

```
</Path.RenderTransform>
</Path>
```

In this example, the [Path](#) object is scaled to 1.5 times the size.

Skew transform

A skew transform skews a [Path](#) object in the 2D x-y coordinate system, and is useful for creating the illusion of 3D depth in a 2D object.

The [SkewTransform](#) class, which derives from the [Transform](#) class, defines the following properties:

- [AngleX](#), of type `double`, which represents the x-axis skew angle, which is measured in degrees counterclockwise from the y-axis. The default value of this property is 0.0.
- [AngleY](#), of type `double`, which represents the y-axis skew angle, which is measured in degrees counterclockwise from the x-axis. The default value of this property is 0.0.
- [CenterX](#), of type `double`, which represents the x-coordinate of the transform center. The default value of this property is 0.0.
- [CenterY](#), of type `double`, which represents the y-coordinate of the transform center. The default value of this property is 0.0.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

To predict the effect of a skew transformation, consider that [AngleX](#) skews x-axis values relative to the original coordinate system. Therefore, for an [AngleX](#) of 30, the y-axis rotates 30 degrees through the origin and skews the values in x by 30 degrees from that origin. Similarly, an [AngleY](#) of 30 skews the y values of the [Path](#) object by 30 degrees from the origin.

ⓘ Note

To skew a [Path](#) object in place, set the `CenterX` and `CenterY` properties to the object's center point.

The following example shows how to skew a [Path](#) object:

```
XAML
```

```

<Path Stroke="Black"
      Aspect="Uniform"
      HorizontalOptions="Center"
      HeightRequest="100"
      WidthRequest="100"
      Data="M13.908992,16.207977L32.000049,16.207977 32.000049,31.999985
13.908992,30.109983z">
  <Path.RenderTransform>
    <SkewTransform CenterX="0"
                  CenterY="0"
                  AngleX="45"
                  AngleY="0" />
  </Path.RenderTransform>
</Path>

```

In this example, a horizontal skew of 45 degrees is applied to the [Path](#) object, from a center point of (0,0).

Translate transform

A translate transform moves an object in the 2D x-y coordinate system.

The [TranslateTransform](#) class, which derives from the [Transform](#) class, defines the following properties:

- [X](#), of type `double`, which represents the distance to move along the x-axis. The default value of this property is 0.0.
- [Y](#), of type `double`, which represents the distance to move along the y-axis. The default value of this property is 0.0.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

Negative [X](#) values move an object to the left, while positive values move an object to the right. Negative [Y](#) values move an object up, while positive values move an object down.

The following example shows how to translate a [Path](#) object:

XAML

```

<Path Stroke="Black"
      Aspect="Uniform"
      HorizontalOptions="Center"
      HeightRequest="100"
      WidthRequest="100"
      Data="M13.908992,16.207977L32.000049,16.207977 32.000049,31.999985
13.908992,30.109983z">

```

```
<Path.RenderTransform>
  <TranslateTransform X="50"
                    Y="50" />
</Path.RenderTransform>
</Path>
```

In this example, the [Path](#) object is moved 50 device-independent units to the right, and 50 device-independent units down.

Multiple transforms

.NET MAUI has two classes that support applying multiple transforms to a [Path](#) object. These are [TransformGroup](#), and [CompositeTransform](#). A [TransformGroup](#) performs transforms in any desired order, while a [CompositeTransform](#) performs transforms in a specific order.

Transform groups

Transform groups represent composite transforms composed of multiple [Transform](#) objects.

The [TransformGroup](#) class, which derives from the [Transform](#) class, defines a [Children](#) property, of type [TransformCollection](#), which represents a collection of [Transform](#) objects. This property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled.

The order of transformations is important in a composite transform that uses the [TransformGroup](#) class. For example, if you first rotate, then scale, then translate, you get a different result than if you first translate, then rotate, then scale. One reason order is significant is that transforms like rotation and scaling are performed respect to the origin of the coordinate system. Scaling an object that is centered at the origin produces a different result to scaling an object that has been moved away from the origin. Similarly, rotating an object that is centered at the origin produces a different result than rotating an object that has been moved away from the origin.

The following example shows how to perform a composite transform using the [TransformGroup](#) class:

XAML

```
<Path Stroke="Black"
      Aspect="Uniform"
      HorizontalOptions="Center"
      HeightRequest="100"
```

```

WidthRequest="100"
Data="M13.908992,16.207977L32.000049,16.207977 32.000049,31.999985
13.908992,30.109983z">
  <Path.RenderTransform>
    <TransformGroup>
      <ScaleTransform ScaleX="1.5"
                      ScaleY="1.5" />
      <RotateTransform Angle="45" />
    </TransformGroup>
  </Path.RenderTransform>
</Path>

```

In this example, the `Path` object is scaled to 1.5 times its size, and then rotated by 45 degrees.

Composite transforms

A composite transform applies multiple transforms to an object.

The `CompositeTransform` class, which derives from the `Transform` class, defines the following properties:

- `CenterX`, of type `double`, which represents the x-coordinate of the center point of this transform. The default value of this property is 0.0.
- `CenterY`, of type `double`, which represents the y-coordinate of the center point of this transform. The default value of this property is 0.0.
- `ScaleX`, of type `double`, which represents the x-axis scale factor. The default value of this property is 1.0.
- `ScaleY`, of type `double`, which represents the y-axis scale factor. The default value of this property is 1.0.
- `SkewX`, of type `double`, which represents the x-axis skew angle, which is measured in degrees counterclockwise from the y-axis. The default value of this property is 0.0.
- `SkewY`, of type `double`, which represents the y-axis skew angle, which is measured in degrees counterclockwise from the x-axis. The default value of this property is 0.0.
- `Rotation`, of type `double`, represents the angle, in degrees, of clockwise rotation. The default value of this property is 0.0.
- `TranslateX`, of type `double`, which represents the distance to move along the x-axis. The default value of this property is 0.0.
- `TranslateY`, of type `double`, which represents the distance to move along the y-axis. The default value of this property is 0.0.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

A [CompositeTransform](#) applies transforms in this order:

1. Scale ([ScaleX](#) and [ScaleY](#)).
2. Skew ([SkewX](#) and [SkewY](#)).
3. Rotate ([Rotation](#)).
4. Translate ([TranslateX](#), [TranslateY](#)).

If you want to apply multiple transforms to an object in a different order, you should create a [TransformGroup](#) and insert the transforms in your intended order.

Important

A [CompositeTransform](#) uses the same center points, `CenterX` and `CenterY`, for all transformations. If you want to specify different center points per transform, use a [TransformGroup](#).

The following example shows how to perform a composite transform using the [CompositeTransform](#) class:

XAML

```
<Path Stroke="Black"
      Aspect="Uniform"
      HorizontalOptions="Center"
      HeightRequest="100"
      WidthRequest="100"
      Data="M13.908992,16.207977L32.000049,16.207977 32.000049,31.999985
13.908992,30.109983z">
  <Path.RenderTransform>
    <CompositeTransform ScaleX="1.5"
                       ScaleY="1.5"
                       Rotation="45"
                       TranslateX="50"
                       TranslateY="50" />
  </Path.RenderTransform>
</Path>
```

In this example, the [Path](#) object is scaled to 1.5 times its size, then rotated by 45 degrees, and then translated by 50 device-independent units.

Transform matrix

A transform can be described in terms of a 3x3 affine transformation matrix, that performs transformations in 2D space. This 3x3 matrix is represented by the [Matrix](#) struct, which is a collection of three rows and three columns of `double` values.

The [Matrix](#) struct defines the following properties:

- [Determinant](#), of type `double`, which gets the determinant of the matrix.
- [HasInverse](#), of type `bool`, which indicates whether the matrix is invertible.
- [Identity](#), of type [Matrix](#), which gets an identity matrix.
- [IsIdentity](#), of type `bool`, which indicates whether the matrix is an identity matrix.
- [M11](#), of type `double`, which represents the value of the first row and first column of the matrix.
- [M12](#), of type `double`, which represents the value of the first row and second column of the matrix.
- [M21](#), of type `double`, which represents the value of the second row and first column of the matrix.
- [M22](#), of type `double`, which represents the value of the second row and second column of the matrix.
- [OffsetX](#), of type `double`, which represents the value of the third row and first column of the matrix.
- [OffsetY](#), of type `double`, which represents the value of the third row and second column of the matrix.

The [OffsetX](#) and [OffsetY](#) properties are so named because they specify the amount to translate the coordinate space along the x-axis, and y-axis, respectively.

In addition, the [Matrix](#) struct exposes a series of methods that can be used to manipulate the matrix values, including [Append](#), [Invert](#), [Multiply](#), [Prepend](#) and many more.

The following table shows the structure of a .NET MAUI matrix:

M11

M12

0.0

M21

M22

0.0

OffsetX

OffsetY

1.0

ⓘ Note

An affine transformation matrix has its final column equal to (0,0,1), so only the members in the first two columns need to be specified.

By manipulating matrix values, you can rotate, scale, skew, and translate [Path](#) objects. For example, if you change the [OffsetX](#) value to 100, you can use it to move a [Path](#) object 100 device-independent units along the x-axis. If you change the [M22](#) value to 3, you can use it to stretch a [Path](#) object to three times its current height. If you change both values, you move the [Path](#) object 100 device-independent units along the x-axis and stretch its height by a factor of 3. In addition, affine transformation matrices can be multiplied to form any number of linear transformations, such as rotation and skew, followed by translation.

Custom transforms

The [MatrixTransform](#) class, which derives from the [Transform](#) class, defines a [Matrix](#) property, of type [Matrix](#), which represents the matrix that defines the transformation. This property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled.

Any transform that you can describe with a [TranslateTransform](#), [ScaleTransform](#), [RotateTransform](#), or [SkewTransform](#) object can equally be described by a [MatrixTransform](#). However, the [TranslateTransform](#), [ScaleTransform](#), [RotateTransform](#), and [SkewTransform](#) classes are easier to conceptualize than setting the vector components in a [Matrix](#). Therefore, the [MatrixTransform](#) class is typically used to create custom transformations that aren't provided by the [RotateTransform](#), [ScaleTransform](#), [SkewTransform](#), or [TranslateTransform](#) classes.

The following example shows how to transform a [Path](#) object using a [MatrixTransform](#):

XAML

```
<Path Stroke="Black"
      Aspect="Uniform"
      HorizontalOptions="Center"
```



```

        Data="M13.908992,16.207977L32.000049,16.207977 32.000049,31.999985
13.908992,30.109983z">
        <Path.RenderTransform>
            <MatrixTransform>
                <MatrixTransform.Matrix>
                    <!-- M11 stretches, M12 skews -->
                    <Matrix OffsetX="10"
                        OffsetY="100"
                        M11="1.5"
                        M12="1" />
                </MatrixTransform.Matrix>
            </MatrixTransform>
        </Path.RenderTransform>
    </Path>

```

In this example, the [Path](#) object is stretched, skewed, and offset in both the X and Y dimensions.

Alternatively, this can be written in a simplified form that uses a type converter that's built into .NET MAUI:

XAML

```

<Path Stroke="Black"
    Aspect="Uniform"
    HorizontalOptions="Center"
    Data="M13.908992,16.207977L32.000049,16.207977 32.000049,31.999985
13.908992,30.109983z">
    <Path.RenderTransform>
        <MatrixTransform Matrix="1.5,1,0,1,10,100" />
    </Path.RenderTransform>
</Path>

```

In this example, the [Matrix](#) property is specified as a comma-delimited string consisting of six members: `M11`, `M12`, `M21`, `M22`, `OffsetX`, `OffsetY`. While the members are comma-delimited in this example, they can also be delimited by one or more spaces.

In addition, the previous example can be simplified even further by specifying the same six members as the value of the [RenderTransform](#) property:

XAML

```

<Path Stroke="Black"
    Aspect="Uniform"
    HorizontalOptions="Center"
    RenderTransform="1.5 1 0 1 10 100"
    Data="M13.908992,16.207977L32.000049,16.207977 32.000049,31.999985
13.908992,30.109983z" />

```

Polygon

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [Polygon](#) class derives from the [Shape](#) class, and can be used to draw polygons, which are connected series of lines that form closed shapes. For information on the properties that the [Polygon](#) class inherits from the [Shape](#) class, see [Shapes](#).

[Polygon](#) defines the following properties:

- [Points](#), of type [PointCollection](#), which is a collection of `Point` structures that describe the vertex points of the polygon.
- [FillRule](#), of type [FillRule](#), which specifies how the interior fill of the shape is determined. The default value of this property is `FillRule.EvenOdd`.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The `PointsCollection` type is an `ObservableCollection` of `Point` objects. The `Point` structure defines `x` and `y` properties, of type `double`, that represent an x- and y-coordinate pair in 2D space. Therefore, the `Points` property should be set to a list of x-coordinate and y-coordinate pairs that describe the polygon vertex points, delimited by a single comma and/or one or more spaces. For example, "40,10 70,80" and "40 10, 70 80" are both valid.

For more information about the [FillRule](#) enumeration, see [Fill rules](#).

Create a Polygon

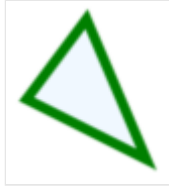
To draw a polygon, create a [Polygon](#) object and set its `Points` property to the vertices of a shape. A line is automatically drawn that connects the first and last points. To paint the inside of the polygon, set its `Fill` property to a [Brush](#)-derived object. To give the polygon an outline, set its `Stroke` property to a [Brush](#)-derived object. The `StrokeThickness` property specifies the thickness of the polygon outline. For more information about [Brush](#) objects, see [Brushes](#).

The following XAML example shows how to draw a filled polygon:

```
XAML
```

```
<Polygon Points="40,10 70,80 10,50"  
  Fill="AliceBlue"  
  Stroke="Green"  
  StrokeThickness="5" />
```

In this example, a filled polygon that represents a triangle is drawn:

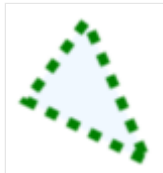


The following XAML example shows how to draw a dashed polygon:

XAML

```
<Polygon Points="40,10 70,80 10,50"  
  Fill="AliceBlue"  
  Stroke="Green"  
  StrokeThickness="5"  
  StrokeDashArray="1,1"  
  StrokeDashOffset="6" />
```

In this example, the polygon outline is dashed:



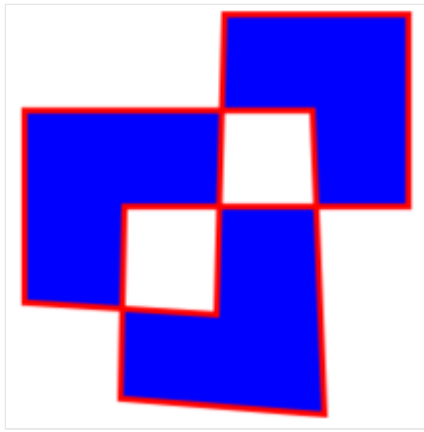
For more information about drawing a dashed polygon, see [Draw dashed shapes](#).

The following XAML example shows a polygon that uses the default fill rule:

XAML

```
<Polygon Points="0 48, 0 144, 96 150, 100 0, 192 0, 192 96, 50 96, 48 192,  
  150 200 144 48"  
  Fill="Blue"  
  Stroke="Red"  
  StrokeThickness="3" />
```

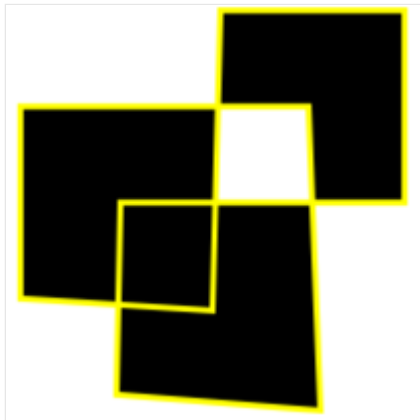
In this example, the fill behavior of each polygon is determined using the [EvenOdd](#) fill rule.



The following XAML example shows a polygon that uses the [Nonzero](#) fill rule:

XAML

```
<Polygon Points="0 48, 0 144, 96 150, 100 0, 192 0, 192 96, 50 96, 48 192,
150 200 144 48"
    Fill="Black"
    FillRule="Nonzero"
    Stroke="Yellow"
    StrokeThickness="3" />
```



In this example, the fill behavior of each polygon is determined using the [Nonzero](#) fill rule.

Polyline

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [Polyline](#) class derives from the [Shape](#) class, and can be used to draw a series of connected straight lines. A polyline is similar to a polygon, except the last point in a polyline is not connected to the first point. For information on the properties that the [Polyline](#) class inherits from the [Shape](#) class, see [Shapes](#).

[Polyline](#) defines the following properties:

- [Points](#), of type [PointCollection](#), which is a collection of `Point` structures that describe the vertex points of the polyline.
- [FillRule](#), of type [FillRule](#), which specifies how the intersecting areas in the polyline are combined. The default value of this property is `FillRule.EvenOdd`.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The `PointsCollection` type is an `ObservableCollection` of `Point` objects. The `Point` structure defines `x` and `y` properties, of type `double`, that represent an x- and y-coordinate pair in 2D space. Therefore, the `Points` property should be set to a list of x-coordinate and y-coordinate pairs that describe the polyline vertex points, delimited by a single comma and/or one or more spaces. For example, "40,10 70,80" and "40 10, 70 80" are both valid.

For more information about the [FillRule](#) enumeration, see [Fill rules](#).

Create a Polyline

To draw a polyline, create a [Polyline](#) object and set its `Points` property to the vertices of a shape. To give the polyline an outline, set its [Stroke](#) property to a [Brush](#)-derived object. The [StrokeThickness](#) property specifies the thickness of the polyline outline. For more information about [Brush](#) objects, see [Brushes](#).

 **Important**

If you set the [Fill](#) property of a [Polyline](#) to a [Brush](#)-derived object, the interior space of the polyline is painted, even if the start point and end point do not intersect.

The following XAML example shows how to draw a polyline:

XAML

```
<Polyline Points="0,0 10,30 15,0 18,60 23,30 35,30 40,0 43,60 48,30 100,30"
          Stroke="Red" />
```

In this example, a red polyline is drawn:



The following XAML example shows how to draw a dashed polyline:

XAML

```
<Polyline Points="0,0 10,30 15,0 18,60 23,30 35,30 40,0 43,60 48,30 100,30"
          Stroke="Red"
          StrokeThickness="2"
          StrokeDashArray="1,1"
          StrokeDashOffset="6" />
```

In this example, the polyline is dashed:



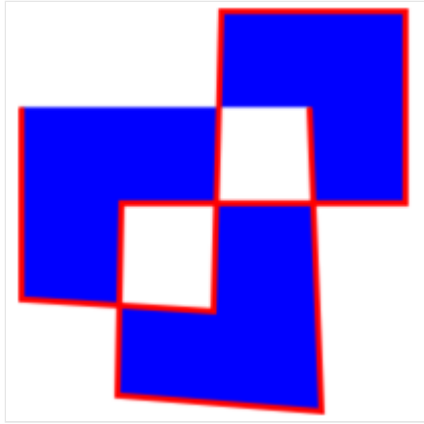
For more information about drawing a dashed polyline, see [Draw dashed shapes](#).

The following XAML example shows a polyline that uses the default fill rule:

XAML

```
<Polyline Points="0 48, 0 144, 96 150, 100 0, 192 0, 192 96, 50 96, 48 192,
150 200 144 48"
          Fill="Blue"
          Stroke="Red"
          StrokeThickness="3" />
```

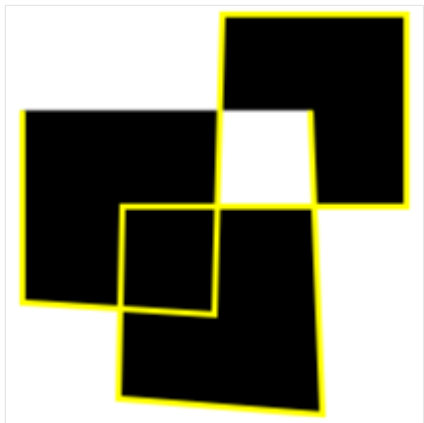
In this example, the fill behavior of the polyline is determined using the [EvenOdd](#) fill rule.



The following XAML example shows a polyline that uses the [Nonzero](#) fill rule:

XAML

```
<Polyline Points="0 48, 0 144, 96 150, 100 0, 192 0, 192 96, 50 96, 48 192,
150 200 144 48"
    Fill="Black"
    FillRule="Nonzero"
    Stroke="Yellow"
    StrokeThickness="3" />
```



In this example, the fill behavior of the polyline is determined using the [Nonzero](#) fill rule.

Rectangle

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [Rectangle](#) class derives from the [Shape](#) class, and can be used to draw rectangles and squares. For information on the properties that the [Rectangle](#) class inherits from the [Shape](#) class, see [.NET MAUI Shapes](#).

[Rectangle](#) defines the following properties:

- [RadiusX](#), of type `double`, which is the x-axis radius that's used to round the corners of the rectangle. The default value of this property is 0.0.
- [RadiusY](#), of type `double`, which is the y-axis radius that's used to round the corners of the rectangle. The default value of this property is 0.0.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [Rectangle](#) class sets the [Aspect](#) property, inherited from the [Shape](#) class, to `Stretch.Fill`. For more information about the [Aspect](#) property, see [Stretch shapes](#).

Create a Rectangle

To draw a rectangle, create a [Rectangle](#) object and sets its [WidthRequest](#) and [HeightRequest](#) properties. To paint the inside of the rectangle, set its [Fill](#) property to a [Brush](#)-derived object. To give the rectangle an outline, set its [Stroke](#) property to a [Brush](#)-derived object. The [StrokeThickness](#) property specifies the thickness of the rectangle outline. For more information about [Brush](#) objects, see [Brushes](#).

To give the rectangle rounded corners, set its [RadiusX](#) and [RadiusY](#) properties. These properties set the x-axis and y-axis radii that's used to round the corners of the rectangle.

Note

There's also a [RoundRectangle](#) class, that has a `CornerRadius` [BindableProperty](#), which can be used to draw rectangles with rounded corners.

To draw a square, make the [WidthRequest](#) and [HeightRequest](#) properties of the [Rectangle](#) object equal.

The following XAML example shows how to draw a filled rectangle:

XAML

```
<Rectangle Fill="Red"
  WidthRequest="150"
  HeightRequest="50"
  HorizontalOptions="Start" />
```

In this example, a red filled rectangle with dimensions 150x50 (device-independent units) is drawn:



The following XAML example shows how to draw a filled rectangle, with rounded corners:

XAML

```
<Rectangle Fill="Blue"
  Stroke="Black"
  StrokeThickness="3"
  RadiusX="50"
  RadiusY="10"
  WidthRequest="200"
  HeightRequest="100"
  HorizontalOptions="Start" />
```

In this example, a blue filled rectangle with rounded corners is drawn:



For information about drawing a dashed rectangle, see [Draw dashed shapes](#).

Slider

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [Slider](#) is a horizontal bar that you can manipulate to select a `double` value from a continuous range.

[Slider](#) defines the following properties:

- `Minimum`, of type `double`, is the minimum of the range, with a default value of 0.
- `Maximum`, of type `double`, is the maximum of the range, with a default value of 1.
- `Value`, of type `double`, is the slider's value, which can range between `Minimum` and `Maximum` and has a default value of 0.
- `MinimumTrackColor`, of type `Color`, is the bar color on the left side of the thumb.
- `MaximumTrackColor`, of type `Color`, is the bar color on the right side of the thumb.
- `ThumbColor` of type `Color`, is the thumb color.
- `ThumbImageSource`, of type `ImageSource`, is the image to use for the thumb, of type `ImageSource`.
- `DragStartedCommand`, of type `ICommand`, which is executed at the beginning of a drag action.
- `DragCompletedCommand`, of type `ICommand`, which is executed at the end of a drag action.

These properties are backed by [BindableProperty](#) objects. The `Value` property has a default binding mode of `BindingMode.TwoWay`, which means that it's suitable as a binding source in an application that uses the Model-View-ViewModel (MVVM) pattern.

ⓘ Note

The `ThumbColor` and `ThumbImageSource` properties are mutually exclusive. If both properties are set, the `ThumbImageSource` property will take precedence.

The [Slider](#) coerces the `Value` property so that it is between `Minimum` and `Maximum`, inclusive. If the `Minimum` property is set to a value greater than the `Value` property, the [Slider](#) sets the `Value` property to `Minimum`. Similarly, if `Maximum` is set to a value less than `Value`, then [Slider](#) sets the `Value` property to `Maximum`. Internally, the [Slider](#) ensures that `Minimum` is less than `Maximum`. If `Minimum` or `Maximum` are ever set so that `Minimum` is not less than `Maximum`, an exception is raised. For more information on setting the `Minimum` and `Maximum` properties, see [Precautions](#).

[Slider](#) defines a `ValueChanged` event that's raised when the `Value` changes, either through user manipulation of the [Slider](#) or when the program sets the `Value` property directly. A `ValueChanged` event is also raised when the `Value` property is coerced as described in the previous paragraph. The `ValueChangedEventArgs` object that accompanies the `ValueChanged` event has `OldValue` and `NewValue` properties, of type `double`. At the time the event is raised, the value of `NewValue` is the same as the `Value` property of the [Slider](#) object.

[Slider](#) also defines `DragStarted` and `DragCompleted` events, that are raised at the beginning and end of the drag action. Unlike the `ValueChanged` event, the `DragStarted` and `DragCompleted` events are only raised through user manipulation of the [Slider](#). When the `DragStarted` event fires, the `DragStartedCommand`, of type [ICommand](#), is executed. Similarly, when the `DragCompleted` event fires, the `DragCompletedCommand`, of type [ICommand](#), is executed.

Warning

Do not use unconstrained horizontal layout options of `Center`, `Start`, or `End` with [Slider](#). Keep the default `HorizontalOptions` setting of `Fill`, and don't use a width of `Auto` when putting [Slider](#) in a [Grid](#) layout.

Create a Slider

The following example shows how to create a [Slider](#), with two [Label](#) objects:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="SliderDemos.BasicSliderXamlPage"
    Title="Basic Slider XAML"
    Padding="10, 0">
    <StackLayout>
        <Label x:Name="rotatingLabel"
            Text="ROTATING TEXT"
            FontSize="18"
            HorizontalOptions="Center"
            VerticalOptions="Center" />
        <Slider Maximum="360"
            ValueChanged="OnSliderValueChanged" />
        <Label x:Name="displayLabel"
            Text="(uninitialized)"
            HorizontalOptions="Center"
            VerticalOptions="Center" />
    </StackLayout>
</ContentPage>
```

```
</StackLayout>
</ContentPage>
```

In this example, the [Slider](#) is initialized to have a `Maximum` property of 360. The second [Label](#) displays the text "(uninitialized)" until the [Slider](#) is manipulated, which causes the first `ValueChanged` event to be raised.

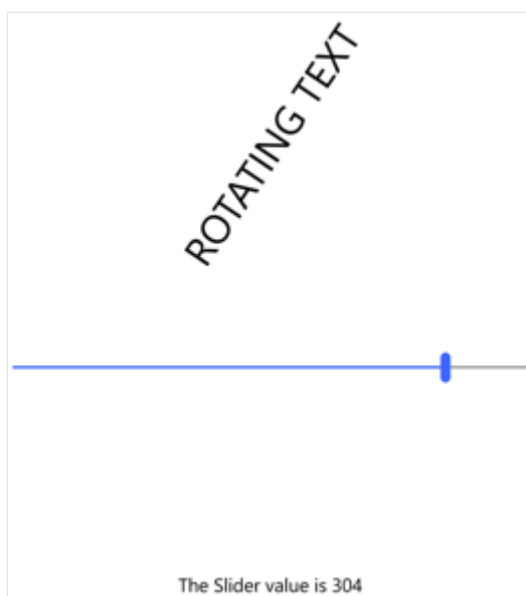
The code-behind file contains the handler for the `ValueChanged` event:

C#

```
public partial class BasicSliderXamlPage : ContentPage
{
    public BasicSliderXamlPage()
    {
        InitializeComponent();

        void OnSliderValueChanged(object sender, ValueChangedEventArgs args)
        {
            double value = args.NewValue;
            rotatingLabel.Rotation = value;
            displayLabel.Text = String.Format("The Slider value is {0}", value);
        }
    }
}
```

The `ValueChanged` handler of the [Slider](#) uses the `Value` property of the `slider` object to set the `Rotation` property of the first [Label](#) and uses the `String.Format` method with the `NewValue` property of the event arguments to set the `Text` property of the second [Label](#):



It's also possible for the event handler to obtain the [Slider](#) that is firing the event through the `sender` argument. The `Value` property contains the current value:

C#

```
double value = ((Slider)sender).Value;
```

If the [Slider](#) object were given a name in the XAML file with an `x:Name` attribute (for example, "slider"), then the event handler could reference that object directly:

C#

```
double value = slider.Value;
```

The equivalent C# code for creating a [Slider](#) is:

C#

```
Slider slider = new Slider
{
    Maximum = 360
};
slider.ValueChanged += (sender, args) =>
{
    rotationLabel.Rotation = slider.Value;
    displayLabel.Text = String.Format("The Slider value is {0}",
args.NewValue);
};
```

Data bind a Slider

The `ValueChanged` event handler can be eliminated by using data binding to respond to the [Slider](#) value changing:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="SliderDemos.BasicSliderBindingsPage"
    Title="Basic Slider Bindings"
    Padding="10, 0">
    <StackLayout>
        <Label Text="ROTATING TEXT"
            Rotation="{Binding Source={x:Reference slider},
                Path=Value}"
            FontSize="18"
            HorizontalOptions="Center"
            VerticalOptions="Center" />
        <Slider x:Name="slider"
            Maximum="360" />
    </StackLayout>
</ContentPage>
```

```

<Label x:Name="displayLabel"
      Text="{Binding Source={x:Reference slider},
                  Path=Value,
                  StringFormat='The Slider value is {0:F0}}'"
      HorizontalOptions="Center"
      VerticalOptions="Center" />
</StackLayout>
</ContentPage>

```

In this example, the `Rotation` property of the first `Label` is bound to the `Value` property of the `Slider`, as is the `Text` property of the second `Label` with a `StringFormat` specification. When the page first appears, the second `Label` displays the text string with the value. To display text without data binding, you'd need to specifically initialize the `Text` property of the `Label` or simulate a firing of the `ValueChanged` event by calling the event handler from the class constructor.

Precautions

The value of the `Minimum` property must always be less than the value of the `Maximum` property. The following example causes the `Slider` to raise an exception:

C#

```

// Throws an exception!
Slider slider = new Slider
{
    Minimum = 10,
    Maximum = 20
};

```

The C# compiler generates code that sets these two properties in sequence, and when the `Minimum` property is set to 10, it is greater than the default `Maximum` value of 1. You can avoid the exception in this case by setting the `Maximum` property first:

C#

```

Slider slider = new Slider
{
    Maximum = 20,
    Minimum = 10
};

```

In this example, setting `Maximum` to 20 is not a problem because it is greater than the default `Minimum` value of 0. When `Minimum` is set, the value is less than the `Maximum` value

of 20.

The same problem exists in XAML. The properties must be set in an order that ensures that `Maximum` is always greater than `Minimum`:

XAML

```
<Slider Maximum="20"  
        Minimum="10" ... />
```

You can set the `Minimum` and `Maximum` values to negative numbers, but only in an order where `Minimum` is always less than `Maximum`:

XAML

```
<Slider Minimum="-20"  
        Maximum="-10" ... />
```

The `Value` property is always greater than or equal to the `Minimum` value and less than or equal to `Maximum`. If `Value` is set to a value outside that range, the value will be coerced to lie within the range, but no exception is raised. For example, the following example won't raise an exception:

C#

```
Slider slider = new Slider  
{  
    Value = 10  
};
```

Instead, the `Value` property is coerced to the `Maximum` value of 1.

A previous example set `Maximum` to 20, and `Minimum` to 10:

C#

```
Slider slider = new Slider  
{  
    Maximum = 20,  
    Minimum = 10  
};
```

When `Minimum` is set to 10, then `Value` is also set to 10.

If a `ValueChanged` event handler has been attached at the time that the `Value` property is coerced to something other than its default value of 0, then a `ValueChanged` event is raised:

XAML

```
<Slider ValueChanged="OnSliderValueChanged"  
        Maximum="20"  
        Minimum="10" />
```

When `Minimum` is set to 10, `Value` is also set to 10, and the `ValueChanged` event is raised. This might occur before the rest of the page has been constructed, and the handler might attempt to reference other elements on the page that have not yet been created. You might want to add some code to the `ValueChanged` handler that checks for `null` values of other elements on the page. Or, you can set the `ValueChanged` event handler after the `Slider` values have been initialized.

Stepper

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [Stepper](#) enables a numeric value to be selected from a range of values. It consists of two buttons labeled with minus and plus signs. These buttons can be manipulated by the user to incrementally select a `double` value from a range of values.

The [Stepper](#) defines four properties of type `double`:

- `Increment` is the amount to change the selected value by, with a default value of 1.
- `Minimum` is the minimum of the range, with a default value of 0.
- `Maximum` is the maximum of the range, with a default value of 100.
- `Value` is the stepper's value, which can range between `Minimum` and `Maximum` and has a default value of 0.

All of these properties are backed by [BindableProperty](#) objects. The `Value` property has a default binding mode of `BindingMode.TwoWay`, which means that it's suitable as a binding source in an application that uses the Model-View-ViewModel (MVVM) pattern.

The [Stepper](#) coerces the `Value` property so that it is between `Minimum` and `Maximum`, inclusive. If the `Minimum` property is set to a value greater than the `Value` property, the [Stepper](#) sets the `Value` property to `Minimum`. Similarly, if `Maximum` is set to a value less than `Value`, then [Stepper](#) sets the `Value` property to `Maximum`. Internally, the [Stepper](#) ensures that `Minimum` is less than `Maximum`. If `Minimum` or `Maximum` are ever set so that `Minimum` is not less than `Maximum`, an exception is raised. For more information on setting the `Minimum` and `Maximum` properties, see [Precautions](#).

[Stepper](#) defines a `ValueChanged` event that's raised when the `Value` changes, either through user manipulation of the [Stepper](#) or when the application sets the `Value` property directly. A `ValueChanged` event is also raised when the `Value` property is coerced as previously described. The `ValueChangedEventArgs` object that accompanies the `ValueChanged` event has `OldValue` and `NewValue`, of type `double`. At the time the event is raised, the value of `NewValue` is the same as the `Value` property of the [Stepper](#) object.

Create a Stepper

The following example shows how to create a [Stepper](#), with two [Label](#) objects:

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="StepperDemo.BasicStepperXAMLPage"
              Title="Basic Stepper XAML">
    <StackLayout Margin="20">
        <Label x:Name="_rotatingLabel"
              Text="ROTATING TEXT"
              FontSize="18"
              HorizontalOptions="Center"
              VerticalOptions="Center" />
        <Stepper Maximum="360"
              Increment="30"
              HorizontalOptions="Center"
              ValueChanged="OnStepperValueChanged" />
        <Label x:Name="_displayLabel"
              Text="(uninitialized)"
              HorizontalOptions="Center"
              VerticalOptions="Center" />
    </StackLayout>
</ContentPage>

```

In this example, the `Stepper` is initialized to have a `Maximum` property of 360, and an `Increment` property of 30. Manipulating the `Stepper` changes the selected value incrementally between `Minimum` to `Maximum` based on the value of the `Increment` property. The second `Label` displays the text "(uninitialized)" until the `Stepper` is manipulated, which causes the first `ValueChanged` event to be raised.

The code-behind file contains the handler for the `ValueChanged` event:

C#

```

public partial class BasicStepperXAMLPage : ContentPage
{
    public BasicStepperXAMLPage()
    {
        InitializeComponent();
    }

    void OnStepperValueChanged(object sender, ValueChangedEventArgs e)
    {
        double value = e.NewValue;
        _rotatingLabel.Rotation = value;
        _displayLabel.Text = string.Format("The Stepper value is {0}",
value);
    }
}

```

The `ValueChanged` handler of the [Stepper](#) uses the `Value` property of the `stepper` object to set the `Rotation` property of the first [Label](#) and uses the `string.Format` method with the `newValue` property of the event arguments to set the `Text` property of the second [Label](#):



It's also possible for the event handler to obtain the [Stepper](#) that is firing the event through the `sender` argument. The `Value` property contains the current value:

C#

```
double value = ((Stepper)sender).Value;
```

If the [Stepper](#) object were given a name in the XAML file with an `x:Name` attribute (for example, "stepper"), then the event handler could reference that object directly:

C#

```
double value = stepper.Value;
```

The equivalent C# code for creating a [Stepper](#) is:

C#

```
Stepper stepper = new Stepper  
{  
    Maximum = 360,  
    Increment = 30,
```

```

        HorizontalOptions = LayoutOptions.Center
    };
    stepper.ValueChanged += (sender, e) =>
    {
        rotationLabel.Rotation = stepper.Value;
        displayLabel.Text = string.Format("The Stepper value is {0}",
e.NewValue);
    };

```

Data bind a Stepper

The `ValueChanged` event handler can be eliminated by using data binding to respond to the `Stepper` value changing:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="StepperDemo.BasicStepperBindingsPage"
    Title="Basic Stepper Bindings">
    <StackLayout Margin="20">
        <Label Text="ROTATING TEXT"
            Rotation="{Binding Source={x:Reference _stepper},
Path=Value}"
            FontSize="18"
            HorizontalOptions="Center"
            VerticalOptions="Center" />
        <Stepper x:Name="_stepper"
            Maximum="360"
            Increment="30"
            HorizontalOptions="Center" />
        <Label Text="{Binding Source={x:Reference _stepper}, Path=Value,
StringFormat='The Stepper value is {0:F0}'}"
            HorizontalOptions="Center"
            VerticalOptions="Center" />
    </StackLayout>
</ContentPage>

```

In this example, the `Rotation` property of the first `Label` is bound to the `Value` property of the `Stepper`, as is the `Text` property of the second `Label` with a `StringFormat` specification. When the page first appears, the second `Label` displays the text string with the value. To display text without data binding, you'd need to specifically initialize the `Text` property of the `Label` or simulate a firing of the `ValueChanged` event by calling the event handler from the class constructor.

Precautions

The value of the `Minimum` property must always be less than the value of the `Maximum` property. The following code example causes the `Stepper` to raise an exception:

C#

```
// Throws an exception!
Stepper stepper = new Stepper
{
    Minimum = 180,
    Maximum = 360
};
```

The C# compiler generates code that sets these two properties in sequence, and when the `Minimum` property is set to 180, it is greater than the default `Maximum` value of 100. You can avoid the exception in this case by setting the `Maximum` property first:

C#

```
Stepper stepper = new Stepper
{
    Maximum = 360,
    Minimum = 180
};
```

In this example, setting `Maximum` to 360 is not a problem because it is greater than the default `Minimum` value of 0. When `Minimum` is set, the value is less than the `Maximum` value of 360.

The same problem exists in XAML. Set the properties in an order that ensures that `Maximum` is always greater than `Minimum`:

XAML

```
<Stepper Maximum="360"
        Minimum="180" ... />
```

You can set the `Minimum` and `Maximum` values to negative numbers, but only in an order where `Minimum` is always less than `Maximum`:

XAML

```
<Stepper Minimum="-360"
        Maximum="-180" ... />
```

The `Value` property is always greater than or equal to the `Minimum` value and less than or equal to `Maximum`. If `Value` is set to a value outside that range, the value will be coerced to lie within the range, but no exception is raised. For example, this code won't raise an exception:

```
C#  
  
Stepper stepper = new Stepper  
{  
    Value = 180  
};
```

Instead, the `Value` property is coerced to the `Maximum` value of 100.

A previous example set `Maximum` to 360 and `Minimum` to 180:

```
C#  
  
Stepper stepper = new Stepper  
{  
    Maximum = 360,  
    Minimum = 180  
};
```

When `Minimum` is set to 180, then `Value` is also set to 180.

If a `ValueChanged` event handler has been attached at the time that the `Value` property is coerced to something other than its default value of 0, then a `ValueChanged` event is raised:

```
XAML  
  
<Stepper ValueChanged="OnStepperValueChanged"  
    Maximum="360"  
    Minimum="180" />
```

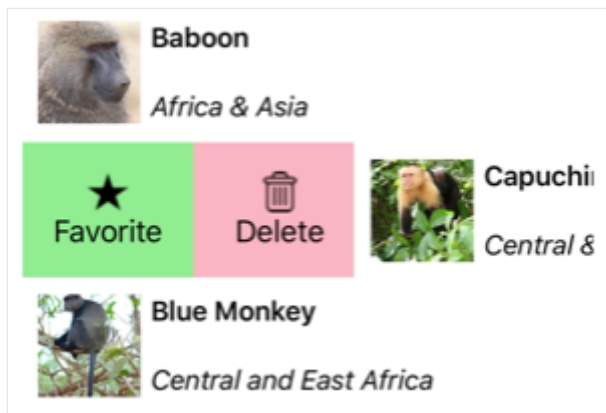
When `Minimum` is set to 180, `Value` is also set to 180, and the `ValueChanged` event is raised. This might occur before the rest of the page has been constructed, and the handler might attempt to reference other elements on the page that have not yet been created. You might want to add some code to the `ValueChanged` handler that checks for `null` values of other elements on the page. Or, you can set the `ValueChanged` event handler after the `Stepper` values have been initialized.

SwipeView

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [SwipeView](#) is a container control that wraps around an item of content, and provides context menu items that are revealed by a swipe gesture:



Important

[SwipeView](#) is designed for touch interfaces. On Windows it can only be swiped in a touch interface and will not function with a pointer device such as a mouse.

[SwipeView](#) defines the following properties:

- `LeftItems`, of type `SwipeItems`, which represents the swipe items that can be invoked when the control is swiped from the left side.
- `RightItems`, of type `SwipeItems`, which represents the swipe items that can be invoked when the control is swiped from the right side.
- `TopItems`, of type `SwipeItems`, which represents the swipe items that can be invoked when the control is swiped from the top down.
- `BottomItems`, of type `SwipeItems`, which represents the swipe items that can be invoked when the control is swiped from the bottom up.
- `Threshold`, of type `double`, which represents the number of device-independent units that trigger a swipe gesture to fully reveal swipe items.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

In addition, the [SwipeView](#) inherits the `Content` property from the [ContentView](#) class. The `Content` property is the content property of the [SwipeView](#) class, and therefore does not need to be explicitly set.

The [SwipeView](#) class also defines three events:

- `SwipeStarted` is raised when a swipe starts. The `SwipeStartedEventArgs` object that accompanies this event has a `SwipeDirection` property, of type `SwipeDirection`.
- `SwipeChanging` is raised as the swipe moves. The `SwipeChangingEventArgs` object that accompanies this event has a `SwipeDirection` property, of type `SwipeDirection`, and an `Offset` property of type `double`.
- `SwipeEnded` is raised when a swipe ends. The `SwipeEndedEventArgs` object that accompanies this event has a `SwipeDirection` property, of type `SwipeDirection`, and an `IsOpen` property of type `bool`.

In addition, [SwipeView](#) includes `Open` and `Close` methods, which programmatically open and close the swipe items, respectively.

ⓘ Note

[SwipeView](#) has a platform-specific on iOS and Android, that controls the transition that's used when opening a [SwipeView](#). For more information, see [SwipeView swipe transition Mode on iOS](#) and [SwipeView swipe transition mode on Android](#).

Create a SwipeView

A [SwipeView](#) must define the content that the [SwipeView](#) wraps around, and the swipe items that are revealed by the swipe gesture. The swipe items are one or more `SwipeItem` objects that are placed in one of the four [SwipeView](#) directional collections - `LeftItems`, `RightItems`, `TopItems`, Or `BottomItems`.

The following example shows how to instantiate a [SwipeView](#) in XAML:

XAML

```
<SwipeView>
  <SwipeView.LeftItems>
    <SwipeItems>
      <SwipeItem Text="Favorite"
                  IconImageSource="favorite.png"
                  BackgroundColor="LightGreen"
                  Invoked="OnFavoriteSwipeItemInvoked" />
      <SwipeItem Text="Delete"
```



```

                IconImageSource="delete.png"
                BackgroundColor="LightPink"
                Invoked="OnDeleteSwipeItemInvoked" />
            </SwipeItems>
        </SwipeView.LeftItems>
        <!-- Content -->
        <Grid HeightRequest="60"
            WidthRequest="300"
            BackgroundColor="LightGray">
            <Label Text="Swipe right"
                HorizontalOptions="Center"
                VerticalOptions="Center" />
        </Grid>
    </SwipeView>

```

The equivalent C# code is:

```

C#

// SwipeItems
SwipeItem favoriteSwipeItem = new SwipeItem
{
    Text = "Favorite",
    IconImageSource = "favorite.png",
    BackgroundColor = Colors.LightGreen
};
favoriteSwipeItem.Invoked += OnFavoriteSwipeItemInvoked;

SwipeItem deleteSwipeItem = new SwipeItem
{
    Text = "Delete",
    IconImageSource = "delete.png",
    BackgroundColor = Colors.LightPink
};
deleteSwipeItem.Invoked += OnDeleteSwipeItemInvoked;

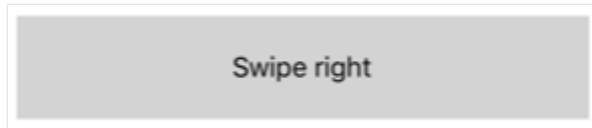
List<SwipeItem> swipeItems = new List<SwipeItem>() { favoriteSwipeItem,
deleteSwipeItem };

// SwipeView content
Grid grid = new Grid
{
    HeightRequest = 60,
    WidthRequest = 300,
    BackgroundColor = Colors.LightGray
};
grid.Add(new Label
{
    Text = "Swipe right",
    HorizontalOptions = LayoutOptions.Center,
    VerticalOptions = LayoutOptions.Center
});

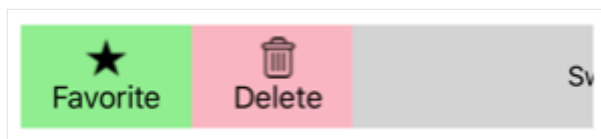
```

```
SwipeView swipeView = new SwipeView
{
    LeftItems = new SwipeItems(swipeItems),
    Content = grid
};
```

In this example, the [SwipeView](#) content is a [Grid](#) that contains a [Label](#):



The swipe items are used to perform actions on the [SwipeView](#) content, and are revealed when the control is swiped from the left side:



By default, a swipe item is executed when it is tapped by the user. However, this behavior can be changed. For more information, see [Swipe mode](#).

Once a swipe item has been executed the swipe items are hidden and the [SwipeView](#) content is re-displayed. However, this behavior can be changed. For more information, see [Swipe behavior](#).

ⓘ Note

Swipe content and swipe items can be placed inline, or defined as resources.

Swipe items

The `LeftItems`, `RightItems`, `TopItems`, and `BottomItems` collections are all of type `SwipeItems`. The `SwipeItems` class defines the following properties:

- `Mode`, of type `SwipeMode`, which indicates the effect of a swipe interaction. For more information about swipe mode, see [Swipe mode](#).
- `SwipeBehaviorOnInvoked`, of type `SwipeBehaviorOnInvoked`, which indicates how a [SwipeView](#) behaves after a swipe item is invoked. For more information about swipe behavior, see [Swipe behavior](#).

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

Each swipe item is defined as a `SwipeItem` object that's placed into one of the four `SwipeItems` directional collections. The `SwipeItem` class derives from the `MenuItem` class, and adds the following members:

- A `BackgroundColor` property, of type `Color`, that defines the background color of the swipe item. This property is backed by a bindable property.
- An `Invoked` event, which is raised when the swipe item is executed.

Important

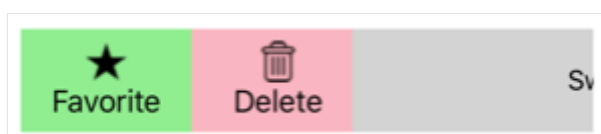
The `MenuItem` class defines several properties, including `Command`, `CommandParameter`, `IconImageSource`, and `Text`. These properties can be set on a `SwipeItem` object to define its appearance, and to define an `ICommand` that executes when the swipe item is invoked. For more information, see [Display menu items](#).

The following example shows two `SwipeItem` objects in the `LeftItems` collection of a `SwipeView`:

XAML

```
<SwipeView>
  <SwipeView.LeftItems>
    <SwipeItems>
      <SwipeItem Text="Favorite"
                  IconImageSource="favorite.png"
                  BackgroundColor="LightGreen"
                  Invoked="OnFavoriteSwipeItemInvoked" />
      <SwipeItem Text="Delete"
                  IconImageSource="delete.png"
                  BackgroundColor="LightPink"
                  Invoked="OnDeleteSwipeItemInvoked" />
    </SwipeItems>
  </SwipeView.LeftItems>
  <!-- Content -->
</SwipeView>
```

The appearance of each `SwipeItem` is defined by a combination of the `Text`, `IconImageSource`, and `BackgroundColor` properties:



When a `SwipeItem` is tapped, its `Invoked` event fires and is handled by its registered event handler. In addition, the `MenuItem.Clicked` event fires. Alternatively, the `Command` property can be set to an `ICommand` implementation that will be executed when the `SwipeItem` is invoked.

⚠ Note

When the appearance of a `SwipeItem` is defined only using the `Text` or `IconImageSource` properties, the content is always centered.

In addition to defining swipe items as `SwipeItem` objects, it's also possible to define custom swipe item views. For more information, see [Custom swipe items](#).

Swipe direction

`SwipeView` supports four different swipe directions, with the swipe direction being defined by the directional `SwipeItems` collection the `SwipeItem` objects are added to. Each swipe direction can hold its own swipe items. For example, the following example shows a `SwipeView` whose swipe items depend on the swipe direction:

XAML

```
<SwipeView>
  <SwipeView.LeftItems>
    <SwipeItems>
      <SwipeItem Text="Delete"
                  IconImageSource="delete.png"
                  BackgroundColor="LightPink"
                  Command="{Binding DeleteCommand}" />
    </SwipeItems>
  </SwipeView.LeftItems>
  <SwipeView.RightItems>
    <SwipeItems>
      <SwipeItem Text="Favorite"
                  IconImageSource="favorite.png"
                  BackgroundColor="LightGreen"
                  Command="{Binding FavoriteCommand}" />
      <SwipeItem Text="Share"
                  IconImageSource="share.png"
                  BackgroundColor="LightYellow"
                  Command="{Binding ShareCommand}" />
    </SwipeItems>
  </SwipeView.RightItems>
  <!-- Content -->
</SwipeView>
```

In this example, the [SwipeView](#) content can be swiped right or left. Swiping to the right will show the **Delete** swipe item, while swiping to the left will show the **Favorite** and **Share** swipe items.

⚠ Warning

Only one instance of a directional `SwipeItems` collection can be set at a time on a [SwipeView](#). Therefore, you cannot have two `LeftItems` definitions on a [SwipeView](#).

The `SwipeStarted`, `SwipeChanging`, and `SwipeEnded` events report the swipe direction via the `SwipeDirection` property in the event arguments. This property is of type `SwipeDirection`, which is an enumeration consisting of four members:

- `Right` indicates that a right swipe occurred.
- `Left` indicates that a left swipe occurred.
- `Up` indicates that an upwards swipe occurred.
- `Down` indicates that a downwards swipe occurred.

Swipe threshold

[SwipeView](#) includes a `Threshold` property, of type `double`, which represents the number of device-independent units that trigger a swipe gesture to fully reveal swipe items.

The following example shows a [SwipeView](#) that sets the `Threshold` property:

XAML

```
<SwipeView Threshold="200">
    <SwipeView.LeftItems>
        <SwipeItems>
            <SwipeItem Text="Favorite"
                        IconImageSource="favorite.png"
                        BackgroundColor="LightGreen" />
        </SwipeItems>
    </SwipeView.LeftItems>
    <!-- Content -->
</SwipeView>
```

In this example, the [SwipeView](#) must be swiped for 200 device-independent units before the `SwipeItem` is fully revealed.

Swipe mode

The `SwipeItems` class has a `Mode` property, which indicates the effect of a swipe interaction. This property should be set to one of the `SwipeMode` enumeration members:

- `Reveal` indicates that a swipe reveals the swipe items. This is the default value of the `SwipeItems.Mode` property.
- `Execute` indicates that a swipe executes the swipe items.

In reveal mode, the user swipes a `SwipeView` to open a menu consisting of one or more swipe items, and must explicitly tap a swipe item to execute it. After the swipe item has been executed the swipe items are closed and the `SwipeView` content is re-displayed. In execute mode, the user swipes a `SwipeView` to open a menu consisting of one more swipe items, which are then automatically executed. Following execution, the swipe items are closed and the `SwipeView` content is re-displayed.

The following example shows a `SwipeView` configured to use execute mode:

XAML

```
<SwipeView>
  <SwipeView.LeftItems>
    <SwipeItems Mode="Execute">
      <SwipeItem Text="Delete"
        IconImageSource="delete.png"
        BackgroundColor="LightPink"
        Command="{Binding DeleteCommand}" />
    </SwipeItems>
  </SwipeView.LeftItems>
  <!-- Content -->
</SwipeView>
```

In this example, the `SwipeView` content can be swiped right to reveal the swipe item, which is executed immediately. Following execution, the `SwipeView` content is re-displayed.

Swipe behavior

The `SwipeItems` class has a `SwipeBehaviorOnInvoked` property, which indicates how a `SwipeView` behaves after a swipe item is invoked. This property should be set to one of the `SwipeBehaviorOnInvoked` enumeration members:

- `Auto` indicates that in reveal mode the `SwipeView` closes after a swipe item is invoked, and in execute mode the `SwipeView` remains open after a swipe item is invoked. This is the default value of the `SwipeItems.SwipeBehaviorOnInvoked` property.

- `Close` indicates that the [SwipeView](#) closes after a swipe item is invoked.
- `RemainOpen` indicates that the [SwipeView](#) remains open after a swipe item is invoked.

The following example shows a [SwipeView](#) configured to remain open after a swipe item is invoked:

XAML

```
<SwipeView>
  <SwipeView.LeftItems>
    <SwipeItems SwipeBehaviorOnInvoked="RemainOpen">
      <SwipeItem Text="Favorite"
        IconImageSource="favorite.png"
        BackgroundColor="LightGreen"
        Invoked="OnFavoriteSwipeItemInvoked" />
      <SwipeItem Text="Delete"
        IconImageSource="delete.png"
        BackgroundColor="LightPink"
        Invoked="OnDeleteSwipeItemInvoked" />
    </SwipeItems>
  </SwipeView.LeftItems>
  <!-- Content -->
</SwipeView>
```

Custom swipe items

Custom swipe items can be defined with the `SwipeItemView` type. The `SwipeItemView` class derives from the [ContentView](#) class, and adds the following properties:

- `Command`, of type [ICommand](#), which is executed when a swipe item is tapped.
- `CommandParameter`, of type `object`, which is the parameter that's passed to the `Command`.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The `SwipeItemView` class also defines an `Invoked` event that's raised when the item is tapped, after the `Command` is executed.

The following example shows a `SwipeItemView` object in the `LeftItems` collection of a [SwipeView](#):

XAML

```

<SwipeView>
  <SwipeView.LeftItems>
    <SwipeItems>
      <SwipeItemView Command="{Binding CheckAnswerCommand}"
                    CommandParameter="{Binding Source={x:Reference
resultEntry}, Path=Text}">
        <StackLayout Margin="10"
                    WidthRequest="300">
          <Entry x:Name="resultEntry"
                Placeholder="Enter answer"
                HorizontalOptions="CenterAndExpand" />
          <Label Text="Check"
                FontAttributes="Bold"
                HorizontalOptions="Center" />
        </StackLayout>
      </SwipeItemView>
    </SwipeItems>
  </SwipeView.LeftItems>
  <!-- Content -->
</SwipeView>

```

In this example, the `SwipeItemView` comprises a `StackLayout` containing an `Entry` and a `Label`. After the user enters input into the `Entry`, the rest of the `SwipeViewItem` can be tapped which executes the `ICommand` defined by the `SwipeItemView.Command` property.

Open and close a SwipeView programmatically

`SwipeView` includes `Open` and `Close` methods, which programmatically open and close the swipe items, respectively. By default, these methods will animate the `SwipeView` when its opened or closed.

The `Open` method requires an `OpenSwipeItem` argument, to specify the direction the `SwipeView` will be opened from. The `OpenSwipeItem` enumeration has four members:

- `LeftItems`, which indicates that the `SwipeView` will be opened from the left, to reveal the swipe items in the `LeftItems` collection.
- `TopItems`, which indicates that the `SwipeView` will be opened from the top, to reveal the swipe items in the `TopItems` collection.
- `RightItems`, which indicates that the `SwipeView` will be opened from the right, to reveal the swipe items in the `RightItems` collection.
- `BottomItems`, which indicates that the `SwipeView` will be opened from the bottom, to reveal the swipe items in the `BottomItems` collection.

In addition, the `Open` method also accepts an optional `bool` argument that defines whether the [SwipeView](#) will be animated when it opens.

Given a [SwipeView](#) named `swipeView`, the following example shows how to open a [SwipeView](#) to reveal the swipe items in the `LeftItems` collection:

```
C#  
  
swipeView.Open(OpenSwipeItem.LeftItems);
```

The `swipeView` can then be closed with the `Close` method:

```
C#  
  
swipeView.Close();
```

ⓘ Note

The `Close` method also accepts an optional `bool` argument that defines whether the [SwipeView](#) will be animated when it closes.

Disable a SwipeView

An app may enter a state where swiping an item of content is not a valid operation. In such cases, the [SwipeView](#) can be disabled by setting its `IsEnabled` property to `false`. This will prevent users from being able to swipe content to reveal swipe items.

In addition, when defining the `Command` property of a `SwipeItem` or `SwipeItemView`, the `CanExecute` delegate of the [ICommand](#) can be specified to enable or disable the swipe item.

Switch

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [Switch](#) control is a horizontal toggle button that can be manipulated by the user to toggle between on and off states, which are represented by a `boolean` value.

The following screenshot shows a [Switch](#) control in its on and off toggle states:



The [Switch](#) control defines the following properties:

- `IsToggled` is a `boolean` value that indicates whether the [Switch](#) is on. The default value of this property is `false`.
- `OnColor` is a [Color](#) that affects how the [Switch](#) is rendered in the toggled, or on state.
- `ThumbColor` is the [Color](#) of the switch thumb.

These properties are backed by [BindableProperty](#) objects, which means they can be styled and be the target of data bindings.

The [Switch](#) control defines a `Toggled` event that's raised when the `IsToggled` property changes, either through user manipulation or when an application sets the `IsToggled` property. The `ToggledEventArgs` object that accompanies the `Toggled` event has a single property named `Value`, of type `bool`. When the event is raised, the value of the `Value` property reflects the new value of the `IsToggled` property.

Create a Switch

A [Switch](#) can be instantiated in XAML. Its `IsToggled` property can be set to toggle the [Switch](#). By default, the `IsToggled` property is `false`. The following example shows how to instantiate a [Switch](#) in XAML with the optional `IsToggled` property set:

XAML

```
<Switch IsToggled="true"/>
```

A [Switch](#) can also be created in code:

C#

```
Switch switchControl = new Switch { IsToggled = true };
```

Switch appearance

In addition to the properties that [Switch](#) inherits from the [View](#) class, [Switch](#) also defines `OnColor` and `ThumbColor` properties. The `OnColor` property can be set to define the [Switch](#) color when it is toggled to its on state, and the `ThumbColor` property can be set to define the [Color](#) of the switch thumb. The following example shows how to instantiate a [Switch](#) in XAML with these properties set:

XAML

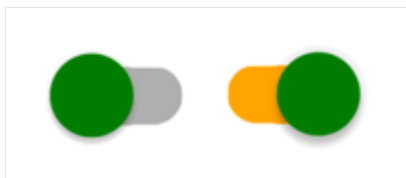
```
<Switch OnColor="Orange"
        ThumbColor="Green" />
```

The properties can also be set when creating a [Switch](#) in code:

C#

```
Switch switch = new Switch { OnColor = Colors.Orange, ThumbColor =  
Colors.Green };
```

The following screenshot shows the [Switch](#) in its on and off toggle states, with the `OnColor` and `ThumbColor` properties set:



Respond to a Switch state change

When the `IsToggled` property changes, either through user manipulation or when an application sets the `IsToggled` property, the `Toggled` event fires. An event handler for this event can be registered to respond to the change:

XAML

```
<Switch Toggled="OnToggled" />
```

The code-behind file contains the handler for the `Toggled` event:

C#

```
void OnToggled(object sender, ToggledEventArgs e)
{
    // Perform an action after examining e.Value
}
```

The `sender` argument in the event handler is the `Switch` responsible for firing this event. You can use the `sender` property to access the `Switch` object, or to distinguish between multiple `Switch` objects sharing the same `Toggled` event handler.

The `Toggled` event handler can also be assigned in code:

C#

```
Switch switchControl = new Switch {...};
switchControl.Toggled += (sender, e) =>
{
    // Perform an action after examining e.Value
};
```

Data bind a Switch

The `Toggled` event handler can be eliminated by using data binding and triggers to respond to a `Switch` changing toggle states.

XAML

```
<Switch x:Name="styleSwitch" />
<Label Text="Lorem ipsum dolor sit amet, elit rutrum, enim hendrerit augue
vitae praesent sed non, lorem aenean quis praesent pede.">
    <Label.Triggers>
        <DataTrigger TargetType="Label"
            Binding="{Binding Source={x:Reference styleSwitch},
Path=IsToggled}"
            Value="true">
            <Setter Property="FontAttributes"
                Value="Italic, Bold" />
            <Setter Property="FontSize"
                Value="18" />
        </DataTrigger>
```

```
</Label.Triggers>
</Label>
```

In this example, the [Label](#) uses a binding expression in a [DataTrigger](#) to monitor the `IsToggled` property of the [Switch](#) named `styleSwitch`. When this property becomes `true`, the `FontAttributes` and `FontSize` properties of the [Label](#) are changed. When the `IsToggled` property returns to `false`, the `FontAttributes` and `FontSize` properties of the [Label](#) are reset to their initial state.

For information about triggers, see [Triggers](#).

Switch visual states

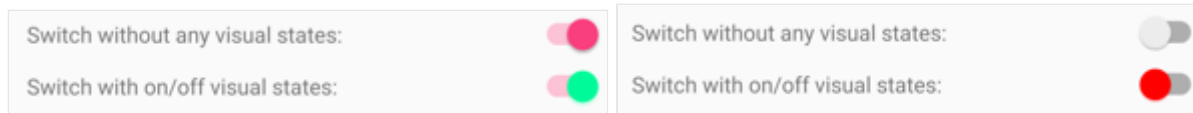
[Switch](#) has `On` and `Off` visual states that can be used to initiate a visual change when the `IsToggled` property changes.

The following XAML example shows how to define visual states for the `On` and `Off` states:

XAML

```
<Switch IsToggled="True">
  <VisualStateManager.VisualStateGroups>
    <VisualStateGroup x:Name="CommonStates">
      <VisualState x:Name="On">
        <VisualState.Setters>
          <Setter Property="ThumbColor"
            Value="MediumSpringGreen" />
        </VisualState.Setters>
      </VisualState>
      <VisualState x:Name="Off">
        <VisualState.Setters>
          <Setter Property="ThumbColor"
            Value="Red" />
        </VisualState.Setters>
      </VisualState>
    </VisualStateGroup>
  </VisualStateManager.VisualStateGroups>
</Switch>
```

In this example, the `On` [VisualState](#) specifies that when the `IsToggled` property is `true`, the `ThumbColor` property will be set to medium spring green. The `Off` [VisualState](#) specifies that when the `IsToggled` property is `false`, the `ThumbColor` property will be set to red. Therefore, the overall effect is that when the [Switch](#) is in an off position its thumb is red, and its thumb is medium spring green when the [Switch](#) is in an on position:



For more information about visual states, see [Visual states](#).

Disable a Switch

An app may enter a state where the [Switch](#) being toggled is not a valid operation. In such cases, the [Switch](#) can be disabled by setting its `IsEnabled` property to `false`. This will prevent users from being able to manipulate the [Switch](#).

TableView

Article • 08/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [TableView](#) displays a table of scrollable items that can be grouped into sections. A [TableView](#) is typically used for displaying items where each row has a different appearance, such as presenting a table of settings.

While [TableView](#) manages the appearance of the table, the appearance of each item in the table is defined by a [Cell](#). .NET MAUI includes five cell types that are used to display different combinations of data, and you can also define custom cells that display any content you want.

[TableView](#) defines the following properties:

- `Intent`, of type `TableIntent`, defines the purpose of the table on iOS.
- `HasUnevenRows`, of type `bool`, indicates whether items in the table can have rows of different heights. The default value of this property is `false`.
- `Root`, of type `TableRoot`, defines the child of the [TableView](#).
- `RowHeight`, of type `int`, determines the height of each row when `HasUnevenRows` is `false`.

The `HasUnevenRows` and `RowHeight` properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The value of the `Intent` property helps to define the [TableView](#) appearance on iOS only. This property should be set to a value of the `TableIntent` enumeration, which defines the following members:

- `Menu`, for presenting a selectable menu.
- `Settings`, for presenting a table of configuration settings.
- `Form`, for presenting a data input form.
- `Data`, for presenting data.

Note

[TableView](#) isn't designed to support binding to a collection of items.

Create a TableView

To create a table, create a [TableView](#) object and set its `Intent` property to a `TableIntent` member. The child of a [TableView](#) must be a `TableRoot` object, which is parent to one or more `TableSection` objects. Each `TableSection` consists of an optional title whose color can also be set, and one or more [Cell](#) objects.

The following example shows how to create a [TableView](#):

XAML

```
<TableView Intent="Menu">
  <TableRoot>
    <TableSection Title="Chapters">
      <TextCell Text="1. Introduction to .NET MAUI"
        Detail="Learn about .NET MAUI and what it provides."
      />
      <TextCell Text="2. Anatomy of an app"
        Detail="Learn about the visual elements in .NET MAUI"
      />
      <TextCell Text="3. Text"
        Detail="Learn about the .NET MAUI controls that
display text." />
      <TextCell Text="4. Dealing with sizes"
        Detail="Learn how to size .NET MAUI controls on
screen." />
      <TextCell Text="5. XAML vs code"
        Detail="Learn more about creating your UI in XAML." />
    </TableSection>
  </TableRoot>
</TableView>
```

In this example, the [TableView](#) defines a menu using [TextCell](#) objects:

Chapters
1. Introduction to .NET MAUI Learn about .NET MAUI and what it provides.
2. Anatomy of an app Learn about the visual elements in .NET MAUI
3. Text Learn about the .NET MAUI controls that display text.
4. Dealing with sizes Learn how to size .NET MAUI controls on screen.
5. XAML vs code Learn more about creating your UI in XAML.

📌 Note

Each [TextCell](#) can execute a command when tapped, provided that the [Command](#) property is set to a valid [ICommand](#) implementation.

Define cell appearance

Each item in a [TableView](#) is defined by a [Cell](#) object, and the [Cell](#) type used defines the appearance of the cell's data. .NET MAUI includes the following built-in cells:

- [TextCell](#), which displays primary and secondary text on separate lines.
- [ImageCell](#), which displays an image with primary and secondary text on separate lines.
- [SwitchCell](#), which displays text and a switch that can be switched on or off.
- [EntryCell](#), which displays a label and text that's editable.
- [ViewCell](#), which is a custom cell whose appearance is defined by a [View](#). This cell type should be used when you want to fully define the appearance of each item in a [TableView](#).

Text cell

A [TextCell](#) displays primary and secondary text on separate lines. [TextCell](#) defines the following properties:

- [Text](#), of type [string](#), defines the primary text to be displayed.
- [TextColor](#), of type [Color](#), represents the color of the primary text.
- [Detail](#), of type [string](#), defines the secondary text to be displayed.
- [DetailColor](#), of type [Color](#), indicates the color of the secondary text.
- [Command](#), of type [ICommand](#), defines the command that's executed when the cell is tapped.
- [CommandParameter](#), of type [object](#), represents the parameter that's passed to the command.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The following example shows using a [TextCell](#) to define the appearance of items in a [TableView](#):

XAML

```
<TableView Intent="Menu">
  <TableRoot>
    <TableSection Title="Chapters">
```

```

        <TextCell Text="1. Introduction to .NET MAUI"
                Detail="Learn about .NET MAUI and what it provides."
        />

        <TextCell Text="2. Anatomy of an app"
                Detail="Learn about the visual elements in .NET MAUI"
        />

        <TextCell Text="3. Text"
                Detail="Learn about the .NET MAUI controls that
display text." />
        <TextCell Text="4. Dealing with sizes"
                Detail="Learn how to size .NET MAUI controls on
screen." />
        <TextCell Text="5. XAML vs code"
                Detail="Learn more about creating your UI in XAML." />
    </TableSection>
</TableRoot>
</TableView>

```

The following screenshot shows the resulting cell appearance:

Chapters
1. Introduction to .NET MAUI Learn about .NET MAUI and what it provides.
2. Anatomy of an app Learn about the visual elements in .NET MAUI
3. Text Learn about the .NET MAUI controls that display text.
4. Dealing with sizes Learn how to size .NET MAUI controls on screen.
5. XAML vs code Learn more about creating your UI in XAML.

Image cell

An [ImageCell](#) displays an image with primary and secondary text on separate lines. [ImageCell](#) inherits the properties from [TextCell](#), and defines the [ImageSource](#) property, of type [ImageSource](#), which specifies the image to be displayed in the cell. This property is backed by a [BindableProperty](#) object, which means it can be the target of data bindings, and be styled.

The following example shows using an [ImageCell](#) to define the appearance of items in a [TableView](#):

XAML

```

<TableView Intent="Menu">
    <TableRoot>
        <TableSection Title="Learn how to use your Xbox">

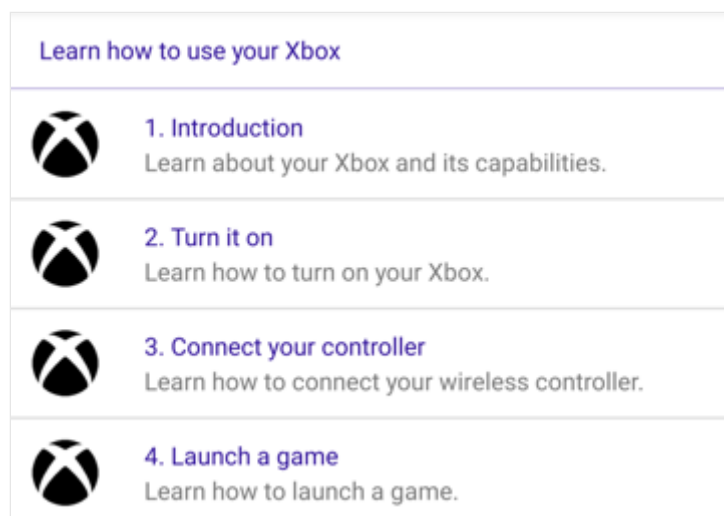
```

```

<ImageCell Text="1. Introduction"
           Detail="Learn about your Xbox and its capabilities."
           ImageSource="xbox.png" />
<ImageCell Text="2. Turn it on"
           Detail="Learn how to turn on your Xbox."
           ImageSource="xbox.png" />
<ImageCell Text="3. Connect your controller"
           Detail="Learn how to connect your wireless
controller."
           ImageSource="xbox.png" />
<ImageCell Text="4. Launch a game"
           Detail="Learn how to launch a game."
           ImageSource="xbox.png" />
</TableSection>
</TableRoot>
</TableView>

```

The following screenshot shows the resulting cell appearance:



Switch cell

A [SwitchCell](#) displays text and a switch that can be switched on or off. [SwitchCell](#) defines the following properties:

- `Text`, of type `string`, defines the text to display next to the switch.
- `On`, of type `bool`, represents whether the switch is on or off.
- `OnColor`, of type `Color`, indicates the color of the switch when in it's on position.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

[SwitchCell](#) also defines an `OnChanged` event that's raised when the switch changes state.

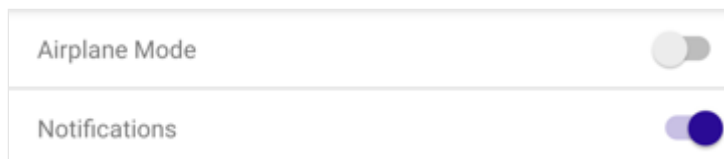
The `ToggledEventArgs` object that accompanies this event defines a `Value` property, that indicates whether the switch is on or off.

The following example shows using a [SwitchCell](#) to define the appearance of items in a [TableView](#):

XAML

```
<TableView Intent="Settings">
  <TableRoot>
    <TableSection>
      <SwitchCell Text="Airplane Mode"
                  On="False" />
      <SwitchCell Text="Notifications"
                  On="True" />
    </TableSection>
  </TableRoot>
</TableView>
```

The following screenshot shows the resulting cell appearance:



Entry cell

An [EntryCell](#) displays a label and text data that's editable. [EntryCell](#) defines the following properties:

- `HorizontalTextAlignment`, of type [TextAlignment](#), represents the horizontal alignment of the text.
- `Keyboard`, of type `Keyboard`, determines the keyboard to display when entering text.
- `Label`, of type `string`, represents the text to display to the left of the editable text.
- `LabelColor`, of type [Color](#), defines the color of the label text.
- `Placeholder`, of type `string`, represents the text that's displayed when the `Text` property is empty.
- `Text`, of type `string`, defines the text that's editable.
- `VerticalTextAlignment`, of type [TextAlignment](#), represents the vertical alignment of the text.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

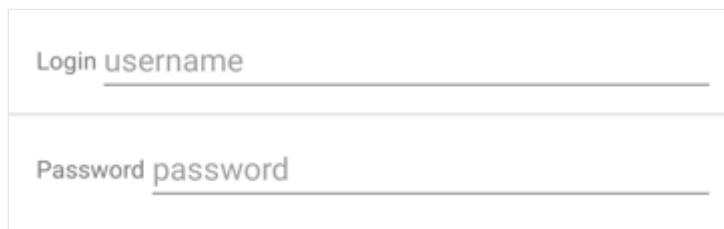
[EntryCell](#) also defines a `Completed` event that's raised when the user hits the return key, to indicate that editing is complete.

The following example shows using an [EntryCell](#) to define the appearance of items in a [TableView](#):

XAML

```
<TableView Intent="Settings">
  <TableRoot>
    <TableSection>
      <EntryCell Label="Login"
                  Placeholder="username" />
      <EntryCell Label="Password"
                  Placeholder="password" />
    </TableSection>
  </TableRoot>
</TableView>
```

The following screenshot shows the resulting cell appearance:



Login	username
Password	password

View cell

A [ViewCell](#) is a custom cell whose appearance is defined by a [View](#). [ViewCell](#) defines a [View](#) property, of type [View](#), which defines the view that represents the content of the cell. This property is backed by a [BindableProperty](#) object, which means it can be the target of data bindings, and be styled.

ⓘ Note

The [View](#) property is the content property of the [ViewCell](#) class, and therefore does not need to be explicitly set from XAML.

The following example shows using a [ViewCell](#) to define the appearance of an item in a [TableView](#):

XAML

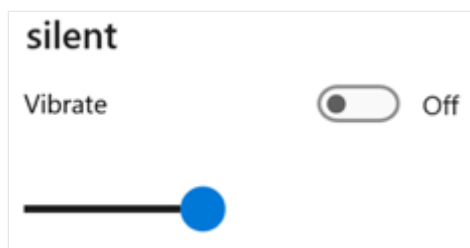
```
<TableView Intent="Settings">
  <TableRoot>
    <TableSection Title="Silent">
      <ViewCell>
```

```

<Grid RowDefinitions="Auto,Auto"
      ColumnDefinitions="0.5*,0.5*">
  <Label Text="Vibrate"
        Margin="10,10,0,0"/>
  <Switch Grid.Column="1"
          HorizontalOptions="End" />
  <Slider Grid.Row="1"
          Grid.ColumnSpan="2"
          Margin="10"
          Minimum="0"
          Maximum="10"
          Value="3" />
</Grid>
</TableViewCell>
</TableSection>
</TableRoot>
</TableView>

```

Inside the [TableViewCell](#), layout can be managed by any .NET MAUI layout. The following screenshot shows the resulting cell appearance:



Size items

By default, all cells of the same type in a [TableView](#) have the same height. However, this behavior can be changed with the `HasUnevenRows` and `RowHeight` properties. By default, the `HasUnevenRows` property is `false`.

The `RowHeight` property can be set to an `int` that represents the height of each item in the [TableView](#), provided that `HasUnevenRows` is `false`. When `HasUnevenRows` is set to `true`, each item in the [TableView](#) can have a different height. The height of each item will be derived from the contents of each cell, and so each item will be sized to its content.

Individual cells can be programmatically resized at runtime by changing layout related properties of elements within the cell, provided that the `HasUnevenRows` property is `true`. The following example changes the height of the cell when it's tapped:

C#

```
void OnViewCellTapped(object sender, EventArgs e)
{
    label.IsVisible = !label.IsVisible;
    viewCell.ForceUpdateSize();
}
```

In this example, the `OnViewCellTapped` event handler is executed in response to the cell being tapped. The event handler updates the visibility of the `Label` object and the `Cell.ForceUpdateSize` method updates the cell's size. If the `Label` has been made visible the cell's height will increase. If the `Label` has been made invisible the cell's height will decrease.

Warning

Overuse of dynamic item sizing can cause `TableView` performance to degrade.

Right-to-left layout

`TableView` can layout its content in a right-to-left flow direction by setting its `FlowDirection` property to `RightToLeft`. However, the `FlowDirection` property should ideally be set on a page or root layout, which causes all the elements within the page, or root layout, to respond to the flow direction:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="TableViewDemos.RightToLeftTablePage"
             Title="Right to left TableView"
             FlowDirection="RightToLeft">
    <TableView Intent="Settings">
        ...
    </TableView>
</ContentPage>
```

The default `FlowDirection` for an element with a parent is `MatchParent`. Therefore, the `TableView` inherits the `FlowDirection` property value from the `ContentPage`.

TimePicker

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [TimePicker](#) invokes the platform's time-picker control and allows you to select a time.

[TimePicker](#) defines the following properties:

- `Time` of type `TimeSpan`, the selected time, which defaults to a `TimeSpan` of 0. The `TimeSpan` type indicates a duration of time since midnight.
- `Format` of type `string`, a [standard](#) or [custom](#) .NET formatting string, which defaults to "t", the short time pattern.
- `TextColor` of type [Color](#), the color used to display the selected time.
- `FontAttributes` of type `FontAttributes`, which defaults to `FontAttributes.None`.
- `FontFamily` of type `string`, which defaults to `null`.
- `FontSize` of type `double`, which defaults to -1.0.
- `CharacterSpacing`, of type `double`, is the spacing between characters of the [TimePicker](#) text.

All of these properties are backed by [BindableProperty](#) objects, which means that they can be styled, and the properties can be targets of data bindings. The `Time` property has a default binding mode of `BindingMode.TwoWay`, which means that it can be a target of a data binding in an application that uses the Model-View-ViewModel (MVVM) pattern.

In addition, [TimePicker](#) defines a [TimeSelected](#) event, which is raised when the selected time changes. The [TimeChangedEventArgs](#) object that accompanies the `TimeSelected` event has `NewTime` and `OldTime` properties, which specify the new and old time, respectively.

Create a TimePicker

When the `Time` property is specified in XAML, the value is converted to a `TimeSpan` and validated to ensure that the number of milliseconds is greater than or equal to 0, and that the number of hours is less than 24. The time components should be separated by colons:

XAML

```
<TimePicker Time="4:15:26" />
```


If the `BindingContext` property of `TimePicker` is set to an instance of a viewmodel containing a property of type `TimeSpan` named `SelectedTime` (for example), you can instantiate the `TimePicker` like this:

XAML

```
<TimePicker Time="{Binding SelectedTime}" />
```

In this example, the `Time` property is initialized to the `SelectedTime` property in the viewmodel. Because the `Time` property has a binding mode of `TwoWay`, any new time that the user selects is automatically propagated to the viewmodel.

In code, you can initialize the `Time` property to a value of type `TimeSpan`:

C#

```
TimePicker timePicker = new TimePicker
{
    Time = new TimeSpan(4, 15, 26) // Time set to "04:15:26"
};
```

For information about setting font properties, see [Fonts](#).

TimePicker and layout

It's possible to use an unconstrained horizontal layout option such as `Center`, `Start`, or `End` with `TimePicker`:

XAML

```
<TimePicker ...
    HorizontalOptions="Center" />
```

However, this is not recommended. Depending on the setting of the `Format` property, selected times might require different display widths. For example, the "T" format string causes the `TimePicker` view to display times in a long format, and "4:15:26 AM" requires a greater display width than the short time format ("t") of "4:15 AM". Depending on the platform, this difference might cause the `TimePicker` view to change width in layout, or for the display to be truncated.



It's best to use the default `HorizontalOptions` setting of `Fill` with [TimePicker](#), and not to use a width of `Auto` when putting [TimePicker](#) in a [Grid](#) cell.

Platform differences

This section describes the platform-specific differences with the [TimePicker](#) control.

Windows

On Windows, the `Format` property only affects whether the hour is formatted for 12-hours or 24-hours. Other settings from the `Format` property are ignored. When the picker is shown by pressing on the control, only the hour, minute, and time of day can be changed. For more information about the Windows control, see [Time picker - Windows apps](#).

TitleBar

Article • 10/15/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [TitleBar](#) is a view that enables you to add a custom title bar to a [Window](#) to match the personality of your app. The following diagram shows the components of the [TitleBar](#):



Important

[TitleBar](#) is only available on Windows. Mac Catalyst support will be added in a future release.

[TitleBar](#) defines the following properties:

- [Content](#), of type [IView](#), which specifies the control for the content that's centered in the title bar, and is allocated the space between the leading and trailing content.
- [DefaultTemplate](#), of type [ControlTemplate](#), which represents the default template for the title bar.
- [ForegroundColor](#), of type [Color](#), which specifies the foreground colour of the title bar, and is used as the color for the title and subtitle text.
- [Icon](#), of type [ImageSource](#), which represents an optional 16x16px icon image for the title bar.
- [LeadingContent](#), of type [IView](#), which specifies the control for the content that follows the icon.
- [PassthroughElements](#), of type `IList<IView>`, which represents a list of elements that should prevent dragging in the title bar region and instead directly handle input.
- [Subtitle](#), of type `string`, which specifies the subtitle text of the title bar. This is usually secondary information about the app or window.
- [Title](#), of type `string`, which specifies the title text of the title bar. This is usually the name of the app or indicates the purpose of the window.
- [TrailingContent](#), of type [IView](#), which specifies the control that follows the `Content` control.

These properties, with the exception of [DefaultTemplate](#) and [PassthroughElements](#), are backed by [BindableProperty](#) objects, which means that they can be styled, and be the target of data bindings.

Important

Views set as the value of the [Content](#), [LeadingContent](#), and [TrailingContent](#) properties will block all input to the title bar region and will directly handle input.

The standard title bar height is 32px, but can be set to a larger value. For information about designing your title bar on Windows, see [Title bar](#).

Create a TitleBar

To add a title bar to a window, set the [Window.TitleBar](#) property to a [TitleBar](#) object.

The following XAML example shows how to add a [TitleBar](#) to a [Window](#):

XAML

```
<Window xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:local="clr-namespace:TitleBarDemo"
        x:Class="TitleBarDemo.MainWindow">
    ...
    <Window.TitleBar>
        <TitleBar Title="{Binding Title}"
                  Subtitle="{Binding Subtitle}"
                  IsVisible="{Binding ShowTitleBar}"
                  BackgroundColor="#512BD4"
                  ForegroundColor="White"
                  HeightRequest="48">
            <TitleBar.Content>
                <SearchBar Placeholder="Search"
                           PlaceholderColor="White"
                           MaximumWidthRequest="300"
                           HorizontalOptions="Fill"
                           VerticalOptions="Center" />
            </TitleBar.Content>
        </TitleBar>
    </Window.TitleBar>
</Window>
```

A [TitleBar](#) can also be defined in C# and added to a [Window](#):

C#

```

Window window = new Window
{
    TitleBar = new TitleBar
    {
        Icon = "titlebar_icon.png"
        Title = "My App",
        Subtitle = "Demo"
        Content = new SearchBar { ... }
    }
};

```

A [TitleBar](#) is highly customizable through its [Content](#), [LeadingContent](#), and [TrailingContent](#) properties:

XAML

```

<TitleBar Title="My App"
    BackgroundColor="#512BD4"
    HeightRequest="48">
    <TitleBar.Content>
        <SearchBar Placeholder="Search"
            MaximumWidthRequest="300"
            HorizontalOptions="FillAndExpand"
            VerticalOptions="Center" />
    </TitleBar.Content>
    <TitleBar.TrailingContent>
        <ImageButton HeightRequest="36"
            WidthRequest="36"
            BorderWidth="0"
            Background="Transparent">
            <ImageButton.Source>
                <FontImageSource Size="16"
                    Glyph="⚙️;"
                    FontFamily="SegoeMDL2"/>
            </ImageButton.Source>
        </ImageButton>
    </TitleBar.TrailingContent>
</TitleBar>

```

The following screenshot shows the resulting appearance:



ⓘ Note

The title bar can be hidden by setting the [IsVisible](#) property, which causes the window content to be displayed in the title bar region.

TitleBar visual states

[TitleBar](#) defines the following visual states that can be used to initiate a visual change to the [TitleBar](#):

- `IconVisible`
- `IconCollapsed`
- `TitleVisible`
- `TitleCollapsed`
- `SubtitleVisible`
- `SubtitleCollapsed`
- `LeadingContentVisible`
- `LeadingContentCollapsed`
- `ContentVisible`
- `ContentCollapsed`
- `TrailingContentVisible`
- `TrailingContentCollapsed`
- `TitleBarTitleActive`
- `TitleBarTitleInactive`

The following XAML example shows how to define a visual state for the `TitleBarTitleActive` and `TitleBarTitleInactive` states:

XAML

```
<TitleBar ...>
    <VisualStateManager.VisualStateGroups>
        <VisualStateGroupList>
            <VisualStateGroup x:Name="TitleActiveStates">
                <VisualState x:Name="TitleBarTitleActive">
                    <VisualState.Setters>
                        <Setter Property="BackgroundColor"
Value="Transparent" />
                        <Setter Property="ForegroundColor" Value="Black" />
                    </VisualState.Setters>
                </VisualState>
                <VisualState x:Name="TitleBarTitleInactive">
                    <VisualState.Setters>
                        <Setter Property="BackgroundColor" Value="White" />
                        <Setter Property="ForegroundColor" Value="Gray" />
                    </VisualState.Setters>
                </VisualState>
            </VisualStateGroup>
        </VisualStateGroupList>
    </VisualStateManager.VisualStateGroups>
</TitleBar>
```

```
<VisualStateManager.VisualStateGroups>  
</TitleBar>
```

In this example, the visual state sets the `BackgroundColor` and `ForegroundColor` properties to specific colors based on whether the title bar is active or inactive.

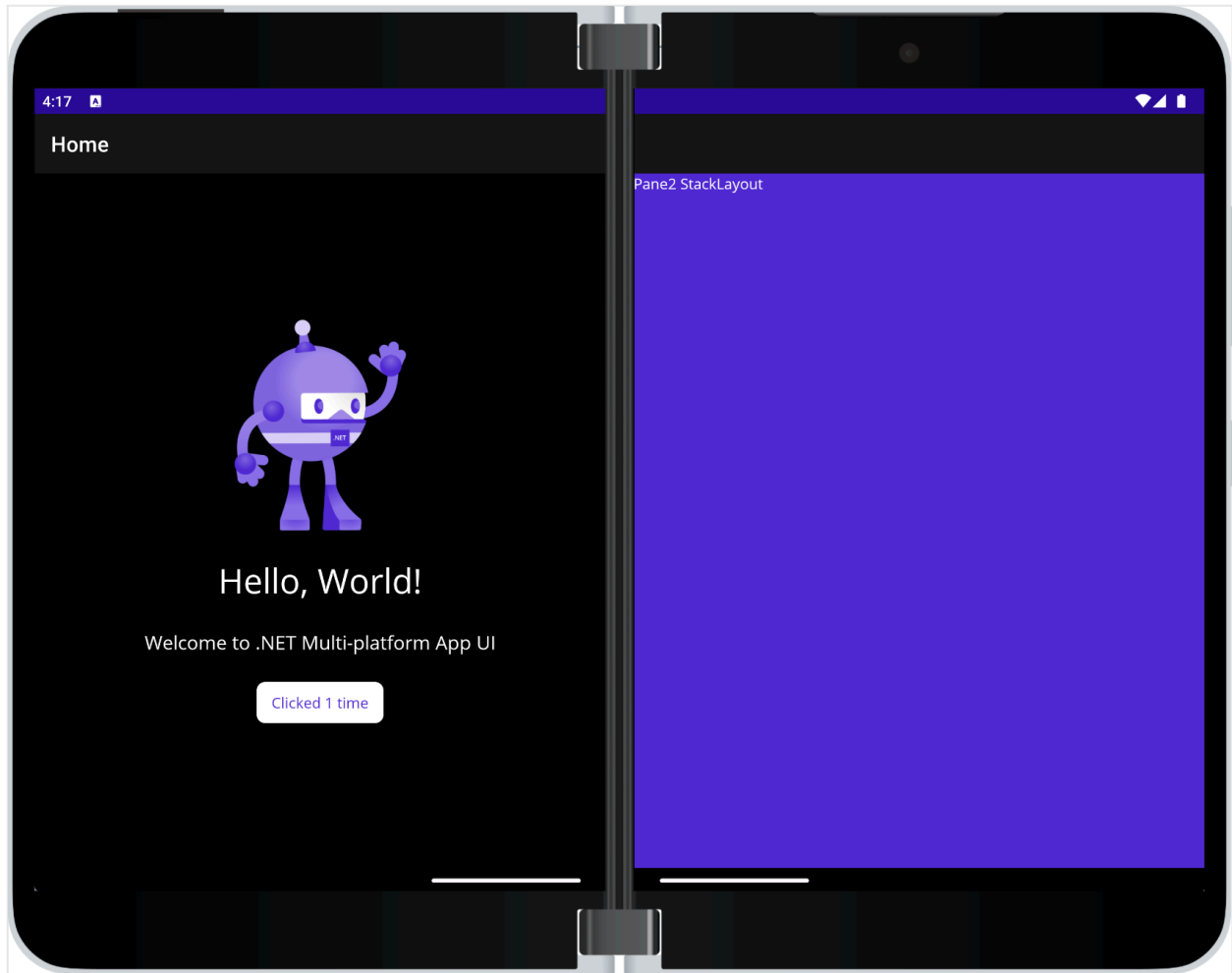
For more information about visual states, see [Visual states](#).

.NET MAUI TwoPaneView layout

Article • 08/30/2024

 [Browse the sample](#)

The [TwoPaneView](#) class represents a container with two views that size and position content in the available space, either side-by-side or top-to-bottom. [TwoPaneView](#) inherits from [Grid](#) so the easiest way to think about these properties is as if they are being applied to a grid.



The layout control is provided by the [Microsoft.Maui.Controls.Foldable](#) NuGet package [↗](#).

Foldable device support overview

Foldable devices include the Microsoft Surface Duo and Android devices from other manufacturers. They bridge the gap between phones and larger screens like tablets and desktops because apps might need to adjust to a variety of screen sizes and orientations on the same device, including adapting to a hinge or fold in the screen.

Visit the [dual-screen developer docs](#) for more information about building apps that target foldable devices, including [design patterns and user experiences](#). There is also a [Surface Duo emulator](#) you can download for Windows, Mac, and Linux.

Important

The [TwoPaneView](#) control only adapts to Android foldable devices that support the Jetpack Window Manager API provided by Google (such as Microsoft Surface Duo).

On all other platforms and devices (i.e. other Android devices, iOS, macOS, Windows) it acts like a configurable and responsive split view that can dynamically show one or two panes, proportionally sized on the screen.

Add and configure the Foldable support NuGet

1. Open the **NuGet Package Manager** dialog for your solution.
2. Under the **Browse** tab, search for `Microsoft.Maui.Controls.Foldable`.
3. Install the `Microsoft.Maui.Controls.Foldable` package to your solution.
4. Add the `UseFoldable()` initialization method (and namespace) call to the project's `MauiApp` class, in the `CreateMauiApp` method:

```
C#  
  
using Microsoft.Maui.Foldable; // ADD THIS NAMESPACE  
...  
public static MauiApp CreateMauiApp()  
{  
    var builder = MauiApp.CreateBuilder();  
    ...  
    builder.UseFoldable(); // ADD THIS LINE TO THE TEMPLATE  
    return builder.Build();  
}
```

The `UseFoldable()` initialization is required for the app to be able to detect changes in the app's state, such as being spanned across a fold.

5. Update the `[Activity(...)]` attribute on the `MainActivity` class in *Platforms/Android*, so that it includes *all* these `ConfigurationChanges` options:

```
C#
```

```
ConfigurationChanges = ConfigChanges.Orientation |  
ConfigChanges.ScreenSize  
    | ConfigChanges.ScreenLayout | ConfigChanges.SmallestScreenSize |  
ConfigChanges.UiMode
```

These values are required so that configuration changes and span state can be more reliably reported for reliable dual-screen support.

Set up TwoPaneView

To add the [TwoPaneView](#) layout to your page:

1. Add a `foldable` namespace alias for the Foldable NuGet:

XAML

```
xmlns:foldable="clr-  
namespace:Microsoft.Maui.Controls.Foldable;assembly=Microsoft.Maui.Cont  
rols.Foldable"
```

2. Add the [TwoPaneView](#) as the root element on the page, and add controls to `Pane1` and `Pane2`:

XAML

```
<foldable:TwoPaneView x:Name="twoPaneView">  
  <foldable:TwoPaneView.Pane1  
    BackgroundColor="#dddddd">  
    <Label  
      Text="Hello, .NET MAUI!"  
      SemanticProperties.HeadingLevel="Level1"  
      FontSize="32"  
      HorizontalOptions="Center" />  
    </foldable:TwoPaneView.Pane1>  
  <foldable:TwoPaneView.Pane2>  
    <StackLayout BackgroundColor="{AppThemeBinding Light=  
{StaticResource Secondary}, Dark={StaticResource Primary}}">  
      <Label Text="Pane2 StackLayout"/>  
    </StackLayout>  
  </foldable:TwoPaneView.Pane2>  
</foldable:TwoPaneView>
```

Understand TwoPaneView modes

Only one of these modes can be active:

- `SinglePane` only one pane is currently visible.
- `Wide` the two panes are laid out horizontally. One pane is on the left and the other is on the right. When on two screens this is the mode when the device is portrait.
- `Tall` the two panes are laid out vertically. One pane is on top and the other is on bottom. When on two screens this is the mode when the device is landscape.

Control `TwoPaneView` when it's only on one screen

The following properties apply when the `TwoPaneView` is occupying a single screen:

- `MinTallModeHeight` indicates the minimum height the control must be to enter `Tall` mode.
- `MinWideModeWidth` indicates the minimum width the control must be to enter `Wide` mode.
- `Pane1Length` sets the width of `Pane1` in `Wide` mode, the height of `Pane1` in `Tall` mode, and has no effect in `SinglePane` mode.
- `Pane2Length` sets the width of `Pane2` in `Wide` mode, the height of `Pane2` in `Tall` mode, and has no effect in `SinglePane` mode.

Important

If the `TwoPaneView` is spanned across a hinge or fold these properties have no effect.

Properties that apply when on one screen or two

The following properties apply when the `TwoPaneView` is occupying a single screen or two screens:

- `TallModeConfiguration` indicates, when in `Tall` mode, the Top/Bottom arrangement or if you only want a single pane visible as defined by the `TwoPaneViewPriority`.
- `WideModeConfiguration` indicates, when in `Wide` mode, the Left/Right arrangement or if you only want a single pane visible as defined by the `TwoPaneViewPriority`.
- `PanePriority` determines whether to show `Pane1` or `Pane2` if in `SinglePane` mode.

Troubleshooting

If the [TwoPaneView](#) layout isn't working as expected, double-check the set-up instructions on this page. Omitting or misconfiguring the `UseFoldable()` method or the `ConfigurationChanges` attribute values are common causes of errors.

WebView

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [WebView](#) displays remote web pages, local HTML files, and HTML strings, in an app. The content displayed a [WebView](#) includes support for Cascading Style Sheets (CSS), and JavaScript. By default, .NET MAUI projects include the platform permissions required for a [WebView](#) to display a remote web page.

[WebView](#) defines the following properties:

- [Cookies](#), of type `CookieContainer`, provides storage for a collection of cookies.
- [CanGoBack](#), of type `bool`, indicates whether the user can navigate to previous pages. This is a read-only property.
- [CanGoForward](#), of type `bool`, indicates whether the user can navigate forward. This is a read-only property.
- [Source](#), of type `WebViewSource`, represents the location that the [WebView](#) displays.
- [UserAgent](#), of type `string`, represents the user agent. The default value is the user agent of the underlying platform browser, or `null` if it can't be determined.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The `Source` property can be set to an `UrlWebViewSource` object or a `HtmlWebViewSource` object, which both derive from `WebViewSource`. A `UrlWebViewSource` is used for loading a web page specified with a URL, while a `HtmlWebViewSource` object is used for loading a local HTML file, or local HTML.

[WebView](#) defines the following events:

- `Navigating`, that's raised when page navigation starts. The `WebNavigatingEventArgs` object that accompanies this event defines a `Cancel` property of type `bool` that can be used to cancel navigation.
- `Navigated`, that's raised when page navigation completes. The `WebNavigatedEventArgs` object that accompanies this event defines a `Result` property of type `WebNavigationResult` that indicates the navigation result.
- `ProcessTerminated`, that's raised when a [WebView](#) process ends unexpectedly. The `WebViewProcessTerminatedEventArgs` object that accompanies this event defines platform-specific properties that indicate why the process failed.

A [WebView](#) must specify its [HeightRequest](#) and [WidthRequest](#) properties when contained in a [HorizontalStackLayout](#), [StackLayout](#), or [VerticalStackLayout](#). If you fail to specify these properties, the [WebView](#) will not render.

Display a web page

To display a remote web page, set the `Source` property to a `string` that specifies the URI:

XAML

```
<WebView Source="https://learn.microsoft.com/dotnet/maui" />
```

The equivalent C# code is:

C#

```
WebView webView = new WebView  
{  
    Source = "https://learn.microsoft.com/dotnet/maui"  
};
```

URIs must be fully formed with the protocol specified.

ⓘ Note

Despite the `Source` property being of type `WebViewSource`, the property can be set to a string-based URI. This is because .NET MAUI includes a type converter, and an implicit conversion operator, that converts the string-based URI to a `UriWebViewSource` object.

Display local HTML

To display inline HTML, set the `Source` property to a `HtmlWebViewSource` object:

XAML

```
<WebView>  
    <WebView.Source>  
        <HtmlWebViewSource Html="&lt;HTML&gt;&lt;BODY&gt;&lt;H1&gt;.NET  
MAUI&lt;/H1&gt;&lt;P&gt;Welcome to  
WebView.&lt;/P&gt;&lt;/BODY&gt;&lt;HTML&gt;" />  
    </WebView.Source>  
</WebView>
```

```
</WebView.Source>
</WebView>
```

In XAML, HTML strings can become unreadable due to escaping the `<` and `>` symbols. Therefore, for greater readability the HTML can be inlined in a `CDATA` section:

XAML

```
<WebView>
  <WebView.Source>
    <HtmlWebViewSource>
      <HtmlWebViewSource.Html>
        <![CDATA[
          <HTML>
            <BODY>
              <H1>.NET MAUI</H1>
              <P>Welcome to WebView.</P>
            </BODY>
          </HTML>
        ]]>
      </HtmlWebViewSource.Html>
    </HtmlWebViewSource>
  </WebView.Source>
</WebView>
```

The equivalent C# code is:

C#

```
WebView webView = new WebView
{
    Source = new HtmlWebViewSource
    {
        Html = @"<HTML><BODY><H1>.NET MAUI</H1><P>Welcome to WebView.</P>
</BODY></HTML>"
    }
};
```

Display a local HTML file

To display a local HTML file, add the file to the *Resources\Raw* folder of your app project and set its build action to **MauiAsset**. Then, the file can be loaded from inline HTML that's defined in a `HtmlWebViewSource` object that's set as the value of the `Source` property:

XAML

```

<WebView>
  <WebView.Source>
    <HtmlWebViewSource>
      <HtmlWebViewSource.Html>
        <![CDATA[
          <html>
            <head>
            </head>
            <body>
              <h1>.NET MAUI</h1>
              <p>The CSS and image are loaded from local files!</p>
              <p><a href="localfile.html">next page</a></p>
            </body>
          </html>
        ]]>
      </HtmlWebViewSource.Html>
    </HtmlWebViewSource>
  </WebView.Source>
</WebView>

```

The local HTML file can load Cascading Style Sheets (CSS), JavaScript, and images, if they've also been added to your app project with the **MauiAsset** build action.

For more information about raw assets, see [Raw assets](#).

Reload content

WebView has a **Reload** method that can be called to reload its source:

C#

```

WebView webView = new WebView();
...
webView.Reload();

```

When the **Reload** method is invoked the **ReloadRequested** event is fired, indicating that a request has been made to reload the current content.

Perform navigation

WebView supports programmatic navigation with the **GoBack** and **GoForward** methods. These methods enable navigation through the **WebView** page stack, and should only be called after inspecting the values of the **CanGoBack** and **CanGoForward** properties:

C#


```

WebView webView = new WebView();
...

// Go backwards, if allowed.
if (webView.CanGoBack)
{
    webView.GoBack();
}

// Go forwards, if allowed.
if (webView.CanGoForward)
{
    webView.GoForward();
}

```

When page navigation occurs in a [WebView](#), either initiated programmatically or by the user, the following events occur:

- **Navigating**, which is raised when page navigation starts. The `WebNavigatingEventArgs` object that accompanies the **Navigating** event defines a `Cancel` property of type `bool` that can be used to cancel navigation.
- **Navigated**, which is raised when page navigation completes. The `WebNavigatedEventArgs` object that accompanies the **Navigated** event defines a `Result` property of type `WebNavigationResult` that indicates the navigation result.

Handle permissions on Android

When browsing to a page that requests access to the device's recording hardware, such as the camera or microphone, permission must be granted by the [WebView](#) control. The `WebView` control uses the [Android.Webkit.WebChromeClient](#) type on Android to react to permission requests. However, the `WebChromeClient` implementation provided by .NET MAUI ignores permission requests. You must create a new type that inherits from `MauiWebChromeClient` and approves the permission requests.

Important

Customizing the `WebView` to approve permission requests, using this approach, requires Android API 26 or later.

The permission requests from a web page to the `WebView` control are different than permission requests from the .NET MAUI app to the user. .NET MAUI app permissions are requested and approved by the user, for the whole app. The `WebView` control is

dependent on the app's ability to access the hardware. To illustrate this concept, consider a web page that requests access to the device's camera. Even if that request is approved by the `WebView` control, yet the .NET MAUI app didn't have approval by the user to access the camera, the web page wouldn't be able to access the camera.

The following steps demonstrate how to intercept permission requests from the `WebView` control to use the camera. If you are trying to use the microphone, the steps would be similar except that you would use microphone-related permissions instead of camera-related permissions.

1. First, add the required app permissions to the Android manifest. Open the *Platforms/Android/AndroidManifest.xml* file and add the following in the `manifest` node:

XML

```
<uses-permission android:name="android.permission.CAMERA" />
```

2. At some point in your app, such as when the page containing a `WebView` control is loaded, request permission from the user to allow the app access to the camera.

C#

```
private async Task RequestCameraPermission()
{
    PermissionStatus status = await
    Permissions.CheckStatusAsync<Permissions.Camera>();

    if (status != PermissionStatus.Granted)
        await Permissions.RequestAsync<Permissions.Camera>();
}
```

3. Add the following class to the *Platforms/Android* folder, changing the root namespace to match your project's namespace:

C#

```
using Android.Webkit;
using Microsoft.Maui.Handlers;
using Microsoft.Maui.Platform;

namespace MauiAppWebViewHandlers.Platforms.Android;

internal class MyWebChromeClient: MauiWebChromeClient
{
    public MyWebChromeClient(IWebViewHandler handler) : base(handler)
    {
    }
}
```

```

    }

    public override void OnPermissionRequest(PermissionRequest request)
    {
        // Process each request
        foreach (var resource in request.GetResources())
        {
            // Check if the web page is requesting permission to the
            camera
            if (resource.Equals(PermissionRequest.ResourceVideoCapture,
                StringComparison.OrdinalIgnoreCase))
            {
                // Get the status of the .NET MAUI app's access to the
                camera
                PermissionStatus status =
                Permissions.CheckStatusAsync<Permissions.Camera>().Result;

                // Deny the web page's request if the app's access to
                the camera is not "Granted"
                if (status != PermissionStatus.Granted)
                    request.Deny();
                else
                    request.Grant(request.GetResources());

                return;
            }
        }

        base.OnPermissionRequest(request);
    }
}

```

In the previous snippet, the `MyWebChromeClient` class inherits from `MauiWebChromeClient`, and overrides the `OnPermissionRequest` method to intercept web page permission requests. Each permission item is checked to see if it matches the `PermissionRequest.ResourceVideoCapture` string constant, which represents the camera. If a camera permission is matched, the code checks to see if the app has permission to use the camera. If it has permission, the web page's request is granted.

4. Use the `SetWebChromeClient` method on the Android's `WebView` control to set the chrome client to `MyWebChromeClient`. The following two items demonstrate how you can set the chrome client:

- Given a .NET MAUI `WebView` control named `theWebViewControl1`, you can set the chrome client directly on the platform view, which is the Android control:

C#

```
((IWebViewHandler)theWebViewControl.Handler).PlatformView.SetWebChromeClient(new MyWebChromeClient((IWebViewHandler)theWebViewControl.Handler));
```

- You can also use handler property mapping to force all `WebView` controls to use your chrome client. For more information, see [Handlers](#).

The following snippet's `CustomizeWebViewHandler` method should be called when the app starts, such as in the `MauiProgram.CreateMauiApp` method.

C#

```
private static void CustomizeWebViewHandler()
{
    #if ANDROID26_0_OR_GREATER
        Microsoft.Maui.Handlers.WebViewHandler.Mapper.ModifyMapping(
            nameof(Android.Webkit.WebView.WebChromeClient),
            (handler, view, args) =>
            handler.PlatformView.SetWebChromeClient(new
            MyWebChromeClient(handler)));
    #endif
}
```

Set cookies

Cookies can be set on a `WebView` so that they are sent with the web request to the specified URL. Set the cookies by adding `Cookie` objects to a `CookieContainer`, and then set the container as the value of the `WebView.Cookies` bindable property. The following code shows an example:

C#

```
using System.Net;

CookieContainer cookieContainer = new CookieContainer();
Uri uri = new Uri("https://learn.microsoft.com/dotnet/maui",
UriKind.RelativeOrAbsolute);

Cookie cookie = new Cookie
{
    Name = "DotNetMAUICookie",
    Expires = DateTime.Now.AddDays(1),
    Value = "My cookie",
    Domain = uri.Host,
    Path = "/"
};
cookieContainer.Add(uri, cookie);
```

```
webView.Cookies = cookieContainer;  
webView.Source = new UriWebViewSource { Url = uri.ToString() };
```

In this example, a single `Cookie` is added to the `CookieContainer` object, which is then set as the value of the `WebView.Cookies` property. When the `WebView` sends a web request to the specified URL, the cookie is sent with the request.

Invoke JavaScript

`WebView` includes the ability to invoke a JavaScript function from C# and return any result to the calling C# code. This interop is accomplished with the `EvaluateJavaScriptAsync` method, which is shown in the following example:

C#

```
Entry numberEntry = new Entry { Text = "5" };  
Label resultLabel = new Label();  
WebView webView = new WebView();  
...  
  
int number = int.Parse(numberEntry.Text);  
string result = await  
webView.EvaluateJavaScriptAsync($"factorial({number})");  
resultLabel.Text = $"Factorial of {number} is {result}.";
```

The `WebView.EvaluateJavaScriptAsync` method evaluates the JavaScript that's specified as the argument, and returns any result as a `string`. In this example, the `factorial` JavaScript function is invoked, which returns the factorial of `number` as a result. This JavaScript function is defined in the local HTML file that the `WebView` loads, and is shown in the following example:

HTML

```
<html>  
<body>  
<script type="text/javascript">  
function factorial(num) {  
    if (num === 0 || num === 1)  
        return 1;  
    for (var i = num - 1; i >= 1; i--) {  
        num *= i;  
    }  
    return num;  
}  
</script>
```

```
</body>  
</html>
```

Launch the system browser

It's possible to open a URI in the system web browser with the `Launcher` class, which is provided by `Microsoft.Maui.Essentials`. Call the launcher's `OpenAsync` method and pass in a `string` or `Uri` argument that represents the URI to open:

```
C#
```

```
await Launcher.OpenAsync("https://learn.microsoft.com/dotnet/maui");
```

For more information, see [Launcher](#).

Tutorial: Create a .NET MAUI app with C# Markup and the Community Toolkit

Article • 04/27/2023

Build a .NET MAUI app with a user interface created without XAML by using **C# Markup** from the [.NET MAUI Community Toolkit](#).

Introduction

The **.NET MAUI Community Toolkit** is a set of extensions, behaviors, animations, and other helpers. One of the features, [C# Markup](#), provides the ability to create a user interface entirely in C# code. In this tutorial, you'll learn how to create a .NET MAUI app for Windows that uses C# Markup to create the user interface.

Setting up the environment

If you haven't already set up your environment for .NET MAUI development, please follow the steps to [Get started with .NET MAUI on Windows](#).

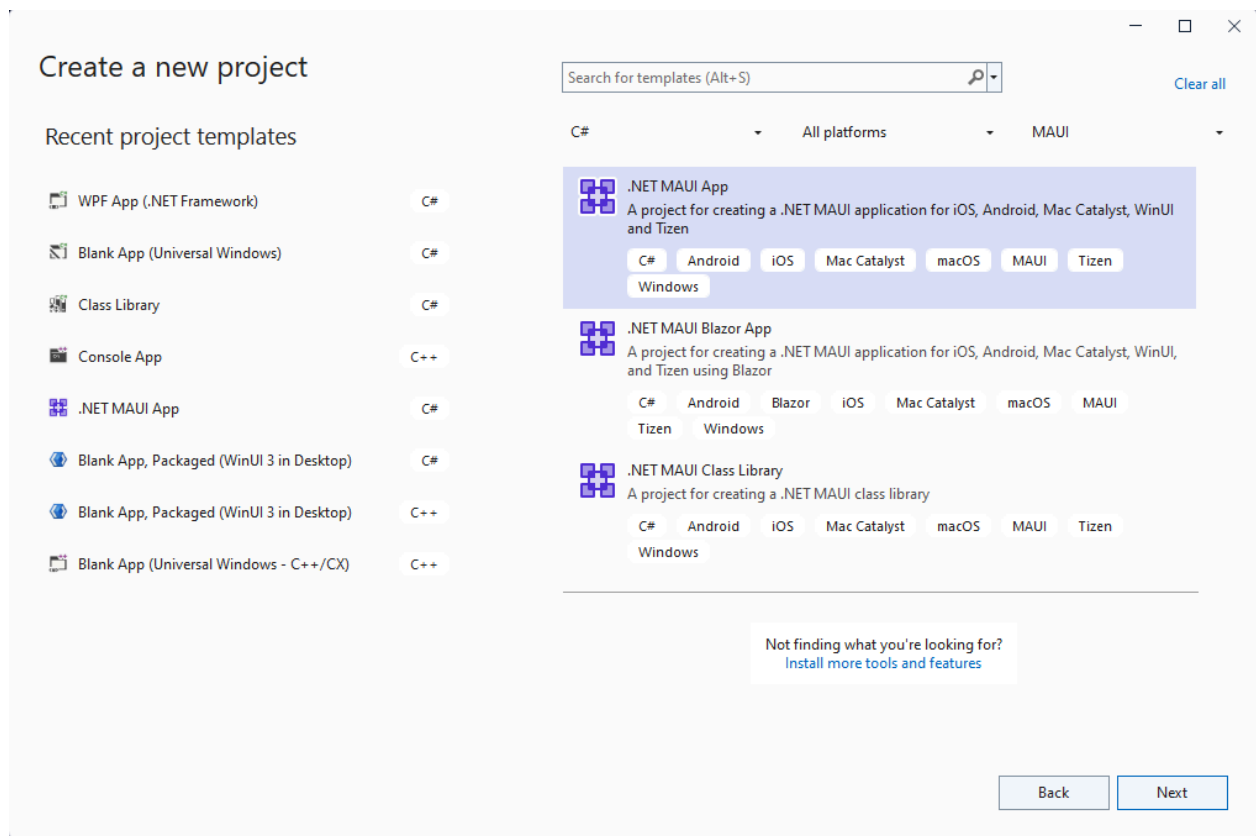
Creating the .NET MAUI project

ⓘ Note

If you are already familiar with setting up a .NET MAUI project, you can skip to the next section.

Launch Visual Studio, and in the start window click **Create a new project** to create a new project.

In the **Create a new project** window, select **MAUI** in the All project types drop-down, select the **.NET MAUI App** template, and click the **Next** button:



Next, on the **Configure your new project** screen, give your project a name, choose a location for it, and click the **Next** button.

On the final screen, **Additional information**, click the **Create** button.

Wait for the project to be created, and for its dependencies to be restored.

In the Visual Studio toolbar, press the **Windows Machine** button to build and run the app. Click the **Click me** button and verify that the button content updates with the number of clicks.

Now that you have verified that the .NET MAUI app on Windows is working as expected, we can integrate the MVVM Toolkit and C# Markup packages. In the next section, you'll add these packages to your new project.

Add C# Markup from the .NET MAUI Community Toolkit

Now that you have your .NET MAUI app running on Windows, let's add a couple of NuGet packages to the project to integrate with the [MVVM Toolkit](#) and **C# Markup** from the **.NET MAUI Community Toolkit**.

Right-click the project in **Solution Explorer** and select **Manage NuGet Packages...** from the context menu.

In the **NuGet Package Manager** window, select the **Browse** tab and search for **CommunityToolkit.MVVM**:

**CommunityToolkit.Mvvm**  by Microsoft, **162K** downloads 8.0.0

This package includes a .NET MVVM library with helpers such as:

- `ObservableObject`: a base class for objects implementing the `INotifyPropertyChanged` interface.

Add the latest stable version of the **CommunityToolkit.MVVM** package to the project by clicking **Install**.

Next, search for **CommunityToolkit.Maui**:

**CommunityToolkit.Maui.Markup**  by Microsoft, **35.3K** downloads 1.2.0

CommunityToolkit.Maui.Markup is a set of fluent helper methods and classes to simplify building declarative .NET MAUI user interfaces in C#

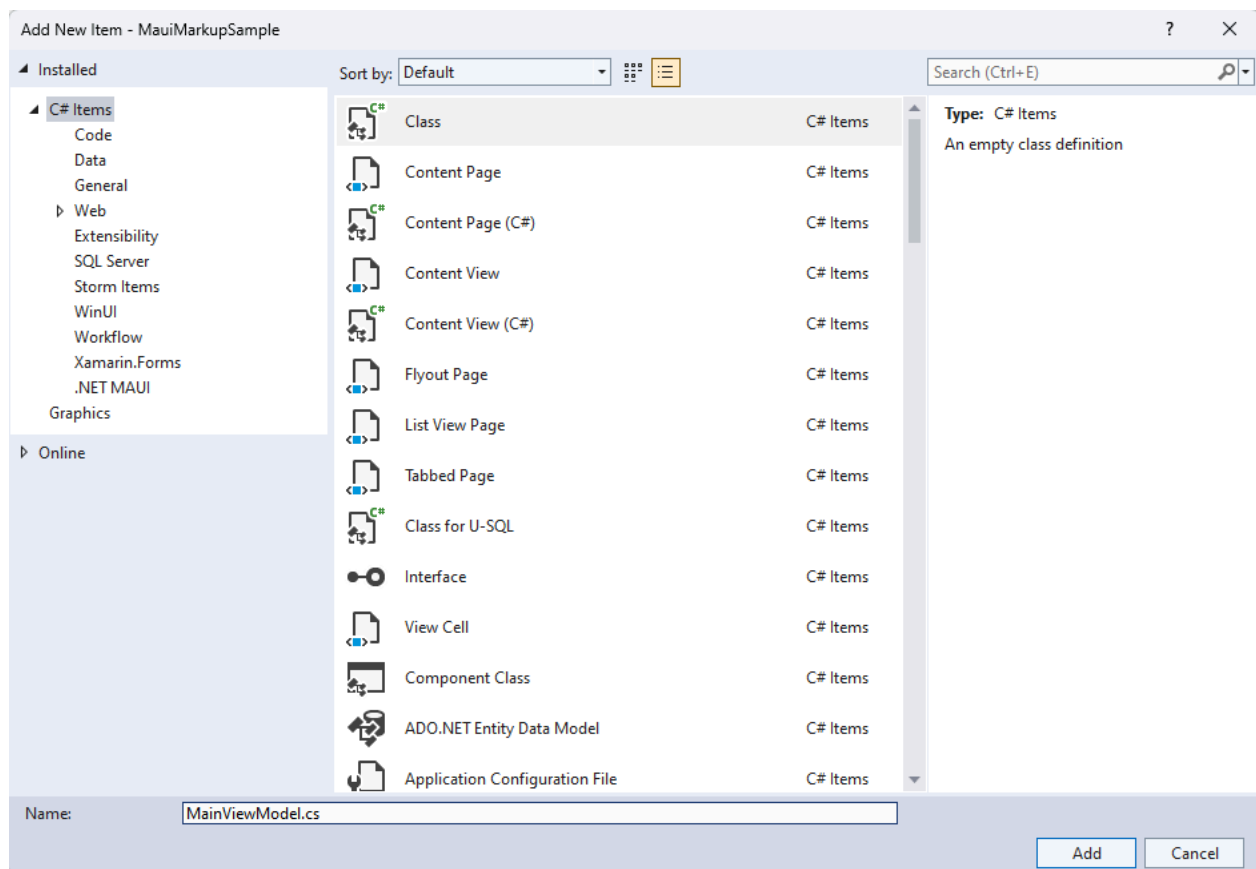
Add the latest stable version of the **CommunityToolkit.Maui.Markup** package to the project by clicking **Install**.

Close the **NuGet Package Manager** window after the new packages have finished installing.

Add a ViewModel to the project

We are going to add a simple **Model-View-ViewModel (MVVM)** implementation with the MVVM Toolkit. Let's start by creating a viewmodel to pair with our view (**MainPage**). Right-click the project again and select **Add | Class** from the context menu.

In the **Add New Item** window that appears, name the class **MainViewModel** and click **Add**:



We are going to leverage the power of the MVVM Toolkit in `MainViewModel1`. Replace the contents of the class with the following code:

C#

```
using CommunityToolkit.Mvvm.ComponentModel;
using System.ComponentModel;
using System.Diagnostics;

namespace MauiMarkupSample
{
    [INotifyPropertyChanged]
    public partial class MainViewModel
    {
        [ObservableProperty]
        private string name;
        partial void OnNameChanging(string value)
        {
            Debug.WriteLine($"Name is about to change to {value}");
        }
        partial void OnNameChanged(string value)
        {
            Debug.WriteLine($"Name has changed to {value}");
        }
    }
}
```

If you have completed the [Build your first .NET MAUI app for Windows](#) tutorial, you will understand what the code above does. The `MainViewModel` class is decorated with the `INotifyPropertyChanged` attribute, which allows the MVVM Toolkit to generate the `INotifyPropertyChanged` implementation for the class. Marking `MainViewModel` as a `partial class` is required for the .NET source generator to work. The `ObservableProperty` attribute on the `name` private field will add a `Name` property for the class with the proper `INotifyPropertyChanged` implementation in the generated partial class. Adding the `OnNameChanging` and `OnNameChanged` partial methods is optional, but allows you to add custom logic when the `Name` property is changing or has changed.

Build a UI with C# Markup

When building a UI with C# Markup, the first step is to update the `CreateMauiApp()` method in `MauiProgram.cs`. Replace the contents of the method with the following code:

C#

```
public static MauiApp CreateMauiApp()
{
    var builder = MauiApp.CreateBuilder();

    builder
        .UseMauiApp<App>()
        .UseMauiCommunityToolkitMarkup();

    return builder.Build();
}
```

You also need to add a new `using` statement to the top of the file: `using CommunityToolkit.Maui.Markup;`. The call to `UseMauiCommunityToolkitMarkup()` will add the C# Markup support to the app, allowing you to construct your UI with C# code instead of XAML.

The `MainPage.xaml` file will no longer be used when rendering the UI, so you can remove the contents of the `ContentPage`.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
```

```
x:Class="MauiMarkupSample.MainPage">
</ContentPage>
```

In **MainPage.xaml.cs**, remove the click event handler and add three private members to the class:

C#

```
private readonly MainViewModel ViewModel = new();
private enum Row { TextEntry }
private enum Column { Description, Input }
```

The **ViewModel** property will create an instance of the **MainViewModel** class to be used when data binding the UI. The **Row** and **Column** enums will be used to define the layout of the UI with C# Markup. It's a simple UI with a single row and two columns which we'll be defining in the next step. You'll also need to add a namespace directive to the top of the file: `using static CommunityToolkit.Maui.Markup.GridRowsColumns;`

Because the UI elements are going to be defined in the C# code, the **InitializeComponent()** method will not be needed. Remove the call and replace it with the following code to create the UI:

C#

```
public MainPage()
{
    Content = new Grid
    {
        RowDefinitions = Rows.Define(
            (Row.TextEntry, 36)),

        ColumnDefinitions = Columns.Define(
            (Column.Description, Star),
            (Column.Input, Stars(2))),

        Children =
        {
            new Label()
                .Text("Customer name:")
                .Row(Row.TextEntry).Column(Column.Description),

            new Entry
            {
                Keyboard = Keyboard.Numeric,
                BackgroundColor = Colors.AliceBlue,
            }.Row(Row.TextEntry).Column(Column.Input)
                .FontSize(15)
                .Placeholder("Enter name")
                .TextColor(Colors.Black)
        }
    }
}
```

```

        .Height(44)
        .Margin(6, 6)
        .Bind(Entry.TextProperty, nameof(ViewModel.Name),
BindingMode.TwoWay)
    }
};
}

```

The new code in the `MainPage` constructor is leveraging C# Markup to define the UI. A `Grid` is set as the `Content` of the page. Our new grid defines a row with a height of 36 pixels and two columns with their widths defined using **Star** values, rather than absolute pixel values. The `Input` column will always be twice the width of the `Description` column. The equivalent XAML for these definitions would be:

XAML

```

<Grid.RowDefinitions>
    <RowDefinition Height="36" />
</Grid.RowDefinitions>
<Grid.ColumnDefinitions>
    <ColumnDefinition Width="*" />
    <ColumnDefinition Width="2*" />
</Grid.ColumnDefinitions>

```

The rest of the code to create the grid adds two `Children`, a `Label` and an `Entry`. The `Text`, `Row`, and `Column` properties are set on the `Label` element, and the `Entry` is created with the following properties:

[Expand table](#)

Property	Value	Description
Row	Row.TextEntry	Defines the row number.
Column	Column.Input	Defines the column number.
FontSize	15	Sets the font size.
Placeholder	"Enter name"	Sets the placeholder text to display when the element is empty.
TextColor	Colors.Black	Sets the text color.
Height	44	Sets the height of the element.
Margin	6, 6	Defines the margin around the element.

Property	Value	Description
Bind	Entry.TextProperty, nameof(ViewModel.Name), BindingMode.TwoWay	Binds the <code>Text</code> property of the element to the <code>Name</code> property of the view model using two-way data binding.

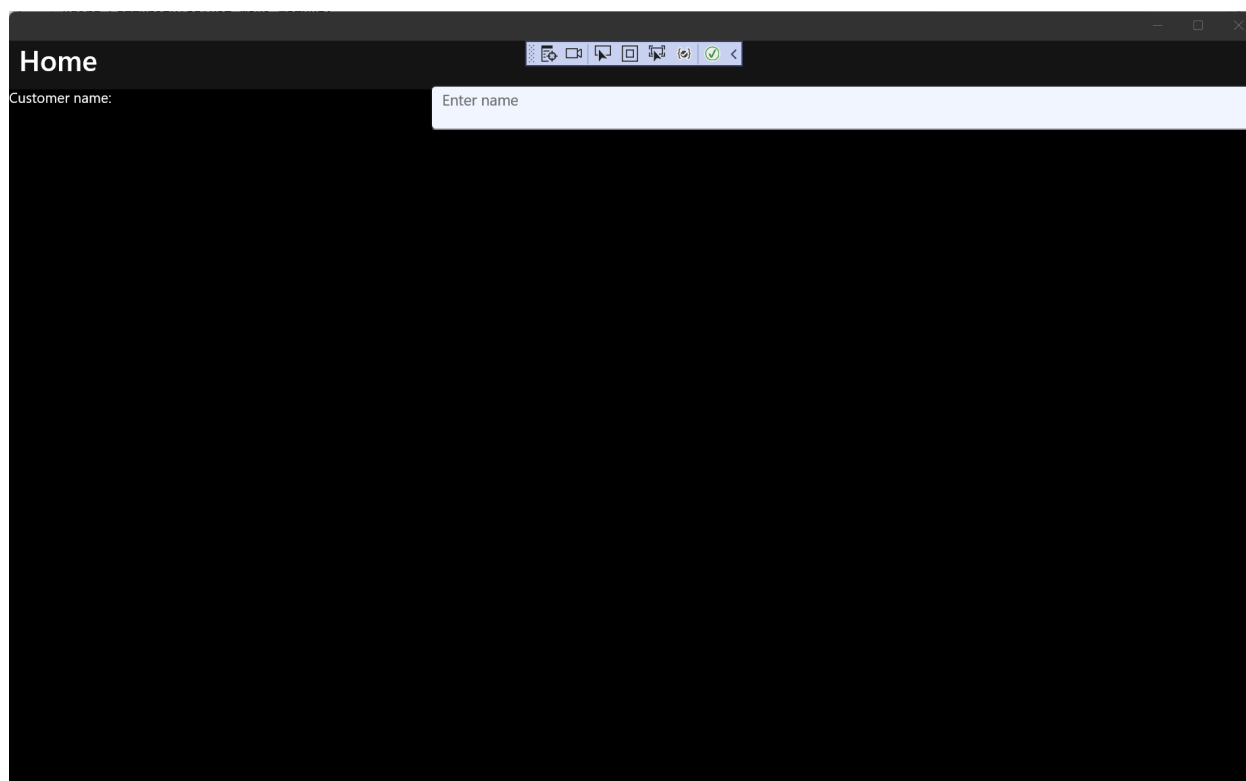
The equivalent XAML to define these child elements would be:

```
XAML

<Label Text="Customer name:"
      Grid.Row="0" Grid.Column="0" />
<Entry Grid.Row="1" Grid.Column="0"
      FontSize="15"
      Placeholder="Enter name"
      HeightRequest="44"
      Margin="6, 6"
      Text="{Binding Path=ViewModel.Name, Mode=TwoWay}" />
```

You may have noticed that the `TextColor` property is not set in the markup above. Setting the `TextColor` of a control requires setting a custom style. For more information about using styles in .NET MAUI, see [Style apps using XAML](#). This is one example where setting properties in C# Markup can be more streamlined than the equivalent XAML. However, using styles in adds ease of reuse and inheritance.

You're now ready to run the app. Press **F5** to build and run the project. The app should look like the following screenshot:



You've now created your first C# Markup app on Windows with .NET MAUI. To learn more about what you can do with C# Markup, see [C# Markup documentation](#).

Related topics

[Resources for learning .NET MAUI](#)

[.NET MAUI Community Toolkit documentation](#)

[C# Markup documentation](#)

Collaborate with us on GitHub

The source for this content can be found on GitHub, where you can also create and review issues and pull requests. For more information, see [our contributor guide](#).



Windows developer feedback

Windows developer is an open source project. Select a link to provide feedback:

 [Open a documentation issue](#)

 [Provide product feedback](#)

Display pop-ups

Article • 09/30/2024

Displaying an alert, asking a user to make a choice, or displaying a prompt is a common UI task. .NET Multi-platform App UI (.NET MAUI) has three methods on the [Page](#) class for interacting with the user via a pop-up: [DisplayAlert](#), [DisplayActionSheet](#), and [DisplayPromptAsync](#). Pop-ups are rendered with native controls on each platform.

Display an alert

All .NET MAUI-supported platforms have a pop-up to alert the user or ask simple questions of them. To display alerts, use the [DisplayAlert](#) method on any [Page](#). The following example shows a simple message to the user:

C#

```
await DisplayAlert("Alert", "You have been alerted", "OK");
```

Alert

You have been alerted

OK

Once the alert is dismissed the user continues interacting with the app.

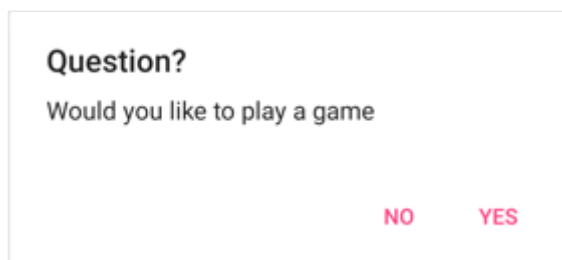
ⓘ Note

On Android, alerts can be dismissed by tapping on the page outside the alert. On desktop platforms, alerts can be dismissed with the escape key.

The [DisplayAlert](#) method can also be used to capture a user's response by presenting two buttons and returning a `bool`. To get a response from an alert, supply text for both buttons and `await` the method:

C#

```
bool answer = await DisplayAlert("Question?", "Would you like to play a game", "Yes", "No");  
Debug.WriteLine("Answer: " + answer);
```

After the user selects one of the options the response will be returned as a `bool`.

The [DisplayAlert](#) method also has overloads that accept a `FlowDirection` argument that specifies the direction in which UI elements flow within the alert. For more information about flow direction, see [Right to left localization](#).

Warning

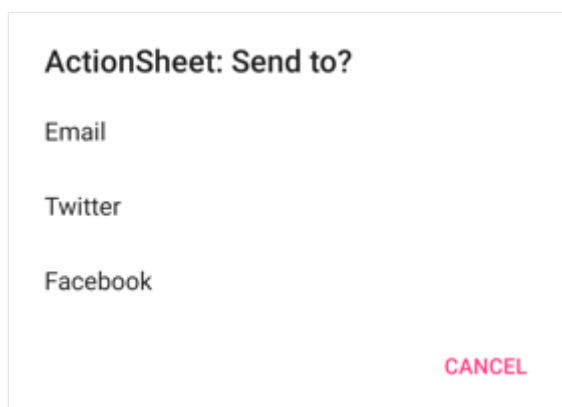
By default on Windows, when an alert is displayed any access keys that are defined on the page behind the alert can still be activated. For more information, see [VisualElement access keys on Windows](#).

Guide users through tasks

An action sheet presents the user with a set of alternatives for how to proceed with a task. To display an action sheet, use the [DisplayActionSheet](#) method on any [Page](#), passing the message and button labels as strings:

C#

```
string action = await DisplayActionSheet("ActionSheet: Send to?", "Cancel",  
null, "Email", "Twitter", "Facebook");  
Debug.WriteLine("Action: " + action);
```



After the user taps one of the buttons, the button label will be returned as a `string`.

📌 Note

Action sheets can be dismissed on touch platforms, and Mac Catalyst, by tapping on the page outside the action sheet. On Windows, action sheets can be dismissed with the escape key and by clicking on the page outside the action sheet.

Action sheets also support a destroy button, which is a button that represents destructive behavior. The destroy button can be specified as the third string argument to the [DisplayActionSheet](#) method, or can be left `null`. The following example specifies a destroy button:

C#

```
async void OnActionSheetCancelDeleteClicked(object sender, EventArgs e)
{
    string action = await DisplayActionSheet("ActionSheet: SavePhoto?",
    "Cancel", "Delete", "Photo Roll", "Email");
    Debug.WriteLine("Action: " + action);
}
```

ActionSheet: SavePhoto?

Photo Roll

Email

DELETE CANCEL

📌 Note

On iOS, the destroy button is rendered differently to the other buttons in the action sheet.

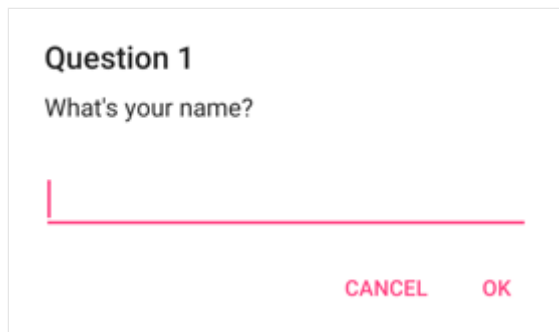
The [DisplayActionSheet](#) method also has an overload that accepts a `FlowDirection` argument that specifies the direction in which UI elements flow within the action sheet. For more information about flow direction, see [Right to left localization](#).

Display a prompt

To display a prompt, call the [DisplayPromptAsync](#) on any [Page](#), passing a title and message as `string` arguments:

C#

```
string result = await DisplayPromptAsync("Question 1", "What's your name?");
```



If the OK button is tapped the entered response is returned as a `string`. If the Cancel button is tapped, `null` is returned.

ⓘ Note

On Android, prompts can be dismissed by tapping on the page outside the alert. On desktop platforms, prompts can be dismissed with the escape key.

The full argument list for the [DisplayPromptAsync](#) method is:

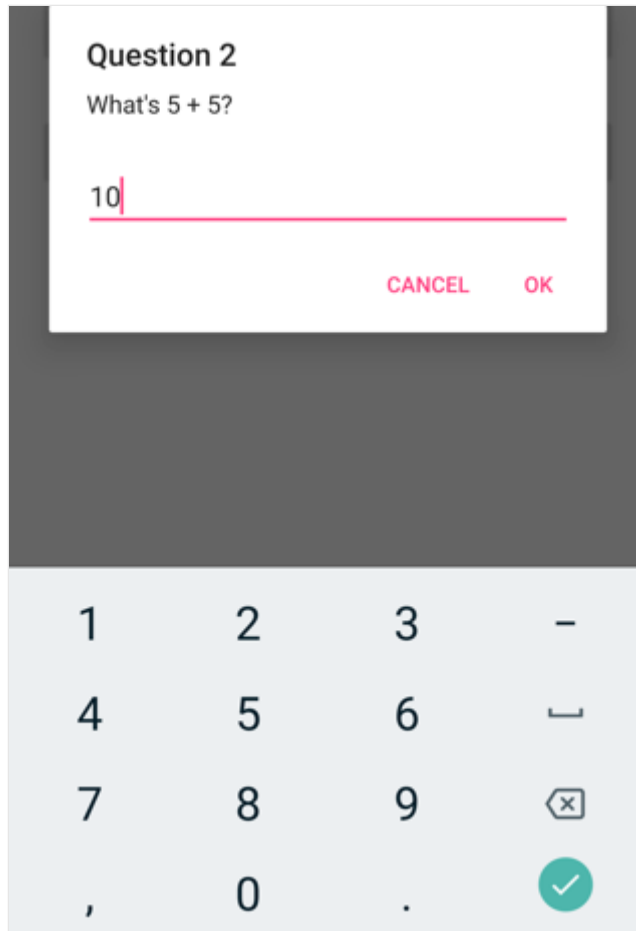
- `title`, of type `string`, is the title to display in the prompt.
- `message`, of type `string`, is the message to display in the prompt.
- `accept`, of type `string`, is the text for the accept button. This is an optional argument, whose default value is OK.
- `cancel`, of type `string`, is the text for the cancel button. This is an optional argument, whose default value is Cancel.
- `placeholder`, of type `string`, is the placeholder text to display in the prompt. This is an optional argument, whose default value is `null`.
- `maxLength`, of type `int`, is the maximum length of the user response. This is an optional argument, whose default value is -1.
- `keyboard`, of type `Keyboard`, is the keyboard type to use for the user response. This is an optional argument, whose default value is `Keyboard.Default`.
- `initialValue`, of type `string`, is a pre-defined response that will be displayed, and which can be edited. This is an optional argument, whose default value is an empty `string`.

The following example shows setting some of the optional arguments:

C#

```
string result = await DisplayPromptAsync("Question 2", "What's 5 + 5?",  
    initialValue: "10", maxLength: 2, keyboard: Keyboard.Numeric);
```

This code displays a predefined response of 10, limits the number of characters that can be input to 2, and displays the numeric keyboard for user input:



Warning

By default on Windows, when a prompt is displayed any access keys that are defined on the page behind the prompt can still be activated. For more information, see [VisualElement access keys on Windows](#).

Display a page as a pop-up

.NET MAUI supports modal page navigation. A modal page encourages users to complete a self-contained task that can't be navigated away from until the task is completed or canceled. For example, to display a form as a pop-up that requires users to enter multiple pieces of data, create a [ContentPage](#) that contains the UI for your form and then push it onto the navigation stack as a modal page. For more information, see [Perform modal navigation](#).

Display toolbar items

Article • 09/30/2024

The .NET Multi-platform App UI (.NET MAUI) [ToolbarItem](#) class is a special type of button that can be added to a [Page](#) object's [ToolbarItems](#) collection. Because the [Shell](#) class derives from [Page](#), [ToolbarItem](#) objects can also be added to the `ToolbarItems` collection of a [Shell](#) object. Each [ToolbarItem](#) object will appear as a button in the app's navigation bar. A [ToolbarItem](#) object can have an icon and appear as a primary or secondary item. The [ToolbarItem](#) class inherits from [MenuItem](#).

The following screenshot shows a [ToolbarItem](#) object in the navigation bar on iOS:



The [ToolbarItem](#) class defines the following properties:

- [Order](#), of type [ToolbarItemOrder](#), determines whether the [ToolbarItem](#) object displays in the primary or secondary menu.
- [Priority](#), of type `int`, determines the display order of items in a [ToolbarItems](#) collection.

The [ToolbarItem](#) class inherits the following typically used properties from the [MenuItem](#) class:

- [Command](#), of type [ICommand](#), allows binding user actions, such as finger taps or clicks, to commands defined on a viewmodel.
- [CommandParameter](#), of type `object`, specifies the parameter that should be passed to the `Command`.
- [IconImageSource](#), of type [ImageSource](#), that determines the display icon on a [ToolbarItem](#) object.
- [Text](#), of type `string`, determines the display text on a [ToolbarItem](#) object.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings.

📌 Note

An alternative to creating a toolbar from [ToolBarItem](#) objects is to set the [TitleViewProperty](#) attached property to a layout class that contains multiple views. For more information, see [Display views in the navigation bar](#).

Create a ToolBarItem

To create a toolbar item, create a [ToolBarItem](#) object and set its properties to define its appearance and behavior. The following example shows how to create a [ToolBarItem](#) with minimal properties set, and add it to a [ContentPage](#)'s [ToolBarItems](#) collection:

XAML

```
<ContentPage.ToolbarItems>
  <ToolBarItem Text="Add item"
               IconImageSource="add.png" />
</ContentPage.ToolbarItems>
```

This example results in a [ToolBarItem](#) object that has text and an icon. However, the appearance of a [ToolBarItem](#) varies across platforms.

A [ToolBarItem](#) can also be created in code and added to the [ToolBarItems](#) collection:

C#

```
ToolBarItem item = new ToolBarItem
{
    Text = "Add item",
    IconImageSource = ImageSource.FromFile("add.png")
};

// "this" refers to a Page object
this.ToolbarItems.Add(item);
```

ⓘ Note

Images can be stored in a single location in your app project. For more information, see [Add images to a .NET MAUI project](#).

Define button behavior

The [ToolBarItem](#) class inherits the [Clicked](#) event from the [MenuItem](#) class. An event handler can be attached to the [Clicked](#) event to react to taps or clicks on [ToolBarItem](#)

objects:

XAML

```
<ToolBarItem ...  
    Clicked="OnItemClicked" />
```

An event handler can also be attached in code:

C#

```
ToolBarItem item = new ToolBarItem { ... };  
item.Clicked += OnItemClicked;
```

These examples reference an `OnItemClicked` event handler, which is shown in the following example:

C#

```
void OnItemClicked(object sender, EventArgs e)  
{  
    ToolBarItem item = (ToolBarItem)sender;  
    messageLabel.Text = $"You clicked the \"{item.Text}\" toolbar item."  
}
```

ⓘ Note

[ToolBarItem](#) objects can also use the [Command](#) and [CommandParameter](#) properties to react to user input without event handlers.

Enable or disable a ToolBarItem at runtime

To enable or disable a [ToolBarItem](#) at runtime, bind its [Command](#) property to an [ICommand](#) implementation, and ensure that its `canExecute` delegate enables and disables the [ICommand](#) as appropriate.

ⓘ Important

Don't bind the `IsEnabled` property to another property when using the `Command` property to enable or disable the [ToolBarItem](#).

Primary and secondary toolbar items

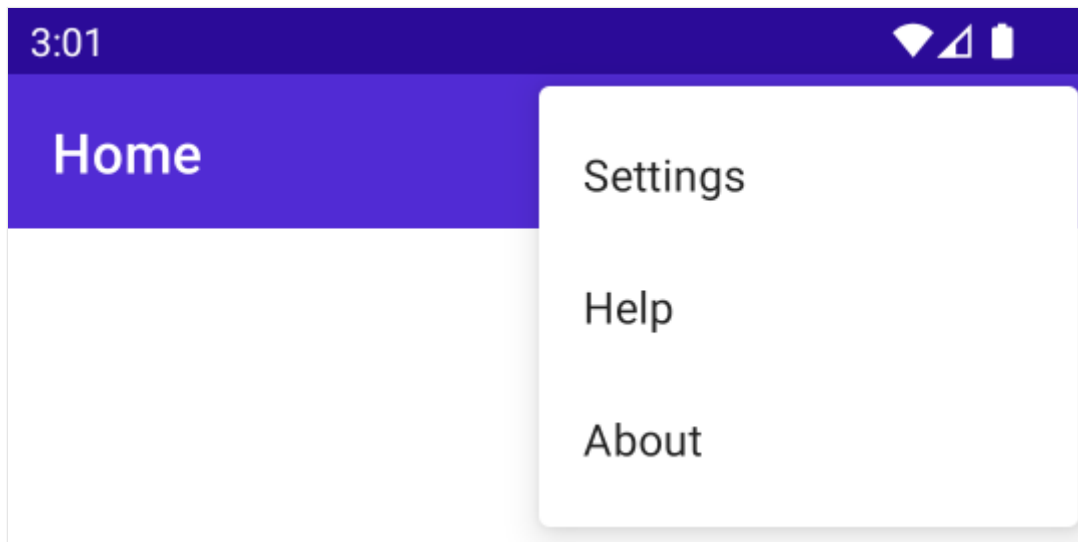
The `ToolBarItemOrder` enum has `Default`, `Primary`, and `Secondary` values.

When the `Order` property is set to `Primary`, the `ToolBarItem` object appears in the navigation bar on all platforms. `ToolBarItem` objects are prioritized over the page title, which will be truncated to make room for the items.

When the `Order` property is set to `Secondary`, behavior varies across platforms. On iOS and Mac Catalyst, `Secondary` toolbar items appear as a horizontal list. On Android and Windows, the `Secondary` items menu appears as three dots that can be tapped:



Tapping the three dots reveals items in a vertical list:



Warning

Icon behavior in `ToolBarItem` objects that have their `Order` property set to `Secondary` is inconsistent across platforms. Avoid setting the `IconImageSource` property on items that appear in the secondary menu.

Display tooltips

Article • 09/30/2024

A .NET Multi-platform App UI (.NET MAUI) tooltip is a small rectangular popup that displays a brief description of a view's purpose, when the user rests the pointer on the [view](#). Tooltips display automatically, typically when the user hovers the pointer over the associated view:

- On Android, tooltips are displayed when users long press the view. Tooltips remain visible for a few seconds after the long press is released.
- On iOS, to show a tooltip your app must be an iPhone or iPad app running on a Mac with Apple silicon. Providing this criteria is met, tooltips are displayed when the pointer is positioned over the view for a few seconds, and remain visible until the pointer moves away from the view. Tooltips on iOS require the use of iOS 15.0+. For more information about using iPhone and iPad apps on a Mac with Apple silicon, see [Use iPhone and iPad apps on Mac with Apple silicon](#) [↗](#).
- On Mac Catalyst, tooltips are displayed when the pointer is positioned over the view for a few seconds. Tooltips remain visible until the pointer moves away from the view. Tooltips on Mac Catalyst require the use of Mac Catalyst 15.0+.
- On Windows, tooltips are displayed when the pointer hovers over the view. Tooltips remain visible for a few seconds, or until the pointer stops hovering over the view.

Tooltips are defined by setting the `ToolTipProperties.Text` attached property, on a view, to a `string`:

XAML

```
<Button Text="Save"
        ToolTipProperties.Text="Click to Save your data" />
```

The equivalent C# code is:

C#

```
Button button = new Button { Text = "Save" };
ToolTipProperties.SetText(button, "Click to Save your data");
```

For more information about attached properties, see [Attached properties](#).

By default, a tooltip is displayed centered above the pointer:

Press to Save your data

Save

ⓘ Note

Configuring tooltip placement is currently unsupported.

Fonts in .NET MAUI

Article • 09/30/2024

By default, .NET Multi-platform App UI (.NET MAUI) apps use the Open Sans font on each platform. However, this default can be changed, and additional fonts can be registered for use in an app.

All controls that display text define properties that can be set to change font appearance:

- `FontFamily`, of type `string`.
- `FontAttributes`, of type `FontAttributes`, which is an enumeration with three members: `None`, `Bold`, and `Italic`. The default value of this property is `None`.
- `FontSize`, of type `double`.
- `FontAutoScalingEnabled`, of type `bool`, which defines whether an app's UI reflects text scaling preferences set in the operating system. The default value of this property is `true`.

These properties are backed by `BindableProperty` objects, which means that they can be targets of data bindings, and styled.

All controls that display text automatically use font scaling, which means that an app's UI reflects text scaling preferences set in the operating system.

Register fonts

True type format (TTF) and open type font (OTF) fonts can be added to your app and referenced by filename or alias, with registration being performed in the `CreateMauiApp` method in the `MauiProgram` class. This is accomplished by invoking the `ConfigureFonts` method on the `MauiApplicationBuilder` object. Then, on the `IFontCollection` object, call the `AddFont` method to add the required font to your app:

C#

```
namespace MyMauiApp
{
    public static class MauiProgram
    {
        public static MauiApp CreateMauiApp()
        {
            var builder = MauiApp.CreateBuilder();
            builder
                .UseMauiApp<App>()
```

```

        .ConfigureFonts(fonts =>
        {
            fonts.AddFont("Lobster-Regular.ttf", "Lobster");
        });

        return builder.Build();
    }
}

```

In the example above, the first argument to the `AddFont` method is the font filename, while the second argument represents an optional alias by which the font can be referenced when consuming it.

A font can be added to your app project by dragging it into the *Resources\Fonts* folder of the project, where its build action will automatically be set to **MauiFont**. This creates a corresponding entry in your project file. Alternatively, all fonts in the app can be registered by using a wildcard in your project file:

XML

```

<ItemGroup>
    <MauiFont Include="Resources\Fonts\*" />
</ItemGroup>

```

Fonts can also be added to other folders of your app project. However, in this scenario their build action must be manually set to **MauiFont** in the **Properties** window.

At build time, fonts are copied to your app package. For information about disabling font packaging, see [Disable font packaging](#).

ⓘ Note

The `*` wildcard character indicates that all the files within the folder will be treated as being font files. In addition, if you want to include files from sub-folders too, then configure it using additional wildcard characters, for example,

```
Resources\Fonts\**\*.
```

Consume fonts

Registered fonts can be consumed by setting the `FontFamily` property of a control that displays text to the font name, without the file extension:

XAML

```
<!-- Use font name -->
<Label Text="Hello .NET MAUI"
      FontFamily="Lobster-Regular" />
```

Alternatively, it can be consumed by referencing its alias:

XAML

```
<!-- Use font alias -->
<Label Text="Hello .NET MAUI"
      FontFamily="Lobster" />
```

The equivalent C# code is:

C#

```
// Use font name
Label label1 = new Label
{
    Text = "Hello .NET MAUI!",
    FontFamily = "Lobster-Regular"
};

// Use font alias
Label label2 = new Label
{
    Text = "Hello .NET MAUI!",
    FontFamily = "Lobster"
};
```

On Android, the following system fonts can be consumed by setting them as the value of the `FontFamily` property:

- monospace
- serif
- sans-serif (or sansserif)
- sans-serif-black (or sansserif-black)
- sans-serif-condensed (or sansserif-condensed)
- sans-serif-condensed-light (or sansserif-condensed-light)
- sans-serif-light (or sansserif-light)
- sans-serif-medium (or sansserif-medium)

For example, the monospace system font can be consumed with the following XAML:

XAML

```
<Label Text="Hello .NET MAUI"
        FontFamily="monospace" />
```

The equivalent C# code is:

C#

```
// Use font name
Label label1 = new Label
{
    Text = "Hello .NET MAUI!",
    FontFamily = "monospace"
};
```

Set font attributes

Controls that display text can set the `FontAttributes` property to specify font attributes:

XAML

```
<Label Text="Italics"
        FontAttributes="Italic" />
<Label Text="Bold and italics"
        FontAttributes="Bold, Italic" />
```

The equivalent C# code is:

C#

```
Label label1 = new Label
{
    Text = "Italics",
    FontAttributes = FontAttributes.Italic
};

Label label2 = new Label
{
    Text = "Bold and italics",
    FontAttributes = FontAttributes.Bold | FontAttributes.Italic
};
```

Set the font size

Controls that display text can set the `FontSize` property to specify the font size. The `FontSize` property can be set to a `double` value:

XAML

```
<Label Text="Font size 24"
        FontSize="24" />
```

The equivalent C# code is:

C#

```
Label label = new Label
{
    Text = "Font size 24",
    FontSize = 24
};
```

ⓘ Note

The `FontSize` value is measured in device-independent units.

Disable font auto scaling

All controls that display text have font scaling enabled by default, which means that an app's UI reflects text scaling preferences set in the operating system. However, this behavior can be disabled by setting the `FontAutoScalingEnabled` property on text-based controls to `false`:

XAML

```
<Label Text="Scaling disabled"
        FontSize="18"
        FontAutoScalingEnabled="False" />
```

This approach is useful when you want to guarantee that text is displayed at a specific size.

ⓘ Note

Font auto scaling also works with font icons. For more information, see [Display font icons](#).

Set font properties per platform

The `OnPlatform` and `On` classes can be used in XAML to set font properties per platform. The example below sets different font families and sizes:

XAML

```
<Label Text="Different font properties on different platforms"
      FontSize="{OnPlatform iOS=20, Android=22, WinUI=24}">
  <Label.FontFamily>
    <OnPlatform x:TypeArguments="x:String">
      <On Platform="iOS" Value="MarkerFelt-Thin" />
      <On Platform="Android" Value="Lobster-Regular" />
      <On Platform="WinUI" Value="ArimaMadurai-Black" />
    </OnPlatform>
  </Label.FontFamily>
</Label>
```

The `DeviceInfo.Platform` property can be used in code to set font properties per platform:

C#

```
Label label = new Label
{
    Text = "Different font properties on different platforms"
};

label.FontSize = DeviceInfo.Platform == DevicePlatform.iOS ? 20 :
    DeviceInfo.Platform == DevicePlatform.Android ? 22 : 24;
label.FontFamily = DeviceInfo.Platform == DevicePlatform.iOS ? "MarkerFelt-
Thin" :
    DeviceInfo.Platform == DevicePlatform.Android ? "Lobster-Regular" :
    "ArimaMadurai-Black";
```

For more information about providing platform-specific values, see [Device information](#). For information about the `OnPlatform` markup extension, see [Customize UI appearance based on the platform](#).

Display font icons

Font icons can be displayed by .NET MAUI apps by specifying the font icon data in a [FontImageSource](#) object. This class, which derives from the [ImageSource](#) class, has the following properties:

- **Glyph** – the unicode character value of the font icon, specified as a `string`.
- **Size** – a `double` value that indicates the size, in device-independent units, of the rendered font icon. The default value is 30. In addition, this property can be set to a named font size.
- **FontFamily** – a `string` representing the font family to which the font icon belongs.
- **Color** – an optional [Color](#) value to be used when displaying the font icon.

This data is used to create a PNG, which can be displayed by any view that can display an [ImageSource](#). This approach permits font icons, such as emojis, to be displayed by multiple views, as opposed to limiting font icon display to a single text presenting view, such as a [Label](#).

Important

Font icons can only currently be specified by their unicode character representation.

The following XAML example has a single font icon being displayed by an [Image](#) view:

XAML

```
<Image BackgroundColor="#D1D1D1">
  <Image.Source>
    <FontImageSource Glyph="&#xf30c;"
                      FontFamily="{OnPlatform iOS=Ionicons,
Android=ionicons.ttf#}"
                      Size="44" />
  </Image.Source>
</Image>
```

This code displays an Xbox icon, from the Ionicons font family, in an [Image](#) view. Note that while the unicode character for this icon is `\uf30c`, it has to be escaped in XAML and so becomes ``. The equivalent C# code is:

C#

```
Image image = new Image { BackgroundColor = Color.FromArgb("#D1D1D1") };
image.Source = new FontImageSource
{
    Glyph = "\uf30c",
    FontFamily = DeviceInfo.Platform == DevicePlatform.iOS ? "Ionicons" :
```

```
"ionicons.ttf#",  
    Size = 44  
};
```

The following screenshot shows several font icons being displayed:



Alternatively, you can display a font icon with the [FontImage](#) markup extension. For more information, see [Load a font icon](#).

Graphics

Article • 09/30/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) provides a cross-platform graphics canvas on which 2D graphics can be drawn using types from the [Microsoft.Maui.Graphics](#) namespace. This canvas supports drawing and painting shapes and images, compositing operations, and graphical object transforms.

There are many similarities between the functionality provided by [Microsoft.Maui.Graphics](#), and the functionality provided by .NET MAUI shapes and brushes. However, each is aimed at different scenarios:

- [Microsoft.Maui.Graphics](#) functionality must be consumed on a drawing canvas, enables performant graphics to be drawn, and provides a convenient approach for writing graphics-based controls. For example, a control that replicates the GitHub contribution profile can be more easily implemented using [Microsoft.Maui.Graphics](#) than by using .NET MAUI shapes.
- .NET MAUI shapes can be consumed directly on a page, and brushes can be consumed by all controls. This functionality is provided to help you produce an attractive UI.

For more information about .NET MAUI shapes, see [Shapes](#).

Draw graphics

In .NET MAUI, the [GraphicsView](#) enables consumption of [Microsoft.Maui.Graphics](#) functionality. [GraphicsView](#) defines the `Drawable` property, of type `IDrawable`, which specifies the content that will be drawn by the control. To specify the content that will be drawn you must create an object that derives from `IDrawable`, and implement its `Draw` method:

C#

```
namespace MyMauiApp
{
    public class GraphicsDrawable : IDrawable
    {
        public void Draw(ICanvas canvas, RectF dirtyRect)
        {
            // Drawing code goes here
        }
    }
}
```

```
}  
}
```

The `Draw` method has `ICanvas` and `RectF` arguments. The `ICanvas` argument is the drawing canvas on which you draw graphical objects. The `RectF` argument is a `struct` that contains data about the size and location of the drawing canvas.

In XAML, the `IDrawable` object can be declared as a resource and then consumed by a `GraphicsView` by specifying its key as the value of the `Drawable` property:

XAML

```
<ContentPage xmlns=http://schemas.microsoft.com/dotnet/2021/maui  
             xmlns:x=http://schemas.microsoft.com/winfx/2009/xaml  
             xmlns:drawable="clr-namespace:MyMauiApp"  
             x:Class="MyMauiApp.MainPage">  
    <ContentPage.Resources>  
        <drawable:GraphicsDrawable x:Key="drawable" />  
    </ContentPage.Resources>  
    <VerticalStackLayout>  
        <GraphicsView Drawable="{StaticResource drawable}"  
                      HeightRequest="300"  
                      WidthRequest="400" />  
    </VerticalStackLayout>  
</ContentPage>
```

For more information about the `GraphicsView`, see [GraphicsView](#).

Drawing canvas

The `GraphicsView` control provides access to an `ICanvas` object, via its `IDrawable` object, on which properties can be set and methods invoked to draw graphical objects. For information about drawing on an `ICanvas`, see [Draw graphical objects](#).

`ICanvas` defines the following properties that affect the appearance of objects that are drawn on the canvas:

- `Alpha`, of type `float`, indicates the opacity of an object.
- `Antialias`, of type `bool`, specifies whether anti-aliasing is enabled.
- `BlendMode`, of type `BlendMode`, defines the blend mode, which determines what happens when an object is rendered on top of an existing object.
- `DisplayScale`, of type `float`, represents the scaling factor to scale the UI by on a canvas.
- `FillColor`, of type `Color`, indicates the color used to paint an object's interior.

- **Font**, of type **IFont**, defines the font when drawing text.
- **FontColor**, of type **Color**, specifies the font color when drawing text.
- **FontSize**, of type `float`, defines the size of the font when drawing text.
- **MiterLimit**, of type `float`, specifies the limit of the miter length of line joins in an object.
- **StrokeColor**, of type **Color**, indicates the color used to paint an object's outline.
- **StrokeDashOffset**, of type `float`, specifies the distance within the dash pattern where a dash begins.
- **StrokeDashPattern**, of type `float[]`, specifies the pattern of dashes and gaps that are used to outline an object.
- **StrokeLineCap**, of type **LineCap**, describes the shape at the start and end of a line.
- **StrokeLineJoin**, of type **LineJoin**, specifies the type of join that is used at the vertices of a shape.
- **StrokeSize**, of type `float`, indicates the width of an object's outline.

By default, an **ICanvas** sets **StrokeSize** to 1, **StrokeColor** to black, **StrokeLineJoin** to `LineJoin.Miter`, and **StrokeLineCap** to `LineJoin.Cap`.

Drawing canvas state

The drawing canvas on each platform has the ability to maintain its state. This enables you to persist the current graphics state, and restore it when required.

However, not all elements of the canvas are elements of the graphics state. The graphics state does not include drawing objects, such as paths, and paint objects, such as gradients. Typical elements of the graphics state on each platform include stroke and fill data, and font data.

The graphics state of each **ICanvas** can be manipulated with the following methods:

- **SaveState**, which saves the current graphics state.
- **RestoreState**, which sets the graphics state to the most recently saved state.
- **ResetState**, which resets the graphics state to its default values.

ⓘ Note

The state that's persisted by these methods is platform dependent.

Blend modes

Article • 09/30/2024

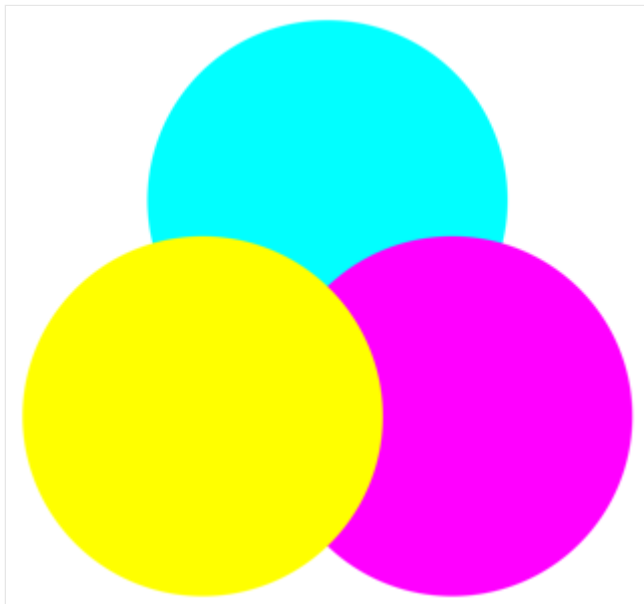
 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) graphics enables different compositing operations for graphical objects to be specified by the `ICanvas.BlendMode` property. This property determines what happens when a graphical object (called the *source*), is rendered on top of an existing graphical object (called the *destination*).

Important

Blend modes are only implemented on iOS and Mac Catalyst. For more information, see [this GitHub issue](#) .

By default, the last drawn object obscures the objects drawn underneath it:



In this example, the cyan circle is drawn first, followed by the magenta circle, then the yellow circle. Each circle obscures the circle drawn underneath it. This occurs because the default blend mode is `Normal`, which means that the source is drawn over the destination. However, it's possible to specify a different blend mode for a different result. For example, if you specify `DestinationOver`, then in the area where the source and destination intersect, the destination is drawn over the source.

The 28 members of the `BlendMode` enumeration can be divided into three categories:

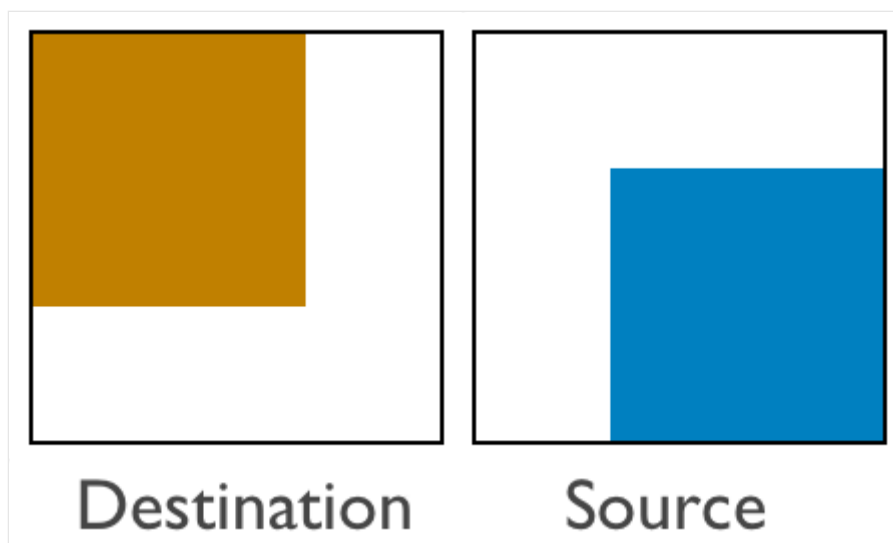
 [Expand table](#)

Separable	Non-Separable	Porter-Duff
Normal	Hue	Clear
Multiply	Saturation	Copy
Screen	Color	SourceIn
Overlay	Luminosity	SourceOut
Darken		SourceAtop
Lighten		DestinationOver
ColorDodge		DestinationIn
ColorBurn		DestinationOut
SoftLight		DestinationAtop
HardLight		Xor
Difference		PlusDarker
Exclusion		PlusLighter

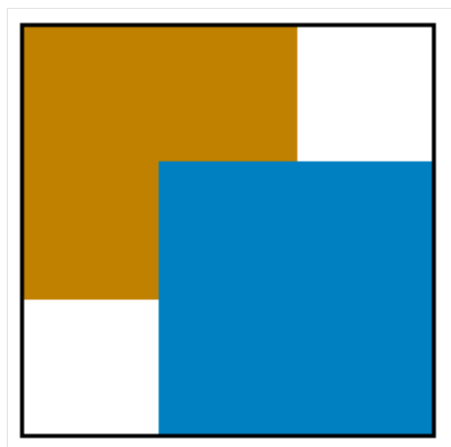
The order that the members are listed in the table above is the same as in the [BlendMode](#) enumeration. The first column lists the 12 *separable* blend modes, while the second column lists the *non-separable* blend modes. Finally, the third column lists the *Porter-Duff* blend modes.

Porter-Duff blend modes

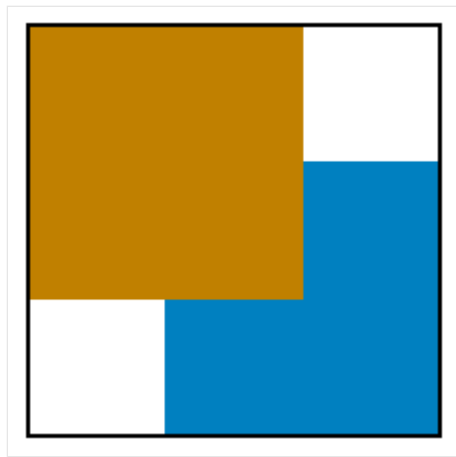
The Porter-Duff blend modes, named after Thomas Porter and Tom Duff, define 12 compositing operators that describe how to compute the color resulting from the composition of the source with the destination. These compositing operators can best be described by considering the case of drawing two rectangles that contain transparent areas:



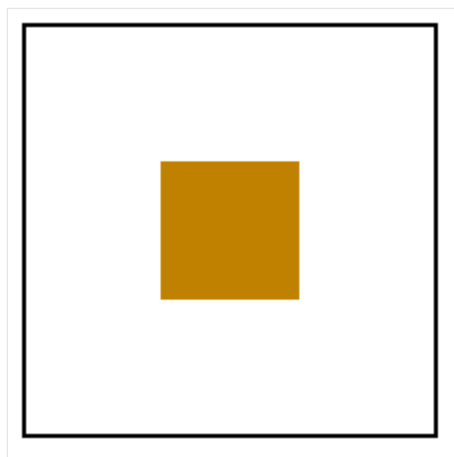
In the image above, the destination is a transparent rectangle except for a brown area that occupies the left and top two-thirds of the display surface. The source is also a transparent rectangle except for a blue area that occupies the right and bottom two-thirds of the display surface. Displaying the source on the destination produces the following result:



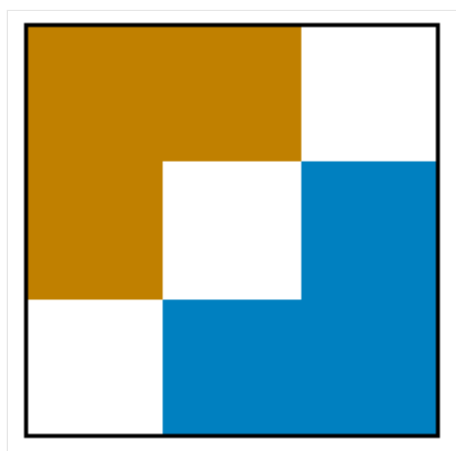
The transparent pixels of the source allow the background to show through, while the blue source pixels obscure the background. This is the normal case, using the default blend mode of `Normal`. However, it's possible to specify that in the area where the source and destination intersect, the destination appears instead of the source, using the `DestinationOver` blend mode:



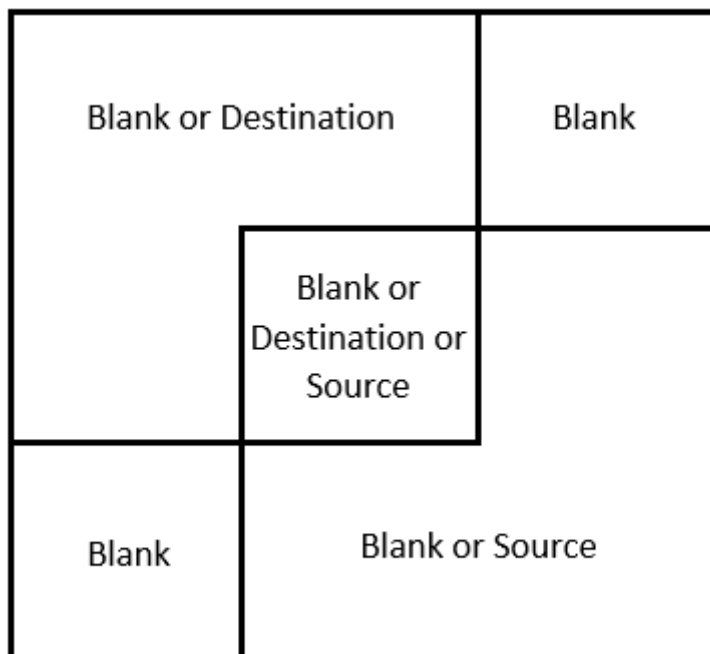
The `DestinationIn` blend mode displays only the area where the destination and source intersect, using the destination color:



The `xor` blend mode causes nothing to appear where the two areas overlap:



The colored destination and source rectangles effectively divide the display surface into four unique areas that can be colored in different ways, corresponding to the presence of the destination and source rectangles:



The upper-right and lower-left rectangles are always blank because both the destination and source are transparent in those areas. The destination color occupies the upper-left area, so that area can either be colored with the destination color or not at all. Similarly, the source color occupies the lower-right area, so that area can be colored with the source color or not at all.

The following table lists the Porter-Duff blend modes provided by [Microsoft.Maui.Graphics](#), and how they color each of the three non-blank areas in the diagram above:

[Expand table](#)

Blend mode	Destination	Intersection	Source
Clear			
Copy		Source	X
SourceIn		Source	
SourceOut			X
SourceAtop	X	Source	
DestinationOver	X	Destination	X
DestinationIn		Destination	
DestinationOut	X		
DestinationAtop		Destination	X

Blend mode	Destination	Intersection	Source
Xor	X		X
PlusDarker	X	Sum	X
PlusLighter	X	Sum	X

The naming convention of the modes follows a few simple rules:

- The *Over* suffix indicates what is visible in the intersection. Either the source or destination is drawn over the other.
- The *In* suffix means that only the intersection is colored. The output is restricted to only the part of the source or destination that is in the other.
- The *Out* suffix means that the intersection isn't colored. The output is only the part of the source or destination that is out of the intersection.
- The *Atop* suffix is the union of *In* and *Out*. It includes the area where the source or destination is atop of the other.

ⓘ Note

These blend modes are symmetrical. The source and destination can be exchanged, and all the modes are still available.

The `PlusLighter` blend mode sums the source and destination. Then, for values above 1, white is displayed. Similarly, the `PlusDarker` blend mode sums the source and destination, but subtracts 1 from the resulting values, with values below 0 becoming black.

Separable blend modes

The separable blend modes alter the individual red, green, and blue color components of a graphical object.

The following table shows the separable blend modes, with brief explanations of what they do. In the table, `Dc` and `Sc` refer to the destination and source colors, and the second column shows the source color that produces no change:

[Expand table](#)

Blend mode	No change	Operation
Normal		No blending, source selected
Multiply	White	Darkens by multiplying $Sc \cdot Dc$
Screen	Black	Complements the product of complements: $Sc + Dc - Sc \cdot Dc$
Overlay	Gray	Inverse of <code>HardLight</code>
Darken	White	Minimum of colors: $\min(Sc, Dc)$
Lighten	Black	Maximum of colors: $\max(Sc, Dc)$
ColorDodge	Black	Brightens the destination based on the source
ColorBurn	White	Darkens the destination based on the source
SoftLight	Gray	Similar to the effect of a soft spotlight
HardLight	Gray	Similar to the effect of a harsh spotlight
Difference	Black	Subtracts the darker color from the lighter color: $Abs(Dc - Sc)$
Exclusion	Black	Similar to <code>Difference</code> but lower contrast

! Note

If the source is transparent, then the separable blend modes have no effect.

The following example uses the `Multiply` blend mode to draw three overlapping circles of cyan, magenta, and yellow:

C#

```
PointF center = new PointF(dirtyRect.Center.X, dirtyRect.Center.Y);
float radius = Math.Min(dirtyRect.Width, dirtyRect.Height) / 4;
float distance = 0.8f * radius;

PointF center1 = new PointF(distance * (float)Math.Cos(9 * Math.PI / 6) +
    center.X,
    distance * (float)Math.Sin(9 * Math.PI / 6) + center.Y);
PointF center2 = new PointF(distance * (float)Math.Cos(1 * Math.PI / 6) +
    center.X,
    distance * (float)Math.Sin(1 * Math.PI / 6) + center.Y);
PointF center3 = new PointF(distance * (float)Math.Cos(5 * Math.PI / 6) +
    center.X,
    distance * (float)Math.Sin(5 * Math.PI / 6) + center.Y);

canvas.BlendMode = BlendMode.Multiply;
```

```
canvas.FillColor = Colors.Cyan;  
canvas.FillCircle(center1, radius);  
  
canvas.FillColor = Colors.Magenta;  
canvas.FillCircle(center2, radius);  
  
canvas.FillColor = Colors.Yellow;  
canvas.FillCircle(center3, radius);
```

The result is that a combination of any two colors produces red, green, and blue, and a combination of all three colors produces black.



Non-separable blend modes

The non-separable blend modes combine hue, saturation, and luminosity values from the destination and the source. Understanding these blend modes requires an understanding of the Hue-Saturation-Luminosity (HSL) color model:

- The hue value represents the dominant wavelength of the color. Hue values range from 0 to 360, and cycle through the additive and subtractive primary colors. Red is the value 0, yellow is 60, green is 120, cyan is 180, blue is 240, magenta is 300, and the cycle returns to red at 360. If there is no dominant color, for example the color is white or black or a gray shade, then the hue is undefined and usually set to 0.
- The saturation value indicates the purity of the color, and can range from 0 to 100. A saturation value of 100 is the purest color while values lower than 100 cause the color to become more grayish. A saturation value of 0 results in a shade of gray.
- The luminosity value indicates how bright the color is. A luminosity value of 0 is black regardless of other values. Similarly, a luminosity value of 100 is white.

The HSL value (0,100,50) is the RGB value (255,0,0), which is pure red. The HSL value (180,100,50) is the RGB value (0, 255, 255), which is pure cyan. As the saturation is decreased, the dominant color component is decreased and the other components are increased. At a saturation level of 0, all the components are the same and color is a gray shade.

The non-separable blend modes conceptually perform the following steps:

- 1. Convert the source and destination objects from their original color space to the HSL color space.
- 2. Create the composited object from a combination of hue, saturation, and luminosity components, from the source and destination objects.
- 3. Convert the result back to the original color space.

The following table lists how which HSL components are composited for each non-separable blend mode:

 Expand table

Blend mode	Source components	Destination components
Hue	Hue	Saturation and Luminosity
Saturation	Saturation	Hue and Luminosity
Color	Hue and Saturation	Luminosity
Luminosity	Luminosity	Hue and Saturation

Colors

Article • 09/30/2024

 [Browse the sample](#)

The [Color](#) class, in the [Microsoft.Maui.Graphics](#) namespace, lets you specify colors as Red-Green-Blue (RGB) values, Hue-Saturation-Luminosity (HSL) values, Hue-Saturation-Value (HSV) values, or with a color name. An Alpha channel is also available to indicate transparency.

[Color](#) objects can be created with [Color](#) constructors, which can be used to specify a gray shade, an RGB value, or an RGB value with transparency. Constructor overloads accept `float` values ranging from 0 to 1, `byte`, and `int` values.

ⓘ Note

The default [Color](#) constructor creates a black [Color](#) object.

You can also use the following static methods to create [Color](#) objects:

- `Color.FromRgb` from `float` RGB values that range from 0 to 1.
- `Color.FromRgb` from `double` RGB values that range from 0 to 1.
- `Color.FromRgb` from `byte` RGB values that range from 0 to 255.
- `Color.FromInt` from `int` RGB values that range from 0 to 255.
- `Color.FromRgba` from `float` RGBA values that range from 0 to 1.
- `Color.FromRgba` from `double` RGBA values that range from 0 to 1.
- `Color.FromRgba` from `byte` RGBA values that range from 0 to 255.
- `Color.FromRgba` from `int` RGBA values that range from 0 to 255.
- `Color.FromRgba` from a `string`-based hexadecimal value in the form "#RRGGBBAA" or "#RRGGBB" or "#RGBA" or "#RGB", where each letter corresponds to a hexadecimal digit for the alpha, red, green, and blue channels.
- `Color.FromHsla` from `float` HSLA values.
- `Color.FromHsla` from `double` HSLA values.
- `Color.FromHsv` from `float` HSV values that range from 0 to 1.
- `Color.FromHsv` from `int` HSV values that range from 0 to 255.
- `Color.FromHsva` from `float` HSVA values.
- `Color.FromHsva` from `int` HSV values.
- `Color.FromInt` from an `int` value calculated as $(B + 256 * (G + 256 * (R + 256 * A)))$.

- `Color.FromUint` from a `uint` value calculated as $(B + 256 * (G + 256 * (R + 256 * A)))$.
- `Color.FromArgb` from a `string`-based hexadecimal value in the form `"#AARRGGBB"` or `"#RRGGBB"` or `"#ARGB"` or `"RGB"`, where each letter corresponds to a hexadecimal digit for the alpha, red, green, and blue channels.

ⓘ Note

In addition to the methods listed above, the `Color` class also has `Parse` and `TryParse` methods that create `Color` objects from `string` arguments.

Once created, a `Color` object is immutable. The characteristics of the color can be obtained from the following `float` fields, that range from 0 to 1:

- `Red`, which represents the red channel of the color.
- `Green`, which represents the green channel of the color.
- `Blue`, which represents the blue channel of the color.
- `Alpha`, which represents the alpha channel of the color.

In addition, the characteristics of the color can be obtained from the following methods:

- `GetHue`, which returns a `float` that represents the hue channel of the color.
- `GetSaturation`, which returns a `float` that represents the saturation channel of the color.
- `GetLuminosity`, which returns a `float` that represents the luminosity channel of the color.

Named colors

The `Colors` class defines 148 public static read-only fields for common colors, such as `AntiqueWhite`, `MidnightBlue`, and `YellowGreen`.

Modify a color

The following instance methods modify an existing color to create a new color:

- `AddLuminosity` returns a `Color` by adding the luminosity value to the supplied delta value.
- `GetComplementary` returns the complementary `Color`.

- `MultiplyAlpha` returns a `Color` by multiplying the alpha value by the supplied `float` value.
- `WithAlpha` returns a `Color`, replacing the alpha value with the supplied `float` value.
- `WithHue` returns a `Color`, replacing the hue value with the supplied `float` value.
- `WithLuminosity` returns a `Color`, replacing the luminosity value with the supplied `float` value.
- `WithSaturation` returns a `Color`, replacing the saturation value with the supplied `float` value.

Conversions

The following instance methods convert a `Color` to an alternative representation:

- `AsPaint` returns a `SolidPaint` object whose `Color` property is set to the color.
- `ToHex` returns a hexadecimal `string` representation of a `Color`.
- `ToArgbHex` returns an ARGB hexadecimal `string` representation of a `Color`.
- `ToRgbaHex` returns an RGBA hexadecimal `string` representation of a `Color`.
- `ToInt` returns an ARGB `int` representation of a `Color`.
- `ToUInt` returns an ARGB `uint` representation of a `Color`.
- `ToRgb` converts a `Color` to RGB `byte` values that are returned as `out` arguments.
- `ToRgba` converts a `Color` to RGBA `byte` values that are returned as `out` arguments.
- `ToHsl` converts a `Color` to HSL `float` values that are passed as `out` arguments.

Examples

In XAML, colors are typically referenced using their named values, or with hexadecimal:

XAML

```
<Label Text="Sea color"
      TextColor="Aqua" />
<Label Text="RGB"
      TextColor="#00FF00" />
<Label Text="Alpha plus RGB"
      TextColor="#CC00FF00" />
<Label Text="Tiny RGB"
      TextColor="#0F0" />
<Label Text="Tiny Alpha plus RGB"
      TextColor="#C0F0" />
```

In C#, colors are typically referenced using their named values, or with their static methods:

C#

```
Label red    = new Label { Text = "Red",    TextColor = Colors.Red };
Label orange = new Label { Text = "Orange", TextColor =
Color.FromHex("FF6A00") };
Label yellow = new Label { Text = "Yellow", TextColor =
Color.FromHsla(0.167, 1.0, 0.5, 1.0) };
Label green  = new Label { Text = "Green",  TextColor = Color.FromRgb (38,
127, 0) };
Label blue   = new Label { Text = "Blue",   TextColor = Color.FromRgba(0,
38, 255, 255) };
Label indigo = new Label { Text = "Indigo", TextColor = Color.FromRgb (0,
72, 255) };
Label violet = new Label { Text = "Violet", TextColor = Color.FromHsla(0.82,
1, 0.25, 1) };
```

The following example uses the `OnPlatform` markup extension to selectively set the color of an [ActivityIndicator](#):

XAML

```
<ActivityIndicator Color="{OnPlatform AliceBlue, iOS=MidnightBlue}"
    IsRunning="True" />
```

The equivalent C# code is:

C#

```
ActivityIndicator activityIndicator = new ActivityIndicator
{
    Color = DeviceInfo.Platform == DevicePlatform.iOS ? Colors.MidnightBlue
    : Colors.AliceBlue,
    IsRunning = true
};
```

Draw graphical objects

Article • 09/30/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) graphics, in the [Microsoft.Maui.Graphics](#) namespace, enables you to draw graphical objects on a canvas that's defined as an [ICanvas](#) object.

The .NET MAUI [GraphicsView](#) control provides access to an [ICanvas](#) object, on which properties can be set and methods invoked to draw graphical objects. For more information about the [GraphicsView](#), see [GraphicsView](#).

ⓘ Note

Many of the graphical objects have `Draw` and `Fill` methods, for example [DrawRectangle](#) and [FillRectangle](#). A `Draw` method draws the outline of the shape, which is unfilled. A `Fill` method draws the outline of the shape and also fills it.

Graphical objects are drawn on an [ICanvas](#) using a device-independent unit that's recognized by each platform. This ensures that graphical objects are scaled appropriately to the pixel density of the underlying platform.

Draw a line

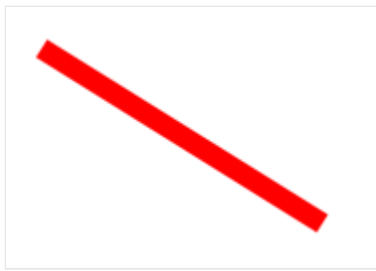
Lines can be drawn on an [ICanvas](#) using the [DrawLine](#) method, which requires four `float` arguments that represent the start and end points of the line.

The following example shows how to draw a line:

C#

```
canvas.StrokeColor = Colors.Red;  
canvas.StrokeSize = 6;  
canvas.DrawLine(10, 10, 90, 100);
```

In this example, a red diagonal line is drawn from (10,10) to (90,100):



ⓘ Note

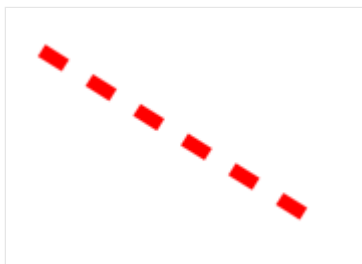
There's also a [DrawLine](#) overload that takes two [PointF](#) arguments.

The following example shows how to draw a dashed line:

C#

```
canvas.StrokeColor = Colors.Red;  
canvas.StrokeSize = 4;  
canvas.StrokeDashPattern = new float[] { 2, 2 };  
canvas.DrawLine(10, 10, 90, 100);
```

In this example, a red dashed diagonal line is drawn from (10,10) to (90,100):



For more information about dashed lines, see [Draw dashed objects](#).

Draw an ellipse

Ellipses and circles can be drawn on an [ICanvas](#) using the [DrawEllipse](#) method, which requires `x`, `y`, `width`, and `height` arguments, of type `float`.

The following example shows how to draw an ellipse:

C#

```
canvas.StrokeColor = Colors.Red;  
canvas.StrokeSize = 4;  
canvas.DrawEllipse(10, 10, 100, 50);
```

In this example, a red ellipse with dimensions 100x50 is drawn at (10,10):

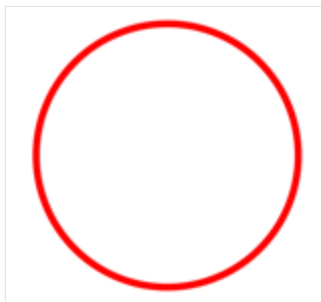


To draw a circle, make the `width` and `height` arguments to the [DrawEllipse](#) method equal:

C#

```
canvas.StrokeColor = Colors.Red;  
canvas.StrokeSize = 4;  
canvas.DrawEllipse(10, 10, 100, 100);
```

In this example, a red circle with dimensions 100x100 is drawn at (10,10):



ⓘ Note

Circles can also be drawn with the [DrawCircle](#) method.

For information about drawing a dashed ellipse, see [Draw dashed objects](#).

A filled ellipse can be drawn with the [FillEllipse](#) method, which also requires `x`, `y`, `width`, and `height` arguments, of type `float`:

C#

```
canvas.FillColor = Colors.Red;  
canvas.FillEllipse(10, 10, 150, 50);
```

In this example, a red filled ellipse with dimensions 150x50 is drawn at (10,10):



The [FillColor](#) property of the [ICanvas](#) object must be set to a [Color](#) before invoking the [FillEllipse](#) method.

Filled circles can also be drawn with the [FillCircle](#) method.

ⓘ Note

There are [DrawEllipse](#) and [FillEllipse](#) overloads that take [Rect](#) and [RectF](#) arguments. In addition, there are also [DrawCircle](#) and [FillCircle](#) overloads.

Draw a rectangle

Rectangles and squares can be drawn on an [ICanvas](#) using the [DrawRectangle](#) method, which requires `x`, `y`, `width`, and `height` arguments, of type `float`.

The following example shows how to draw a rectangle:

C#

```
canvas.StrokeColor = Colors.DarkBlue;  
canvas.StrokeSize = 4;  
canvas.DrawRectangle(10, 10, 100, 50);
```

In this example, a dark blue rectangle with dimensions 100x50 is drawn at (10,10):

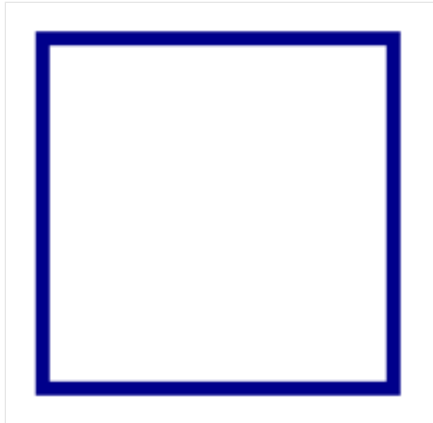


To draw a square, make the `width` and `height` arguments to the [DrawRectangle](#) method equal:

C#

```
canvas.StrokeColor = Colors.DarkBlue;  
canvas.StrokeSize = 4;  
canvas.DrawRectangle(10, 10, 100, 100);
```

In this example, a dark blue square with dimensions 100x100 is drawn at (10,10):



For information about drawing a dashed rectangle, see [Draw dashed objects](#).

A filled rectangle can be drawn with the [FillRectangle](#) method, which also requires `x`, `y`, `width`, and `height` arguments, of type `float`:

C#

```
canvas.FillColor = Colors.DarkBlue;  
canvas.FillRectangle(10, 10, 100, 50);
```

In this example, a dark blue filled rectangle with dimensions 100x50 is drawn at (10,10):



The [FillColor](#) property of the [ICanvas](#) object must be set to a [Color](#) before invoking the [FillRectangle](#) method.

ⓘ Note

There are [DrawRectangle](#) and [FillRectangle](#) overloads that take [Rect](#) and [RectF](#) arguments.

Draw a rounded rectangle

Rounded rectangles and squares can be drawn on an [ICanvas](#) using the [DrawRoundedRectangle](#) method, which requires `x`, `y`, `width`, `height`, and `cornerRadius` arguments, of type `float`. The `cornerRadius` argument specifies the radius used to round the corners of the rectangle.

The following example shows how to draw a rounded rectangle:

C#

```
canvas.StrokeColor = Colors.Green;  
canvas.StrokeSize = 4;  
canvas.DrawRoundedRectangle(10, 10, 100, 50, 12);
```

In this example, a green rectangle with rounded corners and dimensions 100x50 is drawn at (10,10):



For information about drawing a dashed rounded rectangle, see [Draw dashed objects](#).

A filled rounded rectangle can be drawn with the [FillRoundedRectangle](#) method, which also requires `x`, `y`, `width`, `height`, and `cornerRadius` arguments, of type `float`:

C#

```
canvas.FillColor = Colors.Green;  
canvas.FillRoundedRectangle(10, 10, 100, 50, 12);
```

In this example, a green filled rectangle with rounded corners and dimensions 100x50 is drawn at (10,10):



The [FillColor](#) property of the [ICanvas](#) object must be set to a [Color](#) before invoking the [FillRoundedRectangle](#) method.

ⓘ Note

There are [DrawRoundedRectangle](#) and [FillRoundedRectangle](#) overloads that take [Rect](#) and [RectF](#) arguments, and overloads that enable the radius of each corner to be separately specified.

Draw an arc

Arcs can be drawn on an [ICanvas](#) using the [DrawArc](#) method, which requires `x`, `y`, `width`, `height`, `startAngle`, and `endAngle` arguments of type `float`, and `clockwise` and `closed` arguments of type `bool`. The `startAngle` argument specifies the angle from the x-axis to the starting point of the arc. The `endAngle` argument specifies the angle from the x-axis to the end point of the arc. The `clockwise` argument specifies the direction in which the arc is drawn, and the `closed` argument specifies whether the end point of the arc will be connected to the start point.

The following example shows how to draw an arc:

C#

```
canvas.StrokeColor = Colors.Teal;  
canvas.StrokeSize = 4;  
canvas.DrawArc(10, 10, 100, 100, 0, 180, true, false);
```

In this example, a teal arc of dimensions 100x100 is drawn at (10,10). The arc is drawn in a clockwise direction from 0 degrees to 180 degrees, and isn't closed:



For information about drawing a dashed arc, see [Draw dashed objects](#).

A filled arc can be drawn with the [FillArc](#) method, which requires `x`, `y`, `width`, `height`, `startAngle`, and `endAngle` arguments of type `float`, and a `clockwise` argument of type `bool`:

C#

```
canvas.FillColor = Colors.Teal;  
canvas.FillArc(10, 10, 100, 100, 0, 180, true);
```

In this example, a filled teal arc of dimensions 100x100 is drawn at (10,10). The arc is drawn in a clockwise direction from 0 degrees to 180 degrees, and is closed automatically:



The [FillColor](#) property of the [ICanvas](#) object must be set to a [Color](#) before invoking the [FillArc](#) method.

ⓘ Note

There are [DrawArc](#) and [FillArc](#) overloads that take [Rect](#) and [RectF](#) arguments.

Draw a path

A path is a collection of one or more *contours*. Each contour is a collection of *connected* straight lines and curves. Contours are not connected to each other but they might visually overlap. Sometimes a single contour can overlap itself.

Paths are used to draw curves and complex shapes and can be drawn on an [ICanvas](#) using the [DrawPath](#) method, which requires a [PathF](#) argument.

A contour generally begins with a call to the [PathF.MoveTo](#) method, which you can express either as a [PointF](#) value or as separate `x` and `y` coordinates. The [MoveTo](#) call establishes a point at the beginning of the contour and an initial current point. You can then call the following methods to continue the contour with a line or curve from the current point to a point specified in the method, which then becomes the new current point:

- [LineTo](#) to add a straight line to the path.
- [AddArc](#) to add an arc, which is a line on the circumference of a circle or ellipse.
- [CurveTo](#) to add a cubic Bezier spline.
- [QuadTo](#) to add a quadratic Bezier spline.

None of these methods contain all of the data necessary to describe the line or curve. Instead, each method works with the current point established by the method call immediately preceding it. For example, the [LineTo](#) method adds a straight line to the contour based on the current point.

A contour ends with another call to [MoveTo](#), which begins a new contour, or a call to [Close](#), which closes the contour. The [Close](#) method automatically appends a straight line from the current point to the first point of the contour, and marks the path as closed.

The [PathF](#) class also defines other methods and properties. The following methods add entire contours to the path:

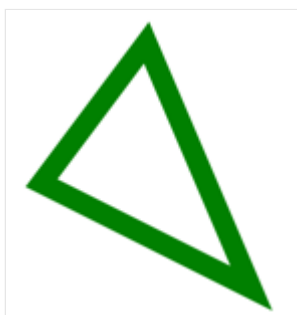
- [AppendEllipse](#) appends a closed ellipse contour to the path.
- [AppendCircle](#) appends a closed circle contour to the path.
- [AppendRectangle](#) appends a closed rectangle contour to the path.
- [AppendRoundedRectangle](#) appends a closed rectangle with rounded corners to the path.

The following example shows how to draw a path:

C#

```
PathF path = new PathF();  
path.MoveTo(40, 10);  
path.LineTo(70, 80);  
path.LineTo(10, 50);  
path.Close();  
canvas.StrokeColor = Colors.Green;  
canvas.StrokeSize = 6;  
canvas.DrawPath(path);
```

In this example, a closed green triangle is drawn:



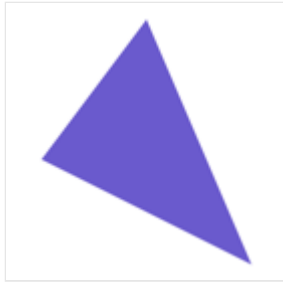
A filled path can be drawn with the [FillPath](#), which also requires a [PathF](#) argument:

C#

```
PathF path = new PathF();  
path.MoveTo(40, 10);
```

```
path.LineTo(70, 80);
path.LineTo(10, 50);
canvas.FillColor = Colors.SlateBlue;
canvas.FillPath(path);
```

In this example, a filled slate blue triangle is drawn:



The [FillColor](#) property of the [ICanvas](#) object must be set to a [Color](#) before invoking the [FillPath](#) method.

📌 Important

The [FillPath](#) method has an overload that enables a [WindingMode](#) to be specified, which sets the fill algorithm that's used. For more information, see [Winding modes](#).

Draw an image

Images can be drawn on an [ICanvas](#) using the [DrawImage](#) method, which requires an [Image](#) argument, and `x`, `y`, `width`, and `height` arguments, of type `float`.

The following example shows how to load an image and draw it to the canvas:

C#

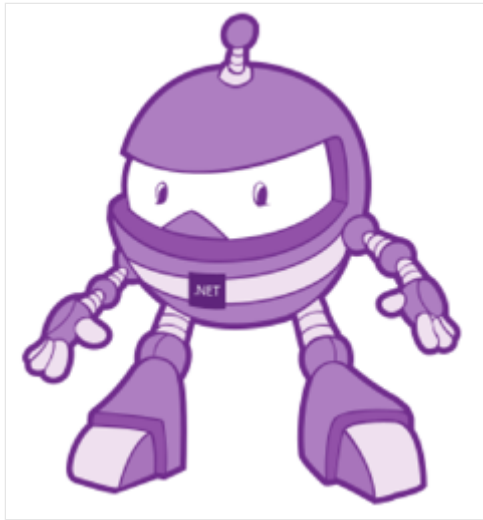
```
using System.Reflection;
using IImage = Microsoft.Maui.Graphics.IImage;
using Microsoft.Maui.Graphics.Platform;

IImage image;
Assembly assembly = GetType().GetTypeInfo().Assembly;
using (Stream stream =
assembly.GetManifestResourceStream("GraphicsViewDemos.Resources.Images.dotnet_bot.png"))
{
    image = PlatformImage.FromStream(stream);
}

if (image != null)
{
```

```
canvas.DrawImage(image, 10, 10, image.Width, image.Height);  
}
```

In this example, an image is retrieved from the assembly and loaded as a stream. It's then drawn at actual size at (10,10):



Important

Loading an image that's embedded in an assembly requires the image to have its build action set to **Embedded Resource** rather than **MauiImage**.

Draw a string

Strings can be drawn on an [ICanvas](#) using one of the [DrawString](#) overloads. The appearance of each string can be defined by setting the [Font](#), [FontColor](#), and [FontSize](#) properties. String alignment can be specified by horizontal and vertical alignment options that perform alignment within the string's bounding box.

Note

The bounding box for a string is defined by its `x`, `y`, `width`, and `height` arguments.

The following examples show how to draw strings:

C#

```
canvas.FontColor = Colors.Blue;  
canvas.FontSize = 18;  
  
canvas.Font = Font.Default;
```

```

canvas.DrawString("Text is left aligned.", 20, 20, 380, 100,
HorizontalAlignment.Left, VerticalAlignment.Top);
canvas.DrawString("Text is centered.", 20, 60, 380, 100,
HorizontalAlignment.Center, VerticalAlignment.Top);
canvas.DrawString("Text is right aligned.", 20, 100, 380, 100,
HorizontalAlignment.Right, VerticalAlignment.Top);

canvas.Font = Font.DefaultBold;
canvas.DrawString("This text is displayed using the bold system font.", 20,
140, 350, 100, HorizontalAlignment.Left, VerticalAlignment.Top);

canvas.Font = new Font("Arial");
canvas.FontColor = Colors.Black;
canvas.SetShadow(new SizeF(6, 6), 4, Colors.Gray);
canvas.DrawString("This text has a shadow.", 20, 200, 300, 100,
HorizontalAlignment.Left, VerticalAlignment.Top);

```

In this example, strings with different appearance and alignment options are displayed:

Text is left aligned.

Text is centered.

Text is right aligned.

This text is displayed using the bold system font.

THIS TEXT HAS A SHADOW.

ⓘ Note

The [DrawString](#) overloads also enable truncation and line spacing to be specified.

For information about drawing shadows, see [Draw a shadow](#).

Draw attributed text

Attributed text can be drawn on an [ICanvas](#) using the [DrawText](#) method, which requires an [IAttributedText](#) argument, and `x`, `y`, `width`, and `height` arguments, of type `float`. Attributed text is a string with associated attributes for parts of its text, that typically represents styling data.

The following example shows how to draw attributed text:

C#

```

using Microsoft.Maui.Graphics.Text;
...

canvas.Font = new Font("Arial");
canvas.FontSize = 18;
canvas.FontColor = Colors.Blue;

string markdownText = @"This is italic text, bold text, underline text, and bold italic text.";
IAttributedText attributedText =
    MarkdownAttributedTextReader.Read(markdownText); // Requires the
    Microsoft.Maui.Graphics.Text.Markdig package
canvas.DrawText(attributedText, 10, 10, 400, 400);

```

In this example, markdown is converted to attributed text and displayed with the correct styling:

THIS IS *ITALIC TEXT*, **BOLD TEXT**, UNDERLINE TEXT, AND ***BOLD ITALIC TEXT***.

Important

Drawing attributed text requires you to have added the `Microsoft.Maui.Graphics.Text.Markdig` NuGet package to your project.

Draw with fill and stroke

Graphical objects with both fill and stroke can be drawn to the canvas by calling a draw method *after* a fill method. For example, to draw an outlined rectangle, set the `FillColor` and `StrokeColor` properties to colors, then call the `FillRectangle` method followed by the `DrawRectangle` method.

The following example draws a filled circle, with a stroke outline, as a path:

```

C#

float radius = Math.Min(dirtyRect.Width, dirtyRect.Height) / 4;

PathF path = new PathF();
path.AppendCircle(dirtyRect.Center.X, dirtyRect.Center.Y, radius);

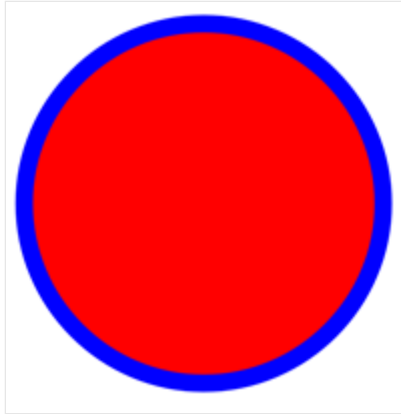
canvas.StrokeColor = Colors.Blue;
canvas.StrokeSize = 10;
canvas.FillColor = Colors.Red;

```



```
canvas.FillPath(path);  
canvas.DrawPath(path);
```

In this example, the stroke and fill colors for a [PathF](#) object are specified. The filled circle is drawn, then the outline stroke of the circle:



Warning

Calling a draw method before a fill method will result in an incorrect z-order. The fill will be drawn over the stroke, and the stroke won't be visible.

Draw a shadow

Graphical objects drawn on an [ICanvas](#) can have a shadow applied using the [SetShadow](#) method, which takes the following arguments:

- `offset`, of type [SizeF](#), specifies an offset for the shadow, which represents the position of a light source that creates the shadow.
- `blur`, of type `float`, represents the amount of blur to apply to the shadow.
- `color`, of type [Color](#), defines the color of the shadow.

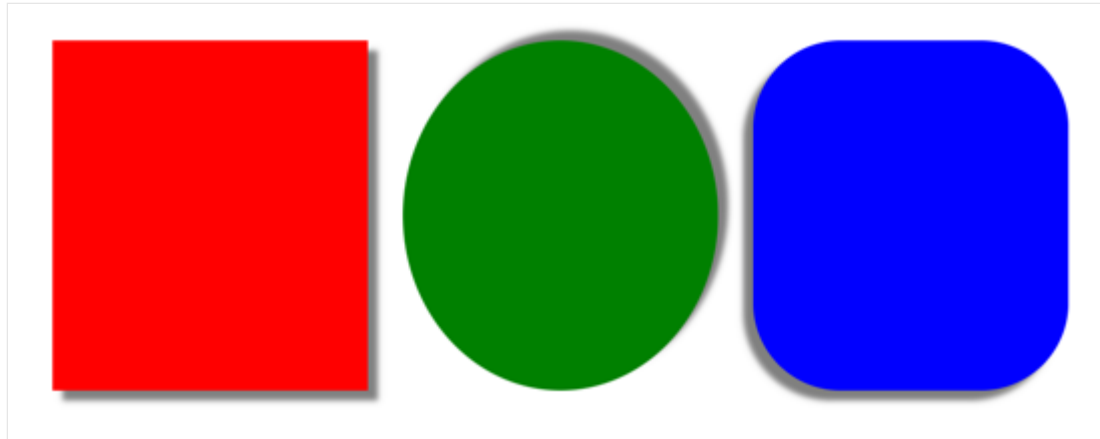
The following examples show how to add shadows to filled objects:

C#

```
canvas.FillColor = Colors.Red;  
canvas.SetShadow(new SizeF(10, 10), 4, Colors.Grey);  
canvas.FillRectangle(10, 10, 90, 100);  
  
canvas.FillColor = Colors.Green;  
canvas.SetShadow(new SizeF(10, -10), 4, Colors.Grey);  
canvas.FillEllipse(110, 10, 90, 100);  
  
canvas.FillColor = Colors.Blue;
```

```
canvas.SetShadow(new SizeF(-10, 10), 4, Colors.Grey);  
canvas.FillRoundedRectangle(210, 10, 90, 100, 25);
```

In these examples, shadows whose light sources are in different positions are added to the filled objects, with identical amounts of blur:



Draw dashed objects

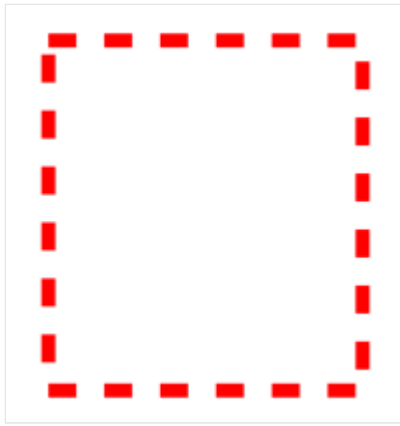
[ICanvas](#) objects have a [StrokeDashPattern](#) property, of type `float[]`. This property is an array of `float` values that indicate the pattern of dashes and gaps that are to be used when drawing the stroke for an object. Each `float` in the array specifies the length of a dash or gap. The first item in the array specifies the length of a dash, while the second item in the array specifies the length of a gap. Therefore, `float` values with an even index value specify dashes, while `float` values with an odd index value specify gaps.

The following example shows how to draw a dashed square, using a regular dash:

C#

```
canvas.StrokeColor = Colors.Red;  
canvas.StrokeSize = 4;  
canvas.StrokeDashPattern = new float[] { 2, 2 };  
canvas.DrawRectangle(10, 10, 90, 100);
```

In this example, a square with a regular dashed stroke is drawn:

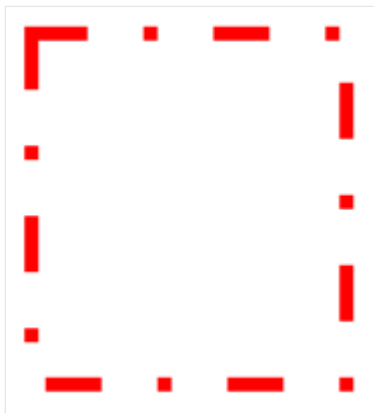


The following example shows how to draw a dashed square, using an irregular dash:

C#

```
canvas.StrokeColor = Colors.Red;  
canvas.StrokeSize = 4;  
canvas.StrokeDashPattern = new float[] { 4, 4, 1, 4 };  
canvas.DrawRectangle(10, 10, 90, 100);
```

In this example, a square with an irregular dashed stroke is drawn:



Control line ends

A line has three parts: start cap, line body, and end cap. The start and end caps describe the start and end of a line.

[Canvas](#) objects have a [StrokeLineCap](#) property, of type [LineCap](#), that describes the start and end of a line. The [LineCap](#) enumeration defines the following members:

- `Butt`, which represents a line with a square end, drawn to extend to the exact endpoint of the line. This is the default value of the [StrokeLineCap](#) property.
- `Round`, which represents a line with a rounded end.
- `Square`, which represents a line with a square end, drawn to extend beyond the endpoint to a distance equal to half the line width.

The following example shows how to set the [StrokeLineCap](#) property:

```
C#

canvas.StrokeSize = 10;
canvas.StrokeColor = Colors.Red;
canvas.StrokeLineCap = LineCap.Round;
canvas.DrawLine(10, 10, 110, 110);
```

In this example, the red line is rounded at the start and end of the line:



Control line joins

[Canvas](#) objects have a [StrokeLineJoin](#) property, of type [LineJoin](#), that specifies the type of join that is used at the vertices of an object. The [LineJoin](#) enumeration defines the following members:

- [Miter](#), which represents angular vertices that produce a sharp or clipped corner. This is the default value of the [StrokeLineJoin](#) property.
- [Round](#), which represents rounded vertices that produce a circular arc at the corner.
- [Bevel](#), which represents beveled vertices that produce a diagonal corner.

ⓘ Note

When the [StrokeLineJoin](#) property is set to [Miter](#), the [MiterLimit](#) property can be set to a [float](#) to limit the miter length of line joins in the object.

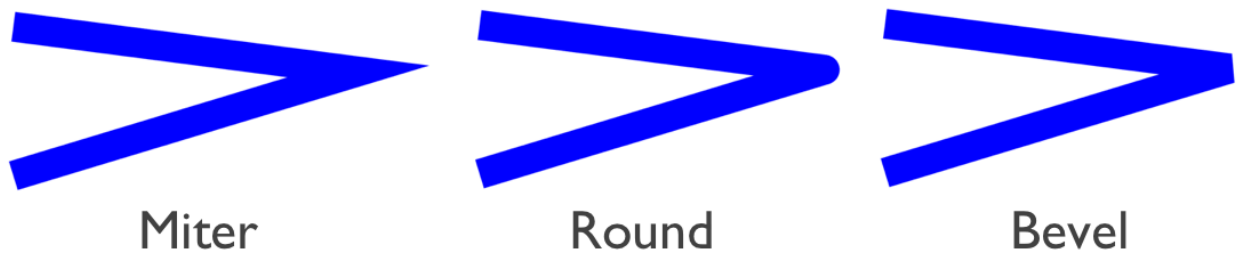
The following example shows how to set the [StrokeLineJoin](#) property:

```
C#

PathF path = new PathF();
path.MoveTo(10, 10);
path.LineTo(110, 50);
path.LineTo(10, 110);
```

```
canvas.StrokeSize = 20;  
canvas.StrokeColor = Colors.Blue;  
canvas.StrokeLineJoin = LineJoin.Round;  
canvas.DrawPath(path);
```

In this example, the blue [PathF](#) object has rounded joins at its vertices:



Clip objects

Graphical objects that are drawn to an [ICanvas](#) can be clipped prior to drawing, with the following methods:

- [ClipPath](#) clips an object so that only the area that's within the region of a [PathF](#) object will be visible.
- [ClipRectangle](#) clips an object so that only the area that's within the region of a rectangle will be visible. The rectangle can be specified using `float` arguments, or by a [Rect](#) or [RectF](#) argument.
- [SubtractFromClip](#) clips an object so that only the area that's outside the region of a rectangle will be visible. The rectangle can be specified using `float` arguments, or by a [Rect](#) or [RectF](#) argument.

The following example shows how to use the [ClipPath](#) method to clip an image:

C#

```
using System.Reflection;  
using IImage = Microsoft.Maui.Graphics.IImage;  
using Microsoft.Maui.Graphics.Platform;  
  
IImage image;  
Assembly assembly = GetType().GetTypeInfo().Assembly;  
using (Stream stream =  
assembly.GetManifestResourceStream("GraphicsViewDemos.Resources.Images.dotnet_bot.png"))  
{  
    image = PlatformImage.FromStream(stream);  
}  
  
if (image != null)
```

```
{
    PathF path = new PathF();
    path.AppendCircle(100, 90, 80);
    canvas.ClipPath(path); // Must be called before DrawImage
    canvas.DrawImage(image, 10, 10, image.Width, image.Height);
}
```

In this example, the image is clipped using a [PathF](#) object that defines a circle that's centered at (100,90) with a radius of 80. The result is that only the part of the image within the circle is visible:



❗ Important

The [ClipPath](#) method has an overload that enables a [WindingMode](#) to be specified, which sets the fill algorithm that's used when clipping. For more information, see [Winding modes](#).

The following example shows how to use the [SubtractFromClip](#) method to clip an image:

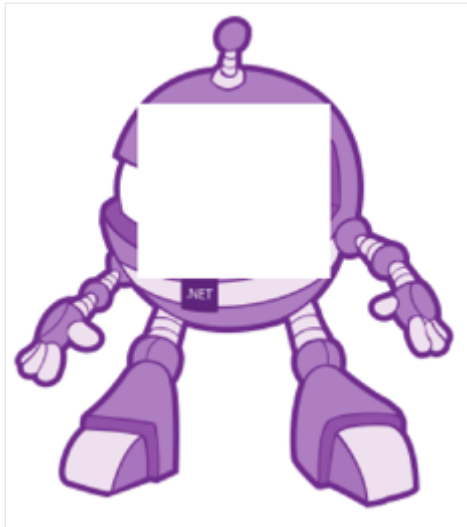
C#

```
using System.Reflection;
using IImage = Microsoft.Maui.Graphics.IImage;
using Microsoft.Maui.Graphics.Platform;

IImage image;
Assembly assembly = GetType().GetTypeInfo().Assembly;
using (Stream stream =
    assembly.GetManifestResourceStream("GraphicsViewDemos.Resources.Images.dotnet_bot.png"))
{
    image = PlatformImage.FromStream(stream);
}
```

```
if (image != null)
{
    canvas.SubtractFromClip(60, 60, 90, 90);
    canvas.DrawImage(image, 10, 10, image.Width, image.Height);
}
```

In this example, the area defined by the rectangle that's specified by the arguments supplied to the [SubtractFromClip](#) method is clipped from the image. The result is that only the parts of the image outside the rectangle are visible:



Images

Article • 09/30/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) graphics includes functionality to load, save, resize, and downsize images. Supported image formats are dependent on the underlying platform.

Images are represented by the [IImage](#) type, which defines the following properties:

- [Width](#), of type `float`, that defines the width of an image.
- [Height](#), of type `float`, that defines the height of an image.

An optional [ImageFormat](#) argument can be specified when loading and saving images. The [ImageFormat](#) enumeration defines `Png`, `Jpeg`, `Gif`, `Tiff`, and `Bmp` members. However, this argument is only used when the image format is supported by the underlying platform.

❗ Note

.NET MAUI contains two different `IImage` interfaces.

`Microsoft.Maui.Graphics.IImage` is used for image display, manipulation, and persistence in when displaying graphics in a [GraphicsView](#). `Microsoft.Maui.IImage` is the interface that abstracts the [Image](#) control.

Load an image

Image loading functionality is provided by the [PlatformImage](#) class. You can load images from a stream by the [FromStream](#) method, or from a byte array using the [PlatformImage](#) constructor.

The following example shows how to load an image:

C#

```
using Microsoft.Maui.Graphics.Platform;
using System.Reflection;
using IImage = Microsoft.Maui.Graphics.IImage;

IImage image;
Assembly assembly = GetType().GetTypeInfo().Assembly;
```



```
using (Stream stream =  
assembly.GetManifestResourceStream("GraphicsViewDemos.Resources.Images.dotnet_bot.png"))  
{  
    image = PlatformImage.FromStream(stream);  
}  
  
if (image != null)  
{  
    canvas.DrawImage(image, 10, 10, image.Width, image.Height);  
}
```

In this example, the image is retrieved from the assembly, loaded as a stream, and displayed.

Important

Loading an image that's embedded in an assembly requires the image to have its build action set to **Embedded Resource** rather than **MauiImage**.

Resize an image

Images can be resized using the [Resize](#) method, which requires `width` and `height` arguments, of type `float`, which represent the target dimensions of the image. The [Resize](#) method also accepts two optional arguments:

- A [ResizeMode](#) argument that controls how the image is resized to fit its target dimensions.
- A `bool` argument that controls whether the source image will be disposed after performing the resize operation. This argument defaults to `false`, indicating that the source image isn't disposed.

The [ResizeMode](#) enumeration defines the following members, which specify how to resize the image to the target size:

- `Fit`, which letterboxes the image so that it fits its target size.
- `Bleed`, which clips the image so that it fits its target size, while preserving its aspect ratio.
- `Stretch`, which stretches the image so it fills the available space. This can result in a change in the image aspect ratio.

The following example shows how to resize an image:

C#

```
using Microsoft.Maui.Graphics.Platform;
using System.Reflection;
using IImage = Microsoft.Maui.Graphics.IImage;

IImage image;
Assembly assembly = GetType().GetTypeInfo().Assembly;
using (Stream stream =
assembly.GetManifestResourceStream("GraphicsViewDemos.Resources.Images.dotnet_bot.png"))
{
    image = PlatformImage.FromStream(stream);
}

if (image != null)
{
    IImage newImage = image.Resize(100, 60, ResizeMode.Stretch, true);
    canvas.DrawImage(newImage, 10, 10, newImage.Width, newImage.Height);
}
```

In this example, the image is retrieved from the assembly and loaded as a stream. The image is resized using the [Resize](#) method, with its arguments specifying the new size, and that it should be stretched to fill the available space. In addition, the source image is disposed. The resized image is then drawn at actual size at (10,10).

Downsize an image

You can downsize images using one of the [Downsize](#) overloads. The first overload requires a single `float` value that represents the maximum width or height of the image, and downsizes the image while maintaining its aspect ratio. The second overload requires two `float` arguments that represent the maximum width and maximum height of the image.

The [Downsize](#) overloads also accept an optional `bool` argument that controls whether the source image should be disposed after performing the downsizing operation. This argument defaults to `false`, indicating that the source image isn't be disposed.

The following example shows how to downsize an image:

C#

```
using Microsoft.Maui.Graphics.Platform;
using System.Reflection;
using IImage = Microsoft.Maui.Graphics.IImage;

IImage image;
```

```

Assembly assembly = GetType().GetTypeInfo().Assembly;
using (Stream stream =
assembly.GetManifestResourceStream("GraphicsViewDemos.Resources.Images.dotnet_bot.png"))
{
    image = PlatformImage.FromStream(stream);
}

if (image != null)
{
    IImage newImage = image.Downsized(100, true);
    canvas.DrawImage(newImage, 10, 10, newImage.Width, newImage.Height);
}

```

In this example, the image is retrieved from the assembly and loaded as a stream. The image is downsized using the [Downsized](#) method, with the argument specifying that its largest dimension should be set to 100 pixels. In addition, the source image is disposed. The downsized image is then drawn at actual size at (10,10).

Save an image

You can save images using the [Save](#) and [SaveAsync](#) methods. Each method saves the [IImage](#) to a [Stream](#), and enables optional [ImageFormat](#) and quality values to be specified.

ⓘ Note

The [Save](#) and [SaveAsync](#) methods on Android and iOS can save images to JPEG and PNG format.

The following example shows how to save an image:

C#

```

using Microsoft.Maui.Graphics.Platform;
using System.Reflection;
using IImage = Microsoft.Maui.Graphics.IImage;

IImage image;
Assembly assembly = GetType().GetTypeInfo().Assembly;
using (Stream stream =
assembly.GetManifestResourceStream("GraphicsViewDemos.Resources.Images.dotnet_bot.png"))
{
    image = PlatformImage.FromStream(stream);
}

```

```
// Save image to a memory stream
if (image != null)
{
    IImage newImage = image.Downsized(150, true);
    using (MemoryStream memStream = new MemoryStream())
    {
        newImage.Save(memStream);
        // Reset destination stream position to 0 if saving to a file
    }
}
```

In this example, the image is retrieved from the assembly and loaded as a stream. The image is downsized using the [Downsized](#) method, with the argument specifying that its largest dimension should be set to 150 pixels. In addition, the source image is disposed. The downsized image is then saved to a stream.

Important

The [Save](#) method doesn't reset the stream position to 0. Therefore, if you want to save the stream to a file you should use the [Seek](#) method to reset the destination stream position to 0 before copying it to a file.

Paint graphical objects

Article • 09/30/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) graphics includes the ability to paint graphical objects with solid colors, gradients, repeating images, and patterns.

The [Paint](#) class is an abstract class that paints an object with its output. Classes that derive from [Paint](#) describe different ways of painting an object. The following list describes the different paint types available in .NET MAUI graphics:

- [SolidPaint](#), which paints an object with a solid color. For more information, see [Paint a solid color](#).
- [ImagePaint](#), which paints an object with an image. For more information, see [Paint an image](#).
- [PatternPaint](#), which paints an object with a pattern. For more information, see [Paint a pattern](#).
- [GradientPaint](#), which paints an object with a gradient. For more information, see [Paint a gradient](#).

Instances of these types can be painted on an [ICanvas](#), typically by using the [SetFillPaint](#) method to set the paint as the fill of a graphical object.

The [Paint](#) class also defines [BackgroundColor](#), and [ForegroundColor](#) properties, of type [Color](#), that can be used to optionally define background and foreground colors for a [Paint](#) object.

Paint a solid color

The [SolidPaint](#) class, that's derived from the [Paint](#) class, is used to paint a graphical object with a solid color.

The [SolidPaint](#) class defines a [Color](#) property, of type [Color](#), which represents the color of the paint. The class also has an [IsTransparent](#) property that returns a `bool` that represents whether the color has an alpha value of less than 1.

Create a SolidPaint object

The color of a [SolidPaint](#) object is typically specified through its constructor, using a [Color](#) argument:

C#

```
SolidPaint solidPaint = new SolidPaint(Colors.Silver);

RectF solidRectangle = new RectF(100, 100, 200, 100);
canvas.SetFillPaint(solidPaint, solidRectangle);
canvas.SetShadow(new SizeF(10, 10), 10, Colors.Grey);
canvas.FillRoundedRectangle(solidRectangle, 12);
```

The [SolidPaint](#) object is specified as the first argument to the [SetFillPaint](#) method. Therefore, a filled rounded rectangle is painted with a silver [SolidPaint](#) object:



Alternatively, the color can be specified with the [Color](#) property:

C#

```
SolidPaint solidPaint = new SolidPaint
{
    Color = Colors.Silver
};

RectF solidRectangle = new RectF(100, 100, 200, 100);
canvas.SetFillPaint(solidPaint, solidRectangle);
canvas.SetShadow(new SizeF(10, 10), 10, Colors.Grey);
canvas.FillRoundedRectangle(solidRectangle, 12);
```

Paint an image

The [ImagePaint](#) class, that's derived from the [Paint](#) class, is used to paint a graphical object with an image.

The [ImagePaint](#) class defines an [Image](#) property, of type [Image](#), which represents the image to paint. The class also has an [IsTransparent](#) property that returns `false`.

Create an ImagePaint object

To paint an object with an image, load the image and assign it to the [Image](#) property of the [ImagePaint](#) object.

ⓘ Note

Loading an image that's embedded in an assembly requires the image to have its build action set to **Embedded Resource**.

The following example shows how to load an image and fill a rectangle with it:

C#

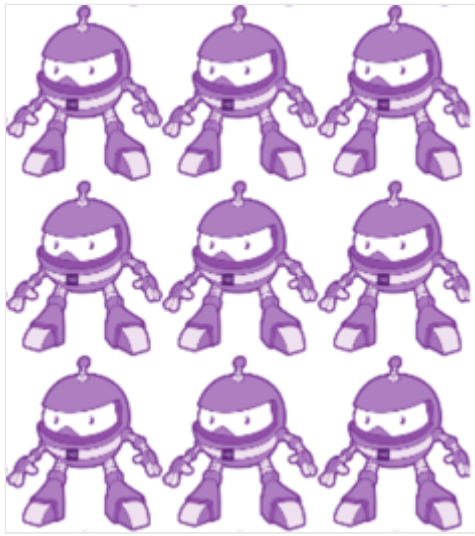
```
using System.Reflection;
using IImage = Microsoft.Maui.Graphics.IImage;
using Microsoft.Maui.Graphics.Platform;

IImage image;
Assembly assembly = GetType().GetTypeInfo().Assembly;
using (Stream stream =
assembly.GetManifestResourceStream("GraphicsViewDemos.Resources.Images.dotnet_bot.png"))
{
    image = PlatformImage.FromStream(stream);
}

if (image != null)
{
    ImagePaint imagePaint = new ImagePaint
    {
        Image = image.Downsized(100)
    };
    canvas.SetFillPaint(imagePaint, RectF.Zero);
    canvas.FillRectangle(0, 0, 240, 300);
}
```

In this example, the image is retrieved from the assembly and loaded as a stream. The image is resized using the [Downsize](#) method, with the argument specifying that its largest dimension should be set to 100 pixels. For more information about downsizing an image, see [Downsize an image](#).

The [Image](#) property of the [ImagePaint](#) object is set to the downsized version of the image, and the [ImagePaint](#) object is set as the paint to fill an object with. A rectangle is then drawn that's filled with the paint:



ⓘ Note

An [ImagePaint](#) object can also be created from an [IImage](#) object by the `AsPaint` extension method.

Alternatively, the [SetFillImage](#) extension method can be used to simplify the code:

```
C#  
  
if (image != null)  
{  
    canvas.SetFillImage(image.Downsized(100));  
    canvas.FillRectangle(0, 0, 240, 300);  
}
```

Paint a pattern

The [PatternPaint](#) class, that's derived from the [Paint](#) class, is used to paint a graphical object with a pattern.

The [PatternPaint](#) class defines a [Pattern](#) property, of type [IPattern](#), which represents the pattern to paint. The class also has an [IsTransparent](#) property that returns a `bool` that represents whether the background or foreground color of the paint has an alpha value of less than 1.

Create a PatternPaint object

To paint an area with a pattern, create the pattern and assign it to the [Pattern](#) property of a [PatternPaint](#) object.

The following example shows how to create a pattern and fill an object with it:

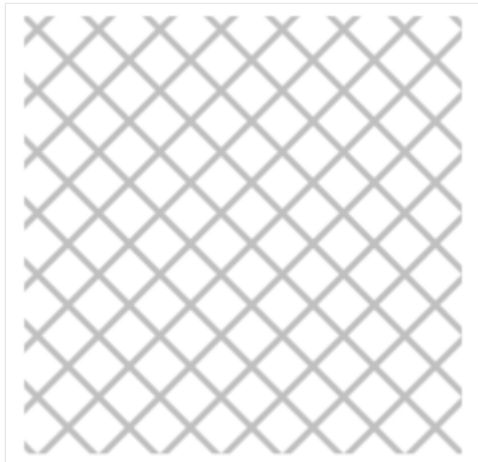
C#

```
IPattern pattern;

// Create a 10x10 template for the pattern
using (PictureCanvas picture = new PictureCanvas(0, 0, 10, 10))
{
    picture.StrokeColor = Colors.Silver;
    picture.DrawLine(0, 0, 10, 10);
    picture.DrawLine(0, 10, 10, 0);
    pattern = new PicturePattern(picture.Picture, 10, 10);
}

// Fill the rectangle with the 10x10 pattern
PatternPaint patternPaint = new PatternPaint
{
    Pattern = pattern
};
canvas.SetFillPaint(patternPaint, RectF.Zero);
canvas.FillRectangle(10, 10, 250, 250);
```

In this example, the pattern is a 10x10 area that contains a diagonal line from (0,0) to (10,10), and a diagonal line from (0,10) to (10,0). The [Pattern](#) property of the [PatternPaint](#) object is set to the pattern, and the [PatternPaint](#) object is set as the paint to fill an object with. A rectangle is then drawn that's filled with the paint:



ⓘ Note

A [PatternPaint](#) object can also be created from a [PicturePattern](#) object by the [AsPaint](#) extension method.

Paint a gradient

The [GradientPaint](#) class, that's derived from the [Paint](#) class, is an abstract base class that describes a gradient, which is composed of gradient steps. A [GradientPaint](#) paints a graphical object with multiple colors that blend into each other along an axis. Classes that derive from [GradientPaint](#) describe different ways of interpreting gradients stops, and .NET MAUI graphics provides the following gradient paints:

- [LinearGradientPaint](#), which paints an object with a linear gradient. For more information, see [Paint a linear gradient](#).
- [RadialGradientPaint](#), which paints an object with a radial gradient. For more information, see [Paint a radial gradient](#).

The [GradientPaint](#) class defines the [GradientStops](#) property, of type [PaintGradientStop](#), which represents the brush's gradient stops, each of which specifies a color and an offset along the gradient axis.

Gradient stops

Gradient stops are the building blocks of a gradient, and specify the colors in the gradient and their location along the gradient axis. Gradient stops are specified using [PaintGradientStop](#) objects.

The [PaintGradientStop](#) class defines the following properties:

- [Color](#), of type [Color](#), which represents the color of the gradient stop.
- [Offset](#), of type `float`, which represents the location of the gradient stop within the gradient vector. Valid values are in the range 0.0-1.0. The closer this value is to 0, the closer the color is to the start of the gradient. Similarly, the closer this value is to 1, the closer the color is to the end of the gradient.

Important

The coordinate system used by gradients is relative to a bounding box for the graphical object. 0 indicates 0 percent of the bounding box, and 1 indicates 100 percent of the bounding box. Therefore, (0.5,0.5) describes a point in the middle of the bounding box, and (1,1) describes a point at the bottom right of the bounding box.

Gradient stops can be added to a [GradientPaint](#) object with the [AddOffset](#) method.

The following example creates a diagonal [LinearGradientPaint](#) with four colors:

```
C#
```

```

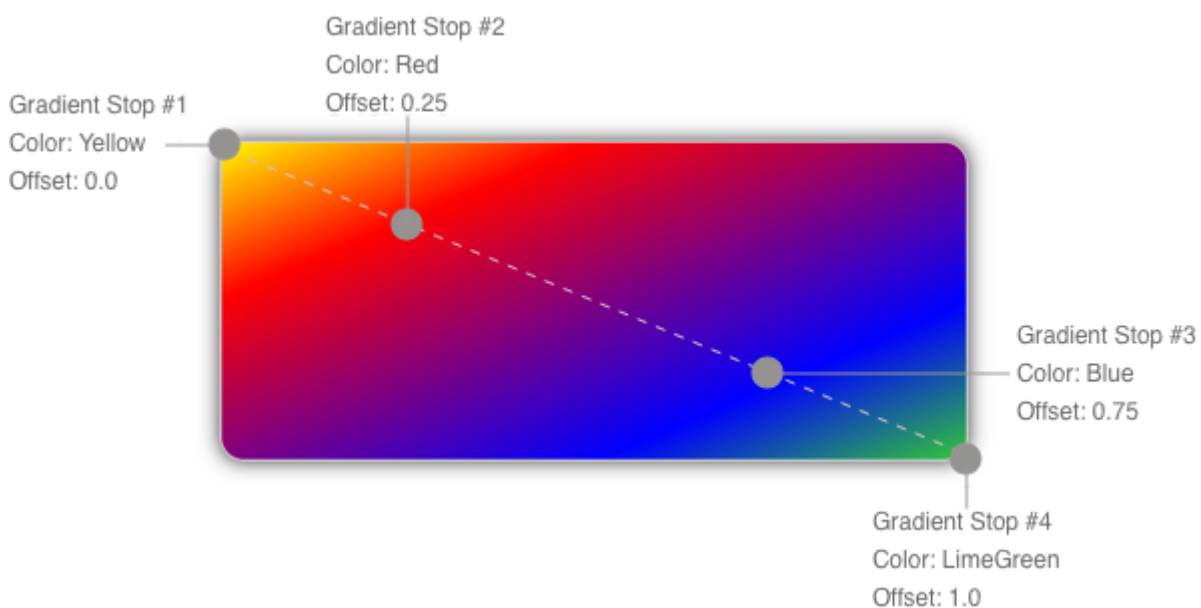
LinearGradientPaint linearGradientPaint = new LinearGradientPaint
{
    StartColor = Colors.Yellow,
    EndColor = Colors.Green,
    StartPoint = new Point(0, 0),
    EndPoint = new Point(1, 1)
};

linearGradientPaint.AddOffset(0.25f, Colors.Red);
linearGradientPaint.AddOffset(0.75f, Colors.Blue);

RectF linearRectangle = new RectF(10, 10, 200, 100);
canvas.SetFillPaint(linearGradientPaint, linearRectangle);
canvas.SetShadow(new SizeF(10, 10), 10, Colors.Grey);
canvas.FillRoundedRectangle(linearRectangle, 12);

```

The color of each point between gradient stops is interpolated as a combination of the color specified by the two bounding gradient stops. The following diagram shows the gradient stops from the previous example:



In this diagram, the circles mark the position of gradient stops, and the dashed line shows the gradient axis. The first gradient stop specifies the color yellow at an offset of 0.0. The second gradient stop specifies the color red at an offset of 0.25. The points between these two gradient stops gradually change from yellow to red as you move from left to right along the gradient axis. The third gradient stop specifies the color blue at an offset of 0.75. The points between the second and third gradient stops gradually change from red to blue. The fourth gradient stop specifies the color lime green at offset of 1.0. The points between the third and fourth gradient stops gradually change from blue to lime green.

Paint a linear gradient

The [LinearGradientPaint](#) class, that's derived from the [GradientPaint](#) class, paints a graphical object with a linear gradient. A linear gradient blends two or more colors along a line known as the gradient axis. [PaintGradientStop](#) objects are used to specify the colors in the gradient and their positions. For more information about [PaintGradientStop](#) objects, see [Paint a gradient](#).

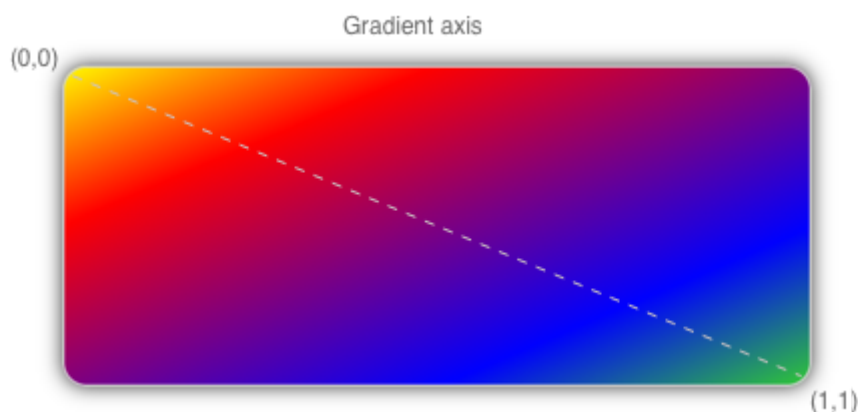
The [LinearGradientPaint](#) class defines the following properties:

- [StartPoint](#), of type [Point](#), which represents the starting two-dimensional coordinates of the linear gradient. The class constructor initializes this property to (0,0).
- [EndPoint](#), of type [Point](#), which represents the ending two-dimensional coordinates of the linear gradient. The class constructor initializes this property to (1,1).

Create a LinearGradientPaint object

A linear gradient's gradient stops are positioned along the gradient axis. The orientation and size of the gradient axis can be changed using the [StartPoint](#) and [EndPoint](#) properties. By manipulating these properties, you can create horizontal, vertical, and diagonal gradients, reverse the gradient direction, condense the gradient spread, and more.

The [StartPoint](#) and [EndPoint](#) properties are relative to the graphical object being painted. (0,0) represents the top-left corner of the object being painted, and (1,1) represents the bottom-right corner of the object being painted. The following diagram shows the gradient axis for a diagonal linear gradient brush:



In this diagram, the dashed line shows the gradient axis, which highlights the interpolation path of the gradient from the start point to the end point.

Create a horizontal linear gradient

To create a horizontal linear gradient, create a [LinearGradientPaint](#) object and set its [StartColor](#) and [EndColor](#) properties. Then, set its [EndPoint](#) to (1,0).

The following example shows how to create a horizontal [LinearGradientPaint](#):

C#

```
LinearGradientPaint linearGradientPaint = new LinearGradientPaint
{
    StartColor = Colors.Yellow,
    EndColor = Colors.Green,
    // StartPoint is already (0,0)
    EndPoint = new Point(1, 0)
};

RectF linearRectangle = new RectF(10, 10, 200, 100);
canvas.SetFillPaint(linearGradientPaint, linearRectangle);
canvas.SetShadow(new SizeF(10, 10), 10, Colors.Grey);
canvas.FillRoundedRectangle(linearRectangle, 12);
```

In this example, the rounded rectangle is painted with a linear gradient that interpolates horizontally from yellow to green:



Create a vertical linear gradient

To create a vertical linear gradient, create a [LinearGradientPaint](#) object and set its [StartColor](#) and [EndColor](#) properties. Then, set its [EndPoint](#) to (0,1).

The following example shows how to create a vertical [LinearGradientPaint](#):

C#

```
LinearGradientPaint linearGradientPaint = new LinearGradientPaint
{
    StartColor = Colors.Yellow,
```

```

        EndColor = Colors.Green,
        // StartPoint is already (0,0)
        EndPoint = new Point(0, 1)
    };

    RectF linearRectangle = new RectF(10, 10, 200, 100);
    canvas.SetFillPaint(linearGradientPaint, linearRectangle);
    canvas.SetShadow(new SizeF(10, 10), 10, Colors.Grey);
    canvas.FillRoundedRectangle(linearRectangle, 12);

```

In this example, the rounded rectangle is painted with a linear gradient that interpolates vertically from yellow to green:



Create a diagonal linear gradient

To create a diagonal linear gradient, create a [LinearGradientPaint](#) object and set its [StartColor](#) and [EndColor](#) properties.

The following example shows how to create a diagonal [LinearGradientPaint](#):

```

C#

LinearGradientPaint linearGradientPaint = new LinearGradientPaint
{
    StartColor = Colors.Yellow,
    EndColor = Colors.Green,
    // StartPoint is already (0,0)
    // EndPoint is already (1,1)
};

RectF linearRectangle = new RectF(10, 10, 200, 100);
canvas.SetFillPaint(linearGradientPaint, linearRectangle);
canvas.SetShadow(new SizeF(10, 10), 10, Colors.Grey);
canvas.FillRoundedRectangle(linearRectangle, 12);

```

In this example, the rounded rectangle is painted with a linear gradient that interpolates diagonally from yellow to green:



Paint a radial gradient

The [RadialGradientPaint](#) class, that's derived from the [GradientPaint](#) class, paints a graphical object with a radial gradient. A radial gradient blends two or more colors across a circle. [PaintGradientStop](#) objects are used to specify the colors in the gradient and their positions. For more information about [PaintGradientStop](#) objects, see [Paint a gradient](#).

The [RadialGradientPaint](#) class defines the following properties:

- [Center](#), of type [Point](#), which represents the center point of the circle for the radial gradient. The class constructor initializes this property to (0.5,0.5).
- [Radius](#), of type `double`, which represents the radius of the circle for the radial gradient. The class constructor initializes this property to 0.5.

Create a RadialGradientPaint object

A radial gradient's gradient stops are positioned along a gradient axis defined by a circle. The gradient axis radiates from the center of the circle to its circumference. The position and size of the circle can be changed using the [Center](#) and [Radius](#) properties. The circle defines the end point of the gradient. Therefore, a gradient stop at 1.0 defines the color at the circle's circumference. A gradient stop at 0.0 defines the color at the center of the circle.

To create a radial gradient, create a [RadialGradientPaint](#) object and set its [StartColor](#) and [EndColor](#) properties. Then, set its [Center](#) and [Radius](#) properties.

The following example shows how to create a centered [RadialGradientPaint](#):

```
C#
```

```
RadialGradientPaint radialGradientPaint = new RadialGradientPaint  
{
```

```

        StartColor = Colors.Red,
        EndColor = Colors.DarkBlue
        // Center is already (0.5,0.5)
        // Radius is already 0.5
    };

    RectF radialRectangle = new RectF(10, 10, 200, 100);
    canvas.SetFillPaint(radialGradientPaint, radialRectangle);
    canvas.SetShadow(new SizeF(10, 10), 10, Colors.Grey);
    canvas.FillRoundedRectangle(radialRectangle, 12);

```

In this example, the rounded rectangle is painted with a radial gradient that interpolates from red to dark blue. The center of the radial gradient is positioned in the center of the rectangle:



The following example moves the center of the radial gradient to the top-left corner of the rectangle:

```

C#

RadialGradientPaint radialGradientPaint = new RadialGradientPaint
{
    StartColor = Colors.Red,
    EndColor = Colors.DarkBlue,
    Center = new Point(0.0, 0.0)
    // Radius is already 0.5
};

RectF radialRectangle = new RectF(10, 10, 200, 100);
canvas.SetFillPaint(radialGradientPaint, radialRectangle);
canvas.SetShadow(new SizeF(10, 10), 10, Colors.Grey);
canvas.FillRoundedRectangle(radialRectangle, 12);

```

In this example, the rounded rectangle is painted with a radial gradient that interpolates from red to dark blue. The center of the radial gradient is positioned in the top-left of the rectangle:



The following example moves the center of the radial gradient to the bottom-right corner of the rectangle:

C#

```
RadialGradientPaint radialGradientPaint = new RadialGradientPaint
{
    StartColor = Colors.Red,
    EndColor = Colors.DarkBlue,
    Center = new Point(1.0, 1.0)
    // Radius is already 0.5
};

RectF radialRectangle = new RectF(10, 10, 200, 100);
canvas.SetFillPaint(radialGradientPaint, radialRectangle);
canvas.SetShadow(new SizeF(10, 10), 10, Colors.Grey);
canvas.FillRoundedRectangle(radialRectangle, 12);
```

In this example, the rounded rectangle is painted with a radial gradient that interpolates from red to dark blue. The center of the radial gradient is positioned in the bottom-right of the rectangle:



Transforms

Article • 09/30/2024



[Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) graphics supports traditional graphics transforms, which are implemented as methods on the [ICanvas](#) object. Mathematically, transforms alter the coordinates and sizes that you specify in [ICanvas](#) drawing methods, when the graphical objects are rendered.

The following transforms are supported:

- *Translate* to shift coordinates from one location to another.
- *Scale* to increase or decrease coordinates and sizes.
- *Rotate* to rotate coordinates around a point.
- *Skew* to shift coordinates horizontally or vertically.

These transforms are known as *affine* transforms. Affine transforms always preserve parallel lines and never cause a coordinate or size to become infinite.

The .NET MAUI [VisualElement](#) class also supports the following transform properties: [TranslationX](#), [TranslationY](#), [Scale](#), [Rotation](#), [RotationX](#), and [RotationY](#). However, there are several differences between [Microsoft.Maui.Graphics](#) transforms and [VisualElement](#) transforms:

- [Microsoft.Maui.Graphics](#) transforms are methods, while [VisualElement](#) transforms are properties. Therefore, [Microsoft.Maui.Graphics](#) transforms perform an operation while [VisualElement](#) transforms set a state. [Microsoft.Maui.Graphics](#) transforms apply to subsequently drawn graphics objects, but not to graphics objects that are drawn before the transform is applied. In contrast, a [VisualElement](#) transform applies to a previously rendered element as soon as the property is set. Therefore, [Microsoft.Maui.Graphics](#) transforms are cumulative as methods are called, while [VisualElement](#) transforms are replaced when the property is set with another value.
- [Microsoft.Maui.Graphics](#) transforms are applied to the [ICanvas](#) object, while [VisualElement](#) transforms are applied to a [VisualElement](#) derived object.
- [Microsoft.Maui.Graphics](#) transforms are relative to the upper-left corner of the [ICanvas](#), while [VisualElement](#) transforms are relative to the upper-left corner of the [VisualElement](#) to which they're applied.

Translate transform

The translate transform shifts graphical objects in the horizontal and vertical directions. Translation can be considered unnecessary because the same result can be accomplished by changing the coordinates of the drawing method you're using. However, when displaying a path, all the coordinates are encapsulated in the path, and so it's often easier to apply a translate transform to shift the entire path.

The [Translate](#) method requires `x` and `y` arguments of type `float` that cause subsequently drawn graphic objects to be shifted horizontally and vertically. Negative `x` values move an object to the left, while positive values move an object to the right. Negative `y` values move up an object, while positive values move down an object.

A common use of the translate transform is for rendering a graphical object that has been originally created using coordinates that are convenient for drawing. The following example creates a [PathF](#) object for an 11-pointed star:

C#

```
PathF path = new PathF();
for (int i = 0; i < 11; i++)
{
    double angle = 5 * i * 2 * Math.PI / 11;
    PointF point = new PointF(100 * (float)Math.Sin(angle), -100 *
(float)Math.Cos(angle));

    if (i == 0)
        path.MoveTo(point);
    else
        path.LineTo(point);
}

canvas.FillColor = Colors.Red;
canvas.Translate(150, 150);
canvas.FillPath(path);
```

The center of the star is at (0,0), and the points of the star are on a circle surrounding that point. Each point is a combination of sine and cosine values of an angle that increases by 5/11ths of 360 degrees. The radius of the circle is set as 100. If the [PathF](#) object is displayed without any transforms, the center of the star will be positioned at the upper-left corner of the [ICanvas](#), and only a quarter of it will be visible. Therefore, a translate transform is used to shift the star horizontally and vertically to (150,150):



Scale transform

The scale transform changes the size of a graphical object, and can also often cause coordinates to move when a graphical object is made larger.

The [Scale](#) method requires `x` and `y` arguments of type `float` that let you specify different values for horizontal and vertical scaling, otherwise known as *anisotropic* scaling. The values of `x` and `y` have a significant impact on the resulting scaling:

- Values between 0 and 1 decrease the width and height of the scaled object.
- Values greater than 1 increase the width and height of the scaled object.
- Values of 1 indicate that the object is not scaled.

The following example demonstrates the [Scale](#) method:

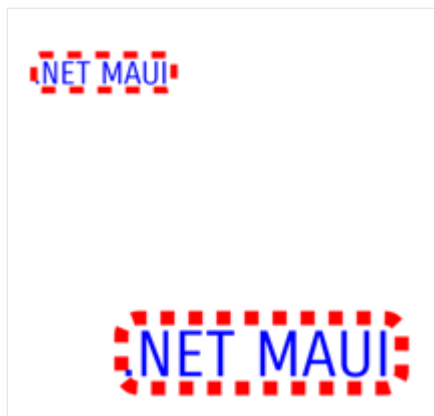
C#

```
canvas.StrokeColor = Colors.Red;
canvas.StrokeSize = 4;
canvas.StrokeDashPattern = new float[] { 2, 2 };
canvas.FontColor = Colors.Blue;
canvas.FontSize = 18;

canvas.DrawRoundedRectangle(50, 50, 80, 20, 5);
canvas.DrawString(".NET MAUI", 50, 50, 80, 20, HorizontalAlignment.Left,
VerticalAlignment.Top);

canvas.Scale(2, 2);
canvas.DrawRoundedRectangle(50, 100, 80, 20, 5);
canvas.DrawString(".NET MAUI", 50, 100, 80, 20, HorizontalAlignment.Left,
VerticalAlignment.Top);
```

In this example, ".NET MAUI" is displayed inside a rounded rectangle stroked with a dashed line. The same graphical objects drawn after the [Scale](#) call increase in size proportionally:



The text and the rounded rectangle are both subject to the same scaling factors.

⚠ Note

Anisotropic scaling causes the stroke size to become different for lines aligned with the horizontal and vertical axes.

Order matters when you combine [Translate](#) and [Scale](#) calls. If the [Translate](#) call comes after the [Scale](#) call, the translation factors are scaled by the scaling factors. If the [Translate](#) call comes before the [Scale](#) call, the translation factors aren't scaled.

Rotate transform

The rotate transform rotates a graphical object around a point. Rotation is clockwise for increasing angles. Negative angles and angles greater than 360 degrees are allowed.

There are two [Rotate](#) overloads. The first requires a `degrees` argument of type `float` that defines the rotation angle, and centers the rotation around the upper-left corner of the canvas (0,0). The following example demonstrates this [Rotate](#) method:

C#

```
canvas.FontColor = Colors.Blue;  
canvas.FontSize = 18;  
  
canvas.Rotate(45);  
canvas.DrawString(".NET MAUI", 50, 50, HorizontalAlignment.Left);
```

In this example, ".NET MAUI" is rotated 45 degrees clockwise:



Alternatively, graphical objects can be rotated centered around a specific point. To rotate around a specific point, use the [Rotate](#) overload that accepts `degrees`, `x`, and `y` arguments of type `float`:

C#

```
canvas.FontColor = Colors.Blue;
canvas.FontSize = 18;

canvas.Rotate(45, dirtyRect.Center.X, dirtyRect.Center.Y);
canvas.DrawString(".NET MAUI", dirtyRect.Center.X, dirtyRect.Center.Y,
HorizontalAlignment.Left);
```

In this example, the .NET MAUI text is rotated 45 degrees around the center of the canvas.

Combine transforms

The simplest way to combine transforms is to begin with global transforms, followed by local transforms. For example, translation, scaling, and rotation can be combined to draw an analog clock. The clock can be drawn using an arbitrary coordinate system based on a circle that's centered at (0,0) with a radius of 100. Translation and scaling expand and center the clock on the canvas, and rotation can then be used to draw the minute and hour marks of the clock and to rotate the hands:

C#

```
canvas.StrokeLineCap = LineCap.Round;
canvas.FillColor = Colors.Gray;

// Translation and scaling
canvas.Translate(dirtyRect.Center.X, dirtyRect.Center.Y);
float scale = Math.Min(dirtyRect.Width / 200f, dirtyRect.Height / 200f);
canvas.Scale(scale, scale);

// Hour and minute marks
for (int angle = 0; angle < 360; angle += 6)
{
    canvas.FillCircle(0, -90, angle % 30 == 0 ? 4 : 2);
```

```

        canvas.Rotate(6);
    }

    DateTime now = DateTime.Now;

    // Hour hand
    canvas.StrokeSize = 20;
    canvas.SaveState();
    canvas.Rotate(30 * now.Hour + now.Minute / 2f);
    canvas.DrawLine(0, 0, 0, -50);
    canvas.RestoreState();

    // Minute hand
    canvas.StrokeSize = 10;
    canvas.SaveState();
    canvas.Rotate(6 * now.Minute + now.Second / 10f);
    canvas.DrawLine(0, 0, 0, -70);
    canvas.RestoreState();

    // Second hand
    canvas.StrokeSize = 2;
    canvas.SaveState();
    canvas.Rotate(6 * now.Second);
    canvas.DrawLine(0, 10, 0, -80);
    canvas.RestoreState();

```

In this example, the [Translate](#) and [Scale](#) calls apply globally to the clock, and so are called before the [Rotate](#) method.

There are 60 marks of two different sizes that are drawn in a circle around the clock. The [FillCircle](#) call draws that circle at (0,-90), which relative to the center of the clock corresponds to 12:00. The [Rotate](#) call increments the rotation angle by 6 degrees after every tick mark. The `angle` variable is used solely to determine if a large circle or a small circle is drawn. Finally, the current time is obtained and rotation degrees are calculated for the hour, minute, and second hands. Each hand is drawn in the 12:00 position so that the rotation angle is relative to that position:



Concatenate transforms

A transform can be described in terms of a 3x3 affine transformation matrix, which performs transformations in 2D space. The following table shows the structure of a 3x3 affine transformation matrix:

M11

M12

0.0

M21

M22

0.0

M31

M32

1.0

An affine transformation matrix has its final column equal to (0,0,1), so only members in the first two columns need to be specified. Therefore, the 3x3 matrix is represented by the [Matrix3x2](#) struct, from the [System.Numerics](#) namespace, which is a collection of three rows and two columns of `float` values.

The six cells in the first two columns of the transform matrix represent values that performing scaling, shearing, and translation:

ScaleX

ShearY

0.0

ShearX

ScaleY

0.0

TranslateX

TranslateY

1.0

For example, if you change the `M31` value to 100, you can use it to translate a graphical object 100 pixels along the x-axis. If you change the `M22` value to 3, you can use it to stretch a graphical object to three times its current height. If you change both values, you move the graphical object 100 pixels along the x-axis and stretch its height by a factor of 3.

You can define a new transform matrix with the `Matrix3x2` constructor. The advantage of specifying transforms with a transform matrix is that composite transforms can be applied as a single transform, which is referred to as *concatenation*. The `Matrix3x2` struct also defines methods that can be used to manipulate matrix values.

The only `ICanvas` method that accepts a `Matrix3x2` argument is the `ConcatenateTransform` method, which combines multiple transforms into a single transform. The following example shows how to use this method to transform a `PathF` object:

C#

```
PathF path = new PathF();
for (int i = 0; i < 11; i++)
{
    double angle = 5 * i * 2 * Math.PI / 11;
    PointF point = new PointF(100 * (float)Math.Sin(angle), -100 *
(float)Math.Cos(angle));

    if (i == 0)
        path.MoveTo(point);
    else
        path.LineTo(point);
}

Matrix3x2 transform = new Matrix3x2(1.5f, 1, 0, 1, 150, 150);
canvas.ConcatenateTransform(transform);
canvas.FillColor = Colors.Red;
canvas.FillPath(path);
```

In this example, the `PathF` object is scaled and sheared on the x-axis, and translated on the x-axis and the y-axis.

Winding modes

Article • 09/30/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) graphics provides a [WindingMode](#) enumeration that enables you to specify the fill algorithm to be used by the [FillPath](#) method. Contours in a path can overlap, and any enclosed area can potentially be filled, but you might not want to fill all the enclosed areas. For more information about paths, see [Draw a path](#).

The [WindingMode](#) enumeration defines `NonZero` and `EvenOdd` members. Each member represents a different algorithm for determining whether a point is in the fill region of an enclosed area.

ⓘ Note

The [ClipPath](#) method has an overload that enables a [WindingMode](#) argument to be specified. By default, this argument is set to `WindingMode.NonZero`.

NonZero

The `NonZero` winding mode draws a hypothetical ray from the point to infinity in any direction and then examines the places where a path contour crosses the ray. The count starts at zero and is incremented each time a contour crosses the ray from left to right and decremented each time a contour crosses the ray from right to left. If the count of crossings is zero, the area isn't filled. Otherwise, the area is filled.

The following example fills a five-pointed star using the `NonZero` winding mode:

C#

```
float radius = 0.45f * Math.Min(dirtyRect.Width, dirtyRect.Height);

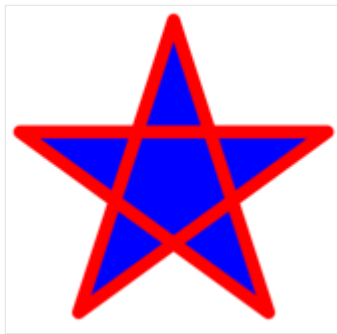
PathF path = new PathF();
path.MoveTo(dirtyRect.Center.X, dirtyRect.Center.Y - radius);

for (int i = 1; i < 5; i++)
{
    double angle = i * 4 * Math.PI / 5;
    path.LineTo(new PointF(radius * (float)Math.Sin(angle) +
        dirtyRect.Center.X, -radius * (float)Math.Cos(angle) + dirtyRect.Center.Y));
}
```

```
path.Close();

canvas.StrokeSize = 15;
canvas.StrokeLineJoin = LineJoin.Round;
canvas.StrokeColor = Colors.Red;
canvas.FillColor = Colors.Blue;
canvas.FillPath(path); // Overload automatically uses a NonZero winding mode
canvas.DrawPath(path);
```

In this example, the path is drawn twice. The [FillPath](#) method is used to fill the path with blue, while the [DrawPath](#) method outlines the path with a red stroke. The [FillPath](#) overload used omits the [WindingMode](#) argument, and instead automatically uses the `NonZero` winding mode. This results in all the enclosed areas of the path being filled:



⚠ Note

For many paths, the `NonZero` winding mode often fills all the enclosed areas of a path.

EvenOdd

The `EvenOdd` winding mode draws a hypothetical ray from the point to infinity in any direction and counts the number of path contours that the ray crosses. If this number is odd, then the area is filled. Otherwise, the area isn't filled.

The following example fills a five-pointed star using the `EvenOdd` winding mode:

```
C#

float radius = 0.45f * Math.Min(dirtyRect.Width, dirtyRect.Height);

PathF path = new PathF();
path.MoveTo(dirtyRect.Center.X, dirtyRect.Center.Y - radius);

for (int i = 1; i < 5; i++)
{
    double angle = i * 4 * Math.PI / 5;
```

```
path.LineTo(new PointF(radius * (float)Math.Sin(angle) +
dirtyRect.Center.X, -radius * (float)Math.Cos(angle) + dirtyRect.Center.Y));
}
path.Close();

canvas.StrokeSize = 15;
canvas.StrokeLineJoin = LineJoin.Round;
canvas.StrokeColor = Colors.Red;
canvas.FillColor = Colors.Blue;
canvas.FillPath(path, WindingMode.EvenOdd);
canvas.DrawPath(path);
```

In this example, the path is drawn twice. The [FillPath](#) method is used to fill the path with blue, while the [DrawPath](#) method outlines the path with a red stroke. The [FillPath](#) overload used specifies that the `EvenOdd` winding mode is used. This mode results in the central area of the star not being filled:



Add an app icon to a .NET MAUI app project

Article • 08/30/2024

Every app has a logo icon that represents it, and that icon typically appears in multiple places. For example, on iOS the app icon appears on the Home screen and throughout the system, such as in Settings, notifications, and search results, and in the App Store. On Android, the app icon appears as a launcher icon and throughout the system, such as on the action bar, notifications, and in the Google Play Store. On Windows, the app icon appears in the app list in the start menu, the taskbar, the app's tile, and in the Microsoft Store.

In a .NET Multi-platform App UI (.NET MAUI) app project, an app icon can be specified in a single location in your app project. At build time, this icon can be automatically resized to the correct resolution for the target platform and device, and added to your app package. This avoids having to manually duplicate and name the app icon on a per platform basis. By default, bitmap (non-vector) image formats aren't automatically resized by .NET MAUI.

A .NET MAUI app icon can use any of the standard platform image formats, including Scalable Vector Graphics (SVG) files.

Important

.NET MAUI converts SVG files to Portable Network Graphic (PNG) files. Therefore, when adding an SVG file to your .NET MAUI app project, it should be referenced from XAML or C# with a *.png* extension. The only reference to the SVG file should be in your project file.

Change the icon

In your .NET MAUI project, the image with the **MauIcon** build action designates the icon to use for your app. This is represented in your project file as the `<MauIcon>` item. You may only have one icon defined for your app. Any subsequent `<MauIcon>` items are ignored.

The icon defined by your app can be composed of a single image, by specifying the file as the `Include` attribute:

XML

```
<ItemGroup>
  <MauiIcon Include="Resources\AppIcon\appicon.svg" />
</ItemGroup>
```

Only the first `<MauiIcon>` item defined in the project file is processed by .NET MAUI. If you want to use a different file as the icon, first delete the existing icon from your project, and then add the new icon by dragging it to the *Resources\AppIcon* folder of your project. Visual Studio will automatically set its build action to **MauiIcon** and will create a corresponding `<MauiIcon>` item in your project file.

⚠ Note

An app icon can also be added to other folders of your app project. However, in this scenario its build action must be manually set to **MauiIcon** in the **Properties** window.

To comply with Android resource naming rules, app icon filenames must be lowercase, start and end with a letter character, and contain only alphanumeric characters or underscores. For more information, see [App resources overview](#) on developer.android.com.

After changing the icon file, you may need to clean the project in Visual Studio. To clean the project, right-click on the project file in the **Solution Explorer** pane, and select **Clean**. You also may need to uninstall the app from the target platform you're testing with.

⊗ Caution

If you don't clean the project and uninstall the app from the target platform, you may not see your new icon.

After changing the icon, review the [Platform specific configuration](#) information.

Composed icon

Alternatively, the app icon can be composed of two images, one image representing the background and another representing the foreground. Since icons are transformed into PNG files, the composed app icon will be first layered with the background image, typically an image of a pattern or solid color, followed by the foreground image. In this

case, the `Include` attribute represents the icon background image, and the `Foreground` attribute represents the foreground image:

XML

```
<ItemGroup>
  <MauiIcon Include="Resources\AppIcon\appicon.svg"
  ForegroundFile="Resources\AppIcon\appiconfg.svg" />
</ItemGroup>
```

On Android, a `ForegroundScale` attribute can be optionally specified to rescale the foreground image so that it fits on the app icon. For more information, see [Adaptive launcher](#).

Important

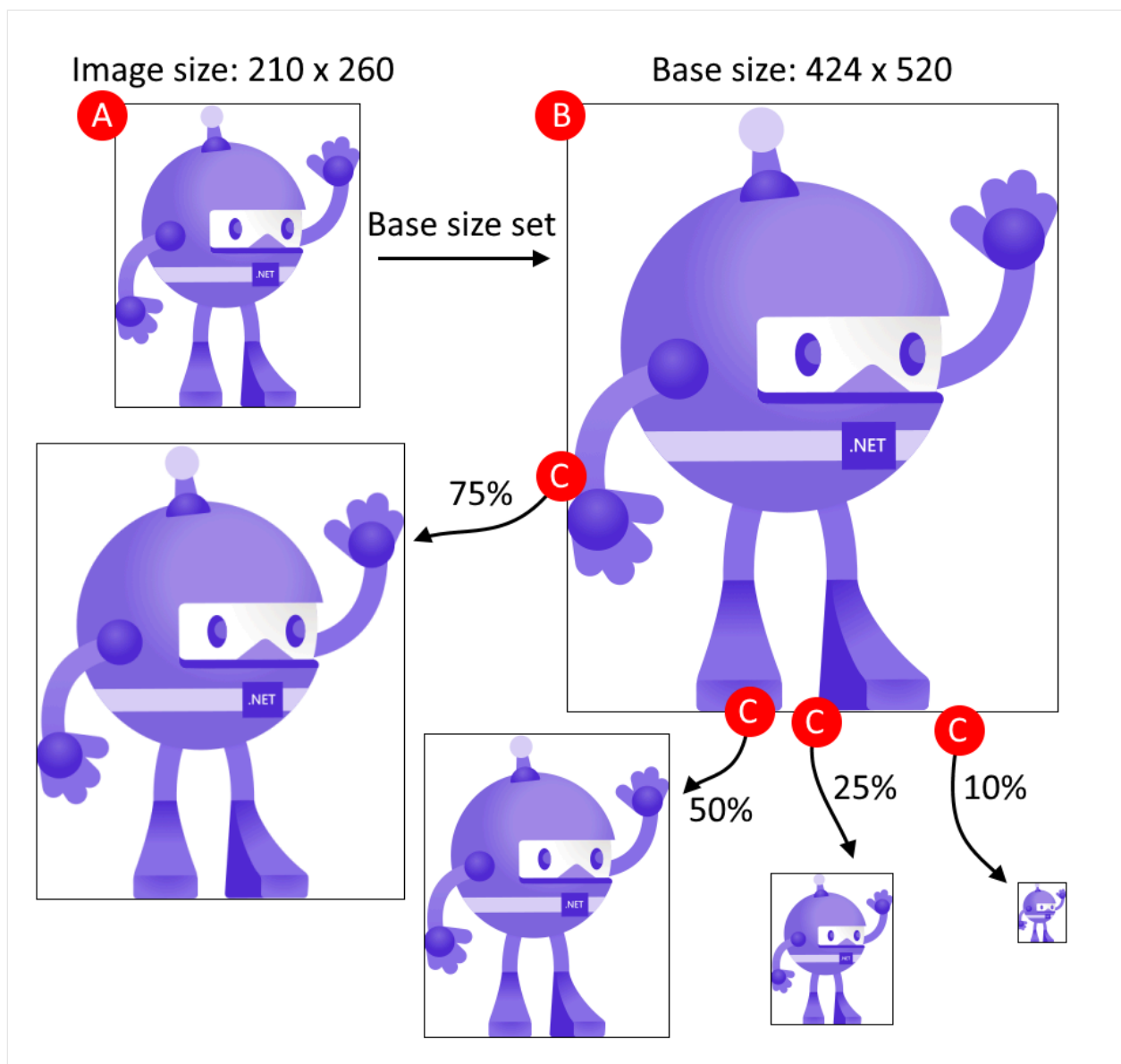
The background image (`Include` attribute) must be specified for the `<MauiIcon>` item. The foreground image (`ForegroundFile` attribute) is optional.

Set the base size

.NET MAUI uses your icon across multiple platforms and devices, and attempts to resize the icon according for each platform and device. The app icon is also used for different purposes, such as a store entry for your app or the icon used to represent the app after it's installed on a device.

The base size of your icon represents baseline density of the image, and is effectively the 1.0 scale factor that all other sizes are derived from. If you don't specify the base size for a bitmap-based app icon, such as a PNG file, the image isn't resized. If you don't specify the base size for a vector-based app icon, such as an SVG file, the dimensions specified in the image are used as the base size. To stop a vector image from being resized, set the `Resize` attribute to `false`.

The following figure illustrates how base size affects an image:



The process shown in the previous figure follows these steps:

- **A:** The image is added as the .NET MAUI icon and has dimensions of 210x260, and the base size is set to 424x520.
- **B:** .NET MAUI automatically scales the image to match the base size of 424x520.
- **C:** As different target platforms require different sizes of the image, .NET MAUI automatically scales the image from the base size to different sizes.

💡 Tip

Use an SVG image as your icon. SVG images can upscale to larger sizes and still look crisp and clean. Bitmap-based images, such as a PNG or JPG image, look blurry when upscaled.

The base size is specified with the `BaseSize="W,H"` attribute, where **W** is the width of the icon and **H** is the height of the icon. The following example sets the base size:

XML

```
<ItemGroup>
  <MauiIcon Include="Resources\AppIcon\appicon.png" BaseSize="128,128" />
</ItemGroup>
```

And the following example stops the automatic resizing of a vector-based image:

XML

```
<ItemGroup>
  <MauiIcon Include="Resources\AppIcon\appicon.svg" Resize="false" />
</ItemGroup>
```

Recolor the background

If the background image used in composing the app icon uses transparency, it can be recolored by specifying the `Color` attribute on the `<MauiIcon>`. The following example sets the background color of the app icon to red:

XML

```
<ItemGroup>
  <MauiIcon Include="Resources\AppIcon\appicon.svg" Color="#FF0000" />
</ItemGroup>
```

Color values can be specified in hexadecimal, using the format: `#RRGGBB` or `#AARRGGBB`. The value of `RR` represents the red channel, `GG` the green channel, `BB` the blue channel, and `AA` the alpha channel. Instead of a hexadecimal value, you may use a named .NET MAUI color, such as `Red` or `PaleVioletRed`.

⊗ Caution

If you don't define a background color for your app icon the background is considered to be transparent on iOS and Mac Catalyst. This will cause an error during App Store Connect verification and you won't be able to upload your app.

Recolor the foreground

If the app icon is composed of a background (`Include`) image and a foreground (`ForegroundFile`) image, the foreground image can be tinted. To tint the foreground

image, specify a color with the `TintColor` attribute. The following example tints the foreground image yellow:

XML

```
<ItemGroup>
  <MauiIcon Include="Resources\AppIcon\appicon.png"
    ForegroundFile="Resources\AppIcon\appiconfg.svg" TintColor="Yellow" />
</ItemGroup>
```

Color values can be specified in hexadecimal, using the format: `#RRGGBB` or `#AARRGGBB`. The value of `RR` represents the red channel, `GG` the green channel, `BB` the blue channel, and `AA` the alpha channel. Instead of a hexadecimal value, you may use a named .NET MAUI color, such as `Red` or `PaleVioletRed`.

Use a different icon per platform

If you want to use different icon resources or settings per platform, add the `Condition` attribute to the `<MauiIcon>` item, and query for the specific platform. If the condition is met, the `<MauiIcon>` item is processed. Only the first valid `<MauiIcon>` item is used by .NET MAUI, so all conditional items should be declared first, followed by a default `<MauiIcon>` item without a condition. The following XML demonstrates declaring a specific icon for Windows and a fallback icon for all other platforms:

XML

```
<ItemGroup>
  <!-- App icon for Windows -->
  <MauiIcon
    Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)'))
    == 'windows'"
    Include="Resources\AppIcon\backicon.png"
    ForegroundFile="Resources\AppIcon\appiconfg.svg" TintColor="#40FF00FF" />

  <!-- App icon for all other platforms -->
  <MauiIcon Include="Resources\AppIcon\appicon.png"
    ForegroundFile="Resources\AppIcon\appiconfg.svg" TintColor="Yellow" />
</ItemGroup>
```

You can set the target platform by changing the value compared in the condition to one of the following values:

- `'ios'`
- `'maccatalyst'`

- 'android'
- 'windows'

For example, a condition that targets Android would be

```
Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) ==  
'android'".
```

Platform-specific configuration

While the project file declares which resources the app icon is composed from, you're still required to update the individual platform configurations with reference to those app icons. The following information describes these platform-specific settings.

Windows

There aren't any other settings to configure for Windows.

The app icon defined by project is used to generate your app's icon assets.

Add images to a .NET MAUI app project

Article • 12/03/2024

Images are a crucial part of app navigation, usability, and branding. However, each platform has differing image requirements that typically involve creating multiple versions of each image at different resolutions. Therefore, a single image typically has to be duplicated multiple times per platform, at different resolutions, with the resulting images having to use different filename and folder conventions on each platform.

In a .NET Multi-platform App UI (.NET MAUI) app project, images can be specified in a single location in your app project, and at build time they can be resized to the correct resolution for the target platform, and added to your app package. This avoids having to manually duplicate and name images on a per platform basis. By default, bitmap (non-vector) image formats, including animated GIFs, are not automatically resized by .NET MAUI.

.NET MAUI images can use any of the standard platform image formats, including Scalable Vector Graphics (SVG) files.

Important

.NET MAUI converts SVG files to PNG files. Therefore, when adding an SVG file to your .NET MAUI app project, it should be referenced from XAML or C# with a .png extension. The only reference to the SVG file should be in your project file.

An image can be added to your app project by dragging it into the *Resources\Images* folder of the project, where its build action will automatically be set to **MauiImage**. This creates a corresponding entry in your project file:

XML

```
<ItemGroup>
  <MauiImage Include="Resources\Images\logo.svg" />
</ItemGroup>
```

Note

Images can also be added to other folders of your app project. However, in this scenario their build action must be manually set to **MauiImage** in the **Properties** window.

To comply with Android resource naming rules, image filenames must be lowercase, start and end with a letter character, and contain only alphanumeric characters or underscores. For more information, see [App resources overview](#)  on developer.android.com.

Image filenames must also be unique, otherwise a build error will occur. For more information, see [Duplicate image filename errors](#).

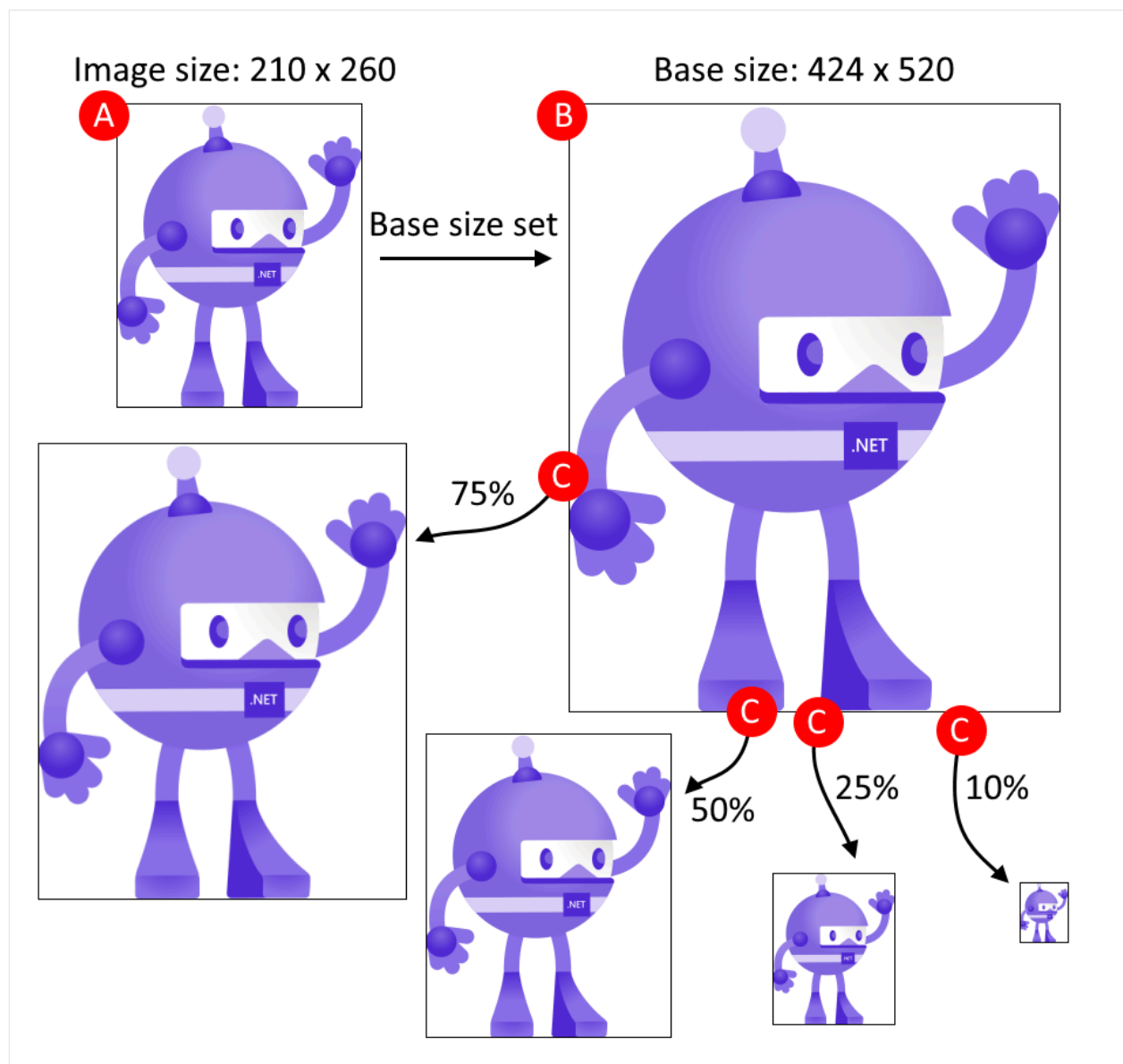
At build time, images can be resized to the correct resolutions for the target platform and device. The resulting images are then added to your app package. For information about disabling image packaging, see [Disable image packaging](#).

For information about displaying images, see [Image](#).

Resize an image

Devices have a range of screen sizes and densities and each platform has functionality for displaying density-dependent images. The base size of an image represents the baseline density of the image, and is effectively the 1.0 scale factor for the image (the size you would typically use in your code to specify the image size) from which all other density sizes are derived. If you don't specify the base size for a bitmap image, the image isn't resized. If you don't specify a base size for a vector image, such as an SVG file, the dimensions specified in the image are used as the base size.

The following diagram illustrates how base size affects an image:



The process shown in the diagram follows these steps:

- **A:** The image has dimensions of 210x260, and the base size is set to 424x520.
- **B:** .NET MAUI scales the image to match the base size of 424x520.
- **C:** As different target platforms require different sizes of the image, .NET MAUI scales the image from the base size to different sizes.

💡 Tip

Use SVG images where possible. SVG images can upscale to larger sizes and still look crisp and clean. Bitmap-based images, such as a PNG or JPG image, look blurry when upscaled.

The base size is specified with the `BaseSize="W,H"` attribute, where **W** is the width of the image and **H** is the height of the image. The following example sets the base size:

XML

```
<MauiImage Include="Resources\Images\logo.jpg" BaseSize="376,678" />
```

At build time, the image will be resized to the correct densities for the target platform. The resulting images are then added to your app package.

To stop vector images being resized, set the `Resize` attribute to `false`:

XML

```
<MauiImage Include="Resources\Images\logo.svg" Resize="false" />
```

Add tint and background color

To add a tint to your images, which is useful when you have icons or simple images you'd like to render in a different color to the source, set the `TintColor` attribute:

XML

```
<MauiImage Include="Resources\Images\logo.svg" TintColor="#66B3FF" />
```

A background color for an image can also be specified:

XML

```
<MauiImage Include="Resources\Images\logo.svg" Color="#512BD4" />
```

Color values can be specified in hexadecimal, or as a .NET MAUI color. For example, `Color="Red"` is valid.

Display an image

Images can be displayed with the [Image](#) control. For more information, see [Image](#).

Add a splash screen to a .NET MAUI app project

Article • 08/30/2024

On Android and iOS, .NET Multi-platform App UI (.NET MAUI) apps can display a splash screen while their initialization process completes. The splash screen is displayed immediately when an app is launched, providing immediate feedback to users while app resources are initialized:



Once the app is ready for interaction, its splash screen is dismissed.

Important

On iOS 16.4+, simulators won't load a splash screen unless your app is signed. For more information, including a workaround, see [GitHub issue 18479](#).

On Android 12+ (API 31+), the splash screen shows an icon that's centred on screen. For more information about splash screens on Android 12+, see [Splash screens](#) on [developer.android.com](#).

In a .NET MAUI app project, a splash screen can be specified in a single location in your app project, and at build time it can be resized to the correct resolution for the target platform, and added to your app package. This avoids having to manually duplicate and

name the splash screen on a per platform basis. By default, bitmap (non-vector) image formats are not automatically resized by .NET MAUI.

A .NET MAUI splash screen can use any of the standard platform image formats, including Scalable Vector Graphics (SVG) files.

Important

.NET MAUI converts SVG files to PNG files. Therefore, when adding an SVG file to your .NET MAUI app project, it should be referenced from XAML or C# with a .png extension. The only reference to the SVG file should be in your project file.

A splash screen can be added to your app project by dragging an image into the *Resources\Splash* folder of the project, where its build action will automatically be set to **MauiSplashScreen**. This creates a corresponding entry in your project file:

XML

```
<ItemGroup>
  <MauiSplashScreen Include="Resources\Splash\splashscreen.svg" />
</ItemGroup>
```

Note

A splash screen can also be added to other folders of your app project. However, in this scenario its build action must be manually set to **MauiSplashScreen** in the **Properties** window.

To comply with Android resource naming rules, splash screen files names must be lowercase, start and end with a letter character, and contain only alphanumeric characters or underscores. For more information, see [App resources overview](#)  on developer.android.com.

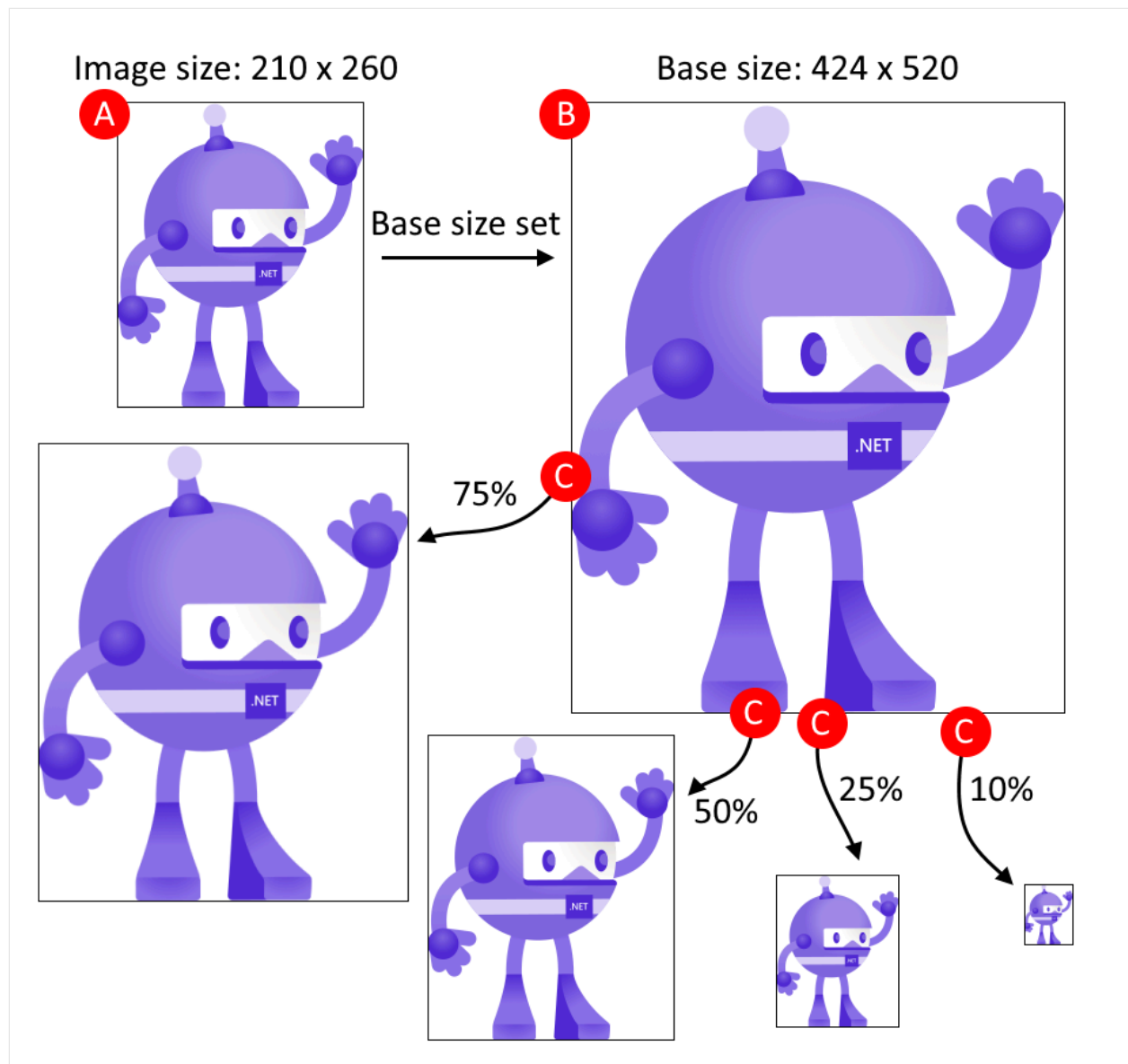
At build time, the splash screen image is resized to the correct size for the target platform and device. The resized splash screen is then added to your app package. For information about disabling splash screen packaging, see [Disable splash screen packaging](#). For information about generating a blank splash screen, see [Generate a blank splash screen](#).

Set the base size

.NET MAUI uses your splash screen across multiple platforms and can resize it for each platform.

The base size of a splash screen image represents the baseline density of the image, and is effectively the 1.0 scale factor for the image (the size you would typically use in your code to specify the splash screen size) from which all other sizes are derived. If you don't specify the base size for a bitmap image, the image isn't resized. If you don't specify a base size for a vector image, such as an SVG file, the dimensions specified in the image are used as the base size.

The following diagram illustrates how base size affects an image:



The process shown in the diagram follows these steps:

- **A:** The image has dimensions of 210x260, and the base size is set to 424x520.
- **B:** .NET MAUI scales the image to match the base size of 424x520.
- **C:** As different target platforms require different sizes of the image, .NET MAUI scales the image from the base size to different sizes.

💡 Tip

Use SVG images where possible. SVG images can upscale to larger sizes and still look crisp and clean. Bitmap-based images, such as a PNG or JPG image, look blurry when upscaled.

The base size is specified with the `BaseSize="W,H"` attribute, where `W` is the width of the image and `H` is the height of the image. The following example sets the base size:

XML

```
<MauiSplashScreen Include="Resources\Splash\splashscreen.svg"
BaseSize="128,128" />
```

At build time, the splash screen will be resized to the correct resolution for the target platform. The resulting splash screen is then added to your app package.

To stop vector images being resized, set the `Resize` attribute to `false`:

XML

```
<MauiSplashScreen Include="Resources\Splash\splashscreen.svg" Resize="false"
/>
```

Add tint and background color

To add a tint to your splash screen, which is useful when you have a simple image you'd like to render in a different color to the source, set the `TintColor` attribute:

XML

```
<MauiSplashScreen Include="Resources\Splash\splashscreen.svg"
TintColor="#66B3FF" />
```

A background color for your splash screen can also be specified:

XML

```
<MauiSplashScreen Include="Resources\Splash\splashscreen.svg"
Color="#512BD4" />
```

Color values can be specified in hexadecimal, or as a .NET MAUI color. For example, `Color="Red"` is valid.

Platform-specific configuration

Android

On Android, the splash screen is added to your app package as **Resources/values/maui_colors.xml** and **Resources/drawable/maui_splash_image.xml**. .NET MAUI apps use the `Maui.SplashTheme` by default, which ensures that a splash screen will be displayed if present. Therefore, you should not specify a different theme in your manifest file or in your `MainActivity` class:

C#

```
using Android.App;
using Android.Content.PM;

namespace MyMauiApp
{
    [Activity(Theme = "@style/Maui.SplashTheme", MainLauncher = true,
        ConfigurationChanges = ConfigChanges.ScreenSize |
        ConfigChanges.Orientation | ConfigChanges.UiMode |
        ConfigChanges.ScreenLayout | ConfigChanges.SmallestScreenSize)]
    public class MainActivity : MauiAppCompatActivity
    {
    }
}
```

For more advanced splash screen scenarios, per-platform approaches apply.

Display a context menu in a .NET MAUI desktop app

Article • 09/30/2024

A context menu, often known as a right-click menu, offers contextual commands that are specific to the control being clicked on. In .NET Multi-platform App UI (.NET MAUI), a context menu can be added to any control that derives from [Element](#), on Mac Catalyst and Windows. This includes all pages, layouts, and views.

A context menu is defined with a `MenuFlyout`, which can consist of the following children:

- `MenuFlyoutItem`, which represents a menu item that can be clicked.
- `MenuFlyoutSubItem`, which represents a sub-menu item that can be clicked.
- `MenuFlyoutSeparator`, which is a horizontal line that separates items in the menu.

`MenuFlyoutSubItem` derives from `MenuFlyoutItem`, which in turn derives from [MenuItem](#). [MenuItem](#) defines multiple properties that enable the appearance and behavior of a menu item to be specified. The appearance of a menu item, or sub-item, can be defined by setting the `Text`, and `IconImageSource` properties. The response to a menu item, or sub-item, click can be defined by setting the `Clicked`, `Command`, and `CommandParameter` properties. For more information about menu items, see [Display menu items](#).

Warning

A context menu on an [Entry](#) is currently unsupported on Mac Catalyst.

Create context menu items

A `MenuFlyout` object can be added to the `FlyoutBase.ContextFlyout` attached property of any control that derives from [Element](#). When the user right-clicks on the control, the context menu will appear at the location where the pointer was clicked.

The following example shows a [WebView](#) that defines a context menu:

XAML

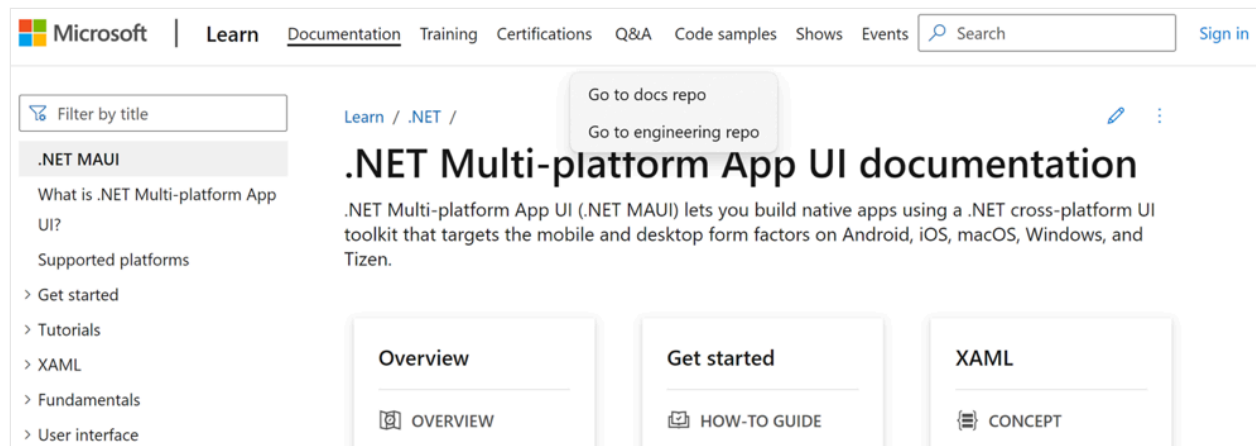
```
<WebView x:Name="webView"
        Source="https://learn.microsoft.com/dotnet/maui"
        MinimumHeightRequest="400">
```

```

<FlyoutBase.ContextFlyout>
  <MenuFlyout>
    <MenuFlyoutItem Text="Go to docs repo"
                  Clicked="OnWebViewGoToRepoClicked"
                  CommandParameter="docs" />
    <MenuFlyoutItem Text="Go to engineering repo"
                  Clicked="OnWebViewGoToRepoClicked"
                  CommandParameter="eng" />
  </MenuFlyout>
</FlyoutBase.ContextFlyout>
</WebView>

```

In this example, the context menu defines two menu items:



When a menu item is clicked upon, the `OnWebViewGoToRepoClicked` event handler is executed:

```

C#

void OnWebViewGoToRepoClicked(object sender, EventArgs e)
{
    MenuFlyoutItem menuItem = sender as MenuFlyoutItem;
    string repo = menuItem.CommandParameter as string;
    string url = repo == "docs" ? "docs-maui" : "maui";
    webView.Source = new UrlWebViewSource { Url =
    $"https://github.com/dotnet/{url}" };
}

```

The `OnWebViewGoToRepoClicked` event handler retrieves the `CommandParameter` property value for the `MenuFlyoutItem` object that was clicked, and uses its value to build the URL that the `WebView` navigates to.

 **Warning**

It's not currently possible to add items to, or remove items from, the `MenuFlyout` at runtime.

Keyboard accelerators can be added to context menu items, so that a context menu item can be invoked through a keyboard shortcut. For more information, see [Keyboard accelerators](#).

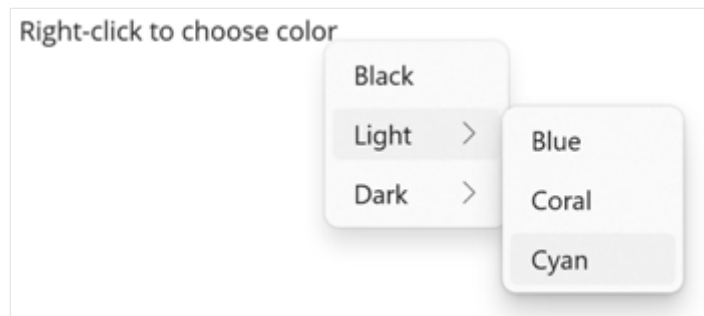
Create sub-menu items

Sub-menu items can be added to a context menu by adding one or more `MenuFlyoutSubItem` objects to the `MenuFlyout`:

XAML

```
<Label x:Name="label"
    Text="Right-click to choose color">
    <FlyoutBase.ContextFlyout>
        <MenuFlyout>
            <MenuFlyoutItem Text="Black"
                Clicked="OnLabelClicked"
                CommandParameter="Black" />
            <MenuFlyoutSubItem Text="Light">
                <MenuFlyoutItem Text="Blue"
                    Clicked="OnLabelClicked"
                    CommandParameter="LightBlue" />
                <MenuFlyoutItem Text="Coral"
                    Clicked="OnLabelClicked"
                    CommandParameter="LightCoral" />
                <MenuFlyoutItem Text="Cyan"
                    Clicked="OnLabelClicked"
                    CommandParameter="LightCyan" />
            </MenuFlyoutSubItem>
            <MenuFlyoutSubItem Text="Dark">
                <MenuFlyoutItem Text="Blue"
                    Clicked="OnLabelClicked"
                    CommandParameter="DarkBlue" />
                <MenuFlyoutItem Text="Cyan"
                    Clicked="OnLabelClicked"
                    CommandParameter="DarkCyan" />
                <MenuFlyoutItem Text="Magenta"
                    Clicked="OnLabelClicked"
                    CommandParameter="DarkMagenta" />
            </MenuFlyoutSubItem>
        </MenuFlyout>
    </FlyoutBase.ContextFlyout>
</Label>
```

In this example, the context menu defines a menu item and two sub-menus that each contain three menu items:



Display icons on menu items

`MenuFlyoutItem` and `MenuFlyoutSubItem` inherit the `IconImageSource` property from `MenuItem`, which enables a small icon to be displayed next to the text for a context menu item. This icon can either be an image, or a font icon.

Warning

Mac Catalyst does not support displaying icons on context menu items.

The following example shows a context menu, where the icons for menu items are defined using font icons:

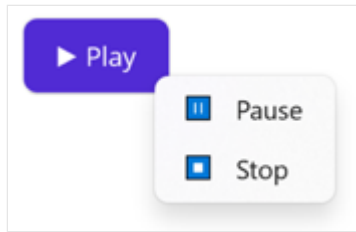
XAML

```
<Button Text="⏮️; Play"
        WidthRequest="80">
    <FlyoutBase.ContextFlyout>
        <MenuFlyout>
            <MenuFlyoutItem Text="Pause"
                            Clicked="OnPauseClicked">
                <MenuFlyoutItem.IconImageSource>
                    <FontImageSource Glyph="⏸️;"
                                      FontFamily="Arial" />
                </MenuFlyoutItem.IconImageSource>
            </MenuFlyoutItem>
            <MenuFlyoutItem Text="Stop"
                            Clicked="OnStopClicked">
                <MenuFlyoutItem.IconImageSource>
                    <FontImageSource Glyph="⏹️;"
                                      FontFamily="Arial" />
                </MenuFlyoutItem.IconImageSource>
            </MenuFlyoutItem>
        </MenuFlyout>
    </FlyoutBase.ContextFlyout>
</Button>
```



```
</FlyoutBase.ContextFlyout>  
</Button>
```

In this example, the context menu defines two menu items that display an icon and text on Windows:



For more information about displaying font icons, see [Display font icons](#). For information about adding images to .NET MAUI projects, see [Add images to a .NET MAUI app project](#).

Display a menu bar in a .NET MAUI desktop app

Article • 09/30/2024

A .NET Multi-platform App UI (.NET MAUI) menu bar is a container that presents a set of menus in a horizontal row, at the top of an app on Mac Catalyst and Windows.

Each top-level menu in the menu bar, known as a menu bar item, is represented by a [MenuBarItem](#) object. [MenuBarItem](#) defines the following properties:

- `Text`, of type `string`, defines the menu text.
- `IsEnabled`, of type `boolean`, specifies whether the menu is enabled. The default value of this property is `true`.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

A [MenuBarItem](#) can consist of the following children:

- [MenuFlyoutItem](#), which represents a menu item that can be clicked.
- [MenuFlyoutSubItem](#), which represents a sub-menu item that can be clicked.
- [MenuFlyoutSeparator](#), which is a horizontal line that separates items in the menu.

[MenuFlyoutSubItem](#) derives from [MenuFlyoutItem](#), which in turn derives from [MenuItem](#). [MenuItem](#) defines multiple properties that enable the appearance and behavior of a menu item to be specified. The appearance of a menu item, or sub-item, can be defined by setting the `Text`, and `IconImageSource` properties. The response to a menu item, or sub-item, click can be defined by setting the `Clicked`, `Command`, and `CommandParameter` properties. For more information about menu items, see [Display menu items](#).

Create menu bar items

[MenuBarItem](#) objects can be added to the [MenuBarItems](#) collection, of type `IList<MenuBarItem>`, on a [ContentPage](#). .NET MAUI desktop apps will display a menu bar, containing menu items, when they are added to any [ContentPage](#) that's hosted in a [NavigationPage](#) or a Shell app.

The following example shows a [ContentPage](#) that defines menu bar items:

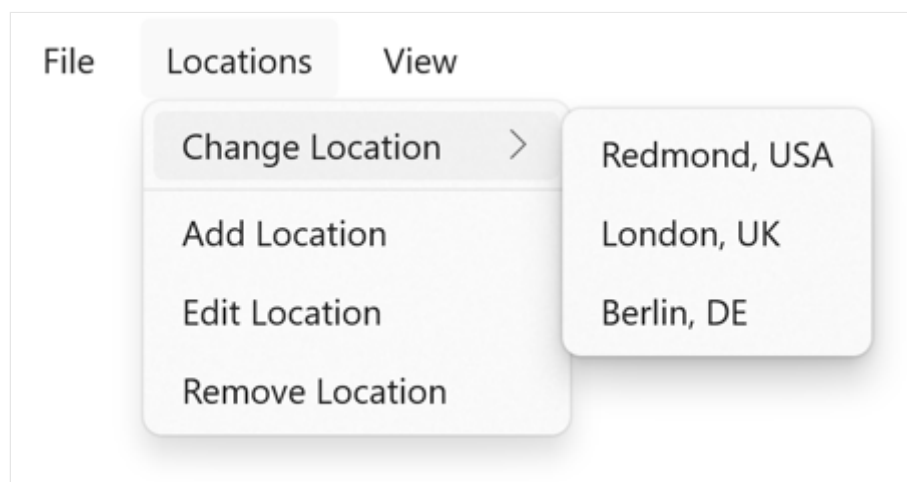
```
XAML
```

```

<ContentPage ...>
  <ContentPage.MenuBarItems>
    <MenuItem Text="File">
      <MenuFlyoutItem Text="Exit"
        Command="{Binding ExitCommand}" />
    </MenuItem>
    <MenuItem Text="Locations">
      <MenuFlyoutSubItem Text="Change Location">
        <MenuFlyoutItem Text="Redmond, USA"
          Command="{Binding ChangeLocationCommand}"
          CommandParameter="Redmond" />
        <MenuFlyoutItem Text="London, UK"
          Command="{Binding ChangeLocationCommand}"
          CommandParameter="London" />
        <MenuFlyoutItem Text="Berlin, DE"
          Command="{Binding ChangeLocationCommand}"
          CommandParameter="Berlin"/>
      </MenuFlyoutSubItem>
      <MenuFlyoutSeparator />
      <MenuFlyoutItem Text="Add Location"
        Command="{Binding AddLocationCommand}" />
      <MenuFlyoutItem Text="Edit Location"
        Command="{Binding EditLocationCommand}" />
      <MenuFlyoutItem Text="Remove Location"
        Command="{Binding RemoveLocationCommand}" />
    </MenuItem>
    <MenuItem Text="View">
      <MenuFlyoutItem Text="Refresh"
        Command="{Binding RefreshCommand}" />
      <MenuFlyoutItem Text="Change Theme"
        Command="{Binding ChangeThemeCommand}" />
    </MenuItem>
  </ContentPage.MenuBarItems>
</ContentPage>

```

This example defines three top-level menus. Each top-level menu has menu items, and the second top-level menu has a sub-menu and a separator:



ⓘ Note

On Mac Catalyst, menu items are added to the system menu bar.

In this example, each [MenuFlyoutItem](#) defines a menu item that executes an [ICommand](#) when selected.

Keyboard accelerators can be added to menu items in a menu bar, so that a menu item can be invoked through a keyboard shortcut. For more information, see [Keyboard accelerators](#).

Display icons on menu items

[MenuFlyoutItem](#) and [MenuFlyoutSubItem](#) inherit the `IconImageSource` property from [MenuItem](#), which enables a small icon to be displayed next to the text for a menu item. This icon can either be an image, or a font icon.

⚠ Warning

Mac Catalyst does not support displaying icons on menu items.

The following example shows a menu bar item, where the icons for menu items are defined using font icons:

XAML

```
<ContentPage.MenuBarItems>
  <MenuBarItem Text="Media">
    <MenuFlyoutItem Text="Play">
      <MenuFlyoutItem.IconImageSource>
        <FontImageSource Glyph="▶"
                          FontFamily="Arial" />
      </MenuFlyoutItem.IconImageSource>
    </MenuFlyoutItem>
    <MenuFlyoutItem Text="Pause"
                    Clicked="OnPauseClicked">
      <MenuFlyoutItem.IconImageSource>
        <FontImageSource Glyph="⏸"
                          FontFamily="Arial" />
      </MenuFlyoutItem.IconImageSource>
    </MenuFlyoutItem>
    <MenuFlyoutItem Text="Stop"
                    Clicked="OnStopClicked">
      <MenuFlyoutItem.IconImageSource>
        <FontImageSource Glyph="⏹"
```

```
FontFamily="Arial" />
    </MenuFlyoutItem.IconImageSource>
</MenuFlyoutItem>
</MenuBarItem>
</ContentPage.MenuBarItems>
```

In this example, the menu bar item defines three menu items that display an icon and text on Windows.

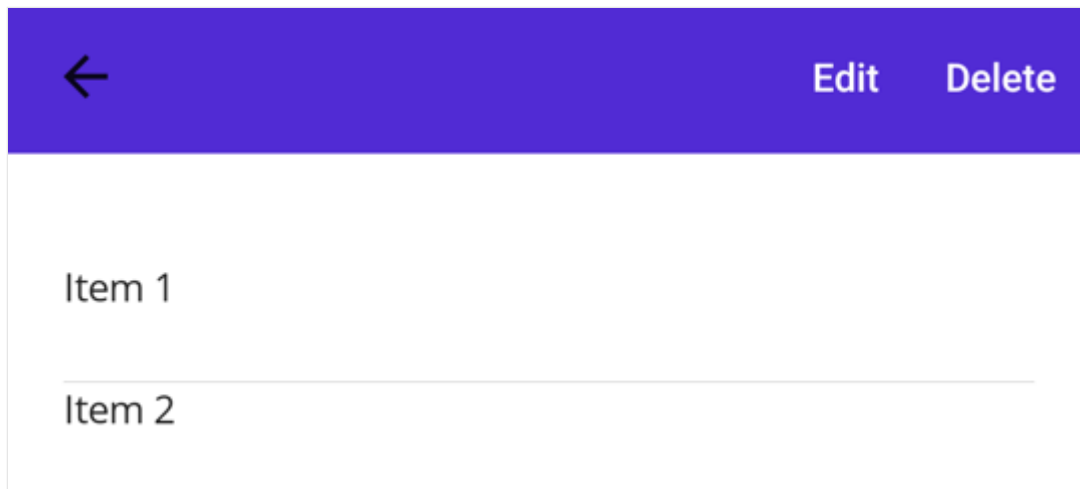
For more information about displaying font icons, see [Display font icons](#). For information about adding images to .NET MAUI projects, see [Add images to a .NET MAUI app project](#).

Display menu items

Article • 09/30/2024

The .NET Multi-platform App UI (.NET MAUI) [MenuItem](#) class can be used to define menu items for menus such as [ListView](#) item context menus and Shell app flyout menus.

The following screenshots show [MenuItem](#) objects in a [ListView](#) context menu on Android:



The [MenuItem](#) class defines the following properties:

- [Command](#), of type [ICommand](#), allows binding user actions, such as finger taps or clicks, to commands defined on a viewmodel.
- [CommandParameter](#), of type `object`, specifies the parameter that should be passed to the `Command`.
- [IconImageSource](#), of type [ImageSource](#), defines the menu item icon.
- [IsDestructive](#), of type `bool`, indicates whether the [MenuItem](#) removes its associated UI element from the list.
- [IsEnabled](#), of type `bool`, indicates whether the menu item responds to user input.
- [Text](#), of type `string`, specifies the menu item text.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings.

Create a MenuItem

To create a menu item, for example as a context menu on a [ListView](#) object's items, create a [MenuItem](#) object within a [ViewCell](#) object that's used as the [DataTemplate](#) object for the [ListView](#) `ItemTemplate`. When the [ListView](#) object is populated it will

create each item using the [DataTemplate](#), exposing the [MenuItem](#) choices when the context menu is activated for an item.

The following example shows how to create a [MenuItem](#) within the context of a [ListView](#) object:

XAML

```
<ListView>
  <ListView.ItemTemplate>
    <DataTemplate>
      <ViewCell>
        <ViewCell.ContextActions>
          <MenuItem Text="Context menu option" />
        </ViewCell.ContextActions>
        <Label Text="{Binding .}" />
      </ViewCell>
    </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

This example will result in a [MenuItem](#) object that has text. However, the appearance of a [MenuItem](#) varies across platforms.

A [MenuItem](#) can also be created in code:

C#

```
// Return a ViewCell instance that is used as the template for each list
item
DataTemplate dataTemplate = new DataTemplate(() =>
{
    // A Label displays the list item text
    Label label = new Label();
    label.SetBinding(Label.TextProperty, ".");

    // A ViewCell serves as the DataTemplate
    ViewCell viewCell = new ViewCell
    {
        View = label
    };

    // Add a MenuItem to the ContextActions
    MenuItem menuItem = new MenuItem
    {
        Text = "Context Menu Option"
    };
    viewCell.ContextActions.Add(menuItem);

    // Return the custom ViewCell to the DataTemplate constructor
    return viewCell;
});
```

```
});

ListView listView = new ListView
{
    ...
    ItemTemplate = dataTemplate
};
```

A context menu in a [ListView](#) is activated and displayed differently on each platform. On Android, the context menu is activated by long-press on a list item. The context menu replaces the title and navigation bar area and [MenuItem](#) options are displayed as horizontal buttons. On iOS, the context menu is activated by swiping on a list item. The context menu is displayed on the list item and [MenuItems](#) are displayed as horizontal buttons. On Windows, the context menu is activated by right-clicking on a list item. The context menu is displayed near the cursor as a vertical list.

Define MenuItem behavior

The [MenuItem](#) class defines a [Clicked](#) event. An event handler can be attached to this event to react to taps or clicks on [MenuItem](#) objects:

XAML

```
<MenuItem ...
    Clicked="OnItemClicked" />
```

An event handler can also be attached in code:

C#

```
MenuItem item = new MenuItem { ... };
item.Clicked += OnItemClicked;
```

These examples reference an [OnItemClicked](#) event handler, which is shown in the following example:

C#

```
void OnItemClicked(object sender, EventArgs e)
{
    MenuItem menuItem = sender as MenuItem;

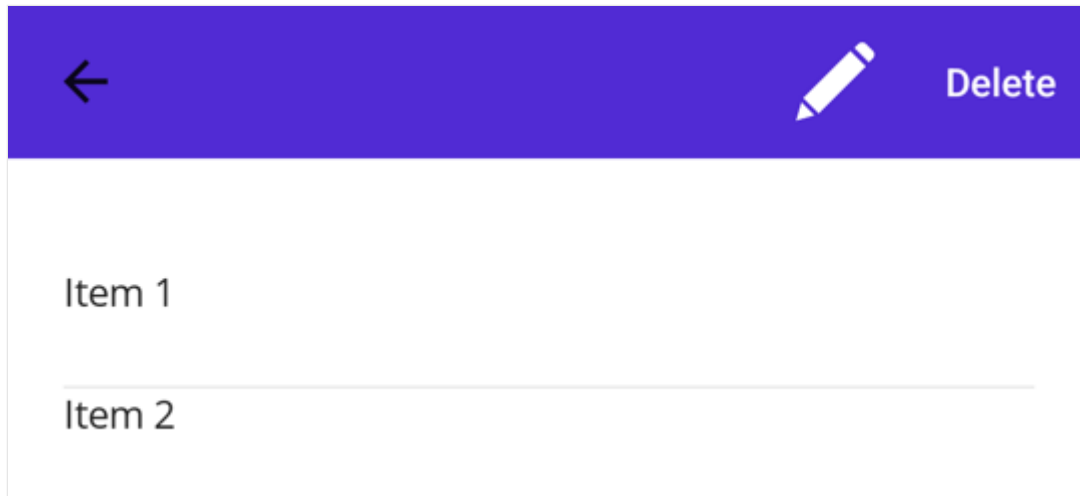
    // Access the list item through the BindingContext
    var contextItem = menuItem.BindingContext;
```



```
// Do something with the contextItem here  
}
```

Define MenuItem appearance

Icons are specified using the [IconImageSource](#) property. If an icon is specified, the text specified by the [Text](#) property won't be displayed. The following screenshot shows a [MenuItem](#) with an icon on Android:



[MenuItem](#) objects only display icons on Android. On other platforms, only the text specified by the [Text](#) property will be displayed.

ⓘ Note

Images can be stored in a single location in your app project. For more information, see [Add images to a .NET MAUI project](#).

Enable or disable a MenuItem at runtime

To enable or disable a [MenuItem](#) at runtime, bind its `Command` property to an [ICommand](#) implementation, and ensure that a `CanExecute` delegate enables and disables the [ICommand](#) as appropriate.

ⓘ Important

Don't bind the `IsEnabled` property to another property when using the `Command` property to enable or disable the [MenuItem](#).

Keyboard accelerators

Article • 11/20/2023

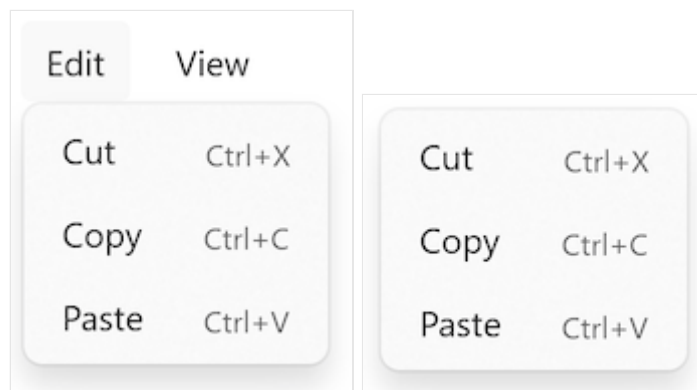
Keyboard accelerators are keyboard shortcuts that improve the usability and accessibility of your .NET Multi-platform App UI (.NET MAUI) apps on Mac Catalyst and Windows by providing an intuitive way for users to invoke common actions or commands without navigating the app UI directly.

A keyboard accelerator is composed of two components:

- Modifiers, which include Shift, Ctrl, and Alt.
- Keys, which include alphanumeric keys, and special keys.

In .NET MAUI, keyboard accelerators are associated with commands exposed in menus, and should be specified with the menu item. Specifically, .NET MAUI keyboard accelerators can be attached to menu items in the menu bar on Mac Catalyst and Windows, and menu items in context menus on Windows. For more information about menu bars, see [Display a menu bar in a .NET MAUI desktop app](#). For more information about context menus, see [Display a context menu in a .NET MAUI desktop app](#).

The following screenshots show menu bar items and context menu items that include keyboard accelerators:



A keyboard accelerator is represented by the [KeyboardAccelerator](#) class, which represents a shortcut key for a [MenuFlyoutItem](#). The [KeyboardAccelerator](#) class defines the following properties:

- [Modifiers](#), of type [KeyboardAcceleratorModifiers](#), which represents the modifier value, such as Ctrl or Shift, for the keyboard shortcut.
- [Key](#), of type `string?`, which represents the key value for the keyboard shortcut.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings.

The `KeyboardAcceleratorModifiers` enumeration defines the following members that be used as values for the `Modifiers` property:

- `None`, which indicates no modifier.
- `Shift`, which indicates the Shift modifier on Mac Catalyst and Windows.
- `Ctrl`, which indicates the Control modifier on Mac Catalyst and Windows.
- `Alt`, which indicates the Option modifier on Mac Catalyst, and the Menu modifier on Windows.
- `Cmd`, which indicates the Command modifier on Mac Catalyst.
- `Windows`, which indicates the Windows modifier on Windows.

 Important

Keyboard accelerators can be attached to `MenuFlyoutItem` objects in a `MenuBarItem` on Mac Catalyst and Windows, and in a `MenuFlyout` on Windows.

The following table outlines the keyboard accelerator formats .NET MAUI supports:

 **Expand table**

Platform	Single key	Multi-key
Mac Catalyst	Keyboard accelerators without a modifier, with a single key. For example, using the F1 key to invoke the action associated with a menu item.	Keyboard accelerators with one or more modifiers, with a single key. For example, using CMD+SHIFT+S or CMD+S to invoke the action associated with a menu item.
Windows	Keyboard accelerators with and without a modifier, with a single key. For example, using the F1 key to invoke the action associated with a menu item.	Keyboard accelerators with one or more modifiers, with a single key. For example, using CTRL+SHIFT+F or CTRL+F to invoke the action associated with a menu item.

Create a keyboard accelerator

A `KeyboardAccelerator` can be attached to a `MenuFlyoutItem` by adding it to its `KeyboardAccelerators` collection:

XAML

```
<MenuFlyoutItem Text="Cut"
                Clicked="OnCutMenuFlyoutItemClicked">
    <MenuFlyoutItem.KeyboardAccelerators>
```

```
<KeyboardAccelerator Modifiers="Ctrl"
                    Key="X" />
</MenuFlyoutItem.KeyboardAccelerators>
</MenuFlyoutItem>
```

Keyboard accelerators can also be specified in code:

C#

```
cutMenuFlyoutItem.KeyboardAccelerators.Add(new KeyboardAccelerator
{
    Modifiers = KeyboardAcceleratorModifiers.Ctrl,
    Key = "X"
});
```

When a keyboard accelerator modifier and key is pressed, the action associated with the [MenuFlyoutItem](#) is invoked.

Important

While multiple [KeyboardAccelerator](#) objects can be added to the `MenuFlyoutItem.KeyboardAccelerators` collection, only the first [KeyboardAccelerator](#) in the collection will have its shortcut displayed on the [MenuFlyoutItem](#). In addition, on Mac Catalyst, only the keyboard shortcut for the first [KeyboardAccelerator](#) in the collection will cause the action associated with the [MenuFlyoutItem](#) to be invoked. However, on Windows, the keyboard shortcuts for all of the [KeyboardAccelerator](#) objects in the `MenuFlyoutItem.KeyboardAccelerators` collection will cause the [MenuFlyoutItem](#) action to be invoked.

Specify multiple modifiers

Multiple modifiers can be specified on a [KeyboardAccelerator](#) on both platforms:

XAML

```
<MenuFlyoutItem Text="Refresh"
                Command="{Binding RefreshCommand}">
    <MenuFlyoutItem.KeyboardAccelerators>
        <KeyboardAccelerator Modifiers="Shift,Ctrl"
                            Key="R" />
    </MenuFlyoutItem.KeyboardAccelerators>
</MenuFlyoutItem>
```

The equivalent C# code is:

C#

```
refreshMenuItem.KeyboardAccelerators.Add(new KeyboardAccelerator
{
    Modifiers = KeyboardAcceleratorModifiers.Shift |
KeyboardAcceleratorModifiers.Ctrl,
    Key = "R"
});
```

Specify keyboard accelerators per platform

Different keyboard accelerator modifiers and keys can be specified per platform in XAML with the [OnPlatform](#) markup extension:

XAML

```
<MenuItem Text="Change Theme"
          Command="{Binding ChangeThemeCommand}">
    <MenuItem.KeyboardAccelerators>
        <KeyboardAccelerator Modifiers="{OnPlatform MacCatalyst=Cmd,
WinUI=Windows}"
                             Key="{OnPlatform MacCatalyst=T, WinUI=C}" />
    </MenuItem.KeyboardAccelerators>
</MenuItem>
```

The equivalent C# code is:

C#

```
KeyboardAcceleratorModifiers modifier = KeyboardAcceleratorModifiers.None;
string key = string.Empty;

if (DeviceInfo.Current.Platform == DevicePlatform.MacCatalyst)
{
    modifier = KeyboardAcceleratorModifiers.Cmd;
    key = "T";
}
else if (DeviceInfo.Current.Platform == DevicePlatform.WinUI)
{
    modifier = KeyboardAcceleratorModifiers.Windows;
    key = "C";
}

myMenuItem.KeyboardAccelerators.Add(new KeyboardAccelerator
{
    Modifiers = modifier,
    Key = key
});
```

Use special keys in a keyboard accelerator

On Windows, special keys can be specified via a string constant or with an integer. For a list of constants and integers, see the table in [VirtualKey](#).

❗ Note

On Windows, single key accelerators (all alphanumeric and punctuation keys, Delete, F2, Spacebar, Esc, Multimedia Key) and multi-key accelerators (Ctrl+Shift+M) are supported. However, Gamepad virtual keys aren't supported.

On Mac Catalyst, special keys can be specified via a string constant. For a list of constants that represent the text input strings that correspond to special keys, see [Input strings for special keys](#) on developer.apple.com.

The following XAML shows an example of defining a keyboard accelerator that uses a special key:

XAML

```
<MenuFlyoutItem Text="Help"
    Command="{Binding HelpCommand}">
    <MenuFlyoutItem.KeyboardAccelerators>
        <!-- Alternatively, 112 can be used to specify F1 on Windows -->
        <KeyboardAccelerator Modifiers="None"
            Key="{OnPlatform MacCatalyst=UIKeyInputF1,
WinUI=F1}" />
    </MenuFlyoutItem.KeyboardAccelerators>
</MenuFlyoutItem>
```

In this example the keyboard accelerator is the F1 key, which is specified via a constant on both platforms. On Windows, it could also be specified by the integer 112.

Localize a keyboard accelerator

Keyboard accelerator keys can be localized via a .NET resource file. The localized key can then be retrieved by using the `x:Static` markup extension:

XAML

```
<MenuFlyoutItem Text="Cut"
    Clicked="OnCutMenuFlyoutItemClicked">
    <MenuFlyoutItem.KeyboardAccelerators>
        <KeyboardAccelerator Modifiers="Ctrl"
```

```
Key="{x:Static
local:AppResources.CutAcceleratorKey}" />
</MenuFlyoutItem.KeyboardAccelerators>
</MenuFlyoutItem>
```

For more information, see [Localization](#).

Disable a keyboard accelerator

When a [MenuFlyoutItem](#) is disabled, the associated keyboard accelerator is also disabled:

XAML

```
<MenuFlyoutItem Text="Cut"
    Clicked="OnCutMenuFlyoutItemClicked"
    IsEnabled="false">
    <MenuFlyoutItem.KeyboardAccelerators>
        <KeyboardAccelerator Modifiers="Ctrl"
            Key="X" />
    </MenuFlyoutItem.KeyboardAccelerators>
</MenuFlyoutItem>
```

In this example, because the `IsEnabled` property of the [MenuFlyoutItem](#) is set to `false`, the associated CTRL+X keyboard accelerator can't be invoked.

Shadow

Article • 09/30/2024

The .NET Multi-platform App UI (.NET MAUI) `Shadow` class paints a shadow around a layout or view. The `VisualElement` class has a `Shadow` bindable property, of type `Shadow`, that enables a shadow to be added to any layout or view.

The `Shadow` class defines the following properties:

- `Radius`, of type `float`, defines the radius of the blur used to generate the shadow. The default value of this property is 10.
- `Opacity`, of type `float`, indicates the opacity of the shadow. The default value of this property is 1.
- `Brush`, of type `Brush`, represents the brush used to colorize the shadow.
- `Offset`, of type `Point`, specifies the offset for the shadow, which represents the position of the light source that creates the shadow.

These properties are backed by `BindableProperty` objects, which means that they can be targets of data bindings, and styled.

❗ Important

The `Brush` property only currently supports a [SolidColorBrush](#).

Create a Shadow

To add a shadow to a control, set the control's `Shadow` property to a `Shadow` object whose properties define its appearance.

The following XAML example shows how to add a shadow to an `Image`:

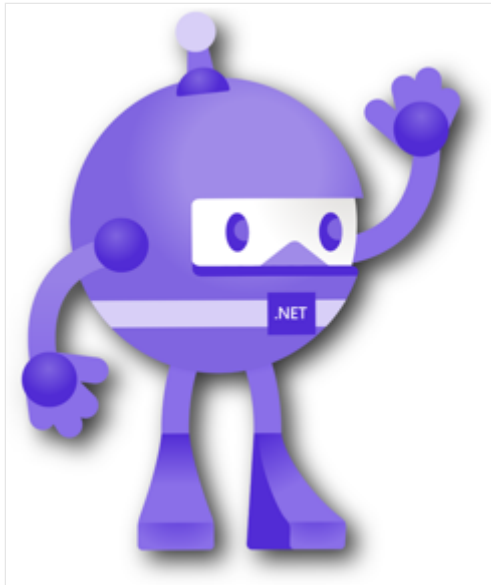
XAML

```
<Image Source="dotnet_bot.png"
        WidthRequest="250"
        HeightRequest="310">
  <Image.Shadow>
    <Shadow Brush="Black"
            Offset="20,20"
            Radius="40"
            Opacity="0.8" />
  </Image.Shadow>
</Image>
```



```
</Image.Shadow>  
</Image>
```

In this example, a black shadow is painted around the outline of the image, with its offset specifying that it appears at the right and bottom of the image:



Shadows can also be added to clipped objects, as shown in the following example:

XAML

```
<Image Source="https://aka.ms/campus.jpg"  
  Aspect="AspectFill"  
  HeightRequest="220"  
  WidthRequest="220"  
  HorizontalOptions="Center">  
  <Image.Clip>  
    <EllipseGeometry Center="220,250"  
      RadiusX="220"  
      RadiusY="220" />  
  </Image.Clip>  
  <Image.Shadow>  
    <Shadow Brush="Black"  
      Offset="10,10"  
      Opacity="0.8" />  
  </Image.Shadow>  
</Image>
```

In this example, a black shadow is painted around the outline of the [EllipseGeometry](#) that clips the image:



For more information about clipping an element, see [Clip with a Geometry](#).

Style apps using XAML

Article • 09/30/2024

.NET Multi-platform App UI (.NET MAUI) apps often contain multiple controls that have an identical appearance. For example, an app may have multiple [Label](#) instances that have the same font options and layout options:

XAML

```
<Label Text="These labels"
      HorizontalOptions="Center"
      VerticalOptions="Center"
      FontSize="18" />
<Label Text="are not"
      HorizontalOptions="Center"
      VerticalOptions="Center"
      FontSize="18" />
<Label Text="using styles"
      HorizontalOptions="Center"
      VerticalOptions="Center"
      FontSize="18" />
```

In this example, each [Label](#) object has identical property values for controlling the appearance of the text displayed by the [Label](#). However, setting the appearance of each individual control can be repetitive and error prone. Instead, a style can be created that defines the appearance, and then applied to the required controls.

Introduction to styles

An app can be styled by using the [Style](#) class to group a collection of property values into one object that can then be applied to multiple visual elements. This helps to reduce repetitive markup, and allows an apps appearance to be more easily changed.

Although styles are designed primarily for XAML-based apps, they can also be created in C#:

- [Style](#) objects created in XAML are typically defined in a [ResourceDictionary](#) that's assigned to the `Resources` collection of a control, page, or to the `Resources` collection of the app.
- [Style](#) objects created in C# are typically defined in the page's class, or in a class that can be globally accessed.

Choosing where to define a [Style](#) impacts where it can be used:

- [Style](#) instances defined at the control-level can only be applied to the control and to its children.
- [Style](#) instances defined at the page-level can only be applied to the page and to its children.
- [Style](#) instances defined at the app-level can be applied throughout the app.

Each [Style](#) object contains a collection of one or more [Setter](#) objects, with each [Setter](#) having a `Property` and a `Value`. The `Property` is the name of the bindable property of the element the style is applied to, and the `Value` is the value that is applied to the property.

Each [Style](#) object can be *explicit*, or *implicit*:

- An *explicit* [Style](#) object is defined by specifying a `TargetType` and an `x:Key` value, and by setting the target element's [Style](#) property to the `x:Key` reference. For more information, see [Explicit styles](#).
- An *implicit* [Style](#) object is defined by specifying only a `TargetType`. The [Style](#) object will then automatically be applied to all elements of that type. However, the subclasses of the `TargetType` do not automatically have the [Style](#) applied. For more information, see [Implicit styles](#).

When creating a [Style](#), the `TargetType` property is always required. The following example shows an *explicit* style:

XAML

```
<Style x:Key="labelStyle" TargetType="Label">
    <Setter Property="HorizontalOptions" Value="Center" />
    <Setter Property="VerticalOptions" Value="Center" />
    <Setter Property="FontSize" Value="18" />
</Style>
```

To apply a [Style](#), the target object must be a [VisualElement](#) that matches the `TargetType` property value of the [Style](#):

XAML

```
<Label Text="Demonstrating an explicit style" Style="{StaticResource
labelStyle}" />
```

Styles lower in the view hierarchy take precedence over those defined higher up. For example, setting a [Style](#) that sets `Label.TextColor` to `Red` at the app-level will be overridden by a page-level style that sets `Label.TextColor` to `Green`. Similarly, a page-

level style will be overridden by a control-level style. In addition, if `Label.TextColor` is set directly on a control property, this takes precedence over any styles.

Styles do not respond to property changes, and remain unchanged for the duration of an app. However, apps can respond to style changes dynamically at runtime by using dynamic resources. For more information, see [Dynamic styles](#).

Explicit styles

To create a [Style](#) at the page-level, a [ResourceDictionary](#) must be added to the page and then one or more [Style](#) declarations can be included in the [ResourceDictionary](#). A [Style](#) is made *explicit* by giving its declaration an `x:Key` attribute, which gives it a descriptive key in the [ResourceDictionary](#). *Explicit* styles must then be applied to specific visual elements by setting their [Style](#) properties.

The following example shows *explicit* styles in a page's [ResourceDictionary](#), and applied to the page's [Label](#) objects:

XAML

```
<ContentPage ...>
  <ContentPage.Resources>
    <Style x:Key="labelRedStyle"
      TargetType="Label">
      <Setter Property="HorizontalOptions" Value="Center" />
      <Setter Property="VerticalOptions" Value="Center" />
      <Setter Property="FontSize" Value="18" />
      <Setter Property="TextColor" Value="Red" />
    </Style>
    <Style x:Key="labelGreenStyle"
      TargetType="Label">
      <Setter Property="HorizontalOptions" Value="Center" />
      <Setter Property="VerticalOptions" Value="Center" />
      <Setter Property="FontSize" Value="18" />
      <Setter Property="TextColor" Value="Green" />
    </Style>
    <Style x:Key="labelBlueStyle"
      TargetType="Label">
      <Setter Property="HorizontalOptions" Value="Center" />
      <Setter Property="VerticalOptions" Value="Center" />
      <Setter Property="FontSize" Value="18" />
      <Setter Property="TextColor" Value="Blue" />
    </Style>
  </ContentPage.Resources>
  <StackLayout>
    <Label Text="These labels"
      Style="{StaticResource labelRedStyle}" />
    <Label Text="are demonstrating"
      Style="{StaticResource labelGreenStyle}" />
```

```

        <Label Text="explicit styles,"
              Style="{StaticResource labelBlueStyle}" />
        <Label Text="and an explicit style override"
              Style="{StaticResource labelBlueStyle}"
              TextColor="Teal" />
    </StackLayout>
</ContentPage>

```

In this example, the [ResourceDictionary](#) defines three styles that are explicitly set on the page's [Label](#) objects. Each [Style](#) is used to display text in a different color, while also setting the font size, and horizontal and vertical layout options. Each [Style](#) is applied to a different [Label](#) by setting its [Style](#) properties using the [StaticResource](#) markup extension. In addition, while the final [Label](#) has a [Style](#) set on it, it also overrides the `TextColor` property to a different [Color](#) value.

Implicit styles

To create a [Style](#) at the page-level, a [ResourceDictionary](#) must be added to the page and then one or more [Style](#) declarations can be included in the [ResourceDictionary](#). A [Style](#) is made *implicit* by not specifying an `x:Key` attribute. The style will then be applied to in scope visual elements that match the `TargetType` exactly, but not to elements that are derived from the `TargetType` value.

The following code example shows an *implicit* style in a page's [ResourceDictionary](#), and applied to the page's [Entry](#) objects:

XAML

```

<ContentPage ...>
    <ContentPage.Resources>
        <Style TargetType="Entry">
            <Setter Property="HorizontalOptions" Value="Fill" />
            <Setter Property="VerticalOptions" Value="Center" />
            <Setter Property="BackgroundColor" Value="Yellow" />
            <Setter Property="FontAttributes" Value="Italic" />
            <Setter Property="TextColor" Value="Blue" />
        </Style>
    </ContentPage.Resources>
    <StackLayout>
        <Entry Text="These entries" />
        <Entry Text="are demonstrating" />
        <Entry Text="implicit styles," />
        <Entry Text="and an implicit style override"
              BackgroundColor="Lime"
              TextColor="Red" />
        <local:CustomEntry Text="Subclassed Entry is not receiving the
style" />
    </StackLayout>
</ContentPage>

```

```
</StackLayout>
</ContentPage>
```

In this example, the `ResourceDictionary` defines a single *implicit* style that are implicitly set on the page's `Entry` objects. The `Style` is used to display blue text on a yellow background, while also setting other appearance options. The `Style` is added to the page's `ResourceDictionary` without specifying an `x:Key` attribute. Therefore, the `Style` is applied to all the `Entry` objects implicitly as they match the `TargetType` property of the `Style` exactly. However, the `Style` is not applied to the `CustomEntry` object, which is a subclassed `Entry`. In addition, the fourth `Entry` overrides the `BackgroundColor` and `TextColor` properties of the style to different `Color` values.

Apply a style to derived types

The `Style.ApplyToDerivedTypes` property enables a style to be applied to controls that are derived from the base type referenced by the `TargetType` property. Therefore, setting this property to `true` enables a single style to target multiple types, provided that the types derive from the base type specified in the `TargetType` property.

The following example shows an implicit style that sets the background color of `Button` instances to red:

XAML

```
<Style TargetType="Button"
      ApplyToDerivedTypes="True">
  <Setter Property="BackgroundColor"
          Value="Red" />
</Style>
```

Placing this style in a page-level `ResourceDictionary` will result in it being applied to all `Button` objects on the page, and also to any controls that derive from `Button`. However, if the `ApplyToDerivedTypes` property remained unset, the style would only be applied to `Button` objects.

Global styles

Styles can be defined globally by adding them to the app's resource dictionary. These styles can then be consumed throughout an app, and help to avoid style duplication across pages and controls.

The following example shows a [Style](#) defined at the app-level:

XAML

```
<Application xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:Styles"
             x:Class="Styles.App">
  <Application.Resources>
    <Style x:Key="buttonStyle" TargetType="Button">
      <Setter Property="HorizontalOptions"
              Value="Center" />
      <Setter Property="VerticalOptions"
              Value="CenterAndExpand" />
      <Setter Property="BorderColor"
              Value="Lime" />
      <Setter Property="CornerRadius"
              Value="5" />
      <Setter Property="BorderWidth"
              Value="5" />
      <Setter Property="WidthRequest"
              Value="200" />
      <Setter Property="TextColor"
              Value="Teal" />
    </Style>
  </Application.Resources>
</Application>
```

In this example, the [ResourceDictionary](#) defines a single *explicit* style, `buttonStyle`, which will be used to set the appearance of [Button](#) objects.

❗ Note

Global styles can be *explicit* or *implicit*.

The following example shows a page consuming the `buttonStyle` on the page's [Button](#) objects:

XAML

```
<ContentPage ...>
  <StackLayout>
    <Button Text="These buttons"
            Style="{StaticResource buttonStyle}" />
    <Button Text="are demonstrating"
            Style="{StaticResource buttonStyle}" />
    <Button Text="application styles"
            Style="{StaticResource buttonStyle}" />
  </StackLayout>
</ContentPage>
```



```
</StackLayout>
</ContentPage>
```

Style inheritance

Styles can inherit from other styles to reduce duplication and enable reuse. This is achieved by setting the `Style.BasedOn` property to an existing [Style](#). In XAML, this can be achieved by setting the `BasedOn` property to a [StaticResource](#) markup extension that references a previously created [Style](#).

Styles that inherit from a base style can include [Setter](#) instances for new properties, or use them to override setters from the base style. In addition, styles that inherit from a base style must target the same type, or a type that derives from the type targeted by the base style. For example, if a base style targets [View](#) objects, styles that are based on the base style can target [View](#) objects or types that derive from the [View](#) class, such as [Label](#) and [Button](#) objects.

A style can only inherit from styles at the same level, or above, in the view hierarchy. This means that:

- An app-level style can only inherit from other app-level styles.
- A page-level style can inherit from app-level styles, and other page-level styles.
- A control-level style can inherit from app-level styles, page-level styles, and other control-level styles.

The following example shows *explicit* style inheritance:

XAML

```
<ContentPage ...>
  <ContentPage.Resources>
    <Style x:Key="baseStyle"
          TargetType="View">
      <Setter Property="HorizontalOptions" Value="Center" />
      <Setter Property="VerticalOptions" Value="Center" />
    </Style>
  </ContentPage.Resources>
  <StackLayout>
    <StackLayout.Resources>
      <Style x:Key="labelStyle"
            TargetType="Label"
            BasedOn="{StaticResource baseStyle}">
        <Setter Property="FontSize" Value="18" />
        <Setter Property="FontAttributes" Value="Italic" />
        <Setter Property="TextColor" Value="Teal" />
      </Style>
    </StackLayout.Resources>
  </StackLayout>
</ContentPage>
```

```

        <Style x:Key="buttonStyle"
              TargetType="Button"
              BasedOn="{StaticResource baseStyle}">
            <Setter Property="BorderColor" Value="Lime" />
            <Setter Property="CornerRadius" Value="5" />
            <Setter Property="BorderWidth" Value="5" />
            <Setter Property="WidthRequest" Value="200" />
            <Setter Property="TextColor" Value="Teal" />
        </Style>
    </StackLayout.Resources>
    <Label Text="This label uses style inheritance"
           Style="{StaticResource labelStyle}" />
    <Button Text="This button uses style inheritance"
            Style="{StaticResource buttonStyle}" />
</StackLayout>
</ContentPage>

```

In this example, the `baseStyle` targets `View` objects, and sets the `HorizontalOptions` and `VerticalOptions` properties. The `baseStyle` is not set directly on any controls. Instead, `labelStyle` and `buttonStyle` inherit from it, setting additional bindable property values. The `labelStyle` and `buttonStyle` objects are then set on a `Label` and `Button`.

Important

An implicit style can be derived from an explicit style, but an explicit style can't be derived from an implicit style.

Dynamic styles

Styles do not respond to property changes, and remain unchanged for the duration of an app. For example, after assigning a `Style` to a visual element, if one of the `Setter` objects is modified, removed, or a new `Setter` added, the changes won't be applied to the visual element. However, apps can respond to style changes dynamically at runtime by using dynamic resources.

The `DynamicResource` markup extension is similar to the `StaticResource` markup extension in that both use a dictionary key to fetch a value from a `ResourceDictionary`. However, while the `StaticResource` performs a single dictionary lookup, the `DynamicResource` maintains a link to the dictionary key. Therefore, if the dictionary entry associated with the key is replaced, the change is applied to the visual element. This enables runtime style changes to be made in an app.

The following example shows *dynamic* styles:

```

<ContentPage ...>
  <ContentPage.Resources>
    <Style x:Key="baseStyle"
      TargetType="View">
      <Setter Property="VerticalOptions" Value="Center" />
    </Style>
    <Style x:Key="blueSearchBarStyle"
      TargetType="SearchBar"
      BasedOn="{StaticResource baseStyle}">
      <Setter Property="FontAttributes" Value="Italic" />
      <Setter Property="PlaceholderColor" Value="Blue" />
    </Style>
    <Style x:Key="greenSearchBarStyle"
      TargetType="SearchBar">
      <Setter Property="FontAttributes" Value="None" />
      <Setter Property="PlaceholderColor" Value="Green" />
    </Style>
  </ContentPage.Resources>
  <StackLayout>
    <SearchBar Placeholder="SearchBar demonstrating dynamic styles"
      Style="{DynamicResource blueSearchBarStyle}" />
  </StackLayout>
</ContentPage>

```

In this example, the `SearchBar` object use the `DynamicResource` markup extension to set a `Style` named `blueSearchBarStyle`. The `SearchBar` can then have its `Style` definition updated in code:

C#

```
Resources["blueSearchBarStyle"] = Resources["greenSearchBarStyle"];
```

In this example, the `blueSearchBarStyle` definition is updated to use the values from the `greenSearchBarStyle` definition. When this code is executed, the `SearchBar` will be updated to use the `Setter` objects defined in `greenSearchBarStyle`.

Dynamic style inheritance

Deriving a style from a dynamic style can't be achieved using the `Style.BasedOn` property. Instead, the `Style` class includes the `BaseResourceKey` property, which can be set to a dictionary key whose value might dynamically change.

The following example shows *dynamic* style inheritance:

```

<ContentPage ...>
  <ContentPage.Resources>
    <Style x:Key="baseStyle"
      TargetType="View">
      <Setter Property="VerticalOptions" Value="Center" />
    </Style>
    <Style x:Key="blueSearchBarStyle"
      TargetType="SearchBar"
      BasedOn="{StaticResource baseStyle}">
      <Setter Property="FontAttributes" Value="Italic" />
      <Setter Property="TextColor" Value="Blue" />
    </Style>
    <Style x:Key="greenSearchBarStyle"
      TargetType="SearchBar">
      <Setter Property="FontAttributes" Value="None" />
      <Setter Property="TextColor" Value="Green" />
    </Style>
    <Style x:Key="tealSearchBarStyle"
      TargetType="SearchBar"
      BaseResourceKey="blueSearchBarStyle">
      <Setter Property="BackgroundColor" Value="Teal" />
      <Setter Property="CancelButtonColor" Value="White" />
    </Style>
  </ContentPage.Resources>
  <StackLayout>
    <SearchBar Text="SearchBar demonstrating dynamic style inheritance"
      Style="{StaticResource tealSearchBarStyle}" />
  </StackLayout>
</ContentPage>

```

In this example, the `SearchBar` object uses the `StaticResource` markup extension to reference a `Style` named `tealSearchBarStyle`. This `Style` sets some additional properties and uses the `BaseResourceKey` property to reference `blueSearchBarStyle`. The `DynamicResource` markup extension is not required because `tealSearchBarStyle` will not change, except for the `Style` it derives from. Therefore, `tealSearchBarStyle` maintains a link to `blueSearchBarStyle` and is updated when the base style changes.

The `blueSearchBarStyle` definition can be updated in code:

C#

```
Resources["blueSearchBarStyle"] = Resources["greenSearchBarStyle"];
```

In this example, the `blueSearchBarStyle` definition is updated to use the values from the `greenSearchBarStyle` definition. When this code is executed, the `SearchBar` will be updated to use the `Setter` objects defined in `greenSearchBarStyle`.

Style classes

Style classes enable multiple styles to be applied to a control, without resorting to style inheritance.

A style class can be created by setting the `Class` property on a [Style](#) to a `string` that represents the class name. The advantage this offers, over defining an explicit style using the `x:Key` attribute, is that multiple style classes can be applied to a [VisualElement](#).

❗ Important

Multiple styles can share the same class name, provided they target different types. This enables multiple style classes, that are identically named, to target different types.

The following example shows three [BoxView](#) style classes, and a [VisualElement](#) style class:

XAML

```
<ContentPage ...>
  <ContentPage.Resources>
    <Style TargetType="BoxView"
      Class="Separator">
      <Setter Property="BackgroundColor"
        Value="#CCCCCC" />
      <Setter Property="HeightRequest"
        Value="1" />
    </Style>

    <Style TargetType="BoxView"
      Class="Rounded">
      <Setter Property="BackgroundColor"
        Value="#1FAECE" />
      <Setter Property="HorizontalOptions"
        Value="Start" />
      <Setter Property="CornerRadius"
        Value="10" />
    </Style>

    <Style TargetType="BoxView"
      Class="Circle">
      <Setter Property="BackgroundColor"
        Value="#1FAECE" />
      <Setter Property="WidthRequest"
        Value="100" />
      <Setter Property="HeightRequest"
        Value="100" />
    </Style>
  </ContentPage.Resources>
</ContentPage>
```

```

        <Setter Property="HorizontalOptions"
            Value="Start" />
        <Setter Property="CornerRadius"
            Value="50" />
    </Style>

    <Style TargetType="VisualElement"
        Class="Rotated"
        ApplyToDerivedTypes="true">
        <Setter Property="Rotation"
            Value="45" />
    </Style>
</ContentPage.Resources>
</ContentPage>

```

In this example, the `Separator`, `Rounded`, and `Circle` style classes each set `BoxView` properties to specific values. The `Rotated` style class has a `TargetType` of `VisualElement`, which means it can only be applied to `VisualElement` instances. However, its `ApplyToDerivedTypes` property is set to `true`, which ensures that it can be applied to any controls that derive from `VisualElement`, such as `BoxView`. For more information about applying a style to a derived type, see [Apply a style to derived types](#).

Style classes can be consumed by setting the `StyleClass` property of the control, which is of type `IList<string>`, to a list of style class names. The style classes will be applied, provided that the type of the control matches the `TargetType` of the style classes.

The following example shows three `BoxView` instances, each set to different style classes:

XAML

```

<ContentPage ...>
    <ContentPage.Resources>
        ...
    </ContentPage.Resources>
    <StackLayout>
        <BoxView StyleClass="Separator" />
        <BoxView WidthRequest="100"
            HeightRequest="100"
            HorizontalOptions="Center"
            StyleClass="Rounded, Rotated" />
        <BoxView HorizontalOptions="Center"
            StyleClass="Circle" />
    </StackLayout>
</ContentPage>

```

In this example, the first `BoxView` is styled to be a line separator, while the third `BoxView` is circular. The second `BoxView` has two style classes applied to it, which give it rounded corners and rotate it 45 degrees:



Important

Multiple style classes can be applied to a control because the `StyleClass` property is of type `IList<string>`. When this occurs, style classes are applied in ascending list order. Therefore, when multiple style classes set identical properties, the property in the style class that's in the highest list position will take precedence.

Style apps using Cascading Style Sheets

Article • 09/30/2024

.NET Multi-platform App UI (.NET MAUI) apps can be styled using Cascading Style Sheets (CSS). A style sheet consists of a list of rules, with each rule consisting of one or more selectors and a declaration block. A declaration block consists of a list of declarations in braces, with each declaration consisting of a property, a colon, and a value. When there are multiple declarations in a block, a semi-colon is inserted as a separator.

The following example shows some .NET MAUI compliant CSS:

CSS

```
navigationpage {
    -maui-bar-background-color: lightgray;
}

^contentpage {
    background-color: lightgray;
}

#listView {
    background-color: lightgray;
}

stacklayout {
    margin: 20;
    -maui-spacing: 6;
}

grid {
    row-gap: 6;
    column-gap: 6;
}

.mainPageTitle {
    font-style: bold;
    font-size: 14;
}

.mainPageSubtitle {
    margin-top: 15;
}

.detailPageTitle {
    font-style: bold;
    font-size: 14;
    text-align: center;
}
```



```
.detailPageSubtitle {  
    text-align: center;  
    font-style: italic;  
}  
  
listview image {  
    height: 60;  
    width: 60;  
}  
  
stacklayout>image {  
    height: 200;  
    width: 200;  
}
```

In .NET MAUI, CSS style sheets are parsed and evaluated at runtime, rather than compile time, and style sheets are re-parsed on use.

Important

It's not possible to fully style a .NET MAUI app using CSS. However, XAML styles can be used to supplement CSS. For more information about XAML styles, see [Style apps using XAML](#).

Consume a style sheet

The process for adding a style sheet to a .NET MAUI app is as follows:

1. Add an empty CSS file to your .NET MAUI app project. The CSS file can be placed in any folder, with the *Resources* folder being the recommended location.
2. Set the build action of the CSS file to **MauiCss**.

Loading a style sheet

There are a number of approaches that can be used to load a style sheet.

Note

It's not possible to change a style sheet at runtime and have the new style sheet applied.

Load a style sheet in XAML

A style sheet can be loaded and parsed with the `StyleSheet` class before being added to a [ResourceDictionary](#):

XAML

```
<Application ...>
  <Application.Resources>
    <StyleSheet Source="/Resources/styles.css" />
  </Application.Resources>
</Application>
```

The `StyleSheet.Source` property specifies the style sheet as a URI relative to the location of the enclosing XAML file, or relative to the project root if the URI starts with a `/`.

Warning

The CSS file will fail to load if its build action is not set to **MauiCss**.

Alternatively, a style sheet can be loaded and parsed with the `StyleSheet` class, before being added to a [ResourceDictionary](#), by inlining it in a `CDATA` section:

XAML

```
<ContentPage ...>
  <ContentPage.Resources>
    <StyleSheet>
      <![CDATA[
        ^contentpage {
          background-color: lightgray;
        }
      ]]>
    </StyleSheet>
  </ContentPage.Resources>
  ...
</ContentPage>
```

For more information about resource dictionaries, see [Resource dictionaries](#).

Load a style sheet in C#

In C#, a style sheet can be loaded from a `StringReader` and added to a [ResourceDictionary](#):

C#

```
using Microsoft.Maui.Controls.StyleSheets;

public partial class MyPage : ContentPage
{
    public MyPage()
    {
        InitializeComponent();

        using (var reader = new StringReader("^contentpage { background-
color: lightgray; }"))
        {
            this.Resources.Add(StyleSheet.FromReader(reader));
        }
    }
}
```

The argument to the `StyleSheet.FromReader` method is the `TextReader` that has read the style sheet.

Select elements and apply properties

CSS uses selectors to determine which elements to target. Styles with matching selectors are applied consecutively, in definition order. Styles defined on a specific item are always applied last. For more information about supported selectors, see [Selector reference](#).

CSS uses properties to style a selected element. Each property has a set of possible values, and some properties can affect any type of element, while others apply to groups of elements. For more information about supported properties, see [Property reference](#).

Child stylesheets always override parent stylesheets if they set the same properties. Therefore, the following precedence rules are followed when applying styles that set the same properties:

- A style defined in the app resources will be overwritten by a style defined in the page resources, if they set the same properties.
- A style defined in page resources will be overwritten by a style defined in the control resources, if they set the same properties.
- A style defined in the app resources will be overwritten by a style defined in the control resources, if they set the same properties.

ⓘ Note

CSS variables are unsupported.

Select elements by type

Elements in the visual tree can be selected by type with the case insensitive `element` selector:

CSS

```
stacklayout {  
    margin: 20;  
}
```

This selector identifies any `StackLayout` elements on pages that consume the style sheet, and sets their margins to a uniform thickness of 20.

ⓘ Note

The `element` selector does not identify subclasses of the specified type.

Selecting elements by base class

Elements in the visual tree can be selected by base class with the case insensitive `^base` selector:

CSS

```
^contentpage {  
    background-color: lightgray;  
}
```

This selector identifies any `ContentPage` elements that consume the style sheet, and sets their background color to `lightgray`.

ⓘ Note

The `^base` selector is specific to .NET MAUI, and isn't part of the CSS specification.

Selecting an element by name

Individual elements in the visual tree can be selected with the case sensitive `#id` selector:

CSS

```
#listView {  
    background-color: lightgray;  
}
```

This selector identifies the element whose `StyleId` property is set to `listView`. However, if the `StyleId` property is not set, the selector will fall back to using the `x:Name` of the element. Therefore, in the following example, the `#listView` selector will identify the `ListView` whose `x:Name` attribute is set to `listView`, and will set its background color to `lightgray`.

XAML

```
<ContentPage ...>  
    <ContentPage.Resources>  
        <StyleSheet Source="/Resources/styles.css" />  
    </ContentPage.Resources>  
    <StackLayout>  
        <ListView x:Name="listView">  
            ...  
        </ListView>  
    </StackLayout>  
</ContentPage>
```

Select elements with a specific class attribute

Elements with a specific class attribute can be selected with the case sensitive `.class` selector:

CSS

```
.detailPageTitle {  
    font-style: bold;  
    font-size: 14;  
    text-align: center;  
}  
  
.detailPageSubtitle {  
    text-align: center;  
    font-style: italic;  
}
```

A CSS class can be assigned to a XAML element by setting the `StyleClass` property of the element to the CSS class name. Therefore, in the following example, the styles defined by the `.detailPageTitle` class are assigned to the first `Label`, while the styles defined by the `.detailPageSubtitle` class are assigned to the second `Label`.

XAML

```
<ContentPage ...>
  <ContentPage.Resources>
    <StyleSheet Source="/Resources/styles.css" />
  </ContentPage.Resources>
  <ScrollView>
    <StackLayout>
      <Label ... StyleClass="detailPageTitle" />
      <Label ... StyleClass="detailPageSubtitle"/>
    </StackLayout>
  </ScrollView>
</ContentPage>
```

Select child elements

Child elements in the visual tree can be selected with the case insensitive `element element` selector:

CSS

```
listview image {
  height: 60;
  width: 60;
}
```

This selector identifies any `Image` elements that are children of `ListView` elements, and sets their height and width to 60. Therefore, in the following XAML example, the `listview image` selector will identify the `Image` that's a child of the `ListView`, and sets its height and width to 60.

XAML

```
<ContentPage ...>
  <ContentPage.Resources>
    <StyleSheet Source="/Resources/styles.css" />
  </ContentPage.Resources>
  <StackLayout>
    <ListView ...>
      <ListView.ItemTemplate>
        <DataTemplate>
          <ViewCell>
```

```

        <Grid>
            ...
            <Image ... />
            ...
        </Grid>
    </ViewCell>
</DataTemplate>
</ListView.ItemTemplate>
</ListView>
</StackLayout>
</ContentPage>

```

ⓘ Note

The `element element` selector does not require the child element to be a *direct* child of the parent – the child element may have a different parent. Selection occurs provided that an ancestor is the specified first element.

Select direct child elements

Direct child elements in the visual tree can be selected with the case insensitive `element>element` selector:

CSS

```

stacklayout>image {
    height: 200;
    width: 200;
}

```

This selector identifies any [Image](#) elements that are direct children of [StackLayout](#) elements, and sets their height and width to 200. Therefore, in the following example, the `stacklayout>image` selector will identify the [Image](#) that's a direct child of the [StackLayout](#), and sets its height and width to 200.

XAML

```

<ContentPage ...>
    <ContentPage.Resources>
        <StyleSheet Source="/Resources/styles.css" />
    </ContentPage.Resources>
    <ScrollView>
        <StackLayout>
            ...
            <Image ... />
            ...
        </StackLayout>
    </ScrollView>
</ContentPage>

```

```
</StackLayout>
</ScrollView>
</ContentPage>
```

ⓘ Note

The `element>element` selector requires that the child element is a *direct* child of the parent.

Selector reference

The following CSS selectors are supported by .NET MAUI:

[🔗 Expand table](#)

Selector	Example	Description
<code>.class</code>	<code>.header</code>	Selects all elements with the <code>StyleClass</code> property containing 'header'. This selector is case sensitive.
<code>#id</code>	<code>#email</code>	Selects all elements with <code>StyleId</code> set to <code>email</code> . If <code>StyleId</code> is not set, fallback to <code>x:Name</code> . When using XAML, <code>x:Name</code> is preferred over <code>StyleId</code> . This selector is case sensitive.
<code>*</code>	<code>*</code>	Selects all elements.
<code>element</code>	<code>label</code>	Selects all elements of type <code>Label</code> , but not subclasses. This selector is case insensitive.
<code>^base</code>	<code>^contentpage</code>	Selects all elements with <code>ContentPage</code> as the base class, including <code>ContentPage</code> itself. This selector is case insensitive and isn't part of the CSS specification.
<code>element,element</code>	<code>label,button</code>	Selects all <code>Button</code> elements and all <code>Label</code> elements. This selector is case insensitive.
<code>element element</code>	<code>stacklayout label</code>	Selects all <code>Label</code> elements inside a <code>StackLayout</code> . This selector is case insensitive.
<code>element>element</code>	<code>stacklayout>label</code>	Selects all <code>Label</code> elements with <code>StackLayout</code> as a direct parent. This selector is case insensitive.
<code>element+element</code>	<code>label+entry</code>	Selects all <code>Entry</code> elements directly after a <code>Label</code> . This selector is case insensitive.
<code>element~element</code>	<code>label~entry</code>	Selects all <code>Entry</code> elements preceded by a <code>Label</code> . This selector is case insensitive.

Styles with matching selectors are applied consecutively, in definition order. Styles defined on a specific item are always applied last.

Tip

Selectors can be combined without limitation, such as `StackLayout>ContentView>label.email`.

The following selectors are unsupported:

- `[attribute]`
- `@media` and `@supports`
- `:` and `::`

Note

Specificity, and specificity overrides are unsupported.

Property reference

The following CSS properties are supported by .NET MAUI (in the **Values** column, types are *italic*, while string literals are `gray`):

 Expand table

Property	Applies to	Values	Example
<code>align-content</code>	FlexLayout	<code>stretch</code> <code>center</code> <code>start</code> <code>end</code> <code>spacebetween</code> <code>spacearound</code> <code>spaceevenly</code> <code>flex-start</code> <code>flex-end</code> <code>space-between</code> <code>space-around</code> <code>initial</code>	<code>align-content: space-between;</code>
<code>align-items</code>	FlexLayout	<code>stretch</code> <code>center</code> <code>start</code> <code>end</code> <code>flex-start</code> <code>flex-end</code> <code>initial</code>	<code>align-items: flex-start;</code>
<code>align-self</code>	VisualElement	<code>auto</code> <code>stretch</code> <code>center</code> <code>start</code> <code>end</code> <code>flex-start</code> <code>flex-end</code> <code>initial</code>	<code>align-self: flex-end;</code>

Property	Applies to	Values	Example
<code>background-color</code>	VisualElement	<i>color</i> <code>initial</code>	<code>background-color: springgreen;</code>
<code>background-image</code>	Page	<i>string</i> <code>initial</code>	<code>background-image: bg.png;</code>
<code>border-color</code>	Button , Frame , ImageButton	<i>color</i> <code>initial</code>	<code>border-color: #9acd32;</code>
<code>border-radius</code>	BoxView , Button , Frame , ImageButton	<i>double</i> <code>initial</code>	<code>border-radius: 10;</code>
<code>border-width</code>	Button , ImageButton	<i>double</i> <code>initial</code>	<code>border-width: .5;</code>
<code>color</code>	ActivityIndicator , BoxView , Button , CheckBox , DatePicker , Editor , Entry , Label , Picker , ProgressBar , SearchBar , Switch , TimePicker	<i>color</i> <code>initial</code>	<code>color: rgba(255, 0, 0, 0.3);</code>
<code>column-gap</code>	Grid	<i>double</i> <code>initial</code>	<code>column-gap: 9;</code>
<code>direction</code>	VisualElement	<code>ltr</code> <code>rtl</code> <code>inherit</code> <code>initial</code>	<code>direction: rtl;</code>
<code>flex-direction</code>	FlexLayout	<code>column</code> <code>columnreverse</code> <code>row</code> <code>rowreverse</code> <code>row-reverse</code> <code>column-reverse</code> <code>initial</code>	<code>flex-direction: column-reverse;</code>
<code>flex-basis</code>	VisualElement	<i>float</i> <code>auto</code> <code>initial</code> . In addition, a percentage in the range 0% to 100% can be specified with the <code>%</code> sign.	<code>flex-basis: 25%;</code>
<code>flex-grow</code>	VisualElement	<i>float</i> <code>initial</code>	<code>flex-grow: 1.5;</code>
<code>flex-shrink</code>	VisualElement	<i>float</i> <code>initial</code>	<code>flex-shrink: 1;</code>
<code>flex-wrap</code>	VisualElement	<code>nowrap</code> <code>wrap</code> <code>reverse</code> <code>wrap-reverse</code> <code>initial</code>	<code>flex-wrap: wrap-reverse;</code>
<code>font-family</code>	Button , DatePicker , Editor , Entry , Label , Picker , SearchBar , TimePicker , Span	<i>string</i> <code>initial</code>	<code>font-family: Consolas;</code>
<code>font-size</code>	Button , DatePicker , Editor , Entry , Label , Picker , SearchBar , TimePicker , Span	<i>double</i> <code>initial</code>	<code>font-size: 12;</code>

Property	Applies to	Values	Example
<code>font-style</code>	Button , DatePicker , Editor , Entry , Label , Picker , SearchBar , TimePicker , Span	<code>bold</code> <code>italic</code> <code>initial</code>	<code>font-style:</code> <code>bold;</code>
<code>height</code>	VisualElement	<code>double</code> <code>initial</code>	<code>height: 250;</code>
<code>justify-content</code>	FlexLayout	<code>start</code> <code>center</code> <code>end</code> <code>spacebetween</code> <code>spacearound</code> <code>spaceevenly</code> <code>flex-start</code> <code>flex-end</code> <code>space-between</code> <code>space-around</code> <code>initial</code>	<code>justify-</code> <code>content: flex-</code> <code>end;</code>
<code>letter-spacing</code>	Button , DatePicker , Editor , Entry , Label , Picker , SearchBar , SearchHandler , Span , TimePicker	<code>double</code> <code>initial</code>	<code>letter-spacing:</code> <code>2.5;</code>
<code>line-height</code>	Label , Span	<code>double</code> <code>initial</code>	<code>line-height:</code> <code>1.8;</code>
<code>margin</code>	View	<code>thickness</code> <code>initial</code>	<code>margin: 6 12;</code>
<code>margin-left</code>	View	<code>thickness</code> <code>initial</code>	<code>margin-left: 3;</code>
<code>margin-top</code>	View	<code>thickness</code> <code>initial</code>	<code>margin-top: 2;</code>
<code>margin-right</code>	View	<code>thickness</code> <code>initial</code>	<code>margin-right:</code> <code>1;</code>
<code>margin-bottom</code>	View	<code>thickness</code> <code>initial</code>	<code>margin-bottom:</code> <code>6;</code>
<code>max-lines</code>	Label	<code>int</code> <code>initial</code>	<code>max-lines: 2;</code>
<code>min-height</code>	VisualElement	<code>double</code> <code>initial</code>	<code>min-height: 50;</code>
<code>min-width</code>	VisualElement	<code>double</code> <code>initial</code>	<code>min-width: 112;</code>
<code>opacity</code>	VisualElement	<code>double</code> <code>initial</code>	<code>opacity: .3;</code>
<code>order</code>	VisualElement	<code>int</code> <code>initial</code>	<code>order: -1;</code>
<code>padding</code>	Button , ImageButton , Layout , Page	<code>thickness</code> <code>initial</code>	<code>padding: 6 12</code> <code>12;</code>
<code>padding-left</code>	Button , ImageButton , Layout , Page	<code>double</code> <code>initial</code>	<code>padding-left:</code> <code>3;</code>
<code>padding-top</code>	Button , ImageButton , Layout ,	<code>double</code> <code>initial</code>	<code>padding-top: 4;</code>

Property	Applies to	Values	Example
	Page		
padding-right	Button, ImageButton, Layout, Page	double initial	padding-right: 2;
padding-bottom	Button, ImageButton, Layout, Page	double initial	padding-bottom: 6;
position	FlexLayout	relative absolute initial	position: absolute;
row-gap	Grid	double initial	row-gap: 12;
text-align	Entry, EntryCell, Label, SearchBar	left top right bottom start center middle end initial. left and right should be avoided in right-to-left environments.	text-align: right;
text-decoration	Label, Span	none underline strikethrough line-through initial	text-decoration: underline, line-through;
text-transform	Button, Editor, Entry, Label, SearchBar, SearchHandler	none default uppercase lowercase initial	text-transform: uppercase;
transform	VisualElement	none, rotate, rotateX, rotateY, scale, scaleX, scaleY, translate, translateX, translateY, initial	transform: rotate(180), scaleX(2.5);
transform-origin	VisualElement	double, double initial	transform-origin: 7.5, 12.5;
vertical-align	Label	left top right bottom start center middle end initial	vertical-align: bottom;
visibility	VisualElement	true visible false hidden collapse initial	visibility: hidden;
width	VisualElement	double initial	width: 320;

📌 Note

`initial` is a valid value for all properties. It clears the value (resets to default) that was set from another style.

The following properties are unsupported:

- `all: initial`.
- Layout properties (box, or grid).
- Shorthand properties, such as `font`, and `border`.

In addition, there's no `inherit` value and so inheritance isn't supported. Therefore you can't, for example, set the `font-size` property on a layout and expect all the [Label](#) instances in the layout to inherit the value. The one exception is the `direction` property, which has a default value of `inherit`.

Important

[Span](#) elements can't be targeted using CSS.

.NET MAUI specific properties

The following .NET MAUI specific CSS properties are also supported (in the **Values** column, types are *italic*, while string literals are `gray`):

 Expand table

Property	Applies to	Values	Example
<code>-maui-bar-background-color</code>	NavigationPage , TabbedPage	<i>color</i> <code>initial</code>	<code>-maui-bar-background-color: teal;</code>
<code>-maui-bar-text-color</code>	NavigationPage , TabbedPage	<i>color</i> <code>initial</code>	<code>-maui-bar-text-color: gray</code>
<code>-maui-horizontal-scroll-bar-visibility</code>	ScrollView	<code>default</code> <code>always</code> <code>never</code> <code>initial</code>	<code>-maui-horizontal-scroll-bar-visibility: never;</code>
<code>-maui-max-length</code>	Entry , Editor , SearchBar	<i>int</i> <code>initial</code>	<code>-maui-max-length: 20;</code>
<code>-maui-max-track-color</code>	Slider	<i>color</i> <code>initial</code>	<code>-maui-max-track-color: red;</code>

Property	Applies to	Values	Example
<code>-maui-min-track-color</code>	Slider	<i>color</i> <code>initial</code>	<code>-maui-min-track-color: yellow;</code>
<code>-maui-orientation</code>	ScrollView , StackLayout	<code>horizontal</code> <code>vertical</code> <code>both</code> <code>initial</code> . <code>both</code> is only supported on a ScrollView .	<code>-maui-orientation: horizontal;</code>
<code>-maui-placeholder</code>	Entry , Editor , SearchBar	<i>quoted text</i> <code>initial</code>	<code>-maui-placeholder: Enter name;</code>
<code>-maui-placeholder-color</code>	Entry , Editor , SearchBar	<i>color</i> <code>initial</code>	<code>-maui-placeholder-color: green;</code>
<code>-maui-spacing</code>	StackLayout	<i>double</i> <code>initial</code>	<code>-maui-spacing: 8;</code>
<code>-maui-thumb-color</code>	Slider , Switch	<i>color</i> <code>initial</code>	<code>-maui-thumb-color: limegreen;</code>
<code>-maui-vertical-scroll-bar-visibility</code>	ScrollView	<code>default</code> <code>always</code> <code>never</code> <code>initial</code>	<code>-maui-vertical-scroll-bar-visibility: always;</code>
<code>-maui-vertical-text-alignment</code>	Label	<code>start</code> <code>center</code> <code>end</code> <code>initial</code>	<code>-maui-vertical-text-alignment: end;</code>
<code>-maui-visual</code>	VisualElement	<i>string</i> <code>initial</code>	<code>-maui-visual: material;</code>

.NET MAUI Shell specific properties

The following .NET MAUI Shell specific CSS properties are also supported (in the **Values** column, types are *italic*, while string literals are `gray`):

 Expand table

Property	Applies to	Values	Example
<code>-maui-flyout-background</code>	Shell	<i>color</i> <code>initial</code>	<code>-maui-flyout-background: red;</code>
<code>-maui-shell-background</code>	Element	<i>color</i> <code>initial</code>	<code>-maui-shell-background: green;</code>
<code>-maui-shell-disabled</code>	Element	<i>color</i> <code>initial</code>	<code>-maui-shell-disabled: blue;</code>

Property	Applies to	Values	Example
<code>-maui-shell-foreground</code>	Element	<code>color</code> <code>initial</code>	<code>-maui-shell-foreground: yellow;</code>
<code>-maui-shell-tabbar-background</code>	Element	<code>color</code> <code>initial</code>	<code>-maui-shell-tabbar-background: white;</code>
<code>-maui-shell-tabbar-disabled</code>	Element	<code>color</code> <code>initial</code>	<code>-maui-shell-tabbar-disabled: black;</code>
<code>-maui-shell-tabbar-foreground</code>	Element	<code>color</code> <code>initial</code>	<code>-maui-shell-tabbar-foreground: gray;</code>
<code>-maui-shell-tabbar-title</code>	Element	<code>color</code> <code>initial</code>	<code>-maui-shell-tabbar-title: lightgray;</code>
<code>-maui-shell-tabbar-unselected</code>	Element	<code>color</code> <code>initial</code>	<code>-maui-shell-tabbar-unselected: cyan;</code>
<code>-maui-shell-title</code>	Element	<code>color</code> <code>initial</code>	<code>-maui-shell-title: teal;</code>
<code>-maui-shell-unselected</code>	Element	<code>color</code> <code>initial</code>	<code>-maui-shell-unselected: limegreen;</code>

Color

The following `color` values are supported:

- `x11 colors` [↗](#), which match CSS colors and .NET MAUI colors. These color values are case insensitive.
- hex colors: `#rgb`, `#argb`, `#rrggbb`, `#aarrggbb`
- rgb colors: `rgb(255,0,0)`, `rgb(100%,0%,0%)`. Values are in the range 0-255, or 0%-100%.
- rgba colors: `rgba(255, 0, 0, 0.8)`, `rgba(100%, 0%, 0%, 0.8)`. The opacity value is in the range 0.0-1.0.
- hsl colors: `hsl(120, 100%, 50%)`. The h value is in the range 0-360, while s and l are in the range 0%-100%.
- hsla colors: `hsla(120, 100%, 50%, .8)`. The opacity value is in the range 0.0-1.0.

Thickness

One, two, three, or four `thickness` values are supported, each separated by white space:

- A single value indicates uniform thickness.
- Two values indicate vertical then horizontal thickness.
- Three values indicate top, then horizontal (left and right), then bottom thickness.
- Four values indicate top, then right, then bottom, then left thickness.

ⓘ Note

CSS `thickness` values differ from XAML `Thickness` values. For example, in XAML a two-value `Thickness` indicates horizontal then vertical thickness, while a four-value `Thickness` indicates left, then top, then right, then bottom thickness. In addition, XAML `Thickness` values are comma delimited.

Functions

Linear and radial gradients can be specified using the `linear-gradient()` and `radial-gradient()` CSS functions, respectively. The result of these functions should be assigned to the `background` property of a control.

Theme an app

Article • 10/01/2024



[Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) apps can respond to style changes dynamically at runtime by using the [DynamicResource](#) markup extension. This markup extension is similar to the [StaticResource](#) markup extension, in that both use a dictionary key to fetch a value from a [ResourceDictionary](#). However, while the [StaticResource](#) markup extension performs a single dictionary lookup, the [DynamicResource](#) markup extension maintains a link to the dictionary key. Therefore, if the value associated with the key is replaced, the change is applied to the [VisualElement](#). This enables runtime theming to be implemented in .NET MAUI apps.

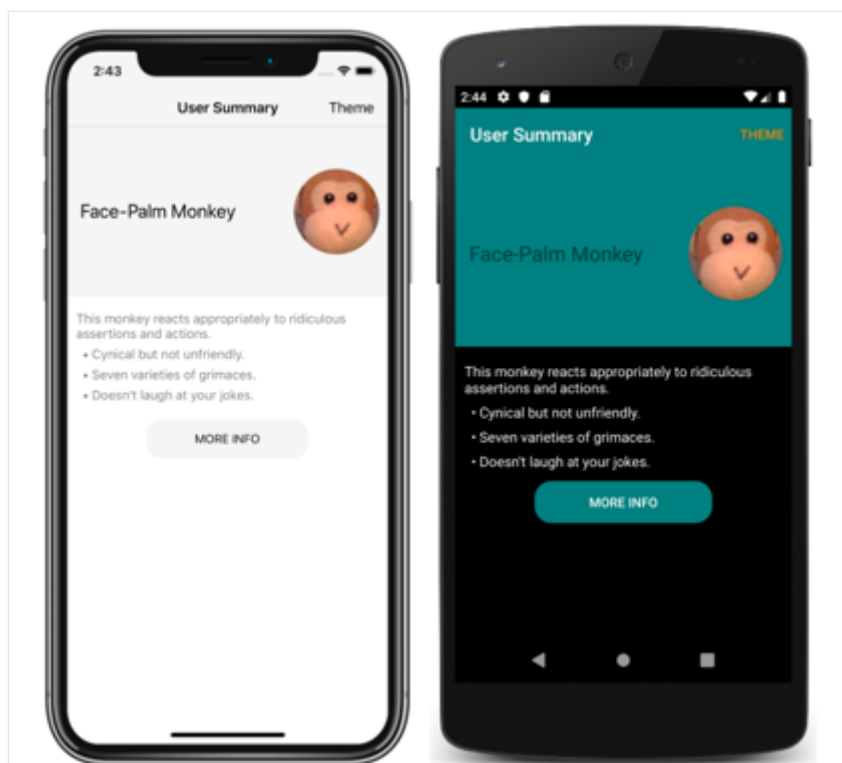
The process for implementing runtime theming in a .NET MAUI app is as follows:

1. Define the resources for each theme in a [ResourceDictionary](#). For more information, see [Define themes](#).
2. Set a default theme in the app's *App.xaml* file. For more information, see [Set a default theme](#).
3. Consume theme resources in the app, using the [DynamicResource](#) markup extension. For more information, see [Consume theme resources](#).
4. Add code to load a theme at runtime. For more information, see [Load a theme at runtime](#).

Important

Use the [StaticResource](#) markup extension if your app doesn't need to change themes dynamically at runtime. If you anticipate switching themes while the app is running, use the [DynamicResource](#) markup extension, which enables resources to be updated at runtime.

The following screenshot shows themed pages, with the iOS app using a light theme and the Android app using a dark theme:



ⓘ Note

Changing a theme at runtime requires the use of XAML or C# style definitions, and isn't possible using CSS.

.NET MAUI also has the ability to respond to system theme changes. The system theme may change for a variety of reasons, depending on the device configuration. This includes the system theme being explicitly changed by the user, it changing due to the time of day, and it changing due to environmental factors such as low light. For more information, see [Respond to system theme changes](#).

Define themes

A theme is defined as a collection of resource objects stored in a [ResourceDictionary](#).

The following example shows a [ResourceDictionary](#) for a light theme named `LightTheme`:

XAML

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
                    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
                    x:Class="ThemingDemo.LightTheme">
    <Color x:Key="PageBackgroundColor">White</Color>
    <Color x:Key="NavigationBarColor">WhiteSmoke</Color>
    <Color x:Key="PrimaryColor">WhiteSmoke</Color>
    <Color x:Key="SecondaryColor">Black</Color>
    <Color x:Key="PrimaryTextColor">Black</Color>
```

```
<Color x:Key="SecondaryTextColor">White</Color>
<Color x:Key="TertiaryTextColor">Gray</Color>
<Color x:Key="TransparentColor">Transparent</Color>
</ResourceDictionary>
```

The following example shows a [ResourceDictionary](#) for a dark theme named `DarkTheme`:

XAML

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="ThemingDemo.DarkTheme">
    <Color x:Key="PageBackgroundColor">Black</Color>
    <Color x:Key="NavigationBarColor">Teal</Color>
    <Color x:Key="PrimaryColor">Teal</Color>
    <Color x:Key="SecondaryColor">White</Color>
    <Color x:Key="PrimaryTextColor">White</Color>
    <Color x:Key="SecondaryTextColor">White</Color>
    <Color x:Key="TertiaryTextColor">WhiteSmoke</Color>
    <Color x:Key="TransparentColor">Transparent</Color>
</ResourceDictionary>
```

Each [ResourceDictionary](#) contains [Color](#) resources that define their respective themes, with each [ResourceDictionary](#) using identical key values. For more information about resource dictionaries, see [Resource Dictionaries](#).

Important

A code behind file is required for each [ResourceDictionary](#), which calls the `InitializeComponent` method. This is necessary so that a CLR object representing the chosen theme can be created at runtime.

Set a default theme

An app requires a default theme, so that controls have values for the resources they consume. A default theme can be set by merging the theme's [ResourceDictionary](#) into the app-level [ResourceDictionary](#) that's defined in *App.xaml*:

XAML

```
<Application xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="ThemingDemo.App">
    <Application.Resources>
        <ResourceDictionary Source="Themes/LightTheme.xaml" />
    </Application.Resources>
</Application>
```

```
</Application.Resources>
</Application>
```

For more information about merging resource dictionaries, see [Merged resource dictionaries](#).

Consume theme resources

When an app wants to consume a resource that's stored in a [ResourceDictionary](#) that represents a theme, it should do so with the [DynamicResource](#) markup extension. This ensures that if a different theme is selected at runtime, the values from the new theme will be applied.

The following example shows three styles from that can be applied to all [Label](#) objects in app:

XAML

```
<Application xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="ThemingDemo.App">
    <Application.Resources>

        <Style x:Key="LargeLabelStyle"
              TargetType="Label">
            <Setter Property="TextColor"
                  Value="{DynamicResource SecondaryTextColor}" />
            <Setter Property="FontSize"
                  Value="30" />
        </Style>

        <Style x:Key="MediumLabelStyle"
              TargetType="Label">
            <Setter Property="TextColor"
                  Value="{DynamicResource PrimaryTextColor}" />
            <Setter Property="FontSize"
                  Value="25" />
        </Style>

        <Style x:Key="SmallLabelStyle"
              TargetType="Label">
            <Setter Property="TextColor"
                  Value="{DynamicResource TertiaryTextColor}" />
            <Setter Property="FontSize"
                  Value="15" />
        </Style>

    </Application.Resources>
</Application>
```

These styles are defined in the app-level resource dictionary, so that they can be consumed by multiple pages. Each style consumes theme resources with the [DynamicResource](#) markup extension.

These styles are then consumed by pages:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:ThemingDemo"
             x:Class="ThemingDemo.UserSummaryPage"
             Title="User Summary"
             BackgroundColor="{DynamicResource PageBackgroundColor}">
    ...
    <ScrollView>
        <Grid>
            <Grid.RowDefinitions>
                <RowDefinition Height="200" />
                <RowDefinition Height="120" />
                <RowDefinition Height="70" />
            </Grid.RowDefinitions>
            <Grid BackgroundColor="{DynamicResource PrimaryColor}">
                <Label Text="Face-Palm Monkey"
                       VerticalOptions="Center"
                       Margin="15"
                       Style="{StaticResource MediumLabelStyle}" />
                ...
            </Grid>
            <StackLayout Grid.Row="1"
                       Margin="10">
                <Label Text="This monkey reacts appropriately to ridiculous
assertions and actions."
                       Style="{StaticResource SmallLabelStyle}" />
                <Label Text="    &#x2022; Cynical but not unfriendly."
                       Style="{StaticResource SmallLabelStyle}" />
                <Label Text="    &#x2022; Seven varieties of grimaces."
                       Style="{StaticResource SmallLabelStyle}" />
                <Label Text="    &#x2022; Doesn't laugh at your jokes."
                       Style="{StaticResource SmallLabelStyle}" />
            </StackLayout>
            ...
        </Grid>
    </ScrollView>
</ContentPage>
```

When a theme resource is consumed directly, it should be consumed with the [DynamicResource](#) markup extension. However, when a style that uses the [DynamicResource](#) markup extension is consumed, it should be consumed with the [StaticResource](#) markup extension.

For more information about styling, see [Style apps using XAML](#). For more information about the [DynamicResource](#) markup extension, see [Dynamic styles](#).

Load a theme at runtime

When a theme is selected at runtime, an app should:

1. Remove the current theme from the app. This is achieved by clearing the `MergedDictionaries` property of the app-level [ResourceDictionary](#).
2. Load the selected theme. This is achieved by adding an instance of the selected theme to the `MergedDictionaries` property of the app-level [ResourceDictionary](#).

Any [VisualElement](#) objects that set properties with the [DynamicResource](#) markup extension will then apply the new theme values. This occurs because the [DynamicResource](#) markup extension maintains a link to dictionary keys. Therefore, when the values associated with keys are replaced, the changes are applied to the [VisualElement](#) objects.

In the sample application, a theme is selected via a modal page that contains a [Picker](#). The following code shows the `OnPickerSelectionChanged` method, which is executed when the selected theme changes:

The following example shows removing the current theme and loading a new theme:

C#

```
ICollection<ResourceDictionary> mergedDictionaries =  
Application.Current.Resources.MergedDictionaries;  
if (mergedDictionaries != null)  
{  
    mergedDictionaries.Clear();  
    mergedDictionaries.Add(new DarkTheme());  
}
```

Respond to system theme changes

Article • 09/30/2024

 [Browse the sample](#)

Devices typically include light and dark themes, which each refer to a broad set of appearance preferences that can be set at the operating system level. Apps should respect these system themes, and respond immediately when the system theme changes.

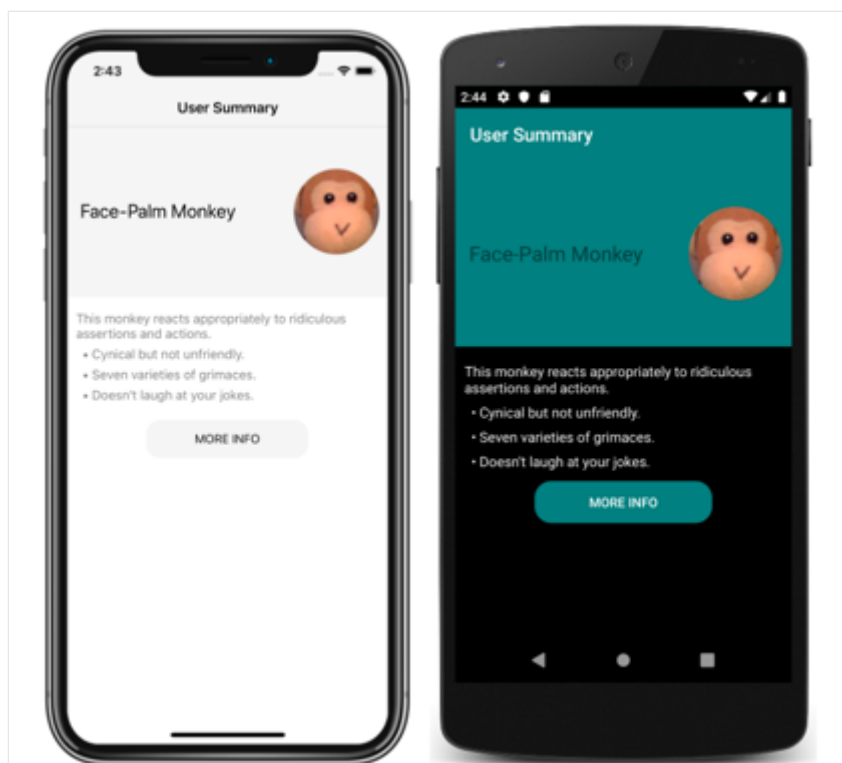
The system theme may change for a variety of reasons, depending on the device configuration. This includes the system theme being explicitly changed by the user, it changing due to the time of day, and it changing due to environmental factors such as low light.

.NET Multi-platform App UI (.NET MAUI) apps can respond to system theme changes by consuming resources with the [AppThemeBinding](#) markup extension, and the `SetAppThemeColor` and `SetAppTheme<T>` extension methods.

Note

.NET MAUI apps can respond to system theme changes on iOS 13 or greater, Android 10 (API 29) or greater, macOS 10.14 or greater, and Windows 10 or greater.

The following screenshot shows themed pages, for the light system theme on iOS and the dark system theme on Android:



Define and consume theme resources

Resources for light and dark themes can be consumed with the [AppThemeBinding](#) markup extension, and the `SetAppThemeColor` and `SetAppTheme<T>` extension methods. With these approaches, resources are automatically applied based on the value of the current system theme. In addition, objects that consume these resources are automatically updated if the system theme changes while an app is running.

AppThemeBinding markup extension

The [AppThemeBinding](#) markup extension enables you to consume a resource, such as an image or color, based on the current system theme.

The [AppThemeBinding](#) markup extension is supported by the [AppThemeBindingExtension](#) class, which defines the following properties:

- `Default`, of type `object`, that you set to the resource to be used by default.
- `Light`, of type `object`, that you set to the resource to be used when the device is using its light theme.
- `Dark`, of type `object`, that you set to the resource to be used when the device is using its dark theme.
- `Value`, of type `object`, that returns the resource that's currently being used by the markup extension.

❗ Note

The XAML parser allows the [AppThemeBindingExtension](#) class to be abbreviated as `AppThemeBinding`.

The `Default` property is the content property of [AppThemeBindingExtension](#). Therefore, for XAML markup expressions expressed with curly braces, you can eliminate the `Default=` part of the expression provided that it's the first argument.

The following XAML example shows how to use the [AppThemeBinding](#) markup extension:

XAML

```
<StackLayout>
    <Label Text="This text is green in light mode, and red in dark mode."
           TextColor="{AppThemeBinding Light=Green, Dark=Red}" />
    <Image Source="{AppThemeBinding Light=lightlogo.png, Dark=darklogo.png}"
           />
</StackLayout>
```

In this example, the text color of the first [Label](#) is set to green when the device is using its light theme, and is set to red when the device is using its dark theme. Similarly, the [Image](#) displays a different image file based upon the current system theme.

Resources defined in a [ResourceDictionary](#) can be consumed in an [AppThemeBinding](#) with the [StaticResource](#) markup extension:

XAML

```
<ContentPage ...>
    <ContentPage.Resources>

        <!-- Light colors -->
        <Color x:Key="LightPrimaryColor">WhiteSmoke</Color>
        <Color x:Key="LightSecondaryColor">Black</Color>

        <!-- Dark colors -->
        <Color x:Key="DarkPrimaryColor">Teal</Color>
        <Color x:Key="DarkSecondaryColor">White</Color>

        <Style x:Key="ButtonStyle"
               TargetType="Button">
            <Setter Property="BackgroundColor"
                   Value="{AppThemeBinding Light={StaticResource
LightPrimaryColor}, Dark={StaticResource DarkPrimaryColor}}" />
            <Setter Property="TextColor"
```

```

        Value="{AppThemeBinding Light={StaticResource
LightSecondaryColor}, Dark={StaticResource DarkSecondaryColor}}" />
    </Style>

</ContentPage.Resources>

<Grid BackgroundColor="{AppThemeBinding Light={StaticResource
LightPrimaryColor}, Dark={StaticResource DarkPrimaryColor}}">
    <Button Text="MORE INFO"
        Style="{StaticResource ButtonStyle}" />
</Grid>
</ContentPage>

```

In this example, the background color of the [Grid](#) and the [Button](#) style changes based on whether the device is using its light theme or dark theme.

In addition, resources defined in a [ResourceDictionary](#) can also be consumed in an [AppThemeBinding](#) with the [DynamicResource](#) markup extension:

XAML

```

<ContentPage ...>
    <ContentPage.Resources>
        <Color x:Key="Primary">DarkGray</Color>
        <Color x:Key="Secondary">HotPink</Color>
        <Color x:Key="Tertiary">Yellow</Color>
        <Style x:Key="labelStyle" TargetType="Label">
            <Setter Property="Padding" Value="5"/>
            <Setter Property="TextColor" Value="{AppThemeBinding Light=
{StaticResource Secondary}, Dark={StaticResource Primary}}" />
            <Setter Property="BackgroundColor" Value="{AppThemeBinding
Light={DynamicResource Primary}, Dark={DynamicResource Secondary}}" />
        </Style>
    </ContentPage.Resources>
    <Label x:Name="myLabel"
        Style="{StaticResource labelStyle}"/>
</ContentPage>

```

Extension methods

.NET MAUI includes `SetAppThemeColor` and `SetAppTheme<T>` extension methods that enable [VisualElement](#) objects to respond to system theme changes.

The `SetAppThemeColor` method enables [Color](#) objects to be specified that will be set on a target property based on the current system theme:

C#

```
Label label = new Label();  
label.SetAppThemeColor(Label.TextColorProperty, Colors.Green, Colors.Red);
```

In this example, the text color of the `Label` is set to green when the device is using its light theme, and is set to red when the device is using its dark theme.

The `SetAppTheme<T>` method enables objects of type `T` to be specified that will be set on a target property based on the current system theme:

C#

```
Image image = new Image();  
image.SetAppTheme<FileImageSource>(Image.SourceProperty, "lightlogo.png",  
"darklogo.png");
```

In this example, the `Image` displays `lightlogo.png` when the device is using its light theme, and `darklogo.png` when the device is using its dark theme.

Detect the current system theme

The current system theme can be detected by getting the value of the `Application.RequestedTheme` property:

C#

```
AppTheme currentTheme = Application.Current.RequestedTheme;
```

The `RequestedTheme` property returns an `AppTheme` enumeration member. The `AppTheme` enumeration defines the following members:

- `Unspecified`, which indicates that the device is using an unspecified theme.
- `Light`, which indicates that the device is using its light theme.
- `Dark`, which indicates that the device is using its dark theme.

Set the current user theme

The theme used by the app can be set with the `Application.UserAppTheme` property, which is of type `AppTheme`, regardless of which system theme is currently operational:

C#

```
Application.Current.UserAppTheme = AppTheme.Dark;
```

In this example, the app is set to use the theme defined for the system dark mode, regardless of which system theme is currently operational.

ⓘ Note

Set the `UserAppTheme` property to `AppTheme.Unspecified` to default to the operational system theme.

React to theme changes

The system theme on a device may change for a variety of reasons, depending on how the device is configured. .NET MAUI apps can be notified when the system theme changes by handling the `Application.RequestedThemeChanged` event:

C#

```
Application.Current.RequestedThemeChanged += (s, a) =>
{
    // Respond to the theme change
};
```

The `AppThemeChangedEventArgs` object, which accompanies the `RequestedThemeChanged` event, has a single property named `RequestedTheme`, of type `AppTheme`. This property can be examined to detect the requested system theme.

ⓘ Important

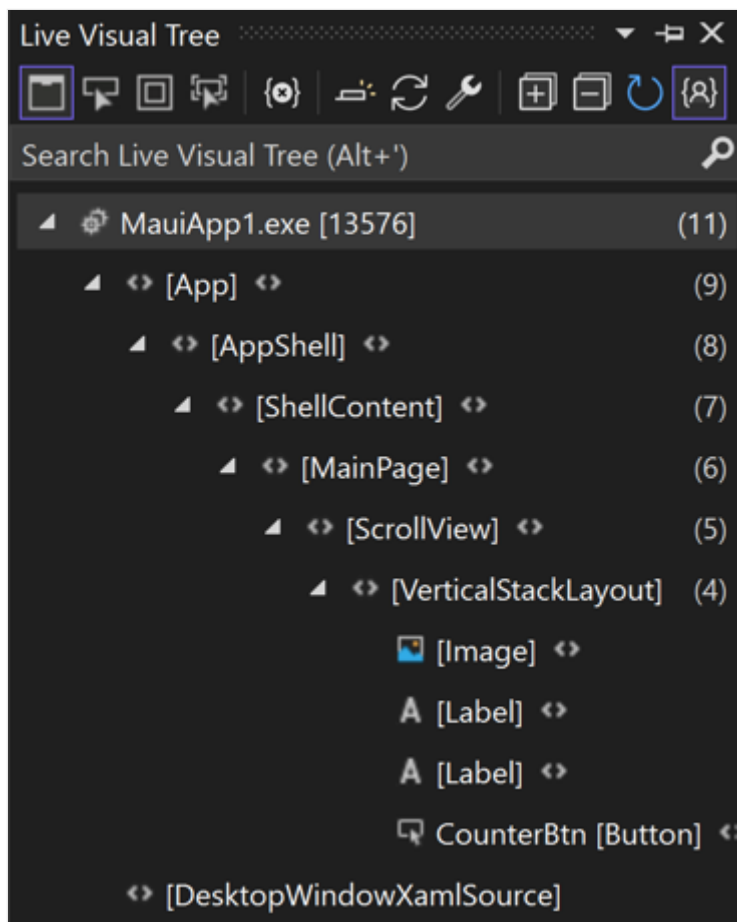
To respond to theme changes on Android your `MainActivity` class must include the `ConfigChanges.UiMode` flag in the `Activity` attribute. .NET MAUI apps created with the Visual Studio project templates automatically include this flag.

Inspect the visual tree of a .NET MAUI app

Article • 08/30/2024

.NET Multi-platform App UI (.NET MAUI) Live Visual Tree is a Visual Studio feature that provides a tree view of the UI elements in your running .NET MAUI app.

When your .NET MAUI app is running in debug configuration, with the debugger attached, the Live Visual Tree window can be opened by selecting **Debug > Windows > Live Visual Tree** from the Visual Studio menu bar:



Provided that Hot Reload is enabled, the **Live Visual Tree** window will display the hierarchy of your app's UI elements regardless of whether the app's UI is built using XAML or C#. However, you will have to disable Just My XAML to display the hierarchy of your app's UI elements for UIs built using C#.

Just My XAML

The view of the UI elements is simplified by default using a feature called Just My XAML. In Visual Studio, clicking the **Show Just My XAML** button disables the feature and shows

all UI elements in the visual tree:

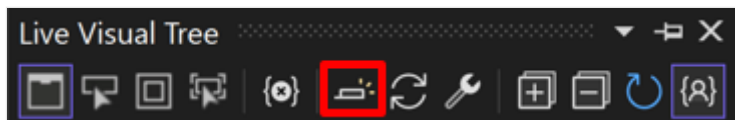


Just My XAML can be permanently disabled by selecting **Debug > Options > XAML Hot Reload** from the Visual Studio menu bar. Next, in the **Options** dialog box, ensure that **Enable Just My XAML in Live Visual Tree** is disabled:



When this mode is enabled, it causes the app window to show horizontal and vertical lines along the bounds of the selected object so you can see what it aligns to, as well as rectangles showing the margins.

- **Preview Selection.** You can enable this mode by clicking the **Preview Selected Item** button in the **Live Visual Tree** toolbar:



This mode shows the XAML source code where the element was declared, provided that you have access to the app source code.

Visual states

Article • 09/30/2024

The .NET Multi-platform App UI (.NET MAUI) Visual State Manager provides a structured way to make visual changes to the user interface from code. In most cases, the user interface of an app is defined in XAML, and this XAML can include markup describing how the Visual State Manager affects the visuals of the user interface.

The Visual State Manager introduces the concept of *visual states*. A .NET MAUI view such as a [Button](#) can have several different visual appearances depending on its underlying state — whether it's disabled, or pressed, or has input focus. These are the button's states. Visual states are collected in *visual state groups*. All the visual states within a visual state group are mutually exclusive. Both visual states and visual state groups are identified by simple text strings.

The .NET MAUI Visual State Manager defines a visual state group named `CommonStates` with the following visual states:

- Normal
- Disabled
- Focused
- Selected
- PointerOver

The `Normal`, `Disabled`, `Focused`, and `PointerOver` visual states are supported on all classes that derive from [VisualElement](#), which is the base class for [View](#) and [Page](#). In addition, you can also define your own visual state groups and visual states.

The advantage of using the Visual State Manager to define appearance, rather than accessing visual elements directly from code-behind, is that you can control how visual elements react to different state entirely in XAML, which keeps all of the UI design in one location.

ⓘ Note

Triggers can also make changes to visuals in the user interface based on changes in a view's properties or the firing of events. However, using triggers to deal with various combinations of these changes can become confusing. With the Visual State Manager, the visual states within a visual state group are always mutually exclusive. At any time, only one state in each group is the current state.

Common visual states

The Visual State Manager allows you to include markup in your XAML file that can change the visual appearance of a view if the view is normal, disabled, has input focus, is selected, or has the mouse cursor hovering over it but not pressed. These are known as the *common states*.

For example, suppose you have an [Entry](#) view on your page, and you want the visual appearance of the [Entry](#) to change in the following ways:

- The [Entry](#) should have a pink background when the [Entry](#) is disabled.
- The [Entry](#) should have a lime background normally.
- The [Entry](#) should expand to twice its normal height when it has input focus.
- The [Entry](#) should have a light blue background when it has the mouse cursor hovering over it but not pressed.

You can attach the Visual State Manager markup to an individual view, or you can define it in a style if it applies to multiple views.

Define visual states on a view

The `VisualStateManager` class defines a `VisualStateGroups` attached property, that's used to attach visual states to a view. The `VisualStateGroups` property is of type `VisualStateGroupList`, which is a collection of `VisualStateGroup` objects. Therefore, the child of the `VisualStateManager.VisualStateGroups` attached property is a `VisualStateGroup` object. This object defines an `x:Name` attribute that indicates the name of the group. Alternatively, the `VisualStateGroup` class defines a `Name` property that you can use instead. For more information about attached properties, see [Attached properties](#).

The `VisualStateGroup` class defines a property named `States`, which is a collection of [VisualState](#) objects. `States` is the content property of the `VisualStateGroups` class so you can include the [VisualState](#) objects as children of the `VisualStateGroup`. Each [VisualState](#) object should be identified using `x:Name` or `Name`.

The [VisualState](#) class defines a property named `Setters`, which is a collection of [Setter](#) objects. These are the same [Setter](#) objects that you use in a [Style](#) object. `Setters` isn't the content property of [VisualState](#), so it's necessary to include property element tags for the `Setters` property. [Setter](#) objects should be inserted as children of `Setters`. Each [Setter](#) object indicates the value of a property when that state is current. Any property referenced by a [Setter](#) object must be backed by a bindable property.

❗ Important

In order for visual state [Setter](#) objects to function correctly, a `VisualStateManager` must contain a [VisualState](#) object for the `Normal` state. If this visual state does not have any [Setter](#) objects, it should be included as an empty visual state (`<VisualState Name="Normal" />`).

The following example shows visual states defined on an [Entry](#):

XAML

```
<Entry FontSize="18">
  <VisualStateManager.VisualStateGroups>
    <VisualStateGroupList>
      <VisualStateGroup Name="CommonStates">
        <VisualState Name="Normal">
          <VisualState.Setters>
            <Setter Property="BackgroundColor" Value="Lime" />
          </VisualState.Setters>
        </VisualState>

        <VisualState Name="Focused">
          <VisualState.Setters>
            <Setter Property="FontSize" Value="36" />
          </VisualState.Setters>
        </VisualState>

        <VisualState Name="Disabled">
          <VisualState.Setters>
            <Setter Property="BackgroundColor" Value="Pink" />
          </VisualState.Setters>
        </VisualState>

        <VisualState Name="PointerOver">
          <VisualState.Setters>
            <Setter Property="BackgroundColor" Value="LightBlue" />
          </VisualState.Setters>
        </VisualState>
      </VisualStateGroup>
    </VisualStateGroupList>
  </VisualStateManager.VisualStateGroups>
</Entry>
```

The following screenshot shows the [Entry](#) in its four defined visual states:



When the [Entry](#) is in the `Normal` state, its background is lime. When the [Entry](#) gains input focus its font size doubles. When the [Entry](#) becomes disabled, its background becomes pink. The [Entry](#) doesn't retain its lime background when it gains input focus. When the mouse pointer hovers over the [Entry](#), but isn't pressed, the [Entry](#) background becomes light blue. As the Visual State Manager switches between the visual states, the properties set by the previous state are unset. Therefore, the visual states are mutually exclusive.

If you want the [Entry](#) to have a lime background in the `Focused` state, add another [Setter](#) to that visual state:

XAML

```
<VisualState Name="Focused">
    <VisualState.Setters>
        <Setter Property="FontSize" Value="36" />
        <Setter Property="BackgroundColor" Value="Lime" />
    </VisualState.Setters>
</VisualState>
```

Define visual states in a style

It's often necessary to share the same visual states in two or more views. In this scenario, the visual states can be defined in a [Style](#). This can be achieved by adding a [Setter](#) object for the `VisualStateManager.VisualStateGroups` property. The content property for the [Setter](#) object is its `Value` property, which can therefore be specified as the child of the [Setter](#) object. The `VisualStateGroups` property is of type `VisualStateGroupList`, and so the child of the [Setter](#) object is a `VisualStateGroupList` to which a `VisualStateGroup` can be added that contains [VisualState](#) objects.

The following example shows an implicit style for an [Entry](#) that defines the common visual states:

XAML

```
<Style TargetType="Entry">
    <Setter Property="FontSize" Value="18" />
    <Setter Property="VisualStateManager.VisualStateGroups">
        <VisualStateGroupList>
            <VisualStateGroup Name="CommonStates">
                <VisualState Name="Normal">
                    <VisualState.Setters>
                        <Setter Property="BackgroundColor" Value="Lime" />
                    </VisualState.Setters>
                </VisualState>
            </VisualStateGroup>
        </VisualStateGroupList>
    </Setter>
</Style>
```

```

        <VisualState Name="Focused">
            <VisualState.Setters>
                <Setter Property="FontSize" Value="36" />
                <Setter Property="BackgroundColor" Value="Lime" />
            </VisualState.Setters>
        </VisualState>
        <VisualState Name="Disabled">
            <VisualState.Setters>
                <Setter Property="BackgroundColor" Value="Pink" />
            </VisualState.Setters>
        </VisualState>
        <VisualState Name="PointerOver">
            <VisualState.Setters>
                <Setter Property="BackgroundColor" Value="LightBlue"
            />
            </VisualState.Setters>
        </VisualState>
    </VisualStateGroup>
</VisualStateGroupList>
</Setter>
</Style>

```

When this style is included in a page-level resource dictionary, the [Style](#) object will be applied to all [Entry](#) objects on the page. Therefore, all [Entry](#) objects on the page will respond in the same way to their visual states.

Visual states in .NET MAUI

The following table lists the visual states that are defined in .NET MAUI:

[Expand table](#)

Class	States	More Information
Button	Pressed	Button visual states
CarouselView	DefaultItem , CurrentItem , PreviousItem , NextItem	CarouselView visual states
CheckBox	IsChecked	CheckBox visual states
CollectionView	Selected	CollectionView visual states
ImageButton	Pressed	ImageButton visual states
RadioButton	Checked , Unchecked	RadioButton visual states
Switch	On , Off	Switch visual states

Class	States	More Information
VisualElement	Normal, Disabled, Focused, PointerOver	Common states

Set state on multiple elements

In the previous examples, visual states were attached to and operated on single elements. However, it's also possible to create visual states that are attached to a single element, but that set properties on other elements within the same scope. This avoids having to repeat visual states on each element the states operate on.

The [Setter](#) type has a `TargetName` property, of type `string`, that represents the target object that the [Setter](#) for a visual state will manipulate. When the `TargetName` property is defined, the [Setter](#) sets the `Property` of the object defined in `TargetName` to `Value`:

XAML

```
<Setter TargetName="label"
        Property="Label.TextColor"
        Value="Red" />
```

In this example, a [Label](#) named `label` will have its `TextColor` property set to `Red`. When setting the `TargetName` property you must specify the full path to the property in `Property`. Therefore, to set the `TextColor` property on a [Label](#), `Property` is specified as `Label.TextColor`.

ⓘ Note

Any property referenced by a [Setter](#) object must be backed by a bindable property.

The following example shows how to set state on multiple objects, from a single visual state group:

XAML

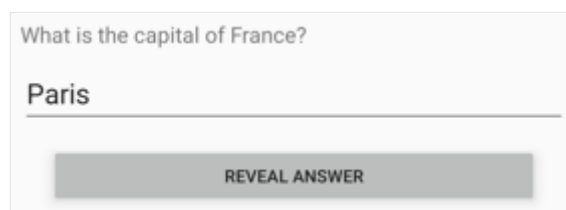
```
<StackLayout>
    <Label Text="What is the capital of France?" />
    <Entry x:Name="entry"
          Placeholder="Enter answer" />
    <Button Text="Reveal answer">
        <VisualStateManager.VisualStateGroups>
            <VisualStateGroup Name="CommonStates">
                <VisualState Name="Normal" />
            </VisualStateGroup>
        </VisualStateManager.VisualStateGroups>
    </Button>
</StackLayout>
```

```

        <VisualState Name="Pressed">
            <VisualState.Setters>
                <Setter Property="Scale"
                    Value="0.8" />
                <Setter TargetName="entry"
                    Property="Entry.Text"
                    Value="Paris" />
            </VisualState.Setters>
        </VisualState>
    </VisualStateManager.VisualStateGroups>
</Button>
</StackLayout>

```

In this example, the `Normal` state is active when the `Button` isn't pressed, and a response can be entered into the `Entry`. The `Pressed` state becomes active when the `Button` is pressed, and specifies that its `Scale` property will be changed from the default value of 1 to 0.8. In addition, the `Entry` named `entry` will have its `Text` property set to Paris. Therefore, the result is that when the `Button` is pressed it's rescaled to be slightly smaller, and the `Entry` displays Paris:



Then, when the `Button` is released it's rescaled to its default value of 1, and the `Entry` displays any previously entered text.

Important

Property paths are unsupported in `Setter` elements that specify the `TargetName` property.

Define custom visual states

Custom visual states can be implemented by defining them as you would define visual states for the common states, but with names of your choosing, and then calling the `VisualStateManager.GoToState` method to activate a state.

The following example shows how to use the Visual State Manager for input validation:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="VsmDemos.VsmValidationPage"
              Title="VSM Validation">
    <StackLayout x:Name="stackLayout"
                Padding="10, 10">
        <VisualStateManager.VisualStateGroups>
            <VisualStateGroup Name="ValidityStates">
                <VisualState Name="Valid">
                    <VisualState.Setters>
                        <Setter TargetName="helpLabel"
                            Property="Label.TextColor"
                            Value="Transparent" />
                        <Setter TargetName="entry"
                            Property="Entry.BackgroundColor"
                            Value="Lime" />
                    </VisualState.Setters>
                </VisualState>
                <VisualState Name="Invalid">
                    <VisualState.Setters>
                        <Setter TargetName="entry"
                            Property="Entry.BackgroundColor"
                            Value="Pink" />
                        <Setter TargetName="submitButton"
                            Property="Button.IsEnabled"
                            Value="False" />
                    </VisualState.Setters>
                </VisualState>
            </VisualStateGroup>
        </VisualStateManager.VisualStateGroups>
        <Label Text="Enter a U.S. phone number:"
              FontSize="18" />
        <Entry x:Name="entry"
              Placeholder="555-555-5555"
              FontSize="18"
              Margin="30, 0, 0, 0"
              TextChanged="OnTextChanged" />
        <Label x:Name="helpLabel"
              Text="Phone number must be of the form 555-555-5555, and not
begin with a 0 or 1" />
        <Button x:Name="submitButton"
              Text="Submit"
              FontSize="18"
              Margin="0, 20"
              VerticalOptions="Center"
              HorizontalOptions="Center" />
    </StackLayout>
</ContentPage>

```

In this example, visual states are attached to the `StackLayout`, and there are two mutually-exclusive states named `Valid` and `Invalid`. If the `Entry` does not contain a valid phone number, then the current state is `Invalid`, and so the `Entry` has a pink

background, the second [Label](#) is visible, and the [Button](#) is disabled. When a valid phone number is entered, then the current state becomes `Valid`. The [Entry](#) gets a lime background, the second [Label](#) disappears, and the [Button](#) is now enabled:

Enter a U.S. phone number:	Enter a U.S. phone number:
234-555-12	234-555-1234
Phone number must be of the form 555-555-5555, and not begin with a 0 or 1	
SUBMIT	SUBMIT
Invalid	Valid

The code-behind file is responsible for handling the `TextChanged` event from the [Entry](#). The handler uses a regular expression to determine if the input string is valid or not. The `GoToState` method in the code-behind file calls the static `VisualStateManager.GoToState` method on the [StackLayout](#) object:

C#

```
public partial class VsmValidationPage : ContentPage
{
    public VsmValidationPage()
    {
        InitializeComponent();

        GoToState(false);
    }

    void OnTextChanged(object sender, TextChangedEventArgs args)
    {
        bool isValid = Regex.IsMatch(args.NewTextValue, @"^[2-9]\d{2}-\d{3}-\d{4}$");
        GoToState(isValid);
    }

    void GoToState(bool isValid)
    {
        string visualState = isValid ? "Valid" : "Invalid";
        VisualStateManager.GoToState(stackLayout, visualState);
    }
}
```

In this example, the `GoToState` method is called from the constructor to initialize the state. There should always be a current state. The code-behind file then calls `VisualStateManager.GoToState`, with a state name, on the object that defines the visual states.

Visual state triggers

Visual states support state triggers, which are a specialized group of triggers that define the conditions under which a [VisualState](#) should be applied.

State triggers are added to the [StateTriggers](#) collection of a [VisualState](#). This collection can contain a single state trigger, or multiple state triggers. A [VisualState](#) will be applied when any state triggers in the collection are active.

When using state triggers to control visual states, .NET MAUI uses the following precedence rules to determine which trigger (and corresponding [VisualState](#)) will be active:

1. Any trigger that derives from [StateTriggerBase](#).
2. An [AdaptiveTrigger](#) activated due to the [MinWindowWidth](#) condition being met.
3. An [AdaptiveTrigger](#) activated due to the [MinWindowHeight](#) condition being met.

If multiple triggers are simultaneously active (for example, two custom triggers) then the first trigger declared in the markup takes precedence.

For more information about state triggers, see [State triggers](#).

Android app links

Article • 02/22/2024

It's often desirable to connect a website and a mobile app so that links on a website launch the mobile app and display content in the mobile app. *App linking*, which is also known as *deep linking*, is a technique that enables a mobile device to respond to a URI and launch content in a mobile app that's represented by the URI.

Android handles app links through the intent system. When you tap on a link in a mobile browser, the browser will dispatch an intent that Android will delegate to a registered app. These links can be based on a custom scheme, such as `myappname://`, or can use the HTTP or HTTPS scheme. For example, clicking on a link on a recipe website would open a mobile app that's associated with that website, and then display a specific recipe to the user. If there's more than one app registered to handle the intent, Android will display a disambiguation dialog that asks the user which app to select to handle the intent. Users who don't have your app installed are taken to content on your website.

Android classifies app links into three categories:

- *Deep links* are URIs of any scheme that take users to specific content in your app. When a deep link is clicked, a disambiguation dialog may appear that asks the user to select an app to handle the deep link.
- *Web links* are deep links that use the HTTP or HTTPS scheme. On Android 12 and higher, a web link always shows content in a web browser. On previous versions of Android, if an app can handle the web link then a disambiguation dialog will appear that asks the user to select an app to handle the web link.
- *Android app links*, which are available on API 23+, are web links that use the HTTP or HTTPS scheme and contain the `autoVerify` attribute. This attribute enables your app to become the default handler for an app link. Therefore, when an app link is clicked your app opens without displaying a disambiguation dialog.

.NET MAUI Android apps can support all three categories of app links. However, this article focuses on Android app links. This requires proving ownership of a domain as well as hosting a digital assets links file JSON file on the domain, which describes the relationship with your app. This enables Android to verify that the app trying to handle a URI has ownership of the URIs domain to prevent malicious apps from intercepting your app links.

The process for handling Android app links in a .NET MAUI Android app is as follows:

1. Verify domain ownership. For more information, see [Verify domain ownership](#).

2. Create and host a digital assets links file on your website. For more information, see [Create and host a digital assets links file](#).
3. Configure an intent filter in your app for the website URIs. For more information, see [Configure the intent filter](#).
4. Read the data from the incoming intent. For more information, see [Read the data from the incoming intent](#).

Important

To use Android app links:

- A version of your app must be live on Google Play.
- A companion website must be registered against the app in Google's Developer Console. Once the app is associated with a website, URIs can be indexed that work for both the website and the app, which can then be served in search results. For more information, see [App Indexing on Google Search](#)  on support.google.com.

For more information about Android app links, see [Handling Android App Links](#) .

Verify domain ownership

You'll be required to verify your ownership of the domain you're serving app links from in the [Google Search Console](#) . Ownership verification means proving that you own a specific website. Google Search Console supports multiple verification approaches. For more information, see [Verify your site ownership](#)  on support.google.com.

Create and host a digital assets links file

Android app links require that Android verify the association between the app and the website before setting the app as the default handler for the URI. This verification will occur when the app is first installed. The digital assets links file is a JSON file that must be hosted by the relevant web domain at the following location:

```
https://domain.name/.well-known/assetlinks.json.
```

The digital asset file contains the metadata necessary for Android to verify the association. The file requires the following key-value pairs:

- `namespace` - the namespace of the Android app.
- `package_name` - the package name of the Android app.

- `sha256_cert_fingerprints` - the SHA256 fingerprints of the signed app, obtained from your `.keystore` file. For information about finding your keystore's signature, see [Find your keystore's signature](#).

The following example *assetlinks.json* file grants link-opening rights to a `com.companyname.myrecipeapp` Android app:

JSON

```
[
  {
    "relation": [
      "delegate_permission/common.handle_all_urls"
    ],
    "target": {
      "namespace": "android_app",
      "package_name": "com.companyname.myrecipeapp",
      "sha256_cert_fingerprints": [
        "14:6D:E9:83:C5:73:06:50:D8:EE:B9:95:2F:34:FC:64:16:A0:83:42:E6:1D:BE:A8:8A:
        04:96:B2:3F:CF:44:E5"
      ]
    }
  }
]
```

It's possible to register more than one SHA256 fingerprint to support different versions or builds of your app. The following *assetlinks.json* file grants link-opening rights to both the `com.companyname.myrecipeapp` and `com.companyname.mycookingapp` Android apps:

JSON

```
[
  {
    "relation": [
      "delegate_permission/common.handle_all_urls"
    ],
    "target": {
      "namespace": "android_app",
      "package_name": "com.companyname.myrecipeapp",
      "sha256_cert_fingerprints": [
        "14:6D:E9:83:C5:73:06:50:D8:EE:B9:95:2F:34:FC:64:16:A0:83:42:E6:1D:BE:A8:8A:
        04:96:B2:3F:CF:44:E5"
      ]
    }
  },
  {
    "relation": [
      "delegate_permission/common.handle_all_urls"
    ],
    "target": {
      "namespace": "android_app",
      "package_name": "com.companyname.mycookingapp",
      "sha256_cert_fingerprints": [
        "14:6D:E9:83:C5:73:06:50:D8:EE:B9:95:2F:34:FC:64:16:A0:83:42:E6:1D:BE:A8:8A:
        04:96:B2:3F:CF:44:E5"
      ]
    }
  }
]
```

```

    ],
    "target": {
      "namespace": "android_app",
      "package_name": "com.companyname.mycookingapp",
      "sha256_cert_fingerprints": [

"14:6D:E9:83:C5:73:06:50:D8:EE:B9:95:2F:34:FC:64:16:A0:83:42:E6:1D:BE:A8:8A:
04:96:B2:3F:CF:44:E5"

      ]
    }
  }
]

```

Tip

Use the [Statement List Generator and Tester](#) tool to help generate the correct JSON, and to validate it.

When publishing your JSON verification file to `https://domain.name/.well-known/assetlinks.json`, you must ensure that:

- The file is served with content-type `application/json`.
- The file must be accessible over HTTPS, regardless of whether your app uses HTTPS as the scheme.
- The file must be accessible without redirects.
- If your app links support multiple domains, then you must publish the *assetlinks.json* file on each domain.

You can confirm that the digital assets file is properly formatted and hosted by using Google's digital asset links API:

HTML

```

https://digitalassetlinks.googleapis.com/v1/statements:list?source.web.site=
https://<WEB SITE
ADDRESS>:&relation=delegate_permission/common.handle_all_urls

```

For more information, see [Declare website associations](#) on developer.android.com.

Configure the intent filter

An intent filter must be configured that maps a URI, or set of URIs, from a website to an activity in your Android app. In .NET MAUI, this can be achieved by adding the

[IntentFilterAttribute](#) to your activity. The intent filter must declare the following information:

- [ActionView](#) - this will register the intent filter to respond to requests to view information.
- [Categories](#) - the intent filter should register both [CategoryDefault](#) and [CategoryBrowsable](#) to be able to correctly handle the web URI.
- [DataScheme](#) - the intent filter must declare a custom scheme, and/or HTTPS and/or HTTPS.
- [DataHost](#) - this is the domain from which URIs will originate.
- [DataPathPrefix](#) - this is an optional path to resources on the website, which must begin with a `/`.
- [AutoVerify](#) - this tells Android to verify the relationship between the app and the website. It must be set to `true` otherwise Android won't verify the association between the app and the website, and therefore won't set your app as the default handler for a URI.

The following example shows how to use the [IntentFilterAttribute](#) to handle links from `https://www.recipe-app.com/recipes`:

C#

```
using Android.App;
using Android.Content;
using Android.Content.PM;

namespace MyNamespace;

[Activity(
    Theme = "@style/Maui.SplashTheme",
    MainLauncher = true,
    ConfigurationChanges = ConfigChanges.ScreenSize |
        ConfigChanges.Orientation |
        ConfigChanges.UiMode |
        ConfigChanges.ScreenLayout |
        ConfigChanges.SmallestScreenSize |
        ConfigChanges.KeyboardHidden |
        ConfigChanges.Density)]
[IntentFilter(
    new string[] { Intent.ActionView },
    Categories = new[] { Intent.CategoryDefault, Intent.CategoryBrowsable },
    DataScheme = "https",
    DataHost = "recipe-app.com",
    DataPath = "/recipe",
    AutoVerify = true,)]
public class MainActivity : MauiAppCompatActivity
{
}
```

ⓘ Note

Multiple schemes and hosts can be specified in your intent filter. For more information, see [Create Deep Links to App Content](#) on developer.android.com.

Android will verify every host that's identified in the intent filters against the digital assets file on the website, before registering the app as the default handler for a URI. All the intent filters must pass verification before Android can establish the app as the default handler. Once you've added an intent filter with a URI for activity content, Android is able to route any intent that has matching URIs to your app at runtime.

It may also be necessary to mark your activity as exportable, so that your activity can be launched by other apps. This can be achieved by adding `Exported = true` to the existing [ActivityAttribute](#). For more information, see [Activity element](#) on developer.android.com.

When a web URI intent is invoked Android tries the following actions until the request succeeds:

1. Opens the preferred app to handle the URI.
2. Opens the only available app to handle the URI.
3. Allows the user to select an app to handle the URI.

For more information about intents and intent filters, see [Intents and intent filters](#) on developer.android.com.

Read the data from the incoming intent

When Android starts your activity through an intent filter, you can use the data provided by the intent to determine what to do. This should be performed in an early lifecycle delegate, ideally `OnCreate`. The `OnCreate` delegate is invoked when an activity is created. For more information about lifecycle delegates, see [Platform lifecycle events](#).

To respond to an Android lifecycle delegate being invoked, call the [ConfigureLifecycleEvents](#) method on the [MauiAppBuilder](#) object in the `CreateMauiapp` method of your `MauiProgram` class. Then, on the [ILifecycleBuilder](#) object, call the `AddAndroid` method and specify the [Action](#) that registers a handler for the required delegate:

C#


```

using Microsoft.Maui.LifecycleEvents;
using Microsoft.Extensions.Logging;

namespace MyNamespace;

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
            })
            .ConfigureLifecycleEvents(lifecycle =>
            {
#if ANDROID
                lifecycle.AddAndroid(android =>
                {
                    android.OnCreate((activity, bundle) =>
                    {
                        var action = activity.Intent?.Action;
                        var data = activity.Intent?.Data?.ToString();

                        if (action == Android.Content.Intent.ActionView &&
data is not null)
                            {
                                Task.Run(() => HandleAppLink(data));
                            }
                    });
                });
#endif

            });

#if DEBUG
            builder.Logging.AddDebug();
#endif

            return builder.Build();
        }

        static void HandleAppLink(string url)
        {
            if (Uri.TryCreate(url, UriKind.RelativeOrAbsolute, out var uri))
                App.Current?.SendOnAppLinkRequestReceived(uri);
        }
    }
}

```

The [Intent.Action](#) property retrieves the action associated with the incoming intent, and the [Intent.Data](#) property retrieves the data associated with the incoming intent. Provided that the intent action is set to [ActionView](#), the intent data can be passed to your `App` class with the [SendOnAppLinkRequestReceived](#) method.

Warning

App links offer a potential attack vector into your app, so ensure you validate all URI parameters and discard any malformed URIs.

In your `App` class, override the [OnAppLinkRequestReceived](#) method to receive and process the intent data:

```
C#

namespace MyNamespace;

public partial class App : Application
{
    ...

    protected override async void OnAppLinkRequestReceived(Uri uri)
    {
        base.OnAppLinkRequestReceived(uri);

        // Show an alert to test that the app link was received.
        await Dispatcher.DispatchAsync(async () =>
        {
            await Windows[0].Page!.DisplayAlert("App link received",
uri.ToString(), "OK");
        });

        Console.WriteLine("App link: " + uri.ToString());
    }
}
```

In the example above, the [OnAppLinkRequestReceived](#) override displays the app link URI. In practice, the app link should take users directly to the content represented by the URI, without any prompts, logins, or other interruptions. Therefore, the [OnAppLinkRequestReceived](#) override is the location from which to invoke navigation to the content represented by the URI.

Test an app link

Provided that the digital asset file is correctly hosted, you can use the Android Debug Bridge, `adb`, with the activity manager tool, `am`, to simulate opening a URI to ensure that your app links work correctly. For example, the following command tries to view a target app activity that's associated with a URI:

Console

```
adb shell am start -W -a android.intent.action.VIEW -c  
android.intent.category.BROWSABLE -d YOUR_URI_HERE
```

This command will dispatch an intent that Android should direct to your mobile app, which should launch and display the activity registered for the URI.

ⓘ Note

You can run `adb` against an emulator or a device.

In addition, you can display the existing link handling policies for the apps installed on a device:

Console

```
adb shell dumpsys package domain-preferred-apps
```

This command will display the following information:

- Package - the package name of the app.
- Domain - the domains, separated by spaces, whose web links will be handled by the app.
- Status - the current link handling status for the app. A value of `always` means that the app has set `AutoVerify` to `true` and has passed system verification. It's followed by a hexadecimal number representing the record of the preference.

For more information about the `adb` command, see [Android Debug Bridge](#).

In addition, you can manage and verify Android app links through the [Play Console](#).

For more information, see [Manage and verify Android App Links](#) on [developer.android.com](#).

For troubleshooting advice, see [Fix common implementation errors](#) on [developer.android.com](#).

Android app manifest

Article • 08/30/2024

Every .NET Multi-platform App UI (.NET MAUI) app on Android has an *AndroidManifest.xml* file, located in the *Platforms\Android* folder, that describes essential information about your app to build tools, the Android operating system, and Google Play.

The manifest file for your .NET MAUI Android app is generated as part of the .NET MAUI build process on Android. This build process takes the XML in the *Platforms\Android\AndroidManifest.xml* file, and merges it with any XML that's generated from specific attributes on your classes. The resulting manifest file can be found in the *obj* folder. For example, it can be found at *obj\Debug\net8.0-android\AndroidManifest.xml* for debug builds on .NET 8.

ⓘ Note

Visual Studio 17.6+ includes an editor that simplifies the process of specifying app details, the target Android version, and required permissions in an Android manifest file.

Generating the manifest

All .NET MAUI app's have a `MainActivity` class that derives from [Activity](#), via the `MauiAppCompatActivity` class, and that has the [ActivityAttribute](#) applied to it. Some apps may include additional classes that derive from [Activity](#) and that have the [ActivityAttribute](#) applied.

At build time, assemblies are scanned for non-`abstract` classes that derive from [Activity](#) and that have the [ActivityAttribute](#) applied. These classes and attributes are used to generate the app's manifest. For example, consider the following code:

C#

```
using Android.App;

namespace MyMauiApp;

public class MyActivity : Activity
{
}
```

This example results in nothing being generated in the manifest file. For an `<activity/>` element to be generated, you'd need to add the [ActivityAttribute](#):

C#

```
using Android.App;

namespace MyMauiApp;

[Activity]
public class MyActivity : Activity
{
}
```

This example causes the following XML fragment to be added to the manifest file:

XML

```
<activity android:name="crc64bdb9c38958c20c7c.MyActivity" />
```

ⓘ Note

[ActivityAttribute](#) has no effect on `abstract` types.

Activity name

The type name of an activity is based on the 64-bit cyclic redundancy check of the assembly-qualified name of the type being exported. This enables the same fully-qualified name to be provided from two different assemblies without receiving a packaging error.

To override this default and explicitly specify the name of your activity, use the [Name](#) property:

C#

```
using Android.App;

namespace MyMauiApp;

[Activity (Name="companyname.mymauiapp.activity")]
public class MyActivity : Activity
{
}
```

This example produces the following XML fragment:

XML

```
<activity android:name="companyname.mymauiapp.activity" />
```

ⓘ Note

You should only use the `Name` property for backward-compatibility reasons, as such renaming can slow down type lookup at runtime.

A typical scenario for setting the `Name` property is when you need to obtain a readable Java name for your activity. This can be useful if another Android app needs to be able to open your app, or if you have a script for launching your app and testing startup time.

Launch from the app chooser

If your .NET MAUI Android app contains multiple activities, and you need to specify which activity should be launchable from the app launcher, use the `MainLauncher` property:

C#

```
using Android.App;

namespace MyMauiApp;

[Activity (Label="My Maui App", MainLauncher = true)]
public class MyActivity : Activity
{
}

```

This example produces the following XML fragment:

XML

```
<activity android:label="My Maui App"
          android:name="crc64bdb9c38958c20c7c.MainActivity">
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>
```

Permissions

When you add permissions to an Android app, they're recorded in the manifest file. For example, if you set the `ACCESS_NETWORK_STATE` permission, the following element is added to the manifest file:

XML

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

The .NET MAUI app project template sets the `INTERNET` and `ACCESS_NETWORK_STATE` permissions in *Platforms\Android\AndroidManifest.xml*, because most apps require internet access. If you remove the `INTERNET` permission from your manifest, debug builds will still include the permission in the generated manifest file.

Tip

If you find that switching to a release build causes your app to lose a permission that was available in the debug build, verify that you've explicitly set the required permission in your manifest file.

Intent actions and features

The Android manifest file provides a way for you to describe the capabilities of your app. This is achieved via [Intents](#) and the [IntentFilterAttribute](#). You can specify which actions are appropriate for your activity with the [IntentFilterAttribute](#) constructor, and which categories are appropriate with the [Categories](#) property. At least one activity must be provided, which is why activities are provided in the constructor. An `[IntentFilter]` can be provided multiple times, and each use results in a separate `<intent-filter/>` element within the `<activity/>`:

C#

```
using Android.App;
using Android.Content;

namespace MyMauiApp;

[Activity(Label = "My Maui App", MainLauncher = true)]
[IntentFilter(new[] {Intent.ActionView},
    Categories = new[] {Intent.CategorySampleCode, "my.custom.category"})]
public class MyActivity : Activity
```

```
{  
}
```

This example produces the following XML fragment:

XML

```
<activity android:label="My Maui App"  
          android:name="crc64bdb9c38958c20c7c.MainActivity">  
  <intent-filter>  
    <action android:name="android.intent.action.MAIN" />  
    <category android:name="android.intent.category.LAUNCHER" />  
  </intent-filter>  
  <intent-filter>  
    <action android:name="android.intent.action.VIEW" />  
    <category android:name="android.intent.category.SAMPLE_CODE" />  
    <category android:name="my.custom.category" />  
  </intent-filter>  
</activity>
```

Application element

The Android manifest file also provides a way for you to declare properties for your entire app. This is achieved via the `<application>` element and its counterpart, the [ApplicationAttribute](#). Typically, you declare `<application>` properties for your entire app and then override these properties as required on an activity basis.

For example, the following `Application` attribute could be added to *MainApplication.cs* to indicate that the app's user-readable name is "My Maui App", and that it uses the `Maui.SplashTheme` style as the default theme for all activities:

C#

```
using Android.App;  
using Android.Runtime;  
  
namespace MyMauiApp;  
  
[Application(Label = "My Maui App", Theme = "@style/Maui.SplashTheme")]  
public class MainApplication : MauiApplication  
{  
    public MainApplication(IntPtr handle, JniHandleOwnership ownership)  
        : base(handle, ownership)  
    {  
    }  
  
    protected override MauiApp CreateMauiApp() =>  
        MauiProgram.CreateMauiApp();  
}
```



```
}
```

This declaration causes the following XML fragment to be generated in *obj\Debug\net8.0-android\AndroidManifest.xml*:

XML

```
<application android:label="MyMauiApp"
  android:theme="@style/Maui.SplashTheme" android:debuggable="true" ...>
```

ⓘ Note

Debug builds automatically set `android:debuggable="true"` so that debuggers and other tooling can attach to your app. However, it isn't set for release builds.

In this example, all activities in the app will default to the `Maui.SplashTheme` style. If you set an activity's theme to `Maui.MyTheme`, only that activity will use the `Maui.MyTheme` style while any other activities in your app will default to the `Maui.SplashTheme` style that's set in the `<application>` element.

The [ApplicationAttribute](#) isn't the only way to configure `<application>` attributes. You can also insert properties directly into the `<application>` element of the manifest file. These properties are then merged into the generated manifest file. For more information, see the properties section of [ApplicationAttribute](#).

ⓘ Important

The content of *Platforms\Android\AndroidManifest.xml* always overrides data provided by attributes.

App title bar

Android app's have a title bar that displays a label. The value of the `$(ApplicationTitle)` build property, in your .NET MAUI app project file, is displayed on the title bar. .NET MAUI includes it in the generated manifest as the value of `android.label` [↗](#):

XML

```
<application android:label="My Maui App" ... />
```

To specify an activities label on the title bar, use the [Label](#) property:

C#

```
using Android.App;

namespace MyMauiApp;

[Activity (Label="My Maui App")]
public class MyActivity : Activity
{
}
```

This example produces the following XML fragment:

XML

```
<activity android:label="My Maui App"
          android:name="crc64bdb9c38958c20c7c.MyActivity" />
```

App icon

By default, your app will be given a .NET icon. For information about specifying a custom icon, see [Change a .NET MAUI app icon](#).

Attributes

The following table shows the .NET for Android attributes that generate Android manifest XML fragments:

 Expand table

Attribute	Description
Android.App.ActivityAttribute	Generates an activity  XML fragment.
Android.App.ApplicationAttribute	Generates an application  XML fragment.
Android.App.InstrumentationAttribute	Generates an instrumentation  XML fragment.
Android.App.IntentFilterAttribute	Generates an intent-filter  XML fragment.

Attribute	Description
Android.App.MetadataAttribute	Generates a meta-data  XML fragment.
Android.App.PermissionAttribute	Generates a permission  XML fragment.
Android.App.PermissionGroupAttribute	Generates a permission-group  XML fragment.
Android.App.PermissionTreeAttribute	Generates a permission-tree  XML fragment.
Android.App.ServiceAttribute	Generates a service  XML fragment.
Android.App.UsesLibraryAttribute	Generates a uses-library  XML fragment.
Android.App.UsesPermissionAttribute	Generates a uses-permission  XML fragment.
Android.Content.BroadcastReceiverAttribute	Generates a receiver  XML fragment.
Android.Content.ContentProviderAttribute	Generates a provider  XML fragment.
Android.Content.GrantUriPermissionAttribute	Generates a grant-uri-permission  XML fragment.

See also

- [App Manifest Overview](#)  on developer.android.com

Android asset packs

Article • 08/20/2024

Assets such as video, audio, or graphics can occupy a lot of space inside your app package. For example, consider the following files in an example project:

```
MyProject.csproj
MainActivity.cs
Assets/
  MyLargeAsset.mp4
  MySmallAsset.json
AndroidManifest.xml
```

MyLargeAsset.mp4 and *MySmallAsset.json* will both be placed into the Android App Bundle (AAB) package when the app is published. However, if *MyLargeAsset.mp4* is greater than 200Mb it can't be placed into the main app bundle due to the package size restrictions enforced by Google Play.

.NET for Android 9 introduces the ability to place assets into separate packages, known as *asset packs*. This enables you to upload apps that would normally be larger than the basic package size allowed by Google Play. By putting these assets into a separate package you gain the ability to upload a package which is up to 2Gb in size, rather than the basic package size of 200Mb.

Important

Asset packs can only contain assets. In the case of .NET for Android this means items that have the `AndroidAsset` build action.

Asset pack metadata

To support asset packages in .NET for Android apps, the `AndroidAsset` item group supports two new metadata attributes:

- `AssetPack`. If present, this attribute controls which asset pack an asset is placed into. If not present, the asset will be placed into the main app bundle.

The name of the asset pack will be `$(AndroidPackage).%(AssetPack)`. Therefore, if your package name is `com.mycompany.myproject` and the `AssetPack` value is

`myassets`, the asset pack name would be `com.mycompany.myproject.myassets`.

ⓘ Note

The `AssetPack` value can also be `base`, which indicates that the asset should be placed into the main app bundle rather than an asset pack. For more information, see [Force an asset into the main app bundle](#).

- `DeliveryType`. If present, this attribute controls the type of asset pack is created. Valid values for this are `InstallTime`, `FastFollow`, and `OnDemand`. If not present, the default value is `InstallTime`. For more information, see [Asset delivery options](#).

Returning to the previous example, *MyLargeAsset.mp4* can be moved into its own asset pack by specifying the following `<ItemGroup>` element in the project's `.csproj` file:

XML

```
<ItemGroup>
  <AndroidAsset Update="Assets/MyLargeAsset.mp4" AssetPack="myassets" />
</ItemGroup>
```

This item group will cause the .NET for Android build system to create a new asset pack called `myassets` and include *MyLargeAsset.mp4* in the pack. The pack will automatically be included in the AAB file. There's no need to specify the delivery type, provided that the default value of `InstallTime` is the required delivery option.

ⓘ Note

This `AndroidAsset` is using the `Update` attribute rather than `Include` because .NET for Android supports auto-import of assets.

In scenarios where you have a large number of assets you can use wildcards to update the auto-imported assets:

XML

```
<ItemGroup>
  <AndroidAsset Update="Assets/*" AssetPack="myassets" />
</ItemGroup>
```

In this example, all of the assets in the `Assets` folder will be placed into an asset pack named `myassets`.

Force an asset into the main app bundle

In scenarios where you have a large number of assets, but you don't want all of them to be placed into an asset pack, you can specify an `AssetPack` value of `base` to force an asset into the main app bundle:

XML

```
<ItemGroup>
  <AndroidAsset Update="Assets/*" AssetPack="myassets" />
  <AndroidAsset Update="Assets/myimportantfile.json" AssetPack="base" />
</ItemGroup>
```

In this example, all of the assets in the `Assets` folder except `myimportantfile.json` will be placed into an asset pack named `myassets`. However, `myimportantfile.json` is placed into the main app bundle via the `AssetPack="base"` value, rather than the `myassets` asset pack.

Asset delivery options

Asset packs can have different delivery options, which control when your assets will install on the device:

- Install time packs are installed at the same time as the app. This pack type can be up to 1Gb in size, but you can only have one of them. This delivery type is specified with `InstallTime` metadata.
- Fast follow packs will install at some point shortly after the app has finished installing. The app will be able to start while this type of pack is being installed so you should check it has finished installing before trying to use the assets. This pack type can be up to 512Mb in size. This delivery type is specified with `FastFollow` metadata.
- On demand packs will never be downloaded to the device unless the app specifically requests it. This delivery type is specified with `OnDemand` metadata.

Important

The total size of all your asset packs can't exceed 2Gb, and you can have up to 50 separate asset packs.

For more information about asset delivery, see [Play Asset Delivery](https://developer.android.com/play-bundle-guide/asset-delivery) on developer.android.com.

Asset packs in .NET MAUI apps

.NET MAUI apps define assets via the `MauiAsset` build action. An asset pack can be specified via the `AssetPack` attribute:

XML

```
<MauiAsset
  Include="Resources\Raw\*"
  LogicalName="% (RecursiveDir)%(Filename)%(Extension) "
  AssetPack="myassetpack" />
```

❗ Note

The additional metadata will be ignored by other platforms.

If you have specific items you want to place in an asset pack you can use the `Update` attribute to define the `AssetPack` metadata:

XML

```
<MauiAsset Update="Resources\Raw\myvideo.mp4" AssetPack="myassets" />
```

In .NET MAUI apps, the delivery type can be specified with the `DeliveryType` attribute on a `MauiAsset`:

XML

```
<MauiAsset Update="Resources\Raw\myvideo.mp4" AssetPack="myassets"
  DeliveryType="FastFollow" />
```

Check the status of `FastFollow` asset packs

If your app uses a `FastFollow` asset pack, it'll need to check that the pack is installed before trying to access its contents.

To do this, add the [Xamarin.Google.Android.Play.Asset.Delivery](#) NuGet package to your project. This NuGet package provides access to the `AssetPackManager` type, which provides the ability to query the location of the asset pack:

C#

```

using Xamarin.Google.Android.Play.Core.AssetPacks;

var assetPackManager = AssetPackManagerFactory.GetInstance(this);
AssetPackLocation assetPackPath =
assetPackManager.GetPackLocation("myfastfollowpack");
string assetsFolderPath = assetPackPath?.AssetsPath() ?? null;
if (assetsFolderPath is null)
{
    // FastFollow asset pack isn't installed.
}

```

The location of the `FastFollow` asset pack is queried with the `GetPackLocation` method, which returns an `AssetPackLocation` object. If the `AssetsPath` method returns `null` for the `AssetPackLocation` object this indicates that the pack hasn't yet been installed. If the `AssetsPath` method returns a value, it represents the install location of the pack.

Download `OnDemand` asset packs

If your app uses an `OnDemand` asset pack, it'll need to download it manually. To do this, add the [Xamarin.Google.Android.Play.Asset.Delivery](#) NuGet package to your project. This NuGet package provides access to the `AssetPackStateUpdateListener` type, that enables you to monitor the progress of the download. However, in .NET this type is wrapped by the `AssetPackStateUpdateListenerWrapper` type, which can be used to register an event handler to monitor the download progress.

To monitor the download progress you'll need to declare some fields:

```

C#

using Xamarin.Google.Android.Play.Core.AssetPacks;

IAssetPackManager assetPackManager;
AssetPackStateUpdateListenerWrapper listener;

```

Then, declare an event handler to monitor the download:

```

C#

using Xamarin.Google.Android.Play.Core.AssetPacks.Model;

void Listener_StateUpdate(object? sender,
AssetPackStateUpdateListenerWrapper.AssetPackStateEventArgs e)
{
    var status = e.State.Status();
    switch (status)

```



```

{
    case AssetPackStatus.Downloading:
        long downloaded = e.State.BytesDownloaded();
        long totalSize = e.State.TotalBytesToDownload();
        double percent = 100.0 * downloaded / totalSize;
        Android.Util.Log.Info ("Listener_StateUpdate", $"Downloading
{percent}");
        break;
    case AssetPackStatus.Completed:
        break;
    case AssetPackStatus.WaitingForWifi:
        assetPackManager.ShowConfirmationDialog(this);
        break;
}
}

```

For information about which `AssetPackStatus` values to use, see [AssetPackStatus](#) on developer.android.com.

Then, create an `IAssetPackManager` instance via the `AssetPackManagerFactory.GetInstance` method and register the `Listener_StateUpdate` event handler against an `AssetPackStateUpdateListenerWrapper` object:

C#

```

assetPackManager = AssetPackManagerFactory.GetInstance (this);
listener = new AssetPackStateUpdateListenerWrapper();
listener.StateUpdate += Listener_StateUpdate;

```

You'll also need to register the listener when the app resumes, and unregister the listener when the app pauses:

C#

```

protected override void OnResume()
{
    assetPackManager.RegisterListener(listener.Listener);
    base.OnResume();
}

protected override void OnPause()
{
    assetPackManager.UnregisterListener(listener.Listener);
    base.OnPause();
}

```

Finally, before you download the `OnDemand` asset pack you'll need to check if it's already been installed. This can be accomplished with the `AssetPackManager.GetPackLocation`

method:

C#

```
using Android.Gms.Extensions;

var assetPackPath = assetPackManager.GetPackLocation ("myondemandpack");
string assetsFolderPath = assetPackPath?.AssetsPath() ?? null;
if (assetsFolderPath is null)
{
    await assetPackManager.Fetch(new string[] { "myondemandpack"
}).AsAsync<AssetPackStates>();
}
```

The location of the `OnDemand` asset pack is queried with the `GetPackLocation` method, which returns an `AssetPackLocation` object. If the `AssetsPath` method returns `null` for the `AssetPackLocation` object this indicates that the pack hasn't yet been installed.

`assetPackManager.Fetch` can then be called to start the download. If the `AssetsPath` method returns a value, it represents the install location of the pack.

ⓘ Note

The `AsAsync<T>` extension method returns a `Task<T>` object, and is available in the `Android.Gms.Extensions` namespace.

The status of the download can then be monitored via the `AssetPackStateUpdateListenerWrapper`.

Test asset packs locally

By default, .NET for Android uses the Android Application Package (APK) format for debugging. However, to test asset packs locally you'll need to ensure you're using the Android App Bundle (AAB) package format. To debug your asset packs update your `.csproj` file with the following build properties:

XML

```
<PropertyGroup Condition="'$(Configuration)' == 'Debug'">
  <AndroidPackageFormat>aab</AndroidPackageFormat>
  <EmbedAssembliesIntoApk>true</EmbedAssembliesIntoApk>
  <AndroidBundleToolExtraArgs>--local-testing</AndroidBundleToolExtraArgs>
</PropertyGroup>
```

The `--local-testing` flag is required for testing on your local device. It tells the `bundletool` app that all the asset packs should be installed in a cached location on the device. It also sets up the `IAssetPackManager` to use a mock downloader that will use the cache. This enables you to test installing `OnDemand` and `FastFollow` asset packs in a `Debug` environment.

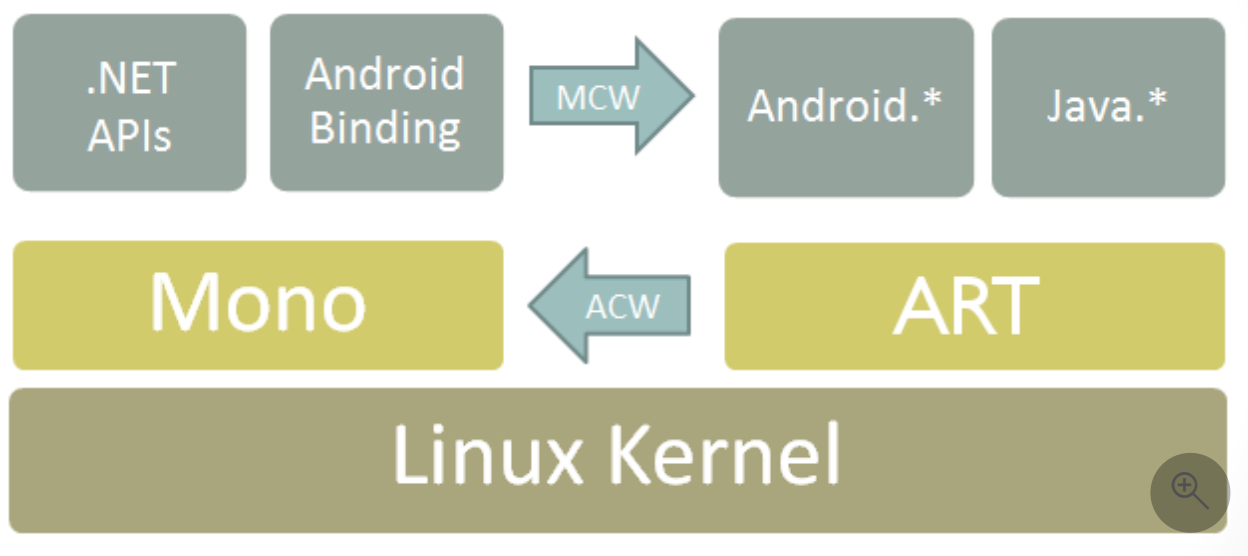
Binding Java libraries

Article • 09/16/2024

The third-party library ecosystem for Android is massive. Because of this, it frequently makes sense to use an existing Android library than to create a new one.

.NET for Android allows you to do this with a *Bindings Library* that automatically wraps the library with C# wrappers so you can invoke Java code via C# calls.

This is implemented by using *Managed Callable Wrappers (MCW)*. MCW is a JNI bridge that is used when managed code needs to invoke Java code. Managed callable wrappers also provide support for subclassing Java types and for overriding virtual methods on Java types. Likewise, whenever Android runtime (ART) code wishes to invoke managed code, it does so via another JNI bridge known as Android Callable Wrappers (ACW). This architecture is illustrated in the following diagram:



A Bindings Library is an assembly containing Managed Callable Wrappers for Java types. For example, here is a Java type, `MyClass`, that we want to wrap in a Bindings Library:

Java

```
package com.contoso.mycode;

public class MyClass
{
    public String myMethod (int i) { ... }
}
```

After we generate a Bindings Library for the `.jar` that contains `MyClass`, we can instantiate it and call methods on it from C#:

C#

```
var instance = new MyClass ();  
  
string result = instance.MyMethod (42);
```

To create this Bindings Library, you use the .NET for Android *Android Java Binding Library* template. The resulting binding project creates a .NET assembly with the MCW classes, **.jar/.aar** file(s), and resources for Android Library projects embedded in it. By referencing the resulting Bindings Library assembly, you can reuse an existing Java library in your .NET for Android project.

When you reference types in your Binding Library, you must use the namespace of your binding library. Typically, you add a `using` directive at the top of your C# source files that is the .NET namespace version of the Java package name. For example, if the Java package name for your bound **.jar** is the following:

C#

```
com.contoso.package
```

Then you would put the following `using` statement at the top of your C# source files to access types in the bound **.jar** file:

C#

```
using Com.Contoso.Package;
```

When binding an existing Android library, it is necessary to keep the following points in mind:

- **Are there any external dependencies for the library?** – Any Java dependencies required by the Android library must be included in the .NET for Android project via a NuGet package or as an **AndroidLibrary**. Any native assemblies must be added to the binding project as an **AndroidNativeLibrary**.
- **What version of the Android API does the Android library target?** – It is not possible to "downgrade" the Android API level; ensure that the .NET for Android binding project is targeting the same API level (or higher) as the Android library.

Adapting Java APIs to C#

The .NET for Android Binding Generator will change some Java idioms and patterns to correspond to .NET patterns. The following list describes how Java is mapped to C#/.NET:

- *Setter/Getter methods* in Java are *Properties* in .NET.
- *Fields* in Java are *Properties* in .NET.
- *Listeners/Listener Interfaces* in Java are *Events* in .NET. The parameters of methods in the callback interfaces will be represented by an `EventArgs` subclass.
- A *Static Nested class* in Java is a *Nested class* in .NET.
- An *Inner class* in Java is a *Nested class* with an instance constructor in C#.

Including a native library in a binding

It may be necessary to include a `.so` library in a .NET for Android binding project as a part of binding a Java library. When the wrapped Java code executes, .NET for Android will fail to make the JNI call and the error message `java.lang.UnsatisfiedLinkError: Native method not found:` will appear in the logcat out for the application.

The fix for this is to manually load the `.so` library with a call to

`Java.Lang.JavaSystem.LoadLibrary`. For example assuming that a .NET for Android project has shared library `libpocketsphinx_jni.so` included in the binding project with a build action of **AndroidNativeLibrary**, the following snippet (executed before using the shared library) will load the `.so` library:

C#

```
Java.Lang.JavaSystem.LoadLibrary("pocketsphinx_jni");
```

Binding scenarios

The following binding scenario guides can help you bind a Java library (or libraries) for incorporation into your app:

- [Binding a Java library](#) is a walkthrough for creating Bindings Libraries for locally available `.jar/.aar` files.
- [Binding a Java library from Maven](#) is a walkthrough for creating Bindings Libraries for `.jar/.aar` files that are hosted in a Maven repository.

- [Customizing bindings](#) explains how to make manual modifications to the binding to resolve build errors and shape the resulting API so that it is more "C#-like".
 - [Troubleshooting bindings](#) lists common binding error scenarios, explains possible causes, and offers suggestions for resolving these errors.
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Binding a Java library

Article • 09/16/2024

The Android community offers many Java libraries that you may want to use in your app. These Java libraries are often packaged in .JAR (Java Archive) or .AAR (Android Archive) format, but you can package a .JAR/.AAR in a *Java Bindings Library* so that its functionality is available to .NET for Android applications. The purpose of the Java Bindings library is to make the APIs in the .JAR/.AAR file available to C# code through automatically-generated code wrappers.

💡 Tip

.NET 9 introduces support for automatically downloading and binding a Java library from a Maven repository. See the [Binding a Java Library from Maven](#) documentation to simplify this scenario.

Walkthrough

In this walkthrough, we are going to bind version `3.1.0` of `CircleImageView`, a library that displays an image in a circular canvas.

From the [Maven repository](#), download `circleimageview-3.1.0.aar` locally to be bound.

Creating the bindings library

First, create a new Bindings Library project. This can be done with the "Android Java Binding Library" project template available in Visual Studio or via the `dotnet` command line with:

```
.NET CLI
```

```
dotnet new android-bindinglib
```

Copy the `circleimageview-3.1.0.aar` file into the project directory.

Like [.NET SDK style projects](#), .NET for Android binding projects automatically include any .JAR/.AAR files in the project directory as an `AndroidLibrary` type file, so no additional configuration is needed.

Now build the project using Visual Studio's Build command, or from the command line:

```
.NET CLI
```

```
dotnet build
```

This Java library has now been bound and it ready to be referenced by a .NET for Android application project or published to NuGet for public consumption.

Additional options

Skip managed bindings

By default, C# bindings will be created for any .JAR/.AAR placed in the project. However C# bindings can be tricky to create and are not necessary if you do not intend to call the Java API from C#.

This is especially the case when the Java library is simply a dependency of another Java library and does not need to be called from C# directly. In this case, the `Bind="false"` attribute can be used to only include the Java dependency but not bind it.

```
XML
```

```
<ItemGroup>
  <AndroidLibrary Update="circleimageview-3.1.0.aar" Bind="false" />
</ItemGroup>
```

Note if using automatic imports you will need to use `Update` to change the automatically imported file instead of adding an additional copy with `Include`.

Java dependencies

A Java library may depend on other Java libraries which will be required to be packaged with your application in order for your application to work. This information is traditionally provided in a .POM file, and it is your responsibility to ensure that any needed dependencies are properly referenced, usually via a NuGet package or by bundling the needed .JAR/.AAR files in your project.

In .NET 9, the Java Dependency Verification feature was added. By providing the .POM file, the binding tooling can help ensure you have met all required Java dependencies.

To enable Java Dependency Verification for our walkthrough, ensure you are using .NET 9+ and your project targets `net9.0-android` or greater.

From the [Maven repository](#), download `circleimageview-3.1.0.pom` locally and place it in your project folder. Note that .POM files will not get detected automatically because they need to be associated with the correct .JAR/.AAR.

Update the automatically imported `AndroidLibrary` to specify the location of the .POM file with `Manifest` attribute. Additionally, specify the `JavaArtifact` and `JavaVersion` of the Java library:

XML

```
<!-- JavaArtifact format is {GroupId}:{ArtifactId} -->
<ItemGroup>
  <AndroidLibrary
    Update="circleimageview-3.1.0.aar"
    Manifest="circleimageview-3.1.0.pom"
    JavaArtifact="de.hdodenhof:circleimageview"
    JavaVersion="3.1.0" />
</ItemGroup>
```

This library is trivial and does not have any Java dependencies, but if it did and they were unmet, error like this would be emitted:

.NET CLI

```
error XA4241: Java dependency 'androidx.collection:collection:1.0.0' is not
satisfied.
error XA4242: Java dependency 'org.jetbrains.kotlin:kotlin-stdlib:1.9.0' is
not satisfied. Microsoft maintains the NuGet package 'Xamarin.Kotlin.StdLib'
that could fulfill this dependency.
```

Additional information on configuring Java Dependency Verification and how to satisfy dependencies can be found in the [documentation](#).

Next steps

- **Customizing Bindings with Metadata** - The Java library bound in the walkthrough is trivial and the binding tooling was able to fully automatically convert it to a C# API. Unfortunately this is often not the case and there will often be compile errors. These errors must be fixed with "metadata" to manually instruct the binding tooling how to resolve differences between the Java and C# languages.

- **Changing Namespaces** - The types in the walkthrough end up in the .NET namespace `DE.Hdodenhof.Circleimageview`. Java package names tend to be more verbose than .NET namespaces and it may be more desirable to change it, for example to `CircleImageViewLibrary` using an `AndroidNamespaceReplacement`:

XML

```
<ItemGroup>
  <AndroidNamespaceReplacement Include='DE.Hdodenhof.Circleimageview'
Replacement='CircleImageViewLibrary' />
</ItemGroup>
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Binding a Java library from Maven

Article • 09/16/2024

A common binding scenario is binding a Java library hosted in a Maven repository (like [Maven Central](#)).

.NET 9 introduces support for automatically downloading a Java library from a Maven repository and verifying its dependencies to help make this scenario easier and more accurate.

💡 Tip

If using a .NET version before .NET 9 or binding a Java library that isn't from Maven, see the [Binding a Java Library](#) documentation.

Walkthrough

In this walkthrough, we are going to bind version `3.1.0` of `CircleImageView`, a library that displays an image in a circular canvas.

From the [Maven repository](#), we can see the following identifiers for this library which will be needed later:

XML

```
<dependency>
  <groupId>de.hdodenhof</groupId>
  <artifactId>circleimageview</artifactId>
  <version>3.1.0</version>
</dependency>
```

Creating the bindings library

First, create a new Bindings Library project. This can be done with the "Android Java Binding Library" project template available in Visual Studio or via the `dotnet` command line with:

.NET CLI

```
dotnet new android-bindinglib
```

Open the project file (`.csproj`) created by the template. We'll add an `AndroidMavenLibrary` element inside an `ItemGroup` to specify the Java library we want to bind:

XML

```
<!-- Include format is {GroupId}:{ArtifactId} -->
<ItemGroup>
  <AndroidMavenLibrary Include="de.hdodenhof:circleimageview"
Version="3.1.0" />
</ItemGroup>
```

Now build the project using Visual Studio's Build command, or from the command line:

.NET CLI

```
dotnet build
```

This Java library has now been bound and it ready to be referenced by a .NET for Android application project or published to NuGet for public consumption.

Additional options

Skip managed bindings

By default, C# bindings will be created for any .JAR/.AAR placed in the project. However C# bindings can be tricky to create and are not necessary if you do not intend to call the Java API from C#.

This is especially the case when the Java library is simply a dependency of another Java library and does not need to be called from C# directly. In this case, the `Bind="false"` attribute can be used to only include the Java dependency but not bind it.

XML

```
<ItemGroup>
  <AndroidMavenLibrary Include="de.hdodenhof:circleimageview"
Version="3.1.0" Bind="false" />
</ItemGroup>
```

Next steps

- **AndroidMavenLibrary Options** - The walkthrough library was automatically downloaded from Maven Central which is the default repository. Other Maven repositories and options can be specified.
- **Java Dependency Verification** - The Java library bound in the walkthrough is trivial and did not depend on any other Java packages. Most libraries will depend on other Java packages and errors will be surfaced to ensure these dependencies can be resolved.

These errors must be fixed before the binding can build, and look like:

.NET CLI

```
error XA4241: Java dependency 'androidx.collection:collection:1.0.0' is not
satisfied.
error XA4242: Java dependency 'org.jetbrains.kotlin:kotlin-stdlib:1.9.0' is
not satisfied. Microsoft maintains the NuGet package 'Xamarin.Kotlin.StdLib'
that could fulfill this dependency.
```

- **Customizing Bindings with Metadata** - The Java library bound in the walkthrough is trivial and the binding tooling was able to fully automatically convert it to a C# API. Unfortunately this is often not the case and there will often be compile errors. These errors must be fixed with "metadata" to manually instruct the binding tooling how to resolve differences between the Java and C# languages.
- **Changing Namespaces** - The types in the walkthrough end up in the .NET namespace `DE.Hdodenhof.Circleimageview`. Java package names tend to be more verbose than .NET namespaces and it may be more desirable to change it, for example to `CircleImageViewLibrary` using an `AndroidNamespaceReplacement`:

XML

```
<ItemGroup>
  <AndroidNamespaceReplacement Include='DE.Hdodenhof.Circleimageview'
Replacement='CircleImageViewLibrary' />
</ItemGroup>
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Customizing bindings

Article • 09/16/2024

.NET for Android automates much of the binding process; however, C# and Java are different languages that do not support exactly the same features, and thus there are cases where manual modification is required to fix differences that cannot be resolved automatically.

Some examples of these issues are:

- Resolving build errors caused by missing types, obfuscated types, duplicate names, class visibility issues, and other situations that cannot be resolved by the .NET for Android tooling.
- Removing unused types that do not need to be bound.
- Adding types that have no counterpart in the underlying Java API.

Additionally it may be desirable to make some ergonomic customizations to make bindings more pleasant to use, like:

- Changing the namespace containing the bound types.

You can make some or all of these changes by modifying the metadata that controls the binding process.

Guides

The following guides describe the metadata that controls the binding process and explain how to modify this metadata to address these issues:

- [Java Bindings Metadata](#) provides an overview of the metadata that goes into a Java binding. It describes the various manual steps that are sometimes required to complete a Java binding library, and it explains how to shape an API exposed by a binding to more closely follow .NET design guidelines.
 - [Namespace Customization](#) explains how to customize the namespace(s) that bound types are placed in.
 - [Creating Enumerations](#) explains how to map collections of Java integer constants into .NET enumerations.
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Customizing namespaces

Article • 09/16/2024

Java package names often do not match C# namespace conventions due to issues such as extra prefixes and capitalization differences.

For example, the Java package name `com.microsoft.streamwriters` would be automatically translated to `Com.Microsoft.Streamwriters` because the binding process automatically translates namespaces to Pascal case. However a better fit would be `Microsoft.StreamWriters`.

This can be accomplished by adding an `<AndroidNamespaceReplacement>` item to the project file:

XML

```
<ItemGroup>
  <AndroidNamespaceReplacement Include='com.microsoft.streamwriters'
  Replacement='Microsoft.StreamWriters' />
</ItemGroup>
```

Although Java packages can also be renamed using traditional `metadata`, it can be tedious to rename many similar packages:

XML

```
<attr path="/api/package[@name='com.microsoft.accessibility']"
name="managedName">Microsoft.Accessibility</attr>
<attr path="/api/package[@name='com.microsoft.content']"
name="managedName">Microsoft.Content</attr>
<attr path="/api/package[@name='com.microsoft.core']"
name="managedName">Microsoft.Core</attr>
...
```

With `<AndroidNamespaceReplacement>`, the same MSBuild transform placed in the project file can be applied to all matching packages:

XML

```
<ItemGroup>
  <AndroidNamespaceReplacement Include='com.microsoft'
  Replacement='Microsoft' />
</ItemGroup>
```

Specification

These replacements will **only** be run for `<package>` elements that do not specify a `metadata @managedName` attribute.

If `@managedName` is used, you are opting to provide the exact desired name, it will not be processed further.

Unlike unused metadata, these replacement will not raise a warning if they are unused.

Case sensitivity

Replacements run **after** the automatic Pascal case transform, but the compare is case-insensitive.

Thus, both of the following are equivalent:

XML

```
<AndroidNamespaceReplacement Include='Androidx' Replacement='AndroidX' />
<AndroidNamespaceReplacement Include='androidx' Replacement='AndroidX' />
```

Word bounds

Replacements take place **only** on full words (namespace parts).

Thus,

XML

```
<AndroidNamespaceReplacement Include='Com' Replacement='' />
```

Matches matches `Com.Google.Library`, but not `Common.Google.Library` or `Google.Imaging.Dicom`.

Multiple full words can be used:

XML

```
<AndroidNamespaceReplacement Include='Com.Google' Replacement='Google' />
<AndroidNamespaceReplacement Include='Com.Androidx'
Replacement='Microsoft.AndroidX' />
```

Word position

The word part match can be constrained to the beginning or end of a namespace by appending a `.` or prepending a `.`, respectively.

XML

```
<AndroidNamespaceReplacement Include='Androidx.'  
Replacement='Microsoft.AndroidX' />
```

matches `Androidx.Core`, but not `Square.OkHttp.Androidx`.

Similarly,

XML

```
<AndroidNamespaceReplacement Include='.Compose' Replacement='ComposeUI' />
```

matches `Google.AndroidX.Compose`, but not `Google.Compose.Writer`.

Both prepending and appending a `.` makes it an exact match.

XML

```
<AndroidNamespaceReplacement Include='.Androidx.'  
Replacement='Microsoft.AndroidX' />
```

matches `Androidx`, but not `Google.Androidx.Core`.

Replacement order

Replacements run in the order specified by the `<ItemGroup>`, however adding to this group at different times may result in an unintended order.

Replacements are run sequentially, and multiple replacements may affect a single namespace.

XML

```
<AndroidNamespaceReplacement Include='Androidx'  
Replacement='Microsoft.AndroidX' />  
<AndroidNamespaceReplacement Include='View' Replacement='Views' />
```

changes `Androidx.View` to `Microsoft.AndroidX.Views`.

Relation to metadata

Technically `@(AndroidNamespaceReplacement)` is implemented as a special case of `<metadata>`, and can be placed directly in a `metadata.xml` file in metadata form if desired.

That is, the MSBuild item:

XML

```
<ItemGroup>
  <AndroidNamespaceReplacement Include='com.microsoft'
  Replacement='Microsoft' />
</ItemGroup>
```

could instead be placed in `metadata.xml` as:

XML

```
<metadata>
  <ns-replace source='com.microsoft' replacement='Microsoft' />
</metadata>
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Java bindings metadata

Article • 09/16/2024

A .NET for Android **Java Binding Library** tries to automate much of the work necessary for binding an existing Android library with the help of a tool sometimes known as the *Bindings Generator*. When binding a Java library, .NET for Android will inspect the Java classes and generate a list of all the packages, types, and members which to be bound. This list of APIs is stored in an XML file that can be found at `{project directory}\obj{Configuration}\api.xml`.

Name	Date Modified	Size	Kind
► Additions	Mar 31, 2016, 2:36 PM	--	Folder
► bin	Apr 6, 2016, 10:47 AM	--	Folder
evernote-android-job.csproj	Jun 23, 2016, 12:37 PM	4 KB	Xamari...Project
► Jars	Apr 1, 2016, 10:55 AM	--	Folder
▼ obj	Jun 23, 2016, 12:55 PM	--	Folder
▼ Debug	Jun 23, 2016, 12:44 PM	--	Folder
__AndroidLibraryProjects__.zip	Jun 23, 2016, 12:44 PM	84 KB	ZIP archive
► __library_projects__	Jun 23, 2016, 12:44 PM	--	Folder
api.xml	Jun 23, 2016, 12:44 PM	103 KB	XML Document
evernote-android-job.csproj.FilesWrittenAbsolute.txt	Jun 23, 2016, 12:44 PM	1 KB	Plain Text
evernote-android-job.dll	Jun 23, 2016, 12:44 PM	176 KB	Microso...library
evernote-android-job.dll.mdb	Jun 23, 2016, 12:44 PM	23 KB	Document
evernoteandroidjob.Jars.cat-1.0.3.jar	Feb 22, 2016, 12:33 PM	12 KB	Java JAR file
evernoteandroidjob.obj.Debug. __AndroidLibraryProjects__.zip	Jun 23, 2016, 12:44 PM	84 KB	ZIP archive

The Bindings Generator will use the `api.xml` file as a guideline for generating the necessary C# wrapper classes. The following snippet is an example of the contents of `api.xml`:

XML

```
<api>
  <package name="android">
    <class abstract="false" deprecated="not deprecated"
extends="java.lang.Object"
    extends-generic-aware="java.lang.Object"
    final="true"
    name="Manifest"
    static="false"
    visibility="public">
      <constructor deprecated="not deprecated" final="false"
        name="Manifest" static="false" type="android.Manifest"
        visibility="public">
      </constructor>
    </class>
  ...
</api>
```

In this example, `api.xml` declares a class in the `android` package named `Manifest` that extends the `java.lang.Object`.

In many cases, human assistance is required to make the Java API feel more ".NET like" or to correct issues that prevent the binding assembly from compiling. For example, it may be necessary to change Java package names to .NET namespaces, rename a class, or change the return type of a method.

These changes should not be achieved by modifying **api.xml** directly. Instead, changes are recorded in special XML files that are provided by the Java Binding Library template. When compiling the .NET for Android binding assembly, the Bindings Generator will be influenced by these mapping files when creating the binding assembly

The **Metadata.xml** file is the most important of these files as it allows general-purpose changes to the binding such as:

- Renaming namespaces, classes, methods, or fields so they follow .NET conventions.
- Removing namespaces, classes, methods, or fields that aren't needed.
- Moving classes to different namespaces.
- Adding additional support classes to make the design of the binding follow .NET framework patterns.

Metadata.xml transform file

As we've already learned, the file **Metadata.xml** is used by the Bindings Generator to influence the creation of the binding assembly. The metadata format uses [XPath](#) syntax.

This implementation is almost a complete implementation of XPath 1.0 and thus supports items in the 1.0 standard. This file is a powerful XPath based mechanism to change, add, hide, or move any element or attribute in the API file. All of the rule elements in the metadata spec include a **path** attribute to identify the node(s) to which the rule is to be applied. The following are the available element types:

- **add-node** – Appends a child node to the node specified by the path attribute.
- **attr** – Sets the value of an attribute of the element specified by the path attribute.
- **remove-node** – Removes nodes matching a specified XPath.

The following is an example of a **Metadata.xml** file:

XML

```
<metadata>
  <!-- Normalize the namespace for .NET -->
  <attr path="/api/package[@name='com.evernote.android.job']">
```

```

        name="managedName">Evernote.AndroidJob</attr>

        <!-- Don't need these packages for the .NET for Android binding/public
API -->
        <remove-node path="/api/package[@name='com.evernote.android.job.v14']"
/>
        <remove-node path="/api/package[@name='com.evernote.android.job.v21']"
/>

        <!-- Change a parameter name from the generic p0 to a more meaningful
one. -->
        <attr
path="/api/package[@name='com.evernote.android.job']/class[@name='JobManager
']/method[@name='forceApi']/parameter[@name='p0']"
        name="name">api</attr>
</metadata>

```

The following lists some of the more commonly used XPath elements for the Java API's:

- `interface` – Used to locate a Java interface. e.g.
`/interface[@name='AuthListener']`.
- `class` – Used to locate a class . e.g. `/class[@name='MapView']`.
- `method` – Used to locate a method on a Java class or interface. e.g.
`/class[@name='MapView']/method[@name='setTitleSource']`.
- `parameter` – Identify a parameter for a method. e.g. `/parameter[@name='p0']`

Adding types

The `add-node` element will tell the .NET for Android binding project to add a new class to `api.xml`. For example, the following snippet will direct the Binding Generator to create a class with a constructor and a single field:

XML

```

<add-node path="/api/package[@name='org.alljoyn.bus']">
    <class abstract="false" deprecated="not deprecated" final="false"
name="AuthListener.AuthRequest" static="true" visibility="public"
extends="java.lang.Object">
        <constructor deprecated="not deprecated" final="false"
name="AuthListener.AuthRequest" static="false"
type="org.alljoyn.bus.AuthListener.AuthRequest" visibility="public" />
        <field name="p0" type="org.alljoyn.bus.AuthListener.Credentials" />
    </class>
</add-node>

```

Removing types

It is possible to instruct the .NET for Android Bindings Generator to ignore a Java type and not bind it. This is done by adding a `remove-node` XML element to the **Metadata.xml** file:

XML

```
<remove-node  
path="/api/package[@name='{package_name}']/class[@name='{name}']" />
```

Renaming members

Renaming members cannot be done by directly editing the **api.xml** file because .NET for Android requires the original Java Native Interface (JNI) names to talk to Java. Therefore, the `//class/@name` attribute cannot be altered; if it is, the binding will not work.

Consider the case where we want to rename a type, `android.Manifest`. To accomplish this, we might try to directly edit **api.xml** and rename the class like so:

XML

```
<attr path="/api/package[@name='android']/class[@name='Manifest']"  
name="name">NewName</attr>
```

This will result in the Bindings Generator creating the following C# code for the wrapper class:

C#

```
[Register ("android/NewName")]  
public class NewName : Java.Lang.Object { ... }
```

Notice that the wrapper class has been renamed to `NewName`, while the original Java type is still `Manifest`. It is no longer possible for the .NET for Android binding class to access any methods on `android.Manifest`; the wrapper class is bound to a non-existent Java type.

To properly change the "managed" name of a wrapped type (or method), it is necessary to set the `managedName` attribute as shown in this example:

XML


```
<attr path="/api/package[@name='android']/class[@name='Manifest']"
      name="managedName">NewName</attr>
```

Using `managedName` is required when trying to rename any member, such as classes, interfaces, methods, and parameters.

Renaming `EventArgs` wrapper classes

When the .NET for Android binding generator identifies an `onXXX` setter method for a *listener type*, a C# event and `EventArgs` subclass will be generated to support a .NET flavoured API for the Java-based listener pattern. As an example, consider the following Java class and method:

XML

```
com.someapp.android.mpa.guidance.NavigationManager.on2DSignNextManuever(Next
ManueverListener listener);
```

.NET for Android will drop the prefix `on` from the setter method and instead use `2DSignNextManuever` as the basis for the name of the `EventArgs` subclass. The subclass will be named something similar to:

C#

```
NavigationManager.2DSignNextManueverEventArgs
```

This is not a legal C# class name. To correct this problem, the binding author must use the `argsType` attribute and provide a valid C# name for the `EventArgs` subclass:

XML

```
<attr path="/api/package[@name='com.someapp.android.mpa.guidance']/
      interface[@name='NavigationManager.Listener']/
      method[@name='on2DSignNextManuever']"
      name="argsType">NavigationManager.TwoDSignNextManueverEventArgs</attr>
```

Supported attributes

The following sections describe some of the attributes for transforming Java APIs.

`argsType`

This attribute is placed on setter methods to name the `EventArg` subclass that will be generated to support Java listeners. This is described in more detail the section [Renaming EventArg Wrapper Classes](#) in this guide.

eventName

Specifies a name for an event. If the name is empty, it prevents event generation. This is described in more detail in the section [Renaming EventArg Wrapper Classes](#).

managedName

This is used to change the name of a package, class, method, or parameter. For example to change the name of the Java class `MyClass` to `NewClassName`:

XML

```
<attr path="/api/package[@name='com.my.application']/class[@name='MyClass']"
      name="managedName">NewClassName</attr>
```

The next example illustrates an XPath expression for renaming the method

`java.lang.object.toString` to `Java.Lang.Object.NewManagedName`:

XML

```
<attr
  path="/api/package[@name='java.lang']/class[@name='Object']/method[@name='toString']"
  name="managedName">NewMethodName</attr>
```

managedType

`managedType` is used to change the return type of a method. In some situations the Bindings Generator will incorrectly infer the return type of a Java method, which will result in a compile time error. One possible solution in this situation is to change the return type of the method.

For example, the Bindings Generator believes that the Java method

`de.neom.neoreadersdk.resolution.compareTo()` should return an `int` and take `Object` as parameters, which results in the error message **Error CS0535**:

'DE.Neom.Neoreadersdk.Resolution' does not implement interface member 'Java.Lang.IComparable.CompareTo(Java.Lang.Object)'. The following snippet

demonstrates how to change the first parameter's type of the generated C# method from a `DE.Neom.Neoreadersdk.Resolution` to a `Java.Lang.Object`:

XML

```
<attr path="/api/package[@name='de.neom.neoreadersdk']/  
  class[@name='Resolution']/  
  method[@name='compareTo' and count(parameter)=1 and  
  parameter[1][@type='de.neom.neoreadersdk.Resolution']]/  
  parameter[1]" name="managedType">Java.Lang.Object</attr>
```

managedType

Changes the return type of a method. This does not change the return attribute (as changes to return attributes can result in incompatible changes to the JNI signature). In the following example, the return type of the `append` method is changed from `SpannableStringBuilder` to `IAppendable`:

XML

```
<attr path="/api/package[@name='android.text']/  
  class[@name='SpannableStringBuilder']/  
  method[@name='append']"  
  name="managedType">Java.Lang.IAppendable</attr>
```

obfuscated

Tools that obfuscate Java libraries may interfere with the .NET for Android Binding Generator and its ability to generate C# wrapper classes. Characteristics of obfuscated classes include:

- The class name includes a \$, i.e. `a$.class`
- The class name is entirely compromised of lower case characters, i.e. `a.class`

This snippet is an example of how to generate an "un-obfuscated" C# type:

XML

```
<attr path="/api/package[@name='{package_name}']/class[@name='{name}']"  
  name="obfuscated">>false</attr>
```

propertyName

This attribute can be used to change the name of a managed property.

A specialized case of using `propertyName` involves the situation where a Java class has only a setter method for a field. In this situation the Binding Generator would want to create a write-only property, something that is discouraged in .NET. The following snippet shows how to "remove" the .NET properties by setting the `propertyName` to an empty string:

XML

```
<attr
  path="/api/package[@name='org.java_websocket.handshake']/class[@name='HandshakeImpl1Client']/method[@name='setResourceDescriptor'
    and count(parameter)=1
    and parameter[1][@type='java.lang.String']]"
  name="propertyName"></attr>
<attr
  path="/api/package[@name='org.java_websocket.handshake']/class[@name='HandshakeImpl1Client']/method[@name='getResourceDescriptor'
    and count(parameter)=0]"
  name="propertyName"></attr>
```

Note that the setter and getter methods will still be created by the Bindings Generator, they just will not be converted to a .NET property.

sender

Specifies which parameter of a method should be the `sender` parameter when the method is mapped to an event. The value can be `true` or `false`. For example:

XML

```
<attr path="/api/package[@name='android.app']/
  interface[@name='TimePickerDialog.OnTimeSetListener']/
  method[@name='onTimeSet']/
  parameter[@name='view']"
  name="sender">true</ attr>
```

visibility

This attribute is used to change the visibility of a class, method, or property. For example, it may be necessary to promote a `protected` Java method so that its corresponding C# wrapper is `public`:

XML

```
<!-- Change the visibility of a class -->
<attr path="/api/package[@name='namespace']/class[@name='ClassName']"
name="visibility">public</attr>

<!-- Change the visibility of a method -->
<attr
path="/api/package[@name='namespace']/class[@name='ClassName']/method[@name=
'MethodName']" name="visibility">public</attr>
```

Related links

- [Binding Java libraries](#)
- [Metadata cheat sheet](#) 
- [Troubleshooting bindings](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Creating enumerations

Article • 09/16/2024

There are cases where Java Android libraries use integer constants to represent states that are passed to properties or methods of the libraries. For widely distributed bindings, it may be useful to bind these integer constants to enums in C# to provide a nicer API for consumers.

For internal or low usage bindings it's usually not worth the effort to set these up, as the consumers can simply use the bound constants instead of an enumeration.

To facilitate this mapping, two files are added to binding projects by the default project template:

- **EnumFields.xml** - This file defines the mapping between Java integer constants and a C# enumeration
- **EnumMethods.xml** - This file defines which methods/properties that currently take an `int` method parameter or have an `int` return type should be modified to use an enumeration instead.

Defining an enum using EnumFields.xml

The **EnumFields.xml** file contains the mapping between Java `int` constants and C# `enums`. Let's take the following example of a C# enum being created for a set of `int` constants:

XML

```
<mapping jni-class="com/skobbler/ngx/map/realreach/SKRealReachSettings" clr-enum-type="Skobbler.Ngx.Map.RealReach.SKMeasurementUnit">
  <field jni-name="UNIT_SECOND" clr-name="Second" value="0" />
  <field jni-name="UNIT_METER" clr-name="Meter" value="1" />
  <field jni-name="UNIT_MILLIWATT_HOURS" clr-name="MilliwattHour" value="2" />
</mapping>
```

Here we have taken the Java class `SKRealReachSettings` and defined a C# enum called `SKMeasurementUnit` in the namespace `Skobbler.Ngx.Map.RealReach`. The `field` entries define the name of the Java constant (example `UNIT_SECOND`), the name of the enum entry (example `Second`), and the integer value represented by both entities (example `0`).

Defining getter/setter methods using EnumMethods.xml

The **EnumMethods.xml** file allows changing method parameters and return types from Java `int` constants to C# `enums`. In other words, it maps the reading and writing of C# `enums` (defined in the **EnumFields.xml** file) to Java `int` constant `get` and `set` methods.

Given the `SKRealReachSettings` enum defined above, the following **EnumMethods.xml** file would define the getter/setter for this enum:

XML

```
<mapping jni-class="com/skobbler/ngx/map/realreach/SKRealReachSettings">
  <method jni-name="getMeasurementUnit" parameter="return" clr-enum-
type="Skobbler.Ngx.Map.RealReach.SKMeasurementUnit" />
  <method jni-name="setMeasurementUnit" parameter="measurementUnit" clr-
enum-type="Skobbler.Ngx.Map.RealReach.SKMeasurementUnit" />
</mapping>
```

The first `method` line maps the return value of the Java `getMeasurementUnit` method to the `SKMeasurementUnit` enum. The second `method` line maps the first parameter of the `setMeasurementUnit` to the same enum.

With all of these changes in place, you can use the follow code in .NET for Android to set the `MeasurementUnit`:

C#

```
realReachSettings.MeasurementUnit = SKMeasurementUnit.Second;
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Binding projects MSBuild properties

Article • 09/16/2024

 Note

In .NET for Android there is technically no distinction between an application and a bindings project, so these properties will work in both. In practice it is highly recommended to create separate application and bindings projects. Build properties that are primarily used in application projects are documented in the [MSBuild properties](#) reference guide.

Build properties

 Expand table

Property	Default	Description
<code>AndroidBoundInterfacesContainConstants</code>	<code>true</code>	A boolean property that specifies if binding constants on interfaces will be supported, or if the legacy workaround of creating an <code>IMyInterfaceConsts</code> class will be used. Documentation
<code>AndroidBoundInterfacesContainStatic</code> <code>AndDefaultInterfaceMethods</code>	<code>true</code>	A boolean property that specifies if default and static members on interfaces will be supported, or the legacy workaround of creating a sibling class containing static members like <code>abstract class MyInterface</code> will be used. Documentation
<code>AndroidBoundInterfacesContainTypes</code>	<code>true</code>	A boolean property that specifies if types nested in interfaces will be supported, or the legacy workaround of creating a non-nested type like <code>IMyInterfaceMyNestedClass</code> will be used. Documentation
<code>AndroidEnableObsoleteOverrideInheritance</code> <i>Added in .NET 8</i>	<code>true</code>	An boolean property that specifies if bound methods that override <code>@Deprecated</code> Java methods are automatically marked as <code>@Deprecated</code>. Documentation

Property	Default	Description
<code>AndroidEnableRestrictToAttributes</code> <i>Added in .NET 8</i>	<code>obsolete</code>	An enum-style property with valid values of <code>obsolete</code> and <code>disable</code> that specifies if the .NET <code>[Obsolete]</code> attribute is added to bound API that is marked with <code>@RestrictTo</code> in a Java library. Documentation
<code>AndroidJavadocVerbosity</code>	<code>intellisense</code>	An enum-style property with valid values <code>intellisense</code> and <code>full</code> that specifies how "verbose" C# XML Documentation Comments should be when importing Javadoc documentation within binding projects using the <code>@(JavaSourceJar)</code> build action. Documentation

AndroidBoundInterfacesContainConstants

A boolean property that specifies if binding constants on interfaces will be supported, or if the legacy workaround of creating an `IMyInterfaceConsts` class will be used.

The default value is `true`.

This is only recommended if trying to maintain public API compatibility with a legacy bindings library created before C# 8 was released.

[More details](#)

AndroidBoundInterfacesContainStaticAndDefaultInterfaceMethod

A boolean property that specifies if default and static members on interfaces will be supported, or if the legacy workaround of creating a sibling class containing static members like `abstract class MyInterface` will be used.

The default value is `true`.

This is only recommended if trying to maintain public API compatibility with a legacy bindings library created before C# 8 was released.

[More details](#)

AndroidBoundInterfacesContainTypes

A boolean property that specifies if types nested in interfaces will be supported, or if the legacy workaround of creating a non-nested type like `IMyInterfaceMyNestedClass` will be used.

The default value is `true`.

This is only recommended if trying to maintain public API compatibility with a legacy bindings library created before C# 8 was released.

[More details](#)

AndroidEnableRestrictToAttributes

An enum-style property with valid values of `obsolete` and `disable` that controls if the .NET `[Obsolete]` attribute is added to bound API that is marked with `@RestrictTo` in a Java library.

This property is set to `obsolete` by default.

When set to `obsolete`, types and members that are marked with the Java annotation `androidx.annotation.RestrictTo` or are in non-exported Java packages will be marked with an `[Obsolete]` attribute in the C# binding.

This `[Obsolete]` attribute has a descriptive message explaining that the Java package owner considers the API to be "internal" and warns against its use.

This attribute also has a custom warning code `XA0BS001` so that it can be suppressed independently of "normal" obsolete API.

When set to `disable`, API will be generated as normal with no additional attributes. (This is the same behavior as before .NET 8.)

Adding `[Obsolete]` attributes instead of automatically removing the API was done to preserve API compatibility with existing packages. If you would instead prefer to *remove* members that have the `@RestrictTo` annotation or are in non-exported Java packages, you can use [metadata](#) in addition to this property to prevent these types from being bound:

XML

```
<remove-node path="//*[@annotated-visibility]" />
```

Support for this property was added in .NET 8.

AndroidEnableObsoleteOverrideInheritance

A boolean property that specifies if bound methods that override `@Deprecated` Java methods are automatically marked as `@Deprecated`.

It is extremely rare to need to change this property.

Support for this property was added in .NET 8.

AndroidJavadocVerbosity

An enum-style property with valid values `intellisense` and `full` that specifies how "verbose" [C# XML Documentation Comments](#) should be when importing Javadoc documentation within binding projects using the [@\(JavaSourceJar\)](#) build action.

This property is set to `intellisense` by default.

- `intellisense`: Only emit the XML comments: `<exception/>`, `<param/>`, `<returns/>`, `<summary/>`.
- `full`: Emit listed `intellisense` elements, as well as `<remarks/>`, `<seealso/>`, and anything else that's supportable.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Binding projects MSBuild items

Article • 09/16/2024

ⓘ **Note**

In .NET for Android there is technically no distinction between an application and a bindings project, so these items will work in both. In practice it is highly recommended to create separate application and bindings projects. Build items that are primarily used in application projects are documented in the [MSBuild items](#) reference guide.

Build items

 Expand table

Item	Description
<code>AndroidAdditionalJavaManifest</code> <i>Added in .NET 9</i>	Represents additional POM files needed to verify Java dependencies. Documentation
<code>AndroidIgnoredJavaDependency</code> <i>Added in .NET 9</i>	Represents a Java dependencies that should be ignored when verifying Java dependency. Documentation
<code>AndroidJavaSource</code>	Represents Java source files (<code>.java</code>) that should be compiled and included in the project. Documentation
<code>AndroidLibrary</code>	Represents a <code>.jar</code> / <code>.aar</code> file to be bound by the bindings project. Documentation
<code>AndroidMavenLibrary</code> <i>Added in .NET 9</i>	Represents a <code>.jar</code> / <code>.aar</code> file that should be downloaded from a Maven repository and bound by the bindings project. Documentation
<code>AndroidNamespaceReplacement</code>	Represents a transform that should be applied to a Java package name to make the resulting managed namespace

Item	Description
	better fit .NET conventions. Documentation
<code>JavaSourceJar</code>	Represents a Java source code <code>.jar</code> that API documentation should be imported from. Documentation

Deprecated build items

These MSBuild items have been deprecated. While they continue to function, it is recommended to migrate to the listed newer items.

[Expand table](#)

Item	Description
<code>AndroidAarLibrary</code> <i>Deprecated</i>	Represents an Android <code>.aar</code> file to be included in the project output. Documentation
<code>AndroidJavaLibrary</code> <i>Deprecated</i>	Represents an Android <code>.jar</code> file to be included in the project output. Documentation
<code>EmbeddedJar</code> <i>Deprecated</i>	Represents an Android <code>.jar</code> file to be bound and included in the project output. Documentation
<code>EmbeddedReferenceJar</code> <i>Deprecated</i>	Represents an Android <code>.jar</code> file to be included in the project output. Documentation
<code>LibraryProjectZip</code> <i>Deprecated</i>	Represents an Android <code>.aar</code> file to be included in the project output. Documentation

AndroidAarLibrary

*This build item is deprecated and is replaced by the **AndroidLibrary** item.*

XML

```
<!-- Deprecated -->
<AndroidAarLibrary Include="mylib.aar" />

<!-- Recommended -->
<AndroidLibrary Include="mylib.aar" />
```

The Build action of `AndroidAarLibrary` should be used to directly reference `.aar` files. This build action will be most commonly used by Xamarin Components. Namely to include references to `.aar` files that are required to get Google Play and other services working.

Files with this Build action will be treated in a similar fashion to the embedded resources found in Library projects. The `.aar` will be extracted into the intermediate directory. Then any assets, resource and `.jar` files will be included in the appropriate item groups.

AndroidAdditionalJavaManifest

`<AndroidAdditionalJavaManifest>` is used in conjunction with [Java Dependency Resolution](#).

It is used to specify additional POM files that will be needed to verify dependencies. These are often parent or imported POM files referenced by a Java library's POM file.

XML

```
<ItemGroup>
  <AndroidAdditionalJavaManifest Include="mylib-parent.pom"
  JavaArtifact="com.example:mylib-parent" JavaVersion="1.0.0" />
</ItemGroup>
```

 Expand table

Item metadata name	Description
JavaArtifact	Required string. The group and artifact id of the Java library matching the specified POM file in the form <code>{GroupId}:{ArtifactId}</code> .
JavaVersion	Required string. The version of the Java library matching the specified POM file.

See the [Java Dependency Resolution documentation](#) for more details.

Support for this build item was introduced in .NET 9.

AndroidIgnoredJavaDependency

`<AndroidIgnoredJavaDependency>` is used in conjunction with [Java Dependency Resolution](#).

It is used to specify a Java dependency that should be ignored. This can be used if a dependency will be fulfilled in a way that Java dependency resolution cannot detect.

XML

```
<!-- Include format is {GroupId}:{ArtifactId} -->
<ItemGroup>
  <AndroidIgnoredJavaDependency
    Include="com.google.errorprone:error_prone_annotations" Version="2.15.0" />
</ItemGroup>
```

 Expand table

Item metadata name	Description
Version	Required string. The version of the Java library matching the specified dependency.

See the [Java Dependency Resolution documentation](#) for more details.

Support for this build item was introduced in .NET 9.

AndroidJavaLibrary

*This build item is deprecated and is replaced by the **AndroidLibrary** item.*

XML

```
<!-- Deprecated -->
<AndroidJavaLibrary Include="mylib.jar" />

<!-- Recommended -->
<AndroidLibrary Include="mylib.jar" />
```

Files with a Build action of `AndroidJavaLibrary` are Java Archives (`.jar` files) that will be included in the final Android package.

AndroidJavaSource

`AndroidJavaSource` files are Java source code that will be compiled and included in the final Android package.

Starting with .NET 7, all `**\*.java` files within the project directory automatically have a Build action of `AndroidJavaSource`, and will be bound prior to the Assembly build. Allows C# code to easily use types and members present within the `**\*.java` files.

[Expand table](#)

Item metadata name	Description
Bind	Optional boolean. Specifies whether the Java source file should have a C# binding generated for it. Defaults to <code>true</code> .

Support for this build item was introduced in .NET 7.

AndroidLibrary

Represents a `.jar/.aar` file to be bound and included in the project.

XML

```
<ItemGroup>
  <AndroidLibrary Include="foo.jar" />
  <AndroidLibrary Include="bar.aar" />
</ItemGroup>
```

[Expand table](#)

Item metadata name	Description
Bind	Optional boolean. Specifies whether the Java library should have a C# binding generated for it. Defaults to <code>true</code> .
Pack	Optional boolean. Specifies whether the Java library should be included in the project output. Defaults to <code>true</code> .

AndroidMavenLibrary

Represents a `.jar/.aar` file that should be downloaded from a Maven repository and bound by the bindings project.

This can be useful to simplify maintenance of .NET for Android bindings for artifacts hosted in Maven.

XML

```
<!-- Include format is {GroupId}:{ArtifactId} -->
<ItemGroup>
  <AndroidMavenLibrary Include="com.squareup.okhttp3:okhttp" Version="4.9.3"
/>
</ItemGroup>
```

 Expand table

Item metadata name	Description
Version	Required string. The version of the Java library that should be downloaded from Maven. Defaults to <code>true</code> .
Repository	Optional string. Specifies Maven repository to use. Supported values are <code>Central</code> , <code>Google</code> , or an <code>https</code> URL to a Maven repository. Defaults to <code>Central</code> .
Bind	Optional boolean. Specifies whether the Java library should have a C# binding generated for it. Defaults to <code>true</code> .
Pack	Optional boolean. Specifies whether the Java library should be included in the project output. Defaults to <code>true</code> .

See the [AndroidMavenLibrary documentation](#) for more details.

Support for this build item was introduced in .NET 9.

EmbeddedJar

*This build item is deprecated and is replaced by the **AndroidLibrary** item.*

XML

```
<!-- Deprecated -->
<EmbeddedJar Include="mylib.jar" />

<!-- Recommended -->
<AndroidLibrary Include="mylib.jar" />
```

In a .NET for Android binding project, the **EmbeddedJar** build action binds the Java/Kotlin library and embeds the `.jar` file into the library. When a .NET for Android

application project consumes the library, it will have access to the Java/Kotlin APIs from C# as well as include the Java/Kotlin code in the final Android application.

EmbeddedReferenceJar

*This build item is deprecated and is replaced by the **AndroidLibrary** item with the **Bind** metadata set to `false`.*

XML

```
<!-- Deprecated -->
<EmbeddedReferenceJar Include="mylib.jar" />

<!-- Recommended -->
<AndroidLibrary Include="mylib.jar" Bind="false" />
```

In a .NET for Android binding project, the **EmbeddedReferenceJar** build action embeds the `.jar` file into the library but does not create a C# binding as **EmbeddedJar** does. When a .NET for Android application project consumes the library, it will include the Java/Kotlin code in the final Android application.

JavaSourceJar

Represents a Java **source code** `.jar` containing [Javadoc documentation comments](#) that API documentation should be imported from.

Javadoc will be converted into [C# XML Documentation Comments](#) within the generated binding source code.

`$(AndroidJavadocVerbosity)` controls how "verbose" or "complete" the imported Javadoc is.

[Expand table](#)

Item metadata name	Description
CopyrightFile	Optional string. A path to a file that contains copyright information for the Javadoc contents, which will be appended to all imported documentation.
UrlPrefix	Optional string. A URL prefix to support linking to online documentation within imported documentation.
UrlStyle	Optional string. The "style" of URLs to generate when linking to online documentation. Only one style is currently supported:

Item metadata name	Description
	<code>developer.android.com/reference@2020-Nov.</code>
DocRootUrl	Optional string. A URL prefix to use in place of all <code>{@docroot}</code> instances in the imported documentation.

LibraryProjectZip

*This build item is deprecated and is replaced by the **AndroidLibrary** build item.*

XML

```
<!-- Deprecated -->
<LibraryProjectZip Include="mylib.aar" />

<!-- Recommended -->
<AndroidLibrary Include="mylib.aar" />
```

The **LibraryProjectZip** build action binds the Java/Kotlin library and embeds the `.zip` or `.aar` file into the library. When a .NET for Android application project consumes the library, it will have access to the Java/Kotlin APIs from C# as well as include the Java/Kotlin code in the final Android application.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

AndroidMavenLibrary

Article • 09/16/2024

ⓘ Note

This feature is only available in .NET 9+.

`<AndroidMavenLibrary>` allows a Maven artifact to be specified which will automatically be downloaded and added to a .NET for Android binding project. This can be useful to simplify maintenance of .NET for Android bindings for artifacts hosted in Maven.

Specification

A basic use of `<AndroidMavenLibrary>` looks like:

XML

```
<!-- Include format is {GroupId}:{ArtifactId} -->
<ItemGroup>
  <AndroidMavenLibrary Include="com.squareup.okhttp3:okhttp" Version="4.9.3"
/>
</ItemGroup>
```

This will do several things at build time:

- Download the Java [artifact](#) with group id `com.squareup.okhttp3`, artifact id `okhttp`, and version `4.9.3` from [Maven Central](#) to a local cache (if not already cached).
- Add the cached package to the .NET for Android bindings build as an `<AndroidLibrary>`.
- Download the Java artifact's POM file (and any needed parent/imported POM files) to enable [Java Dependency Verification](#). To opt out of this feature, add `VerifyDependencies="false"` to the `<AndroidMavenLibrary>` item.

Note that only the requested Java artifact is added to the .NET for Android bindings build. Any artifact dependencies are not added. If the requested artifact has dependencies, they must be fulfilled individually.

Options

`<AndroidMavenLibrary>` defaults to using Maven Central, however it should support any Maven repository that does not require authentication. This can be controlled with the `Repository` attribute.

Supported values are `Central` (default), `Google`, or a URL to another Maven repository.

XML

```
<ItemGroup>
  <AndroidMavenLibrary
    Include="androidx.core:core"
    Version="1.9.0"
    Repository="Google" />
</ItemGroup>
```

XML

```
<ItemGroup>
  <AndroidMavenLibrary
    Include="com.github.chrisbanes:PhotoView"
    Version="2.3.0"

Repository="https://repository.mulesoft.org/nexus/content/repositories/publi
c" />
</ItemGroup>
```

Additionally, any attributes applied to the `<AndroidMavenLibrary>` element will be copied to the `<AndroidLibrary>` it creates internally. Thus, [attributes](#) like `Bind` and `Pack` can be used to control the binding process. (Both default to `true`.)

XML

```
<ItemGroup>
  <AndroidMavenLibrary
    Include="androidx.core:core"
    Version="1.9.0"
    Repository="Google"
    Bind="false"
    Pack="false" />
</ItemGroup>
```

Feedback

Was this page helpful?

 Yes

 No

Java dependency verification

Article • 09/16/2024

ⓘ Note

This feature is only available in .NET 9+.

A common problem when creating Java binding libraries for .NET for Android is not providing the required Java dependencies. The binding process ignores API that requires missing dependencies, so this can result in large portions of desired API not being bound.

Unlike .NET assemblies, a Java library does not specify its dependencies in the package. The dependency information is stored in external files called POM files. In order to consume this information to ensure correct dependencies an additional layer of files must be added to a binding project.

💡 Tip

The preferred way of interacting with this system is to use [<AndroidMavenLibrary>](#) which will automatically download any needed POM files.

For example:

XML

```
<AndroidMavenLibrary Include="com.squareup.okio:okio" Version="1.17.4" />
```

automatically gets expanded to:

XML

```
<AndroidLibrary
  Include="
  <MavenCacheDir>/Central/com.squareup.okio/okio/1.17.4/com.squareup.okio_okio.jar"
  Manifest="
  <MavenCacheDir>/Central/com.squareup.okio/okio/1.17.4/com.squareup.okio_okio.pom"
  JavaArtifact="com.squareup.okio:okio:1.17.4" />

<AndroidAdditionalJavaManifest
  Include="<MavenCacheDir>/Central/com.squareup.okio/okio-
```

```
parent/1.17.4/okio-parent-1.17.4.pom"
  JavaArtifact="com.squareup.okio:okio-parent:1.17.4" />
```

etc.

However it is also possible to manually opt in to Java dependency verification using the build items documented here.

Specification

To manually opt in to Java dependency verification, add the `Manifest` and `JavaArtifact` attributes to an `<AndroidLibrary>` item:

XML

```
<!-- JavaArtifact format is {GroupId}:{ArtifactId}:{Version} -->
<ItemGroup>
  <AndroidLibrary
    Include="my_binding_library.jar"
    Manifest="my_binding_library.pom"
    JavaArtifact="com.example:mybinding:1.0.0" />
</ItemGroup>
```

Building the binding project now should result in verification errors if `my_binding_library.pom` specifies dependencies that are not met.

For example:

```
error : Java dependency 'androidx.collection:collection' version '1.0.0' is
not satisfied.
```

Seeing these error(s) or no errors should indicate that the Java dependency verification is working. Follow the [Resolving Java Dependencies](#) guide to fix any missing dependency errors.

Additional POM files

POM files can sometimes have some optional features in use that make them more complicated than the above example.

That is, a POM file can depend on a "parent" POM file:

XML

```
<parent>
  <groupId>com.squareup.okio</groupId>
  <artifactId>okio-parent</artifactId>
  <version>1.17.4</version>
</parent>
```

Additionally, a POM file can "import" dependency information from another POM file:

XML

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.squareup.okio</groupId>
      <artifactId>okio-bom</artifactId>
      <version>3.0.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Dependency information cannot be accurately determined without also having access to these additional POM files, and will result in an error like:

```
error : Unable to resolve POM for artifact 'com.squareup.okio:okio-
parent:1.17.4'.
```

In this case, we need to provide the POM file for `com.squareup.okio:okio-parent:1.17.4`:

XML

```
<!-- JavaArtifact format is {GroupId}:{ArtifactId}:{Version} -->
<ItemGroup>
  <AndroidAdditionalJavaManifest
    Include="com.squareup.okio.okio-parent.1.17.4.pom"
    JavaArtifact="com.squareup.okio:okio-parent:1.17.4" />
</ItemGroup>
```

Note that as "Parent" and "Import" POMs can themselves have parent and imported POMs, this step may need to be repeated until all POM files can be resolved.

Note also that if using `<AndroidMavenLibrary>` this should all be handled automatically.

At this point, if there are dependency errors, follow the [Resolving Java Dependencies](#) guide to fix any missing dependency errors.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Resolving Java dependencies

Article • 09/16/2024

ⓘ Note

This feature is only available in .NET 9+.

Once Java dependency verification has been enabled for a bindings project, either automatically via `<AndroidMavenLibrary>` or manually via `<AndroidLibrary>`, there may be errors to resolve, such as:

```
error : Java dependency 'androidx.collection:collection' version '1.0.0' is
not satisfied.
```

These dependencies can be fulfilled in many different ways.

<PackageReference>

In the best case scenario, there is already an existing binding of the Java dependency available on NuGet.org. This package may be provided by Microsoft or the .NET community. Packages maintained by Microsoft may be surfaced in the error message like this:

```
error : Java dependency 'androidx.collection:collection' version '1.0.0' is
not satisfied. Microsoft maintains the NuGet package
'Xamarin.AndroidX.Collection' that could fulfill this dependency.
```

Adding the `Xamarin.AndroidX.Collection` package to the project should automatically resolve this error, as the package provides metadata to advertise that it provides the `androidx.collection:collection` dependency. This is done by looking for a specially crafted NuGet tag. For example, for the AndroidX Collection library, the tag looks like this:

XML

```
<!-- artifact={GroupId}:{ArtifactId}:{Java Library Version} -->
```

```
<PackageTags>artifact=androidx.collection:collection:1.0.0</PackageTags>
```

However there may be NuGet packages which fulfill a dependency but have not had this metadata added to it. In this case, you will need to explicitly specify which dependency the package contains with `JavaArtifact`:

XML

```
<PackageReference  
  Include="Xamarin.Kotlin.StdLib"  
  Version="1.7.10"  
  JavaArtifact="org.jetbrains.kotlin:kotlin-stdlib:1.7.10" />
```

With this, the binding process knows the Java dependency is satisfied by the NuGet package.

ⓘ Note

NuGet packages specify their own dependencies, so you will not need to worry about transitive dependencies.

<ProjectReference>

If the needed Java dependency is provided by another project in your solution, you can annotate the `<ProjectReference>` to specify the dependency it fulfills:

XML

```
<ProjectReference  
  Include="..\My.Other.Binding\My.Other.Binding.csproj"  
  JavaArtifact="my.other.binding:helperlib:1.0.0" />
```

With this, the binding process knows the Java dependency is satisfied by the referenced project.

ⓘ Note

Each project specifies their own dependencies, so you will not need to worry about transitive dependencies.

<AndroidLibrary>

If you are creating a public NuGet package, you will want to follow NuGet's "one library per package" policy so that the NuGet dependency graph works. However, if creating a binding for private use, you may want to include your Java dependencies directly inside the parent binding.

This can be done by adding additional `<AndroidLibrary>` items to the project:

XML

```
<ItemGroup>
  <AndroidLibrary Include="mydependency.jar"
    JavaArtifact="my.library:dependency-library:1.0.0" />
</ItemGroup>
```

To include the Java library but not produce C# bindings for it, mark it with `Bind="false"`:

XML

```
<ItemGroup>
  <AndroidLibrary Include="mydependency.jar"
    JavaArtifact="my.library:dependency-library:1.0.0" Bind="false" />
</ItemGroup>
```

Alternatively, `<AndroidMavenLibrary>` can be used to retrieve a Java library from a Maven repository:

XML

```
<ItemGroup>
  <AndroidMavenLibrary Include="my.library:dependency-library"
    Version="1.0.0" />
  <!-- or, if the Java library doesn't need to be bound -->
  <AndroidMavenLibrary Include="my.library:dependency-library"
    Version="1.0.0" Bind="false" />
</ItemGroup>
```

ⓘ Note

If the dependency library has its own dependencies, you will be required to ensure they are fulfilled.

<AndroidIgnoredJavaDependency>

As a last resort, a needed Java dependency can be ignored. An example of when this is useful is if the dependency library is a collection of Java annotations that are only used at compile type and not runtime.

Note that while the error message will go away, it does not mean the package will magically work. If the dependency is actually needed at runtime and not provided the Android application will crash with a `Java.Lang.NoClassDefFoundError` error.

XML

```
<ItemGroup>
  <AndroidIgnoredJavaDependency
Include="com.google.errorprone:error_prone_annotations:2.15.0" />
</ItemGroup>
```

ⓘ Note

Any usage of `JavaArtifact` can specify multiple artifacts by delimiting them with a comma or semicolon.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Distributing bindings libraries

Article • 09/16/2024

Once a Java library has been bound to be used in .NET for Android, there are multiple ways it can be consumed:

- **Project Reference** - If the binding project and the application project are in the same solution file, using a `<ProjectReference>` is generally the easiest way to consume the binding.
- **NuGet Package** - A NuGet package is ideal for public publishing or for an internal distributed development environment that has an internal NuGet server.
- **File Reference** - A consuming application can directly add a `<Reference>` to a binding `.dll` if neither the binding project nor a NuGet server is available.

Controlling binding and packaging options

A bindings library project has two options for controlling if an `<AndroidLibrary>` gets bound and gets redistributed:

- **Bind** (`true/false`) - Defaults to `true`, meaning managed bindings are created for the specified `<AndroidLibrary>`. Setting to `false` means the Java library will be included in the output, but will not have managed bindings. This is useful if the library is a needed dependency of another Java library, but it will not be called from C#.

XML

```
<!-- Java library will have C# bindings and be included in the output -->
<AndroidLibrary Include="okhttp-4.12.0.jar" />

<!-- Java library will not have C# bindings but will still be included in
the output -->
<AndroidLibrary Include="okio-3.9.0.jar" Bind="false" />
```

- **Pack** (`true/false`) - Defaults to `true`, meaning the specified `<AndroidLibrary>` will be included in the output (such as a NuGet package). Setting to `false` means the Java library will not be included in the output. This is a rare scenario in case the dependency is already being provided via alternative means.

XML

```
<!-- Java library will have C# bindings and be included in the output -->
<AndroidLibrary Include="okhttp-4.12.0.jar" />

<!-- Java library will have C# bindings but will *not* be included in the
output -->
<AndroidLibrary Include="okio-3.9.0.jar" Pack="false" />
```

ProjectReference

If the binding project and the application project are in the same solution file, using a `<ProjectReference>` is generally the easiest way to consume the binding:

XML

```
<ProjectReference Include="mybindinglib.csproj" />
```

The build system will be responsible for adding the managed bindings as well as any `.jar/.aar` files to the application project.

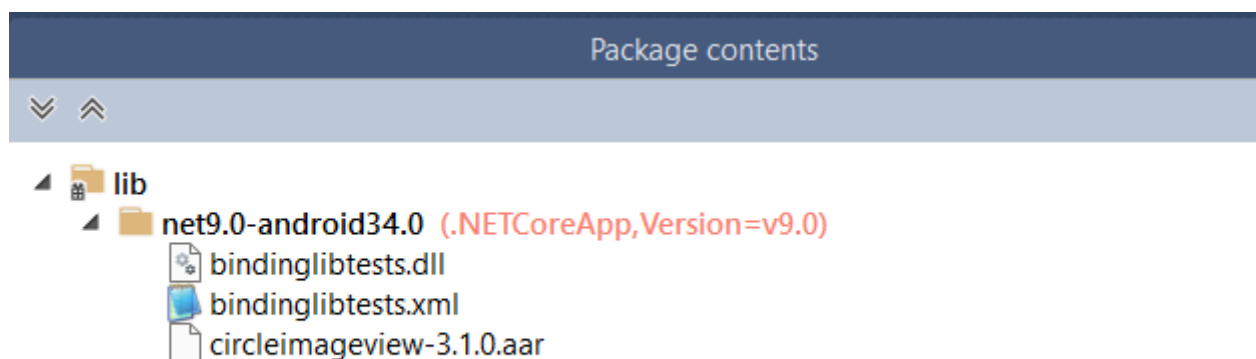
NuGet package

As a property inherited from .NET SDK-style projects, any bindings library can be trivially packaged into a redistributable NuGet package by using the "Pack" command in Visual Studio or from the command line:

.NET CLI

```
dotnet pack
```

The bindings library as well as the Java library will be included in the NuGet package:



Use the `Pack` attribute documented above to control which Java libraries are desired in the NuGet package.

NuGet packages can be customized using the [standard .NET MSBuild elements](#).

File reference

If neither of the above options are possible, the binding `.dll` can be referenced directly with a `<Reference>` element. Note that any Java libraries that are part of the binding project output must be in the **same directory** as the binding `.dll`.

If you move/copy just the binding `.dll` to another location and do not also move/copy any `.jar`/`.dll` files they will not end up in your application and the binding will fail at runtime.

❗ Important

In Classic Xamarin.Android using an item action like `EmbeddedJar` would place the `.jar` file inside the `.dll` and only one file would be needed. Support for this was removed in .NET for Android because it greatly increased application build time to scan and extract embedded Java library files. Any needed `.jar`/`.dll` files **MUST** be located in the same directory as the binding `.dll`.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Native library interop

Article • 10/31/2024

Native library interop (formerly referred to as the "Slim Binding" approach), refers to a pattern for accessing native SDKs in .NET for Android and .NET MAUI projects.

Starting in .NET 9, the .NET for Android SDK supports building Gradle projects by using the `@(AndroidGradleProject)` build action. This is declared in an MSBuild ItemGroup in a project file:

XML

```
<ItemGroup>
  <AndroidGradleProject Include="path/to/project/build.gradle.kts"
  ModuleName="mylibrary" />
</ItemGroup>
```

When an `@(AndroidGradleProject)` item is added to a .NET for Android project, the build process will attempt to create an AAR or APK file from the specified Gradle project. Any AAR output files will be added to the .NET project as an `@(AndroidLibrary)` to be bound.

See also

- The [.NET MAUI Community Toolkit - Native Library Interop](#) guide for more detailed docs.
- The [build-items](#) docs for more information about the `@(AndroidGradleProject)` build action.
- The [Maui.NativeLibraryInterop](#) [git repository](#) for code samples.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) [Get help at Microsoft Q&A](#)

Troubleshooting bindings

Article • 09/16/2024

Binding an Android library (an `.aar` or a `.jar`) file is seldom a straightforward affair; it usually requires additional effort to mitigate issues that result from the differences between Java and .NET. These issues will prevent .NET for Android from binding the Android library and present themselves as error messages in the build log. This guide will provide some tips for troubleshooting the issues, list some of the more common problems/scenarios, and provide possible solutions to successfully binding the Android library.

When binding an existing Android library, it is necessary to keep in mind the following points:

- **The external dependencies for the library** – Any Java dependencies required by the Android library must be included in the .NET for Android project via a NuGet package or as an `AndroidLibrary`.
- **The Android API level that the Android library is targeting** – It is not possible to "downgrade" the Android API level; ensure that the .NET for Android binding project is targeting the same API level (or higher) as the Android library.

Tip

The [Binding Tooling GitHub repository wiki](#) [↗] is a great resource and contains additional troubleshooting information that may help with specific cases.

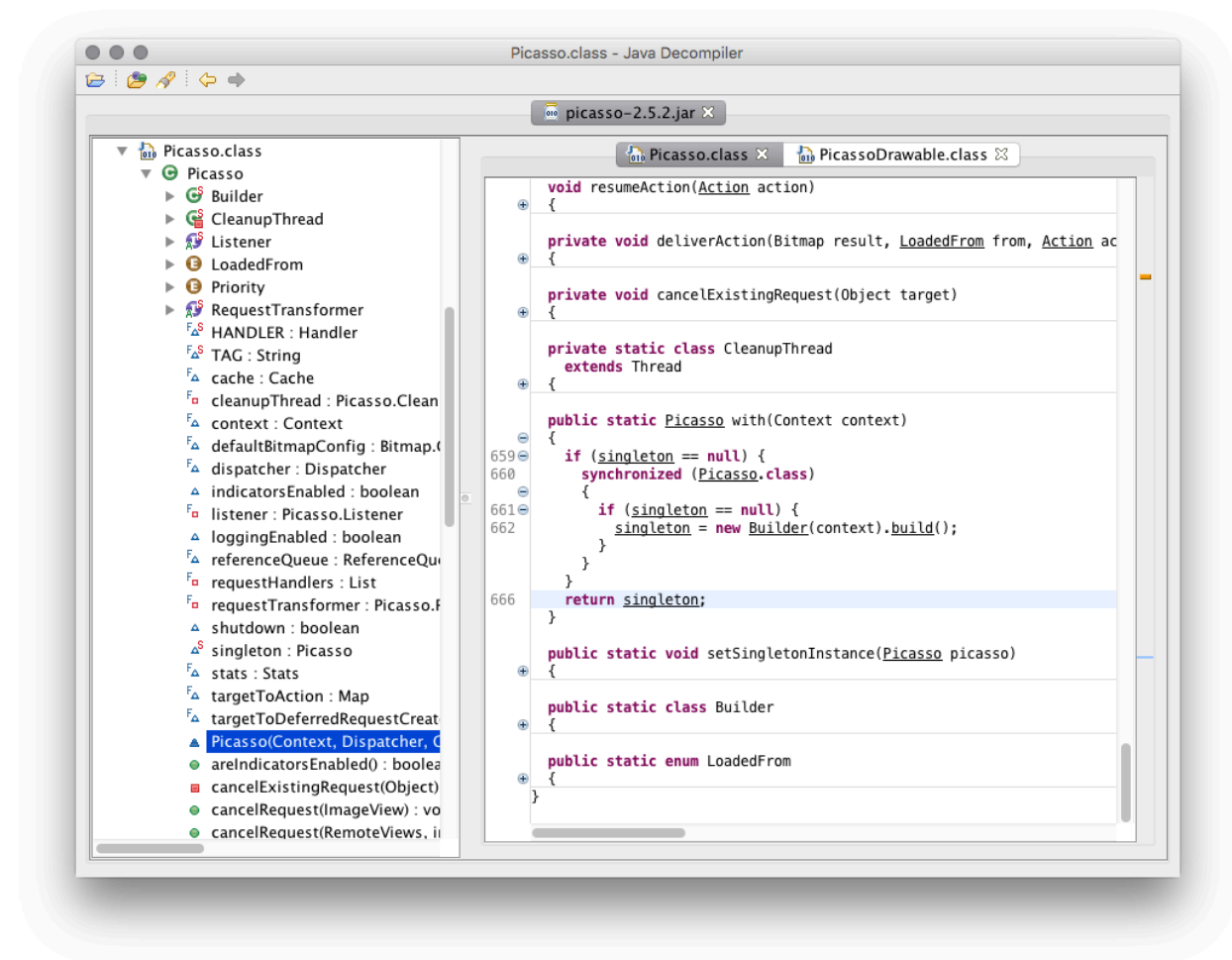
The first step to troubleshooting issues with binding a .NET for Android library is to enable [diagnostic MSBuild output](#). After enabling the diagnostic output, rebuild the .NET for Android binding project and examine the build log to locate clues about what the cause of problem is.

It can also prove helpful to decompile the Android library and examine the types and methods that .NET for Android is trying to bind. This is covered in more detail later on in this guide.

Decompiling an Android library

Inspecting the classes and methods of the Java classes can provide valuable information that will assist in binding a library. [JD-GUI](#) [↗] is a graphical utility that can display Java source code from the `CLASS` files contained in a JAR.

To decompile an Android library open the .JAR file with the Java decompiler. If the library is an .AAR file, the Java source code will be in the **classes.jar** entry of the archive file. The following is a sample screenshot of using JD-GUI to analyze the [Picasso](#) JAR:



Once you have decompiled the Android library, examine the source code. Generally speaking, look for :

- **Classes that have characteristics of obfuscation** – Characteristics of obfuscated classes include:
 - The class name includes a \$, i.e. **a\$.class**
 - The class name is entirely comprised of lower case characters, i.e. **a.class**
- **import statements for unreferenced libraries** – Identify the unreferenced library and add those dependencies to the .NET for Android binding project with an appropriate binding from NuGet or with a **Build Action of AndroidLibrary**.

ⓘ Note

Decompiling a Java library may be prohibited or subject to legal restrictions based on local laws or the license under which the Java library was published. If necessary,

enlist the services of a legal professional before attempting to decompile a Java library and inspect the source code.

Inspect api.xml

As a part of building a binding project, .NET for Android will generate an XML file name `obj/Debug/api.xml`:

Name	Date Modified	Size	Kind
▶ Additions	Mar 31, 2016, 2:36 PM	--	Folder
▶ bin	Apr 6, 2016, 10:47 AM	--	Folder
✖ evernote-android-job.csproj	Jun 23, 2016, 12:37 PM	4 KB	Xamari...Project
▶ Jars	Apr 1, 2016, 10:55 AM	--	Folder
▼ obj	Jun 23, 2016, 12:55 PM	--	Folder
▼ Debug	Jun 23, 2016, 12:44 PM	--	Folder
__AndroidLibraryProjects__.zip	Jun 23, 2016, 12:44 PM	84 KB	ZIP archive
▶ __library_projects__	Jun 23, 2016, 12:44 PM	--	Folder
api.xml	Jun 23, 2016, 12:44 PM	103 KB	XML Document
evernote-android-job.csproj.FilesWrittenAbsolute.txt	Jun 23, 2016, 12:44 PM	1 KB	Plain Text
evernote-android-job.dll	Jun 23, 2016, 12:44 PM	176 KB	Microso...library
evernote-android-job.dll.mdb	Jun 23, 2016, 12:44 PM	23 KB	Document
evernoteandroidjob.Jars.cat-1.0.3.jar	Feb 22, 2016, 12:33 PM	12 KB	Java JAR file
evernoteandroidjob.Debug. AndroidLibraryProjects .zip	Jun 23, 2016, 12:44 PM	84 KB	ZIP archive

This file provides a list of all the Java APIs that .NET for Android is trying to bind. The contents of this file can help identify any missing types or methods, duplicate binding. Although inspection of this file is tedious and time consuming, it can provide clues on what might be causing any binding problems. For example, `api.xml` might reveal that a property is returning an inappropriate type, or that there are two types that share the same managed name.

Known issues

This section will list some of the common error messages or symptoms that may occur when trying to bind an Android library.

Problem: Missing C# types in generated output.

The binding `.dll` builds but misses some Java types, or the generated C# source does not build due to an error stating there are missing types.

Possible causes:

This error may occur due to several reasons as listed below:

- The library being bound may reference a second Java library. If the public API for the bound library uses types from the second library, you must reference a

managed binding for the second library as well.

- Java allows deriving a public class from non-public class, but this is unsupported in .NET. Since the binding generator does not generate bindings for non-public classes, derived classes such as these cannot be generated correctly. To fix this, either remove the metadata entry for those derived classes using the remove-node in **Metadata.xml**, or fix the metadata that is making the non-public class public. Although the latter solution will create the binding so that the C# source will build, the non-public class should not be used.

For example:

XML

```
<attr  
  path="/api/package[@name='com.some.package']/class[@name='SomeClass']"  
  name="visibility">public</attr>
```

- Tools that obfuscate Java libraries may interfere with the .NET for Android Binding Generator and its ability to generate C# wrapper classes. The following snippet shows how to update **Metadata.xml** to unobfuscate a class name:

XML

```
<attr path="/api/package[@name='{package_name}']/class[@name='{name}']"  
  name="obfuscated">>false</attr>
```

Problem: Generated C# source does not build due to parameter type mismatch

The generated C# source does not build. Overridden method's parameter types do not match.

Possible causes:

.NET for Android includes a variety of Java fields that are mapped to enums in the C# bindings. These can cause type incompatibilities in the generated bindings. To resolve this, the method signatures created from the binding generator need to be modified to use the enums. For more information, please see [Creating enumerations](#).

Problem: Duplicate custom EventArgs types

Build fails due to duplicate custom EventArgs types. An error like this occurs:

shell

```
error CS0102: The type `Com.Google.Ads.Mediation.DismissScreenEventArgs'
already contains a definition for `p0'
```

Possible causes:

This is because there is some conflict between event types that come from more than one interface "listener" type that shares methods having identical names. For example, if there are two Java interfaces as seen in the example below, the generator creates

`DismissScreenEventArgs` for both `MediationBannerListener` and `MediationInterstitialListener`, resulting in the error.

Java

```
// Java:
public interface MediationBannerListener {
    void onDismissScreen(MediationBannerAdapter p0);
}
public interface MediationInterstitialListener {
    void onDismissScreen(MediationInterstitialAdapter p0);
}
```

This is by design so that lengthy names on event argument types are avoided. To avoid these conflicts, some metadata transformation is required. Edit **Transforms\Metadata.xml** and add an `argsType` attribute on either of the interfaces (or on the interface method):

XML

```
<attr path="/api/package[@name='com.google.ads.mediation']/"
interface[@name='MediationBannerListener']/method[@name='onDismissScreen']"
    name="argsType">BannerDismissScreenEventArgs</attr>

<attr path="/api/package[@name='com.google.ads.mediation']/"
interface[@name='MediationInterstitialListener']/method[@name='onDismissScreen']"
    name="argsType">IntersitionalDismissScreenEventArgs</attr>

<attr path="/api/package[@name='android.content']/"
interface[@name='DialogInterface.OnClickListener']"
    name="argsType">DialogClickEventArgs</attr>
```

Problem: Class does not implement interface method

An error message is produced indicating that a generated class does not implement a method that is required for an interface which the generated class implements. However, looking at the generated code, you can see that the method is implemented.

Here is an example of the error:

shell

```
obj\Debug\generated\src\Oauth.Signpost.Basic.HttpURLConnectionRequestAdapter
.cs(8,23):
error CS0738: 'Oauth.Signpost.Basic.HttpURLConnectionRequestAdapter' does
not
implement interface member 'Oauth.Signpost.Http.IHttpRequest.Unwrap()'.
'Oauth.Signpost.Basic.HttpURLConnectionRequestAdapter.Unwrap()' cannot
implement
'Oauth.Signpost.Http.IHttpRequest.Unwrap()' because it does not have the
matching
return type of 'Java.Lang.Object'
```

Possible causes:

This is a problem that occurs with binding Java methods with covariant return types. In this example, the method `Oauth.Signpost.Http.IHttpRequest.UnWrap()` needs to return `Java.Lang.Object`. However, the method `Oauth.Signpost.Basic.HttpURLConnectionRequestAdapter.UnWrap()` has a return type of `HttpURLConnection`. There are two ways to fix this issue:

- Add a partial class declaration for `HttpURLConnectionRequestAdapter` and explicitly implement `IHttpRequest.Unwrap()`:

C#

```
namespace Oauth.Signpost.Basic {
    partial class HttpURLConnectionRequestAdapter {
        Java.Lang.Object OauthSignpost.Http.IHttpRequest.Unwrap() {
            return Unwrap();
        }
    }
}
```

- Remove the covariance from the generated C# code. This involves adding the following transform to `Transforms\Metadata.xml` which will cause the generated C# code to have a return type of `Java.Lang.Object`:

XML

```
<attr  
  
path="/api/package[@name='oauth.signpost.basic']/class[@name='HttpURLConnectionRequestAdapter']/method[@name='unwrap']"  
    name="managedReturn">Java.Lang.Object  
</attr>
```

Problem: Name collisions on inner classes / properties

Conflicting visibility on inherited objects.

In Java, it's not required that a derived class have the same visibility as its parent. Java will just fix that for you. In C#, that has to be explicit, so you need to make sure all classes in the hierarchy have the appropriate visibility. The following example shows how to change a Java package name from `com.evernote.android.job` to

`Evernote.AndroidJob`:

XML

```
<!-- Change the visibility of a class -->  
<attr path="/api/package[@name='namespace']/class[@name='ClassName']"  
    name="visibility">public</attr>  
  
<!-- Change the visibility of a method -->  
<attr  
path="/api/package[@name='namespace']/class[@name='ClassName']/method[@name=  
'MethodName']" name="visibility">public</attr>
```

Problem: A .so library required by the binding is not loading

Some binding projects may also depend on functionality in a .so library. It is possible that .NET for Android will not automatically load the .so library. When the wrapped Java code executes, .NET for Android will fail to make the JNI call and the error message *java.lang.UnsatisfiedLinkError: Native method not found:* will appear in the logcat out for the application.

The fix for this is to manually load the .so library with a call to

`Java.Lang.JavaSystem.LoadLibrary`. For example assuming that a .NET for Android project has shared library `libpocketsphinx_jni.so` included in the binding project with a

build action of **EmbeddedNativeLibrary**, the following snippet (executed before using the shared library) will load the **.so** library:

C#

```
Java.Lang.JavaSystem.LoadLibrary("pocketsphinx_jni");
```

Related links

- [Troubleshooting Tips on Binding Tooling GitHub repository wiki](#) 
- [Library Projects](#) 
- [Enable Diagnostic Output](#) 
- [JD-GUI](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Build process

Article • 05/08/2024

The .NET for Android build process is responsible for gluing everything together: [generating Resource.designer.cs](#), supporting the `@(AndroidAsset)`, `@(AndroidResource)`, and other [build actions](#), generating [Android-callable wrappers](#), and generating a `.apk` for execution on Android devices.

Application packages

In broad terms, there are two types of Android application packages (`.apk` files) which the .NET for Android build system can generate:

- **Release** builds, which are fully self-contained and don't require extra packages to execute. These are the packages that are provided to an App store.
- **Debug** builds, which are not.

These package types match the MSBuild `Configuration` which produces the package.

Fast deployment

Fast deployment works by further shrinking Android application package size. This is done by excluding the app's assemblies from the package, and instead deploying the app's assemblies directly to the application's internal `files` directory, usually located in `/data/data/com.some.package`. The internal `files` directory is not a globally writable folder, so the `run-as` tool is used to execute all the commands to copy the files into that directory.

This process speeds up the build/deploy/debug cycle because the package is not reinstalled when *only* assemblies are changed. Only the updated assemblies are resynchronized to the target device.

Warning

Fast deployment is known to fail on devices which block `run-as`, which often includes devices older than Android 5.0.

Fast deployment is enabled by default, and may be disabled in Debug builds by setting the `$(EmbedAssembliesIntoApk)` property to `True`.

The [Enhanced Fast Deployment](#) mode can be used in conjunction with this feature to speed up deployments even further. This will deploy both assemblies, native libraries, typemaps and dexes to the `files` directory. But you should only really need to enable this if you are changing native libraries, bindings or Java code.

MSBuild projects

The .NET for Android build process is based on MSBuild, which is also the project file format used by Visual Studio for Mac and Visual Studio. Ordinarily, users will not need to edit the MSBuild files by hand – the IDE creates fully functional projects and updates them with any changes made, and automatically invoke build targets as needed.

Advanced users may wish to do things not supported by the IDE's GUI, so the build process is customizable by editing the project file directly. This page documents only the .NET for Android-specific features and customizations – many more things are possible with the normal MSBuild items, properties and targets.

Binding projects

The following MSBuild properties are used with [Binding projects](#):

- [\\$\(AndroidClassParser\)](#)
- [\\$\(AndroidCodegenTarget\)](#)

`Resource.designer.cs` Generation

The Following MSBuild properties are used to control generation of the `Resource.designer.cs` file:

- [\\$\(AndroidAapt2CompileExtraArgs\)](#)
- [\\$\(AndroidAapt2LinkExtraArgs\)](#)
- [\\$\(AndroidExplicitCrunch\)](#)
- [\\$\(AndroidR8IgnoreWarnings\)](#)
- [\\$\(AndroidResgenExtraArgs\)](#)
- [\\$\(AndroidResgenFile\)](#)
- [\\$\(AndroidUseAapt2\)](#)
- [\\$\(MonoAndroidResourcePrefix\)](#)

Signing properties

Signing properties control how the Application package is signed so that it may be installed onto an Android device. To allow quicker build iteration, the .NET for Android tasks do not sign packages during the build process, because signing is quite slow. Instead, they are signed (if necessary) before installation or during export, by the IDE or the *Install* build target. Invoking the *SignAndroidPackage* target will produce a package with the `-Signed.apk` suffix in the output directory.

By default, the signing target generates a new debug-signing key if necessary. If you wish to use a specific key, for example on a build server, the following MSBuild properties are used:

- `$(AndroidDebugKeyAlgorithm)`
- `$(AndroidDebugKeyValidity)`
- `$(AndroidDebugStoreType)`
- `$(AndroidKeyStore)`
- `$(AndroidSigningKeyAlias)`
- `$(AndroidSigningKeyPass)`
- `$(AndroidSigningKeyStore)`
- `$(AndroidSigningStorePass)`
- `$(JarsignerTimestampAuthorityCertificateAlias)`
- `$(JarsignerTimestampAuthorityUrl)`

keytool Option Mapping

Consider the following `keytool` invocation:

shell

```
$ keytool -genkey -v -keystore filename.keystore -alias keystore.alias -  
keyalg RSA -keysize 2048 -validity 10000  
Enter keystore password: keystore.filename password  
Re-enter new password: keystore.filename password  
...  
Is CN=... correct?  
[no]: yes  
  
Generating 2,048 bit RSA key pair and self-signed certificate (SHA1withRSA)  
with a validity of 10,000 days  
for: ...  
Enter key password for keystore.alias  
(RETURN if same as keystore password): keystore.alias password  
[Storing filename.keystore]
```

To use the keystore generated above, use the property group:

XML

```
<PropertyGroup>
  <AndroidKeyStore>True</AndroidKeyStore>
  <AndroidSigningKeyStore>filename.keystore</AndroidSigningKeyStore>
  <AndroidSigningStorePass>keystore.filename
password</AndroidSigningStorePass>
  <AndroidSigningKeyAlias>keystore.alias</AndroidSigningKeyAlias>
  <AndroidSigningKeyPass>keystore.alias password</AndroidSigningKeyPass>
</PropertyGroup>
```

Build extension points

The .NET for Android build system exposes a few public extension points for users wanting to hook into our build process. To use one of these extension points you will need to add your custom target to the appropriate MSBuild property in a

`PropertyGroup`. For example:

XML

```
<PropertyGroup>
  <AfterGenerateAndroidManifest>
    $(AfterGenerateAndroidManifest);
    YourTarget;
  </AfterGenerateAndroidManifest>
</PropertyGroup>
```

Extension points include:

- ``$(AfterGenerateAndroidManifest)`
- ``$(AndroidPrepareForBuildDependsOn)`
- ``$(BeforeGenerateAndroidManifest)`
- `$(BeforeBuildAndroidAssetPacks)`

A word of caution about extending the build process: If not written correctly, build extensions can affect your build performance, especially if they run on every build. It is highly recommended that you read the MSBuild [documentation](#) before implementing such extensions.

Target definitions

The .NET for Android-specific parts of the build process are defined in

`$(MSBuildExtensionsPath)\Xamarin\Android\Xamarin.Android.CSharp.targets`, but normal

language-specific targets such as *Microsoft.CSharp.targets* are also required to build the assembly.

The following build properties must be set before importing any language targets:

XML

```
<PropertyGroup>
  <TargetFrameworkIdentifier>MonoDroid</TargetFrameworkIdentifier>
  <MonoDroidVersion>v1.0</MonoDroidVersion>
  <TargetFrameworkVersion>v2.2</TargetFrameworkVersion>
</PropertyGroup>
```

All of these targets and properties can be included for C# by importing *Xamarin.Android.CSharp.targets*:

XML

```
<Import
Project="$(MSBuildExtensionsPath)\Xamarin\Android\Xamarin.Android.CSharp.targets" />
```

This file can easily be adapted for other languages.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Build targets

Article • 04/18/2024

The following build targets are defined in .NET for Android projects.

Build

Builds the source code within a project and all dependencies.

This target *does not* create an Android package (`.apk` file). To create an Android package, use the [SignAndroidPackage](#) target, or set the `$(AndroidBuildApplicationPackage)` property to `True` when building:

```
shell
```

```
msbuild /p:AndroidBuildApplicationPackage=True App.sln
```

BuildAndStartAotProfiling

Builds the app with an embedded AOT profiler, sets the profiler TCP port to [\\$\(AndroidAotProfilerPort\)](#), and starts the default activity.

The default TCP port is `9999`.

Added in Xamarin.Android 10.2.

Clean

Removes all files generated by the build process.

FinishAotProfiling

Must be called *after* the [BuildAndStartAotProfiling](#) target.

Collects the AOT profiler data from the device or emulator through the TCP port [\\$\(AndroidAotProfilerPort\)](#) and writes them to [\\$\(AndroidAotCustomProfilePath\)](#).

The default values for port and custom profile are `9999` and `custom.aprof`.

To pass additional options to `aprofutil`, set them in the `$(AProfUtilExtraOptions)` property.

This is equivalent to:

shell

```
aprofutil $(AProfUtilExtraOptions) -s -v -f -p $(AndroidAotProfilerPort) -o  
"$(AndroidAotCustomProfilePath)"
```

Added in Xamarin.Android 10.2.

GetAndroidDependencies

Creates the `@(AndroidDependency)` item group, which is used by the `InstallAndroidDependencies` target to determine which Android SDK packages to install.

Install

`Creates`, `signs`, and installs the Android package onto the default device or virtual device.

The `$(AdbTarget)` property specifies the Android target device the Android package may be installed to or removed from.

Bash

```
# Install package onto emulator via -e  
# Use `/Library/Frameworks/Mono.framework/Commands/msbuild` on OS X  
MSBuild /t:Install ProjectName.csproj /p:AdbTarget=-e
```

InstallAndroidDependencies

Calls the `GetAndroidDependencies` target, then installs the Android SDK packages specified in the `@(AndroidDependency)` item group.

.NET CLI

```
dotnet build -t:InstallAndroidDependencies -f net8.0-android "-  
p:AndroidSdkDirectory=<path to sdk>" "-p:JavaSdkDirectory=<path to java  
sdk>"
```

The `-f net8.0-android` is required as this target is a .NET for Android specific target. If you omit this argument you will get the following error:

```
error MSB4057: The target "InstallAndroidDependencies" does not exist in the project.
```

The `AndroidSdkDirectory` and `JavaSdkDirectory` properties are required as we need to know where to install the required components. These directories can be empty or existing. Sdk components will be installed on top of an existing sdk installation.

The `$(AndroidManifestType)` MSBuild property controls which [Visual Studio SDK Manager repository](#) is used for package name and package version detection, and URLs to download.

RunWithLogging

Runs the application with additional logging enabled. Helpful when reporting or investigating an issue with either the application or the runtime. If successful, messages printed to the screen will show location of the logcat file with the logged messages.

Properties which affect how the target works:

- `/p:RunLogVerbose=true` enables even more verbose logging from MonoVM
- `/p:RunLogDelayInMS=X` where `X` should be replaced with time in milliseconds to wait before writing the log output to file. Defaults to `1000`.

SignAndroidPackage

Creates and signs the Android package (`.apk`) file.

Use with `/p:Configuration=Release` to generate self-contained "Release" packages.

StartAndroidActivity

Starts the default activity on the device or the running emulator.

To start a different activity, set the `$(AndroidLaunchActivity)` property to the activity name.

This is equivalent to:

```
shell
```

```
adb shell am start -S -n @PACKAGE_NAME@/$(AndroidLaunchActivity)
```

Added in Xamarin.Android 10.2.

StopAndroidPackage

Completely stops the application package on the device or the running emulator.

This is equivalent to:

```
shell
```

```
adb shell am force-stop @PACKAGE_NAME@
```

Added in Xamarin.Android 10.2.

Uninstall

Uninstalls the Android package from the default device or virtual device.

The [\\$\(AdbTarget\)](#) property specifies the Android target device the Android package may be installed to or removed from.

UpdateAndroidResources

Updates the `Resource.designer.cs` file.

This target is usually called by the IDE when new resources are added to the project.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Build properties

Article • 10/31/2024

MSBuild properties control the behavior of the [targets](#).

They're specified within the project file, for example **MyApp.csproj**, within an [MSBuild PropertyGroup](#).

ⓘ Note

In .NET for Android there is technically no distinction between an application and a bindings project, so properties will work in both. In practice it is highly recommended to create separate application and bindings projects. Properties that are primarily used in bindings projects are documented in the [MSBuild bindings project properties](#) reference guide.

AdbTarget

The `$(AdbTarget)` property specifies the Android target device the Android package may be installed to or removed from. The value of this property is the same as the [adb Target Device option](#).

AfterGenerateAndroidManifest

MSBuild Targets listed in this property will run directly after the internal `_GenerateJavaStubs` target, which is where the `AndroidManifest.xml` file is generated in the `$(IntermediateOutputPath)`. If you want to make any modifications to the generated `AndroidManifest.xml` file, you can do that using this extension point.

AndroidAapt2CompileExtraArgs

Specifies command-line options to pass to the `aapt2 compile` command when processing Android assets and resources.

AndroidAapt2LinkExtraArgs

Specifies command-line options to pass to the `aapt2 link` command when processing Android assets and resources.

AndroidAddKeepAlives

A boolean property that controls whether the linker will insert `GC.KeepAlive()` invocations within binding projects to prevent premature object collection.

The default value is `True` for Release configuration builds.

AndroidAotAdditionalArguments

A string property that allows passing options to the Mono compiler during the `Aot` task for projects that have either `$(AndroidEnableProfiledAot)` or `$(AotAssemblies)` set to `true`. The string value of the property is added to the response file when calling the Mono cross-compiler.

In general, this property should be left blank, but in certain special scenarios it might provide useful flexibility.

The `$(AndroidAotAdditionalArguments)` property is different from the related `$(AndroidExtraAotOptions)` property; `$(AndroidAotAdditionalArguments)` passes full standalone space-separated options like `--verbose` or `--debug` to the AOT compiler, while `$(AndroidExtraAotOptions)` contains comma-separated arguments which are part of the `--aot` option of the AOT compiler.

AndroidAotCustomProfilePath

The file that `aprofutil` should create to hold profiler data.

AndroidAotProfiles

A string property that allows the developer to add AOT profiles from the command line. It's a semicolon or comma-separated list of absolute paths.

AndroidAotProfilerPort

The port that `aprofutil` should connect to when obtaining profiling data.

AndroidAotEnableLazyLoad

Enable lazy (delayed) loading of AOT-d assemblies, instead of preloading them at the startup. The default value is `True` for Release builds with any form of AOT enabled.

Introduced in .NET 6.

AndroidApkDigestAlgorithm

A string value that specifies the digest algorithm to use with `jarsigner -digestalg`.

The default value is `SHA-256`.

AndroidApkSignerAdditionalArguments

A string property that allows the developer to provide arguments to the `apksigner` tool.

AndroidApkSigningAlgorithm

A string value that specifies the signing algorithm to use with `jarsigner -sigalg`.

The default value is `SHA256withRSA`.

AndroidApplication

A boolean value that indicates whether the project is for an Android Application (`True`) or for an Android Library Project (`False` or not present).

Only one project with `<AndroidApplication>True</AndroidApplication>` may be present within an Android package. (Unfortunately this requirement isn't verified, which can result in subtle and bizarre errors regarding Android resources.)

AndroidApplicationJavaClass

The full Java class name to use in place of `android.app.Application` when a class inherits from [Android.App.Application](#).

The `$(AndroidApplicationJavaClass)` property is generally set by *other* properties, such as the [\\$\(AndroidEnableMultiDex\)](#) MSBuild property.

AndroidAvoidEmitForPerformance

A boolean property that determines whether or not `System.Reflection.Emit` is "avoided" to improve startup performance. This property is `True` by default.

Usage of `System.Reflection.Emit` has a noticeable impact on startup performance on Android. This behavior is disabled by default for the following feature switches:

- `Switch.System.Reflection.ForceInterpretedInvoke`: after the second call to `MethodInfo.Invoke()` or `ConstructorInfo.Invoke()`, code is emitted to improve performance of repeated calls.
- `Microsoft.Extensions.DependencyInjection.DisableDynamicEngine`: after the second call to retrieve a service from a dependency injection container, code is emitted to improve performance of repeated calls.

It is desirable in most Android applications to disable this behavior.

See the [Base Class Libraries Feature Switches documentation](#) for details about available feature switches.

Added in .NET 8.

AndroidBinUtilsPath

A path to a directory containing the Android [binutils](#) such as `ld`, the native linker, and `as`, the native assembler. These tools are included in the .NET for Android workload.

The default value is `$(MonoAndroidBinDirectory)\binutils\bin\`.

AndroidBoundExceptionType

A string value that specifies how exceptions should be propagated when a .NET for Android-provided type implements a .NET type or interface in terms of Java types, for example `Android.Runtime.InputStreamInvoker` and `System.IO.Stream`, or `Android.Runtime.JavaDictionary` and `System.Collections.IDictionary`.

- `Java`: The original Java exception type is propagated as-is.

`Java` means that, for example, `InputStreamInvoker` doesn't properly implement the `System.IO.Stream` API because `Java.IO.IOException` may be thrown from `Stream.Read()` instead of `System.IO.IOException`.

- `System`: The original Java exception type is caught and wrapped in an appropriate .NET exception type.

`System` means that, for example, `InputStreamInvoker` properly implements `System.IO.Stream`, and `Stream.Read()` will *not* throw `Java.IO.IOException` instances. (It may instead throw a `System.IO.IOException` containing a `Java.IO.IOException` as the `Exception.InnerException` value.)

`System` is the default value.

AndroidBoundInterfacesContainConstants

A boolean property that determines whether binding constants on interfaces will be supported, or the workaround of creating an `IMyInterfaceConsts` class will be used.

The default value is `True`.

AndroidBoundInterfacesContainStaticAndDefaultInterfaceMembers

A boolean property that whether default and static members on interfaces will be supported, or old workaround of creating a sibling class containing static members like `abstract class MyInterface`.

The default value is `True` in .NET 6 and `False` for legacy.

AndroidBoundInterfacesContainTypes

A boolean property that whether types nested in interfaces will be supported, or the workaround of creating a non-nested type like `IMyInterfaceMyNestedClass`.

The default value is `True` in .NET 6 and `False` for legacy.

AndroidBuildApplicationPackage

A boolean value that indicates whether to create and sign the package (.apk). Setting this value to `True` is equivalent to using the [SignAndroidPackage](#) build target.

This property is `False` by default.

AndroidBundleConfigurationFile

Specifies a filename to use as a [configuration file](#) for `bundletool` when building an Android App Bundle. This file controls some aspects of how APKs are generated from the bundle, such as on what dimensions the bundle is split to produce APKs. .NET for Android configures some of these settings automatically, including the list of file extensions to leave uncompressed.

This property is only relevant if `$(AndroidPackageFormat)` is set to `aab`.

AndroidBundleToolExtraArgs

Specifies command-line options to pass to the `bundletool` command when build app bundles.

AndroidClassParser

A string property that controls how `.jar` files are parsed. Possible values include:

- `class-parse`: Uses `class-parse.exe` to parse Java bytecode directly, without assistance of a JVM.
- `jar2xml`: this value is obsolete and is no longer supported.

AndroidCodegenTarget

A string property that controls the code generation target ABI. Possible values include:

- **XamarinAndroid**: this value is obsolete and is no longer supported.
- **XAJavaInterop1**: Use Java.Interop for JNI invocations. Binding assemblies using `XAJavaInterop1` can only build and execute with Xamarin.Android 6.1 or later. Xamarin.Android 6.1 and later bind `Mono.Android.dll` with this value.

The default value is `XAJavaInterop1`.

AndroidCreatePackagePerAbi

A boolean property that determines if a set of files--one per ABI specified in `$(AndroidSupportedAbis)`--should be created instead of having support for all ABIs in a single `.apk`.

See also the [Building ABI-Specific APKs](#) guide.

AndroidCreateProguardMappingFile

A boolean property that controls if a proguard mapping file is generated as part of the build process.

Adding the following to your csproj will cause the file to be generated, and uses the [AndroidProguardMappingFile](#) property to control the location of the final mapping file.

XML

```
<AndroidCreateProguardMappingFile>True</AndroidCreateProguardMappingFile>
```

When producing `.aab` files, the mapping file is automatically included in your package. There is no need to upload it to the Google Play Store manually. When using `.apk` files, the [AndroidProguardMappingFile](#) will need to be manually uploaded.

The default value is `True` when using `$(AndroidLinkTool)=r8`.

AndroidDebugKeyAlgorithm

Specifies the default algorithm to use for the `debug.keystore`. The default value is `RSA`.

AndroidDebugKeyValidity

Specifies the default validity to use for the `debug.keystore`. The default value is `10950` or `30 * 365` or `30 years`.

AndroidDebugStoreType

Specifies the key store file format to use for the `debug.keystore`. It defaults to `pkcs12`.

AndroidDeviceUserId

Allows deploying and debugging the application under guest or work accounts. The value is the `uid` value you get from the following adb command:


```
shell
```

```
adb shell pm list users
```

The above command will return the following data:

```
Users:
  UserInfo{0:Owner:c13} running
  UserInfo{10:Guest:404}
```

The `uid` is the first integer value. In the above output, they're `0` and `10`.

AndroidDexTool

An enum-style property with a valid value of `d8`. *Previously, a value of `dx` was supported in Xamarin.Android.*

Indicates which Android [dex](#) compiler is used during the .NET for Android build process. The default value is `d8`. See our documentation on [D8 and R8](#).

AndroidEnableDesugar

A boolean property that determines if `desugar` is enabled. Android doesn't currently support all Java 8 features, and the default toolchain implements the new language features by performing bytecode transformations, called `desugar`, on the output of the `javac` compiler. The default value is `False` if using `$(AndroidDexTool)=dx` and `True` if using `$(AndroidDexTool)=d8`.

AndroidEnableGooglePlayStoreChecks

A bool property that allows developers to disable the following Google Play Store checks: XA1004, XA1005 and XA1006. Disabling these checks is useful for developers who are not targeting the Google Play Store and do not wish to run those checks.

AndroidEnableMarshalMethods

A bool property, that determines whether or not LLVM marshal methods are enabled. LLVM marshal methods are an app startup optimization which uses native entry points for Java `native` method registration.

This property is `False` by default.

Added in .NET 8.

AndroidEnableMultiDex

A boolean property that determines whether or not multi-dex support will be used in the final `.apk`.

This property is `False` by default.

AndroidEnableObsoleteOverrideInheritance

A boolean property that determines if bound methods automatically inherit `[Obsolete]` attributes from methods they override.

Support for this property was added in .NET 8.

This property is `True` by default.

AndroidEnablePreloadAssemblies

A boolean property that controls whether or not all managed assemblies bundled within the application package are loaded during process startup or not.

When set to `True`, all assemblies bundled within the application package will be loaded during process startup, before any application code is invoked.

When set to `False`, assemblies will only be loaded on an as-needed basis. Loading assemblies on an as-needed basis allows applications to launch faster, and is also more consistent with desktop .NET semantics. To see the time savings, set the `debug.mono.log` System Property to include `timing`, and look for the `Finished loading assemblies: preloaded` message within `adb logcat`.

Applications or libraries, which use dependency injection may *require* that this property be `True` if they in turn require that `AppDomain.CurrentDomain.GetAssemblies()` return all assemblies within the application bundle, even if the assembly wouldn't otherwise have been needed.

By default this value is `False`.

AndroidEnableProfiledAot

A boolean property that determines whether or not the AOT profiles are used during Ahead-of-Time compilation.

The profiles are listed in [@\(AndroidAotProfile\)](#) item group. This ItemGroup contains default profile(s). It can be overridden by removing the existing one(s) and adding your own AOT profiles.

This property is `False` by default.

AndroidEnableRestrictToAttributes

An enum-style property with valid values of `obsolete` and `disable`.

When set to `obsolete`, types and members that are marked with the Java annotation `androidx.annotation.RestrictTo` or are in non-exported Java packages will be marked with an `[Obsolete]` attribute in the C# binding.

This `[Obsolete]` attribute has a descriptive message explaining that the Java package owner considers the API to be "internal" and warns against its use.

This attribute also has a custom warning code `XA0BS001` so that it can be suppressed independently of "normal" obsolete API.

When set to `disable`, API will be generated as normal with no additional attributes. (This is the same behavior as before .NET 8.)

Adding `[Obsolete]` attributes instead of automatically removing the API was done to preserve API compatibility with existing packages. If you would instead prefer to *remove* members that have the `@RestrictTo` annotation or are in

non-exported Java packages, you can use [Transform files](#) in addition to this property to prevent these types from being bound:

XML

```
<remove-node path="//*[@annotated-visibility]" />
```

Support for this property was added in .NET 8.

This property is set to `obsolete` by default.

AndroidEnableSGenConcurrent

A boolean property that determines whether or not Mono's [concurrent GC collector](#) will be used.

This property is `False` by default.

AndroidErrorOnCustomJavaObject

A boolean property that determines whether types may implement `Android.Runtime.IJavaObject` *without* also inheriting from `Java.Lang.Object` or `Java.Lang.Throwable`:

C#

```
class BadType : IJavaObject {
    public IntPtr Handle {
        get {return IntPtr.Zero;}
    }

    public void Dispose()
    {
    }
}
```

When True, such types will generate an XA4212 error, otherwise an XA4212 warning will be generated.

This property is `True` by default.

AndroidExplicitCrunch

This property is no longer supported.

AndroidExtraAotOptions

A string property that allows passing options to the Mono compiler during the `Aot` task for projects that have either `$(AndroidEnableProfiledAot)` or `$(AotAssemblies)` set to `true`. The string value of the property is added to the response file when calling the Mono cross-compiler.

In general, this property should be left blank, but in certain special scenarios it might provide useful flexibility.

The `$(AndroidExtraAotOptions)` property is different from the related `$(AndroidAotAdditionalArguments)` property; `$(AndroidAotAdditionalArguments)` places comma-separated arguments into the `--aot` option of the Mono compiler. `$(AndroidExtraAotOptions)` instead passes full standalone space-separated options like `--verbose` or `--debug` to the compiler.

AndroidFastDeploymentType

A `:` (colon)-separated list of values to control what types can be deployed to the [Fast Deployment directory](#) on the target device when the `$(EmbedAssembliesIntoApk)` MSBuild property is `False`. If a resource is fast deployed, it is *not* embedded into the generated `.apk`, which can speed up deployment times. (The more that is fast deployed, then the less frequently the `.apk` needs to be rebuilt, and the install process can be faster.) Valid values include:

- `Assemblies`: Deploy application assemblies.
- `Dexes`: Deploy `.dex` files, native libraries and typemaps. The `Dexes` value can *only* be used on devices running **Android 4.4 or later (API-19)**.

The default value is `Assemblies`.

Support for Fast Deploying resources and assets via that system was removed in commit [f0d565fe](#). This was because it required the use of deprecated API's to work.

****Support for this feature was removed in .NET 9**

Experimental.

AndroidFragmentType

Specifies the default fully qualified type to be used for all `<fragment>` layout elements when generating the layout bindings code. The default value is the standard Android `Android.App.Fragment` type.

AndroidGenerateJniMarshalMethods

A bool property that enables generating of JNI marshal methods as part of the build process. This greatly reduces the `System.Reflection` usage in the binding helper code.

The default value is `False`. If developers wish to use the new JNI marshal methods feature, they can set

XML

```
<AndroidGenerateJniMarshalMethods>True</AndroidGenerateJniMarshalMethods>
```

in their `.csproj`. Alternatively provide the property on the command line via

shell

```
-p:AndroidGenerateJniMarshalMethods=True
```

Experimental. The default value is `False`.

AndroidGenerateJniMarshalMethodsAdditionalArguments

A string property that can be used to add parameters to the `jnimarshalmethod-gen.exe` invocation, and is useful for debugging, so that options such as `-v`, `-d`, or `--keeptemp` can be used.

Default value is empty string. It can be set in the `.csproj` file or on the command line. For example:

XML

```
<AndroidGenerateJniMarshalMethodsAdditionalArguments>-v -d --
keeptemp</AndroidGenerateJniMarshalMethodsAdditionalArguments>
```

or:

```
shell
```

```
-p:AndroidGenerateJniMarshalMethodsAdditionalArguments="-v -d --keeptemp"
```

AndroidGenerateLayoutBindings

Enables generation of [layout code-behind](#) if set to `true` or disables it completely if set to `false`.

The default value is `false`.

AndroidGenerateResourceDesigner

The default value is `true`. When set to `false`, disables the generation of `Resource.designer.cs`.

AndroidHttpClientHandlerType

Controls the default `System.Net.Http.HttpMessageHandler` implementation which will be used by the `System.Net.Http.HttpClient` default constructor. The value is an assembly-qualified type name of an `HttpMessageHandler` subclass, suitable for use with [System.Type.GetType\(string\)](#).

In .NET 6 and newer, this property has effect only when used together with `$(UseNativeHttpHandler)=true`. The most common values for this property are:

- `Xamarin.Android.Net.AndroidMessageHandler`: Use the Android Java APIs to perform HTTP requests. It is similar to the legacy `Xamarin.Android.Net.AndroidClientHandler` with several improvements. It supports HTTP 1.1 and TLS 1.2. It is the default HTTP message handler.
- `System.Net.Http.SocketsHttpHandler`, `System.Net.Http`: The default message handler in .NET. It supports HTTP/2, TLS 1.2, and it is the recommended HTTP message handler to use with [Grpc.Net.Client](#). This value is equivalent to `$(UseNativeHttpHandler)=false`.
- Unset/the empty string, which is equivalent to `System.Net.Http.HttpClientHandler`, `System.Net.Http`

Corresponds to the **Default** option in the Visual Studio property pages.

The new project wizard selects this option for new projects when the **Minimum Android Version** is configured to **Android 4.4.87** or lower in Visual Studio or when **Target Platforms** is set to **Modern Development** or **Maximum Compatibility** in Visual Studio for Mac.

- `System.Net.Http.HttpClientHandler`, `System.Net.Http`: Use the managed `HttpMessageHandler`.

Corresponds to the **Managed** option in the Visual Studio property pages.

Note

In .NET 6, the type you specify must not be `Xamarin.Android.Net.AndroidClientHandler` or `System.Net.Http.HttpClientHandler` or inherit from either of these classes. If you are migrating from "classic"

Xamarin.Android, use `AndroidMessageHandler` or derive your custom handler from it instead.

ⓘ Note

Support for the `$(AndroidHttpClientHandlerType)` property works by setting the **XA HTTP CLIENT HANDLER TYPE environment variable**. A `$XA_HTTP_CLIENT_HANDLER_TYPE` value found in a file with a Build action of `@(AndroidEnvironment)` will take precedence.

AndroidIncludeWrapSh

A boolean value that indicates whether the Android wrapper script ([wrap.sh](#)) should be packaged into the APK. The default value is `false` since the wrapper script may significantly influence the way the application starts up and works and the script should be included only when necessary, for example when debugging or otherwise changing the application startup/runtime behavior.

The script is added to the project using the `@(AndroidNativeLibrary)` build action, because it is placed in the same directory as architecture-specific native libraries, and must be named `wrap.sh`.

The easiest way to specify path to the `wrap.sh` script is to put it in a directory named after the target architecture. This approach will work if you have just one `wrap.sh` per architecture:

XML

```
<AndroidNativeLibrary Include="path/to/arm64-v8a/wrap.sh" />
```

However, if your project needs more than one `wrap.sh` per architecture, for different purposes, this approach won't work. Instead, in such cases the name can be specified using the `Link` metadata of the `AndroidNativeLibrary`:

XML

```
<AndroidNativeLibrary Include="/path/to/my/arm64-wrap.sh">
  <Link>lib\arm64-v8a\wrap.sh</Link>
</AndroidNativeLibrary>
```

If the `Link` metadata is used, the path specified in its value must be a valid native architecture-specific library path, relative to the APK root directory. The format of the path is `lib\ARCH\wrap.sh` where `ARCH` can be one of:

- `arm64-v8a`
- `armeabi-v7a`
- `x86_64`
- `x86`

AndroidIncludeAssetPacksInPackage

This property controls if an Asset Packs build automatically are auto included in the final `.aab` file. It will default to `true`.

In certain cases the user might want to release an interim release. In these cases the user does not need to update the asset pack. Especially if the contents of the asset pack have not changed. This property allows the user to skip the asset packs if they are not required.

Added in .NET 9

AndroidInstallJavaDependencies

The default value is `true` for command line builds. When set to `true`, enables installation of the Java SDK when running the `<InstallAndroidDependencies/>` target.

Support for this property was added in .NET 9.

AndroidJavadocVerbosity

Specifies how "verbose" [C# XML Documentation Comments](#) should be when importing Javadoc documentation within binding projects.

Requires use of the [@\(JavaSourceJar\)](#) build action.

The `$(AndroidJavadocVerbosity)` property is enum-like, with possible values of `full` or `intellisense`:

- `intellisense`: Only emit the XML comments: `<exception/>`, `<param/>`, `<returns/>`, `<summary/>`.
- `full`: Emit `intellisense` elements, as well as `<remarks/>`, `<seealso/>`, and anything else that's supportable.

The default value is `intellisense`.

AndroidKeyStore

A boolean value that indicates whether custom signing information should be used. The default value is `False`, meaning that the default debug-signing key will be used to sign packages.

AndroidLaunchActivity

The Android activity to launch.

AndroidLinkMode

Specifies which type of [linking](#) should be performed on assemblies contained within the Android package. Only used in Android Application projects. The default value is `SdkOnly`. Valid values are:

- None**: No linking will be attempted.
- SdkOnly**: Linking will be performed on the base class libraries only, not user's assemblies.
- Full**: Linking will be performed on base class libraries and user assemblies.

ⓘ Note

Using an `AndroidLinkMode` value of `Full` often results in broken apps, particularly when Reflection is used. Avoid unless you *really* know what you're doing.

XML

```
<AndroidLinkMode>SdkOnly</AndroidLinkMode>
```

AndroidLinkResources

When `true`, the build system will link out the Nested Types of the `Resource.Designer.cs` `Resource` class in all assemblies. The IL code that uses those types will be updated to use the values directly rather than accessing fields.

Linking out the nested types can have a small impact on reducing the apk size, and can also help with startup performance. Only "Release" builds are linked.

Experimental. Only designed to work with code such as

C#

```
var view = FindViewById<Resources.Ids.foo>;
```

Any other scenarios (such as reflection) will not be supported.

AndroidLinkSkip

Specifies a semicolon-delimited (`;`) list of assembly names, without file extensions, of assemblies that should not be linked. Only used within Android Application projects.

XML

```
<AndroidLinkSkip>Assembly1;Assembly2</AndroidLinkSkip>
```

AndroidLinkTool

An enum-style property with valid values of `proguard` or `r8`. Indicates which code shrinker is used for Java code. The default value is an empty string, or `proguard` if `$(AndroidEnableProguard)` is `True`. See our documentation on [D8 and R8](#).

AndroidLintEnabled

A bool property that allows the developer to run the android `lint` tool as part of the packaging process.

When `$(AndroidLintEnabled)=True`, the following properties are used:

- `$(AndroidLintEnabledIssues)`:
- `$(AndroidLintDisabledIssues)`:
- `$(AndroidLintCheckIssues)`:

The following build actions may also be used:

- `@(AndroidLintConfig)`:

See [Lint Help](#) for more details on the android `lint` tooling.

AndroidLintEnabledIssues

A string property that is a comma-separated list of lint issues to enable.

Only used when `$(AndroidLintEnabled)=True`.

AndroidLintDisabledIssues

A string property that is a comma-separated list of lint issues to disable.

Only used when `$(AndroidLintEnabled)=True`.

AndroidLintCheckIssues

A string property that is a comma-separated list of lint issues to check.

Only used when `$(AndroidLintEnabled)=True`.

Note: only these issues will be checked.

AndroidManagedSymbols

A boolean property that controls whether sequence points are generated so that file name and line number information can be extracted from `Release` stack traces.

AndroidManifest

Specifies a filename to use as the template for the app's [AndroidManifest.xml](#). During the build, any other necessary values will be merged into to produce the actual `AndroidManifest.xml`. The `$(AndroidManifest)` must contain the package name in the `/manifest/@package` attribute.

AndroidManifestMerger

Specifies the implementation for merging *AndroidManifest.xml* files. This is an enum-style property where `legacy` selects the original C# implementation and `manifestmerger.jar` selects Google's Java implementation.

The default value is currently `manifestmerger.jar`. If you want to use the old version add the following to your csproj

XML

```
<AndroidManifestMerger>legacy</AndroidManifestMerger>
```

Google's merger enables support for `xmlns:tools="http://schemas.android.com/tools"` as described in the [Android documentation](#).

AndroidManifestMergerExtraArgs

A string property to provide arguments to the [Android documentation](#) tool.

If you want detailed output from the tool you can add the following to the `.csproj`.

XML

```
<AndroidManifestMergerExtraArgs>--log VERBOSE</AndroidManifestMergerExtraArgs>
```

AndroidManifestType

An enum-style property with valid values of `Xamarin` or `GoogleV2`. This controls which repository is used by the [InstallAndroidDependencies](#) target to determine which Android packages and package versions are available and can

be installed.

Xamarin is the **Approved List (Recommended)** repository within the [Visual Studio SDK Manager](#).

GoogleV2 is the **Full List (Unsupported)** repository within the [Visual Studio SDK Manager](#).

If `$(AndroidManifestType)` is not set, then `Xamarin` is used.

AndroidManifestPlaceholders

A semicolon-separated list of key-value replacement pairs for *AndroidManifest.xml*, where each pair has the format `key=value`.

For example, a property value of `assemblyName=$(AssemblyName)` defines an `${assemblyName}` placeholder that can then appear in *AndroidManifest.xml*:

XML

```
<application android:label="${assemblyName}"
```

This provides a way to insert variables from the build process into the *AndroidManifest.xml* file.

AndroidMultiDexClassListExtraArgs

A string property which allows developers to pass arguments to the `com.android.multidex.MainDexListBuilder` when generating the `multidex.keep` file.

One specific case is if you are getting the following error during the `dx` compilation.

text

```
com.android.dex.DexException: Too many classes in --main-dex-list, main dex capacity exceeded
```

If you are getting this error you can add the following to the `.csproj`.

XML

```
<DxExtraArguments>--force-jumbo </DxExtraArguments>
<AndroidMultiDexClassListExtraArgs>--disable-annotation-resolution-
workaround</AndroidMultiDexClassListExtraArgs>
```

which will allow the `dx` step to succeed.

AndroidPackageFormat

An enum-style property with valid values of `apk` or `aab`. Indicates if you want to package the Android application as an [APK file](#) or [Android App Bundle](#). App Bundles are a new format for `Release` builds that are intended for submission on Google Play. The default value is `apk`.

When `$(AndroidPackageFormat)` is set to `aab`, other MSBuild properties are set, which are required for Android App Bundles:

- `$(AndroidUseAapt2)` is `True`.
- `$(AndroidUseApkSigner)` is `False`.

- `$(AndroidCreatePackagePerAbi)` is `False`.

This property will be deprecated for .net 6. Users should switch over to the newer [AndroidPackageFormats](#).

AndroidPackageFormats

A semi-colon delimited property with valid values of `apk` and `aab`. Indicates if you want to package the Android application as an [APK file](#) or [Android App Bundle](#). App Bundles are a new format for `Release` builds that are intended for submission on Google Play.

When building a Release build you might want to generate both and `aab` and an `apk` for distribution to various stores.

Setting `AndroidPackageFormats` to `aab;apk` will result in both being generated. Setting `AndroidPackageFormats` to either `aab` or `apk` will generate only one file.

The default value is `aab;apk` for `Release` builds only. It is recommended that you continue to use just `apk` for debugging.

AndroidPackageNamingPolicy

An enum-style property for specifying the Java package names of generated Java source code.

The only supported value is `LowercaseCamel64`.

AndroidPrepareForBuildDependsOn

A semi-colon delimited property that can be used to extend the Android build process. MSBuild targets added to this property will execute early in the build for both Application and Library project types. This property is empty by default.

Example:

XML

```
<PropertyGroup>
  <AndroidPrepareForBuildDependsOn>MyCustomTarget</AndroidPrepareForBuildDependsOn>
</PropertyGroup>

<Target Name="MyCustomTarget" >
  <Message Text="Running target: 'MyCustomTarget'" Importance="high" />
</Target>
```

AndroidProguardMappingFile

Specifies the `-printmapping` proguard rule for `r8`. This will mean the `mapping.txt` file will be produced in the `$(OutputPath)` folder. This file can then be used when uploading packages to the Google Play Store.

By default this file is produced automatically when using `AndroidLinkTool=r8` and will generate the following file `$(OutputPath)mapping.txt`.

If you do not want to generate this mapping file you can use the [AndroidCreateProguardMappingFile](#) property to stop creating it. Add the following in your project

XML

```
<AndroidCreateProguardMappingFile>False</AndroidCreateProguardMappingFile>
```

or use `-p:AndroidCreateProguardMappingFile=False` on the command line.

AndroidD8IgnoreWarnings

Specifies `--map-diagnostics warning info` to be passed to `d8`. The default value is `True`, but can be set to `False` to enforce more strict behavior. See the [D8 and R8 source code](#) for details.

Added in .NET 8.

AndroidR8IgnoreWarnings

Specifies the `-ignorewarnings` proguard rule for `r8`. This allows `r8` to continue with dex compilation even if certain warnings are encountered. The default value is `True`, but can be set to `False` to enforce more strict behavior. See the [ProGuard manual](#) for details.

Starting in .NET 8, specifies `--map-diagnostics warning info`. See the [D8 and R8 source code](#) for details.

AndroidR8JarPath

The path to `r8.jar` for use with the `r8` dex-compiler and shrinker. The default value is a path into the .NET for Android workload installation. For further information see our documentation on [D8 and R8](#).

AndroidResgenExtraArgs

Specifies command-line options to pass to the `aapt` command when processing Android assets and resources.

AndroidResgenFile

Specifies the name of the Resource file to generate. The default template sets this to `Resource.designer.cs`.

AndroidResourceDesignerClassModifier

Specifies the class modifier for the intermediate `Resource` class which is generated. Valid values are `public` and `internal`.

By default this will be `public`.

Added in .NET 9.

AndroidSdkBuildToolsVersion

The Android SDK build-tools package provides the `aapt` and `zipalign` tools, among others. Multiple different versions of the build-tools package may be installed simultaneously. The build-tools package chosen for packaging is done by checking for and using a "preferred" build-tools version if it is present; if the "preferred" version is *not* present, then the highest versioned installed build-tools package is used.

The `$(AndroidSdkBuildToolsVersion)` MSBuild property contains the preferred build-tools version. The .NET for Android build system provides a default value in `Xamarin.Android.Common.targets`, and the default value may be overridden within your project file to choose an alternate build-tools version, if (for example) the latest aapt is crashing out while a previous aapt version is known to work.

AndroidSigningKeyAlias

Specifies the alias for the key in the keystore. This is the `keytool -alias` value used when creating the keystore.

AndroidSigningKeyPass

Specifies the password of the key within the keystore file. This is the value entered when `keytool` asks **Enter key password for** `$(AndroidSigningKeyAlias)`.

This property also supports `env:` and `file:` prefixes that can be used to specify an environment variable or file that contains the password. These options provide a way to prevent the password from appearing in build logs.

For example, to use an environment variable named *AndroidSigningPassword*:

XML

```
<PropertyGroup>
  <AndroidSigningKeyPass>env:AndroidSigningPassword</AndroidSigningKeyPass>
</PropertyGroup>
```

To use a file located at `C:\Users\user1\AndroidSigningPassword.txt`:

XML

```
<PropertyGroup>
  <AndroidSigningKeyPass>file:C:\Users\user1\AndroidSigningPassword.txt</AndroidSigningKeyPass>
</PropertyGroup>
```

Note

The `env:` prefix is not supported when `$(AndroidPackageFormat)` is set to `aab`.

AndroidSigningKeyStore

Specifies the filename of the keystore file created by `keytool`. This corresponds to the value provided to the `keytool -keystore` option.

AndroidSigningStorePass

Specifies the password to `$(AndroidSigningKeyStore)`. This is the value provided to `keytool` when creating the keystore file and asked **Enter keystore password:**.

This property also supports `env:` and `file:` prefixes that can be used to specify an environment variable or file that contains the password. These options provide a way to prevent the password from appearing in build logs.

For example, to use an environment variable named *AndroidSigningPassword*:

XML

```
<PropertyGroup>
  <AndroidSigningStorePass>env:AndroidSigningPassword</AndroidSigningStorePass>
</PropertyGroup>
```

To use a file located at `C:\Users\user1\AndroidSigningPassword.txt`:

XML

```
<PropertyGroup>
  <AndroidSigningStorePass>file:C:\Users\user1\AndroidSigningPassword.txt</AndroidSigningStorePass>
</PropertyGroup>
```

ⓘ Note

The `env:` prefix is not supported when `$(AndroidPackageFormat)` is set to `aab`.

AndroidSigningPlatformKey

Specifies the key file to use to sign the apk. This is only used when building `system` applications.

AndroidSigningPlatformCert

Specifies the certificate file to use to sign the apk. This is only used when building `system` applications.

AndroidStripILAfterAOT

A bool property that specifies whether or not the *method bodies* of AOT compiled methods will be removed.

The default value is `false`, and the method bodies of AOT compiled methods will *not* be removed.

When set to `true`, `$(AndroidEnableProfiledAot)` is set to `false` by default. This means that in Release configuration builds -- in which `$(RunAOTCompilation)` is `true` by default -- AOT is enabled for *everything*. This can result in increased app sizes. This behavior can be overridden by explicitly setting `$(AndroidEnableProfiledAot)` to `true` within your project file.

Support for this property was added in .NET 8.

AndroidSupportedAbis

A string property that contains a semicolon (;)-delimited list of ABIs which should be included into the `.apk`.

Supported values include:

- `armeabi-v7a`
- `x86`
- `arm64-v8a`
- `x86_64`

AndroidTlsProvider

This property is obsolete and should not be used.

AndroidUseAapt2

A boolean property that allows the developer to control the use of the `aapt2` tool for packaging. By default this will be `True`.

This property cannot be set to `false`.

AndroidUseApkSigner

A bool property that allows the developer to use the `apksigner` tool rather than `jarsigner`.

AndroidUseDefaultAotProfile

A bool property that allows the developer to suppress usage of the default AOT profiles.

To suppress the default AOT profiles, set the property to `false`.

AndroidUseDesignerAssembly

A bool property which controls if the build system will generate an `_Microsoft.Android.Resource.Designer.dll` as apposed to a `Resource.Designer.cs` file. The benefits of this are smaller applications and faster startup time.

The default value is `true` in .NET 8.

As a Nuget Author it is recommended that you ship three versions of the assembly if you want to maintain backward compatibility. One for MonoAndroid, one for net6.0-android and one for net8.0-android. You can do this by using [Xamarin.Legacy.Sdk](#). This is only required if your Nuget Library project makes use of `AndroidResource` items in the project or via a dependency.

XML

```
<TargetFrameworks>monoandroid90;net6.0-android;net8.0-android</TargetFrameworks>
```

Alternatively turn this setting off until such time as both Classic and net7.0-android have been deprecated.

.NET 8 Projects which choose to turn this setting off will not be able to consume references which do use it. If you try to use an assembly which does have this feature enabled in a project that does not, you will get a `XA1034` build error.

Added in .NET 8.

AndroidUseInterpreter

A boolean property that causes the `.apk` to contain the mono *interpreter*, and not the normal JIT.

Experimental.

AndroidUseLegacyVersionCode

A boolean property that allows the developer to revert the versionCode calculation back to its old pre Xamarin.Android 8.2 behavior. This should ONLY be used for developers with existing applications in the Google Play

Store. It is highly recommended that the new `$(AndroidVersionCodePattern)` property is used.

AndroidUseManagedDesignTimeResourceGenerator

A boolean property that will switch over the design time builds to use the managed resource parser rather than `aapt`.

AndroidUseNegotiateAuthentication

A boolean property that enables support for NTLMv2/Negotiate authentication in `AndroidMessageHandler`. The default value is `False`.

Added in .NET 7.

AndroidUseSharedRuntime

This property is obsolete and should not be used.

AndroidVersionCode

An MSBuild property that can be used as an alternative to `/manifest/@android:versionCode` in the [AndroidManifest.xml](#) file. To opt into this feature you must also enable `<GenerateApplicationManifest>true</GenerateApplicationManifest>`. This will be the default going forward in .NET 6.

This property is ignored if `$(AndroidCreatePackagePerAbi)` and `$(AndroidVersionCodePattern)` are used.

`@android:versionCode` is an integer value that must be incremented for each Google Play release. See the [Android documentation](#) for further details about the requirements for `/manifest/@android:versionCode`.

AndroidVersionCodePattern

A string property that allows the developer to customize the `versionCode` in the manifest. See [Creating the Version Code for the APK](#) for information on deciding a `versionCode`.

Some examples, if `abi` is `armeabi` and `versionCode` in the manifest is `123`, `{abi}{versionCode}` will produce a `versionCode` of `1123` when `$(AndroidCreatePackagePerAbi)` is `True`, otherwise will produce a value of `123`. If `abi` is `x86_64` and `versionCode` in the manifest is `44`. This will produce `544` when `$(AndroidCreatePackagePerAbi)` is `True`, otherwise will produce a value of `44`.

If we include a left padding format string `{abi}{versionCode:0000}`, it would produce `50044` because we are left padding the `versionCode` with `0`. Alternatively, you can use the decimal padding such as `{abi}{versionCode:D4}` which does the same as the previous example.

Only '0' and 'Dx' padding format strings are supported since the value MUST be an integer.

Pre-defined key items

- **abi** – Inserts the targeted abi for the app
 - 2 – `armeabi-v7a`
 - 3 – `x86`
 - 4 – `arm64-v8a`
 - 5 – `x86_64`

- **minSDK** – Inserts the minimum supported Sdk value from the `AndroidManifest.xml` or `11` if none is defined.
- **versionCode** – Uses the version code directly from `Properties\AndroidManifest.xml`.

You can define custom items using the `$(AndroidVersionCodeProperties)` property (defined next).

By default the value will be set to `{abi}{versionCode:D6}`. If a developer wants to keep the old behavior you can override the default by setting the `$(AndroidUseLegacyVersionCode)` property to `true`

AndroidVersionCodeProperties

A string property that allows the developer to define custom items to use with the `$(AndroidVersionCodePattern)`.

They are in the form of a `key=value` pair. All items in the `value` should be integer values. For example:

`screen=23;target=$( _AndroidApiLevel)`. As you can see you can make use of existing or custom MSBuild properties in the string.

ApplicationId

An MSBuild property that can be used as an alternative to `/manifest/@package` in the `AndroidManifest.xml` file. To opt into this feature you must also enable `<GenerateApplicationManifest>true</GenerateApplicationManifest>`. This will be the default going forward in .NET 6.

See the [Android documentation](#) for further details about the requirements for `/manifest/@package`.

ApplicationTitle

An MSBuild property that can be used as an alternative to `/manifest/application/@android:label` in the `AndroidManifest.xml` file. To opt into this feature you must also enable

`<GenerateApplicationManifest>true</GenerateApplicationManifest>`. This will be the default going forward in .NET 6.

See the [Android documentation](#) for further details about the requirements for

`/manifest/application/@android:label`.

ApplicationVersion

An MSBuild property that can be used as an alternative to `/manifest/@android:versionName` in the `AndroidManifest.xml` file. To opt into this feature you must also enable

`<GenerateApplicationManifest>true</GenerateApplicationManifest>`. This will be the default going forward in .NET 6.

See the [Android documentation](#) for further details about the requirements for `/manifest/@android:versionName`.

AotAssemblies

A boolean property that determines whether or not assemblies will be Ahead-of-Time compiled into native code and included in applications. This property is `False` by default.

Deprecated in .NET 7. Migrate to the new `$(RunAOTCompilation)` MSBuild property instead, as support for `$(AotAssemblies)` will be removed in a future release.

AProfUtilExtraOptions

Extra options to pass to `aprofutil`.

BeforeBuildAndroidAssetPacks

MSBuild Targets listed in this property will run directly before the `AssetPack` items are built.

Added in .NET 9

BeforeGenerateAndroidManifest

MSBuild Targets listed in this property will run directly before `_GenerateJavaStubs`.

Configuration

Specifies the build configuration to use, such as "Debug" or "Release". The Configuration property is used to determine default values for other properties which determine target behavior. Additional configurations may be created within your IDE.

By default, the `Debug` configuration will result in the [Install](#) and [SignAndroidPackage](#) targets creating a smaller Android package which requires the presence of other files and packages to operate.

The default `Release` configuration will result in the [Install](#) and [SignAndroidPackage](#) targets creating an Android package which is *stand-alone*, and may be used without installing any other packages or files.

DebugSymbols

A boolean value that determines whether the Android package is *debuggable*, in combination with the `$(DebugType)` property. A debuggable package contains debug symbols, sets the `//application/@android:debuggable` attribute [↗](#) to `true`, and automatically adds the [INTERNET](#) [↗](#) permission so that a debugger can attach to the process. An application is debuggable if `DebugSymbols` is `True` and `DebugType` is either the empty string or `Full`.

DebugType

Specifies the [type of debug symbols](#) to generate as part of the build, which also impacts whether the Application is debuggable. Possible values include:

- **Full:** Full symbols are generated. If the [DebugSymbols](#) MSBuild property is also `True`, then the Application package is debuggable.
- **PdbOnly:** "PDB" symbols are generated. The Application package is not debuggable.

If `DebugType` is not set or is the empty string, then the `DebugSymbols` property controls whether or not the Application is debuggable.

EmbedAssembliesIntoApk

A boolean property that determines whether or not the app's assemblies should be embedded into the Application package.

This property should be `True` for Release builds and `False` for Debug builds. It *may* need to be `True` in Debug builds if Fast Deployment doesn't support the target device.

When this property is `False`, then the `$(AndroidFastDeploymentType)` MSBuild property also controls what will be embedded into the `.apk`, which can impact deployment and rebuild times.

EnableLLVM

A boolean property that determines whether or not LLVM will be used when Ahead-of-Time compiling assemblies into native code.

The Android NDK must be installed to build a project that has this property enabled.

This property is `False` by default.

This property is ignored unless the `$(AotAssemblies)` MSBuild property is `True`.

EnableProguard

A boolean property that determines whether or not [proguard](#) is run as part of the packaging process to link Java code.

This property is `False` by default.

When `True`, `@(ProguardConfiguration)` files will be used to control `proguard` execution.

GenerateApplicationManifest

Enables or disables the following MSBuild properties that emit values in the final `AndroidManifest.xml` file:

- `$(AndroidVersionCode)`
- `$(ApplicationId)`
- `$(ApplicationTitle)`
- `$(ApplicationVersion)`

The default value `$(GenerateApplicationManifest)` is `true`.

JavaMaximumHeapSize

Specifies the value of the `java -Xmx` parameter value to use when building the `.dex` file as part of the packaging process. If not specified, then the `-Xmx` option supplies `java` with a value of `1G`. This was found to be commonly required on Windows in comparison to other platforms.

Specifying this property is necessary if the `_CompileDex` target throws a `java.lang.OutOfMemoryError`.

Customize the value by changing:

XML

```
<JavaMaximumHeapSize>1G</JavaMaximumHeapSize>
```

JavaOptions

Specifies command-line options to pass to `java` when building the `.dex` file.

JarsignerTimestampAuthorityCertificateAlias

This property allows you to specify an alias in the keystore for a timestamp authority. See the Java [Signature Timestamp Support](#) [↗](#) documentation for more details.

XML

```
<PropertyGroup>
  <JarsignerTimestampAuthorityCertificateAlias>Alias</JarsignerTimestampAuthorityCertificateAlias>
</PropertyGroup>
```

JarsignerTimestampAuthorityUrl

This property allows you to specify a URL to a timestamp authority service. This can be used to make sure your `.apk` signature includes a timestamp. See the Java [Signature Timestamp Support](#) [↗](#) documentation for more details.

XML

```
<PropertyGroup>
  <JarsignerTimestampAuthorityUrl>http://example.tsa.url</JarsignerTimestampAuthorityUrl>
</PropertyGroup>
```

LinkerDumpDependencies

A bool property that enables generating of linker dependencies file. This file can be used as input for [illinkanalyzer](#) [↗](#) tool.

The dependencies file named `linker-dependencies.xml.gz` is written to the project directory. On .NET5/6 it is written next to the linked assemblies in `obj/<Configuration>/android<ABI>/linked` directory.

The default value is False.

MandroidI18n

This MSBuild property is obsolete and is no longer supported.

MonoAndroidResourcePrefix

Specifies a *path prefix* that is removed from the start of filenames with a Build action of `AndroidResource`. This is to allow changing where resources are located.

The default value is `Resources`. Change this to `res` for the Java project structure.

MonoSymbolArchive

A boolean property that controls whether `.msym` artifacts are created for later use with `mono-symbolicate`, to extract “real” filename and line number information from Release stack traces.

This is True by default for “Release” apps which have debugging symbols enabled: `$(EmbedAssembliesIntoApk)` is True, `$(DebugSymbols)` is True, and `$(Optimize)` is True.

RunAOTCompilation

A boolean property that determines whether or not assemblies will be Ahead-of-Time compiled into native code and included in applications. This property is `False` by default for `Debug` builds and `True` by default for `Release` builds.

This MSBuild property replaces the `$(AotAssemblies)` MSBuild property from Xamarin.Android. This is the same property used for [Blazor WASM](#).

Feedback

Was this page helpful?



[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Build items

Article • 10/31/2024

Build items control how a .NET for Android application or library project is built.

They're specified within the project file, for example **MyApp.csproj**, within an [MSBuild ItemGroup](#).

ⓘ Note

In .NET for Android there is technically no distinction between an application and a bindings project, so build items will work in both. In practice it is highly recommended to create separate application and bindings projects. Build items that are primarily used in bindings projects are documented in the [MSBuild bindings project items](#) reference guide.

AndroidAdditionalJavaManifest

`<AndroidAdditionalJavaManifest>` is used in conjunction with [Java Dependency Resolution](#) to specify additional POM files that will be needed to verify dependencies. These are often parent or imported POM files referenced by a Java library's POM file.

XML

```
<ItemGroup>
  <AndroidAdditionalJavaManifest Include="mylib-parent.pom"
  JavaArtifact="com.example:mylib-parent" JavaVersion="1.0.0" />
</ItemGroup>
```

The following MSBuild metadata are required:

- `%(JavaArtifact)`: The group and artifact id of the Java library matching the specified POM file in the form `{GroupId}:{ArtifactId}`.
- `%(JavaVersion)`: The version of the Java library matching the specified POM file.

See the [Java Dependency Resolution documentation](#) for more details.

This build action was introduced in .NET 9.

AndroidAsset

Supports [Android Assets](#), files that would be included in the `assets` folder in a Java Android project.

Starting with .NET 9 the `@(AndroidAsset)` build action also supports additional metadata for generating [Asset Packs](#). The `%(AndroidAsset.AssetPack)` metadata can be used to automatically generate an asset pack of that name. This feature is only supported when the `$(AndroidPackageFormat)` is set to `.aab`. The following example will place `movie2.mp4` and `movie3.mp4` in separate asset packs.

XML

```
<ItemGroup>
  <AndroidAsset Update="Asset/movie.mp4" />
  <AndroidAsset Update="Asset/movie2.mp4" AssetPack="assets1" />
  <AndroidAsset Update="Asset/movie3.mp4" AssetPack="assets2" />
</ItemGroup>
```

This feature can be used to include large files in your application which would normally exceed the max package size limits of Google Play.

If you have a large number of assets it might be more efficient to make use of the `base` asset pack. In this scenario you update ALL assets to be in a single asset pack then use the `AssetPack="base"` metadata to declare which specific assets end up in the base aab file. With this you can use wildcards to move most assets into the asset pack.

XML

```
<ItemGroup>
  <AndroidAsset Update="Assets/*" AssetPack="assets1" />
  <AndroidAsset Update="Assets/movie.mp4" AssetPack="base" />
  <AndroidAsset Update="Assets/some.png" AssetPack="base" />
</ItemGroup>
```

In this example, `movie.mp4` and `some.png` will end up in the `base` aab file, while all the other assets will end up in the `assets1` asset pack.

The additional metadata is only supported on .NET for Android 9 and above.

AndroidAarLibrary

The Build action of `AndroidAarLibrary` should be used to directly reference `.aar` files. This build action will be most commonly used by Xamarin Components. Namely to

include references to `.aar` files that are required to get Google Play and other services working.

Files with this Build action will be treated in a similar fashion to the embedded resources found in Library projects. The `.aar` will be extracted into the intermediate directory. Then any assets, resource and `.jar` files will be included in the appropriate item groups.

AndroidAotProfile

Used to provide an AOT profile, for use with profile-guided AOT.

It can be also used from Visual Studio by setting the `AndroidAotProfile` build action to a file containing an AOT profile.

AndroidAppBundleMetaDataFile

Specifies a file that will be included as metadata in the Android App Bundle. The format of the flag value is `<bundle-path>:<physical-file>` where `bundle-path` denotes the file location inside the App Bundle's metadata directory, and `physical-file` is an existing file containing the raw data to be stored.

XML

```
<ItemGroup>
  <AndroidAppBundleMetaDataFile

  Include="com.android.tools.build.obfuscation/proguard.map:$(OutputPath)mapping.txt"
  />
</ItemGroup>
```

See [bundletool](#) documentation for more details.

AndroidBoundLayout

Indicates that the layout file is to have `code-behind generated` for it in case when the `$(AndroidGenerateLayoutBindings)` property is set to `false`. In all other aspects it is identical to `AndroidResource`.

This action can be used **only** with layout files:

XML


```
<AndroidBoundLayout Include="Resources\layout\Main.xml" />
```

AndroidEnvironment

Files with a Build action of `AndroidEnvironment` are used to [initialize environment variables and system properties during process startup](#). The `AndroidEnvironment` Build action may be applied to multiple files, and they will be evaluated in no particular order (so don't specify the same environment variable or system property in multiple files).

AndroidGradleProject

`<AndroidGradleProject>` can be used to build and consume the outputs of Android Gradle projects created in Android Studio or elsewhere.

The `Include` metadata should point to the top level `build.gradle` or `build.gradle.kts` file that will be used to build the project. This will be found in the root directory of your Gradle project, which should also contain `gradlew` wrapper scripts.

XML

```
<ItemGroup>
  <AndroidGradleProject Include="path/to/project/build.gradle.kts"
  ModuleName="mylibrary" />
</ItemGroup>
```

The following MSBuild metadata are supported:

- `%(Configuration)`: The name of the configuration to use to build or assemble the project or project module specified. The default value is `Release`.
- `%(ModuleName)`: The name of the [module or subproject](#) [↗](#) that should be built. The default value is empty.
- `%(OutputPath)`: Can be set to override the build output path of the Gradle project. The default value is `$(IntermediateOutputPath)gradle/%(ModuleName)%(Configuration)-{Hash}`.
- `%(CreateAndroidLibrary)`: Output AAR files will be added as an [AndroidLibrary](#) to the project. Metadata supported by `<AndroidLibrary>` like `%(Bind)` or `%(Pack)` will be forwarded if set. The default value is `true`.

This build action was introduced in .NET 9.

AndroidJavaLibrary

Files with a Build action of `AndroidJavaLibrary` are Java Archives (`.jar` files) that will be included in the final Android package.

AndroidIgnoredJavaDependency

`<AndroidIgnoredJavaDependency>` is used in conjunction with [Java Dependency Resolution](#).

It is used to specify a Java dependency that should be ignored. This can be used if a dependency will be fulfilled in a way that Java dependency resolution cannot detect.

XML

```
<!-- Include format is {GroupId}:{ArtifactId} -->
<ItemGroup>
  <AndroidIgnoredJavaDependency
    Include="com.google.errorprone:error_prone_annotations" Version="2.15.0" />
</ItemGroup>
```

The following MSBuild metadata are required:

- `%(Version)`: The version of the Java library matching the specified `%(Include)`.

See the [Java Dependency Resolution documentation](#) for more details.

This build action was introduced in .NET 9.

AndroidJavaSource

Files with a Build action of `AndroidJavaSource` are Java source code that will be included in the final Android package.

Starting with .NET 7, all `**\*.java` files within the project directory automatically have a Build action of `AndroidJavaSource`, and will be bound prior to the Assembly build. Allows C# code to easily use types and members present within the `**\*.java` files.

Set `%(AndroidJavaSource.Bind)` to False to disable this behavior.

AndroidLibrary

AndroidLibrary is a new build action for simplifying how `.jar` and `.aar` files are included in projects.

Any project can specify:

XML

```
<ItemGroup>
  <AndroidLibrary Include="foo.jar" />
  <AndroidLibrary Include="bar.aar" />
</ItemGroup>
```

The result of the above code snippet has a different effect for each .NET for Android project type:

- Application and class library projects:
 - `foo.jar` maps to [AndroidJavaLibrary](#).
 - `bar.aar` maps to [AndroidAarLibrary](#).
- Java binding projects:
 - `foo.jar` maps to [EmbeddedJar](#).
 - `foo.jar` maps to [EmbeddedReferenceJar](#) if `Bind="false"` metadata is added.
 - `bar.aar` maps to [LibraryProjectZip](#).

This simplification means you can use **AndroidLibrary** everywhere.

AndroidLintConfig

The Build action 'AndroidLintConfig' should be used in conjunction with the `$(AndroidLintEnabled)` property. Files with this build action will be merged together and passed to the android `lint` tooling. They should be XML files containing information on tests to enable and disable.

See the [lint documentation](#) for more details.

AndroidManifestOverlay

The `AndroidManifestOverlay` build action can be used to provide `AndroidManifest.xml` files to the [Manifest Merger](#) tool. Files with this build action will be passed to the Manifest Merger along with the main `AndroidManifest.xml` file and manifest files from references. These will then be merged into the final manifest.

You can use this build action to provide changes and settings to your app depending on your build configuration. For example, if you need to have a specific permission only while debugging, you can use the overlay to inject that permission when debugging. For example, given the following overlay file contents:

XML

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.CAMERA" />
</manifest>
```

You can use the following to add a manifest overlay for a debug build:

XML

```
<ItemGroup>
  <AndroidManifestOverlay Include="DebugPermissions.xml" Condition="
'$(Configuration)' == 'Debug' " />
</ItemGroup>
```

AndroidInstallModules

Specifies the modules that get installed by **bundletool** command when installing app bundles.

AndroidMavenLibrary

`<AndroidMavenLibrary>` allows a Maven artifact to be specified which will automatically be downloaded and added to a .NET for Android binding project. This can be useful to simplify maintenance of .NET for Android bindings for artifacts hosted in Maven.

XML

```
<!-- Include format is {GroupId}:{ArtifactId} -->
<ItemGroup>
  <AndroidMavenLibrary Include="com.squareup.okhttp3:okhttp" Version="4.9.3"
/>
</ItemGroup>
```

The following MSBuild metadata are supported:

- `%(Version)`: Required version of the Java library referenced by `%(Include)`.

- `%(Repository)`: Optional Maven repository to use. Supported values are `Central` (default), `Google`, or an `https` URL to a Maven repository.

The `<AndroidMavenLibrary>` item is translated to [AndroidLibrary](#), so any metadata supported by `<AndroidLibrary>` like `%(Bind)` or `%(Pack)` are also supported.

See the [AndroidMavenLibrary documentation](#) for more details.

This build action was introduced in .NET 9.

AndroidNativeLibrary

[Native libraries](#) are added to the build by setting their Build action to `AndroidNativeLibrary`.

Note that since Android supports multiple Application Binary Interfaces (ABIs), the build system must know the ABI the native library is built for. There are two ways the ABI can be specified:

1. Path "sniffing".
2. Using the `%(Abi)` item metadata.

With path sniffing, the parent directory name of the native library is used to specify the ABI that the library targets. Thus, if you add `lib/armeabi-v7a/libfoo.so` to the build, then the ABI will be "sniffed" as `armeabi-v7a`.

Item attribute name

Abi – Specifies the ABI of the native library.

XML

```
<ItemGroup>
  <AndroidNativeLibrary Include="path/to/libfoo.so">
    <Abi>armeabi-v7a</Abi>
  </AndroidNativeLibrary>
</ItemGroup>
```

AndroidPackagingOptionsExclude

A set of file glob compatible items which will allow for items to be excluded from the final package. The default values are as follows

XML

```
<ItemGroup>
  <AndroidPackagingOptionsExclude Include="DebugProbesKt.bin" />
  <AndroidPackagingOptionsExclude
Include="$([MSBuild]::Escape('*.kotlin_*'))" />
</ItemGroup>
```

Items can use file blob characters for wildcards such as `*` and `?`. However these Items MUST be URL encoded or use `$([MSBuild]::Escape(""))`. This is so MSBuild does not try to interpret them as actual file wildcards.

For example

XML

```
<ItemGroup>
  <AndroidPackagingOptionsExclude Include="%2A.foo_%2A" />
  <AndroidPackagingOptionsExclude Include="$([MSBuild]::Escape('*.foo'))" />
</ItemGroup>
```

NOTE: `*`, `?` and `.` will be replaced in the `BuildApk` task with the appropriate file globs.

If the default file glob is too restrictive you can remove it by adding the following to your csproj

XML

```
<ItemGroup>
  <AndroidPackagingOptionsExclude
Remove="$([MSBuild]::Escape('*.kotlin_*'))" />
</ItemGroup>
```

Added in .NET 7.

AndroidPackagingOptionsInclude

A set of file glob compatible items which will allow for items to be included from the final package. The default values are as follows

XML

```
<ItemGroup>
  <AndroidPackagingOptionsInclude
```

```
Include="$([MSBuild]::Escape('*.*.kotlin_builtins'))" />
</ItemGroup>
```

Items can use file blob characters for wildcards such as `*` and `?`. However these Items MUST use URL encoding or `'$([MSBuild]::Escape(''))'`. This is so MSBuild does not try to interpret them as actual file wildcards. For example

XML

```
<ItemGroup>
  <AndroidPackagingOptionsInclude Include="%2A.foo_%2A" />
  <AndroidPackagingOptionsInclude Include="$([MSBuild]::Escape('*.*.foo'))" />
</ItemGroup>
```

NOTE: `*`, `?` and `.` will be replaced in the `BuildApk` task with the appropriate file globs.

Added in .NET 9.

AndroidResource

All files with an *AndroidResource* build action are compiled into Android resources during the build process and made accessible via `$(AndroidResgenFile)`.

XML

```
<ItemGroup>
  <AndroidResource Include="Resources\values\strings.xml" />
</ItemGroup>
```

More advanced users might perhaps wish to have different resources used in different configurations but with the same effective path. This can be achieved by having multiple resource directories and having files with the same relative paths within these different directories, and using MSBuild conditions to conditionally include different files in different configurations. For example:

XML

```
<ItemGroup Condition=" '$(Configuration)' != 'Debug' ">
  <AndroidResource Include="Resources\values\strings.xml" />
</ItemGroup>
<ItemGroup Condition=" '$(Configuration)' == 'Debug' ">
  <AndroidResource Include="Resources-Debug\values\strings.xml"/>
</ItemGroup>
<PropertyGroup>
  <MonoAndroidResourcePrefix>Resources;Resources-
```

```
Debug</MonoAndroidResourcePrefix>
</PropertyGroup>
```

LogicalName – Specifies the resource path explicitly. Allows “aliasing” files so that they will be available as multiple distinct resource names.

XML

```
<ItemGroup Condition="'$(Configuration)'!='Debug'">
  <AndroidResource Include="Resources/values/strings.xml"/>
</ItemGroup>
<ItemGroup Condition="'$(Configuration)'=='Debug'">
  <AndroidResource Include="Resources-Debug/values/strings.xml">
    <LogicalName>values/strings.xml</LogicalName>
  </AndroidResource>
</ItemGroup>
```

Content

The normal `Content` Build action is not supported (as we haven't figured out how to support it without a possibly costly first-run step).

Attempting to use the `@(Content)` Build action will result in a [XA0101](#) warning.

EmbeddedJar

In a .NET for Android binding project, the **EmbeddedJar** build action binds the Java/Kotlin library and embeds the `.jar` file into the library. When a .NET for Android application project consumes the library, it will have access to the Java/Kotlin APIs from C# as well as include the Java/Kotlin code in the final Android application.

You should instead use the [AndroidLibrary](#) build action as an alternative such as:

XML

```
<Project>
  <ItemGroup>
    <AndroidLibrary Include="Library.jar" />
  </ItemGroup>
</Project>
```

EmbeddedNativeLibrary

In a .NET for Android class library or Java binding project, the **EmbeddedNativeLibrary** build action bundles a native library such as `lib/armeabi-v7a/libfoo.so` into the library. When a .NET for Android application consumes the library, the `libfoo.so` file will be included in the final Android application.

You can use the [AndroidNativeLibrary](#) build action as an alternative.

EmbeddedReferenceJar

In a .NET for Android binding project, the **EmbeddedReferenceJar** build action embeds the `.jar` file into the library but does not create a C# binding as [EmbeddedJar](#) does. When a .NET for Android application project consumes the library, it will include the Java/Kotlin code in the final Android application.

You can use the [AndroidLibrary](#) build action as an alternative such as `<AndroidLibrary Include="..." Bind="false" />`:

XML

```
<Project>
  <ItemGroup>
    <!-- A .jar file to bind & embed -->
    <AndroidLibrary Include="Library.jar" />
    <!-- A .jar file to only embed -->
    <AndroidLibrary Include="Dependency.jar" Bind="false" />
  </ItemGroup>
</Project>
```

JavaSourceJar

In a .NET for Android binding project, the **JavaSourceJar** build action is used on `.jar` files that contain *Java source code*, that contain [Javadoc documentation comments](#) [↗](#).

Javadoc will instead be converted into [C# XML Documentation Comments](#) within the generated binding source code.

`$(AndroidJavadocVerbosity)` controls how "verbose" or "complete" the imported Javadoc is.

The following MSBuild metadata is supported:

- `%(CopyrightFile)`: A path to a file that contains copyright information for the Javadoc contents, which will be appended to all imported documentation.

- `%(UrlPrefix)`: A URL prefix to support linking to online documentation within imported documentation.
- `%(UrlStyle)`: The "style" of URLs to generate when linking to online documentation. Only one style is currently supported:
`developer.android.com/reference@2020-Nov`.
- `%(DocRootUrl)`: A URL prefix to use in place of all `{@docroot}` instances in the imported documentation.

LibraryProjectZip

The `LibraryProjectZip` build action binds the Java/Kotlin library and embeds the `.zip` or `.aar` file into the library. When a .NET for Android application project consumes the library, it will have access to the Java/Kotlin APIs from C# as well as include the Java/Kotlin code in the final Android application.

LinkDescription

Files with a `LinkDescription` build action are used to [control linker behavior](#).

ProguardConfiguration

Files with a `ProguardConfiguration` build action contain options which are used to control `proguard` behavior. For more information about this build action, see [ProGuard](#).

These files are ignored unless the `$(EnableProguard)` MSBuild property is `True`.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Overview

Article • 04/18/2024

As part of the .NET for Android build, [Android resources](#) are processed, exposing [Android IDs](#) via a generated `_Microsoft.Android.Resource.Designer.dll` assembly. For example, given the file `Resources\layout\Main.axml` with contents:

XML

```
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android">
  <Button android:id="@+id/myButton" />
  <fragment
    android:id="@+id/log_fragment"
    android:name="commonsamplelibrary.LogFragment"
  />
  <fragment
    android:id="@+id/secondary_log_fragment"
    android:name="CommonSampleLibrary.LogFragment"
  />
</LinearLayout>
```

Then during build-time a `_Microsoft.Android.Resource.Designer.dll` assembly with contents similar to:

C#

```
namespace _Microsoft.Android.Resource.Designer;

partial class Resource {
    partial class Id {
        public static int myButton {get;}
        public static int log_fragment {get;}
        public static int secondary_log_fragment {get;}
    }
    partial class Layout {
        public static int Main {get;}
    }
}
```

Traditionally, interacting with Resources would be done in code, using the constants from the `Resource` type and the `FindViewById<T>()` method:

C#

```
partial class MainActivity : Activity {
```

```
protected override void OnCreate (Bundle savedInstanceState)
{
    base.OnCreate (savedInstanceState);
    SetContentView (Resource.Layout.Main);
    Button button = FindViewById<Button>(Resource.Id.myButton);
    button.Click += delegate {
        button.Text = $"{count++} clicks!";
    };
}
}
```

Starting with Xamarin.Android 8.4, there are two additional ways to interact with Android resources when using C#:

1. [Bindings](#)
2. [Code-Behind](#)

To enable these new features, set the [\\$\(AndroidGenerateLayoutBindings\)](#) MSBuild property to `True` either on the msbuild command line:

.NET CLI

```
dotnet build -p:AndroidGenerateLayoutBindings=true MyProject.csproj
```

or in your .csproj file:

XML

```
<PropertyGroup>
  <AndroidGenerateLayoutBindings>true</AndroidGenerateLayoutBindings>
</PropertyGroup>
```

Bindings

A *binding* is a generated class, one per *Android layout file*, which contains strongly typed properties for all of the *ids* within the layout file. Binding types are generated into the `global::Bindings` namespace, with type names which mirror the filename of the layout file.

Binding types are created for all layout files which contain any Android IDs.

Given the Android Layout file `Resources\layout\Main.axml`:

XML

```

<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:xamarin="http://schemas.xamarin.com/android/xamarin/tools">
    <Button android:id="@+id/myButton" />
    <fragment
        android:id="@+id/fragmentWithExplicitManagedType"
        android:name="commonsamplelibrary.LogFragment"
        xamarin:managedType="CommonSampleLibrary.LogFragment"
    />
    <fragment
        android:id="@+id/fragmentWithInferredType"
        android:name="CommonSampleLibrary.LogFragment"
    />
</LinearLayout>

```

then following type will be generated:

C#

```

// Generated code
namespace Binding {
    sealed class Main : global::Xamarin.Android.Design.LayoutBinding {

        [global::Android.Runtime.PreserveAttribute (Conditional=true)]
        public Main (
            global::Android.App.Activity client,
            global::Xamarin.Android.Design.OnLayoutItemNotFoundHandler
itemNotFoundHandler = null)
            : base (client, itemNotFoundHandler) {}

        [global::Android.Runtime.PreserveAttribute (Conditional=true)]
        public Main (
            global::Android.Views.View client,
            global::Xamarin.Android.Design.OnLayoutItemNotFoundHandler
itemNotFoundHandler = null)
            : base (client, itemNotFoundHandler) {}

        Button __myButton;
        public Button myButton => FindView
(global::Xamarin.Android.Tests.CodeBehindFew.Resource.Id.myButton, ref
__myButton);

        CommonSampleLibrary.LogFragment __fragmentWithExplicitManagedType;
        public CommonSampleLibrary.LogFragment fragmentWithExplicitManagedType
=>
            FindFragment
(global::Xamarin.Android.Tests.CodeBehindFew.Resource.Id.fragmentWithExplici
tManagedType, __fragmentWithExplicitManagedType, ref
__fragmentWithExplicitManagedType);

        global::Android.App.Fragment __fragmentWithInferredType;
        public global::Android.App.Fragment fragmentWithInferredType =>

```

```

        FindFragment
        (global::Xamarin.Android.Tests.CodeBehindFew.Resource.Id.fragmentWithInferredType, __fragmentWithInferredType, ref __fragmentWithInferredType);
    }
}

```

The binding's base type, `Xamarin.Android.Design.LayoutBinding` is **not** part of the .NET for Android class library but rather shipped with .NET for Android in source form and included in the application's build automatically whenever bindings are used.

The generated binding type can be created around `Activity` instances, allowing for strongly-typed access to IDs within the layout file:

```

C#

// User-written code
partial class MainActivity : Activity {

    protected override void OnCreate (Bundle savedInstanceState)
    {
        base.OnCreate (savedInstanceState);

        SetContentView (Resource.Layout.Main);
        var binding      = new Binding.Main (this);
        Button button    = binding.myButton;
        button.Click += delegate {
            button.Text = $"{count++} clicks!";
        };
    }
}

```

Binding types may also be constructed around `View` instances, allowing strongly-typed access to Resource IDs *within* the View or its children:

```

C#

var binding = new Binding.Main (some_view);

```

Missing Resource IDs

Properties on binding types still use `FindViewById<T>()` in their implementation. If `FindViewById<T>()` returns `null`, then the default behavior is for the property to throw an `InvalidOperationException` instead of returning `null`.

This default behavior may be overridden by passing an error handler delegate to the generated binding on its instantiation:

C#

```
// User-written code
partial class MainActivity : Activity {

    Java.Lang.Object? OnLayoutItemNotFound (int resourceId, Type
expectedViewType)
    {
        // Find and return the View or Fragment identified by `resourceId`
        // or `null` if unknown
        return null;
    }

    protected override void OnCreate (Bundle savedInstanceState)
    {
        base.OnCreate (savedInstanceState);

        SetContentView (Resource.Layout.Main);
        var binding      = new Binding.Main (this, OnLayoutItemNotFound);
    }
}
```

The `OnLayoutItemNotFound()` method is invoked when a resource ID for a `View` or a `Fragment` could not be found.

The handler *must* return either `null`, in which case the `InvalidOperationException` will be thrown or, preferably, return the `View` or `Fragment` instance that corresponds to the ID passed to the handler. The returned object **must** be of the correct type matching the type of the corresponding Binding property. The returned value is cast to that type, so if the object isn't correctly typed an exception will be thrown.

Code-Behind

Code-Behind involves build-time generation of a `partial` class which contains strongly typed properties for all of the *ids* within the layout file.

Code-Behind builds atop the Binding mechanism, while requiring that layout files "opt-in" to Code-Behind generation by using the new `xamarin:classes` XML attribute, which is a `;`-separated list of full class names to be generated.

Given the Android Layout file `Resources\layout\Main.xml`:

XML

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:xamarin="http://schemas.xamarin.com/android/xamarin/tools"
```

```

        xamarin:classes="Example.MainActivity">
<Button android:id="@+id/myButton" />
<fragment
    android:id="@+id/fragmentWithExplicitManagedType"
    android:name="commonsamplerlibrary.LogFragment"
    xamarin:managedType="CommonSampleLibrary.LogFragment"
/>
<fragment
    android:id="@+id/fragmentWithInferredType"
    android:name="CommonSampleLibrary.LogFragment"
/>
</LinearLayout>

```

at build time the following type will be produced:

C#

```

// Generated code
namespace Example {
    partial class MainActivity {
        Binding.Main __layout_binding;

        public override void SetContentView (global::Android.Views.View view);
        void SetContentView (global::Android.Views.View view,

global::Xamarin.Android.Design.LayoutBinding.OnLayoutItemNotFoundHandler
onLayoutItemNotFound);

        public override void SetContentView (global::Android.Views.View view,
global::Android.Views.ViewGroup.LayoutParams @params);
        void SetContentView (global::Android.Views.View view,
global::Android.Views.ViewGroup.LayoutParams @params,

global::Xamarin.Android.Design.LayoutBinding.OnLayoutItemNotFoundHandler
onLayoutItemNotFound);

        public override void SetContentView (int layoutResID);
        void SetContentView (int layoutResID,

global::Xamarin.Android.Design.LayoutBinding.OnLayoutItemNotFoundHandler
onLayoutItemNotFound);

        partial void OnSetContentView (global::Android.Views.View view, ref bool
callBaseAfterReturn);
        partial void OnSetContentView (global::Android.Views.View view,
global::Android.Views.ViewGroup.LayoutParams @params, ref bool
callBaseAfterReturn);
        partial void OnSetContentView (int layoutResID, ref bool
callBaseAfterReturn);

        public Button myButton => __layout_binding?.myButton;
        public CommonSampleLibrary.LogFragment fragmentWithExplicitManagedType
=> __layout_binding?.fragmentWithExplicitManagedType;

```



```

        public global::Android.App.Fragment fragmentWithInferredType =>
        __layout_binding?.fragmentWithInferredType;
    }
}

```

This allows for more "intuitive" use of Resource IDs within the layout:

C#

```

// User-written code
partial class MainActivity : Activity {
    protected override void OnCreate (Bundle savedInstanceState)
    {
        base.OnCreate (savedInstanceState);

        SetContentView (Resource.Layout.Main);

        myButton.Click += delegate {
            button.Text = $"{count++} clicks!";
        };
    }
}

```

The `OnLayoutItemNotFound` error handler can be passed as the last parameter of whatever overload of `SetContentView` the activity is using:

C#

```

// User-written code
partial class MainActivity : Activity {
    protected override void OnCreate (Bundle savedInstanceState)
    {
        base.OnCreate (savedInstanceState);

        SetContentView (Resource.Layout.Main, OnLayoutItemNotFound);
    }

    Java.Lang.Object? OnLayoutItemNotFound (int resourceId, Type
expectedViewType)
    {
        // Find and return the View or Fragment identified by `resourceId`
        // or `null` if unknown
        return null;
    }
}

```

As Code-Behind relies on partial classes, *all* declarations of a partial class *must* use `partial class` in their declaration, otherwise a [CS0260](#) C# compiler error will be generated at build time.

Customization

The generated Code Behind type *always* overrides `Activity.SetContentView()`, and by default it *always* calls `base.SetContentView()`, forwarding the parameters. If this is not desired, then one of the `OnSetContentView()` *partial methods* should be overridden, setting `callBaseAfterReturn` to `false`:

C#

```
// Generated code
namespace Example
{
    partial class MainActivity {
        partial void OnSetContentView (global::Android.Views.View view, ref bool
callBaseAfterReturn);
        partial void OnSetContentView (global::Android.Views.View view,
global::Android.Views.ViewGroup.LayoutParams @params, ref bool
callBaseAfterReturn);
        partial void OnSetContentView (int layoutResID, ref bool
callBaseAfterReturn);
    }
}
```

Example Generated Code

C#

```
// Generated code
namespace Example
{
    partial class MainActivity {

        Binding.Main? __layout_binding;

        public override void SetContentView (global::Android.Views.View view)
        {
            __layout_binding = new global::Binding.Main (view);
            bool callBase = true;
            OnSetContentView (view, ref callBase);
            if (callBase) {
                base.SetContentView (view);
            }
        }

        void SetContentView (global::Android.Views.View view,
global::Xamarin.Android.Design.LayoutBinding.OnLayoutItemNotFoundHandler
onLayoutItemNotFound)
        {
            __layout_binding = new global::Binding.Main (view,
```

```

onLayoutItemNotFound);
    bool callBase = true;
    OnSetContentView (view, ref callBase);
    if (callBase) {
        base.SetContentView (view);
    }
}

    public override void SetContentView (global::Android.Views.View view,
global::Android.Views.ViewGroup.LayoutParams @params)
    {
        __layout_binding = new global::Binding.Main (view);
        bool callBase = true;
        OnSetContentView (view, @params, ref callBase);
        if (callBase) {
            base.SetContentView (view, @params);
        }
    }

    void SetContentView (global::Android.Views.View view,
global::Android.Views.ViewGroup.LayoutParams @params,
global::Xamarin.Android.Design.LayoutBinding.OnLayoutItemNotFoundHandler
onLayoutItemNotFound)
    {
        __layout_binding = new global::Binding.Main (view,
onLayoutItemNotFound);
        bool callBase = true;
        OnSetContentView (view, @params, ref callBase);
        if (callBase) {
            base.SetContentView (view, @params);
        }
    }

    public override void SetContentView (int layoutResID)
    {
        __layout_binding = new global::Binding.Main (this);
        bool callBase = true;
        OnSetContentView (layoutResID, ref callBase);
        if (callBase) {
            base.SetContentView (layoutResID);
        }
    }

    void SetContentView (int layoutResID,
global::Xamarin.Android.Design.LayoutBinding.OnLayoutItemNotFoundHandler
onLayoutItemNotFound)
    {
        __layout_binding = new global::Binding.Main (this,
onLayoutItemNotFound);
        bool callBase = true;
        OnSetContentView (layoutResID, ref callBase);
        if (callBase) {
            base.SetContentView (layoutResID);
        }
    }
}

```

```

    partial void OnSetContentView (global::Android.Views.View view, ref bool
callBaseAfterReturn);
    partial void OnSetContentView (global::Android.Views.View view,
global::Android.Views.ViewGroup.LayoutParams @params, ref bool
callBaseAfterReturn);
    partial void OnSetContentView (int layoutResID, ref bool
callBaseAfterReturn);

    public Button myButton
=> __layout_binding?.myButton;
    public CommonSampleLibrary.LogFragment fragmentWithExplicitManagedType
=> __layout_binding?.fragmentWithExplicitManagedType;
    public global::Android.App.Fragment fragmentWithInferredType
=> __layout_binding?.fragmentWithInferredType;
}
}

```

Layout XML Attributes

Many new Layout XML attributes control Binding and Code-Behind behavior, which are within the `xamarin` XML namespace

(`xmlns:xamarin="http://schemas.xamarin.com/android/xamarin/tools"`). These include:

- `xamarin:classes`
- `xamarin:managedType`

`xamarin:classes`

The `xamarin:classes` XML attribute is used as part of Code-Behind to specify which types should be generated.

The `xamarin:classes` XML attribute contains a `;`-separated list of *full class names* that should be generated.

`xamarin:managedType`

The `xamarin:managedType` layout attribute is used to explicitly specify the managed type to expose the bound ID as. If not specified, the type will be inferred from the declaring context, e.g. `<Button/>` will result in an `Android.Widget.Button`, and `<fragment/>` will result in an `Android.App.Fragment`.

The `xamarin:managedType` attribute allows for more explicit type declarations.

Managed type mapping

It is quite common to use widget names based on the Java package they come from and, equally as often, the managed .NET name of such type will have a different (.NET style) name in the managed land. The code generator can perform a number of very simple adjustments to try to match the code, such as:

- Capitalize all the components of the type namespace and name. For instance `java.package.myButton` would become `Java.Package.MyButton`
- Capitalize two-letter components of the type namespace. For instance `android.os.SomeType` would become `Android.OS.SomeType`
- Look up a number of hard-coded namespaces which have known mappings. Currently the list includes the following mappings:
 - `android.view` -> `Android.Views`
 - `com.actionbarsherlock` -> `ABSherlock`
 - `com.actionbarsherlock.widget` -> `ABSherlock.Widget`
 - `com.actionbarsherlock.view` -> `ABSherlock.View`
 - `com.actionbarsherlock.app` -> `ABSherlock.App`
- Look up a number of hard-coded types in internal tables. Currently the list includes the following types:
 - `WebView` -> `Android.Webkit.WebView`
- Strip number of hard-coded namespace *prefixes*. Currently the list includes the following prefixes:
 - `com.google.`

If, however, the above attempts fail, you will need to modify the layout which uses a widget with such an unmapped type to add both the `xamarin` XML namespace declaration to the root element of the layout and the `xamarin:managedType` to the element requiring the mapping. For instance:

XML

```
<fragment
  android:id="@+id/log_fragment"
  android:name="commonsamplelibrary.LogFragment"
  xamarin:managedType="CommonSampleLibrary.LogFragment"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
/>
```

Will use the `CommonSampleLibrary.LogFragment` type for the native type `commonsamplelibrary.LogFragment`.

You can avoid adding the XML namespace declaration and the `xamarin:managedType` attribute by simply naming the type using its managed name, for instance the above fragment could be redeclared as follows:

XML

```
<fragment
  android:name="CommonSampleLibrary.LogFragment"
  android:id="@+id/secondary_log_fragment"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
/>
```

Fragments: a special case

The Android ecosystem currently supports two distinct implementations of the `Fragment` widget:

- [Android.App.Fragment](#) The "classic" Fragment shipped with the base Android system
- `AndroidX.Fragment.App.Fragment`, in the [Xamarin.AndroidX.Fragment](#) [NuGet](#) package.

These classes are **not** compatible with each other and so special care must be taken when generating binding code for `<fragment>` elements in the layout files. .NET for Android must choose one `Fragment` implementation as the default one to be used if the `<fragment>` element does not have any specific type (managed or otherwise) specified. Binding code generator uses the `$(AndroidFragmentType)` MSBuild property for that purpose. The property can be overridden by the user to specify a type different than the default one. The property is set to `Android.App.Fragment` by default, and is overridden by the AndroidX NuGet packages.

If the generated code does not build, the layout file must be amended by specifying the managed type of the fragment in question.

Code-behind layout selection and processing

Selection

By default code-behind generation is disabled. To enable processing for all layouts in any of the `Resource\layout*` directories that contain at least a single element with the `/**/@android:id` attribute, set the `$(AndroidGenerateLayoutBindings)` MSBuild property to `True` either on the msbuild command line:

.NET CLI

```
dotnet build -p:AndroidGenerateLayoutBindings=true MyProject.csproj
```

or in your .csproj file:

XML

```
<PropertyGroup>
  <AndroidGenerateLayoutBindings>true</AndroidGenerateLayoutBindings>
</PropertyGroup>
```

Alternatively, you can leave code-behind disabled globally and enable it only for specific files. To enable Code-Behind for a particular `.axml` file, change the file to have a **Build action** of `@(AndroidBoundLayout)` by editing your `.csproj` file and replacing `AndroidResource` with `AndroidBoundLayout`:

XML

```
<!-- This -->
<AndroidResource Include="Resources\layout\Main.xml" />
<!-- should become this -->
<AndroidBoundLayout Include="Resources\layout\Main.xml" />
```

Processing

Layouts are grouped by name, with like-named templates from *different* `Resource\layout*` directories comprising a single group. Such groups are processed as if they were a single layout. It is possible that in such case there will be a type clash between two widgets found in different layouts belonging to the same group. In such case the generated property will not be able to have the exact widget type, but rather a "decayed" one. Decaying follows the algorithm below:

1. If all of the conflicting widgets are `View` derivatives, the property type will be `Android.Views.View`

2. If all of the conflicting types are `Fragment` derivatives, the property type will be `Android.App.Fragment`
3. If the conflicting widgets contain both a `View` and a `Fragment`, the property type will be `global::System.Object`

Generated code

If you are interested in how the generated code looks for your layouts, please take a look in the `obj\$(Configuration)\generated` folder in your solution directory.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Java and managed code interoperability

Article • 05/07/2024

App developers expect to be able to call native Android APIs and receive calls, or react to events, from the Android APIs using code written in one of the .NET managed languages. .NET for Android employs a number of approaches, at build and at runtime, to bridge the Java VM (ART in Android OS) and the Managed VM (MonoVM).

The Java VM and the Managed VM co-exist in the same process or app as separate entities. Despite sharing the same process resources, there's no direct way to call Java/Kotlin APIs from .NET, and there's no direct way for Java/Kotlin code to invoke managed code APIs. To enable this communication, .NET for Android uses the [Java Native Interface \(JNI\)](#). This is an approach that enables native code (.NET managed code being native in this context) to register implementations of Java methods, which are written outside the Java VM and in languages other than Java or Kotlin. These methods need to be declared in Java code, for example:

Java

```
class MainActivity extends androidx.appcompat.app.AppCompatActivity
{
    public void onCreate (android.os.Bundle p0)
    {
        n_onCreate (p0);
    }

    private native void n_onCreate (android.os.Bundle p0);
}
```

Each native method is declared using the `native` keyword, and whenever it is invoked from Java code, the Java VM will use the JNI to invoke the target method.

Native methods can be registered dynamically or statically, by providing a native shared library which exports a symbol with an appropriate name that points to the native function implementing the Java method.

Java callable wrappers

.NET for Android wraps Android APIs by generating C# code that mirrors the Java/Kotlin APIs. Each generated class that corresponds to a Java/Kotlin type is derived from the `Java.Lang.Object` class (implemented in the `Mono.Android` assembly), which marks it as a Java interoperable type. This means that it can implement or override virtual Java

methods. To make registration and invocation of these methods possible, it's necessary to generate a Java class that mirrors the managed class and that provides an entry point to the Java to managed transition. Java classes are generated during app build, as well as during the .NET for Android build, and are known as *Java Callable Wrappers* (JCW). The following example shows a managed class that overrides two Java virtual methods:

C#

```
public class MainActivity : AppCompatActivity
{
    public override Android.Views.View? OnCreateView(Android.Views.View?
parent, string name, Android.Content.Context context,
Android.Util.IAttributeSet attrs)
    {
        return base.OnCreateView(parent, name, context, attrs);
    }

    protected override void OnCreate(Bundle savedInstanceState)
    {
        base.OnCreate(savedInstanceState);
        DoSomething(savedInstanceState);
    }

    void DoSomething(Bundle bundle)
    {
        // do something with the bundle
    }
}
```

In this example, the managed class overrides the `OnCreateView` and `OnCreate` Java virtual methods that are found in the `AppCompatActivity` type. The `DoSomething` method doesn't correspond to any method found in the base Java type, and therefore it won't be included in the JCW.

The following example shows the JCW generated for the above class (with some generated methods omitted for clarity):

Java

```
public class MainActivity extends androidx.appcompat.app.AppCompatActivity
{
    public android.view.View onCreateView (android.view.View p0,
java.lang.String p1, android.content.Context p2, android.util.AttributeSet
p3)
    {
        return n_onCreateView (p0, p1, p2, p3);
    }
    private native android.view.View n_onCreateView (android.view.View p0,
java.lang.String p1, android.content.Context p2, android.util.AttributeSet
```

```

p3);

    public void onCreate (android.os.Bundle p0)
    {
        n_onCreate (p0);
    }
    private native void n_onCreate (android.os.Bundle p0);
}

```

Registration

Both method registration approaches rely on generation of JCWs, with dynamic registration requiring more code to be generated so that the registration can be performed at runtime.

JCWs are generated only for types that derive from the `Java.Lang.Object` type. Finding such types is the task of [Java.Interop](#), which reads all the assemblies referenced by the app and its libraries. The returned list of assemblies is then used by a variety of tasks, with JCW being one of them.

After all types are found, each method in each type is analyzed to look for those that override a virtual Java method and therefore need to be included in the wrapper class code. The generator also optionally checks whether the given method can be registered statically.

The generator looks for methods decorated with the `Register` attribute, which most frequently are created by invoking its constructor with the Java method name, the JNI method signature, and the connector method name. The connector is a static method that creates a delegate that subsequently allows invocation of the native callback method:

```

C#

public class MainActivity : AppCompatActivity
{
    // Connector backing field
    static Delegate? cb_onCreate_Landroid_os_Bundle_;

    // Connector method
    static Delegate GetOnCreate_Landroid_os_Bundle_Handler()
    {
        if (cb_onCreate_Landroid_os_Bundle_ == null)
            cb_onCreate_Landroid_os_Bundle_ =
                JNINativeWrapper.CreateDelegate((_JniMarshal_PPL_V)n_OnCreate_Landroid_os_Bu
                ndle_);
        return cb_onCreate_Landroid_os_Bundle_;
    }
}

```

```

// Native callback
static void n_OnCreate_Landroid_os_Bundle_(IntPtr jnienv, IntPtr
native__this, IntPtr native_savedInstanceState)
{
    var __this =
global::Java.Lang.Object.GetObject<Android.App.Activity>(jnienv,
native__this, JniHandleOwnership.DoNotTransfer!);
    var savedInstanceState =
global::Java.Lang.Object.GetObject<Android.OS.Bundle>
(native_savedInstanceState, JniHandleOwnership.DoNotTransfer);
    __this.OnCreate(savedInstanceState);
}

// Target method
[Register("onCreate", "(Landroid/os/Bundle;)V",
"GetOnCreate_Landroid_os_Bundle_Handler")]
protected virtual unsafe void OnCreate(Android.OS.Bundle?
savedInstanceState)
{
    const string __id = "onCreate.(Landroid/os/Bundle;)V";
    try
    {
        JniArgumentValue* __args = stackalloc JniArgumentValue[1];
        __args[0] = new JniArgumentValue((savedInstanceState == null) ?
IntPtr.Zero : ((global::Java.Lang.Object)savedInstanceState).Handle);
        _members.InstanceMethods.InvokeVirtualVoidMethod(__id, this,
__args);
    }
    finally
    {
        global::System.GC.KeepAlive(savedInstanceState);
    }
}
}

```

The above code is generated in the `Android.App.Activity` class when .NET for Android is built. What happens with this code depends on the registration approach.

Dynamic registration

This registration approach is used by .NET for Android when an app is built in the Debug configuration, or when [marshal methods](#) is turned off.

With dynamic registration, the following Java code is generated for the C# example shown in [Java callable wrappers](#) (with some generated methods omitted for clarity):

Java

```

public class MainActivity extends androidx.appcompat.app.AppCompatActivity
{
    public static final String __md_methods;
    static
    {
        __md_methods =
            "n_onCreateView:
(Landroid/view/View;Ljava/lang/String;Landroid/content/Context;Landroid/util/
/AttributeSet;)Landroid/view/View;:GetOnCreateView_Landroid_view_View_Ljava_
lang_String_Landroid_content_Context_Landroid_util_AttributeSet_Handler\n" +
            "n_onCreate:
(Landroid/os/Bundle;)V:GetOnCreate_Landroid_os_Bundle_Handler\n" +
            "";
        mono.android.Runtime.register ("HelloAndroid.MainActivity,
HelloAndroid", MainActivity.class, __md_methods);
    }

    public android.view.View onCreateView (android.view.View p0,
java.lang.String p1, android.content.Context p2, android.util.AttributeSet
p3)
    {
        return n_onCreateView (p0, p1, p2, p3);
    }

    private native android.view.View n_onCreateView (android.view.View p0,
java.lang.String p1, android.content.Context p2, android.util.AttributeSet
p3);

    public void onCreate (android.os.Bundle p0)
    {
        n_onCreate (p0);
    }

    private native void n_onCreate (android.os.Bundle p0);
}

```

The code that takes part in registration is the class's static constructor. For each method registered for the type (that is, implemented or overridden in managed code), the JCW generator outputs a single string that contains information about the type and method to register. Each registration string is terminated with the newline character and the entire sequence ends with an empty string. All the registration strings are concatenated and placed in the `__md_methods` static variable. The `mono.android.Runtime.register` method is then invoked to register all the methods.

Dynamic registration call sequence

All the native methods declared in the generated Java type are registered when the type is constructed or accessed for the first time. This is when the Java VM invokes the type's

static constructor, initiating a sequence of calls that ends with all of the type's methods being registered with the JNI:

1. `mono.android.Runtime.register` is a native method, declared in the `Runtime` class of .NET for Android's Java runtime code, and implemented in the native .NET for Android runtime. The purpose of this method is to prepare a call into .NET for Android's managed runtime code.
2. `Android.Runtime.JNIEnv::RegisterJniNatives` is passed the name of the managed type for which to register Java methods, and uses .NET reflection to load that type followed by a call to cache the type. It ends with a call to the `Android.Runtime.AndroidTypeManager::RegisterNativeMembers` method.
3. `Android.Runtime.AndroidTypeManager::RegisterNativeMembers` calls the `Java.Interop.JniEnvironment.Types::RegisterNatives` method which first generates a delegate to the native callback method, using `System.Reflection.Emit`, and invokes Java JNI's `RegisterNatives` function to register the native methods for a managed type.

⚠ Note

The `System.Reflection.Emit` call is a costly operation, and is repeated for each registered method.

For more information about Java type registration, see [Java Type Registration](#).

Marshal methods

The marshal methods registration approach takes advantage of the JNI's ability to look up implementations of `native` Java methods in native libraries. Such symbols must have names that follow a set of rules, so that the JNI is able to locate them.

The goal of this approach is to bypass the dynamic registration approach, replacing it with native code generated and compiled during app build. This reduces app startup time. To achieve this goal, this approach uses classes that [generate native code](#) and [modify assemblies](#) which contain the registered methods.

The marshal methods classifier recognizes the standard method registration pattern, which consists of:

- The connector method, which is `GetOnCreate_Landroid_os_Bundle_Handler` in the example in [Registration](#).

- The delegate backing field, which is `cb_onCreate_Landroid_os_Bundle_` in the example in [Registration](#).
- The native callback method, which is `n_OnCreate_Landroid_os_Bundle_` in the example in [Registration](#).
- The virtual target method that dispatches the call to the actual object, which is `OnCreate` in the example in [Registration](#).

Whenever the classifier runs, it's passed the methods declaring type and the `Register` attribute instance which it uses to check whether the method being registered conforms to the registration pattern. The connector, native callback methods, and backing field must be private and static for the registered method to be considered as a candidate for static registration.

ⓘ Note

Registered methods that don't follow the standard pattern will be registered dynamically.

With marshal methods, the following Java code is generated for the C# example shown in [Java callable wrappers](#) (with some generated methods omitted for clarity):

Java

```
public class MainActivity extends androidx.appcompat.app.AppCompatActivity
{
    public android.view.View onCreateView (android.view.View p0,
    java.lang.String p1, android.content.Context p2, android.util.AttributeSet
    p3)
    {
        return n_onCreateView (p0, p1, p2, p3);
    }

    private native android.view.View n_onCreateView (android.view.View p0,
    java.lang.String p1, android.content.Context p2, android.util.AttributeSet
    p3);

    public void onCreate (android.os.Bundle p0)
    {
        n_onCreate (p0);
    }

    private native void n_onCreate (android.os.Bundle p0);
}
```

Compared to the code generated for dynamic registration, there is no static constructor. However, the rest of the code is identical.

JNI requirements

The JNI specifies a [series of rules](#) that govern how the native symbol name is constructed. A native method name is concatenated from the following components:

- Each symbol starts with the `Java_` prefix.
- A mangled *fully qualified class name*.
- A `_` character serves as a separator.
- A mangled *method name*.
- Optionally, a double underscore `__` and the mangled method argument signature.

Mangling is a way of encoding certain characters that aren't directly representable in the source code and in the native symbol name. The JNI specification allows for direct use of ASCII letters (capital and lowercase) and digits, while all the other characters are either represented by placeholders or encoded as 16-bit hexadecimal Unicode character codes:

[Expand table](#)

Escape sequence	Denotes
<code>_0XXXX</code>	a Unicode character XXXX, all lower case.
<code>_1</code>	The <code>_</code> character.
<code>_2</code>	The <code>;</code> character in signatures.
<code>_3</code>	The <code>[</code> character in signatures.
<code>-</code>	The <code>.</code> or <code>/</code> characters.

Generation of JNI symbol names is performed by one of the .NET for Android build tasks while generating the native function source code.

JNI supports a short and long form of the native symbol name. The short form is looked up first by the Java VM, followed by the long form. The long forms needs to be used only for overloaded methods.

LLVM intermediate representation code generation

One of the .NET for Android build tasks uses the LLVM intermediate representation (IR) generator infrastructure to output data and executable code for all the marshal methods wrappers. Consider the following C++ code, which serves as a guide to understanding how the marshal method runtime invocation works:

C++

```
using get_function_pointer_fn = void (*)(uint32_t mono_image_index, uint32_t
class_index, uint32_t method_token, void*& target_ptr);

static get_function_pointer_fn get_function_pointer;

void xamarin_app_init (get_function_pointer_fn fn) noexcept
{
    get_function_pointer = fn;
}

using android_app_activity_on_create_bundle_fn = void (*) (JNIEnv *env,
jclass klass, jobject savedInstanceState);
static android_app_activity_on_create_bundle_fn
android_app_activity_on_create_bundle = nullptr;

extern "C" JNIEXPORT void
JNICALL Java_helloandroid_MainActivity_n_1onCreate__Landroid_os_Bundle_2
(JNIEnv *env, jclass klass, jobject savedInstanceState) noexcept
{
    if (android_app_activity_on_create_bundle == nullptr) {
        get_function_pointer (
            16, // mono image index
            0, // class index
            0x0600055B, // method token
            reinterpret_cast<void*>(android_app_activity_on_create_bundle) //
target pointer
        );
    }

    android_app_activity_on_create_bundle (env, klass, savedInstanceState);
}
```

The `xamarin_app_init` function is output only once and is called by the .NET for Android runtime twice during app startup. The first time it's called is to pass `get_function_pointer_fn`, and the second time it's called is just before handing control over to the Mono VM, to pass a pointer to `get_function_pointer_fn`.

The `Java_helloandroid_MainActivity_n_1onCreate__Landroid_os_Bundle_2` function is a template that's repeated for each Java native function, with each function having its own set of arguments and its own callback backing field (`android_app_activity_on_create_bundle` here).

The `get_function_pointer` function takes indexes into several tables as parameters. One is for `MonoImage*` pointers and the other for `MonoClass*` pointers - both of which are generated at app build time and allow for very fast lookup at runtime. Target methods

are retrieved by their token value, within the specified `MonoImage*` (essentially a pointer to managed assembly image in memory) and class.

The method identified using this approach must be decorated in managed code with the `UnmanagedCallersOnly` attribute so that it can be invoked directly, as if it was a native method itself, with minimal managed marshalling overhead.

Assembly rewriting

Managed assemblies, including `Mono.Android.dll`, that contain Java types need to be usable with dynamic registration and with marshal methods. However, both of these approaches have different requirements. It can't be assumed that every assembly will have marshal methods compliant code. Therefore, an approach is required that ensures that code meets this requirement. This is achieved by reading assemblies and modifying them by altering the definition of the native callbacks and removing the code that's no longer used by marshal methods. This task is performed by one of the .NET for Android build tasks that's invoked during app build after all the assemblies are linked but before type maps are generated.

The exact modifications that are applied are:

- Removal of the *connector backing field*.
- Removal of the *connector method*.
- Generation of a *native callback wrapper* method, which catches and propagates unhandled exceptions thrown by the native callback or the target method. This method is decorated with the `UnmanagedCallersOnly` attribute and called directly from the native code.
- Optionally, generate code in the *native callback wrapper* to handle [non-blittable types](#).

After modifications, the assembly contains the equivalent of the following C# code for each marshal method:

C#

```
public class MainActivity : AppCompatActivity
{
    // Native callback
    static void n_OnCreate_Landroid_os_Bundle_(IntPtr jnienv, IntPtr
native__this, IntPtr native_savedInstanceState)
    {
        var __this =
global::Java.Lang.Object.GetObject<Android.App.Activity>(jnienv,
native__this, JniHandleOwnership.DoNotTransfer!);
        var savedInstanceState =
```

```

global::Java.Lang.Object.GetObject<Android.OS.Bundle>
(native_savedInstanceState, JNIEnvOwnership.DoNotTransfer);
    __this.OnCreate(savedInstanceState);
}

// Native callback exception wrapper
[UnmanagedCallersOnly]
static void n_OnCreate_Landroid_os_Bundle__mm_wrapper(IntPtr jnienv,
IntPtr native__this, IntPtr native_savedInstanceState)
{
    try
    {
        n_OnCreate_Landroid_os_Bundle_(jnienv, native__this,
native_savedInstanceState)
    }
    catch (Exception ex)
    {
        Android.Runtime.AndroidEnvironmentInternal.UnhandledException(ex);
    }
}

// Target method
[Register("onCreate", "(Landroid/os/Bundle;)V",
"GetOnCreate_Landroid_os_Bundle_Handler")]
protected virtual unsafe void OnCreate(Android.OS.Bundle?
savedInstanceState)
{
    const string __id = "onCreate.(Landroid/os/Bundle;)V";
    try
    {
        JNIEnvArgumentValue* __args = stackalloc JNIEnvArgumentValue[1];
        __args[0] = new JNIEnvArgumentValue((savedInstanceState == null) ?
IntPtr.Zero : ((global::Java.Lang.Object)savedInstanceState).Handle);
        __members.InstanceMethods.InvokeVirtualVoidMethod(__id, this,
__args);
    }
    finally
    {
        global::System.GC.KeepAlive(savedInstanceState);
    }
}
}

```

Wrappers for methods with non-blittable types

The `UnmanagedCallersOnly` attribute requires that all the argument types, and the method return type, are `blittable`. Among these types is the `bool` type that's commonly used by managed classes implementing Java methods. This is currently the only non-blittable type encountered in bindings, and therefore is the only one supported by the assembly rewriter.

Whenever a method with a non-blittable type is encountered, a wrapper is generated for it so that it can be decorated with the [UnmanagedCallersOnly](#) attribute. This is less error prone than modifying the native callback method's IL stream to implement the necessary conversion. An example of such a method is

`Android.Views.View.IOnTouchListener::OnTouch:`

C#

```
static bool n_OnTouch_Landroid_view_View_Landroid_view_MotionEvent(IntPtr
jnienv, IntPtr native__this, IntPtr native_v, IntPtr native_e)
{
    var __this =
global::Java.Lang.Object.GetObject<Android.Views.View.IOnTouchListener>
(jnienv, native__this, JniHandleOwnership.DoNotTransfer)!;
    var v = global::Java.Lang.Object.GetObject<Android.Views.View>(native_v,
JniHandleOwnership.DoNotTransfer);
    var e = global::Java.Lang.Object.GetObject<Android.Views.MotionEvent>
(native_e, JniHandleOwnership.DoNotTransfer);
    bool __ret = __this.OnTouch(v, e);
    return __ret;
}
```

This method returns a `bool` value, and so needs a wrapper to cast the return value correctly. Each wrapper method retains the native callback method name, but appends the `_mm_wrapper` suffix to it:

C#

```
[UnmanagedCallersOnly]
static byte
n_OnTouch_Landroid_view_View_Landroid_view_MotionEvent__mm_wrapper(IntPtr
jnienv, IntPtr native__this, IntPtr native_v, IntPtr native_e)
{
    try
    {
        return
n_OnTouch_Landroid_view_View_Landroid_view_MotionEvent_(jnienv,
native__this, native_v, native_e) ? 1 : 0;
    }
    catch (Exception ex)
    {
        Android.Runtime.AndroidEnvironmentInternal.UnhandledException(ex);
        return default;
    }
}
```

The wrapper's return statement uses the ternary operator to cast the boolean value to 1 or 0 because the value of `bool` across the managed runtime can take a range of values -

0 for false, -1 or 1 for true, and != 0 for true.

Because the `bool` type in C# can be 1, 2 or 4 bytes long, it's cast to a type of a known and static size. The `byte` managed type is used as it corresponds to the Java/JNI `jboolean` type, defined as an unsigned 8-bit type.

Whenever an argument value needs to be converted between `byte` and `bool`, code is generated that's equivalent to the `argument != 0` comparison. For example, for the `Android.Views.View.IOnFocusChangeListener::OnFocusChange` method:

C#

```
[UnmanagedCallersOnly]
static void n_OnFocusChange_Landroid_view_View_Z(IntPtr jnienv, IntPtr
native__this, IntPtr native_v, byte hasFocus)
{
    n_OnFocusChange_Landroid_view_View_Z(jnienv, native__this, native_v,
hasFocus != 0);
}

static void n_OnFocusChange_Landroid_view_View_Z(IntPtr jnienv, IntPtr
native__this, IntPtr native_v, bool hasFocus)
{
    var __this =
global::Java.Lang.Object.GetObject<Android.Views.View.IOnFocusChangeListener
>(jnienv, native__this, JniHandleOwnership.DoNotTransfer)!;
    var v = global::Java.Lang.Object.GetObject<Android.Views.View>(native_v,
JniHandleOwnership.DoNotTransfer);
    __this.OnFocusChange(v, hasFocus);
}
```

UnmanagedCallersOnly attribute

Each marshal methods native callback method is decorated with the `UnmanagedCallersOnly` attribute, to be able to invoke the callback directly from native code with minimal overhead.

Marshal methods registration call sequence

What's common for dynamic and marshal methods registration is the resolution of the native function target performed by the Java VM runtime. In both cases, the method declared in a Java class as `native` is looked up by the Java VM when first JIT-ing the code. The difference is in the way this lookup is performed.

Dynamic registration uses the [RegisterNatives](#) JNI function at runtime, which stores a pointer to the registered method inside the structure that describes a Java class in the Java VM.

Marshal methods, however, don't register anything with the JNI. Instead, they rely on the symbol lookup approach of the Java VM. Whenever a call to a `native` Java method is JIT'd and isn't registered previously using the `RegisterNatives` JNI function, the Java VM will proceed to look for symbols in the process runtime image and, having found a matching symbol, use a pointer to it as the target of the `native` Java method call.

.NET for Android errors and warnings reference

Article • 04/17/2024

ADBxxxx: ADB tooling

- [ADB0000](#): Generic `adb` error/warning.
- [ADB0010](#): Generic `adb` APK installation error/warning.
- [ADB0020](#): The package does not support the CPU architecture of this device.
- [ADB0030](#): APK installation failed due to a conflict with the existing package.
- [ADB0040](#): The device does not support the minimum SDK level specified in the manifest.
- [ADB0050](#): Package {packageName} already exists on device.
- [ADB0060](#): There is not enough storage space on the device to store package: {packageName}. Free up some space and try again.

ANDXXxxxx: Generic Android tooling

- [ANDAS0000](#): Generic `apksigner` error/warning.
- [ANDJS0000](#): Generic `jarsigner` error/warning.
- [ANDKT0000](#): Generic `keytool` error/warning.
- [ANDZA0000](#): Generic `zipalign` error/warning.

APTxxxx: AAPT tooling

- [APT0000](#): Generic `aapt` error/warning.
- [APT0001](#): Unknown option `{option}`. Please check ``$(AndroidAapt2CompileExtraArgs)`` and ``$(AndroidAapt2LinkExtraArgs)`` to see if they include any ``aapt`` command line arguments that are no longer valid for ``aapt2`` and ensure that all other arguments are valid for `aapt2`.
- [APT0002](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0003](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0004](#): Invalid file name: It must start with either A-Z or a-z or an underscore.
- [APT2264](#): The system cannot find the file specified. (2).
- [APT2265](#): The system cannot find the file specified. (2).

JAVAXxxx: Java Tool

- [JAVA0000](#): Generic Java tool error

JAVACxxxx: Java compiler

- [JAVAC0000](#): Generic Java compiler error

XA0xxx: Environment issue or missing tooling

- [XA0000](#): Could not determine `$(AndroidApiLevel)` or `$(TargetFrameworkVersion)`.
- [XA0001](#): Invalid or unsupported `$(TargetFrameworkVersion)` value.
- [XA0002](#): Could not find mono.android.jar
- [XA0003](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. It must be an integer value.
- [XA0004](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. The value must be in the range of 0 to `{maxVersionCode}`.
- [XA0030](#): Building with JDK version `{versionNumber}` is not supported.
- [XA0031](#): Java SDK `{requiredJavaForFrameworkVersion}` or above is required when using `$(TargetFrameworkVersion)` `{targetFrameworkVersion}`.
- [XA0032](#): Java SDK `{requiredJavaForBuildTools}` or above is required when using Android SDK Build-Tools `{buildToolsVersion}`.
- [XA0033](#): Failed to get the Java SDK version because the returned value does not appear to contain a valid version number.
- [XA0034](#): Failed to get the Java SDK version.
- [XA0035](#): Failed to determine the Android ABI for the project.
- [XA0036](#): `$(AndroidSupportedAbis)` is not supported in .NET 6 and higher.
- [XA0100](#): `EmbeddedNativeLibrary` is invalid in Android Application projects. Please use `AndroidNativeLibrary` instead.
- [XA0101](#): warning XA0101: `@(Content)` build action is not supported.
- [XA0102](#): Generic `lint` Warning.
- [XA0103](#): Generic `lint` Error.
- [XA0104](#): Invalid value for ``$(AndroidSequencePointsMode)``
- [XA0105](#): The `$(TargetFrameworkVersion)` for a library is greater than the `$(TargetFrameworkVersion)` for the application project.
- [XA0107](#): `{Assmebly}` is a Reference Assembly.
- [XA0108](#): Could not get version from `lint`.
- [XA0109](#): Unsupported or invalid `$(TargetFrameworkVersion)` value of 'v4.5'.
- [XA0111](#): Could not get the `aapt2` version. Please check it is installed correctly.

- [XA0112](#): `aapt2` is not installed. Disabling `aapt2` support. Please check it is installed correctly.
- [XA0113](#): Google Play requires that new applications and updates must use a TargetFrameworkVersion of v11.0 (API level 30) or above.
- [XA0115](#): Invalid value 'armeabi' in \$(AndroidSupportedAbis). This ABI is no longer supported. Please update your project properties to remove the old value. If the properties page does not show an 'armeabi' checkbox, un-check and re-check one of the other ABIs and save the changes.
- [XA0116](#): Unable to find `EmbeddedResource` named `{ResourceName}`.
- [XA0117](#): The TargetFrameworkVersion {TargetFrameworkVersion} is deprecated. Please update it to be v4.4 or higher.
- [XA0118](#): Could not parse '{TargetMoniker}'
- [XA0119](#): A non-ideal configuration was found in the project.
- [XA0121](#): Assembly '{assembly}' is using '[assembly: Java.Interop.JavaLibraryReferenceAttribute]', which is no longer supported. Use a newer version of this NuGet package or notify the library author.
- [XA0122](#): Assembly '{assembly}' is using a deprecated attribute '[assembly: Java.Interop.DoNotPackageAttribute]'. Use a newer version of this NuGet package or notify the library author.
- [XA0123](#): Removing {issue} from {propertyName}. Lint {version} does not support this check.
- [XA0125](#): `{Project}` is using a deprecated debug information level. Set the debugging information to Portable in the Visual Studio project property pages or edit the project file in a text editor and set the 'DebugType' MSBuild property to 'portable' to use the newer, cross-platform debug information level. If this file comes from a NuGet package, update to a newer version of the NuGet package or notify the library author.
- [XA0126](#): Error installing FastDev Tools. This device does not support Fast Deployment. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0127](#): There was an issue deploying {destination} using {FastDevTool}. We encountered the following error {output}. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0128](#): Stdio Redirection is enabled. Please disable it to use Fast Deployment.
- [XA0129](#): Error deploying `{File}`. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0130](#): Sorry. Fast deployment is only supported on devices running Android 5.0 (API level 21) or higher. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.

- [XA0131](#): The 'run-as' tool has been disabled on this device. Either enable it by activating the developer options on the device or by setting `ro.boot.disable_runas` to `false`.
- [XA0132](#): The package was not installed. Please check you do not have it installed under any other user. If the package does show up on the device, try manually uninstalling it then try again. You should be able to uninstall the app via the Settings app on the device.
- [XA0133](#): The 'run-as' tool required by the Fast Deployment system has been disabled on this device by the manufacturer. Please disable Fast Deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0134](#): The application does not have the 'android:debuggable' attribute set in the AndroidManifest.xml. This is required in order for Fast Deployment to work. This is normally enabled by default by the .NET for Android build system for Debug builds.
- [XA0135](#): The package is a 'system' application. These are applications which install under the 'system' user on a device. These types of applications cannot use 'run-as'.
- [XA0136](#): The currently installation of the package is corrupt. Please manually uninstall the package from all the users on device and try again. If that does not work you can disable Fast Deployment.
- [XA0137](#): The 'run-as' command failed with '{0}'. Fast Deployment is not currently supported on this device. Please file an issue with the exact error message using the 'Help->Send Feedback->Report a Problem' menu item in Visual Studio or 'Help->Report a Problem' in Visual Studio for Mac.
- [XA0138](#): %(AndroidAsset.AssetPack) and %(AndroidAsset.AssetPack) item metadata are only supported when `$(AndroidApplication)` is `true`.
- [XA0139](#): `@(AndroidAsset) {0}` has invalid `DeliveryType` metadata of `{1}`. Supported values are `installtime`, `ondemand` or `fastfollow`
- [XA0140](#):
- [XA0141](#): NuGet package '{0}' version '{1}' contains a shared library '{2}' which is not correctly aligned. See <https://developer.android.com/guide/practices/page-sizes> for more details
- [XA0142](#): Command '{0}' failed.\n{1}

XA1xxx: Project related

- [XA1000](#): There was a problem parsing {file}. This is likely due to incomplete or invalid XML.

- [XA1001](#): AndroidResgen: Warning while updating Resource XML '{filename}': {Message}
- [XA1002](#): The closest match found for '{customViewName}' is '{customViewLookupName}', but the capitalization does not match. Please correct the capitalization.
- [XA1003](#): '{zip}' does not exist. Please rebuild the project.
- [XA1004](#): There was an error opening {filename}. The file is probably corrupt. Try deleting it and building again.
- [XA1005](#): Attempting basic type name matching for element with ID '{id}' and type '{managedType}'
- [XA1006](#): The TargetFrameworkVersion (Android API level {compileSdk}) is higher than the targetSdkVersion ({targetSdk}).
- [XA1007](#): The minSdkVersion ({minSdk}) is greater than targetSdkVersion. Please change the value such that minSdkVersion is less than or equal to targetSdkVersion ({targetSdk}).
- [XA1008](#): The TargetFrameworkVersion (Android API level {compileSdk}) is lower than the targetSdkVersion ({targetSdk}).
- [XA1009](#): The {assembly} is Obsolete. Please upgrade to {assembly} {version}
- [XA1010](#): Invalid `\$(AndroidManifestPlaceholders)` value for Android manifest placeholders. Please use `key1=value1;key2=value2` format. The specified value was: {placeholders}
- [XA1011](#): Using ProGuard with the D8 DEX compiler is no longer supported. Please set the code shrinker to 'r8' in the Visual Studio project property pages or edit the project file in a text editor and set the 'AndroidLinkTool' MSBuild property to 'r8'.
- XA1012: Included layout root element override ID '{id}' is not valid.
- XA1013: Failed to parse ID of node '{name}' in the layout file '{file}'.
- XA1014: JAR library references with identical file names but different contents were found: {libraries}. Please remove any conflicting libraries from EmbeddedJar, InputJar and AndroidJavaLibrary.
- XA1015: More than one Android Wear project is specified as the paired project. It can be at most one.
- XA1016: Target Wear application's project '{project}' does not specify required 'AndroidManifest' project property.
- XA1017: Target Wear application's AndroidManifest.xml does not specify required 'package' attribute.
- XA1018: Specified AndroidManifest file does not exist: {file}.
- XA1019: `LibraryProjectProperties` file `{file}` is located in a parent directory of the bindings project's intermediate output directory. Please adjust the path to use the original `project.properties` file directly from the Android library project directory.

- XA1020: At least one Java library is required for binding. Check that a Java library is included in the project and has the appropriate build action: 'LibraryProjectZip' (for AAR or ZIP), 'EmbeddedJar', 'InputJar' (for JAR), or 'LibraryProjectProperties' (project.properties).
- XA1021: Specified source Java library not found: {file}
- XA1022: Specified reference Java library not found: {file}
- [XA1023](#): Using the DX DEX Compiler is deprecated.
- [XA1024](#): Ignoring configuration file 'Foo.dll.config'. .NET configuration files are not supported in .NET for Android projects that target .NET 6 or higher.
- [XA1025](#): The experimental 'Hybrid' value for the 'AndroidAotMode' MSBuild property is not currently compatible with the armeabi-v7a target ABI.
- [XA1027](#): The 'EnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1028](#): The 'AndroidEnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1029](#): The 'AotAssemblies' MSBuild property is deprecated. Edit the project file in a text editor to remove this property, and use the 'RunAOTCompilation' MSBuild property instead.
- [XA1031](#): The 'AndroidHttpClientHandlerType' has an invalid value.
- [XA1032](#): Failed to resolve '{0}' from '{1}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1033](#): Could not resolve '{0}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1035](#): The 'BundleAssemblies' property is deprecated and it has no effect on the application build. Equivalent functionality is implemented by the 'AndroidUseAssemblyStore' and 'AndroidEnableAssemblyCompression' properties.
- [XA1036](#): AndroidManifest.xml //uses-sdk/@android:minSdkVersion '29' does not match the \$(SupportedOSPlatformVersion) value '21' in the project file (if there is no \$(SupportedOSPlatformVersion) value in the project file, then a default value has been assumed). Either change the value in the AndroidManifest.xml to match the \$(SupportedOSPlatformVersion) value, or remove the value in the AndroidManifest.xml (and add a \$(SupportedOSPlatformVersion) value to the project file if it doesn't already exist).
- [XA1037](#): The '{0}' MSBuild property is deprecated and will be removed in .NET {1}. See <https://aka.ms/net-android-deprecations> for more details.
- [XA1038](#): The '{0}' MSBuild property has an invalid value. Value values are {1}.

XA2xxx: Linker

- [XA2000](#): Use of AppDomain.CreateDomain() detected in assembly: {assembly}. .NET 6 will only support a single AppDomain, so this API will no longer be available in .NET for Android once .NET 6 is released.
- [XA2001](#): Source file '{filename}' could not be found.
- [XA2002](#): Can not resolve reference: `{missing}`, referenced by {assembly}. Perhaps it doesn't exist in the Mono for Android profile?
- XA2006: Could not resolve reference to '{member}' (defined in assembly '{assembly}') with scope '{scope}'. When the scope is different from the defining assembly, it usually means that the type is forwarded.
- XA2007: Exception while loading assemblies: {exception}
- XA2008: In referenced assembly {assembly}, Java.Interop.DoNotPackageAttribute requires non-null file name.

XA3xxx: Unmanaged code compilation

- XA3001: Could not AOT the assembly: {assembly}
- XA3002: Invalid AOT mode: {mode}
- XA3003: Could not strip IL of assembly: {assembly}
- XA3004: Android NDK r10d is buggy and provides an incompatible x86_64 libm.so.
- XA3005: The detected Android NDK version is incompatible with the targeted LLVM configuration.
- XA3006: Could not compile native assembly file: {file}
- XA3007: Could not link native shared library: {library}

XA4xxx: Code generation

- XA4209: Failed to generate Java type for class: {managedType} due to {exception}
- XA4210: Please add a reference to Mono.Android.Export.dll when using ExportAttribute or ExportFieldAttribute.
- XA4211: AndroidManifest.xml //uses-sdk/@android:targetSdkVersion '{targetSdk}' is less than \$(TargetFrameworkVersion) '{targetFramework}'. Using API-{targetFrameworkApi} for ACW compilation.
- XA4213: The type '{type}' must provide a public default constructor
- [XA4214](#): The managed type `Library1.Class1` exists in multiple assemblies: Library1, Library2. Please refactor the managed type names in these assemblies so that they are not identical.
- [XA4215](#): The Java type `com.contoso.library1.Class1` is generated by more than one managed type. Please change the [Register] attribute so that the same Java type is not emitted.

- [XA4216](#): The deployment target '19' is not supported (the minimum is '21'). Please increase the \$(SupportedOSPlatformVersion) property value in your project file.
- XA4217: Cannot override Kotlin-generated method '{method}' because it is not a valid Java method name. This method can only be overridden from Kotlin.
- [XA4218](#): Unable to find //manifest/application/uses-library at path: {path}
- XA4219: Cannot find binding generator for language {language} or {defaultLanguage}.
- XA4220: Partial class item '{file}' does not have an associated binding for layout '{layout}'.
- XA4221: No layout binding source files were generated.
- XA4222: No widgets found for layout ({layoutFiles}).
- XA4223: Malformed full class name '{name}'. Missing namespace.
- XA4224: Malformed full class name '{name}'. Missing class name.
- XA4225: Widget '{widget}' in layout '{layout}' has multiple instances with different types. The property type will be set to: {type}
- XA4226: Resource item '{file}' does not have the required metadata item '{metadataName}'.
- XA4228: Unable to find specified //activity-alias/@android:targetActivity: '{targetActivity}'
- XA4229: Unrecognized `TransformFile` root element: {element}.
- XA4230: Error parsing XML: {exception}
- [XA4231](#): The Android class parser value 'jar2xml' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'class-parse'.
- [XA4232](#): The Android code generation target 'XamarinAndroid' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'XJavaInterop1'.
- [XA4234](#): '<{item}>' item '{itemspec}' is missing required attribute '{name}'.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id'.
- [XA4236](#): Cannot download Maven artifact '{group}:{artifact}'. - {jar}: {exception} - {aar}: {exception}
- [XA4237](#): Cannot download POM file for Maven artifact '{artifact}'. - {exception}
- [XA4239](#): Unknown Maven repository: '{repository}'.
- [XA4241](#): Java dependency '{artifact}' is not satisfied.
- [XA4242](#): Java dependency '{artifact}' is not satisfied. Microsoft maintains the NuGet package '{nugetId}' that could fulfill this dependency.
- [XA4243](#): Attribute '{name}' is required when using '{name}' for '{element}' item '{itemspec}'.
- [XA4244](#): Attribute '{name}' cannot be empty for '{element}' item '{itemspec}'.
- [XA4245](#): Specified POM file '{file}' does not exist.

- [XA4246](#): Could not parse POM file '{file}'. - {exception}
- [XA4247](#): Could not resolve POM file for artifact '{artifact}'.
- [XA4248](#): Could not find NuGet package '{nugetId}' version '{version}' in lock file. Ensure NuGet Restore has run since this `<PackageReference>` was added.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id:version'.
- XA4300: Native library '{library}' will not be bundled because it has an unsupported ABI.
- [XA4301](#): Apk already contains the item `xxx`.
- [XA4302](#): Unhandled exception merging `AndroidManifest.xml`: {ex}
- [XA4303](#): Error extracting resources from "{assemblyPath}": {ex}
- [XA4304](#): ProGuard configuration file '{file}' was not found.
- [XA4305](#): Multidex is enabled, but `\$\_AndroidMainDexListFile` is empty.
- [XA4306](#): R8 does not support `@(MultiDexMainDexList)` files when `android:minSdkVersion >= 21`
- [XA4307](#): Invalid ProGuard configuration file.
- [XA4308](#): Failed to generate type maps
- [XA4309](#): 'MultiDexMainDexList' file '{file}' does not exist.
- [XA4310](#): `\$(AndroidSigningKeyStore)` file '{keystore}' could not be found.
- XA4311: The application won't contain the paired Wear package because the Wear application package APK is not created yet. If building on the command line, be sure to build the "SignAndroidPackage" target.
- [XA4312](#): Referencing an Android Wear application project from an Android application project is deprecated.
- [XA4313](#): Framework assembly has been deprecated.
- [XA4314](#): `$(Property)` is empty. A value for `$(Property)` should be provided.
- [XA4315](#): Ignoring {file}. Manifest does not have the required 'package' attribute on the manifest element.

XA5xxx: GCC and toolchain

- XA5101: Missing Android NDK toolchains directory '{path}'. Please install the Android NDK.
- XA5102: Conversion from assembly to native code failed. Exit code {exitCode}
- XA5103: NDK C compiler exited with an error. Exit code {0}
- XA5104: Could not locate the Android NDK.
- XA5105: Toolchain utility '{utility}' for target {arch} was not found. Tried in path: "{path}"
- XA5201: NDK linker exited with an error. Exit code {0}
- [XA5205](#): Cannot find `{ToolName}` in the Android SDK.

- [XA5207](#): Could not find android.jar for API level `{compileSdk}`.
- [XA5211](#): Embedded Wear app package name differs from handheld app package name (`{wearPackageName} != {handheldPackageName}`).
- [XA5213](#): `java.lang.OutOfMemoryError`. Consider increasing the value of `$(JavaMaximumHeapSize)`. Java ran out of memory while executing '`{tool}` `{arguments}`'
- [XA5300](#): The Android/Java SDK Directory could not be found.
- [XA5301](#): Failed to generate Java type for class: `{managedType}` due to `MAX_PATH`: `{exception}`
- [XA5302](#): Two processes may be building this project at once. Lock file exists at path: `{path}`

XA6xxx: Internal tools

XAccc7xxx: Unhandled MSBuild Exceptions

Exceptions that have not been gracefully handled yet. Ideally these will be fixed or replaced with better errors in the future.

These take the form of `XACCC7NNN`, where `CCC` is a 3 character code denoting the MSBuild Task that is throwing the exception, and `NNN` is a 3 digit number indicating the type of the unhandled `Exception`.

Tasks:

- `A2C` - `Aapt2Compile`
- `A2L` - `Aapt2Link`
- `AAS` - `AndroidApkSigner`
- `ACD` - `AndroidCreateDebugKey`
- `ACM` - `AppendCustomMetadataToItemGroup`
- `ADB` - `Adb`
- `AJV` - `AdjustJavacVersionArguments`
- `AOT` - `Aot`
- `APT` - `Aapt`
- `ASP` - `AndroidSignPackage`
- `AZA` - `AndroidZipAlign`
- `BAB` - `BuildAppBundle`
- `BAS` - `BuildApkSet`
- `BBA` - `BuildBaseAppBundle`

- BGN - BindingsGenerator
- BLD - BuildApk
- CAL - CreateAdditionalLibraryResourceCache
- CAR - CalculateAdditionalResourceCacheDirectories
- CCR - CopyAndConvertResources
- CCV - ConvertCustomView
- CDF - ConvertDebuggingFiles
- CDJ - CheckDuplicateJavaLibraries
- CFI - CheckForInvalidResourceFileNames
- CFR - CheckForRemovedItems
- CGJ - CopyGeneratedJavaResourceClasses
- CGS - CheckGoogleSdkRequirements
- CIC - CopyIfChanged
- CIL - CilStrip
- CLA - CollectLibraryAssets
- CLC - CalculateLayoutCodeBehind
- CLP - ClassParse
- CLR - CreateLibraryResourceArchive
- CMD - CreateMultiDexMainDexClassList
- CML - CreateManagedLibraryResourceArchive
- CMM - CreateMsymManifest
- CNA - CompileNativeAssembly
- CNE - CollectNonEmptyDirectories
- CNL - CreateNativeLibraryArchive
- CPD - CalculateProjectDependencies
- CPF - CollectPdbFiles
- CPI - CheckProjectItems
- CPR - CopyResource
- CPT - ComputeHash
- CRC - ConvertResourcesCases
- CRM - CreateResgenManifest
- CRN - Crunch
- CRP - AndroidComputeResPaths
- CTD - CreateTemporaryDirectory
- CTX - CompileToDalvik
- DES - Desugar
- DJL - DetermineJavaLibrariesToCompile
- DX8 - D8

- FD - FastDeploy
- FLB - FindLayoutsToBind
- FLT - FilterAssemblies
- GAD - GetAndroidDefineConstants
- GAP - GetAndroidPackageName
- GAR - GetAdditionalResourcesFromAssemblies
- GAS - GetAppSettingsDirectory
- GCB - GenerateCodeBehindForLayout
- GCJ - GetConvertedJavaLibraries
- GEP - GetExtraPackages
- GFT - GetFilesThatExist
- GIL - GetImportedLibraries
- GJP - GetJavaPlatformJar
- GJS - GenerateJavaStubs
- GLB - GenerateLayoutBindings
- GLR - GenerateLibraryResources
- GMA - GenerateManagedAidlProxies
- GMJ - GetMonoPlatformJar
- GPM - GeneratePackageManagerJava
- GRD - GenerateResourceDesigner
- IAS - InstallApkSet
- IJD - ImportJavaDoc
- JDC - JavaDoc
- JVC - Javac
- JTX - JarToXml
- KEY - KeyTool
- LAS - LinkApplicationSharedLibraries
- LEF - LogErrorsForFiles
- LNK - LinkAssemblies
- LNS - LinkAssembliesNoShrink
- LNT - Lint
- LWF - LogWarningsForFiles
- MBN - MakeBundleNativeCodeExternal
- MDC - MDoc
- PAI - PrepareAbiItems
- PAW - ParseAndroidWearProjectAndManifest
- PRO - Proguard
- PWA - PrepareWearApplicationFiles

- R8D - R8
- RAM - ReadAndroidManifest
- RAR - ReadAdditionalResourcesFromAssemblyCache
- RAT - ResolveAndroidTooling
- RDF - RemoveDirFixed
- RIL - ReadImportedLibrariesCache
- RJJ - ResolveJdkJvmPath
- RLC - ReadLibraryProjectImportsCache
- RLP - ResolveLibraryProjectImports
- RRA - RemoveRegisterAttribute
- RSA - ResolveAssemblies
- RSD - ResolveSdks
- RUF - RemoveUnknownFiles
- SPL - SplitProperty
- SVM - SetVsMonoAndroidRegistryKey
- UNZ - Unzip
- VJV - ValidateJavaVersion
- WLF - WriteLockFile

Exceptions:

- 7000 - Other Exception
- 7001 - NullPointerException
- 7002 - ArgumentOutOfRangeException
- 7003 - ArgumentNullException
- 7004 - ArgumentException
- 7005 - FormatException
- 7006 - IndexOutOfRangeException
- 7007 - InvalidCastException
- 7008 - ObjectDisposedException
- 7009 - InvalidOperationException
- 7010 - InvalidProgramException
- 7011 - KeyNotFoundException
- 7012 - TaskCanceledException
- 7013 - OperationCanceledException
- 7014 - OutOfMemoryException
- 7015 - NotSupportedException
- 7016 - StackOverflowException

- 7017 - `TimeoutException`
- 7018 - `TypeInitializationException`
- 7019 - `UnauthorizedAccessException`
- 7020 - `ApplicationException`
- 7021 - `KeyNotFoundException`
- 7022 - `PathTooLongException`
- 7023 - `DirectoryNotFoundException`
- 7024 - `IOException`
- 7025 - `DriveNotFoundException`
- 7026 - `EndOfStreamException`
- 7027 - `FileLoadException`
- 7028 - `FileNotFoundException`
- 7029 - `PipeException`

XA8xxx: Linker Step Errors

- [XA8000/IL8000](#): Could not find Android Resource '@anim/enterfromright'. Please update `@(AndroidResource)` to add the missing resource.

XA9xxx: Licensing

Removed messages

Removed in Xamarin.Android 10.4

- XA5215: Duplicate Resource found for {elementName}. Duplicates are in {filenames}
- XA5216: Resource entry {elementName} is already defined in {filename}

Removed in Xamarin.Android 10.3

- [XA0110](#): Disabling `$(AndroidExplicitCrunch)` as it is not supported by `aapt2`. If you wish to use `$(AndroidExplicitCrunch)` please set `$(AndroidUseAapt2)` to false.

Removed in Xamarin.Android 10.2

- [XA0120](#): Failed to use SHA1 hash algorithm

Removed in Xamarin.Android 9.3

- [XA0114](#): Google Play requires that application updates must use a `$(TargetFrameworkVersion)` of v8.0 (API level 26) or above.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android errors and warnings reference

Article • 04/17/2024

ADBxxxx: ADB tooling

- [ADB0000](#): Generic `adb` error/warning.
- [ADB0010](#): Generic `adb` APK installation error/warning.
- [ADB0020](#): The package does not support the CPU architecture of this device.
- [ADB0030](#): APK installation failed due to a conflict with the existing package.
- [ADB0040](#): The device does not support the minimum SDK level specified in the manifest.
- [ADB0050](#): Package {packageName} already exists on device.
- [ADB0060](#): There is not enough storage space on the device to store package: {packageName}. Free up some space and try again.

ANDXXxxxx: Generic Android tooling

- [ANDAS0000](#): Generic `apksigner` error/warning.
- [ANDJS0000](#): Generic `jarsigner` error/warning.
- [ANDKT0000](#): Generic `keytool` error/warning.
- [ANDZA0000](#): Generic `zipalign` error/warning.

APTxxxx: AAPT tooling

- [APT0000](#): Generic `aapt` error/warning.
- [APT0001](#): Unknown option `{option}`. Please check ``$(AndroidAapt2CompileExtraArgs)`` and ``$(AndroidAapt2LinkExtraArgs)`` to see if they include any ``aapt`` command line arguments that are no longer valid for ``aapt2`` and ensure that all other arguments are valid for `aapt2`.
- [APT0002](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0003](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0004](#): Invalid file name: It must start with either A-Z or a-z or an underscore.
- [APT2264](#): The system cannot find the file specified. (2).
- [APT2265](#): The system cannot find the file specified. (2).

JAVAxxxx: Java Tool

- [JAVA0000](#): Generic Java tool error

JAVACxxxx: Java compiler

- [JAVAC0000](#): Generic Java compiler error

XA0xxx: Environment issue or missing tooling

- [XA0000](#): Could not determine `$(AndroidApiLevel)` or `$(TargetFrameworkVersion)`.
- [XA0001](#): Invalid or unsupported `$(TargetFrameworkVersion)` value.
- [XA0002](#): Could not find mono.android.jar
- [XA0003](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. It must be an integer value.
- [XA0004](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. The value must be in the range of 0 to `{maxVersionCode}`.
- [XA0030](#): Building with JDK version `{versionNumber}` is not supported.
- [XA0031](#): Java SDK `{requiredJavaForFrameworkVersion}` or above is required when using `$(TargetFrameworkVersion)` `{targetFrameworkVersion}`.
- [XA0032](#): Java SDK `{requiredJavaForBuildTools}` or above is required when using Android SDK Build-Tools `{buildToolsVersion}`.
- [XA0033](#): Failed to get the Java SDK version because the returned value does not appear to contain a valid version number.
- [XA0034](#): Failed to get the Java SDK version.
- [XA0035](#): Failed to determine the Android ABI for the project.
- [XA0036](#): `$(AndroidSupportedAbis)` is not supported in .NET 6 and higher.
- [XA0100](#): `EmbeddedNativeLibrary` is invalid in Android Application projects. Please use `AndroidNativeLibrary` instead.
- [XA0101](#): warning XA0101: `@(Content)` build action is not supported.
- [XA0102](#): Generic `lint` Warning.
- [XA0103](#): Generic `lint` Error.
- [XA0104](#): Invalid value for ``$(AndroidSequencePointsMode)``
- [XA0105](#): The `$(TargetFrameworkVersion)` for a library is greater than the `$(TargetFrameworkVersion)` for the application project.
- [XA0107](#): `{Assmebly}` is a Reference Assembly.
- [XA0108](#): Could not get version from `lint`.
- [XA0109](#): Unsupported or invalid `$(TargetFrameworkVersion)` value of `'v4.5'`.
- [XA0111](#): Could not get the `aapt2` version. Please check it is installed correctly.

- [XA0112](#): `aapt2` is not installed. Disabling `aapt2` support. Please check it is installed correctly.
- [XA0113](#): Google Play requires that new applications and updates must use a TargetFrameworkVersion of v11.0 (API level 30) or above.
- [XA0115](#): Invalid value 'armeabi' in \$(AndroidSupportedAbis). This ABI is no longer supported. Please update your project properties to remove the old value. If the properties page does not show an 'armeabi' checkbox, un-check and re-check one of the other ABIs and save the changes.
- [XA0116](#): Unable to find `EmbeddedResource` named `{ResourceName}`.
- [XA0117](#): The TargetFrameworkVersion {TargetFrameworkVersion} is deprecated. Please update it to be v4.4 or higher.
- [XA0118](#): Could not parse '{TargetMoniker}'
- [XA0119](#): A non-ideal configuration was found in the project.
- [XA0121](#): Assembly '{assembly}' is using '[assembly: Java.Interop.JavaLibraryReferenceAttribute]', which is no longer supported. Use a newer version of this NuGet package or notify the library author.
- [XA0122](#): Assembly '{assembly}' is using a deprecated attribute '[assembly: Java.Interop.DoNotPackageAttribute]'. Use a newer version of this NuGet package or notify the library author.
- [XA0123](#): Removing {issue} from {propertyName}. Lint {version} does not support this check.
- [XA0125](#): `{Project}` is using a deprecated debug information level. Set the debugging information to Portable in the Visual Studio project property pages or edit the project file in a text editor and set the 'DebugType' MSBuild property to 'portable' to use the newer, cross-platform debug information level. If this file comes from a NuGet package, update to a newer version of the NuGet package or notify the library author.
- [XA0126](#): Error installing FastDev Tools. This device does not support Fast Deployment. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0127](#): There was an issue deploying {destination} using {FastDevTool}. We encountered the following error {output}. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0128](#): Stdio Redirection is enabled. Please disable it to use Fast Deployment.
- [XA0129](#): Error deploying `{File}`. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0130](#): Sorry. Fast deployment is only supported on devices running Android 5.0 (API level 21) or higher. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.

- [XA0131](#): The 'run-as' tool has been disabled on this device. Either enable it by activating the developer options on the device or by setting `ro.boot.disable_runas` to `false`.
- [XA0132](#): The package was not installed. Please check you do not have it installed under any other user. If the package does show up on the device, try manually uninstalling it then try again. You should be able to uninstall the app via the Settings app on the device.
- [XA0133](#): The 'run-as' tool required by the Fast Deployment system has been disabled on this device by the manufacturer. Please disable Fast Deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0134](#): The application does not have the 'android:debuggable' attribute set in the AndroidManifest.xml. This is required in order for Fast Deployment to work. This is normally enabled by default by the .NET for Android build system for Debug builds.
- [XA0135](#): The package is a 'system' application. These are applications which install under the 'system' user on a device. These types of applications cannot use 'run-as'.
- [XA0136](#): The currently installation of the package is corrupt. Please manually uninstall the package from all the users on device and try again. If that does not work you can disable Fast Deployment.
- [XA0137](#): The 'run-as' command failed with '{0}'. Fast Deployment is not currently supported on this device. Please file an issue with the exact error message using the 'Help->Send Feedback->Report a Problem' menu item in Visual Studio or 'Help->Report a Problem' in Visual Studio for Mac.
- [XA0138](#): %(AndroidAsset.AssetPack) and %(AndroidAsset.AssetPack) item metadata are only supported when `$(AndroidApplication)` is `true`.
- [XA0139](#): `@(AndroidAsset) {0}` has invalid `DeliveryType` metadata of `{1}`. Supported values are `installtime`, `ondemand` or `fastfollow`
- [XA0140](#):
- [XA0141](#): NuGet package '{0}' version '{1}' contains a shared library '{2}' which is not correctly aligned. See <https://developer.android.com/guide/practices/page-sizes> for more details
- [XA0142](#): Command '{0}' failed.\n{1}

XA1xxx: Project related

- [XA1000](#): There was a problem parsing {file}. This is likely due to incomplete or invalid XML.

- [XA1001](#): AndroidResgen: Warning while updating Resource XML '{filename}': {Message}
- [XA1002](#): The closest match found for '{customViewName}' is '{customViewLookupName}', but the capitalization does not match. Please correct the capitalization.
- [XA1003](#): '{zip}' does not exist. Please rebuild the project.
- [XA1004](#): There was an error opening {filename}. The file is probably corrupt. Try deleting it and building again.
- [XA1005](#): Attempting basic type name matching for element with ID '{id}' and type '{managedType}'
- [XA1006](#): The TargetFrameworkVersion (Android API level {compileSdk}) is higher than the targetSdkVersion ({targetSdk}).
- [XA1007](#): The minSdkVersion ({minSdk}) is greater than targetSdkVersion. Please change the value such that minSdkVersion is less than or equal to targetSdkVersion ({targetSdk}).
- [XA1008](#): The TargetFrameworkVersion (Android API level {compileSdk}) is lower than the targetSdkVersion ({targetSdk}).
- [XA1009](#): The {assembly} is Obsolete. Please upgrade to {assembly} {version}
- [XA1010](#): Invalid `\$(AndroidManifestPlaceholders)` value for Android manifest placeholders. Please use `key1=value1;key2=value2` format. The specified value was: {placeholders}
- [XA1011](#): Using ProGuard with the D8 DEX compiler is no longer supported. Please set the code shrinker to 'r8' in the Visual Studio project property pages or edit the project file in a text editor and set the 'AndroidLinkTool' MSBuild property to 'r8'.
- XA1012: Included layout root element override ID '{id}' is not valid.
- XA1013: Failed to parse ID of node '{name}' in the layout file '{file}'.
- XA1014: JAR library references with identical file names but different contents were found: {libraries}. Please remove any conflicting libraries from EmbeddedJar, InputJar and AndroidJavaLibrary.
- XA1015: More than one Android Wear project is specified as the paired project. It can be at most one.
- XA1016: Target Wear application's project '{project}' does not specify required 'AndroidManifest' project property.
- XA1017: Target Wear application's AndroidManifest.xml does not specify required 'package' attribute.
- XA1018: Specified AndroidManifest file does not exist: {file}.
- XA1019: `LibraryProjectProperties` file `{file}` is located in a parent directory of the bindings project's intermediate output directory. Please adjust the path to use the original `project.properties` file directly from the Android library project directory.

- XA1020: At least one Java library is required for binding. Check that a Java library is included in the project and has the appropriate build action: 'LibraryProjectZip' (for AAR or ZIP), 'EmbeddedJar', 'InputJar' (for JAR), or 'LibraryProjectProperties' (project.properties).
- XA1021: Specified source Java library not found: {file}
- XA1022: Specified reference Java library not found: {file}
- [XA1023](#): Using the DX DEX Compiler is deprecated.
- [XA1024](#): Ignoring configuration file 'Foo.dll.config'. .NET configuration files are not supported in .NET for Android projects that target .NET 6 or higher.
- [XA1025](#): The experimental 'Hybrid' value for the 'AndroidAotMode' MSBuild property is not currently compatible with the armeabi-v7a target ABI.
- [XA1027](#): The 'EnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1028](#): The 'AndroidEnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1029](#): The 'AotAssemblies' MSBuild property is deprecated. Edit the project file in a text editor to remove this property, and use the 'RunAOTCompilation' MSBuild property instead.
- [XA1031](#): The 'AndroidHttpClientHandlerType' has an invalid value.
- [XA1032](#): Failed to resolve '{0}' from '{1}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1033](#): Could not resolve '{0}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1035](#): The 'BundleAssemblies' property is deprecated and it has no effect on the application build. Equivalent functionality is implemented by the 'AndroidUseAssemblyStore' and 'AndroidEnableAssemblyCompression' properties.
- [XA1036](#): AndroidManifest.xml //uses-sdk/@android:minSdkVersion '29' does not match the \$(SupportedOSPlatformVersion) value '21' in the project file (if there is no \$(SupportedOSPlatformVersion) value in the project file, then a default value has been assumed). Either change the value in the AndroidManifest.xml to match the \$(SupportedOSPlatformVersion) value, or remove the value in the AndroidManifest.xml (and add a \$(SupportedOSPlatformVersion) value to the project file if it doesn't already exist).
- [XA1037](#): The '{0}' MSBuild property is deprecated and will be removed in .NET {1}. See <https://aka.ms/net-android-deprecations> for more details.
- [XA1038](#): The '{0}' MSBuild property has an invalid value. Value values are {1}.

XA2xxx: Linker

- [XA2000](#): Use of AppDomain.CreateDomain() detected in assembly: {assembly}. .NET 6 will only support a single AppDomain, so this API will no longer be available in .NET for Android once .NET 6 is released.
- [XA2001](#): Source file '{filename}' could not be found.
- [XA2002](#): Can not resolve reference: `{missing}`, referenced by {assembly}. Perhaps it doesn't exist in the Mono for Android profile?
- XA2006: Could not resolve reference to '{member}' (defined in assembly '{assembly}') with scope '{scope}'. When the scope is different from the defining assembly, it usually means that the type is forwarded.
- XA2007: Exception while loading assemblies: {exception}
- XA2008: In referenced assembly {assembly}, Java.Interop.DoNotPackageAttribute requires non-null file name.

XA3xxx: Unmanaged code compilation

- XA3001: Could not AOT the assembly: {assembly}
- XA3002: Invalid AOT mode: {mode}
- XA3003: Could not strip IL of assembly: {assembly}
- XA3004: Android NDK r10d is buggy and provides an incompatible x86_64 libm.so.
- XA3005: The detected Android NDK version is incompatible with the targeted LLVM configuration.
- XA3006: Could not compile native assembly file: {file}
- XA3007: Could not link native shared library: {library}

XA4xxx: Code generation

- XA4209: Failed to generate Java type for class: {managedType} due to {exception}
- XA4210: Please add a reference to Mono.Android.Export.dll when using ExportAttribute or ExportFieldAttribute.
- XA4211: AndroidManifest.xml //uses-sdk/@android:targetSdkVersion '{targetSdk}' is less than \$(TargetFrameworkVersion) '{targetFramework}'. Using API-{targetFrameworkApi} for ACW compilation.
- XA4213: The type '{type}' must provide a public default constructor
- [XA4214](#): The managed type `Library1.Class1` exists in multiple assemblies: Library1, Library2. Please refactor the managed type names in these assemblies so that they are not identical.
- [XA4215](#): The Java type `com.contoso.library1.Class1` is generated by more than one managed type. Please change the [Register] attribute so that the same Java type is not emitted.

- [XA4216](#): The deployment target '19' is not supported (the minimum is '21'). Please increase the \$(SupportedOSPlatformVersion) property value in your project file.
- XA4217: Cannot override Kotlin-generated method '{method}' because it is not a valid Java method name. This method can only be overridden from Kotlin.
- [XA4218](#): Unable to find //manifest/application/uses-library at path: {path}
- XA4219: Cannot find binding generator for language {language} or {defaultLanguage}.
- XA4220: Partial class item '{file}' does not have an associated binding for layout '{layout}'.
- XA4221: No layout binding source files were generated.
- XA4222: No widgets found for layout ({layoutFiles}).
- XA4223: Malformed full class name '{name}'. Missing namespace.
- XA4224: Malformed full class name '{name}'. Missing class name.
- XA4225: Widget '{widget}' in layout '{layout}' has multiple instances with different types. The property type will be set to: {type}
- XA4226: Resource item '{file}' does not have the required metadata item '{metadataName}'.
- XA4228: Unable to find specified //activity-alias/@android:targetActivity: '{targetActivity}'
- XA4229: Unrecognized `TransformFile` root element: {element}.
- XA4230: Error parsing XML: {exception}
- [XA4231](#): The Android class parser value 'jar2xml' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'class-parse'.
- [XA4232](#): The Android code generation target 'XamarinAndroid' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'XJavaInterop1'.
- [XA4234](#): '<{item}>' item '{itemspec}' is missing required attribute '{name}'.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id'.
- [XA4236](#): Cannot download Maven artifact '{group}:{artifact}'. - {jar}: {exception} - {aar}: {exception}
- [XA4237](#): Cannot download POM file for Maven artifact '{artifact}'. - {exception}
- [XA4239](#): Unknown Maven repository: '{repository}'.
- [XA4241](#): Java dependency '{artifact}' is not satisfied.
- [XA4242](#): Java dependency '{artifact}' is not satisfied. Microsoft maintains the NuGet package '{nugetId}' that could fulfill this dependency.
- [XA4243](#): Attribute '{name}' is required when using '{name}' for '{element}' item '{itemspec}'.
- [XA4244](#): Attribute '{name}' cannot be empty for '{element}' item '{itemspec}'.
- [XA4245](#): Specified POM file '{file}' does not exist.

- [XA4246](#): Could not parse POM file '{file}'. - {exception}
- [XA4247](#): Could not resolve POM file for artifact '{artifact}'.
- [XA4248](#): Could not find NuGet package '{nugetId}' version '{version}' in lock file. Ensure NuGet Restore has run since this `<PackageReference>` was added.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id:version'.
- XA4300: Native library '{library}' will not be bundled because it has an unsupported ABI.
- [XA4301](#): Apk already contains the item `xxx`.
- [XA4302](#): Unhandled exception merging `AndroidManifest.xml`: {ex}
- [XA4303](#): Error extracting resources from "{assemblyPath}": {ex}
- [XA4304](#): ProGuard configuration file '{file}' was not found.
- [XA4305](#): Multidex is enabled, but `\$\_AndroidMainDexListFile` is empty.
- [XA4306](#): R8 does not support `@(MultiDexMainDexList)` files when `android:minSdkVersion >= 21`
- [XA4307](#): Invalid ProGuard configuration file.
- [XA4308](#): Failed to generate type maps
- [XA4309](#): 'MultiDexMainDexList' file '{file}' does not exist.
- [XA4310](#): `\$(AndroidSigningKeyStore)` file '{keystore}' could not be found.
- XA4311: The application won't contain the paired Wear package because the Wear application package APK is not created yet. If building on the command line, be sure to build the "SignAndroidPackage" target.
- [XA4312](#): Referencing an Android Wear application project from an Android application project is deprecated.
- [XA4313](#): Framework assembly has been deprecated.
- [XA4314](#): `$(Property)` is empty. A value for `$(Property)` should be provided.
- [XA4315](#): Ignoring {file}. Manifest does not have the required 'package' attribute on the manifest element.

XA5xxx: GCC and toolchain

- XA5101: Missing Android NDK toolchains directory '{path}'. Please install the Android NDK.
- XA5102: Conversion from assembly to native code failed. Exit code {exitCode}
- XA5103: NDK C compiler exited with an error. Exit code {0}
- XA5104: Could not locate the Android NDK.
- XA5105: Toolchain utility '{utility}' for target {arch} was not found. Tried in path: "{path}"
- XA5201: NDK linker exited with an error. Exit code {0}
- [XA5205](#): Cannot find `{ToolName}` in the Android SDK.

- [XA5207](#): Could not find android.jar for API level `{compileSdk}`.
- [XA5211](#): Embedded Wear app package name differs from handheld app package name (`{wearPackageName} != {handheldPackageName}`).
- [XA5213](#): `java.lang.OutOfMemoryError`. Consider increasing the value of `$(JavaMaximumHeapSize)`. Java ran out of memory while executing '`{tool}` `{arguments}`'
- [XA5300](#): The Android/Java SDK Directory could not be found.
- [XA5301](#): Failed to generate Java type for class: `{managedType}` due to MAX_PATH: `{exception}`
- [XA5302](#): Two processes may be building this project at once. Lock file exists at path: `{path}`

XA6xxx: Internal tools

XAccc7xxx: Unhandled MSBuild Exceptions

Exceptions that have not been gracefully handled yet. Ideally these will be fixed or replaced with better errors in the future.

These take the form of `XACCC7NNN`, where `CCC` is a 3 character code denoting the MSBuild Task that is throwing the exception, and `NNN` is a 3 digit number indicating the type of the unhandled `Exception`.

Tasks:

- `A2C` - `Aapt2Compile`
- `A2L` - `Aapt2Link`
- `AAS` - `AndroidApkSigner`
- `ACD` - `AndroidCreateDebugKey`
- `ACM` - `AppendCustomMetadataToItemGroup`
- `ADB` - `Adb`
- `AJV` - `AdjustJavacVersionArguments`
- `AOT` - `Aot`
- `APT` - `Aapt`
- `ASP` - `AndroidSignPackage`
- `AZA` - `AndroidZipAlign`
- `BAB` - `BuildAppBundle`
- `BAS` - `BuildApkSet`
- `BBA` - `BuildBaseAppBundle`

- BGN - BindingsGenerator
- BLD - BuildApk
- CAL - CreateAdditionalLibraryResourceCache
- CAR - CalculateAdditionalResourceCacheDirectories
- CCR - CopyAndConvertResources
- CCV - ConvertCustomView
- CDF - ConvertDebuggingFiles
- CDJ - CheckDuplicateJavaLibraries
- CFI - CheckForInvalidResourceFileNames
- CFR - CheckForRemovedItems
- CGJ - CopyGeneratedJavaResourceClasses
- CGS - CheckGoogleSdkRequirements
- CIC - CopyIfChanged
- CIL - CilStrip
- CLA - CollectLibraryAssets
- CLC - CalculateLayoutCodeBehind
- CLP - ClassParse
- CLR - CreateLibraryResourceArchive
- CMD - CreateMultiDexMainDexClassList
- CML - CreateManagedLibraryResourceArchive
- CMM - CreateMsymManifest
- CNA - CompileNativeAssembly
- CNE - CollectNonEmptyDirectories
- CNL - CreateNativeLibraryArchive
- CPD - CalculateProjectDependencies
- CPF - CollectPdbFiles
- CPI - CheckProjectItems
- CPR - CopyResource
- CPT - ComputeHash
- CRC - ConvertResourcesCases
- CRM - CreateResgenManifest
- CRN - Crunch
- CRP - AndroidComputeResPaths
- CTD - CreateTemporaryDirectory
- CTX - CompileToDalvik
- DES - Desugar
- DJL - DetermineJavaLibrariesToCompile
- DX8 - D8

- FD - FastDeploy
- FLB - FindLayoutsToBind
- FLT - FilterAssemblies
- GAD - GetAndroidDefineConstants
- GAP - GetAndroidPackageName
- GAR - GetAdditionalResourcesFromAssemblies
- GAS - GetAppSettingsDirectory
- GCB - GenerateCodeBehindForLayout
- GCJ - GetConvertedJavaLibraries
- GEP - GetExtraPackages
- GFT - GetFilesThatExist
- GIL - GetImportedLibraries
- GJP - GetJavaPlatformJar
- GJS - GenerateJavaStubs
- GLB - GenerateLayoutBindings
- GLR - GenerateLibraryResources
- GMA - GenerateManagedAidlProxies
- GMJ - GetMonoPlatformJar
- GPM - GeneratePackageManagerJava
- GRD - GenerateResourceDesigner
- IAS - InstallApkSet
- IJD - ImportJavaDoc
- JDC - JavaDoc
- JVC - Javac
- JTX - JarToXml
- KEY - KeyTool
- LAS - LinkApplicationSharedLibraries
- LEF - LogErrorsForFiles
- LNK - LinkAssemblies
- LNS - LinkAssembliesNoShrink
- LNT - Lint
- LWF - LogWarningsForFiles
- MBN - MakeBundleNativeCodeExternal
- MDC - MDoc
- PAI - PrepareAbiItems
- PAW - ParseAndroidWearProjectAndManifest
- PRO - Proguard
- PWA - PrepareWearApplicationFiles

- R8D - R8
- RAM - ReadAndroidManifest
- RAR - ReadAdditionalResourcesFromAssemblyCache
- RAT - ResolveAndroidTooling
- RDF - RemoveDirFixed
- RIL - ReadImportedLibrariesCache
- RJJ - ResolveJdkJvmPath
- RLC - ReadLibraryProjectImportsCache
- RLP - ResolveLibraryProjectImports
- RRA - RemoveRegisterAttribute
- RSA - ResolveAssemblies
- RSD - ResolveSdks
- RUF - RemoveUnknownFiles
- SPL - SplitProperty
- SVM - SetVsMonoAndroidRegistryKey
- UNZ - Unzip
- VJV - ValidateJavaVersion
- WLF - WriteLockFile

Exceptions:

- 7000 - Other Exception
- 7001 - NullPointerException
- 7002 - ArgumentOutOfRangeException
- 7003 - ArgumentNullException
- 7004 - ArgumentException
- 7005 - FormatException
- 7006 - IndexOutOfRangeException
- 7007 - InvalidCastException
- 7008 - ObjectDisposedException
- 7009 - InvalidOperationException
- 7010 - InvalidProgramException
- 7011 - KeyNotFoundException
- 7012 - TaskCanceledException
- 7013 - OperationCanceledException
- 7014 - OutOfMemoryException
- 7015 - NotSupportedException
- 7016 - StackOverflowException

- 7017 - `TimeoutException`
- 7018 - `TypeInitializationException`
- 7019 - `UnauthorizedAccessException`
- 7020 - `ApplicationException`
- 7021 - `KeyNotFoundException`
- 7022 - `PathTooLongException`
- 7023 - `DirectoryNotFoundException`
- 7024 - `IOException`
- 7025 - `DriveNotFoundException`
- 7026 - `EndOfStreamException`
- 7027 - `FileLoadException`
- 7028 - `FileNotFoundException`
- 7029 - `PipeException`

XA8xxx: Linker Step Errors

- [XA8000/IL8000](#): Could not find Android Resource '@anim/enterfromright'. Please update `@(AndroidResource)` to add the missing resource.

XA9xxx: Licensing

Removed messages

Removed in Xamarin.Android 10.4

- XA5215: Duplicate Resource found for {elementName}. Duplicates are in {filenames}
- XA5216: Resource entry {elementName} is already defined in {filename}

Removed in Xamarin.Android 10.3

- [XA0110](#): Disabling `$(AndroidExplicitCrunch)` as it is not supported by `aapt2`. If you wish to use `$(AndroidExplicitCrunch)` please set `$(AndroidUseAapt2)` to false.

Removed in Xamarin.Android 10.2

- [XA0120](#): Failed to use SHA1 hash algorithm

Removed in Xamarin.Android 9.3

- [XA0114](#): Google Play requires that application updates must use a `$(TargetFrameworkVersion)` of v8.0 (API level 26) or above.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error/warning ADB0000

Article • 04/17/2024

Example messages

```
error ADB0000: error: no devices/emulators found
```

Issue

This message indicates that `adb` (Android Debug Bridge) reported an unhandled/unexpected error or warning. `adb` is part of the Android SDK and is used internally by .NET for Android to communicate with Android emulators and devices.

Solution

To learn more about `adb`, see the [Android documentation](#).

Errors reported by `adb` are outside of .NET for Android's control, so a general error code of ADB0000 is used for unexpected errors reporting the exact message coming from `adb`.

Implementation notes

Note that nothing in the open source <https://github.com/xamarin/xamarin-android> repository emits ADB0000, as features such as debugging and "fast deployment" are implemented in the proprietary .NET for Android additions.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error/warning ADB0010

Article • 04/17/2024

Issue

This message indicates that `adb` (Android Debug Bridge) reported an unhandled/unexpected error during APK installation. `adb` is part of the Android SDK and is used internally by .NET for Android to communicate with Android emulators and devices.

Solution

To learn more about `adb`, see the [Android documentation](#).

Errors reported by `adb` are outside of .NET for Android's control, so a general error code of ADB0010 is used for unexpected errors related to APK installation. An error message of the form `[INSTALL_FAILED_*]` will emit this error code.

Implementation notes

Note that nothing in the open source <https://github.com/xamarin/xamarin-android> repository emits ADB0010, as features such as debugging and "fast deployment" are implemented in the proprietary .NET for Android additions.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error ADB0020

Article • 04/17/2024

Example messages

```
error ADB0020: The package does not support the CPU architecture of this device.
```

Issue

ADB0020 means that the built Android APK did not contain a matching Android architecture for the emulator or device it was deployed to.

This message indicates that `adb` (Android Debug Bridge) reported an `INSTALL_FAILED_CPU_ABI_INCOMPATIBLE` or `INSTALL_FAILED_NO_MATCHING_ABIS` error. `adb` is part of the Android SDK and is used internally by .NET for Android to communicate with Android emulators and devices. Learn more about `adb` from the [Android documentation](#).

Solution

A solution is to add an additional architecture under the **Supported architectures** in your project options.

You may also modify the MSBuild property, as in the following example that includes all ABIs:

XML

```
<AndroidSupportedAbis>armeabi-v7a;x86;x86_64;arm64-v8a</AndroidSupportedAbis>
```

Implementation notes

Note that nothing in the open source <https://github.com/xamarin/xamarin-android> repository emits ADB0020, as features such as debugging and "fast deployment" are

implemented in the proprietary .NET for Android additions.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error ADB0030

Article • 04/17/2024

Example messages

```
error ADB0030: Failure [INSTALL_PARSE_FAILED_INCONSISTENT_CERTIFICATES]
```

```
error ADB0030: The installed package is incompatible. Please manually  
uninstall and try again.
```

Issue

ADB0030 means that you must manually uninstall your APK before you can deploy your .NET for Android application to the attached device or emulator. This situation can happen if you had deployed your .NET for Android application in the past, but it was signed with a different Android keystore file.

This message indicates that `adb` (Android Debug Bridge) reported an `INSTALL_PARSE_FAILED_INCONSISTENT_CERTIFICATES`, `INSTALL_FAILED_UPDATE_INCOMPATIBLE`, or `INSTALL_FAILED_VERSION_DOWNGRADE` error. `adb` is part of the Android SDK and is used internally by .NET for Android to communicate with Android emulators and devices. Learn more about `adb` from the [Android documentation](#).

Solution

Manually uninstall your APK from the attached device or emulator.

Implementation notes

Note that nothing in the open source <https://github.com/xamarin/xamarin-android> repository emits ADB0030, as features such as debugging and "fast deployment" are implemented in the proprietary .NET for Android additions.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error ADB0040

Article • 04/17/2024

Example messages

```
error ADB0040: The device does not support the minimum SDK level specified
in the manifest.
```

Issue

ADB0040 means that you are trying to deploy to an emulator or device that has an older Android version than what your .NET for Android application supports.

This message indicates that `adb` (Android Debug Bridge) reported an `INSTALL_FAILED_OLDER_SDK` error. `adb` is part of the Android SDK and is used internally by .NET for Android to communicate with Android emulators and devices. Learn more about `adb` from the [Android documentation](#).

Solution

Verify you are setting the appropriate values for `uses-sdk` in your *AndroidManifest.xml*:

XML

```
<uses-sdk android:minSdkVersion="15" android:targetSdkVersion="27"/>
```

Your attached device must at least be able to support `minSdkVersion`.

Implementation notes

Note that nothing in the open source <https://github.com/xamarin/xamarin-android> repository emits ADB0040, as features such as debugging and "fast deployment" are implemented in the proprietary .NET for Android additions.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error ADB0050

Article • 04/17/2024

Example messages

```
error ADB0050: Package {packageName} already exists on device.
```

Issue

ADB0050 is likely an error in the .NET for Android build chain, since it means there was an absence of the `-r` flag during the `adb install` command.

This message indicates that `adb` (Android Debug Bridge) reported an `INSTALL_FAILED_ALREADY_EXISTS` error. `adb` is part of the Android SDK and is used internally by .NET for Android to communicate with Android emulators and devices. Learn more about `adb` from the [Android documentation](#).

Solution

Consider submitting a [bug](#) if you are getting this warning under normal circumstances.

Implementation notes

Note that nothing in the open source <https://github.com/xamarin/xamarin-android> repository emits ADB0050, as features such as debugging and "fast deployment" are implemented in the proprietary .NET for Android additions.

Feedback

Was this page helpful?

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error ADB0060

Article • 04/17/2024

Example messages

```
error ADB0060: There is not enough storage space on the device to store
package: {packageName}. Free up some space and try again.
```

```
error ADB0060: There is not enough storage space on the device to store
package: {packageName}. Free up some space or use an SD card and try again.
```

Issue

ADB0060 means that the internal or external disk space is full on your Android emulator or device.

This message indicates that `adb` (Android Debug Bridge) reported an `INSTALL_FAILED_INSUFFICIENT_STORAGE` or `INSTALL_FAILED_MEDIA_UNAVAILABLE` error. `adb` is part of the Android SDK and is used internally by .NET for Android to communicate with Android emulators and devices. Learn more about `adb` from the [Android documentation](#).

Solutions

Consider uninstalling applications or adding an SD card for additional storage.

Implementation notes

Note that nothing in the open source <https://github.com/xamarin/xamarin-android> repository emits ADB0060, as features such as debugging and "fast deployment" are implemented in the proprietary .NET for Android additions.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android errors and warnings reference

Article • 04/17/2024

ADBxxxx: ADB tooling

- [ADB0000](#): Generic `adb` error/warning.
- [ADB0010](#): Generic `adb` APK installation error/warning.
- [ADB0020](#): The package does not support the CPU architecture of this device.
- [ADB0030](#): APK installation failed due to a conflict with the existing package.
- [ADB0040](#): The device does not support the minimum SDK level specified in the manifest.
- [ADB0050](#): Package {packageName} already exists on device.
- [ADB0060](#): There is not enough storage space on the device to store package: {packageName}. Free up some space and try again.

ANDXXxxxx: Generic Android tooling

- [ANDAS0000](#): Generic `apksigner` error/warning.
- [ANDJS0000](#): Generic `jarsigner` error/warning.
- [ANDKT0000](#): Generic `keytool` error/warning.
- [ANDZA0000](#): Generic `zipalign` error/warning.

APTxxxx: AAPT tooling

- [APT0000](#): Generic `aapt` error/warning.
- [APT0001](#): Unknown option `{option}`. Please check ``$(AndroidAapt2CompileExtraArgs)`` and ``$(AndroidAapt2LinkExtraArgs)`` to see if they include any ``aapt`` command line arguments that are no longer valid for ``aapt2`` and ensure that all other arguments are valid for `aapt2`.
- [APT0002](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0003](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0004](#): Invalid file name: It must start with either A-Z or a-z or an underscore.
- [APT2264](#): The system cannot find the file specified. (2).
- [APT2265](#): The system cannot find the file specified. (2).

JAVAxxxx: Java Tool

- [JAVA0000](#): Generic Java tool error

JAVACxxxx: Java compiler

- [JAVAC0000](#): Generic Java compiler error

XA0xxx: Environment issue or missing tooling

- [XA0000](#): Could not determine `$(AndroidApiLevel)` or `$(TargetFrameworkVersion)`.
- [XA0001](#): Invalid or unsupported `$(TargetFrameworkVersion)` value.
- [XA0002](#): Could not find mono.android.jar
- [XA0003](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. It must be an integer value.
- [XA0004](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. The value must be in the range of 0 to `{maxVersionCode}`.
- [XA0030](#): Building with JDK version `{versionNumber}` is not supported.
- [XA0031](#): Java SDK `{requiredJavaForFrameworkVersion}` or above is required when using `$(TargetFrameworkVersion)` `{targetFrameworkVersion}`.
- [XA0032](#): Java SDK `{requiredJavaForBuildTools}` or above is required when using Android SDK Build-Tools `{buildToolsVersion}`.
- [XA0033](#): Failed to get the Java SDK version because the returned value does not appear to contain a valid version number.
- [XA0034](#): Failed to get the Java SDK version.
- [XA0035](#): Failed to determine the Android ABI for the project.
- [XA0036](#): `$(AndroidSupportedAbis)` is not supported in .NET 6 and higher.
- [XA0100](#): `EmbeddedNativeLibrary` is invalid in Android Application projects. Please use `AndroidNativeLibrary` instead.
- [XA0101](#): warning XA0101: `@(Content)` build action is not supported.
- [XA0102](#): Generic `lint` Warning.
- [XA0103](#): Generic `lint` Error.
- [XA0104](#): Invalid value for ``$(AndroidSequencePointsMode)``
- [XA0105](#): The `$(TargetFrameworkVersion)` for a library is greater than the `$(TargetFrameworkVersion)` for the application project.
- [XA0107](#): `{Assmebly}` is a Reference Assembly.
- [XA0108](#): Could not get version from `lint`.
- [XA0109](#): Unsupported or invalid `$(TargetFrameworkVersion)` value of 'v4.5'.
- [XA0111](#): Could not get the `aapt2` version. Please check it is installed correctly.

- [XA0112](#): `aapt2` is not installed. Disabling `aapt2` support. Please check it is installed correctly.
- [XA0113](#): Google Play requires that new applications and updates must use a TargetFrameworkVersion of v11.0 (API level 30) or above.
- [XA0115](#): Invalid value 'armeabi' in \$(AndroidSupportedAbis). This ABI is no longer supported. Please update your project properties to remove the old value. If the properties page does not show an 'armeabi' checkbox, un-check and re-check one of the other ABIs and save the changes.
- [XA0116](#): Unable to find `EmbeddedResource` named `{ResourceName}`.
- [XA0117](#): The TargetFrameworkVersion {TargetFrameworkVersion} is deprecated. Please update it to be v4.4 or higher.
- [XA0118](#): Could not parse '{TargetMoniker}'
- [XA0119](#): A non-ideal configuration was found in the project.
- [XA0121](#): Assembly '{assembly}' is using '[assembly: Java.Interop.JavaLibraryReferenceAttribute]', which is no longer supported. Use a newer version of this NuGet package or notify the library author.
- [XA0122](#): Assembly '{assembly}' is using a deprecated attribute '[assembly: Java.Interop.DoNotPackageAttribute]'. Use a newer version of this NuGet package or notify the library author.
- [XA0123](#): Removing {issue} from {propertyName}. Lint {version} does not support this check.
- [XA0125](#): `{Project}` is using a deprecated debug information level. Set the debugging information to Portable in the Visual Studio project property pages or edit the project file in a text editor and set the 'DebugType' MSBuild property to 'portable' to use the newer, cross-platform debug information level. If this file comes from a NuGet package, update to a newer version of the NuGet package or notify the library author.
- [XA0126](#): Error installing FastDev Tools. This device does not support Fast Deployment. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0127](#): There was an issue deploying {destination} using {FastDevTool}. We encountered the following error {output}. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0128](#): Stdio Redirection is enabled. Please disable it to use Fast Deployment.
- [XA0129](#): Error deploying `{File}`. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0130](#): Sorry. Fast deployment is only supported on devices running Android 5.0 (API level 21) or higher. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.

- [XA0131](#): The 'run-as' tool has been disabled on this device. Either enable it by activating the developer options on the device or by setting `ro.boot.disable_runas` to `false`.
- [XA0132](#): The package was not installed. Please check you do not have it installed under any other user. If the package does show up on the device, try manually uninstalling it then try again. You should be able to uninstall the app via the Settings app on the device.
- [XA0133](#): The 'run-as' tool required by the Fast Deployment system has been disabled on this device by the manufacturer. Please disable Fast Deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0134](#): The application does not have the 'android:debuggable' attribute set in the AndroidManifest.xml. This is required in order for Fast Deployment to work. This is normally enabled by default by the .NET for Android build system for Debug builds.
- [XA0135](#): The package is a 'system' application. These are applications which install under the 'system' user on a device. These types of applications cannot use 'run-as'.
- [XA0136](#): The currently installation of the package is corrupt. Please manually uninstall the package from all the users on device and try again. If that does not work you can disable Fast Deployment.
- [XA0137](#): The 'run-as' command failed with '{0}'. Fast Deployment is not currently supported on this device. Please file an issue with the exact error message using the 'Help->Send Feedback->Report a Problem' menu item in Visual Studio or 'Help->Report a Problem' in Visual Studio for Mac.
- [XA0138](#): `%(AndroidAsset.AssetPack)` and `%(AndroidAsset.AssetPack)` item metadata are only supported when `$(AndroidApplication)` is `true`.
- [XA0139](#): `@(AndroidAsset) {0}` has invalid `DeliveryType` metadata of `{1}`. Supported values are `installtime`, `ondemand` or `fastfollow`
- [XA0140](#):
- [XA0141](#): NuGet package '{0}' version '{1}' contains a shared library '{2}' which is not correctly aligned. See <https://developer.android.com/guide/practices/page-sizes> for more details
- [XA0142](#): Command '{0}' failed.\n{1}

XA1xxx: Project related

- [XA1000](#): There was a problem parsing {file}. This is likely due to incomplete or invalid XML.

- [XA1001](#): AndroidResgen: Warning while updating Resource XML '{filename}': {Message}
- [XA1002](#): The closest match found for '{customViewName}' is '{customViewLookupName}', but the capitalization does not match. Please correct the capitalization.
- [XA1003](#): '{zip}' does not exist. Please rebuild the project.
- [XA1004](#): There was an error opening {filename}. The file is probably corrupt. Try deleting it and building again.
- [XA1005](#): Attempting basic type name matching for element with ID '{id}' and type '{managedType}'
- [XA1006](#): The TargetFrameworkVersion (Android API level {compileSdk}) is higher than the targetSdkVersion ({targetSdk}).
- [XA1007](#): The minSdkVersion ({minSdk}) is greater than targetSdkVersion. Please change the value such that minSdkVersion is less than or equal to targetSdkVersion ({targetSdk}).
- [XA1008](#): The TargetFrameworkVersion (Android API level {compileSdk}) is lower than the targetSdkVersion ({targetSdk}).
- [XA1009](#): The {assembly} is Obsolete. Please upgrade to {assembly} {version}
- [XA1010](#): Invalid `\$(AndroidManifestPlaceholders)` value for Android manifest placeholders. Please use `key1=value1;key2=value2` format. The specified value was: {placeholders}
- [XA1011](#): Using ProGuard with the D8 DEX compiler is no longer supported. Please set the code shrinker to 'r8' in the Visual Studio project property pages or edit the project file in a text editor and set the 'AndroidLinkTool' MSBuild property to 'r8'.
- XA1012: Included layout root element override ID '{id}' is not valid.
- XA1013: Failed to parse ID of node '{name}' in the layout file '{file}'.
- XA1014: JAR library references with identical file names but different contents were found: {libraries}. Please remove any conflicting libraries from EmbeddedJar, InputJar and AndroidJavaLibrary.
- XA1015: More than one Android Wear project is specified as the paired project. It can be at most one.
- XA1016: Target Wear application's project '{project}' does not specify required 'AndroidManifest' project property.
- XA1017: Target Wear application's AndroidManifest.xml does not specify required 'package' attribute.
- XA1018: Specified AndroidManifest file does not exist: {file}.
- XA1019: `LibraryProjectProperties` file `{file}` is located in a parent directory of the bindings project's intermediate output directory. Please adjust the path to use the original `project.properties` file directly from the Android library project directory.

- XA1020: At least one Java library is required for binding. Check that a Java library is included in the project and has the appropriate build action: 'LibraryProjectZip' (for AAR or ZIP), 'EmbeddedJar', 'InputJar' (for JAR), or 'LibraryProjectProperties' (project.properties).
- XA1021: Specified source Java library not found: {file}
- XA1022: Specified reference Java library not found: {file}
- [XA1023](#): Using the DX DEX Compiler is deprecated.
- [XA1024](#): Ignoring configuration file 'Foo.dll.config'. .NET configuration files are not supported in .NET for Android projects that target .NET 6 or higher.
- [XA1025](#): The experimental 'Hybrid' value for the 'AndroidAotMode' MSBuild property is not currently compatible with the armeabi-v7a target ABI.
- [XA1027](#): The 'EnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1028](#): The 'AndroidEnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1029](#): The 'AotAssemblies' MSBuild property is deprecated. Edit the project file in a text editor to remove this property, and use the 'RunAOTCompilation' MSBuild property instead.
- [XA1031](#): The 'AndroidHttpClientHandlerType' has an invalid value.
- [XA1032](#): Failed to resolve '{0}' from '{1}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1033](#): Could not resolve '{0}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1035](#): The 'BundleAssemblies' property is deprecated and it has no effect on the application build. Equivalent functionality is implemented by the 'AndroidUseAssemblyStore' and 'AndroidEnableAssemblyCompression' properties.
- [XA1036](#): AndroidManifest.xml //uses-sdk/@android:minSdkVersion '29' does not match the \$(SupportedOSPlatformVersion) value '21' in the project file (if there is no \$(SupportedOSPlatformVersion) value in the project file, then a default value has been assumed). Either change the value in the AndroidManifest.xml to match the \$(SupportedOSPlatformVersion) value, or remove the value in the AndroidManifest.xml (and add a \$(SupportedOSPlatformVersion) value to the project file if it doesn't already exist).
- [XA1037](#): The '{0}' MSBuild property is deprecated and will be removed in .NET {1}. See <https://aka.ms/net-android-deprecations> for more details.
- [XA1038](#): The '{0}' MSBuild property has an invalid value. Value values are {1}.

XA2xxx: Linker

- [XA2000](#): Use of AppDomain.CreateDomain() detected in assembly: {assembly}. .NET 6 will only support a single AppDomain, so this API will no longer be available in .NET for Android once .NET 6 is released.
- [XA2001](#): Source file '{filename}' could not be found.
- [XA2002](#): Can not resolve reference: `{missing}`, referenced by {assembly}. Perhaps it doesn't exist in the Mono for Android profile?
- XA2006: Could not resolve reference to '{member}' (defined in assembly '{assembly}') with scope '{scope}'. When the scope is different from the defining assembly, it usually means that the type is forwarded.
- XA2007: Exception while loading assemblies: {exception}
- XA2008: In referenced assembly {assembly}, Java.Interop.DoNotPackageAttribute requires non-null file name.

XA3xxx: Unmanaged code compilation

- XA3001: Could not AOT the assembly: {assembly}
- XA3002: Invalid AOT mode: {mode}
- XA3003: Could not strip IL of assembly: {assembly}
- XA3004: Android NDK r10d is buggy and provides an incompatible x86_64 libm.so.
- XA3005: The detected Android NDK version is incompatible with the targeted LLVM configuration.
- XA3006: Could not compile native assembly file: {file}
- XA3007: Could not link native shared library: {library}

XA4xxx: Code generation

- XA4209: Failed to generate Java type for class: {managedType} due to {exception}
- XA4210: Please add a reference to Mono.Android.Export.dll when using ExportAttribute or ExportFieldAttribute.
- XA4211: AndroidManifest.xml //uses-sdk/@android:targetSdkVersion '{targetSdk}' is less than \$(TargetFrameworkVersion) '{targetFramework}'. Using API-{targetFrameworkApi} for ACW compilation.
- XA4213: The type '{type}' must provide a public default constructor
- [XA4214](#): The managed type `Library1.Class1` exists in multiple assemblies: Library1, Library2. Please refactor the managed type names in these assemblies so that they are not identical.
- [XA4215](#): The Java type `com.contoso.library1.Class1` is generated by more than one managed type. Please change the [Register] attribute so that the same Java type is not emitted.

- [XA4216](#): The deployment target '19' is not supported (the minimum is '21'). Please increase the \$(SupportedOSPlatformVersion) property value in your project file.
- XA4217: Cannot override Kotlin-generated method '{method}' because it is not a valid Java method name. This method can only be overridden from Kotlin.
- [XA4218](#): Unable to find //manifest/application/uses-library at path: {path}
- XA4219: Cannot find binding generator for language {language} or {defaultLanguage}.
- XA4220: Partial class item '{file}' does not have an associated binding for layout '{layout}'.
- XA4221: No layout binding source files were generated.
- XA4222: No widgets found for layout ({layoutFiles}).
- XA4223: Malformed full class name '{name}'. Missing namespace.
- XA4224: Malformed full class name '{name}'. Missing class name.
- XA4225: Widget '{widget}' in layout '{layout}' has multiple instances with different types. The property type will be set to: {type}
- XA4226: Resource item '{file}' does not have the required metadata item '{metadataName}'.
- XA4228: Unable to find specified //activity-alias/@android:targetActivity: '{targetActivity}'
- XA4229: Unrecognized `TransformFile` root element: {element}.
- XA4230: Error parsing XML: {exception}
- [XA4231](#): The Android class parser value 'jar2xml' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'class-parse'.
- [XA4232](#): The Android code generation target 'XamarinAndroid' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'XJavaInterop1'.
- [XA4234](#): '<{item}>' item '{itemspec}' is missing required attribute '{name}'.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id'.
- [XA4236](#): Cannot download Maven artifact '{group}:{artifact}'. - {jar}: {exception} - {aar}: {exception}
- [XA4237](#): Cannot download POM file for Maven artifact '{artifact}'. - {exception}
- [XA4239](#): Unknown Maven repository: '{repository}'.
- [XA4241](#): Java dependency '{artifact}' is not satisfied.
- [XA4242](#): Java dependency '{artifact}' is not satisfied. Microsoft maintains the NuGet package '{nugetId}' that could fulfill this dependency.
- [XA4243](#): Attribute '{name}' is required when using '{name}' for '{element}' item '{itemspec}'.
- [XA4244](#): Attribute '{name}' cannot be empty for '{element}' item '{itemspec}'.
- [XA4245](#): Specified POM file '{file}' does not exist.

- [XA4246](#): Could not parse POM file '{file}'. - {exception}
- [XA4247](#): Could not resolve POM file for artifact '{artifact}'.
- [XA4248](#): Could not find NuGet package '{nugetId}' version '{version}' in lock file. Ensure NuGet Restore has run since this `<PackageReference>` was added.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id:version'.
- XA4300: Native library '{library}' will not be bundled because it has an unsupported ABI.
- [XA4301](#): Apk already contains the item `xxx`.
- [XA4302](#): Unhandled exception merging `AndroidManifest.xml`: {ex}
- [XA4303](#): Error extracting resources from "{assemblyPath}": {ex}
- [XA4304](#): ProGuard configuration file '{file}' was not found.
- [XA4305](#): Multidex is enabled, but `\$\_AndroidMainDexListFile` is empty.
- [XA4306](#): R8 does not support `@(MultiDexMainDexList)` files when `android:minSdkVersion >= 21`
- [XA4307](#): Invalid ProGuard configuration file.
- [XA4308](#): Failed to generate type maps
- [XA4309](#): 'MultiDexMainDexList' file '{file}' does not exist.
- [XA4310](#): `\$(AndroidSigningKeyStore)` file '{keystore}' could not be found.
- XA4311: The application won't contain the paired Wear package because the Wear application package APK is not created yet. If building on the command line, be sure to build the "SignAndroidPackage" target.
- [XA4312](#): Referencing an Android Wear application project from an Android application project is deprecated.
- [XA4313](#): Framework assembly has been deprecated.
- [XA4314](#): `$(Property)` is empty. A value for `$(Property)` should be provided.
- [XA4315](#): Ignoring {file}. Manifest does not have the required 'package' attribute on the manifest element.

XA5xxx: GCC and toolchain

- XA5101: Missing Android NDK toolchains directory '{path}'. Please install the Android NDK.
- XA5102: Conversion from assembly to native code failed. Exit code {exitCode}
- XA5103: NDK C compiler exited with an error. Exit code {0}
- XA5104: Could not locate the Android NDK.
- XA5105: Toolchain utility '{utility}' for target {arch} was not found. Tried in path: "{path}"
- XA5201: NDK linker exited with an error. Exit code {0}
- [XA5205](#): Cannot find `{ToolName}` in the Android SDK.

- [XA5207](#): Could not find android.jar for API level `{compileSdk}`.
- [XA5211](#): Embedded Wear app package name differs from handheld app package name (`{wearPackageName} != {handheldPackageName}`).
- [XA5213](#): `java.lang.OutOfMemoryError`. Consider increasing the value of `$(JavaMaximumHeapSize)`. Java ran out of memory while executing '`{tool}` `{arguments}`'
- [XA5300](#): The Android/Java SDK Directory could not be found.
- [XA5301](#): Failed to generate Java type for class: `{managedType}` due to `MAX_PATH`: `{exception}`
- [XA5302](#): Two processes may be building this project at once. Lock file exists at path: `{path}`

XA6xxx: Internal tools

XAccc7xxx: Unhandled MSBuild Exceptions

Exceptions that have not been gracefully handled yet. Ideally these will be fixed or replaced with better errors in the future.

These take the form of `XACCC7NNN`, where `CCC` is a 3 character code denoting the MSBuild Task that is throwing the exception, and `NNN` is a 3 digit number indicating the type of the unhandled `Exception`.

Tasks:

- `A2C` - `Aapt2Compile`
- `A2L` - `Aapt2Link`
- `AAS` - `AndroidApkSigner`
- `ACD` - `AndroidCreateDebugKey`
- `ACM` - `AppendCustomMetadataToItemGroup`
- `ADB` - `Adb`
- `AJV` - `AdjustJavacVersionArguments`
- `AOT` - `Aot`
- `APT` - `Aapt`
- `ASP` - `AndroidSignPackage`
- `AZA` - `AndroidZipAlign`
- `BAB` - `BuildAppBundle`
- `BAS` - `BuildApkSet`
- `BBA` - `BuildBaseAppBundle`

- BGN - BindingsGenerator
- BLD - BuildApk
- CAL - CreateAdditionalLibraryResourceCache
- CAR - CalculateAdditionalResourceCacheDirectories
- CCR - CopyAndConvertResources
- CCV - ConvertCustomView
- CDF - ConvertDebuggingFiles
- CDJ - CheckDuplicateJavaLibraries
- CFI - CheckForInvalidResourceFileNames
- CFR - CheckForRemovedItems
- CGJ - CopyGeneratedJavaResourceClasses
- CGS - CheckGoogleSdkRequirements
- CIC - CopyIfChanged
- CIL - CilStrip
- CLA - CollectLibraryAssets
- CLC - CalculateLayoutCodeBehind
- CLP - ClassParse
- CLR - CreateLibraryResourceArchive
- CMD - CreateMultiDexMainDexClassList
- CML - CreateManagedLibraryResourceArchive
- CMM - CreateMsymManifest
- CNA - CompileNativeAssembly
- CNE - CollectNonEmptyDirectories
- CNL - CreateNativeLibraryArchive
- CPD - CalculateProjectDependencies
- CPF - CollectPdbFiles
- CPI - CheckProjectItems
- CPR - CopyResource
- CPT - ComputeHash
- CRC - ConvertResourcesCases
- CRM - CreateResgenManifest
- CRN - Crunch
- CRP - AndroidComputeResPaths
- CTD - CreateTemporaryDirectory
- CTX - CompileToDalvik
- DES - Desugar
- DJL - DetermineJavaLibrariesToCompile
- DX8 - D8

- FD - FastDeploy
- FLB - FindLayoutsToBind
- FLT - FilterAssemblies
- GAD - GetAndroidDefineConstants
- GAP - GetAndroidPackageName
- GAR - GetAdditionalResourcesFromAssemblies
- GAS - GetAppSettingsDirectory
- GCB - GenerateCodeBehindForLayout
- GCJ - GetConvertedJavaLibraries
- GEP - GetExtraPackages
- GFT - GetFilesThatExist
- GIL - GetImportedLibraries
- GJP - GetJavaPlatformJar
- GJS - GenerateJavaStubs
- GLB - GenerateLayoutBindings
- GLR - GenerateLibraryResources
- GMA - GenerateManagedAidlProxies
- GMJ - GetMonoPlatformJar
- GPM - GeneratePackageManagerJava
- GRD - GenerateResourceDesigner
- IAS - InstallApkSet
- IJD - ImportJavaDoc
- JDC - JavaDoc
- JVC - Javac
- JTX - JarToXml
- KEY - KeyTool
- LAS - LinkApplicationSharedLibraries
- LEF - LogErrorsForFiles
- LNK - LinkAssemblies
- LNS - LinkAssembliesNoShrink
- LNT - Lint
- LWF - LogWarningsForFiles
- MBN - MakeBundleNativeCodeExternal
- MDC - MDoc
- PAI - PrepareAbiItems
- PAW - ParseAndroidWearProjectAndManifest
- PRO - Proguard
- PWA - PrepareWearApplicationFiles

- R8D - R8
- RAM - ReadAndroidManifest
- RAR - ReadAdditionalResourcesFromAssemblyCache
- RAT - ResolveAndroidTooling
- RDF - RemoveDirFixed
- RIL - ReadImportedLibrariesCache
- RJJ - ResolveJdkJvmPath
- RLC - ReadLibraryProjectImportsCache
- RLP - ResolveLibraryProjectImports
- RRA - RemoveRegisterAttribute
- RSA - ResolveAssemblies
- RSD - ResolveSdks
- RUF - RemoveUnknownFiles
- SPL - SplitProperty
- SVM - SetVsMonoAndroidRegistryKey
- UNZ - Unzip
- VJV - ValidateJavaVersion
- WLF - WriteLockFile

Exceptions:

- 7000 - Other Exception
- 7001 - NullPointerException
- 7002 - ArgumentOutOfRangeException
- 7003 - ArgumentNullException
- 7004 - ArgumentException
- 7005 - FormatException
- 7006 - IndexOutOfRangeException
- 7007 - InvalidCastException
- 7008 - ObjectDisposedException
- 7009 - InvalidOperationException
- 7010 - InvalidProgramException
- 7011 - KeyNotFoundException
- 7012 - TaskCanceledException
- 7013 - OperationCanceledException
- 7014 - OutOfMemoryException
- 7015 - NotSupportedException
- 7016 - StackOverflowException

- 7017 - `TimeoutException`
- 7018 - `TypeInitializationException`
- 7019 - `UnauthorizedAccessException`
- 7020 - `ApplicationException`
- 7021 - `KeyNotFoundException`
- 7022 - `PathTooLongException`
- 7023 - `DirectoryNotFoundException`
- 7024 - `IOException`
- 7025 - `DriveNotFoundException`
- 7026 - `EndOfStreamException`
- 7027 - `FileLoadException`
- 7028 - `FileNotFoundException`
- 7029 - `PipeException`

XA8xxx: Linker Step Errors

- [XA8000/IL8000](#): Could not find Android Resource '@anim/enterfromright'. Please update `@(AndroidResource)` to add the missing resource.

XA9xxx: Licensing

Removed messages

Removed in Xamarin.Android 10.4

- XA5215: Duplicate Resource found for {elementName}. Duplicates are in {filenames}
- XA5216: Resource entry {elementName} is already defined in {filename}

Removed in Xamarin.Android 10.3

- [XA0110](#): Disabling `$(AndroidExplicitCrunch)` as it is not supported by `aapt2`. If you wish to use `$(AndroidExplicitCrunch)` please set `$(AndroidUseAapt2)` to false.

Removed in Xamarin.Android 10.2

- [XA0120](#): Failed to use SHA1 hash algorithm

Removed in Xamarin.Android 9.3

- [XA0114](#): Google Play requires that application updates must use a `$(TargetFrameworkVersion)` of v8.0 (API level 26) or above.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error/warning ANDAS0000

Article • 04/17/2024

Issue

This message indicates that the Android `apksigner` command line tool used by .NET for Android reported an error or warning.

Errors reported by `apksigner` and other Android command line tooling are outside of .NET for Android's control, so a general error code of ANDAS0000 is used reporting the exact message.

Solution

To learn more about `apksigner` and its usage, see the Android documentation [here](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error/warning ANDJS0000

Article • 04/17/2024

Issue

This message indicates that the Java `jarsigner` command line tool used by .NET for Android reported an error or warning.

Errors reported by `jarsigner` and other Android command line tooling are outside of .NET for Android's control, so a general error code of ANDJS0000 is used reporting the exact message.

Solution

To learn more about `jarsigner` and its usage, see the Java documentation [here](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error/warning ANDKT0000

Article • 04/17/2024

Issue

This message indicates that the Java `keytool` command line tool used by .NET for Android reported an error or warning.

Errors reported by `keytool` and other Android command line tooling are outside of .NET for Android's control, so a general error code of ANDKT0000 is used reporting the exact message.

Solution

To learn more about `keytool` and its usage, see the Java documentation [here](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error/warning ANDZA0000

Article • 04/17/2024

Issue

This message indicates that the Android `zipalign` command line tool used by .NET for Android reported an error or warning.

Errors reported by `zipalign` and other Android command line tooling are outside of .NET for Android's control, so a general error code of ANDZA0000 is used reporting the exact message.

Solution

To learn more about `zipalign` and its usage, see the Android documentation [here](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android errors and warnings reference

Article • 04/17/2024

ADBxxxx: ADB tooling

- [ADB0000](#): Generic `adb` error/warning.
- [ADB0010](#): Generic `adb` APK installation error/warning.
- [ADB0020](#): The package does not support the CPU architecture of this device.
- [ADB0030](#): APK installation failed due to a conflict with the existing package.
- [ADB0040](#): The device does not support the minimum SDK level specified in the manifest.
- [ADB0050](#): Package {packageName} already exists on device.
- [ADB0060](#): There is not enough storage space on the device to store package: {packageName}. Free up some space and try again.

ANDXXxxxx: Generic Android tooling

- [ANDAS0000](#): Generic `apksigner` error/warning.
- [ANDJS0000](#): Generic `jarsigner` error/warning.
- [ANDKT0000](#): Generic `keytool` error/warning.
- [ANDZA0000](#): Generic `zipalign` error/warning.

APTxxxx: AAPT tooling

- [APT0000](#): Generic `aapt` error/warning.
- [APT0001](#): Unknown option `{option}`. Please check ``$(AndroidAapt2CompileExtraArgs)`` and ``$(AndroidAapt2LinkExtraArgs)`` to see if they include any ``aapt`` command line arguments that are no longer valid for ``aapt2`` and ensure that all other arguments are valid for `aapt2`.
- [APT0002](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0003](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0004](#): Invalid file name: It must start with either A-Z or a-z or an underscore.
- [APT2264](#): The system cannot find the file specified. (2).
- [APT2265](#): The system cannot find the file specified. (2).

JAVAXxxx: Java Tool

- [JAVA0000](#): Generic Java tool error

JAVACxxxx: Java compiler

- [JAVAC0000](#): Generic Java compiler error

XA0xxx: Environment issue or missing tooling

- [XA0000](#): Could not determine `$(AndroidApiLevel)` or `$(TargetFrameworkVersion)`.
- [XA0001](#): Invalid or unsupported `$(TargetFrameworkVersion)` value.
- [XA0002](#): Could not find mono.android.jar
- [XA0003](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. It must be an integer value.
- [XA0004](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. The value must be in the range of 0 to `{maxVersionCode}`.
- [XA0030](#): Building with JDK version `{versionNumber}` is not supported.
- [XA0031](#): Java SDK `{requiredJavaForFrameworkVersion}` or above is required when using `$(TargetFrameworkVersion)` `{targetFrameworkVersion}`.
- [XA0032](#): Java SDK `{requiredJavaForBuildTools}` or above is required when using Android SDK Build-Tools `{buildToolsVersion}`.
- [XA0033](#): Failed to get the Java SDK version because the returned value does not appear to contain a valid version number.
- [XA0034](#): Failed to get the Java SDK version.
- [XA0035](#): Failed to determine the Android ABI for the project.
- [XA0036](#): `$(AndroidSupportedAbis)` is not supported in .NET 6 and higher.
- [XA0100](#): `EmbeddedNativeLibrary` is invalid in Android Application projects. Please use `AndroidNativeLibrary` instead.
- [XA0101](#): warning XA0101: `@(Content)` build action is not supported.
- [XA0102](#): Generic `lint` Warning.
- [XA0103](#): Generic `lint` Error.
- [XA0104](#): Invalid value for ``$(AndroidSequencePointsMode)``
- [XA0105](#): The `$(TargetFrameworkVersion)` for a library is greater than the `$(TargetFrameworkVersion)` for the application project.
- [XA0107](#): `{Assmebly}` is a Reference Assembly.
- [XA0108](#): Could not get version from `lint`.
- [XA0109](#): Unsupported or invalid `$(TargetFrameworkVersion)` value of `'v4.5'`.
- [XA0111](#): Could not get the `aapt2` version. Please check it is installed correctly.

- [XA0112](#): `aapt2` is not installed. Disabling `aapt2` support. Please check it is installed correctly.
- [XA0113](#): Google Play requires that new applications and updates must use a TargetFrameworkVersion of v11.0 (API level 30) or above.
- [XA0115](#): Invalid value 'armeabi' in \$(AndroidSupportedAbis). This ABI is no longer supported. Please update your project properties to remove the old value. If the properties page does not show an 'armeabi' checkbox, un-check and re-check one of the other ABIs and save the changes.
- [XA0116](#): Unable to find `EmbeddedResource` named `{ResourceName}`.
- [XA0117](#): The TargetFrameworkVersion {TargetFrameworkVersion} is deprecated. Please update it to be v4.4 or higher.
- [XA0118](#): Could not parse '{TargetMoniker}'
- [XA0119](#): A non-ideal configuration was found in the project.
- [XA0121](#): Assembly '{assembly}' is using '[assembly: Java.Interop.JavaLibraryReferenceAttribute]', which is no longer supported. Use a newer version of this NuGet package or notify the library author.
- [XA0122](#): Assembly '{assembly}' is using a deprecated attribute '[assembly: Java.Interop.DoNotPackageAttribute]'. Use a newer version of this NuGet package or notify the library author.
- [XA0123](#): Removing {issue} from {propertyName}. Lint {version} does not support this check.
- [XA0125](#): `{Project}` is using a deprecated debug information level. Set the debugging information to Portable in the Visual Studio project property pages or edit the project file in a text editor and set the 'DebugType' MSBuild property to 'portable' to use the newer, cross-platform debug information level. If this file comes from a NuGet package, update to a newer version of the NuGet package or notify the library author.
- [XA0126](#): Error installing FastDev Tools. This device does not support Fast Deployment. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0127](#): There was an issue deploying {destination} using {FastDevTool}. We encountered the following error {output}. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0128](#): Stdio Redirection is enabled. Please disable it to use Fast Deployment.
- [XA0129](#): Error deploying `{File}`. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0130](#): Sorry. Fast deployment is only supported on devices running Android 5.0 (API level 21) or higher. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.

- [XA0131](#): The 'run-as' tool has been disabled on this device. Either enable it by activating the developer options on the device or by setting `ro.boot.disable_runas` to `false`.
- [XA0132](#): The package was not installed. Please check you do not have it installed under any other user. If the package does show up on the device, try manually uninstalling it then try again. You should be able to uninstall the app via the Settings app on the device.
- [XA0133](#): The 'run-as' tool required by the Fast Deployment system has been disabled on this device by the manufacturer. Please disable Fast Deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0134](#): The application does not have the 'android:debuggable' attribute set in the AndroidManifest.xml. This is required in order for Fast Deployment to work. This is normally enabled by default by the .NET for Android build system for Debug builds.
- [XA0135](#): The package is a 'system' application. These are applications which install under the 'system' user on a device. These types of applications cannot use 'run-as'.
- [XA0136](#): The currently installation of the package is corrupt. Please manually uninstall the package from all the users on device and try again. If that does not work you can disable Fast Deployment.
- [XA0137](#): The 'run-as' command failed with '{0}'. Fast Deployment is not currently supported on this device. Please file an issue with the exact error message using the 'Help->Send Feedback->Report a Problem' menu item in Visual Studio or 'Help->Report a Problem' in Visual Studio for Mac.
- [XA0138](#): `%(AndroidAsset.AssetPack)` and `%(AndroidAsset.AssetPack)` item metadata are only supported when `$(AndroidApplication)` is `true`.
- [XA0139](#): `@(AndroidAsset) {0}` has invalid `DeliveryType` metadata of `{1}`. Supported values are `installtime`, `ondemand` or `fastfollow`
- [XA0140](#):
- [XA0141](#): NuGet package '{0}' version '{1}' contains a shared library '{2}' which is not correctly aligned. See <https://developer.android.com/guide/practices/page-sizes> for more details
- [XA0142](#): Command '{0}' failed.\n{1}

XA1xxx: Project related

- [XA1000](#): There was a problem parsing {file}. This is likely due to incomplete or invalid XML.

- [XA1001](#): AndroidResgen: Warning while updating Resource XML '{filename}': {Message}
- [XA1002](#): The closest match found for '{customViewName}' is '{customViewLookupName}', but the capitalization does not match. Please correct the capitalization.
- [XA1003](#): '{zip}' does not exist. Please rebuild the project.
- [XA1004](#): There was an error opening {filename}. The file is probably corrupt. Try deleting it and building again.
- [XA1005](#): Attempting basic type name matching for element with ID '{id}' and type '{managedType}'
- [XA1006](#): The TargetFrameworkVersion (Android API level {compileSdk}) is higher than the targetSdkVersion ({targetSdk}).
- [XA1007](#): The minSdkVersion ({minSdk}) is greater than targetSdkVersion. Please change the value such that minSdkVersion is less than or equal to targetSdkVersion ({targetSdk}).
- [XA1008](#): The TargetFrameworkVersion (Android API level {compileSdk}) is lower than the targetSdkVersion ({targetSdk}).
- [XA1009](#): The {assembly} is Obsolete. Please upgrade to {assembly} {version}
- [XA1010](#): Invalid `\$(AndroidManifestPlaceholders)` value for Android manifest placeholders. Please use `key1=value1;key2=value2` format. The specified value was: {placeholders}
- [XA1011](#): Using ProGuard with the D8 DEX compiler is no longer supported. Please set the code shrinker to 'r8' in the Visual Studio project property pages or edit the project file in a text editor and set the 'AndroidLinkTool' MSBuild property to 'r8'.
- XA1012: Included layout root element override ID '{id}' is not valid.
- XA1013: Failed to parse ID of node '{name}' in the layout file '{file}'.
- XA1014: JAR library references with identical file names but different contents were found: {libraries}. Please remove any conflicting libraries from EmbeddedJar, InputJar and AndroidJavaLibrary.
- XA1015: More than one Android Wear project is specified as the paired project. It can be at most one.
- XA1016: Target Wear application's project '{project}' does not specify required 'AndroidManifest' project property.
- XA1017: Target Wear application's AndroidManifest.xml does not specify required 'package' attribute.
- XA1018: Specified AndroidManifest file does not exist: {file}.
- XA1019: `LibraryProjectProperties` file `{file}` is located in a parent directory of the bindings project's intermediate output directory. Please adjust the path to use the original `project.properties` file directly from the Android library project directory.

- XA1020: At least one Java library is required for binding. Check that a Java library is included in the project and has the appropriate build action: 'LibraryProjectZip' (for AAR or ZIP), 'EmbeddedJar', 'InputJar' (for JAR), or 'LibraryProjectProperties' (project.properties).
- XA1021: Specified source Java library not found: {file}
- XA1022: Specified reference Java library not found: {file}
- [XA1023](#): Using the DX DEX Compiler is deprecated.
- [XA1024](#): Ignoring configuration file 'Foo.dll.config'. .NET configuration files are not supported in .NET for Android projects that target .NET 6 or higher.
- [XA1025](#): The experimental 'Hybrid' value for the 'AndroidAotMode' MSBuild property is not currently compatible with the armeabi-v7a target ABI.
- [XA1027](#): The 'EnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1028](#): The 'AndroidEnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1029](#): The 'AotAssemblies' MSBuild property is deprecated. Edit the project file in a text editor to remove this property, and use the 'RunAOTCompilation' MSBuild property instead.
- [XA1031](#): The 'AndroidHttpClientHandlerType' has an invalid value.
- [XA1032](#): Failed to resolve '{0}' from '{1}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1033](#): Could not resolve '{0}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1035](#): The 'BundleAssemblies' property is deprecated and it has no effect on the application build. Equivalent functionality is implemented by the 'AndroidUseAssemblyStore' and 'AndroidEnableAssemblyCompression' properties.
- [XA1036](#): AndroidManifest.xml //uses-sdk/@android:minSdkVersion '29' does not match the \$(SupportedOSPlatformVersion) value '21' in the project file (if there is no \$(SupportedOSPlatformVersion) value in the project file, then a default value has been assumed). Either change the value in the AndroidManifest.xml to match the \$(SupportedOSPlatformVersion) value, or remove the value in the AndroidManifest.xml (and add a \$(SupportedOSPlatformVersion) value to the project file if it doesn't already exist).
- [XA1037](#): The '{0}' MSBuild property is deprecated and will be removed in .NET {1}. See <https://aka.ms/net-android-deprecations> for more details.
- [XA1038](#): The '{0}' MSBuild property has an invalid value. Value values are {1}.

XA2xxx: Linker

- [XA2000](#): Use of AppDomain.CreateDomain() detected in assembly: {assembly}. .NET 6 will only support a single AppDomain, so this API will no longer be available in .NET for Android once .NET 6 is released.
- [XA2001](#): Source file '{filename}' could not be found.
- [XA2002](#): Can not resolve reference: `{missing}`, referenced by {assembly}. Perhaps it doesn't exist in the Mono for Android profile?
- XA2006: Could not resolve reference to '{member}' (defined in assembly '{assembly}') with scope '{scope}'. When the scope is different from the defining assembly, it usually means that the type is forwarded.
- XA2007: Exception while loading assemblies: {exception}
- XA2008: In referenced assembly {assembly}, Java.Interop.DoNotPackageAttribute requires non-null file name.

XA3xxx: Unmanaged code compilation

- XA3001: Could not AOT the assembly: {assembly}
- XA3002: Invalid AOT mode: {mode}
- XA3003: Could not strip IL of assembly: {assembly}
- XA3004: Android NDK r10d is buggy and provides an incompatible x86_64 libm.so.
- XA3005: The detected Android NDK version is incompatible with the targeted LLVM configuration.
- XA3006: Could not compile native assembly file: {file}
- XA3007: Could not link native shared library: {library}

XA4xxx: Code generation

- XA4209: Failed to generate Java type for class: {managedType} due to {exception}
- XA4210: Please add a reference to Mono.Android.Export.dll when using ExportAttribute or ExportFieldAttribute.
- XA4211: AndroidManifest.xml //uses-sdk/@android:targetSdkVersion '{targetSdk}' is less than \$(TargetFrameworkVersion) '{targetFramework}'. Using API-{targetFrameworkApi} for ACW compilation.
- XA4213: The type '{type}' must provide a public default constructor
- [XA4214](#): The managed type `Library1.Class1` exists in multiple assemblies: Library1, Library2. Please refactor the managed type names in these assemblies so that they are not identical.
- [XA4215](#): The Java type `com.contoso.library1.Class1` is generated by more than one managed type. Please change the [Register] attribute so that the same Java type is not emitted.

- [XA4216](#): The deployment target '19' is not supported (the minimum is '21'). Please increase the \$(SupportedOSPlatformVersion) property value in your project file.
- XA4217: Cannot override Kotlin-generated method '{method}' because it is not a valid Java method name. This method can only be overridden from Kotlin.
- [XA4218](#): Unable to find //manifest/application/uses-library at path: {path}
- XA4219: Cannot find binding generator for language {language} or {defaultLanguage}.
- XA4220: Partial class item '{file}' does not have an associated binding for layout '{layout}'.
- XA4221: No layout binding source files were generated.
- XA4222: No widgets found for layout ({layoutFiles}).
- XA4223: Malformed full class name '{name}'. Missing namespace.
- XA4224: Malformed full class name '{name}'. Missing class name.
- XA4225: Widget '{widget}' in layout '{layout}' has multiple instances with different types. The property type will be set to: {type}
- XA4226: Resource item '{file}' does not have the required metadata item '{metadataName}'.
- XA4228: Unable to find specified //activity-alias/@android:targetActivity: '{targetActivity}'
- XA4229: Unrecognized `TransformFile` root element: {element}.
- XA4230: Error parsing XML: {exception}
- [XA4231](#): The Android class parser value 'jar2xml' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'class-parse'.
- [XA4232](#): The Android code generation target 'XamarinAndroid' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'XJavaInterop1'.
- [XA4234](#): '<{item}>' item '{itemspec}' is missing required attribute '{name}'.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id'.
- [XA4236](#): Cannot download Maven artifact '{group}:{artifact}'. - {jar}: {exception} - {aar}: {exception}
- [XA4237](#): Cannot download POM file for Maven artifact '{artifact}'. - {exception}
- [XA4239](#): Unknown Maven repository: '{repository}'.
- [XA4241](#): Java dependency '{artifact}' is not satisfied.
- [XA4242](#): Java dependency '{artifact}' is not satisfied. Microsoft maintains the NuGet package '{nugetId}' that could fulfill this dependency.
- [XA4243](#): Attribute '{name}' is required when using '{name}' for '{element}' item '{itemspec}'.
- [XA4244](#): Attribute '{name}' cannot be empty for '{element}' item '{itemspec}'.
- [XA4245](#): Specified POM file '{file}' does not exist.

- [XA4246](#): Could not parse POM file '{file}'. - {exception}
- [XA4247](#): Could not resolve POM file for artifact '{artifact}'.
- [XA4248](#): Could not find NuGet package '{nugetId}' version '{version}' in lock file. Ensure NuGet Restore has run since this `<PackageReference>` was added.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id:version'.
- XA4300: Native library '{library}' will not be bundled because it has an unsupported ABI.
- [XA4301](#): Apk already contains the item `xxx`.
- [XA4302](#): Unhandled exception merging `AndroidManifest.xml`: {ex}
- [XA4303](#): Error extracting resources from "{assemblyPath}": {ex}
- [XA4304](#): ProGuard configuration file '{file}' was not found.
- [XA4305](#): Multidex is enabled, but `\$\_AndroidMainDexListFile` is empty.
- [XA4306](#): R8 does not support `@(MultiDexMainDexList)` files when `android:minSdkVersion >= 21`
- [XA4307](#): Invalid ProGuard configuration file.
- [XA4308](#): Failed to generate type maps
- [XA4309](#): 'MultiDexMainDexList' file '{file}' does not exist.
- [XA4310](#): `\$(AndroidSigningKeyStore)` file '{keystore}' could not be found.
- XA4311: The application won't contain the paired Wear package because the Wear application package APK is not created yet. If building on the command line, be sure to build the "SignAndroidPackage" target.
- [XA4312](#): Referencing an Android Wear application project from an Android application project is deprecated.
- [XA4313](#): Framework assembly has been deprecated.
- [XA4314](#): `$(Property)` is empty. A value for `$(Property)` should be provided.
- [XA4315](#): Ignoring {file}. Manifest does not have the required 'package' attribute on the manifest element.

XA5xxx: GCC and toolchain

- XA5101: Missing Android NDK toolchains directory '{path}'. Please install the Android NDK.
- XA5102: Conversion from assembly to native code failed. Exit code {exitCode}
- XA5103: NDK C compiler exited with an error. Exit code {0}
- XA5104: Could not locate the Android NDK.
- XA5105: Toolchain utility '{utility}' for target {arch} was not found. Tried in path: "{path}"
- XA5201: NDK linker exited with an error. Exit code {0}
- [XA5205](#): Cannot find `{ToolName}` in the Android SDK.

- [XA5207](#): Could not find android.jar for API level `{compileSdk}`.
- XA5211: Embedded Wear app package name differs from handheld app package name (`{wearPackageName} != {handheldPackageName}`).
- XA5213: `java.lang.OutOfMemoryError`. Consider increasing the value of `$(JavaMaximumHeapSize)`. Java ran out of memory while executing '`{tool} {arguments}`'
- [XA5300](#): The Android/Java SDK Directory could not be found.
- [XA5301](#): Failed to generate Java type for class: `{managedType}` due to MAX_PATH: `{exception}`
- [XA5302](#): Two processes may be building this project at once. Lock file exists at path: `{path}`

XA6xxx: Internal tools

XAccc7xxx: Unhandled MSBuild Exceptions

Exceptions that have not been gracefully handled yet. Ideally these will be fixed or replaced with better errors in the future.

These take the form of `XACCC7NNN`, where `CCC` is a 3 character code denoting the MSBuild Task that is throwing the exception, and `NNN` is a 3 digit number indicating the type of the unhandled `Exception`.

Tasks:

- `A2C` - `Aapt2Compile`
- `A2L` - `Aapt2Link`
- `AAS` - `AndroidApkSigner`
- `ACD` - `AndroidCreateDebugKey`
- `ACM` - `AppendCustomMetadataToItemGroup`
- `ADB` - `Adb`
- `AJV` - `AdjustJavacVersionArguments`
- `AOT` - `Aot`
- `APT` - `Aapt`
- `ASP` - `AndroidSignPackage`
- `AZA` - `AndroidZipAlign`
- `BAB` - `BuildAppBundle`
- `BAS` - `BuildApkSet`
- `BBA` - `BuildBaseAppBundle`

- BGN - BindingsGenerator
- BLD - BuildApk
- CAL - CreateAdditionalLibraryResourceCache
- CAR - CalculateAdditionalResourceCacheDirectories
- CCR - CopyAndConvertResources
- CCV - ConvertCustomView
- CDF - ConvertDebuggingFiles
- CDJ - CheckDuplicateJavaLibraries
- CFI - CheckForInvalidResourceFileNames
- CFR - CheckForRemovedItems
- CGJ - CopyGeneratedJavaResourceClasses
- CGS - CheckGoogleSdkRequirements
- CIC - CopyIfChanged
- CIL - CilStrip
- CLA - CollectLibraryAssets
- CLC - CalculateLayoutCodeBehind
- CLP - ClassParse
- CLR - CreateLibraryResourceArchive
- CMD - CreateMultiDexMainDexClassList
- CML - CreateManagedLibraryResourceArchive
- CMM - CreateMsymManifest
- CNA - CompileNativeAssembly
- CNE - CollectNonEmptyDirectories
- CNL - CreateNativeLibraryArchive
- CPD - CalculateProjectDependencies
- CPF - CollectPdbFiles
- CPI - CheckProjectItems
- CPR - CopyResource
- CPT - ComputeHash
- CRC - ConvertResourcesCases
- CRM - CreateResgenManifest
- CRN - Crunch
- CRP - AndroidComputeResPaths
- CTD - CreateTemporaryDirectory
- CTX - CompileToDalvik
- DES - Desugar
- DJL - DetermineJavaLibrariesToCompile
- DX8 - D8

- FD - FastDeploy
- FLB - FindLayoutsToBind
- FLT - FilterAssemblies
- GAD - GetAndroidDefineConstants
- GAP - GetAndroidPackageName
- GAR - GetAdditionalResourcesFromAssemblies
- GAS - GetAppSettingsDirectory
- GCB - GenerateCodeBehindForLayout
- GCJ - GetConvertedJavaLibraries
- GEP - GetExtraPackages
- GFT - GetFilesThatExist
- GIL - GetImportedLibraries
- GJP - GetJavaPlatformJar
- GJS - GenerateJavaStubs
- GLB - GenerateLayoutBindings
- GLR - GenerateLibraryResources
- GMA - GenerateManagedAidlProxies
- GMJ - GetMonoPlatformJar
- GPM - GeneratePackageManagerJava
- GRD - GenerateResourceDesigner
- IAS - InstallApkSet
- IJD - ImportJavaDoc
- JDC - JavaDoc
- JVC - Javac
- JTX - JarToXml
- KEY - KeyTool
- LAS - LinkApplicationSharedLibraries
- LEF - LogErrorsForFiles
- LNK - LinkAssemblies
- LNS - LinkAssembliesNoShrink
- LNT - Lint
- LWF - LogWarningsForFiles
- MBN - MakeBundleNativeCodeExternal
- MDC - MDoc
- PAI - PrepareAbiItems
- PAW - ParseAndroidWearProjectAndManifest
- PRO - Proguard
- PWA - PrepareWearApplicationFiles

- R8D - R8
- RAM - ReadAndroidManifest
- RAR - ReadAdditionalResourcesFromAssemblyCache
- RAT - ResolveAndroidTooling
- RDF - RemoveDirFixed
- RIL - ReadImportedLibrariesCache
- RJJ - ResolveJdkJvmPath
- RLC - ReadLibraryProjectImportsCache
- RLP - ResolveLibraryProjectImports
- RRA - RemoveRegisterAttribute
- RSA - ResolveAssemblies
- RSD - ResolveSdks
- RUF - RemoveUnknownFiles
- SPL - SplitProperty
- SVM - SetVsMonoAndroidRegistryKey
- UNZ - Unzip
- VJV - ValidateJavaVersion
- WLF - WriteLockFile

Exceptions:

- 7000 - Other Exception
- 7001 - NullPointerException
- 7002 - ArgumentOutOfRangeException
- 7003 - ArgumentNullException
- 7004 - ArgumentException
- 7005 - FormatException
- 7006 - IndexOutOfRangeException
- 7007 - InvalidCastException
- 7008 - ObjectDisposedException
- 7009 - InvalidOperationException
- 7010 - InvalidProgramException
- 7011 - KeyNotFoundException
- 7012 - TaskCanceledException
- 7013 - OperationCanceledException
- 7014 - OutOfMemoryException
- 7015 - NotSupportedException
- 7016 - StackOverflowException

- 7017 - `TimeoutException`
- 7018 - `TypeInitializationException`
- 7019 - `UnauthorizedAccessException`
- 7020 - `ApplicationException`
- 7021 - `KeyNotFoundException`
- 7022 - `PathTooLongException`
- 7023 - `DirectoryNotFoundException`
- 7024 - `IOException`
- 7025 - `DriveNotFoundException`
- 7026 - `EndOfStreamException`
- 7027 - `FileLoadException`
- 7028 - `FileNotFoundException`
- 7029 - `PipeException`

XA8xxx: Linker Step Errors

- [XA8000/IL8000](#): Could not find Android Resource '@anim/enterfromright'. Please update `@(AndroidResource)` to add the missing resource.

XA9xxx: Licensing

Removed messages

Removed in Xamarin.Android 10.4

- XA5215: Duplicate Resource found for {elementName}. Duplicates are in {filenames}
- XA5216: Resource entry {elementName} is already defined in {filename}

Removed in Xamarin.Android 10.3

- [XA0110](#): Disabling `$(AndroidExplicitCrunch)` as it is not supported by `aapt2`. If you wish to use `$(AndroidExplicitCrunch)` please set `$(AndroidUseAapt2)` to false.

Removed in Xamarin.Android 10.2

- [XA0120](#): Failed to use SHA1 hash algorithm

Removed in Xamarin.Android 9.3

- [XA0114](#): Google Play requires that application updates must use a `$(TargetFrameworkVersion)` of v8.0 (API level 26) or above.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error/warning

APT0000

Article • 04/17/2024

Example messages

```
error APT0000: res\drawable\image (1).png: Invalid file name: must contain only [a-z0-9_.]
```

```
error APT0000: Resource entry resource_name is already defined.
```

```
error APT0000: No resource found that matches the given name (at 'resource_name' with value '@string/foo').
```

```
error APT0000: invalid resource directory name: obj\Debug\dir with spaces "dir with spaces".
```

```
warning APT0000: warning: string 'resource_name' has no default translation.
```

Issue

This message indicates that `aapt` (Android Asset Packaging Tool) reported an error or warning. `aapt` is part of the Android SDK and is used internally by .NET for Android to process and compile resources into binary assets.

Errors reported by `aapt` are outside of .NET for Android's control, so a general error code of APT0000 is used reporting the exact message coming from `aapt`.

Solution

To learn more about `aapt` and Android resources, see the [Android documentation](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error APT0001

Article • 04/17/2024

Example messages

```
error APT0001: Unknown option `--ignore-assets`. Please check
`$(AndroidAapt2CompileExtraArgs)` and `$(AndroidAapt2LinkExtraArgs)` to see
if they include any `aapt` command line arguments that are no longer valid
for `aapt2` and ensure that all other arguments are valid for `aapt2`.
```

Issue

The new Android `aapt2` resource tool has different commandline arguments to the older `aapt` tool. These are generally incompatible with each other.

Solution

If you are receiving this error, check that the values provided in

XML

```
<AndroidAapt2LinkExtraArgs/>
<AndroidAapt2CompileExtraArgs>
```

are valid for `aapt2`. You can use `aapt2 compile` and `aapt2 link` to view the list of valid arguments.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error APT2264

Article • 04/17/2024

Issue

The tool `aapt2` is unable to resolve one of the files it was passed. This is generally caused by the path being longer than the Maximum Path length allowed on windows.

Solution

The best way to avoid this is to ensure that your project is not located deep in the folder structure. For example if you create all of your projects in folders such as

```
C:\Users\shelly\Visual Studio
2022\Android\MyProjects\Com.SomeReallyLongCompanyName.MyBrilliantApplication\
MyBrilliantApplicaition.Android\
```

you may well encounter problems with not only `aapt2` but also Ahead of Time compilation. Keeping your project names and folder structures short, concise will help work around these issues. For example instead of the above you could use

```
C:\Work\Android\MyBrilliantApp
```

Which is much shorter and much less likely to encounter path issues.

However this is not always possible. Sometimes a project or a environment requires deep folder structures. Enabling long path support in Windows *might* be enough to get your project working. Details on how to do this can be found [here](#).

If long path support does not work changing the location of the `$(BaseIntermediateOutputPath)` can help solve these problems. In order for this to work the setting MUST be changed before ANY build or restore occurs. To do this you can make use of the MSBuild `Directory.Build.props` support.

Creating a `Directory.Build.props` file in your solution or project directory which redefines the `$(BaseIntermediateOutputPath)` to somewhere nearer the root of the drive

with solve these issues. Adding a file with the following contents will create the `obj` directory in a different location of your choosing.

XML

```
<Project>
  <PropertyGroup>
    <BaseIntermediateOutputPath Condition=" '$(OS)' == 'Windows_NT'
">C:\Intermediate\$(ProjectName)</BaseIntermediateOutputPath>
    <BaseIntermediateOutputPath Condition=" '$(OS)' != 'Windows_NT'
">/tmp/Intermediate/$(ProjectName)</BaseIntermediateOutputPath>
  </PropertyGroup>
</Project>
```

Using this technique will reduce the lengths of the paths sent to the various tools like `aapt2`. Note this is generally only a Windows issue. So there is no need to override the `$(BaseIntermediateOutputPath)` on Mac or Linux based environments. However you might want to override everywhere to be consistent.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error APT2265

Article • 04/17/2024

Issue

The tool `aapt2` is unable to resolve one of the files it was passed. This is generally caused by non-ASCII characters in the project name or the path to the project.

Solution

The best way to avoid this is to ensure that your project does not contain non-ASCII characters. For example if you create all of your projects in folders such as

```
C:\Users\shëllly\Visual Studio  
2022\Android\MyProjects\Com.SomeReallyLongCompanyName.MyBrilliantApplication\MyBrill  
iantApplicaiton.Android\
```

you may well encounter problems with not only `aapt2` but also Ahead of Time compilation. Keeping your project names and folder structures short, concise and ASCII will help work around these issues. For example instead of the above you could use

```
C:\Work\Android\MyBrilliantApp
```

Which is much shorter, contains no non-ASCII characters and much less likely to encounter issues.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android errors and warnings reference

Article • 04/17/2024

ADBxxxx: ADB tooling

- [ADB0000](#): Generic `adb` error/warning.
- [ADB0010](#): Generic `adb` APK installation error/warning.
- [ADB0020](#): The package does not support the CPU architecture of this device.
- [ADB0030](#): APK installation failed due to a conflict with the existing package.
- [ADB0040](#): The device does not support the minimum SDK level specified in the manifest.
- [ADB0050](#): Package {packageName} already exists on device.
- [ADB0060](#): There is not enough storage space on the device to store package: {packageName}. Free up some space and try again.

ANDXXxxxx: Generic Android tooling

- [ANDAS0000](#): Generic `apksigner` error/warning.
- [ANDJS0000](#): Generic `jarsigner` error/warning.
- [ANDKT0000](#): Generic `keytool` error/warning.
- [ANDZA0000](#): Generic `zipalign` error/warning.

APTxxxx: AAPT tooling

- [APT0000](#): Generic `aapt` error/warning.
- [APT0001](#): Unknown option `{option}`. Please check ``$(AndroidAapt2CompileExtraArgs)`` and ``$(AndroidAapt2LinkExtraArgs)`` to see if they include any ``aapt`` command line arguments that are no longer valid for ``aapt2`` and ensure that all other arguments are valid for `aapt2`.
- [APT0002](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0003](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0004](#): Invalid file name: It must start with either A-Z or a-z or an underscore.
- [APT2264](#): The system cannot find the file specified. (2).
- [APT2265](#): The system cannot find the file specified. (2).

JAVAxxxx: Java Tool

- [JAVA0000](#): Generic Java tool error

JAVACxxxx: Java compiler

- [JAVAC0000](#): Generic Java compiler error

XA0xxx: Environment issue or missing tooling

- [XA0000](#): Could not determine `$(AndroidApiLevel)` or `$(TargetFrameworkVersion)`.
- [XA0001](#): Invalid or unsupported `$(TargetFrameworkVersion)` value.
- [XA0002](#): Could not find mono.android.jar
- [XA0003](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. It must be an integer value.
- [XA0004](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. The value must be in the range of 0 to `{maxVersionCode}`.
- [XA0030](#): Building with JDK version `{versionNumber}` is not supported.
- [XA0031](#): Java SDK `{requiredJavaForFrameworkVersion}` or above is required when using `$(TargetFrameworkVersion)` `{targetFrameworkVersion}`.
- [XA0032](#): Java SDK `{requiredJavaForBuildTools}` or above is required when using Android SDK Build-Tools `{buildToolsVersion}`.
- [XA0033](#): Failed to get the Java SDK version because the returned value does not appear to contain a valid version number.
- [XA0034](#): Failed to get the Java SDK version.
- [XA0035](#): Failed to determine the Android ABI for the project.
- [XA0036](#): `$(AndroidSupportedAbis)` is not supported in .NET 6 and higher.
- [XA0100](#): `EmbeddedNativeLibrary` is invalid in Android Application projects. Please use `AndroidNativeLibrary` instead.
- [XA0101](#): warning XA0101: `@(Content)` build action is not supported.
- [XA0102](#): Generic `lint` Warning.
- [XA0103](#): Generic `lint` Error.
- [XA0104](#): Invalid value for ``$(AndroidSequencePointsMode)``
- [XA0105](#): The `$(TargetFrameworkVersion)` for a library is greater than the `$(TargetFrameworkVersion)` for the application project.
- [XA0107](#): `{Assmebly}` is a Reference Assembly.
- [XA0108](#): Could not get version from `lint`.
- [XA0109](#): Unsupported or invalid `$(TargetFrameworkVersion)` value of `'v4.5'`.
- [XA0111](#): Could not get the `aapt2` version. Please check it is installed correctly.

- [XA0112](#): `aapt2` is not installed. Disabling `aapt2` support. Please check it is installed correctly.
- [XA0113](#): Google Play requires that new applications and updates must use a TargetFrameworkVersion of v11.0 (API level 30) or above.
- [XA0115](#): Invalid value 'armeabi' in \$(AndroidSupportedAbis). This ABI is no longer supported. Please update your project properties to remove the old value. If the properties page does not show an 'armeabi' checkbox, un-check and re-check one of the other ABIs and save the changes.
- [XA0116](#): Unable to find `EmbeddedResource` named `{ResourceName}`.
- [XA0117](#): The TargetFrameworkVersion {TargetFrameworkVersion} is deprecated. Please update it to be v4.4 or higher.
- [XA0118](#): Could not parse '{TargetMoniker}'
- [XA0119](#): A non-ideal configuration was found in the project.
- [XA0121](#): Assembly '{assembly}' is using '[assembly: Java.Interop.JavaLibraryReferenceAttribute]', which is no longer supported. Use a newer version of this NuGet package or notify the library author.
- [XA0122](#): Assembly '{assembly}' is using a deprecated attribute '[assembly: Java.Interop.DoNotPackageAttribute]'. Use a newer version of this NuGet package or notify the library author.
- [XA0123](#): Removing {issue} from {propertyName}. Lint {version} does not support this check.
- [XA0125](#): `{Project}` is using a deprecated debug information level. Set the debugging information to Portable in the Visual Studio project property pages or edit the project file in a text editor and set the 'DebugType' MSBuild property to 'portable' to use the newer, cross-platform debug information level. If this file comes from a NuGet package, update to a newer version of the NuGet package or notify the library author.
- [XA0126](#): Error installing FastDev Tools. This device does not support Fast Deployment. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0127](#): There was an issue deploying {destination} using {FastDevTool}. We encountered the following error {output}. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0128](#): Stdio Redirection is enabled. Please disable it to use Fast Deployment.
- [XA0129](#): Error deploying `{File}`. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0130](#): Sorry. Fast deployment is only supported on devices running Android 5.0 (API level 21) or higher. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.

- [XA0131](#): The 'run-as' tool has been disabled on this device. Either enable it by activating the developer options on the device or by setting `ro.boot.disable_runas` to `false`.
- [XA0132](#): The package was not installed. Please check you do not have it installed under any other user. If the package does show up on the device, try manually uninstalling it then try again. You should be able to uninstall the app via the Settings app on the device.
- [XA0133](#): The 'run-as' tool required by the Fast Deployment system has been disabled on this device by the manufacturer. Please disable Fast Deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0134](#): The application does not have the 'android:debuggable' attribute set in the AndroidManifest.xml. This is required in order for Fast Deployment to work. This is normally enabled by default by the .NET for Android build system for Debug builds.
- [XA0135](#): The package is a 'system' application. These are applications which install under the 'system' user on a device. These types of applications cannot use 'run-as'.
- [XA0136](#): The currently installation of the package is corrupt. Please manually uninstall the package from all the users on device and try again. If that does not work you can disable Fast Deployment.
- [XA0137](#): The 'run-as' command failed with '{0}'. Fast Deployment is not currently supported on this device. Please file an issue with the exact error message using the 'Help->Send Feedback->Report a Problem' menu item in Visual Studio or 'Help->Report a Problem' in Visual Studio for Mac.
- [XA0138](#): %(AndroidAsset.AssetPack) and %(AndroidAsset.AssetPack) item metadata are only supported when `$(AndroidApplication)` is `true`.
- [XA0139](#): `@(AndroidAsset) {0}` has invalid `DeliveryType` metadata of `{1}`. Supported values are `installtime`, `ondemand` or `fastfollow`
- [XA0140](#):
- [XA0141](#): NuGet package '{0}' version '{1}' contains a shared library '{2}' which is not correctly aligned. See <https://developer.android.com/guide/practices/page-sizes> for more details
- [XA0142](#): Command '{0}' failed.\n{1}

XA1xxx: Project related

- [XA1000](#): There was a problem parsing {file}. This is likely due to incomplete or invalid XML.

- [XA1001](#): AndroidResgen: Warning while updating Resource XML '{filename}': {Message}
- [XA1002](#): The closest match found for '{customViewName}' is '{customViewLookupName}', but the capitalization does not match. Please correct the capitalization.
- [XA1003](#): '{zip}' does not exist. Please rebuild the project.
- [XA1004](#): There was an error opening {filename}. The file is probably corrupt. Try deleting it and building again.
- [XA1005](#): Attempting basic type name matching for element with ID '{id}' and type '{managedType}'
- [XA1006](#): The TargetFrameworkVersion (Android API level {compileSdk}) is higher than the targetSdkVersion ({targetSdk}).
- [XA1007](#): The minSdkVersion ({minSdk}) is greater than targetSdkVersion. Please change the value such that minSdkVersion is less than or equal to targetSdkVersion ({targetSdk}).
- [XA1008](#): The TargetFrameworkVersion (Android API level {compileSdk}) is lower than the targetSdkVersion ({targetSdk}).
- [XA1009](#): The {assembly} is Obsolete. Please upgrade to {assembly} {version}
- [XA1010](#): Invalid `\$(AndroidManifestPlaceholders)` value for Android manifest placeholders. Please use `key1=value1;key2=value2` format. The specified value was: {placeholders}
- [XA1011](#): Using ProGuard with the D8 DEX compiler is no longer supported. Please set the code shrinker to 'r8' in the Visual Studio project property pages or edit the project file in a text editor and set the 'AndroidLinkTool' MSBuild property to 'r8'.
- XA1012: Included layout root element override ID '{id}' is not valid.
- XA1013: Failed to parse ID of node '{name}' in the layout file '{file}'.
- XA1014: JAR library references with identical file names but different contents were found: {libraries}. Please remove any conflicting libraries from EmbeddedJar, InputJar and AndroidJavaLibrary.
- XA1015: More than one Android Wear project is specified as the paired project. It can be at most one.
- XA1016: Target Wear application's project '{project}' does not specify required 'AndroidManifest' project property.
- XA1017: Target Wear application's AndroidManifest.xml does not specify required 'package' attribute.
- XA1018: Specified AndroidManifest file does not exist: {file}.
- XA1019: `LibraryProjectProperties` file `{file}` is located in a parent directory of the bindings project's intermediate output directory. Please adjust the path to use the original `project.properties` file directly from the Android library project directory.

- XA1020: At least one Java library is required for binding. Check that a Java library is included in the project and has the appropriate build action: 'LibraryProjectZip' (for AAR or ZIP), 'EmbeddedJar', 'InputJar' (for JAR), or 'LibraryProjectProperties' (project.properties).
- XA1021: Specified source Java library not found: {file}
- XA1022: Specified reference Java library not found: {file}
- [XA1023](#): Using the DX DEX Compiler is deprecated.
- [XA1024](#): Ignoring configuration file 'Foo.dll.config'. .NET configuration files are not supported in .NET for Android projects that target .NET 6 or higher.
- [XA1025](#): The experimental 'Hybrid' value for the 'AndroidAotMode' MSBuild property is not currently compatible with the armeabi-v7a target ABI.
- [XA1027](#): The 'EnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1028](#): The 'AndroidEnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1029](#): The 'AotAssemblies' MSBuild property is deprecated. Edit the project file in a text editor to remove this property, and use the 'RunAOTCompilation' MSBuild property instead.
- [XA1031](#): The 'AndroidHttpClientHandlerType' has an invalid value.
- [XA1032](#): Failed to resolve '{0}' from '{1}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1033](#): Could not resolve '{0}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1035](#): The 'BundleAssemblies' property is deprecated and it has no effect on the application build. Equivalent functionality is implemented by the 'AndroidUseAssemblyStore' and 'AndroidEnableAssemblyCompression' properties.
- [XA1036](#): AndroidManifest.xml //uses-sdk/@android:minSdkVersion '29' does not match the \$(SupportedOSPlatformVersion) value '21' in the project file (if there is no \$(SupportedOSPlatformVersion) value in the project file, then a default value has been assumed). Either change the value in the AndroidManifest.xml to match the \$(SupportedOSPlatformVersion) value, or remove the value in the AndroidManifest.xml (and add a \$(SupportedOSPlatformVersion) value to the project file if it doesn't already exist).
- [XA1037](#): The '{0}' MSBuild property is deprecated and will be removed in .NET {1}. See <https://aka.ms/net-android-deprecations> for more details.
- [XA1038](#): The '{0}' MSBuild property has an invalid value. Value values are {1}.

XA2xxx: Linker

- [XA2000](#): Use of AppDomain.CreateDomain() detected in assembly: {assembly}. .NET 6 will only support a single AppDomain, so this API will no longer be available in .NET for Android once .NET 6 is released.
- [XA2001](#): Source file '{filename}' could not be found.
- [XA2002](#): Can not resolve reference: `{missing}`, referenced by {assembly}. Perhaps it doesn't exist in the Mono for Android profile?
- XA2006: Could not resolve reference to '{member}' (defined in assembly '{assembly}') with scope '{scope}'. When the scope is different from the defining assembly, it usually means that the type is forwarded.
- XA2007: Exception while loading assemblies: {exception}
- XA2008: In referenced assembly {assembly}, Java.Interop.DoNotPackageAttribute requires non-null file name.

XA3xxx: Unmanaged code compilation

- XA3001: Could not AOT the assembly: {assembly}
- XA3002: Invalid AOT mode: {mode}
- XA3003: Could not strip IL of assembly: {assembly}
- XA3004: Android NDK r10d is buggy and provides an incompatible x86_64 libm.so.
- XA3005: The detected Android NDK version is incompatible with the targeted LLVM configuration.
- XA3006: Could not compile native assembly file: {file}
- XA3007: Could not link native shared library: {library}

XA4xxx: Code generation

- XA4209: Failed to generate Java type for class: {managedType} due to {exception}
- XA4210: Please add a reference to Mono.Android.Export.dll when using ExportAttribute or ExportFieldAttribute.
- XA4211: AndroidManifest.xml //uses-sdk/@android:targetSdkVersion '{targetSdk}' is less than \$(TargetFrameworkVersion) '{targetFramework}'. Using API-{targetFrameworkApi} for ACW compilation.
- XA4213: The type '{type}' must provide a public default constructor
- [XA4214](#): The managed type `Library1.Class1` exists in multiple assemblies: Library1, Library2. Please refactor the managed type names in these assemblies so that they are not identical.
- [XA4215](#): The Java type `com.contoso.library1.Class1` is generated by more than one managed type. Please change the [Register] attribute so that the same Java type is not emitted.

- [XA4216](#): The deployment target '19' is not supported (the minimum is '21'). Please increase the \$(SupportedOSPlatformVersion) property value in your project file.
- XA4217: Cannot override Kotlin-generated method '{method}' because it is not a valid Java method name. This method can only be overridden from Kotlin.
- [XA4218](#): Unable to find //manifest/application/uses-library at path: {path}
- XA4219: Cannot find binding generator for language {language} or {defaultLanguage}.
- XA4220: Partial class item '{file}' does not have an associated binding for layout '{layout}'.
- XA4221: No layout binding source files were generated.
- XA4222: No widgets found for layout ({layoutFiles}).
- XA4223: Malformed full class name '{name}'. Missing namespace.
- XA4224: Malformed full class name '{name}'. Missing class name.
- XA4225: Widget '{widget}' in layout '{layout}' has multiple instances with different types. The property type will be set to: {type}
- XA4226: Resource item '{file}' does not have the required metadata item '{metadataName}'.
- XA4228: Unable to find specified //activity-alias/@android:targetActivity: '{targetActivity}'
- XA4229: Unrecognized `TransformFile` root element: {element}.
- XA4230: Error parsing XML: {exception}
- [XA4231](#): The Android class parser value 'jar2xml' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'class-parse'.
- [XA4232](#): The Android code generation target 'XamarinAndroid' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'XJavaInterop1'.
- [XA4234](#): '<{item}>' item '{itemspec}' is missing required attribute '{name}'.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id'.
- [XA4236](#): Cannot download Maven artifact '{group}:{artifact}'. - {jar}: {exception} - {aar}: {exception}
- [XA4237](#): Cannot download POM file for Maven artifact '{artifact}'. - {exception}
- [XA4239](#): Unknown Maven repository: '{repository}'.
- [XA4241](#): Java dependency '{artifact}' is not satisfied.
- [XA4242](#): Java dependency '{artifact}' is not satisfied. Microsoft maintains the NuGet package '{nugetId}' that could fulfill this dependency.
- [XA4243](#): Attribute '{name}' is required when using '{name}' for '{element}' item '{itemspec}'.
- [XA4244](#): Attribute '{name}' cannot be empty for '{element}' item '{itemspec}'.
- [XA4245](#): Specified POM file '{file}' does not exist.

- [XA4246](#): Could not parse POM file '{file}'. - {exception}
- [XA4247](#): Could not resolve POM file for artifact '{artifact}'.
- [XA4248](#): Could not find NuGet package '{nugetId}' version '{version}' in lock file. Ensure NuGet Restore has run since this `<PackageReference>` was added.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id:version'.
- XA4300: Native library '{library}' will not be bundled because it has an unsupported ABI.
- [XA4301](#): Apk already contains the item `xxx`.
- [XA4302](#): Unhandled exception merging `AndroidManifest.xml`: {ex}
- [XA4303](#): Error extracting resources from "{assemblyPath}": {ex}
- [XA4304](#): ProGuard configuration file '{file}' was not found.
- [XA4305](#): Multidex is enabled, but `\$\_AndroidMainDexListFile` is empty.
- [XA4306](#): R8 does not support `@(MultiDexMainDexList)` files when `android:minSdkVersion >= 21`
- [XA4307](#): Invalid ProGuard configuration file.
- [XA4308](#): Failed to generate type maps
- [XA4309](#): 'MultiDexMainDexList' file '{file}' does not exist.
- [XA4310](#): `\$(AndroidSigningKeyStore)` file '{keystore}` could not be found.
- XA4311: The application won't contain the paired Wear package because the Wear application package APK is not created yet. If building on the command line, be sure to build the "SignAndroidPackage" target.
- [XA4312](#): Referencing an Android Wear application project from an Android application project is deprecated.
- [XA4313](#): Framework assembly has been deprecated.
- [XA4314](#): `$(Property)` is empty. A value for `$(Property)` should be provided.
- [XA4315](#): Ignoring {file}. Manifest does not have the required 'package' attribute on the manifest element.

XA5xxx: GCC and toolchain

- XA5101: Missing Android NDK toolchains directory '{path}'. Please install the Android NDK.
- XA5102: Conversion from assembly to native code failed. Exit code {exitCode}
- XA5103: NDK C compiler exited with an error. Exit code {0}
- XA5104: Could not locate the Android NDK.
- XA5105: Toolchain utility '{utility}' for target {arch} was not found. Tried in path: "{path}"
- XA5201: NDK linker exited with an error. Exit code {0}
- [XA5205](#): Cannot find `{ToolName}` in the Android SDK.

- [XA5207](#): Could not find android.jar for API level `{compileSdk}`.
- [XA5211](#): Embedded Wear app package name differs from handheld app package name (`{wearPackageName} != {handheldPackageName}`).
- [XA5213](#): `java.lang.OutOfMemoryError`. Consider increasing the value of `$(JavaMaximumHeapSize)`. Java ran out of memory while executing '`{tool}` `{arguments}`'
- [XA5300](#): The Android/Java SDK Directory could not be found.
- [XA5301](#): Failed to generate Java type for class: `{managedType}` due to `MAX_PATH`: `{exception}`
- [XA5302](#): Two processes may be building this project at once. Lock file exists at path: `{path}`

XA6xxx: Internal tools

XAccc7xxx: Unhandled MSBuild Exceptions

Exceptions that have not been gracefully handled yet. Ideally these will be fixed or replaced with better errors in the future.

These take the form of `XACCC7NNN`, where `CCC` is a 3 character code denoting the MSBuild Task that is throwing the exception, and `NNN` is a 3 digit number indicating the type of the unhandled `Exception`.

Tasks:

- `A2C` - `Aapt2Compile`
- `A2L` - `Aapt2Link`
- `AAS` - `AndroidApkSigner`
- `ACD` - `AndroidCreateDebugKey`
- `ACM` - `AppendCustomMetadataToItemGroup`
- `ADB` - `Adb`
- `AJV` - `AdjustJavacVersionArguments`
- `AOT` - `Aot`
- `APT` - `Aapt`
- `ASP` - `AndroidSignPackage`
- `AZA` - `AndroidZipAlign`
- `BAB` - `BuildAppBundle`
- `BAS` - `BuildApkSet`
- `BBA` - `BuildBaseAppBundle`

- BGN - BindingsGenerator
- BLD - BuildApk
- CAL - CreateAdditionalLibraryResourceCache
- CAR - CalculateAdditionalResourceCacheDirectories
- CCR - CopyAndConvertResources
- CCV - ConvertCustomView
- CDF - ConvertDebuggingFiles
- CDJ - CheckDuplicateJavaLibraries
- CFI - CheckForInvalidResourceFileNames
- CFR - CheckForRemovedItems
- CGJ - CopyGeneratedJavaResourceClasses
- CGS - CheckGoogleSdkRequirements
- CIC - CopyIfChanged
- CIL - CilStrip
- CLA - CollectLibraryAssets
- CLC - CalculateLayoutCodeBehind
- CLP - ClassParse
- CLR - CreateLibraryResourceArchive
- CMD - CreateMultiDexMainDexClassList
- CML - CreateManagedLibraryResourceArchive
- CMM - CreateMsymManifest
- CNA - CompileNativeAssembly
- CNE - CollectNonEmptyDirectories
- CNL - CreateNativeLibraryArchive
- CPD - CalculateProjectDependencies
- CPF - CollectPdbFiles
- CPI - CheckProjectItems
- CPR - CopyResource
- CPT - ComputeHash
- CRC - ConvertResourcesCases
- CRM - CreateResgenManifest
- CRN - Crunch
- CRP - AndroidComputeResPaths
- CTD - CreateTemporaryDirectory
- CTX - CompileToDalvik
- DES - Desugar
- DJL - DetermineJavaLibrariesToCompile
- DX8 - D8

- FD - FastDeploy
- FLB - FindLayoutsToBind
- FLT - FilterAssemblies
- GAD - GetAndroidDefineConstants
- GAP - GetAndroidPackageName
- GAR - GetAdditionalResourcesFromAssemblies
- GAS - GetAppSettingsDirectory
- GCB - GenerateCodeBehindForLayout
- GCJ - GetConvertedJavaLibraries
- GEP - GetExtraPackages
- GFT - GetFilesThatExist
- GIL - GetImportedLibraries
- GJP - GetJavaPlatformJar
- GJS - GenerateJavaStubs
- GLB - GenerateLayoutBindings
- GLR - GenerateLibraryResources
- GMA - GenerateManagedAidlProxies
- GMJ - GetMonoPlatformJar
- GPM - GeneratePackageManagerJava
- GRD - GenerateResourceDesigner
- IAS - InstallApkSet
- IJD - ImportJavaDoc
- JDC - JavaDoc
- JVC - Javac
- JTX - JarToXml
- KEY - KeyTool
- LAS - LinkApplicationSharedLibraries
- LEF - LogErrorsForFiles
- LNK - LinkAssemblies
- LNS - LinkAssembliesNoShrink
- LNT - Lint
- LWF - LogWarningsForFiles
- MBN - MakeBundleNativeCodeExternal
- MDC - MDoc
- PAI - PrepareAbiItems
- PAW - ParseAndroidWearProjectAndManifest
- PRO - Proguard
- PWA - PrepareWearApplicationFiles

- R8D - R8
- RAM - ReadAndroidManifest
- RAR - ReadAdditionalResourcesFromAssemblyCache
- RAT - ResolveAndroidTooling
- RDF - RemoveDirFixed
- RIL - ReadImportedLibrariesCache
- RJJ - ResolveJdkJvmPath
- RLC - ReadLibraryProjectImportsCache
- RLP - ResolveLibraryProjectImports
- RRA - RemoveRegisterAttribute
- RSA - ResolveAssemblies
- RSD - ResolveSdks
- RUF - RemoveUnknownFiles
- SPL - SplitProperty
- SVM - SetVsMonoAndroidRegistryKey
- UNZ - Unzip
- VJV - ValidateJavaVersion
- WLF - WriteLockFile

Exceptions:

- 7000 - Other Exception
- 7001 - NullPointerException
- 7002 - ArgumentOutOfRangeException
- 7003 - ArgumentNullException
- 7004 - ArgumentException
- 7005 - FormatException
- 7006 - IndexOutOfRangeException
- 7007 - InvalidCastException
- 7008 - ObjectDisposedException
- 7009 - InvalidOperationException
- 7010 - InvalidProgramException
- 7011 - KeyNotFoundException
- 7012 - TaskCanceledException
- 7013 - OperationCanceledException
- 7014 - OutOfMemoryException
- 7015 - NotSupportedException
- 7016 - StackOverflowException

- 7017 - `TimeoutException`
- 7018 - `TypeInitializationException`
- 7019 - `UnauthorizedAccessException`
- 7020 - `ApplicationException`
- 7021 - `KeyNotFoundException`
- 7022 - `PathTooLongException`
- 7023 - `DirectoryNotFoundException`
- 7024 - `IOException`
- 7025 - `DriveNotFoundException`
- 7026 - `EndOfStreamException`
- 7027 - `FileLoadException`
- 7028 - `FileNotFoundException`
- 7029 - `PipeException`

XA8xxx: Linker Step Errors

- [XA8000/IL8000](#): Could not find Android Resource '@anim/enterfromright'. Please update `@(AndroidResource)` to add the missing resource.

XA9xxx: Licensing

Removed messages

Removed in Xamarin.Android 10.4

- XA5215: Duplicate Resource found for {elementName}. Duplicates are in {filenames}
- XA5216: Resource entry {elementName} is already defined in {filename}

Removed in Xamarin.Android 10.3

- [XA0110](#): Disabling `$(AndroidExplicitCrunch)` as it is not supported by `aapt2`. If you wish to use `$(AndroidExplicitCrunch)` please set `$(AndroidUseAapt2)` to false.

Removed in Xamarin.Android 10.2

- [XA0120](#): Failed to use SHA1 hash algorithm

Removed in Xamarin.Android 9.3

- [XA0114](#): Google Play requires that application updates must use a `$(TargetFrameworkVersion)` of v8.0 (API level 26) or above.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error JAVA0000

Article • 04/17/2024

Example messages

```
MSBUILD : java error JAVA0000: Error in obj/Debug/net7.0-  
android/lp/59/jl/classes.jar:android/support/v4/app/INotificationSideChannel  
$Stub.class: [Example.csproj]  
MSBUILD : java error JAVA0000: Type  
android.support.v4.app.INotificationSideChannel$Stub is defined multiple  
times: obj/Debug/net7.0-  
android/lp/59/jl/classes.jar:android/support/v4/app/INotificationSideChannel  
$Stub.class, obj/Debug/net7.0-  
android/lp/4/jl/bin/classes.jar:android/support/v4/app/INotificationSideChan  
nel$Stub.class [Example.csproj]  
MSBUILD : java error JAVA0000: Compilation failed [Example.csproj]  
MSBUILD : java error JAVA0000: java.lang.RuntimeException:  
com.android.tools.r8.CompilationFailedException: Compilation failed to  
complete, origin: obj/Debug/net7.0-android/lp/59/jl/classes.jar  
[Example.csproj]  
MSBUILD : java error JAVA0000:  
android/support/v4/app/INotificationSideChannel$Stub.class [Example.csproj]  
MSBUILD : java error JAVA0000: at  
com.android.tools.r8.internal.Bj.a(R8_3.3.28_2aaf796388b4e9f6bed752d926eca11  
0512a53a3f09a8d755196089c1cfd799:98) [Example.csproj]  
MSBUILD : java error JAVA0000: at  
com.android.tools.r8.D8.main(R8_3.3.28_2aaf796388b4e9f6bed752d926eca110512a5  
3a3f09a8d755196089c1cfd799:4) [Example.csproj]
```

Issue

During a normal .NET for Android build various java tools are run. This message indicates that [java][java] tool which was run raised an error.

Solution

Look at the stack trace included in the error. This should provide some clues into why the error occurred.

Consider submitting a [bug](#) if you are getting this error under normal circumstances.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android errors and warnings reference

Article • 04/17/2024

ADBxxxx: ADB tooling

- [ADB0000](#): Generic `adb` error/warning.
- [ADB0010](#): Generic `adb` APK installation error/warning.
- [ADB0020](#): The package does not support the CPU architecture of this device.
- [ADB0030](#): APK installation failed due to a conflict with the existing package.
- [ADB0040](#): The device does not support the minimum SDK level specified in the manifest.
- [ADB0050](#): Package {packageName} already exists on device.
- [ADB0060](#): There is not enough storage space on the device to store package: {packageName}. Free up some space and try again.

ANDXXxxxx: Generic Android tooling

- [ANDAS0000](#): Generic `apksigner` error/warning.
- [ANDJS0000](#): Generic `jarsigner` error/warning.
- [ANDKT0000](#): Generic `keytool` error/warning.
- [ANDZA0000](#): Generic `zipalign` error/warning.

APTxxxx: AAPT tooling

- [APT0000](#): Generic `aapt` error/warning.
- [APT0001](#): Unknown option `{option}`. Please check ``$(AndroidAapt2CompileExtraArgs)`` and ``$(AndroidAapt2LinkExtraArgs)`` to see if they include any ``aapt`` command line arguments that are no longer valid for ``aapt2`` and ensure that all other arguments are valid for `aapt2`.
- [APT0002](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0003](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0004](#): Invalid file name: It must start with either A-Z or a-z or an underscore.
- [APT2264](#): The system cannot find the file specified. (2).
- [APT2265](#): The system cannot find the file specified. (2).

JAVAxxxx: Java Tool

- [JAVA0000](#): Generic Java tool error

JAVACxxxx: Java compiler

- [JAVAC0000](#): Generic Java compiler error

XA0xxx: Environment issue or missing tooling

- [XA0000](#): Could not determine `$(AndroidApiLevel)` or `$(TargetFrameworkVersion)`.
- [XA0001](#): Invalid or unsupported `$(TargetFrameworkVersion)` value.
- [XA0002](#): Could not find mono.android.jar
- [XA0003](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. It must be an integer value.
- [XA0004](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. The value must be in the range of 0 to `{maxVersionCode}`.
- [XA0030](#): Building with JDK version `{versionNumber}` is not supported.
- [XA0031](#): Java SDK `{requiredJavaForFrameworkVersion}` or above is required when using `$(TargetFrameworkVersion)` `{targetFrameworkVersion}`.
- [XA0032](#): Java SDK `{requiredJavaForBuildTools}` or above is required when using Android SDK Build-Tools `{buildToolsVersion}`.
- [XA0033](#): Failed to get the Java SDK version because the returned value does not appear to contain a valid version number.
- [XA0034](#): Failed to get the Java SDK version.
- [XA0035](#): Failed to determine the Android ABI for the project.
- [XA0036](#): `$(AndroidSupportedAbis)` is not supported in .NET 6 and higher.
- [XA0100](#): `EmbeddedNativeLibrary` is invalid in Android Application projects. Please use `AndroidNativeLibrary` instead.
- [XA0101](#): warning XA0101: `@(Content)` build action is not supported.
- [XA0102](#): Generic `lint` Warning.
- [XA0103](#): Generic `lint` Error.
- [XA0104](#): Invalid value for ``$(AndroidSequencePointsMode)``
- [XA0105](#): The `$(TargetFrameworkVersion)` for a library is greater than the `$(TargetFrameworkVersion)` for the application project.
- [XA0107](#): `{Assmebly}` is a Reference Assembly.
- [XA0108](#): Could not get version from `lint`.
- [XA0109](#): Unsupported or invalid `$(TargetFrameworkVersion)` value of 'v4.5'.
- [XA0111](#): Could not get the `aapt2` version. Please check it is installed correctly.

- [XA0112](#): `aapt2` is not installed. Disabling `aapt2` support. Please check it is installed correctly.
- [XA0113](#): Google Play requires that new applications and updates must use a TargetFrameworkVersion of v11.0 (API level 30) or above.
- [XA0115](#): Invalid value 'armeabi' in \$(AndroidSupportedAbis). This ABI is no longer supported. Please update your project properties to remove the old value. If the properties page does not show an 'armeabi' checkbox, un-check and re-check one of the other ABIs and save the changes.
- [XA0116](#): Unable to find `EmbeddedResource` named `{ResourceName}`.
- [XA0117](#): The TargetFrameworkVersion {TargetFrameworkVersion} is deprecated. Please update it to be v4.4 or higher.
- [XA0118](#): Could not parse '{TargetMoniker}'
- [XA0119](#): A non-ideal configuration was found in the project.
- [XA0121](#): Assembly '{assembly}' is using '[assembly: Java.Interop.JavaLibraryReferenceAttribute]', which is no longer supported. Use a newer version of this NuGet package or notify the library author.
- [XA0122](#): Assembly '{assembly}' is using a deprecated attribute '[assembly: Java.Interop.DoNotPackageAttribute]'. Use a newer version of this NuGet package or notify the library author.
- [XA0123](#): Removing {issue} from {propertyName}. Lint {version} does not support this check.
- [XA0125](#): `{Project}` is using a deprecated debug information level. Set the debugging information to Portable in the Visual Studio project property pages or edit the project file in a text editor and set the 'DebugType' MSBuild property to 'portable' to use the newer, cross-platform debug information level. If this file comes from a NuGet package, update to a newer version of the NuGet package or notify the library author.
- [XA0126](#): Error installing FastDev Tools. This device does not support Fast Deployment. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0127](#): There was an issue deploying {destination} using {FastDevTool}. We encountered the following error {output}. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0128](#): Stdio Redirection is enabled. Please disable it to use Fast Deployment.
- [XA0129](#): Error deploying `{File}`. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0130](#): Sorry. Fast deployment is only supported on devices running Android 5.0 (API level 21) or higher. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.

- [XA0131](#): The 'run-as' tool has been disabled on this device. Either enable it by activating the developer options on the device or by setting `ro.boot.disable_runas` to `false`.
- [XA0132](#): The package was not installed. Please check you do not have it installed under any other user. If the package does show up on the device, try manually uninstalling it then try again. You should be able to uninstall the app via the Settings app on the device.
- [XA0133](#): The 'run-as' tool required by the Fast Deployment system has been disabled on this device by the manufacturer. Please disable Fast Deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0134](#): The application does not have the 'android:debuggable' attribute set in the AndroidManifest.xml. This is required in order for Fast Deployment to work. This is normally enabled by default by the .NET for Android build system for Debug builds.
- [XA0135](#): The package is a 'system' application. These are applications which install under the 'system' user on a device. These types of applications cannot use 'run-as'.
- [XA0136](#): The currently installation of the package is corrupt. Please manually uninstall the package from all the users on device and try again. If that does not work you can disable Fast Deployment.
- [XA0137](#): The 'run-as' command failed with '{0}'. Fast Deployment is not currently supported on this device. Please file an issue with the exact error message using the 'Help->Send Feedback->Report a Problem' menu item in Visual Studio or 'Help->Report a Problem' in Visual Studio for Mac.
- [XA0138](#): %(AndroidAsset.AssetPack) and %(AndroidAsset.AssetPack) item metadata are only supported when `$(AndroidApplication)` is `true`.
- [XA0139](#): `@(AndroidAsset) {0}` has invalid `DeliveryType` metadata of `{1}`. Supported values are `installtime`, `ondemand` or `fastfollow`
- [XA0140](#):
- [XA0141](#): NuGet package '{0}' version '{1}' contains a shared library '{2}' which is not correctly aligned. See <https://developer.android.com/guide/practices/page-sizes> for more details
- [XA0142](#): Command '{0}' failed.\n{1}

XA1xxx: Project related

- [XA1000](#): There was a problem parsing {file}. This is likely due to incomplete or invalid XML.

- [XA1001](#): AndroidResgen: Warning while updating Resource XML '{filename}': {Message}
- [XA1002](#): The closest match found for '{customViewName}' is '{customViewLookupName}', but the capitalization does not match. Please correct the capitalization.
- [XA1003](#): '{zip}' does not exist. Please rebuild the project.
- [XA1004](#): There was an error opening {filename}. The file is probably corrupt. Try deleting it and building again.
- [XA1005](#): Attempting basic type name matching for element with ID '{id}' and type '{managedType}'
- [XA1006](#): The TargetFrameworkVersion (Android API level {compileSdk}) is higher than the targetSdkVersion ({targetSdk}).
- [XA1007](#): The minSdkVersion ({minSdk}) is greater than targetSdkVersion. Please change the value such that minSdkVersion is less than or equal to targetSdkVersion ({targetSdk}).
- [XA1008](#): The TargetFrameworkVersion (Android API level {compileSdk}) is lower than the targetSdkVersion ({targetSdk}).
- [XA1009](#): The {assembly} is Obsolete. Please upgrade to {assembly} {version}
- [XA1010](#): Invalid `\$(AndroidManifestPlaceholders)` value for Android manifest placeholders. Please use `key1=value1;key2=value2` format. The specified value was: {placeholders}
- [XA1011](#): Using ProGuard with the D8 DEX compiler is no longer supported. Please set the code shrinker to 'r8' in the Visual Studio project property pages or edit the project file in a text editor and set the 'AndroidLinkTool' MSBuild property to 'r8'.
- XA1012: Included layout root element override ID '{id}' is not valid.
- XA1013: Failed to parse ID of node '{name}' in the layout file '{file}'.
- XA1014: JAR library references with identical file names but different contents were found: {libraries}. Please remove any conflicting libraries from EmbeddedJar, InputJar and AndroidJavaLibrary.
- XA1015: More than one Android Wear project is specified as the paired project. It can be at most one.
- XA1016: Target Wear application's project '{project}' does not specify required 'AndroidManifest' project property.
- XA1017: Target Wear application's AndroidManifest.xml does not specify required 'package' attribute.
- XA1018: Specified AndroidManifest file does not exist: {file}.
- XA1019: `LibraryProjectProperties` file `{file}` is located in a parent directory of the bindings project's intermediate output directory. Please adjust the path to use the original `project.properties` file directly from the Android library project directory.

- XA1020: At least one Java library is required for binding. Check that a Java library is included in the project and has the appropriate build action: 'LibraryProjectZip' (for AAR or ZIP), 'EmbeddedJar', 'InputJar' (for JAR), or 'LibraryProjectProperties' (project.properties).
- XA1021: Specified source Java library not found: {file}
- XA1022: Specified reference Java library not found: {file}
- [XA1023](#): Using the DX DEX Compiler is deprecated.
- [XA1024](#): Ignoring configuration file 'Foo.dll.config'. .NET configuration files are not supported in .NET for Android projects that target .NET 6 or higher.
- [XA1025](#): The experimental 'Hybrid' value for the 'AndroidAotMode' MSBuild property is not currently compatible with the armeabi-v7a target ABI.
- [XA1027](#): The 'EnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1028](#): The 'AndroidEnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1029](#): The 'AotAssemblies' MSBuild property is deprecated. Edit the project file in a text editor to remove this property, and use the 'RunAOTCompilation' MSBuild property instead.
- [XA1031](#): The 'AndroidHttpClientHandlerType' has an invalid value.
- [XA1032](#): Failed to resolve '{0}' from '{1}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1033](#): Could not resolve '{0}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1035](#): The 'BundleAssemblies' property is deprecated and it has no effect on the application build. Equivalent functionality is implemented by the 'AndroidUseAssemblyStore' and 'AndroidEnableAssemblyCompression' properties.
- [XA1036](#): AndroidManifest.xml //uses-sdk/@android:minSdkVersion '29' does not match the \$(SupportedOSPlatformVersion) value '21' in the project file (if there is no \$(SupportedOSPlatformVersion) value in the project file, then a default value has been assumed). Either change the value in the AndroidManifest.xml to match the \$(SupportedOSPlatformVersion) value, or remove the value in the AndroidManifest.xml (and add a \$(SupportedOSPlatformVersion) value to the project file if it doesn't already exist).
- [XA1037](#): The '{0}' MSBuild property is deprecated and will be removed in .NET {1}. See <https://aka.ms/net-android-deprecations> for more details.
- [XA1038](#): The '{0}' MSBuild property has an invalid value. Value values are {1}.

XA2xxx: Linker

- [XA2000](#): Use of AppDomain.CreateDomain() detected in assembly: {assembly}. .NET 6 will only support a single AppDomain, so this API will no longer be available in .NET for Android once .NET 6 is released.
- [XA2001](#): Source file '{filename}' could not be found.
- [XA2002](#): Can not resolve reference: `{missing}`, referenced by {assembly}. Perhaps it doesn't exist in the Mono for Android profile?
- XA2006: Could not resolve reference to '{member}' (defined in assembly '{assembly}') with scope '{scope}'. When the scope is different from the defining assembly, it usually means that the type is forwarded.
- XA2007: Exception while loading assemblies: {exception}
- XA2008: In referenced assembly {assembly}, Java.Interop.DoNotPackageAttribute requires non-null file name.

XA3xxx: Unmanaged code compilation

- XA3001: Could not AOT the assembly: {assembly}
- XA3002: Invalid AOT mode: {mode}
- XA3003: Could not strip IL of assembly: {assembly}
- XA3004: Android NDK r10d is buggy and provides an incompatible x86_64 libm.so.
- XA3005: The detected Android NDK version is incompatible with the targeted LLVM configuration.
- XA3006: Could not compile native assembly file: {file}
- XA3007: Could not link native shared library: {library}

XA4xxx: Code generation

- XA4209: Failed to generate Java type for class: {managedType} due to {exception}
- XA4210: Please add a reference to Mono.Android.Export.dll when using ExportAttribute or ExportFieldAttribute.
- XA4211: AndroidManifest.xml //uses-sdk/@android:targetSdkVersion '{targetSdk}' is less than \$(TargetFrameworkVersion) '{targetFramework}'. Using API-{targetFrameworkApi} for ACW compilation.
- XA4213: The type '{type}' must provide a public default constructor
- [XA4214](#): The managed type `Library1.Class1` exists in multiple assemblies: Library1, Library2. Please refactor the managed type names in these assemblies so that they are not identical.
- [XA4215](#): The Java type `com.contoso.library1.Class1` is generated by more than one managed type. Please change the [Register] attribute so that the same Java type is not emitted.

- [XA4216](#): The deployment target '19' is not supported (the minimum is '21'). Please increase the \$(SupportedOSPlatformVersion) property value in your project file.
- XA4217: Cannot override Kotlin-generated method '{method}' because it is not a valid Java method name. This method can only be overridden from Kotlin.
- [XA4218](#): Unable to find //manifest/application/uses-library at path: {path}
- XA4219: Cannot find binding generator for language {language} or {defaultLanguage}.
- XA4220: Partial class item '{file}' does not have an associated binding for layout '{layout}'.
- XA4221: No layout binding source files were generated.
- XA4222: No widgets found for layout ({layoutFiles}).
- XA4223: Malformed full class name '{name}'. Missing namespace.
- XA4224: Malformed full class name '{name}'. Missing class name.
- XA4225: Widget '{widget}' in layout '{layout}' has multiple instances with different types. The property type will be set to: {type}
- XA4226: Resource item '{file}' does not have the required metadata item '{metadataName}'.
- XA4228: Unable to find specified //activity-alias/@android:targetActivity: '{targetActivity}'
- XA4229: Unrecognized `TransformFile` root element: {element}.
- XA4230: Error parsing XML: {exception}
- [XA4231](#): The Android class parser value 'jar2xml' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'class-parse'.
- [XA4232](#): The Android code generation target 'XamarinAndroid' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'XJavaInterop1'.
- [XA4234](#): '<{item}>' item '{itemspec}' is missing required attribute '{name}'.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id'.
- [XA4236](#): Cannot download Maven artifact '{group}:{artifact}'. - {jar}: {exception} - {aar}: {exception}
- [XA4237](#): Cannot download POM file for Maven artifact '{artifact}'. - {exception}
- [XA4239](#): Unknown Maven repository: '{repository}'.
- [XA4241](#): Java dependency '{artifact}' is not satisfied.
- [XA4242](#): Java dependency '{artifact}' is not satisfied. Microsoft maintains the NuGet package '{nugetId}' that could fulfill this dependency.
- [XA4243](#): Attribute '{name}' is required when using '{name}' for '{element}' item '{itemspec}'.
- [XA4244](#): Attribute '{name}' cannot be empty for '{element}' item '{itemspec}'.
- [XA4245](#): Specified POM file '{file}' does not exist.

- [XA4246](#): Could not parse POM file '{file}'. - {exception}
- [XA4247](#): Could not resolve POM file for artifact '{artifact}'.
- [XA4248](#): Could not find NuGet package '{nugetId}' version '{version}' in lock file. Ensure NuGet Restore has run since this `<PackageReference>` was added.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id:version'.
- XA4300: Native library '{library}' will not be bundled because it has an unsupported ABI.
- [XA4301](#): Apk already contains the item `xxx`.
- [XA4302](#): Unhandled exception merging `AndroidManifest.xml`: {ex}
- [XA4303](#): Error extracting resources from "{assemblyPath}": {ex}
- [XA4304](#): ProGuard configuration file '{file}' was not found.
- [XA4305](#): Multidex is enabled, but `\$\_AndroidMainDexListFile` is empty.
- [XA4306](#): R8 does not support `@(MultiDexMainDexList)` files when `android:minSdkVersion >= 21`
- [XA4307](#): Invalid ProGuard configuration file.
- [XA4308](#): Failed to generate type maps
- [XA4309](#): 'MultiDexMainDexList' file '{file}' does not exist.
- [XA4310](#): `\$(AndroidSigningKeyStore)` file '{keystore}' could not be found.
- XA4311: The application won't contain the paired Wear package because the Wear application package APK is not created yet. If building on the command line, be sure to build the "SignAndroidPackage" target.
- [XA4312](#): Referencing an Android Wear application project from an Android application project is deprecated.
- [XA4313](#): Framework assembly has been deprecated.
- [XA4314](#): `$(Property)` is empty. A value for `$(Property)` should be provided.
- [XA4315](#): Ignoring {file}. Manifest does not have the required 'package' attribute on the manifest element.

XA5xxx: GCC and toolchain

- XA5101: Missing Android NDK toolchains directory '{path}'. Please install the Android NDK.
- XA5102: Conversion from assembly to native code failed. Exit code {exitCode}
- XA5103: NDK C compiler exited with an error. Exit code {0}
- XA5104: Could not locate the Android NDK.
- XA5105: Toolchain utility '{utility}' for target {arch} was not found. Tried in path: "{path}"
- XA5201: NDK linker exited with an error. Exit code {0}
- [XA5205](#): Cannot find `{ToolName}` in the Android SDK.

- [XA5207](#): Could not find android.jar for API level `{compileSdk}`.
- [XA5211](#): Embedded Wear app package name differs from handheld app package name (`{wearPackageName} != {handheldPackageName}`).
- [XA5213](#): `java.lang.OutOfMemoryError`. Consider increasing the value of `$(JavaMaximumHeapSize)`. Java ran out of memory while executing '`{tool}` `{arguments}`'
- [XA5300](#): The Android/Java SDK Directory could not be found.
- [XA5301](#): Failed to generate Java type for class: `{managedType}` due to `MAX_PATH`: `{exception}`
- [XA5302](#): Two processes may be building this project at once. Lock file exists at path: `{path}`

XA6xxx: Internal tools

XAccc7xxx: Unhandled MSBuild Exceptions

Exceptions that have not been gracefully handled yet. Ideally these will be fixed or replaced with better errors in the future.

These take the form of `XACCC7NNN`, where `CCC` is a 3 character code denoting the MSBuild Task that is throwing the exception, and `NNN` is a 3 digit number indicating the type of the unhandled `Exception`.

Tasks:

- `A2C` - `Aapt2Compile`
- `A2L` - `Aapt2Link`
- `AAS` - `AndroidApkSigner`
- `ACD` - `AndroidCreateDebugKey`
- `ACM` - `AppendCustomMetadataToItemGroup`
- `ADB` - `Adb`
- `AJV` - `AdjustJavacVersionArguments`
- `AOT` - `Aot`
- `APT` - `Aapt`
- `ASP` - `AndroidSignPackage`
- `AZA` - `AndroidZipAlign`
- `BAB` - `BuildAppBundle`
- `BAS` - `BuildApkSet`
- `BBA` - `BuildBaseAppBundle`

- BGN - BindingsGenerator
- BLD - BuildApk
- CAL - CreateAdditionalLibraryResourceCache
- CAR - CalculateAdditionalResourceCacheDirectories
- CCR - CopyAndConvertResources
- CCV - ConvertCustomView
- CDF - ConvertDebuggingFiles
- CDJ - CheckDuplicateJavaLibraries
- CFI - CheckForInvalidResourceFileNames
- CFR - CheckForRemovedItems
- CGJ - CopyGeneratedJavaResourceClasses
- CGS - CheckGoogleSdkRequirements
- CIC - CopyIfChanged
- CIL - CilStrip
- CLA - CollectLibraryAssets
- CLC - CalculateLayoutCodeBehind
- CLP - ClassParse
- CLR - CreateLibraryResourceArchive
- CMD - CreateMultiDexMainDexClassList
- CML - CreateManagedLibraryResourceArchive
- CMM - CreateMsymManifest
- CNA - CompileNativeAssembly
- CNE - CollectNonEmptyDirectories
- CNL - CreateNativeLibraryArchive
- CPD - CalculateProjectDependencies
- CPF - CollectPdbFiles
- CPI - CheckProjectItems
- CPR - CopyResource
- CPT - ComputeHash
- CRC - ConvertResourcesCases
- CRM - CreateResgenManifest
- CRN - Crunch
- CRP - AndroidComputeResPaths
- CTD - CreateTemporaryDirectory
- CTX - CompileToDalvik
- DES - Desugar
- DJL - DetermineJavaLibrariesToCompile
- DX8 - D8

- FD - FastDeploy
- FLB - FindLayoutsToBind
- FLT - FilterAssemblies
- GAD - GetAndroidDefineConstants
- GAP - GetAndroidPackageName
- GAR - GetAdditionalResourcesFromAssemblies
- GAS - GetAppSettingsDirectory
- GCB - GenerateCodeBehindForLayout
- GCJ - GetConvertedJavaLibraries
- GEP - GetExtraPackages
- GFT - GetFilesThatExist
- GIL - GetImportedLibraries
- GJP - GetJavaPlatformJar
- GJS - GenerateJavaStubs
- GLB - GenerateLayoutBindings
- GLR - GenerateLibraryResources
- GMA - GenerateManagedAidlProxies
- GMJ - GetMonoPlatformJar
- GPM - GeneratePackageManagerJava
- GRD - GenerateResourceDesigner
- IAS - InstallApkSet
- IJD - ImportJavaDoc
- JDC - JavaDoc
- JVC - Javac
- JTX - JarToXml
- KEY - KeyTool
- LAS - LinkApplicationSharedLibraries
- LEF - LogErrorsForFiles
- LNK - LinkAssemblies
- LNS - LinkAssembliesNoShrink
- LNT - Lint
- LWF - LogWarningsForFiles
- MBN - MakeBundleNativeCodeExternal
- MDC - MDoc
- PAI - PrepareAbiItems
- PAW - ParseAndroidWearProjectAndManifest
- PRO - Proguard
- PWA - PrepareWearApplicationFiles

- R8D - R8
- RAM - ReadAndroidManifest
- RAR - ReadAdditionalResourcesFromAssemblyCache
- RAT - ResolveAndroidTooling
- RDF - RemoveDirFixed
- RIL - ReadImportedLibrariesCache
- RJJ - ResolveJdkJvmPath
- RLC - ReadLibraryProjectImportsCache
- RLP - ResolveLibraryProjectImports
- RRA - RemoveRegisterAttribute
- RSA - ResolveAssemblies
- RSD - ResolveSdks
- RUF - RemoveUnknownFiles
- SPL - SplitProperty
- SVM - SetVsMonoAndroidRegistryKey
- UNZ - Unzip
- VJV - ValidateJavaVersion
- WLF - WriteLockFile

Exceptions:

- 7000 - Other Exception
- 7001 - NullPointerException
- 7002 - ArgumentOutOfRangeException
- 7003 - ArgumentNullException
- 7004 - ArgumentException
- 7005 - FormatException
- 7006 - IndexOutOfRangeException
- 7007 - InvalidCastException
- 7008 - ObjectDisposedException
- 7009 - InvalidOperationException
- 7010 - InvalidProgramException
- 7011 - KeyNotFoundException
- 7012 - TaskCanceledException
- 7013 - OperationCanceledException
- 7014 - OutOfMemoryException
- 7015 - NotSupportedException
- 7016 - StackOverflowException

- 7017 - `TimeoutException`
- 7018 - `TypeInitializationException`
- 7019 - `UnauthorizedAccessException`
- 7020 - `ApplicationException`
- 7021 - `KeyNotFoundException`
- 7022 - `PathTooLongException`
- 7023 - `DirectoryNotFoundException`
- 7024 - `IOException`
- 7025 - `DriveNotFoundException`
- 7026 - `EndOfStreamException`
- 7027 - `FileLoadException`
- 7028 - `FileNotFoundException`
- 7029 - `PipeException`

XA8xxx: Linker Step Errors

- [XA8000/IL8000](#): Could not find Android Resource '@anim/enterfromright'. Please update `@(AndroidResource)` to add the missing resource.

XA9xxx: Licensing

Removed messages

Removed in Xamarin.Android 10.4

- XA5215: Duplicate Resource found for {elementName}. Duplicates are in {filenames}
- XA5216: Resource entry {elementName} is already defined in {filename}

Removed in Xamarin.Android 10.3

- [XA0110](#): Disabling `$(AndroidExplicitCrunch)` as it is not supported by `aapt2`. If you wish to use `$(AndroidExplicitCrunch)` please set `$(AndroidUseAapt2)` to false.

Removed in Xamarin.Android 10.2

- [XA0120](#): Failed to use SHA1 hash algorithm

Removed in Xamarin.Android 9.3

- [XA0114](#): Google Play requires that application updates must use a `$(TargetFrameworkVersion)` of v8.0 (API level 26) or above.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error JAVAC0000

Article • 04/17/2024

Example messages

```
error JAVAC0000: Foo.java(1,8): javac error: class, interface, or enum expected
```

```
error JAVAC0000: Foo.java(1,41): javac error: ';' expected
```

Issue

During a normal .NET for Android build, Java source code is generated and compiled. This message indicates that [javac](#), the Java programming language compiler, failed to compile Java source code.

Solution

If you have Java source code in your project with a build action of `AndroidJavaSource`, verify your Java syntax is correct.

Consider submitting a [bug](#) if you are getting this error under normal circumstances.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android errors and warnings reference

Article • 04/17/2024

ADBxxxx: ADB tooling

- [ADB0000](#): Generic `adb` error/warning.
- [ADB0010](#): Generic `adb` APK installation error/warning.
- [ADB0020](#): The package does not support the CPU architecture of this device.
- [ADB0030](#): APK installation failed due to a conflict with the existing package.
- [ADB0040](#): The device does not support the minimum SDK level specified in the manifest.
- [ADB0050](#): Package {packageName} already exists on device.
- [ADB0060](#): There is not enough storage space on the device to store package: {packageName}. Free up some space and try again.

ANDXXxxxx: Generic Android tooling

- [ANDAS0000](#): Generic `apksigner` error/warning.
- [ANDJS0000](#): Generic `jarsigner` error/warning.
- [ANDKT0000](#): Generic `keytool` error/warning.
- [ANDZA0000](#): Generic `zipalign` error/warning.

APTxxxx: AAPT tooling

- [APT0000](#): Generic `aapt` error/warning.
- [APT0001](#): Unknown option `{option}`. Please check ``$(AndroidAapt2CompileExtraArgs)`` and ``$(AndroidAapt2LinkExtraArgs)`` to see if they include any ``aapt`` command line arguments that are no longer valid for ``aapt2`` and ensure that all other arguments are valid for `aapt2`.
- [APT0002](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0003](#): Invalid file name: It must contain only `[^a-zA-Z0-9_-.]+`.
- [APT0004](#): Invalid file name: It must start with either A-Z or a-z or an underscore.
- [APT2264](#): The system cannot find the file specified. (2).
- [APT2265](#): The system cannot find the file specified. (2).

JAVAXxxx: Java Tool

- [JAVA0000](#): Generic Java tool error

JAVACxxxx: Java compiler

- [JAVAC0000](#): Generic Java compiler error

XA0xxx: Environment issue or missing tooling

- [XA0000](#): Could not determine `$(AndroidApiLevel)` or `$(TargetFrameworkVersion)`.
- [XA0001](#): Invalid or unsupported `$(TargetFrameworkVersion)` value.
- [XA0002](#): Could not find mono.android.jar
- [XA0003](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. It must be an integer value.
- [XA0004](#): Invalid ``android:versionCode`` value `{code}` in ``AndroidManifest.xml``. The value must be in the range of 0 to `{maxVersionCode}`.
- [XA0030](#): Building with JDK version `{versionNumber}` is not supported.
- [XA0031](#): Java SDK `{requiredJavaForFrameworkVersion}` or above is required when using `$(TargetFrameworkVersion)` `{targetFrameworkVersion}`.
- [XA0032](#): Java SDK `{requiredJavaForBuildTools}` or above is required when using Android SDK Build-Tools `{buildToolsVersion}`.
- [XA0033](#): Failed to get the Java SDK version because the returned value does not appear to contain a valid version number.
- [XA0034](#): Failed to get the Java SDK version.
- [XA0035](#): Failed to determine the Android ABI for the project.
- [XA0036](#): `$(AndroidSupportedAbis)` is not supported in .NET 6 and higher.
- [XA0100](#): `EmbeddedNativeLibrary` is invalid in Android Application projects. Please use `AndroidNativeLibrary` instead.
- [XA0101](#): warning XA0101: `@(Content)` build action is not supported.
- [XA0102](#): Generic `lint` Warning.
- [XA0103](#): Generic `lint` Error.
- [XA0104](#): Invalid value for ``$(AndroidSequencePointsMode)``
- [XA0105](#): The `$(TargetFrameworkVersion)` for a library is greater than the `$(TargetFrameworkVersion)` for the application project.
- [XA0107](#): `{Assmebly}` is a Reference Assembly.
- [XA0108](#): Could not get version from `lint`.
- [XA0109](#): Unsupported or invalid `$(TargetFrameworkVersion)` value of 'v4.5'.
- [XA0111](#): Could not get the `aapt2` version. Please check it is installed correctly.

- [XA0112](#): `aapt2` is not installed. Disabling `aapt2` support. Please check it is installed correctly.
- [XA0113](#): Google Play requires that new applications and updates must use a TargetFrameworkVersion of v11.0 (API level 30) or above.
- [XA0115](#): Invalid value 'armeabi' in \$(AndroidSupportedAbis). This ABI is no longer supported. Please update your project properties to remove the old value. If the properties page does not show an 'armeabi' checkbox, un-check and re-check one of the other ABIs and save the changes.
- [XA0116](#): Unable to find `EmbeddedResource` named `{ResourceName}`.
- [XA0117](#): The TargetFrameworkVersion {TargetFrameworkVersion} is deprecated. Please update it to be v4.4 or higher.
- [XA0118](#): Could not parse '{TargetMoniker}'
- [XA0119](#): A non-ideal configuration was found in the project.
- [XA0121](#): Assembly '{assembly}' is using '[assembly: Java.Interop.JavaLibraryReferenceAttribute]', which is no longer supported. Use a newer version of this NuGet package or notify the library author.
- [XA0122](#): Assembly '{assembly}' is using a deprecated attribute '[assembly: Java.Interop.DoNotPackageAttribute]'. Use a newer version of this NuGet package or notify the library author.
- [XA0123](#): Removing {issue} from {propertyName}. Lint {version} does not support this check.
- [XA0125](#): `{Project}` is using a deprecated debug information level. Set the debugging information to Portable in the Visual Studio project property pages or edit the project file in a text editor and set the 'DebugType' MSBuild property to 'portable' to use the newer, cross-platform debug information level. If this file comes from a NuGet package, update to a newer version of the NuGet package or notify the library author.
- [XA0126](#): Error installing FastDev Tools. This device does not support Fast Deployment. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0127](#): There was an issue deploying {destination} using {FastDevTool}. We encountered the following error {output}. Please rebuild your app using `EmbedAssembliesIntoApk = True`.
- [XA0128](#): Stdio Redirection is enabled. Please disable it to use Fast Deployment.
- [XA0129](#): Error deploying `{File}`. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0130](#): Sorry. Fast deployment is only supported on devices running Android 5.0 (API level 21) or higher. Please disable fast deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.

- [XA0131](#): The 'run-as' tool has been disabled on this device. Either enable it by activating the developer options on the device or by setting `ro.boot.disable_runas` to `false`.
- [XA0132](#): The package was not installed. Please check you do not have it installed under any other user. If the package does show up on the device, try manually uninstalling it then try again. You should be able to uninstall the app via the Settings app on the device.
- [XA0133](#): The 'run-as' tool required by the Fast Deployment system has been disabled on this device by the manufacturer. Please disable Fast Deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.
- [XA0134](#): The application does not have the 'android:debuggable' attribute set in the AndroidManifest.xml. This is required in order for Fast Deployment to work. This is normally enabled by default by the .NET for Android build system for Debug builds.
- [XA0135](#): The package is a 'system' application. These are applications which install under the 'system' user on a device. These types of applications cannot use 'run-as'.
- [XA0136](#): The currently installation of the package is corrupt. Please manually uninstall the package from all the users on device and try again. If that does not work you can disable Fast Deployment.
- [XA0137](#): The 'run-as' command failed with '{0}'. Fast Deployment is not currently supported on this device. Please file an issue with the exact error message using the 'Help->Send Feedback->Report a Problem' menu item in Visual Studio or 'Help->Report a Problem' in Visual Studio for Mac.
- [XA0138](#): `%(AndroidAsset.AssetPack)` and `%(AndroidAsset.AssetPack)` item metadata are only supported when `$(AndroidApplication)` is `true`.
- [XA0139](#): `@(AndroidAsset) {0}` has invalid `DeliveryType` metadata of `{1}`. Supported values are `installtime`, `ondemand` or `fastfollow`
- [XA0140](#):
- [XA0141](#): NuGet package '{0}' version '{1}' contains a shared library '{2}' which is not correctly aligned. See <https://developer.android.com/guide/practices/page-sizes> for more details
- [XA0142](#): Command '{0}' failed.\n{1}

XA1xxx: Project related

- [XA1000](#): There was a problem parsing {file}. This is likely due to incomplete or invalid XML.

- [XA1001](#): AndroidResgen: Warning while updating Resource XML '{filename}': {Message}
- [XA1002](#): The closest match found for '{customViewName}' is '{customViewLookupName}', but the capitalization does not match. Please correct the capitalization.
- [XA1003](#): '{zip}' does not exist. Please rebuild the project.
- [XA1004](#): There was an error opening {filename}. The file is probably corrupt. Try deleting it and building again.
- [XA1005](#): Attempting basic type name matching for element with ID '{id}' and type '{managedType}'
- [XA1006](#): The TargetFrameworkVersion (Android API level {compileSdk}) is higher than the targetSdkVersion ({targetSdk}).
- [XA1007](#): The minSdkVersion ({minSdk}) is greater than targetSdkVersion. Please change the value such that minSdkVersion is less than or equal to targetSdkVersion ({targetSdk}).
- [XA1008](#): The TargetFrameworkVersion (Android API level {compileSdk}) is lower than the targetSdkVersion ({targetSdk}).
- [XA1009](#): The {assembly} is Obsolete. Please upgrade to {assembly} {version}
- [XA1010](#): Invalid `\$(AndroidManifestPlaceholders)` value for Android manifest placeholders. Please use `key1=value1;key2=value2` format. The specified value was: {placeholders}
- [XA1011](#): Using ProGuard with the D8 DEX compiler is no longer supported. Please set the code shrinker to 'r8' in the Visual Studio project property pages or edit the project file in a text editor and set the 'AndroidLinkTool' MSBuild property to 'r8'.
- XA1012: Included layout root element override ID '{id}' is not valid.
- XA1013: Failed to parse ID of node '{name}' in the layout file '{file}'.
- XA1014: JAR library references with identical file names but different contents were found: {libraries}. Please remove any conflicting libraries from EmbeddedJar, InputJar and AndroidJavaLibrary.
- XA1015: More than one Android Wear project is specified as the paired project. It can be at most one.
- XA1016: Target Wear application's project '{project}' does not specify required 'AndroidManifest' project property.
- XA1017: Target Wear application's AndroidManifest.xml does not specify required 'package' attribute.
- XA1018: Specified AndroidManifest file does not exist: {file}.
- XA1019: `LibraryProjectProperties` file `{file}` is located in a parent directory of the bindings project's intermediate output directory. Please adjust the path to use the original `project.properties` file directly from the Android library project directory.

- XA1020: At least one Java library is required for binding. Check that a Java library is included in the project and has the appropriate build action: 'LibraryProjectZip' (for AAR or ZIP), 'EmbeddedJar', 'InputJar' (for JAR), or 'LibraryProjectProperties' (project.properties).
- XA1021: Specified source Java library not found: {file}
- XA1022: Specified reference Java library not found: {file}
- [XA1023](#): Using the DX DEX Compiler is deprecated.
- [XA1024](#): Ignoring configuration file 'Foo.dll.config'. .NET configuration files are not supported in .NET for Android projects that target .NET 6 or higher.
- [XA1025](#): The experimental 'Hybrid' value for the 'AndroidAotMode' MSBuild property is not currently compatible with the armeabi-v7a target ABI.
- [XA1027](#): The 'EnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1028](#): The 'AndroidEnableProguard' MSBuild property is set to 'true' and the 'AndroidLinkTool' MSBuild property is empty, so 'AndroidLinkTool' will default to 'proguard'.
- [XA1029](#): The 'AotAssemblies' MSBuild property is deprecated. Edit the project file in a text editor to remove this property, and use the 'RunAOTCompilation' MSBuild property instead.
- [XA1031](#): The 'AndroidHttpClientHandlerType' has an invalid value.
- [XA1032](#): Failed to resolve '{0}' from '{1}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1033](#): Could not resolve '{0}'. Please check your `AndroidHttpClientHandlerType` setting.
- [XA1035](#): The 'BundleAssemblies' property is deprecated and it has no effect on the application build. Equivalent functionality is implemented by the 'AndroidUseAssemblyStore' and 'AndroidEnableAssemblyCompression' properties.
- [XA1036](#): AndroidManifest.xml //uses-sdk/@android:minSdkVersion '29' does not match the \$(SupportedOSPlatformVersion) value '21' in the project file (if there is no \$(SupportedOSPlatformVersion) value in the project file, then a default value has been assumed). Either change the value in the AndroidManifest.xml to match the \$(SupportedOSPlatformVersion) value, or remove the value in the AndroidManifest.xml (and add a \$(SupportedOSPlatformVersion) value to the project file if it doesn't already exist).
- [XA1037](#): The '{0}' MSBuild property is deprecated and will be removed in .NET {1}. See <https://aka.ms/net-android-deprecations> for more details.
- [XA1038](#): The '{0}' MSBuild property has an invalid value. Value values are {1}.

XA2xxx: Linker

- [XA2000](#): Use of AppDomain.CreateDomain() detected in assembly: {assembly}. .NET 6 will only support a single AppDomain, so this API will no longer be available in .NET for Android once .NET 6 is released.
- [XA2001](#): Source file '{filename}' could not be found.
- [XA2002](#): Can not resolve reference: `{missing}`, referenced by {assembly}. Perhaps it doesn't exist in the Mono for Android profile?
- XA2006: Could not resolve reference to '{member}' (defined in assembly '{assembly}') with scope '{scope}'. When the scope is different from the defining assembly, it usually means that the type is forwarded.
- XA2007: Exception while loading assemblies: {exception}
- XA2008: In referenced assembly {assembly}, Java.Interop.DoNotPackageAttribute requires non-null file name.

XA3xxx: Unmanaged code compilation

- XA3001: Could not AOT the assembly: {assembly}
- XA3002: Invalid AOT mode: {mode}
- XA3003: Could not strip IL of assembly: {assembly}
- XA3004: Android NDK r10d is buggy and provides an incompatible x86_64 libm.so.
- XA3005: The detected Android NDK version is incompatible with the targeted LLVM configuration.
- XA3006: Could not compile native assembly file: {file}
- XA3007: Could not link native shared library: {library}

XA4xxx: Code generation

- XA4209: Failed to generate Java type for class: {managedType} due to {exception}
- XA4210: Please add a reference to Mono.Android.Export.dll when using ExportAttribute or ExportFieldAttribute.
- XA4211: AndroidManifest.xml //uses-sdk/@android:targetSdkVersion '{targetSdk}' is less than \$(TargetFrameworkVersion) '{targetFramework}'. Using API-{targetFrameworkApi} for ACW compilation.
- XA4213: The type '{type}' must provide a public default constructor
- [XA4214](#): The managed type `Library1.Class1` exists in multiple assemblies: Library1, Library2. Please refactor the managed type names in these assemblies so that they are not identical.
- [XA4215](#): The Java type `com.contoso.library1.Class1` is generated by more than one managed type. Please change the [Register] attribute so that the same Java type is not emitted.

- [XA4216](#): The deployment target '19' is not supported (the minimum is '21'). Please increase the \$(SupportedOSPlatformVersion) property value in your project file.
- XA4217: Cannot override Kotlin-generated method '{method}' because it is not a valid Java method name. This method can only be overridden from Kotlin.
- [XA4218](#): Unable to find //manifest/application/uses-library at path: {path}
- XA4219: Cannot find binding generator for language {language} or {defaultLanguage}.
- XA4220: Partial class item '{file}' does not have an associated binding for layout '{layout}'.
- XA4221: No layout binding source files were generated.
- XA4222: No widgets found for layout ({layoutFiles}).
- XA4223: Malformed full class name '{name}'. Missing namespace.
- XA4224: Malformed full class name '{name}'. Missing class name.
- XA4225: Widget '{widget}' in layout '{layout}' has multiple instances with different types. The property type will be set to: {type}
- XA4226: Resource item '{file}' does not have the required metadata item '{metadataName}'.
- XA4228: Unable to find specified //activity-alias/@android:targetActivity: '{targetActivity}'
- XA4229: Unrecognized `TransformFile` root element: {element}.
- XA4230: Error parsing XML: {exception}
- [XA4231](#): The Android class parser value 'jar2xml' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'class-parse'.
- [XA4232](#): The Android code generation target 'XamarinAndroid' is deprecated and will be removed in a future version of .NET for Android. Update the project properties to use 'XJavaInterop1'.
- [XA4234](#): '<{item}>' item '{itemspec}' is missing required attribute '{name}'.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id'.
- [XA4236](#): Cannot download Maven artifact '{group}:{artifact}'. - {jar}: {exception} - {aar}: {exception}
- [XA4237](#): Cannot download POM file for Maven artifact '{artifact}'. - {exception}
- [XA4239](#): Unknown Maven repository: '{repository}'.
- [XA4241](#): Java dependency '{artifact}' is not satisfied.
- [XA4242](#): Java dependency '{artifact}' is not satisfied. Microsoft maintains the NuGet package '{nugetId}' that could fulfill this dependency.
- [XA4243](#): Attribute '{name}' is required when using '{name}' for '{element}' item '{itemspec}'.
- [XA4244](#): Attribute '{name}' cannot be empty for '{element}' item '{itemspec}'.
- [XA4245](#): Specified POM file '{file}' does not exist.

- [XA4246](#): Could not parse POM file '{file}'. - {exception}
- [XA4247](#): Could not resolve POM file for artifact '{artifact}'.
- [XA4248](#): Could not find NuGet package '{nugetId}' version '{version}' in lock file. Ensure NuGet Restore has run since this `<PackageReference>` was added.
- [XA4235](#): Maven artifact specification '{artifact}' is invalid. The correct format is 'group_id:artifact_id:version'.
- XA4300: Native library '{library}' will not be bundled because it has an unsupported ABI.
- [XA4301](#): Apk already contains the item `xxx`.
- [XA4302](#): Unhandled exception merging `AndroidManifest.xml`: {ex}
- [XA4303](#): Error extracting resources from "{assemblyPath}": {ex}
- [XA4304](#): ProGuard configuration file '{file}' was not found.
- [XA4305](#): Multidex is enabled, but `\$\_AndroidMainDexListFile` is empty.
- [XA4306](#): R8 does not support `@(MultiDexMainDexList)` files when `android:minSdkVersion >= 21`
- [XA4307](#): Invalid ProGuard configuration file.
- [XA4308](#): Failed to generate type maps
- [XA4309](#): 'MultiDexMainDexList' file '{file}' does not exist.
- [XA4310](#): `\$(AndroidSigningKeyStore)` file '{keystore}` could not be found.
- XA4311: The application won't contain the paired Wear package because the Wear application package APK is not created yet. If building on the command line, be sure to build the "SignAndroidPackage" target.
- [XA4312](#): Referencing an Android Wear application project from an Android application project is deprecated.
- [XA4313](#): Framework assembly has been deprecated.
- [XA4314](#): `$(Property)` is empty. A value for `$(Property)` should be provided.
- [XA4315](#): Ignoring {file}. Manifest does not have the required 'package' attribute on the manifest element.

XA5xxx: GCC and toolchain

- XA5101: Missing Android NDK toolchains directory '{path}'. Please install the Android NDK.
- XA5102: Conversion from assembly to native code failed. Exit code {exitCode}
- XA5103: NDK C compiler exited with an error. Exit code {0}
- XA5104: Could not locate the Android NDK.
- XA5105: Toolchain utility '{utility}' for target {arch} was not found. Tried in path: "{path}"
- XA5201: NDK linker exited with an error. Exit code {0}
- [XA5205](#): Cannot find `{ToolName}` in the Android SDK.

- [XA5207](#): Could not find android.jar for API level `{compileSdk}`.
- [XA5211](#): Embedded Wear app package name differs from handheld app package name (`{wearPackageName} != {handheldPackageName}`).
- [XA5213](#): `java.lang.OutOfMemoryError`. Consider increasing the value of `$(JavaMaximumHeapSize)`. Java ran out of memory while executing '`{tool}` `{arguments}`'
- [XA5300](#): The Android/Java SDK Directory could not be found.
- [XA5301](#): Failed to generate Java type for class: `{managedType}` due to `MAX_PATH`: `{exception}`
- [XA5302](#): Two processes may be building this project at once. Lock file exists at path: `{path}`

XA6xxx: Internal tools

XAccc7xxx: Unhandled MSBuild Exceptions

Exceptions that have not been gracefully handled yet. Ideally these will be fixed or replaced with better errors in the future.

These take the form of `XACCC7NNN`, where `CCC` is a 3 character code denoting the MSBuild Task that is throwing the exception, and `NNN` is a 3 digit number indicating the type of the unhandled `Exception`.

Tasks:

- `A2C` - `Aapt2Compile`
- `A2L` - `Aapt2Link`
- `AAS` - `AndroidApkSigner`
- `ACD` - `AndroidCreateDebugKey`
- `ACM` - `AppendCustomMetadataToItemGroup`
- `ADB` - `Adb`
- `AJV` - `AdjustJavacVersionArguments`
- `AOT` - `Aot`
- `APT` - `Aapt`
- `ASP` - `AndroidSignPackage`
- `AZA` - `AndroidZipAlign`
- `BAB` - `BuildAppBundle`
- `BAS` - `BuildApkSet`
- `BBA` - `BuildBaseAppBundle`

- BGN - BindingsGenerator
- BLD - BuildApk
- CAL - CreateAdditionalLibraryResourceCache
- CAR - CalculateAdditionalResourceCacheDirectories
- CCR - CopyAndConvertResources
- CCV - ConvertCustomView
- CDF - ConvertDebuggingFiles
- CDJ - CheckDuplicateJavaLibraries
- CFI - CheckForInvalidResourceFileNames
- CFR - CheckForRemovedItems
- CGJ - CopyGeneratedJavaResourceClasses
- CGS - CheckGoogleSdkRequirements
- CIC - CopyIfChanged
- CIL - CilStrip
- CLA - CollectLibraryAssets
- CLC - CalculateLayoutCodeBehind
- CLP - ClassParse
- CLR - CreateLibraryResourceArchive
- CMD - CreateMultiDexMainDexClassList
- CML - CreateManagedLibraryResourceArchive
- CMM - CreateMsymManifest
- CNA - CompileNativeAssembly
- CNE - CollectNonEmptyDirectories
- CNL - CreateNativeLibraryArchive
- CPD - CalculateProjectDependencies
- CPF - CollectPdbFiles
- CPI - CheckProjectItems
- CPR - CopyResource
- CPT - ComputeHash
- CRC - ConvertResourcesCases
- CRM - CreateResgenManifest
- CRN - Crunch
- CRP - AndroidComputeResPaths
- CTD - CreateTemporaryDirectory
- CTX - CompileToDalvik
- DES - Desugar
- DJL - DetermineJavaLibrariesToCompile
- DX8 - D8

- FD - FastDeploy
- FLB - FindLayoutsToBind
- FLT - FilterAssemblies
- GAD - GetAndroidDefineConstants
- GAP - GetAndroidPackageName
- GAR - GetAdditionalResourcesFromAssemblies
- GAS - GetAppSettingsDirectory
- GCB - GenerateCodeBehindForLayout
- GCJ - GetConvertedJavaLibraries
- GEP - GetExtraPackages
- GFT - GetFilesThatExist
- GIL - GetImportedLibraries
- GJP - GetJavaPlatformJar
- GJS - GenerateJavaStubs
- GLB - GenerateLayoutBindings
- GLR - GenerateLibraryResources
- GMA - GenerateManagedAidlProxies
- GMJ - GetMonoPlatformJar
- GPM - GeneratePackageManagerJava
- GRD - GenerateResourceDesigner
- IAS - InstallApkSet
- IJD - ImportJavaDoc
- JDC - JavaDoc
- JVC - Javac
- JTX - JarToXml
- KEY - KeyTool
- LAS - LinkApplicationSharedLibraries
- LEF - LogErrorsForFiles
- LNK - LinkAssemblies
- LNS - LinkAssembliesNoShrink
- LNT - Lint
- LWF - LogWarningsForFiles
- MBN - MakeBundleNativeCodeExternal
- MDC - MDoc
- PAI - PrepareAbiItems
- PAW - ParseAndroidWearProjectAndManifest
- PRO - Proguard
- PWA - PrepareWearApplicationFiles

- R8D - R8
- RAM - ReadAndroidManifest
- RAR - ReadAdditionalResourcesFromAssemblyCache
- RAT - ResolveAndroidTooling
- RDF - RemoveDirFixed
- RIL - ReadImportedLibrariesCache
- RJJ - ResolveJdkJvmPath
- RLC - ReadLibraryProjectImportsCache
- RLP - ResolveLibraryProjectImports
- RRA - RemoveRegisterAttribute
- RSA - ResolveAssemblies
- RSD - ResolveSdks
- RUF - RemoveUnknownFiles
- SPL - SplitProperty
- SVM - SetVsMonoAndroidRegistryKey
- UNZ - Unzip
- VJV - ValidateJavaVersion
- WLF - WriteLockFile

Exceptions:

- 7000 - Other Exception
- 7001 - NullPointerException
- 7002 - ArgumentOutOfRangeException
- 7003 - ArgumentNullException
- 7004 - ArgumentException
- 7005 - FormatException
- 7006 - IndexOutOfRangeException
- 7007 - InvalidCastException
- 7008 - ObjectDisposedException
- 7009 - InvalidOperationException
- 7010 - InvalidProgramException
- 7011 - KeyNotFoundException
- 7012 - TaskCanceledException
- 7013 - OperationCanceledException
- 7014 - OutOfMemoryException
- 7015 - NotSupportedException
- 7016 - StackOverflowException

- 7017 - `TimeoutException`
- 7018 - `TypeInitializationException`
- 7019 - `UnauthorizedAccessException`
- 7020 - `ApplicationException`
- 7021 - `KeyNotFoundException`
- 7022 - `PathTooLongException`
- 7023 - `DirectoryNotFoundException`
- 7024 - `IOException`
- 7025 - `DriveNotFoundException`
- 7026 - `EndOfStreamException`
- 7027 - `FileLoadException`
- 7028 - `FileNotFoundException`
- 7029 - `PipeException`

XA8xxx: Linker Step Errors

- [XA8000/IL8000](#): Could not find Android Resource '@anim/enterfromright'. Please update `@(AndroidResource)` to add the missing resource.

XA9xxx: Licensing

Removed messages

Removed in Xamarin.Android 10.4

- XA5215: Duplicate Resource found for {elementName}. Duplicates are in {filenames}
- XA5216: Resource entry {elementName} is already defined in {filename}

Removed in Xamarin.Android 10.3

- [XA0110](#): Disabling `$(AndroidExplicitCrunch)` as it is not supported by `aapt2`. If you wish to use `$(AndroidExplicitCrunch)` please set `$(AndroidUseAapt2)` to false.

Removed in Xamarin.Android 10.2

- [XA0120](#): Failed to use SHA1 hash algorithm

Removed in Xamarin.Android 9.3

- [XA0114](#): Google Play requires that application updates must use a `$(TargetFrameworkVersion)` of v8.0 (API level 26) or above.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0000

Article • 04/17/2024

Issue

An unhandled exception (or error case) was encountered in one of .NET for Android's MSBuild tasks.

Solution

Consider submitting a [bug](#) if you are getting this warning under normal circumstances.

Specific scenario 1

Issue

An exception was thrown and logged, including a C# exception stack trace showing where the failure occurred.

Solution

Likely, the only option is to submit a [bug](#).

Specific scenario 2

Issue

You are using a `$(TargetFrameworkVersion)` which cannot be mapped to a valid android api-level.

Solution

Check you have selected a valid `$(TargetFrameworkVersion)` and the required android api-level is installed in `$(AndroidSdkDirectory)\platforms`.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0001

Article • 04/17/2024

Issue

This error means you are using a `$(TargetFrameworkVersion)` which is either invalid or unsupported.

Solution

Check you have selected a valid `$(TargetFrameworkVersion)` and the required android api-level is installed in `$(AndroidSdkDirectory)\platforms`.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0002

Article • 04/17/2024

Issue

This error means the build system could not locate the *mono.android.jar* file. This probably means your installation is corrupt in some way.

Solution

Please reinstall .NET for Android.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0003

Article • 04/17/2024

Issue

This error means the value for `android:versionCode` in the `AndroidManifest.xml` file is not an integer value.

Google requires that the value be an integer within the range of 0 to 2100000000. See <https://developer.android.com/studio/publish/versioning> for more details.

Solution

Correct the `android:versionCode` in the `AndroidManifest.xml` to be an integer in the range of 0 to 2100000000.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error XA0004

Article • 04/17/2024

Issue

This error means the value for `android:versionCode` in the `AndroidManifest.xml` file is not an integer but is not within the value range of 0 to 2100000000.

Google requires that the value be an integer within the range of 0 to 2100000000. See <https://developer.android.com/studio/publish/versioning> for more details.

Solution

Correct the `android:versionCode` in the `AndroidManifest.xml` to be an integer in the range of 0 to 2100000000.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error XA0030

Article • 04/17/2024

Issue

Building with the detected JDK version is not supported. This is due to limitations with the Android SDK tooling.

See <https://aka.ms/xamarin/jdk9-errors> ↗

Solution

Make sure you install a compatible JDK version.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) ↗ | [Get help at Microsoft Q&A](#)

.NET for Android error XA0031

Article • 04/17/2024

Issue

The Android SDK platform you are targeting only works with certain versions of Java. If you get this error, it means either:

1. You don't have a Java SDK installed, or
2. Your Java SDK version is too old or is otherwise not compatible with the targeted Android platform.

Solution

Make sure you install a compatible JDK version, such as the [Microsoft Build of OpenJDK](#).

ⓘ Note

Java SDK 11.0 is required to use `$(TargetFrameworkVersion) v12.0` (API-31) and later, and to use `$(TargetFramework) = net6.0-android31.0` in .NET 6 and later.

[Use of Java SDK 11.0 will break the Android Designer in Visual Studio 16.11 and earlier](#) [↗](#).

Example messages

```
error XA0031: Java SDK 11.0 or above is required when using
$(TargetFrameworkVersion) v12.0.
Download the latest JDK at: https://aka.ms/msopenjdk
Note: the Android Designer is incompatible with Java SDK 11.0:
https://aka.ms/vs2019-and-jdk-11
```

Feedback

Was this page helpful?

[👍 Yes](#)

[👎 No](#)

.NET for Android error XA0032

Article • 04/17/2024

Issue

The Android SDK Build tools only work with certain versions of Java. If you get this error, it means either your Java version is too old or is not compatible with the installed Android SDK.

Solution

Make sure you install a compatible JDK version.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0033

Article • 04/17/2024

Issue

Failed to get the Java SDK version as it does not appear to contain a valid version number. The call to `javac -version` returned a version number which was unknown or could not be parsed.

Solution

Make sure you have a valid Java installation installed.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0034

Article • 04/17/2024

Issue

An exception was raised when .NET for Android tried to get the Java SDK version.

Solution

Make sure you have a valid Java installation installed.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0035

Article • 04/17/2024

Issue

.NET for Android was not able to determine the application's target [Android ABIs](#) as specified by the `.csproj` file.

Solution

Open the project file in [Visual Studio](#) or another text editor and make sure all of the values in the `RuntimeIdentifiers` MSBuild property are valid:

XML

```
<PropertyGroup>
  <RuntimeIdentifiers>android-arm;android-arm64;android-x86;android-
x64</RuntimeIdentifiers>
</PropertyGroup>
```

See the Microsoft documentation on [runtime identifiers](#) for more information.

Example messages

```
error XA0035: Unable to determine the Android ABI from the value 'XYZ'. Edit
the project file in a text editor and set the 'RuntimeIdentifiers' MSBuild
property to contain only valid identifiers for the Android platform.
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0036

Article • 04/17/2024

Issue

The `$(AndroidSupportedAbis)` MSBuild property is no longer supported in .NET 6 and higher.

Solution

Open the project file in [Visual Studio](#) or another text editor and remove `<AndroidSupportedAbis/>`. Use the `RuntimeIdentifiers` MSBuild property instead:

XML

```
<PropertyGroup>
  <RuntimeIdentifiers>android-arm;android-arm64;android-x86;android-
x64</RuntimeIdentifiers>
</PropertyGroup>
```

See the Microsoft documentation on [runtime identifiers](#) for more information.

Example messages

```
warning XA0036: The 'AndroidSupportedAbis' MSBuild property is no longer
supported. Edit the project file in a text editor, remove any uses of
'AndroidSupportedAbis', and use the 'RuntimeIdentifiers' MSBuild property
instead.
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0101

Article • 04/17/2024

Issue

The normal `@(Content)` Build action is not supported (as we haven't figured out how to support it without a possibly costly first-run step).

Solution

If you wish to include additional content in your APK, such as textures or audio for games, you should consider using `@(AndroidAsset)` or `@(EmbeddedResource)`.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0102

Article • 04/17/2024

Issue

This message indicates that `lint` (Android Code Checking Tool) reported an error or warning. `lint` is part of the Android SDK and can be used by .NET for Android to check for Android based issues.

Solution

Learn more about `lint` from the [Android Lint documentation](#).

Learn more about [using lint from .NET for Android](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error XA0103

Article • 04/17/2024

Issue

This message indicates that `lint` (Android Code Checking Tool) reported an error. `lint` is part of the Android SDK and can be used by .NET for Android to check for Android based issues.

Solution

Learn more about `lint` from the [Android Lint documentation](#).

Learn more about [using lint from .NET for Android](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0105

Article • 04/17/2024

Issue

The .NET for Android build will emit a XA0105 warning if it detects that one of the referenced libraries or projects is using a `$(TargetFrameworkVersion)` higher than the project being built. Using libraries which are targeting a higher `$(TargetFrameworkVersion)` can cause unexpected results at runtime, especially when running on older devices.

Solution

The fix is to increase the `$(TargetFrameworkVersion)` for your project to be equal to or higher than the referenced library projects.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0107

Article • 04/17/2024

Issue

The .NET for Android build will emit a XA0107 warning if it is using a "Reference Assembly" during the build, where it shouldn't be. These are sanity checks that likely mean there is a bug in .NET for Android.

XA0107 means that a file such as *System.dll* located within the .NET for Android installation is being passed as input to one of the following MSBuild tasks:

- `<BuildApk />`
- `<ResolveAssemblies />`

Solution

Consider submitting a [bug](#) if you are getting this warning under normal circumstances.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0108

Article • 04/17/2024

Issue

This message indicates that the .NET for Android tooling was unable to determine the version of the `lint` tool. As a result, none of the normally disabled checks which produce false positives will be disabled.

Solution

Consider submitting a [bug](#) if you are getting this warning under normal circumstances.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0109

Article • 04/17/2024

Issue

This warning is raised because android version v4.5 no longer exists. As a result, it is not supported by .NET for Android.

Solution

If you see this warning, you should change your `$(TargetFrameworkVersion)` to be one of the supported [versions](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0111

Article • 04/17/2024

Issue

.NET for Android could not get the AAPT2 version via the command:

```
aapt2 -version
```

`aapt2` did not return an expected value.

Solution

Check for any `<Aapt2ToolPath>` elements in the `.csproj` file and remove them. .NET for Android redistributes its own recommended version of AAPT2, so setting a custom value for the `Aapt2ToolPath` MSBuild property is generally not recommended. Alternatively, try re-installing the .NET for Android workload in Visual Studio.

AAPT is part of the Android SDK and is used internally by .NET for Android to process and compile resources into binary assets. Learn more about AAPT and Android resources from the [Android documentation](#).

AAPT2 is version 2.0 of AAPT. Learn more about AAPT2 from the [Android documentation](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0112

Article • 04/17/2024

Issue

The AAPT2 command-line tool was not found.

Solution

Check for any `<Aapt2ToolPath>` elements in the `.csproj` file and remove them. .NET for Android redistributes its own recommended version of AAPT2, so setting a custom value for the `Aapt2ToolPath` MSBuild property is generally not recommended. Alternatively, try re-installing the .NET for Android workload in Visual Studio.

AAPT is part of the Android SDK and is used internally by .NET for Android to process and compile resources into binary assets. Learn more about AAPT and Android resources from the [Android documentation](#).

AAPT2 is version 2.0 of AAPT. Learn more about AAPT2 from the [Android documentation](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0113

Article • 04/17/2024

Issue

As of November 1st 2021, new applications and application updates uploaded to the Google Play store need to target v11.0 (API 30) or above.

Solution

If you are seeing this warning, you should update the `$(TargetFrameworkVersion)` of your projects to be v11.0 or above.

If you are not targeting the Google Play store and wish to hide these warnings, you can make use of the `/warnasmessages:XA0113` command line switch. Alternatively, add

XML

```
<PropertyGroup>
  <MSBuildWarningsAsMessages>XA0113</MSBuildWarningsAsMessages>
</PropertyGroup>
```

to your `.csproj` file.

[Google Play's target API level requirement](#) [↗] [Target API level requirements for the Play Console](#) [↗]

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) [↗] | [Get help at Microsoft Q&A](#)

.NET for Android error XA0115

Article • 04/17/2024

Example messages

```
Invalid value 'armeabi' in $(AndroidSupportedAbis). This ABI is no longer supported. Please update your project properties to remove the old value. If the properties page does not show an 'armeabi' checkbox, un-check and re-check one of the other ABIs and save the changes.
```

Issue

Due to the [removal of armeabi support in Android NDK r17](#), .NET for Android 9.1 is the last version that supports the armeabi architecture.

Example `.csproj` file element for `$(AndroidSupportedAbis)` that will cause the error:

XML

```
<AndroidSupportedAbis>armeabi;armeabi-v7a;arm64-v8a</AndroidSupportedAbis>
```

Solution

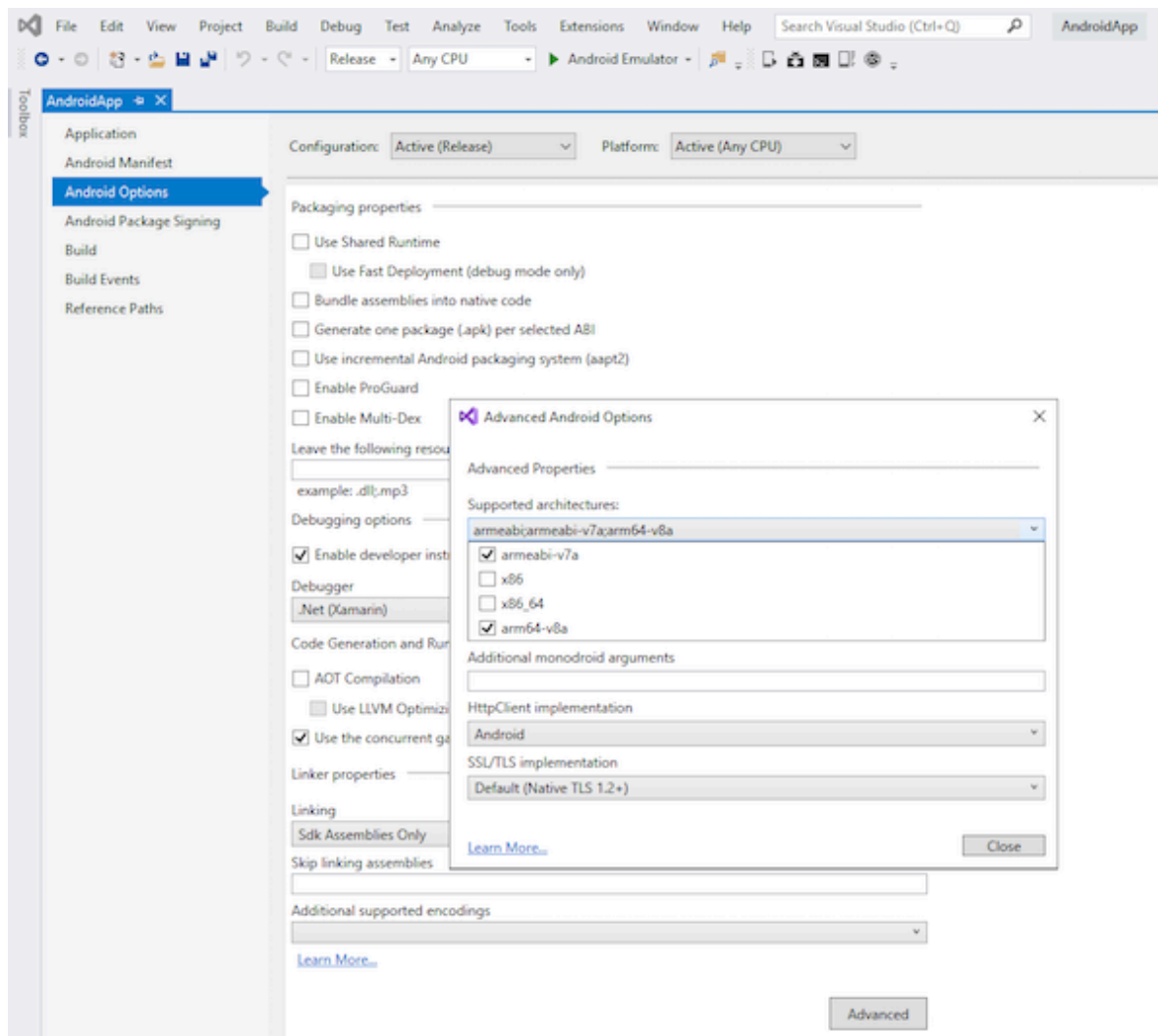
Projects that have this old ABI selected in the `$(AndroidSupportedAbis)` property will need to be updated to remove it before they will build successfully with newer versions of .NET for Android. The newer armeabi-v7a ABI should now be used instead.

The `armeabi` value can be removed from this property either by editing the `.csproj` directly or by updating the setting in the Visual Studio property pages on Windows or macOS.

Updating the setting on Windows

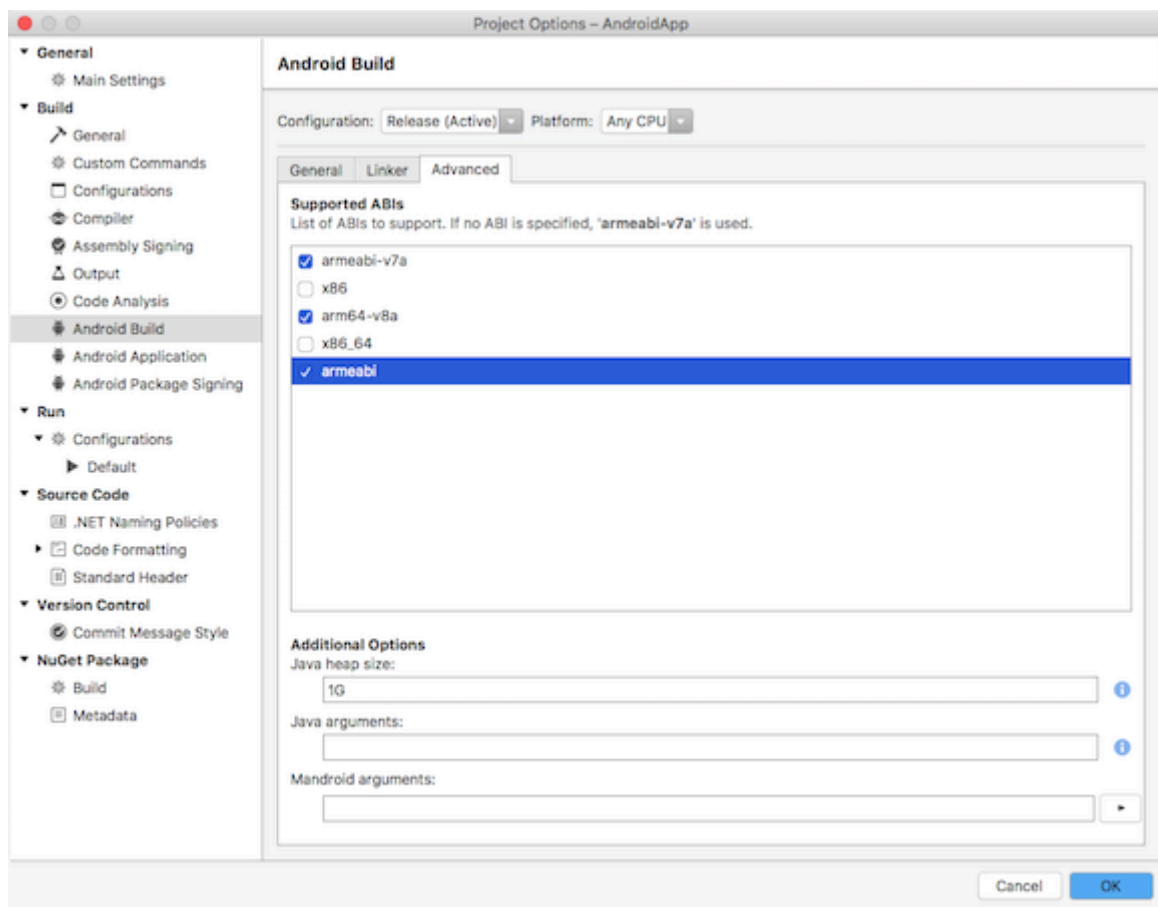
1. Select the project in the **Solution Explorer** and click the **Properties** icon, or right-click the project and select **Properties**.
2. In the side pane, choose **Android Options**.

3. Select the **Advanced** button.
4. The **Supported architectures** list no longer includes an **armeabi** checkbox, so to remove the old armeabi setting, un-check and re-check one of the other ABIs, click the **Close** button, and then save the changes.



Updating the setting on macOS

1. Control-click on the project in the **Solution** pad and select **Options**.
2. In the side pane, choose **Android Build**.
3. Select the **Advanced** tab.
4. In the **Supported ABIs** list, un-check the **armeabi** checkbox and click the **OK** button to save the changes.



Feedback

Was this page helpful?

Yes

No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error XA0116

Article • 04/17/2024

Issue

.NET for Android's build process extracts various `EmbeddedResource` files from its assemblies during a build. These include internal Java source files that are required for a .NET for Android application to start successfully. A XA0116 error would only be encountered if one of these required files is missing.

Solution

Consider submitting a [bug](#) if you are getting this error under normal circumstances.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0117

Article • 04/17/2024

Issue

As the Android platform evolves certain versions will be deprecated and removed. How we decide if a platform is deprecated largely depends on whether the [Android NDK](#) continues to support the platform in question. It also depends on whether the [Android Support Libraries support that platform](#).

If both the Android NDK and the Android Support Libraries have removed support for a particular `apiLevel`, then it is highly likely that support for the API level will be deprecated and removed from .NET for Android.

If you are seeing this warning, it is because your `$(TargetFrameworkVersion)` is set to a value which is likely to be deprecated in the near future.

Solution

To remove this warning, update your `$(TargetFrameworkVersion)` to use the latest stable platform.

Your `android:minSdk` value within *AndroidManifest.xml* can still mention older versions of the android platform.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0118

Article • 04/17/2024

Issue

These warnings indicate an issue with the `$(NuGetTargetMoniker)` MSBuild property in a .NET for Android project. This property is normally generated automatically based on the `$(TargetFrameworkIdentifier)` and `$(TargetFrameworkVersion)` properties, so these warnings should not arise in normal use.

Solution

Consider submitting a [bug](#) if you are getting one of these warnings under normal circumstances.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0119

Article • 04/17/2024

Issue

This warning indicate a non-ideal configuration in your .NET for Android project.

Solution

Remove the following options from `Debug` configurations:

- Linker
 - `<AndroidLinkMode>SdkOnly</AndroidLinkMode>`
 - `<AndroidLinkMode>Full</AndroidLinkMode>`
- AOT
 - `<AotAssemblies>True</AotAssemblies>`
- Code Shrinker
 - `<AndroidEnableProguard>True</AndroidEnableProguard>`
 - `<EnableProguard>True</EnableProguard>`
 - `<AndroidLinkTool>proguard</AndroidLinkTool>`
 - `<AndroidLinkTool>r8</AndroidLinkTool>`
- App Bundles
 - `<AndroidPackageFormat>aab</AndroidPackageFormat>`

Remove the following from `Release` configurations:

- Hot Reload support
 - `<UseInterpreter>true</UseInterpreter>`

DO use the following options for `Debug` configurations:

- `<AndroidLinkMode>None</AndroidLinkMode>`
- `<EmbedAssembliesIntoApk>False</EmbedAssembliesIntoApk>`
- `<UseInterpreter>true</UseInterpreter>`

DO use the following options for `Release` configurations:

- `<EmbedAssembliesIntoApk>True</EmbedAssembliesIntoApk>`
- `<AotAssemblies>True</AotAssemblies>` or in .NET 6
`<RunAOTCompilation>True</RunAOTCompilation>`

Consider submitting a [bug](#) if you are getting one of these warnings under normal circumstances.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error XA0121

Article • 04/17/2024

Issue

The behavior implemented in the `<GetAdditionalResourcesFromAssemblies/>` MSBuild task is no longer supported.

This MSBuild task is a precursor to [Xamarin.Build.Download](#) that enables downloading of Android packages from the internet.

Libraries using any of the following custom assembly-level attributes will encounter this error:

- `IncludeAndroidResourcesFromAttribute`
- `NativeLibraryReferenceAttribute`
- `JavaLibraryReferenceAttribute`

Solution

The [Xamarin Support Libraries](#), can be simply updated to a newer version on NuGet.

Library authors will need to remove usage of these attributes. Their functionality was removed in .NET for Android 10.2.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0122

Article • 04/17/2024

Example messages

```
warning XA0122: Assembly 'Library1' is using a deprecated attribute  
'[assembly: Java.Interop.DoNotPackageAttribute]'. Use a newer version of  
this NuGet package or notify the library author.
```

Issue

The behavior implemented in the `DoNotPackageAttribute` is deprecated:

C#

```
[assembly: Java.Interop.DoNotPackage ("foo.jar")]
```

This would prevent `foo.jar` from being packaged in the app.

Alternatively, you can use the `@(AndroidExternalJavaLibrary)` item group for including `foo.jar`. The java library will only be used at compile time, and will not be packaged in the final Android app.

Solution

Some libraries using this feature can be simply updated to a newer version on NuGet.

Library authors will need to remove usage of this attribute. Its functionality will be removed in a future release.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0125

Article • 04/17/2024

Example messages

```
warning XA0125: 'AndroidApp1.pdb' is using a deprecated debug
information level. Set the debugging information to Portable in the
Visual Studio project property pages or edit the project file in a
text editor and set the 'DebugType' MSBuild property to 'portable' to
use the newer, cross-platform debug information level. If this file
comes from a NuGet package, update to a newer version of the NuGet
package or notify the library author.
```

Issue

Support for *.mdb* or *.pdb* symbols files that were built with the `DebugType` MSBuild property set to `full` or `pdbonly` is now deprecated. This applies to *.mdb* and *.pdb* files in application projects as well as in referenced libraries, including NuGet packages.

Solution

Set `DebugType` to `portable` in the application project as well all library references. `portable` is the recommended setting for all projects from now on. The older `full` and `pdbonly` settings are for older Windows-specific file formats. .NET 6 and higher will not support those older formats.

In Visual Studio, go to **Properties > Build > Advanced** in the project property pages and change **Debugging information** to **Portable**.

In Visual Studio for Mac, go to **Build > Compiler > Debug information** in the project property pages and change **Debug information** to **Portable**.

If the problematic symbol file comes from a NuGet package, update to a newer version of the package or notify the library author.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0126

Article • 04/17/2024

Issue

This issue happens when you are trying to use fast deployment on a device which does not support it. Fast deployment requires features which are not available on devices running API 20 or lower. The Fast Deployment system makes use of the [run-as](#) feature of the Android OS. This feature was either not available or had limited capabilities in API 20 and earlier.

Solution

Disable Fast Deployment by setting `EmbedAssembliesIntoApk = True` in your .csproj. Or turn off `Fast Deployment` in the IDE. You will still be able to debug on the device, all the required files will be packaged inside the .apk.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error XA0127

Article • 04/17/2024

Issue

If you encounter this issue something unexpected happened with the Fast Deployment native tooling. This is most likely to be a native crash.

Solution

To work around the issue disable Fast Deployment by setting `EmbedAssembliesIntoApk = True` in your .csproj. Or turn off `Fast Deployment` in the IDE. You will still be able to debug on the device, all the required files will be packaged inside the .apk.

In addition raise an issue at <https://github.com/xamarin/xamarin-android/issues/new/choose>. Please provide the deployment log and as much detail as possible.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error XA0128

Article • 04/17/2024

Issue

The Fast Deployment system relies on reading the output from the various system applications on the device. This is done by reading `System.out`, which by default is where all output will be redirected. If `log.redirect-stdio` this will cause all of the Fast Deployment tooling to fail.

Solution

Disable the redirection of stdio in order to use Fast Deployment. This can be done using the following commands.

```
adb shell stop
adb shell setprop log.redirect-stdio false
adb shell start
```

Depending on the device you might need to run `adb root` before running the above commands.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0129

Article • 04/17/2024

Issue

This issue happens when you are trying to use fast deployment on a device which does not support it. In this case both the normal and backup types of fast deployment failed. The Fast Deployment system makes use of the [run-as](#) feature of the Android OS. This feature was either not available or had limited capabilities in API 20 and earlier

Solution

Disable Fast Deployment by setting `EmbedAssembliesIntoApk = True` in your .csproj. Or turn off `Fast Deployment` in the IDE. You will still be able to debug on the device, all the required files will be packaged inside the .apk.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error XA0130

Article • 04/17/2024

Issue

This issue happens when you are trying to use fast deployment on a device which does not support it. Fast deployment requires features which are not available on devices running API 20 or lower. The Fast Deployment system makes use of the [run-as](#) feature of the Android OS. This feature was either not available or had limited capabilities in API 20 and earlier.

Solution

Disable Fast Deployment by setting `EmbedAssembliesIntoApk = True` in your .csproj. Or turn off `Fast Deployment` in the IDE. You will still be able to debug on the device, all the required files will be packaged inside the .apk.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error XA0131

Article • 04/17/2024

Issue

The 'run-as' tool has been disabled on this device by the Manufacturer. Fast Deployment requires 'run-as' in order to function.

Solution

Either enable it by activating the developer options on the device or by setting 'ro.boot.disable_runas' to 'false'. If you are unable to activate the developer options you can disable Fast Deployment by setting `EmbedAssembliesIntoApk = True` in your .csproj. Or turn off `Fast Deployment` in the IDE. You will still be able to debug on the device, all the required files will be packaged inside the .apk.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0132

Article • 04/17/2024

Issue

The package was not installed. Please check you do not have it installed under any other user. If the package does show up on the device, try manually uninstalling it then try again. You should be able to uninstall the app via the Settings app on the device.

Solution

This error only seems to happen when the same app is installed under multiple user account. Check that you do not have the app installed under a Guest user or a secondary account on you device.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0133

Article • 04/17/2024

Issue

The 'run-as' tool required by the Fast Deployment system has been disabled on this device by the manufacturer. Please disable Fast Deployment in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.

Solution

Unfortunately the manufacturer of the device has explicitly disabled the tool we need for Fast Deployment. The only option currently is to disable Fast Deployment from the IDE or by setting the the 'EmbedAssembliesIntoApk' MSBuild property to 'true' in the csproj.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0134

Article • 04/17/2024

Issue

The application does not have the 'android:debuggable' attribute set in the 'AndroidManifest.xml'. This is required in order for Fast Deployment to work. This is normally enabled by default by the .NET for Android build system for Debug builds.

Solution

Please check that you do not have the 'android:debuggable' attribute set on the 'application' element in your 'AndroidManifest.xml'. If you have a class that derives from 'Android.App.Application' and are using the '[Application]' make sure the 'Debuggable' property is not set at all as it will override the value for debug builds. If neither of these solutions work please raise an issue at <https://github.com/xamarin/xamarin-android/issues/new/choose>.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android error XA0135

Article • 04/17/2024

Issue

The package is a 'system' application. These are applications which install under the 'system' user on a device. These types of applications cannot use 'run-as'.

Solution

The Fast Deployment system should have handled this particular error automatically and used a backup installation path. However if you are seeing this error please file an issue with the exact error message using the 'Help->Send Feedback->Report a Problem' menu item in Visual Studio or 'Help->Report a Problem' in Visual Studio for Mac. If possible attach a full diagnostic build log to the feedback report as this will help us diagnose the issue.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0136

Article • 04/17/2024

Issue

The 'run-as' command failed with '{0}'.

Solution

The currently installation of the package is corrupt. Please manually uninstall the package from all the users on device and try again. If that does not work you can disable Fast Deployment. Fast Deployment can be disabled in the Visual Studio project property pages or edit the project file in a text editor and set the 'EmbedAssembliesIntoApk' MSBuild property to 'true'.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0137

Article • 04/17/2024

Issue

The 'run-as' command failed with '{0}'.

Solution

Fast Deployment is not currently supported on this device. Please file an issue with the exact error message using the 'Help->Send Feedback->Report a Problem' menu item in Visual Studio or 'Help->Report a Problem' in Visual Studio for Mac. If possible attach a full diagnostic build log to the feedback report as this will help us diagnose the issue.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0138

Article • 04/17/2024

Issue

`%(AndroidAsset.AssetPack)` and `%(AndroidAsset.AssetPack)` item metadata are only supported when `$(AndroidApplication)` is `true`.

Solution

Remove the 'AssetPack' or 'DeliveryType' Metadata from your `AndroidAsset` build Items in the project the error was raised for.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0139

Article • 04/17/2024

Issue

`@(AndroidAsset) {0}` has an invalid `DeliveryType` metadata of `{1}`. Supported values are `installtime`, `ondemand` or `fastfollow`.

Solution

Make sure that all `DeliveryType` attributes are one of the following valid values, `installtime`, `ondemand` or `fastfollow`.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android error XA0140

Article • 04/17/2024

Issue

The AssetPack value defined for `{0}` has invalid characters. `{1}` should only contain A-z, a-z, 0-9 or an underscore.

Solution

Make sure that all `AssetPack` attributes only contain valid characters.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0141

Article • 10/31/2024

Issue

NuGet package '{0}' version '{1}' contains a shared library '{2}' which is not correctly aligned. See <https://developer.android.com/guide/practices/page-sizes> for more details

Solution

The indicated native shared library must be recompiled and relinked with the 16k alignment, as per URL indicated in the message.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

.NET for Android warning XA0142

Article • 10/31/2024

Issue

Command '{0}' failed.\n{1}

Solution

Examine logged output of the failed command for indications of what caused the issue. If no immediate solution is suggested by the logged messages, please file an issue at <https://github.com/dotnet/android> including the full error message and steps that led to the command failing (possibly including a small repro application).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)